

Curso de Programação Estatística

Andressa Cerqueira, Danilo Lourenço Lopes, Rafael Izbicki, Thiago Ramos

2026-09-01

Table of contents

Curso de Programação Estatística	12
Objetivo Geral	12
A quem se destina	12
Como o livro está organizado	13
Como usar o livro	13
Notação	14
Autores	14
 Breve Introdução ao Python	 16
Onde escrever seu código	16
Variáveis e Tipos de Dados	17
O Comando help() em Python	18
Armazenando Resultados em Variáveis	18
Variáveis Lógicas	19
Caracteres/Strings em Python	21
Listas	24
Tuplas	24
Dicionários	25
Funções em Python	26
Loops	28
Estruturas Condicionais: if, else e elif em Python	30
Bibliotecas Essenciais para Estatística: NumPy, Pandas e Matplotlib	31
Exercícios	50
 Breve Introdução ao R	 53
Onde escrever seu código	53
Variáveis Numéricas	53
Variáveis Lógicas	56
Caracteres/Strings	58
Vetores e Sequências	58
Funções	62
Laços	63
If/Else	65
Listas	66
Estatística Descritiva	68

O operador %>% (pipe)	72
O pacote dplyr	74
O pacote ggplot2	80
O pacote tidyr	89
Leitura de Arquivos com readr	93
Exercícios	94
1 Por que Simular?	97
1.1 Estimando π com pontos aleatórios	97
2 R	98
3 Python	100
3.1 Simulando um dado viciado	102
4 R	103
5 Python	105
5.1 E se tivéssemos apenas uma uniforme?	107
6 R	108
7 Python	111
7.1 Aleatoriedade também aparece em aprendizado de máquina	114
8 R	115
9 Python	117
9.1 O caminho daqui em diante	119
9.2 Exercícios	119
10 Números Pseudoaleatórios	121
10.1 O Gerador Linear Congruente (LCG)	121
11 R	124
12 Python	125
12.1 Exemplo 1: implementando o LCG	125
13 R	127
14 Python	130
14.1 Escolhendo os parâmetros	131
15 R	134

16 Python	135
16.1 Exemplo 2: o efeito de parâmetros ruins	136
17 R	137
18 Python	140
18.1 Exemplo 3: a estrutura escondida nos pares	142
19 R	143
20 Python	145
20.1 Exemplo 4: gerando números uniformes com uma moeda	147
21 R	149
22 Python	151
22.1 Exercícios	152
23 Técnica da Inversão para Variáveis Discretas	155
23.1 Inversa da f.d.a.	155
23.2 Geração de Variáveis Aleatórias Discretas Genéricas	156
24 R	157
25 Python	159
25.1 Por que o método funciona	160
25.2 Exemplo 1: Geração de uma Bernoulli por inversão	161
26 R	162
27 Python	164
27.1 Exemplo 2: Geração de Variáveis Aleatórias com Distribuição Geométrica	166
28 R	168
29 Python	171
29.1 Exemplo 3: Geração de Variáveis Aleatórias com Distribuição Poisson	173
30 R	175
31 Python	179
31.1 Exemplo 4: permutações e amostragem sem reposição	182
32 R	184
33 Python	186

34 R	189
35 Python	191
35.1 Exercícios	192
36 Técnica da Inversão para Variáveis Contínuas	196
36.1 A inversa da f.d.a. no caso contínuo	196
37 R	198
38 Python	200
38.1 Método da Inversão	202
38.2 Exemplo 1: uma potência	203
39 R	204
40 Python	206
40.1 Exemplo 2: distribuição exponencial	207
41 R	208
42 Python	210
42.1 Exemplo 3: distribuições truncadas	211
43 R	214
44 Python	216
44.1 Exemplo 4: quando não há fórmula para a inversa	218
45 R	220
46 Python	222
46.1 Exercícios	224
47 Método da Rejeição para Variáveis Discretas	227
47.1 Algoritmo	227
47.2 Exemplo: uma distribuição sobre $\{1, 2, \dots, 10\}$	228
48 R	230
49 Python	232
50 R	234
51 Python	238
51.1 Por que o método funciona	241

51.2	Eficiência e escolha da distribuição proposta	243
51.3	Exemplo: um dado honesto a partir de moedas	243
52	R	245
53	Python	247
53.1	Quando a rejeição é preferível à inversão	248
53.2	Exercícios	249
54	Método da Rejeição para Variáveis Contínuas	253
54.1	Algoritmo	253
54.2	Interpretação geométrica	254
55	R	255
56	Python	257
57	R	259
58	Python	261
58.1	Exemplo 1: uma densidade em $(0, 1)$ com proposta uniforme	262
59	R	264
60	Python	266
60.1	Exemplo 2: gerando uma normal a partir de exponenciais	268
61	R	270
62	Python	273
62.1	Por que o método funciona	275
62.2	Eficiência e escolha da distribuição proposta	276
62.3	Exercícios	277
63	Transformações e Misturas	282
63.1	Transformação de v.a.	282
63.2	Exemplo 1: simulando $Y \sim \text{Unif}(1, 2)$	283
64	R	284
65	Python	286
65.1	Exemplo 2: simulando $Y \sim \text{Gama}(n, \lambda)$	287
66	R	289

67 Python	291
67.1 Exemplo 3: distribuição de Weibull	292
68 R	295
69 Python	297
69.1 Exemplo 4: distribuição qui-quadrado	298
70 R	300
71 Python	302
71.1 Exemplo 5: um vetor de normais correlacionadas	303
72 R	306
73 Python	309
73.1 Misturas	311
73.2 Exemplo 6: mistura de duas normais	313
74 R	314
75 Python	316
75.1 Exemplo 7: distribuição t de Student	317
76 R	320
77 Python	322
77.1 Exemplo 8: um modelo hierárquico	324
78 R	326
79 Python	328
79.1 Exercícios	330
80 Método de Box-Muller	337
80.1 A ideia: olhar para o par, não para a coordenada	337
80.2 Representação em coordenadas polares	338
80.3 A distribuição de R e Θ	339
80.4 O algoritmo	340
80.5 Exemplo 1: gerando normais padrão	342
81 R	343
82 Python	346
82.1 Exemplo 2: de $N(0, 1)$ para $N(\mu, \sigma^2)$	349

83 R	351
84 Python	353
84.1 Exemplo 3: um par de normais correlacionadas	354
85 R	356
86 Python	358
86.1 O método polar de Marsaglia	360
87 R	361
88 Python	363
88.1 Exercícios	365
89 Método de Monte Carlo	370
89.1 A ideia do método	370
89.2 Exemplo 1: Estimativa de uma Integral	372
90 R	373
91 Python	374
92 R	375
93 Python	377
93.1 Escolhendo a densidade: integrais em outros domínios	378
93.2 Exemplo 2: Uma integral em um intervalo qualquer	379
94 R	380
95 Python	381
95.1 Exemplo 3: Uma integral em um domínio ilimitado	381
96 R	383
97 Python	384
98 R	385
99 Python	386
99.1 Exemplo 4: Aproximando uma Probabilidade	387
100R	388

101Python	389
101.1Exemplo 5: Aproximando o valor de π	390
102R	391
103Python	392
104R	395
105Python	397
105.1Quão preciso é o método?	398
105.2Intervalos de confiança para estimativas de Monte Carlo	399
105.3Exemplo 6: Intervalo de Confiança para a Estimativa de π	400
106R	401
107Python	402
108R	404
109Python	405
109.1Exemplo 7: Quando o método falha	406
110R	407
111Python	409
111.1Exemplo 8: Problema das Figurinhas	410
112R	411
113Python	412
114R	413
115Python	414
116R	416
117Python	417
118R	418
119Python	419
120R	420

121Python	421
121.1Exercícios	421
122Técnicas de Redução de Variância	427
122.1Medindo o ganho	427
122.2Técnica 1: Variáveis Antitéticas	429
123R	433
124Python	434
125R	436
126Python	437
126.1Técnica 2: Variáveis de Controle	438
127R	442
128Python	444
129R	446
130Python	447
131R	449
132Python	450
132.1Técnica 3: Condicionamento	451
133R	454
134Python	456
135R	458
136Python	460
136.1Técnica 4: Amostragem Estratificada	461
137R	465
138Python	467
138.1Colocando as técnicas lado a lado	468
139R	469

140Python	471
140.1Números aleatórios comuns	472
141R	475
142Python	477
142.1Exercícios	479
143Amostragem por Importância	486
143.1Quando o Monte Carlo usual falha	486
144R	487
145Python	488
145.1A ideia da amostragem por importância	489
145.2A escolha da proposta	492
146R	494
147Python	496
147.1De volta aos eventos raros	498
148R	499
149Python	501
149.1Diagnóstico: o tamanho amostral efetivo	502
150R	504
151Python	505
151.1Ajustando os parâmetros da proposta	505
152R	508
153Python	509
153.1Amostragem por importância autonormalizada	510
154R	512
155Python	514
155.1Exercícios	515
References	521

Curso de Programação Estatística

Este é o material da disciplina **Programação Estatística**, da graduação em Estatística da UF-SCar. O livro trata de uma ideia que atravessa boa parte da Estatística moderna: quando uma quantidade de interesse — uma probabilidade, uma esperança, uma integral, a distribuição de um estimador — não tem fórmula fechada, ainda assim podemos estimá-la **repetindo o fenômeno muitas vezes no computador e olhando para o que acontece**.

Enunciar essa ideia é fácil; executá-la exige responder a três perguntas, e é isso que os capítulos fazem, nesta ordem: de onde o computador tira números aleatórios, como transformar esses números na distribuição que nos interessa, e o que fazer com a amostra obtida.

Objetivo Geral

Este curso visa explorar o uso do computador como ferramenta de análise estatística, com atenção ao impacto das representações numéricas sobre os resultados dos algoritmos. O foco será na programação, visualização e preparação de dados, além de discutir tópicos importantes como aleatoriedade, pseudoaleatoriedade, erros de truncamento e arredondamento, entre outros. O curso inclui ainda uma introdução à inferência por simulação estocástica, utilizando métodos como Monte Carlo e integrações numéricas. O material do curso foi amplamente baseado nas discussões apresentadas em Ross (2006).

A quem se destina

O livro supõe que você já cursou uma disciplina de **probabilidade**: usamos variáveis aleatórias, funções de densidade e de distribuição acumulada, esperança e variância, e os dois teoremas que sustentam tudo o que faremos — a Lei dos Grandes Números e o Teorema Central do Limite.

Já de **programação**, não supomos nada. Os dois primeiros capítulos ([Python](#) e [R](#)) revisam o necessário do zero, incluindo onde instalar e escrever o código; eles não fazem parte do conteúdo do curso propriamente dito, e servem de consulta. Se você já programa em uma das duas linguagens, pode ir direto para [Por que Simular?](#) e voltar a esses capítulos quando precisar.

Como o livro está organizado

A aleatoriedade e sua origem. O primeiro capítulo mostra a simulação funcionando, sem ainda justificar nada, para estabelecer o que queremos ser capazes de fazer. O segundo abre a caixa-preta do `runif`/`np.random.uniform` e mostra como um algoritmo determinístico produz números que *parecem* aleatórios.

- [Por que Simular?](#) — quatro exemplos que motivam o restante do livro
- [Números Pseudoaleatórios](#) — o Gerador Linear Congruente e a noção de semente

De $\text{Unif}(0, 1)$ para qualquer distribuição. Este é o coração do livro. A partir daqui, supomos resolvido o problema de gerar números uniformes em $[0, 1)$, e a pergunta passa a ser: dada uma distribuição qualquer, como construir uma amostra dela usando apenas uniformes? Há mais de uma resposta, e cada capítulo apresenta um método, com suas hipóteses e seu custo.

- [Inversão: caso discreto e caso contínuo](#) — quando sabemos inverter a f.d.a.
- [Rejeição: caso discreto e caso contínuo](#) — quando não sabemos, mas temos uma distribuição auxiliar fácil de simular
- [Transformações e Misturas](#) — construindo distribuições novas a partir de outras já conhecidas
- [Método de Box-Muller](#) — o caso da normal, que escapa dos métodos anteriores

O que fazer com a amostra. Ter a amostra é meio caminho. Os capítulos finais tratam de usá-la para estimar quantidades, de medir o erro dessa estimativa e de reduzi-lo sem simplesmente aumentar o número de simulações.

- [Método de Monte Carlo](#) — estimação, erro padrão e intervalos de confiança
- [Redução de Variância](#) — variáveis antitéticas, variáveis de controle, condicionamento e amostragem estratificada
- [Amostragem por Importância](#) — simulando de uma distribuição para aprender sobre outra, útil sobretudo para eventos raros

Como usar o livro

Todo exemplo executável aparece em [R](#) e em [Python](#), em versões que fazem exatamente a mesma coisa. Você não precisa saber as duas linguagens: escolha uma e siga por ela.

 Navegando pelos blocos de código

- As abas **R** e **Python** ficam **sincronizadas**: ao escolher uma delas em qualquer ponto do livro, todos os demais blocos passam a exibir a mesma linguagem.

- O código vem recolhido por padrão. Clique em **Mostrar código** para vê-lo.
- Vale a pena tentar escrever o código antes de abrir o bloco: todos os métodos são apresentados como um **pseudo-algoritmo** em português, logo antes da implementação, e é justamente a passagem de um para o outro que se pretende ensinar aqui.

Os capítulos com simulação começam fixando uma **semente** (`set.seed` no R, `np.random.seed` no Python). Isso faz com que o código gere sempre os mesmos números, e é o que permite que os resultados que você obtém coincidam com os impressos no livro. Ao mudar a semente, os valores mudam — mas as conclusões, não; fazer isso de propósito é um bom teste de que a ideia foi entendida.

Cada capítulo termina com uma lista de **exercícios**. Alguns trazem a marcação (**Desafio**): são exercícios substancialmente mais difíceis que os demais — pedem uma demonstração, uma otimização analítica ou um argumento que o capítulo não desenvolveu — e ficam sempre ao final da lista.

Notação

O livro segue estas convenções:

Símbolo	Significado
v.a.	variável aleatória
f.d.a.	função de distribuição acumulada
$\mathbb{P}(A)$, $\mathbb{E}[X]$, $\text{Var}(X)$	probabilidade, esperança e variância
$\text{Unif}(0, 1)$, $\text{Exp}(\lambda)$, $N(\mu, \sigma^2)$	as distribuições usuais
$\hat{\theta}_n$	um estimador de θ
B	número de simulações de Monte Carlo
n	tamanho de uma amostra de dados reais

A distinção entre B e n é proposital e importante: n é o tamanho da amostra que você tem, e não se pode escolher; B é quantas vezes você decidiu rodar a simulação, e aumentá-lo custa apenas tempo de computador.

Autores

[Andressa Cerqueira](#), [Danilo Lourenço Lopes](#), [Rafael Izbicki](#), [Thiago Rodrigo Ramos](#).

Encontrou um erro ou tem uma sugestão? O material é aberto e vive em <https://github.com/thiagorr162/prog-estat>.

Breve Introdução ao Python

Aqui faremos uma breve introdução ao Python. Para uma introdução mais detalhada, você pode explorar:

- [Documentação Oficial do Python](#)
- [Real Python](#)

Onde escrever seu código

Assim como no R, o Python é o motor que executa os comandos, e escrevemos o código em um editor separado. Uma forma prática de instalar o Python já com as bibliotecas que usaremos (NumPy, Pandas, Matplotlib, entre outras) é a distribuição [Anaconda](#). Para escrever o código, as opções gratuitas mais comuns são:

- **VS Code** (<https://code.visualstudio.com>): o editor de código de uso geral mais usado hoje. Instale a extensão *Python*, da Microsoft; a extensão *Jupyter* permite ainda rodar o código em células, como em um notebook.
- **Positron** (<https://positron.posit.co>): uma IDE feita pela Posit — a mesma empresa do RStudio — sobre a base do VS Code, mas voltada especificamente para análise de dados. Traz painéis de console, variáveis, gráficos e ajuda, e trata Python e R como linguagens de primeira classe, o que é conveniente neste curso, em que todo exemplo aparece nas duas.
- **Jupyter Notebook / JupyterLab** (<https://jupyter.org>): o formato de *notebook*, em que texto e código convivem em células executadas uma a uma. É bastante usado em aulas e em análises exploratórias.
- **Google Colab** (<https://colab.research.google.com>): notebooks Jupyter que rodam no navegador, sem instalar nada no seu computador. Uma saída útil caso a instalação local dê trabalho.

Se você não tem preferência, o Positron é uma boa escolha para este curso, porque o mesmo programa serve para os capítulos em Python e para os em R. Nenhuma dessas escolhas muda o que o código faz: ele roda igual em qualquer uma delas.

Variáveis e Tipos de Dados

Em Python, variáveis são usadas para armazenar informações que podem ser manipuladas ao longo do código. Existem diferentes tipos de dados que podem ser atribuídos a uma variável. Vamos explorar os principais tipos básicos em Python: `int`, `float`, `str`, `bool`, e `None`.

```
# Exemplo de variáveis e tipos de dados
nome = "Thiago"           # String (str)
idade = 35                # Inteiro (int)
saldo_bancario = 1023.75 # Número com casas decimais (float)
estudante = True          # Booleano (bool)
endereco = None           # None, indicando ausência de valor

# Usando o método .format() para formatar a string
print("Nome: {} (Tipo: {})".format(nome, type(nome)))
print("Idade: {} (Tipo: {})".format(idade, type(idade)))
print("Saldo Bancário: {} (Tipo: {})".format(saldo_bancario, type(saldo_bancario)))
print("Estudante: {} (Tipo: {})".format(estudante, type(estudante)))
print("Endereço: {} (Tipo: {})".format(endereco, type(endereco)))
```

```
Nome: Thiago (Tipo: <class 'str'>)
Idade: 35 (Tipo: <class 'int'>)
Saldo Bancário: 1023.75 (Tipo: <class 'float'>)
Estudante: True (Tipo: <class 'bool'>)
Endereço: None (Tipo: <class 'NoneType'>)
```

f-strings: Formatação de Strings em Python (*)

Introduzidas na versão Python 3.6, as f-strings (ou “formatted string literals”) são uma forma eficiente e legível de formatar strings, permitindo a inclusão de expressões e variáveis diretamente dentro de uma string.

A sintaxe das f-strings utiliza a letra `f` antes da string e permite a inclusão de expressões dentro de chaves `{}`. Essas expressões são avaliadas em tempo de execução, e seus resultados são inseridos na string.

```
# Usando f-strings para formatar a string
print(f"Nome: {nome} (Tipo: {type(nome)})")
print(f"Idade: {idade} (Tipo: {type(idade)})")
print(f"Saldo Bancário: {saldo_bancario} (Tipo: {type(saldo_bancario)})")
print(f"Estudante: {estudante} (Tipo: {type(estudante)})")
print(f"Endereço: {endereco} (Tipo: {type(endereco)})")
```

```
Nome: Thiago (Tipo: <class 'str'>)
Idade: 35 (Tipo: <class 'int'>)
Saldo Bancário: 1023.75 (Tipo: <class 'float'>)
Estudante: True (Tipo: <class 'bool'>)
Endereço: None (Tipo: <class 'NoneType'>)
```

O Comando `help()` em Python

Python possui uma função embutida chamada `help()`, que é muito útil para obter informações sobre funções, módulos, objetos, e classes. Ele fornece uma explicação detalhada de como determinado elemento funciona, quais parâmetros aceita e o que retorna, entre outros detalhes. Essa função é especialmente útil quando você está começando ou precisa de uma rápida referência sem ter que sair do ambiente de desenvolvimento.

```
help(len)
```

Help on built-in function len in module builtins:

```
len(obj, /)
    Return the number of items in a container.
```

```
def exemplo():
    """Esta é uma função de exemplo."""
    pass

help(exemplo)
```

Help on function exemplo in module __main__:

```
exemplo()
    Esta é uma função de exemplo.
```

Armazenando Resultados em Variáveis

Em Python, as variáveis são usadas para armazenar resultados de cálculos ou operações para uso posterior no código. Ao armazenar um valor em uma variável, você pode acessá-lo facilmente quando precisar, sem ter que repetir o cálculo ou operação.

Quando você atribui um valor ou resultado de uma operação a uma variável, o Python guarda esse valor na memória e o associa ao nome que você escolheu para a variável. Isso permite que você reutilize o valor sempre que necessário.

```
resultado = 5 + 3 # Armazenando a soma de 5 e 3 em uma variável
print(resultado) # Resultado: 8
```

8

```
x = 2 + 3
print(x)
```

5

```
y = 2 * x
y
```

10

Variáveis Lógicas

Python, assim como R, possui operadores lógicos para comparar valores e trabalhar com variáveis booleanas. Abaixo estão alguns dos principais operadores lógicos usados em Python:

#	Operador	Descrição
1	<code>x < y</code>	x é menor que y?
2	<code>x <= y</code>	x é menor ou igual a y?
3	<code>x > y</code>	x é maior que y?
4	<code>x >= y</code>	x é maior ou igual a y?
5	<code>x == y</code>	x é igual a y?
6	<code>x != y</code>	x é diferente de y?
7	<code>not x</code>	Negativa de x (inverte o valor lógico)
8	<code>x or y</code>	x ou y são verdadeiros? (ou inclusivo)
9	<code>x and y</code>	x e y são verdadeiros? (e lógico)
10	<code>x ^ y</code>	x ou y, mas não ambos, são verdadeiros? (xor lógico)

```
a = 5
b = 10
print(a < b)  # True
print(a == b) # False
```

True
False

```
x = True
print(not x)  # False
```

False

```
x = False
y = True
print(x or y)  # True
```

True

```
x = True
y = False
print(x and y)  # False
```

False

```
x = True
y = False
print(x ^ y)  # True (apenas um dos dois é verdadeiro)
```

True

 Atenção: $\wedge$ não é exponenciação em Python

Em Python, o operador $\wedge$ **não** é usado para exponenciação — ele é o XOR bit a bit (ou exclusivo). Para elevar um número a uma potência, use `**`: escreve-se `5 ** 3`, e não `5 ^ 3`. O erro é traiçoeiro porque `5 ^ 3` não dá erro nenhum: devolve silenciosamente 6, que é o XOR dos dois números. No R, ao contrário, $\wedge$ é mesmo a exponenciação.

O operador XOR compara os bits de dois números. Ele retorna 1 quando os bits são diferentes e 0 quando os bits são iguais.

```
a = 5    # Em binário: 101
b = 3    # Em binário: 011

resultado = a ** b # Faz a exponenciação 5^3, e NÃO o XOR

print(resultado)  # Saída: 125
```

125

```
a = 5    # Em binário: 101
b = 3    # Em binário: 011

resultado = a ^ b # Faz XOR bit a bit

print(resultado)  # Saída: 6 (Em binário: 110)
```

6

Caracteres/Strings em Python

Em Python, uma string é uma sequência de caracteres que pode ser usada para armazenar e manipular textos. Strings são um dos tipos de dados mais comuns e úteis, e Python oferece uma grande variedade de métodos e operações para trabalhar com elas.

As strings em Python podem ser definidas de diferentes maneiras, usando aspas simples (') ou aspas duplas (").

```
# Definindo strings
nome = 'Thiago'
cidade = "São Carlos"
```

- Concatenar Strings: Você pode unir duas ou mais strings usando o operador +.

```
# Concatenação
nome_completo = "Thiago" + " " + "Rodrigo"
print(nome_completo) # Resultado: Thiago Rodrigo
```

Thiago Rodrigo

- Repetir Strings: Você pode repetir uma string múltiplas vezes usando o operador *.

```
repeticao = "Oi! " * 3
print(repeticao) # Resultado: Oi! Oi! Oi!
```

Oi! Oi! Oi!

- Acessar Caracteres por Índice: As strings em Python são indexadas, e você pode acessar um caractere específico usando o índice (começando em 0).

```
nome = "Thiago"
print(nome[0]) # Resultado: T
print(nome[-1]) # Resultado: o (último caractere)
```

T
o

- Fatiar Strings (Slicing): Você pode pegar uma parte da string usando fatias (substrings).

```
nome = "Thiago"
print(nome[0:3]) # Resultado: Thi (caracteres do índice 0 ao 2)
```

Thi

⚠ Atenção: índices em Python e em R

Duas diferenças que costumam confundir quem usa as duas linguagens:

- em Python a contagem começa em **0** (o primeiro caractere é `nome[0]`), enquanto no R começa em **1** (`nome[1]`);
- em Python a fatia `nome[0:3]` **não inclui** a posição 3: ela devolve os índices 0, 1 e 2. No R, `x[1:3]` inclui as três posições.

O índice negativo também significa coisas diferentes: em Python `nome[-1]` é o *último* elemento; no R, `x[-1]` quer dizer “todos menos o primeiro”.

- Comprimento de Strings: Para saber quantos caracteres uma string tem, use a função `len()`.

```
nome = "Thiago"
print(len(nome)) # Resultado: 6
```

6

Métodos Úteis para Strings

- `lower()` e `upper()`: Convertem todas as letras para minúsculas ou maiúsculas, respectivamente.

```
nome = "Thiago"
print(nome.lower()) # Resultado: thiago
print(nome.upper()) # Resultado: THIAGO
```

thiago
THIAGO

- `strip()`: Remove espaços em branco no início e no final da string.

```
frase = "   Olá!   "
print(frase.strip()) # Resultado: Olá!
```

Olá!

- `replace()`: Substitui parte de uma string por outra.

```
frase = "Eu gosto de R"
nova_frase = frase.replace("R", "Python")
print(nova_frase) # Resultado: Eu gosto de Python
```

Eu gosto de Python

- `split()`: Divide uma string em uma lista, utilizando um delimitador (por padrão, espaço).

```
frase = "Eu gosto de Python"
palavras = frase.split()
print(palavras) # Resultado: ['Eu', 'gosto', 'de', 'Python']
```

['Eu', 'gosto', 'de', 'Python']

- `join()`: Une uma lista de strings em uma única string, usando um delimitador.

```
lista = ['Thiago', 'Rodrigo', 'Ramos']
nome_completo = " ".join(lista)
print(nome_completo) # Resultado: Thiago Rodrigo Ramos
```

Thiago Rodrigo Ramos

Listas

Em Python, listas são coleções ordenadas e mutáveis, o que significa que você pode modificar os elementos após sua criação. Elas são definidas usando colchetes `[]` e podem armazenar múltiplos tipos de dados, como inteiros, strings ou até mesmo outras listas.

Características principais:

- Mutáveis: você pode adicionar, remover ou modificar elementos.
- Ordenadas: os elementos mantêm a ordem em que são inseridos.
- Acesso por índice: os elementos podem ser acessados pelo índice, começando por 0.

```
# Criando uma lista
frutas = ['maçã', 'banana', 'laranja']

# Acessando elementos
print(frutas[1]) # banana

# Modificando a lista
frutas.append('uva') # Adiciona 'uva' ao final
frutas[0] = 'kiwi'   # Substitui 'maçã' por 'kiwi'

# Removendo um elemento
frutas.remove('banana')

print(frutas) # ['kiwi', 'laranja', 'uva']
```

```
banana
['kiwi', 'laranja', 'uva']
```

Tuplas

As tuplas em Python são semelhantes às listas, porém, diferentemente das listas, são imutáveis, ou seja, seus elementos não podem ser modificados após a criação. Elas são úteis quando você deseja garantir que uma sequência de valores permaneça inalterada. As tuplas são definidas usando parênteses `()`. Características principais:

- Imutáveis: uma vez criadas, seus elementos não podem ser alterados, adicionados ou removidos.
- Ordenadas: os elementos mantêm a ordem de inserção.
- Acesso por índice: assim como nas listas, os elementos podem ser acessados por índices, começando do 0.


```
# Criando uma tupla
coordenadas = (10, 20)

# Acessando elementos
print(coordenadas[0]) # 10

# Tuplas podem armazenar diferentes tipos de dados
dados = ('Thiago', 30, True)

print(dados) # ('Thiago', 30, True)
```

```
10
('Thiago', 30, True)
```

Tentar modificar uma tupla resulta em erro. O código abaixo não é executado no livro justamente porque interromperia a execução — experimente rodá-lo no seu computador:

```
coordenadas[0] = 15
# TypeError: 'tuple' object does not support item assignment
```

Dicionários

Os dicionários em Python são coleções não ordenadas de pares chave-valor. Eles permitem associar valores a uma chave específica, sendo muito úteis quando você precisa acessar elementos por meio de uma chave, em vez de um índice. Eles são definidos com chaves {}.

Características principais:

- Mutáveis: você pode adicionar, remover ou modificar pares chave-valor.
- Ordem de inserção: a partir do Python 3.7, os elementos são percorridos na ordem em que foram inseridos (em versões anteriores essa ordem não era garantida). Ainda assim, o acesso é sempre feito pela chave, e não pela posição.
- Acesso por chave: os valores são acessados por suas chaves, que podem ser de tipos imutáveis (como strings ou números).

```
# Criando um dicionário
estudante = {
    'nome': 'Ana',
    'idade': 22,
```

```

    'curso': 'Estatística'
}

# Acessando valores
print(estudante['nome']) # Ana

# Modificando o dicionário
estudante['idade'] = 23 # Atualiza o valor da chave 'idade'

# Adicionando um novo par chave-valor
estudante['matricula'] = 12345

# Removendo um elemento
del estudante['curso']

print(estudante) # {'nome': 'Ana', 'idade': 23, 'matricula': 12345}

```

```

Ana
{'nome': 'Ana', 'idade': 23, 'matricula': 12345}

```

Funções em Python

As funções em Python são blocos de código reutilizáveis que realizam uma tarefa específica. Elas ajudam a organizar o código, tornando-o mais modular e legível. Você pode definir suas próprias funções usando a palavra-chave `def`, e elas podem receber parâmetros, retornar valores ou simplesmente executar uma ação. Características principais:

- Reutilizáveis: uma vez definida, a função pode ser chamada várias vezes no código.
- Modulares: permitem dividir o código em partes menores e mais organizadas.
- Parâmetros opcionais ou obrigatórios: funções podem receber parâmetros (ou argumentos) para realizar operações com base neles.

```

def saudacao(nome):
    """Exibe uma saudação personalizada."""
    print(f"Olá, {nome}!")

# Chamando a função
saudacao("Thiago") # Olá, Thiago!

```

```

Olá, Thiago!

```

Return

Funções podem retornar valores usando a palavra-chave `return`. Isso permite que o resultado da função seja usado em outras partes do código.

```
def soma(a, b):  
    """Retorna a soma de dois números."""  
    return a + b  
  
resultado = soma(3, 5)  
print(resultado) # 8
```

8

Parâmetros Opcionais e Valores Padrão

Você pode definir parâmetros opcionais em uma função, atribuindo valores padrão a eles.

```
def saudacao(nome="amigo"):  
    """Exibe uma saudação personalizada, com um nome padrão."""  
    print(f"Olá, {nome}!")  
  
saudacao() # Olá, amigo!  
saudacao("Rafael") # Olá, Rafael!
```

Olá, amigo!
Olá, Rafael!

Lambda

Python também oferece funções anônimas (ou funções `lambda`), que são funções curtas e sem nome. Elas são úteis quando você precisa de uma função simples e temporária.

```
# Função lambda para multiplicar dois números  
multiplicar = lambda x, y: x * y  
print(multiplicar(3, 4)) # 12
```

12

Loops

Os loops são estruturas fundamentais de programação que permitem executar um bloco de código repetidamente enquanto uma condição for verdadeira ou para cada item de uma sequência. Python oferece dois tipos principais de loops: for e while.

for

O loop for é usado para iterar sobre uma sequência (como uma lista, tupla, string ou range) e executar um bloco de código para cada item dessa sequência. É especialmente útil quando o número de iterações é conhecido ou quando você deseja percorrer uma estrutura de dados.

```
# Iterando sobre uma lista
frutas = ['maçã', 'banana', 'laranja']
for fruta in frutas:
    print(fruta)
```

```
maçã
banana
laranja
```

```
# Iterando de 0 a 4
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

```
frutas = ['maçã', 'banana', 'laranja']
for i, fruta in enumerate(frutas):
    print(i, fruta)
```

```
0 maçã
1 banana
2 laranja
```

while

O loop while executa um bloco de código repetidamente enquanto uma condição for verdadeira. Ele é útil quando o número de iterações não é conhecido antecipadamente e depende de uma condição dinâmica.

```
# Loop enquanto o valor de x for menor que 5
x = 0
while x < 5:
    print(x)
    x += 1
```

0
1
2
3
4

Controle de Loops: break e continue

break: interrompe o loop imediatamente, mesmo que a condição ainda seja verdadeira (no caso do while) ou ainda restem itens na sequência (no caso do for).

continue: pula para a próxima iteração do loop, ignorando o código restante naquela iteração.

```
for i in range(10):
    if i == 5:
        break # Interrompe o loop quando i é igual a 5
    print(i)
```

0
1
2
3
4

```
for i in range(5):
    if i == 3:
        continue # Pula a iteração quando i é igual a 3
    print(i)
```

0
1
2
4

Estruturas Condicionais: if, else e elif em Python

As estruturas condicionais em Python permitem que o programa tome decisões e execute blocos de código diferentes com base em condições específicas. As principais instruções condicionais são if, else e elif.

if

A instrução if é usada para executar um bloco de código se uma condição for verdadeira. Se a condição avaliada for True, o bloco de código indentado após o if será executado.

```
idade = 20
if idade >= 18:
    print("Você é maior de idade")
```

Você é maior de idade

else

A instrução else é usada para executar um bloco de código se a condição do if for falsa. É como uma segunda opção, caso a primeira condição não seja atendida.

```
idade = 16
if idade >= 18:
    print("Você é maior de idade")
else:
    print("Você é menor de idade")
```

Você é menor de idade

elif

A instrução elif (abreviação de else if) permite testar múltiplas condições. Se a condição if for falsa, o Python verificará a condição do elif. Você pode ter vários blocos elif em uma estrutura condicional.

```
nota = 85
if nota >= 90:
    print("Aprovado com excelência")
elif nota >= 70:
    print("Aprovado")
else:
    print("Reprovado")
```

Aprovado

Operadores lógicos

As condições podem usar operadores lógicos para combinar mais de uma verificação:

- and: todas as condições devem ser verdadeiras.
- or: pelo menos uma condição deve ser verdadeira.
- not: inverte o resultado da condição.

```
idade = 20
possui_carteira = True

if idade >= 18 and possui_carteira:
    print("Você pode dirigir")
else:
    print("Você não pode dirigir")
```

Você pode dirigir

Bibliotecas Essenciais para Estatística: NumPy, Pandas e Matplotlib

Em Python, as bibliotecas NumPy, Pandas, e Matplotlib são amplamente utilizadas para análise de dados e computação científica. Elas fornecem ferramentas poderosas para lidar com grandes volumes de dados, realizar cálculos matemáticos eficientes e criar visualizações de alta qualidade. Vamos detalhar cada uma delas:

NumPy

NumPy (Numerical Python) é uma biblioteca fundamental para cálculos matemáticos em Python. Ela fornece suporte para arrays e matrizes multidimensionais, além de uma coleção de funções para operações com esses arrays. Principais características:

- Array multidimensional (ndarray): O ndarray é a estrutura central da NumPy, oferecendo suporte a vetores e matrizes de várias dimensões, com operações eficientes em termos de memória e tempo de execução.
- Operações vetorizadas: Permite realizar operações em todos os elementos de um array sem a necessidade de loops explícitos.
- Funções matemáticas: Inclui uma vasta gama de funções para álgebra linear, estatística, trigonometria, entre outros.

```
import numpy as np

# Criando um array NumPy
array = np.array([1, 2, 3, 4, 5])

# Operações elementares
print(array * 2) # Multiplicação por escalar: [2 4 6 8 10]

# Matrizes e operações matriciais
matriz = np.array([[1, 2], [3, 4]])
print(np.dot(matriz, matriz)) # Multiplicação de matrizes
```

```
[ 2  4  6  8 10]
[[ 7 10]
 [15 22]]
```

```
dados = np.array([1, 2, 3, 4, 5])
media = np.mean(dados)
print(media) # Saída: 3.0
```

3.0

```
mediana = np.median(dados)
print(mediana) # Saída: 3.0
```

3.0


```
variancia = np.var(dados)
print(variancia) # Saída: 2.0
```

2.0

```
desvio_padrao = np.std(dados)
print(desvio_padrao) # Saída: 1.4142135623730951
```

1.4142135623730951

```
minimo = np.min(dados)
maximo = np.max(dados)
print(minimo, maximo) # Saída: 1 5
```

1 5

```
p25 = np.percentile(dados, 25)
p50 = np.percentile(dados, 50)
p75 = np.percentile(dados, 75)
print(p25, p50, p75) # Saída: 2.0 3.0 4.0
```

2.0 3.0 4.0

Pandas

Pandas é uma biblioteca poderosa para manipulação de dados em Python, frequentemente usada em projetos de ciência de dados, análise estatística e processamento de dados tabulares. Ela oferece estruturas de dados como DataFrames e Series que facilitam a organização, limpeza e análise de dados, tornando-a uma das ferramentas mais populares para lidar com grandes volumes de dados.

Séries

Uma Series é uma coluna de dados unidimensional, semelhante a um array de NumPy, mas com rótulos (índices) associados a cada valor. A Series pode armazenar qualquer tipo de dado, como inteiros, floats, strings, ou objetos.

```
import pandas as pd

# Criando uma Series
s = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
print(s)

# Acessando elementos por índice
print(s['b']) # Saída: 2
```

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
2
```

DataFrame

O DataFrame é a estrutura de dados mais importante do Pandas. Ele é uma tabela bidimensional (semelhante a uma planilha ou tabela SQL) com rótulos para as linhas e colunas. Cada coluna de um DataFrame é uma Series, e as colunas podem ter diferentes tipos de dados.

```
# Criando um DataFrame a partir de um dicionário
dados = {
    'Nome': ['Ana', 'Pedro', 'João'],
    'Idade': [23, 34, 19],
    'Cidade': ['São Paulo', 'Rio de Janeiro', 'Curitiba']
}

df = pd.DataFrame(dados)
print(df)
```

	Nome	Idade	Cidade
0	Ana	23	São Paulo
1	Pedro	34	Rio de Janeiro
2	João	19	Curitiba

Operações Essenciais com DataFrames

1. Selecionar Colunas
2. Filtragem de Dados
3. Alteração de Dados
4. Agrupamento e Agregação
5. Tratamento de Dados Faltantes

```
# Selecionando uma única coluna
print(df['Nome'])
```

```
0      Ana
1    Pedro
2     João
Name: Nome, dtype: object
```

```
# Selecionando múltiplas colunas
print(df[['Nome', 'Cidade']])
```

```
      Nome      Cidade
0     Ana  São Paulo
1  Pedro  Rio de Janeiro
2   João    Curitiba
```

```
# Filtrando o DataFrame para mostrar apenas pessoas com mais de 20 anos
filtro = df[df['Idade'] > 20]
print(filtro)
```

```
      Nome  Idade      Cidade
0     Ana    23  São Paulo
1  Pedro    34  Rio de Janeiro
```

```
# Alterando o valor de uma célula
df.loc[0, 'Idade'] = 24
# Alterando uma coluna inteira
df['Idade'] = df['Idade'] + 1
```

```
# Agrupando por 'Cidade' e calculando a média de 'Idade'
agrupado = df.groupby('Cidade')['Idade'].mean()
print(agrupado)
```

```
Cidade
Curitiba      20.0
Rio de Janeiro 35.0
São Paulo      25.0
Name: Idade, dtype: float64
```

Para ver o tratamento de dados faltantes funcionando, vamos criar uma cópia do DataFrame em que uma das idades é desconhecida. Em Pandas, um valor faltante é representado por `np.nan`:

```
import numpy as np

# Cópia do DataFrame com uma idade faltante
df_faltantes = df.copy()
df_faltantes.loc[1, 'Idade'] = np.nan
print(df_faltantes)
```

	Nome	Idade	Cidade
0	Ana	25.0	São Paulo
1	Pedro	NaN	Rio de Janeiro
2	João	20.0	Curitiba

```
# Contando quantos valores faltantes há em cada coluna
print(df_faltantes.isnull().sum())
```

```
Nome      0
Idade     1
Cidade    0
dtype: int64
```

```
# Removendo as linhas que têm algum dado faltante
df_limpo = df_faltantes.dropna()
print(df_limpo)
```

	Nome	Idade	Cidade
0	Ana	25.0	São Paulo
2	João	20.0	Curitiba

```
# Preenchendo os valores faltantes com a média da coluna.
# Atenção: fillna devolve uma nova coluna, então é preciso atribuir o resultado
# de volta ao DataFrame - só chamar fillna não altera df_faltantes.
df_faltantes['Idade'] = df_faltantes['Idade'].fillna(df_faltantes['Idade'].mean())
print(df_faltantes)
```

	Nome	Idade	Cidade
0	Ana	25.0	São Paulo
1	Pedro	22.5	Rio de Janeiro
2	João	20.0	Curitiba

Estatísticas e Operações com Dados

Pandas facilita o cálculo de estatísticas descritivas, como média, desvio padrão, contagem, entre outras.

```
# Estatísticas descritivas
print(df.describe())
```

	Idade
count	3.000000
mean	26.666667
std	7.637626
min	20.000000
25%	22.500000
50%	25.000000
75%	30.000000
max	35.000000

```
# Soma, contagem e média de colunas específicas
print(df['Idade'].sum())
print(df['Idade'].count())
print(df['Idade'].mean())
```

```
80
3
26.666666666666668
```

Matplotlib: Biblioteca de Visualização de Dados em Python

Matplotlib é uma biblioteca poderosa e flexível para visualização de dados em Python. Ela permite criar uma ampla gama de gráficos, desde simples gráficos de linha até visualizações complexas e altamente customizadas. Embora outras bibliotecas como Seaborn ou Plotly sejam populares para visualizações avançadas, Matplotlib continua sendo o núcleo para muitas dessas bibliotecas e é amplamente utilizado por sua versatilidade e integração com NumPy e Pandas.

A maneira mais comum de usar Matplotlib é através do submódulo pyplot, que fornece uma interface simples para criar gráficos. A estrutura básica de um gráfico com pyplot envolve definir os dados e criar o gráfico com funções que controlam o comportamento do gráfico.

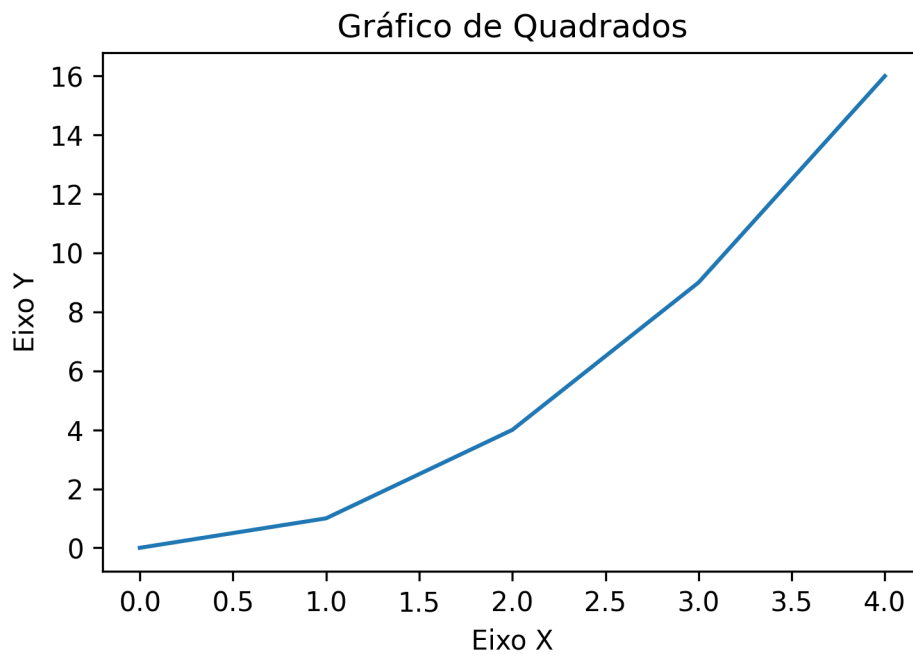
```
import matplotlib.pyplot as plt

# Dados
x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]

# Criando um gráfico de linha
plt.plot(x, y)

# Adicionando título e rótulos aos eixos
plt.title('Gráfico de Quadrados')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')

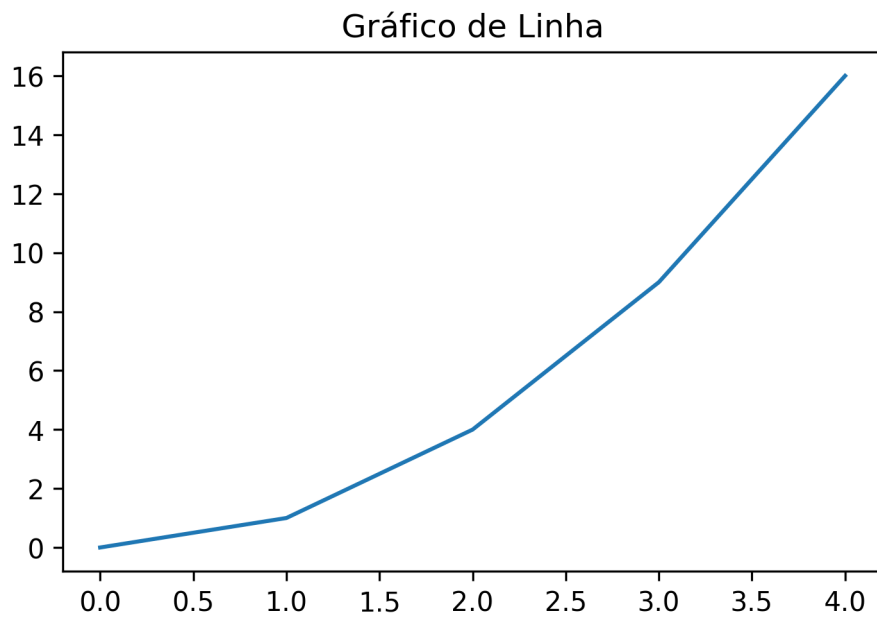
# Exibindo o gráfico
plt.show()
```



Tipos de Gráficos Comuns

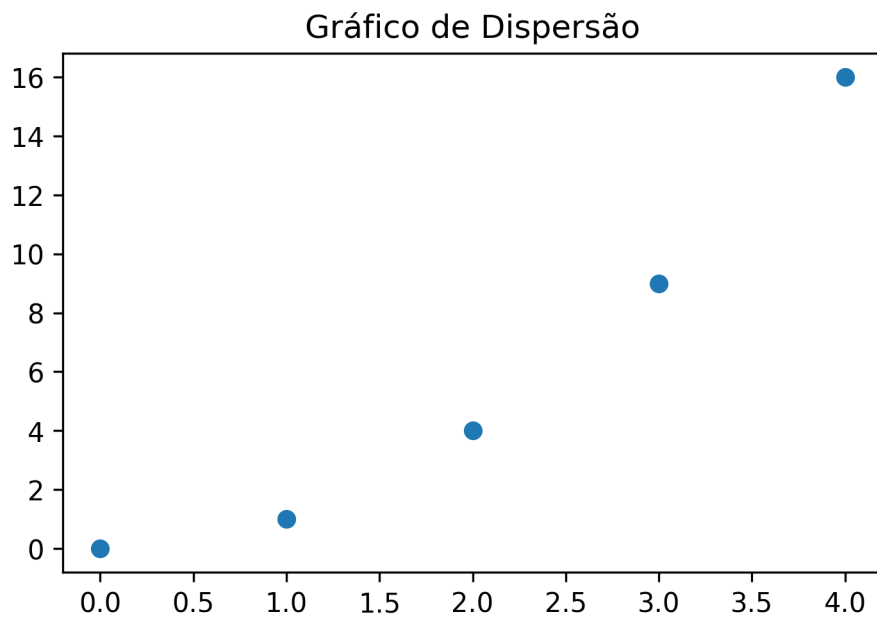
Linha

```
x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]
plt.plot(x, y)
plt.title('Gráfico de Linha')
plt.show()
```



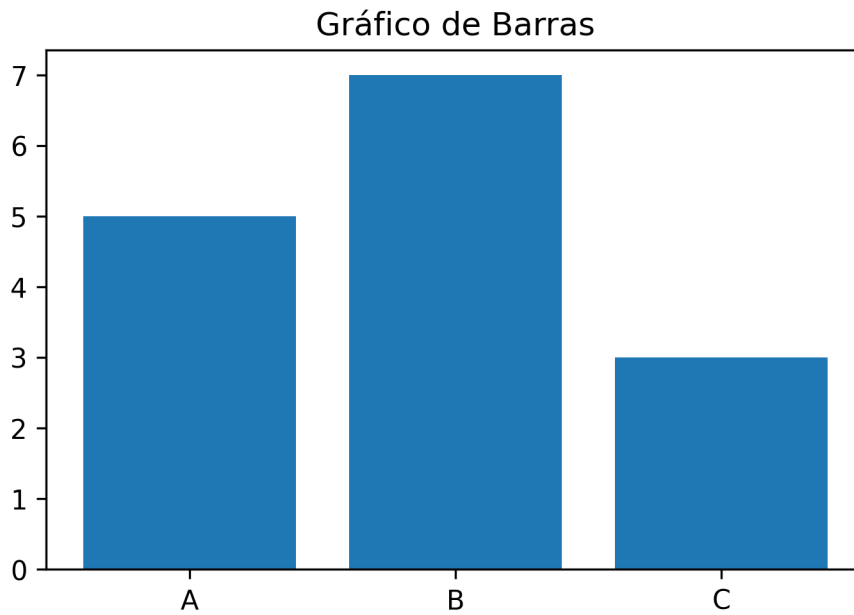
Dispersão

```
plt.scatter(x, y)
plt.title('Gráfico de Dispersão')
plt.show()
```

Barras

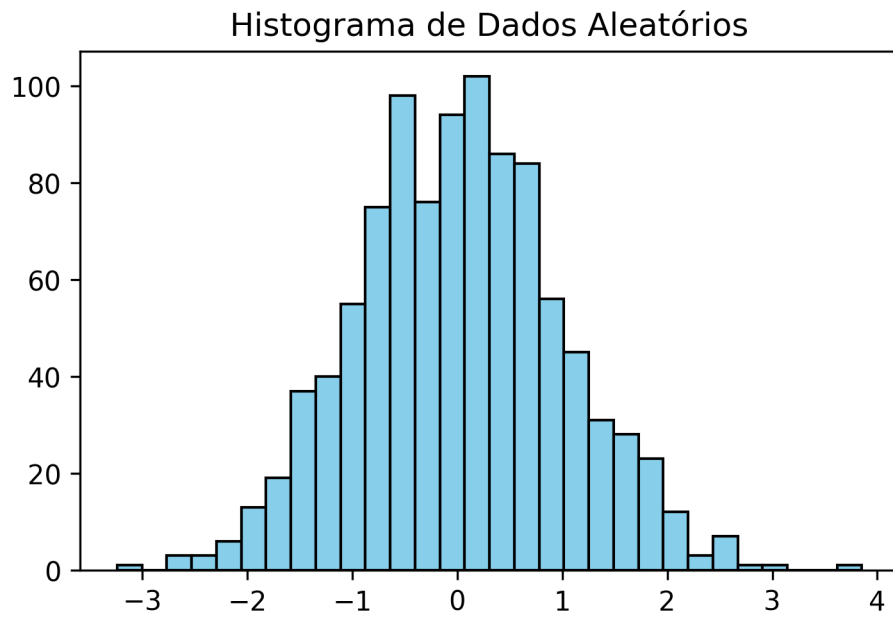
```
categorias = ['A', 'B', 'C']  
valores = [5, 7, 3]  
plt.bar(categorias, valores)  
plt.title('Gráfico de Barras')  
plt.show()
```



Histograma

```
import numpy as np

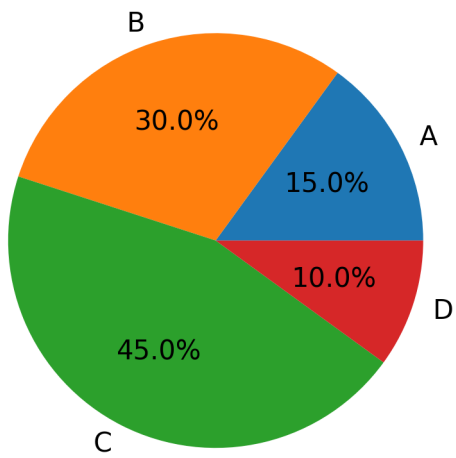
np.random.seed(42) # fixa a semente para que o gráfico seja sempre o mesmo
dados = np.random.randn(1000)
plt.hist(dados, bins=30, color='skyblue', edgecolor='black')
plt.title('Histograma de Dados Aleatórios')
plt.show()
```



Pizza

```
tamanhos = [15, 30, 45, 10]
labels = ['A', 'B', 'C', 'D']
plt.pie(tamanhos, labels=labels, autopct='%1.1f%%')
plt.title('Gráfico de Pizza')
plt.show()
```

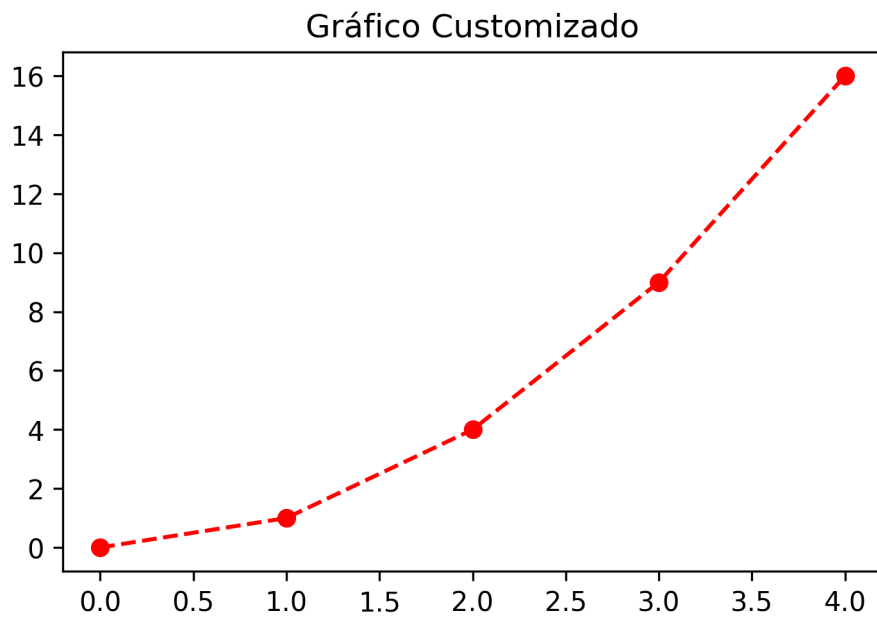
Gráfico de Pizza



Customização

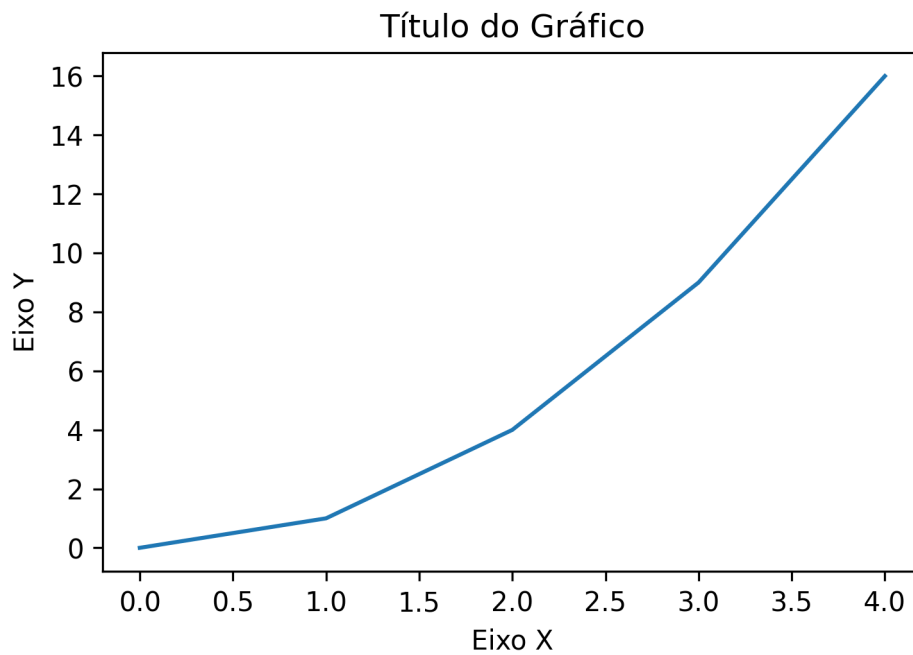
Alterando Cores e Estilos de Linha

```
plt.plot(x, y, color='red', linestyle='--', marker='o') # Linha vermelha com marcador  
plt.title('Gráfico Customizado')  
plt.show()
```



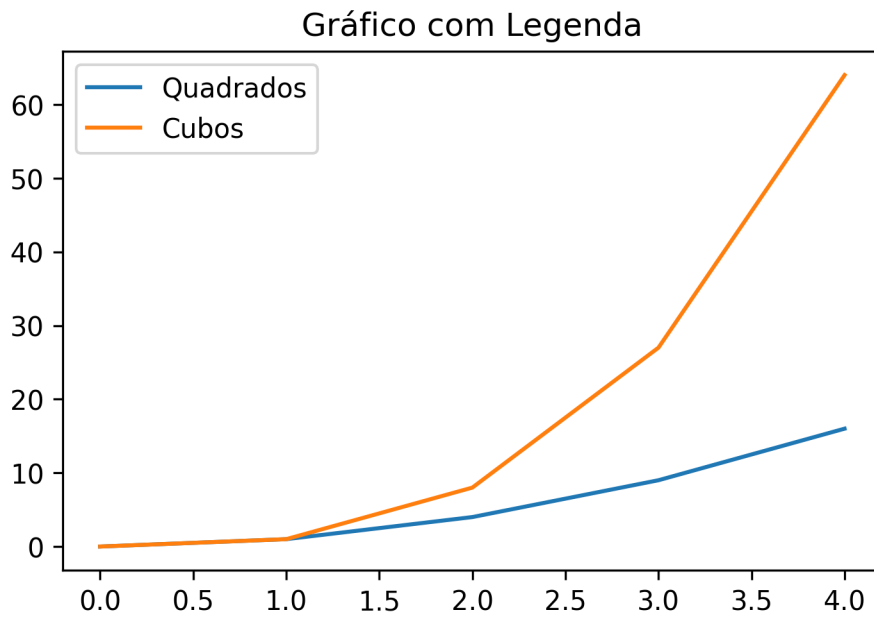
Adicionando Títulos e Rótulos aos Eixos

```
plt.plot(x, y)
plt.title('Título do Gráfico')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.show()
```



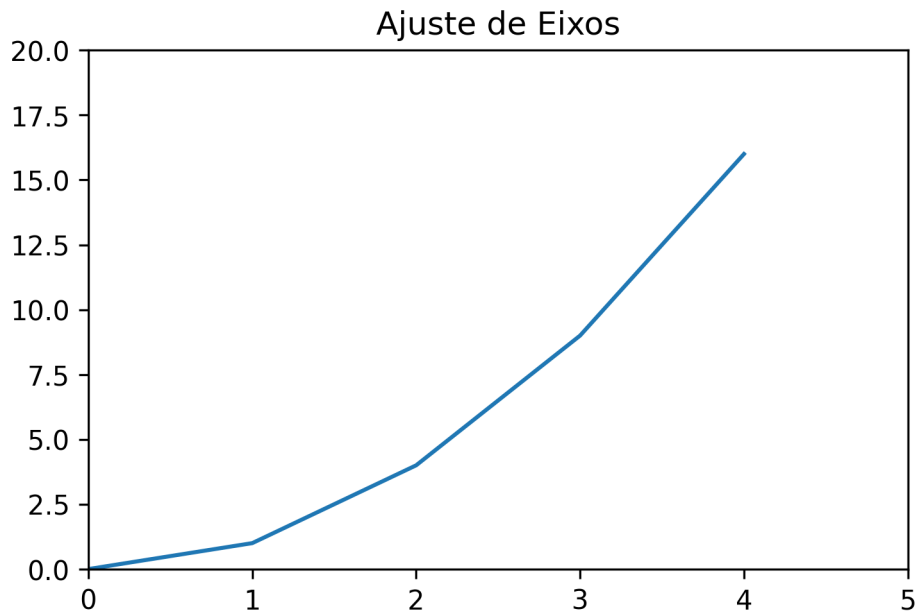
Legendas

```
plt.plot(x, y, label='Quadrados')
plt.plot(x, [i**3 for i in x], label='Cubos')
plt.title('Gráfico com Legenda')
plt.legend() # Adiciona legenda
plt.show()
```



Ajustando os Eixos

```
plt.plot(x, y)
plt.xlim(0, 5)
plt.ylim(0, 20)
plt.title('Ajuste de Eixos')
plt.show()
```



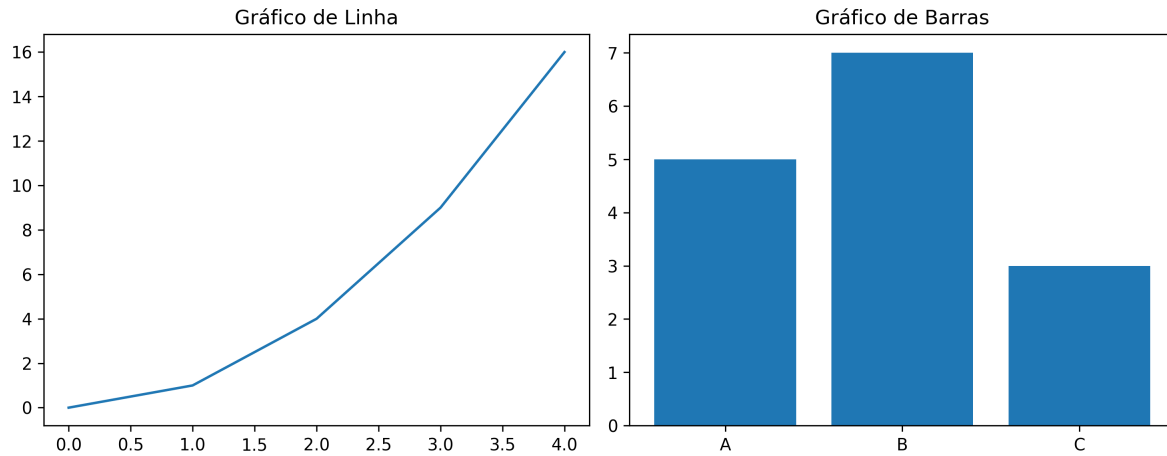
Subplot

```
# Criando uma figura com dois gráficos lado a lado
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1) # 1 linha, 2 colunas, 1º gráfico
plt.plot(x, y)
plt.title('Gráfico de Linha')

plt.subplot(1, 2, 2) # 1 linha, 2 colunas, 2º gráfico
plt.bar(categorias, valores)
plt.title('Gráfico de Barras')

plt.tight_layout()
plt.show()
```

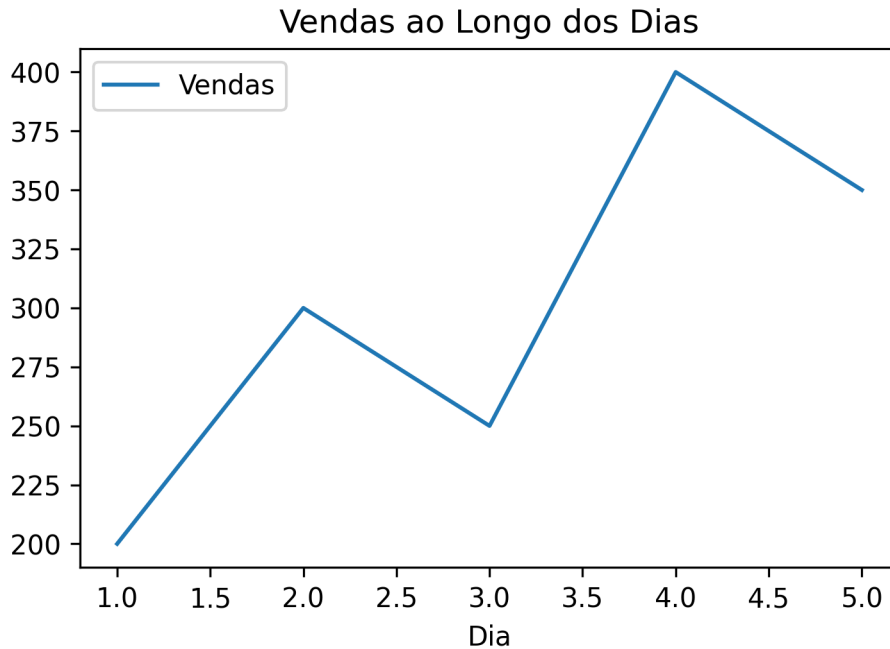



Integração com Pandas

```
import pandas as pd

# Criando um DataFrame
dados = {
    'Dia': [1, 2, 3, 4, 5],
    'Vendas': [200, 300, 250, 400, 350]
}
df = pd.DataFrame(dados)

# Plotando os dados
df.plot(x='Dia', y='Vendas', kind='line')
plt.title('Vendas ao Longo dos Dias')
plt.show()
```



Exercícios

Exercício 1. Execute as seguintes operações no Python e interprete os resultados:

- Use a função `math.log()` para calcular o logaritmo natural de 10 e depois o logaritmo na base 10 de 1000.
- Descubra a função que calcula a raiz quadrada de um número utilizando a função `help()`, e aplique-a ao número 144.

Exercício 2. Armazene o resultado de `2 * 7` em uma variável chamada `resultado`. Use `resultado` para calcular o valor de `resultado ** 2`.

Liste todas as variáveis no ambiente do Python usando a função `globals()`. Exclua a variável `resultado` usando `del` e verifique novamente as variáveis presentes.

Exercício 3. Crie um vetor NumPy `v` contendo os números 4, 8, 15, 16, 23, 42. Use `subsetting` para selecionar:

- O segundo e o quarto elemento.
- Todos os elementos exceto o terceiro.
- Os elementos que são maiores que 10.

Multiplique cada elemento do vetor NumPy `v` por 2 e armazene o resultado em um novo vetor `v2`.

Exercício 4. Verifique se `8 > 5` e se `8 == 10`.

Crie dois vetores NumPy lógicos `a = np.array([True, False, True])` e `b = np.array([False, True, False])`. Aplique os operadores `&`, `|` e `np.logical_xor()` nesses vetores.

Exercício 5. Escreva uma função `soma_quadrados` que receba dois números como argumentos e retorne a soma de seus quadrados. Teste sua função com os números 3 e 4.

Modifique a função `soma_quadrados` para que o segundo argumento tenha um valor padrão de 2. Teste a função chamando-a com apenas um argumento.

Exercício 6. Escreva um laço `for` que calcule a soma dos números de 1 a 100.

Crie um laço `while` que multiplique os números de 1 a 6 e retorne o resultado.

Exercício 7. Crie duas variáveis com o seu primeiro e último nome. Use uma f-string para juntar as duas variáveis em uma frase que diga “Meu nome completo é [nome completo]”.

Altere o separador para um traço `-` e junte novamente as variáveis.

Exercício 8. Crie um DataFrame com os dados fictícios a seguir utilizando `pandas`:

Nome	Idade	Cidade
Ana	23	São Paulo
Pedro	34	Rio de Janeiro
João	19	Curitiba

Filtre as observações onde a idade é maior que 20.

Ordene o DataFrame pela coluna `Idade` em ordem decrescente.

Exercício 9. Usando o DataFrame criado no exercício anterior, crie um gráfico de barras (`plt.bar`) que mostre a idade de cada pessoa.

No gráfico anterior, mude a cor de cada barra.

Exercício 10. Crie um DataFrame com os dados de temperatura de uma semana:

Dia	Temperatura (°C)
Seg	22
Ter	24
Qua	19
Qui	21
Sex	23

Dia	Temperatura (°C)
Sáb	25
Dom	20

Agora, responda às perguntas abaixo:

1. Filtre todos os dias em que a temperatura foi maior que 21°C.
2. Calcule a média das temperaturas da semana.
3. Crie um gráfico de linha (`plt.plot`) que mostre a variação da temperatura ao longo da semana.

Breve Introdução ao R

Aqui faremos uma breve introdução ao R. Para uma introdução mais detalhada, veja <https://curso-r.com/material/> e <https://r4ds.had.co.nz/>.

Onde escrever seu código

O R propriamente dito é só o motor que executa os comandos; na prática, escrevemos o código em um editor que conversa com esse motor. Ou seja, são dois programas diferentes: primeiro instale o R (<https://cran.r-project.org>), depois um dos ambientes abaixo. Todos são gratuitos e rodam em Windows, macOS e Linux.

- **RStudio** (<https://posit.co/download/rstudio-desktop/>): a IDE clássica para R e a mais usada em cursos de Estatística. Reúne na mesma janela o console, a lista de variáveis criadas, os gráficos e a ajuda.
- **Positron** (<https://positron.posit.co>): uma IDE mais recente, feita pela Posit — a mesma empresa do RStudio — sobre a base do VS Code. Tem painéis parecidos com os do RStudio, mas trabalha com R e Python no mesmo programa, o que é conveniente aqui: todos os exemplos deste livro aparecem nas duas linguagens.
- **VS Code** (<https://code.visualstudio.com>): editor de uso geral, muito popular fora da Estatística. Para programar em R nele, instale a extensão *R* e, dentro do R, o pacote *languageserver*. É a opção que exige mais configuração inicial.

Se você não tem preferência, comece pelo RStudio (é o caminho mais curto) ou pelo Positron (se quiser um único programa para os capítulos em R e os em Python).

Vale notar que nenhuma dessas escolhas muda o que o código faz: o R é o mesmo nos três, e todo o código deste livro roda igual em qualquer um deles. Os três também abrem os arquivos *.qmd* (Quarto) em que este livro foi escrito — RStudio e Positron já vêm com suporte a Quarto, e no VS Code basta instalar a extensão *Quarto*.

Variáveis Numéricas

Aqui vão alguns exemplos para começarmos a entender como o R funciona. Inicialmente, veremos como podemos usar o R como calculadora.

```
2 * 7
```

```
[1] 14
```

```
0 / 0
```

```
[1] NaN
```

```
10^1000
```

```
[1] Inf
```

```
log(1)
```

```
[1] 0
```

```
exp(1)
```

```
[1] 2.718282
```

Repare em duas saídas especiais: `0/0` devolve `NaN` (de *not a number*, uma conta sem resultado definido) e `10^1000` devolve `Inf`, porque o resultado é grande demais para ser representado. O R não interrompe a execução nesses casos — ele segue em frente carregando esses valores, então vale ficar atento quando eles aparecem.

Para entender o que uma função faz, você pode digitar o símbolo de interrogação seguido do nome da função, por exemplo:

```
?exp
```

O help contém as seguintes informações:

- **Description:** Resumo geral sobre o uso da função.
- **Usage:** Mostra como a função deve ser utilizada e quais argumentos podem ser especificados.
- **Arguments:** Explica o que é cada um dos argumentos.
- **Details:** Explica alguns detalhes sobre o uso e aplicação da função.
- **Value:** Mostra o que sai no output após usar a função (os resultados).
- **Note:** Notas sobre a função.
- **Authors:** Lista os autores da função.
- **References:** Referências para os métodos usados.
- **See also:** Outras funções relacionadas que podem ser consultadas.
- **Examples:** Exemplos do uso da função.

Armazenando resultados em variáveis

Podemos também armazenar resultados de contas em variáveis. Por exemplo:

```
x <- 2 + 3  
x
```

```
[1] 5
```

```
y <- 2 * x  
y
```

```
[1] 10
```

```
print(y)
```

```
[1] 10
```

i <- ou =?

O R aceita os dois símbolos para atribuir um valor a uma variável: `x <- 2` e `x = 2` fazem a mesma coisa. Por convenção, usa-se `<-` para atribuição e reserva-se `=` para nomear argumentos de funções, como em `seq(from = 1, to = 10)`. É essa convenção que seguiremos no livro todo. No RStudio e no Positron, o atalho `alt + =` escreve `<-` de uma vez.

Números grandes são impressos usando notação científica:

```
y <- 2 * 10^10  
print(y)
```

```
[1] 2e+10
```

Para listar quais variáveis estão declaradas no ambiente, podemos usar:

```
ls()
```

```
[1] "pandoc_dir"      "quarto_bin_path" "x"                "y"
```

Para remover uma variável:

```
rm(x)
ls()
```

```
[1] "pandoc_dir"      "quarto_bin_path" "y"
```

Para remover todas as variáveis existentes:

```
rm(list = ls()) # Essa função apaga todas as variáveis existentes
gc()           # Essa função libera a memória utilizada
```

```
      used (Mb) gc trigger (Mb) max used (Mb)
Ncells 613273 32.8   1295652 69.2  1295652 69.2
Vcells 1157172  8.9   8388608 64.0   1973977 15.1
```

Variáveis Lógicas

Alguns operadores lógicos definidos no R são mostrados na tabela abaixo:

#	Operador	Descrição
1	$x < y$	x menor que y?
2	$x \leq y$	x menor ou igual a y?
3	$x > y$	x maior que y?
4	$x \geq y$	x maior ou igual a y?
5	$x == y$	x igual a y?
6	$x != y$	x diferente de y?
7	$!x$	Negativa de x
8	$x \mid y$	x ou y são verdadeiros?
9	$x \& y$	x e y são verdadeiros?
10	$\text{xor}(x, y)$	Apenas um dos dois é verdadeiro?

Exemplos de uso de operadores lógicos

```
1 == 3
```

```
[1] FALSE
```



```
1 == 1
```

```
[1] TRUE
```

```
1 <= 3
```

```
[1] TRUE
```

```
x <- 1 > 3  
print(x)
```

```
[1] FALSE
```

```
x <- 1; y <- 3  
x < y
```

```
[1] TRUE
```

```
(x == 1) & (y == 3)
```

```
[1] TRUE
```

```
(x == 1) & (y != 3)
```

```
[1] FALSE
```

```
!TRUE
```

```
[1] FALSE
```

```
x <- TRUE; y <- FALSE  
x | y
```

```
[1] TRUE
```

O número um tem o valor verdadeiro, e o número zero tem o valor falso:

```
x <- 1; y <- 0
x | y
```

```
[1] TRUE
```

Note que ao fazermos contas com variáveis lógicas, elas são transformadas em zero e um:

```
(TRUE | FALSE) + 2
```

```
[1] 3
```

Caracteres/Strings

Para declarar cadeias de caracteres, usamos aspas no R:

```
x <- "Rafael"
y <- "Izbicki"
```

Várias operações podem ser feitas com esses objetos. Um exemplo é a função `paste`:

```
paste(x, y)
```

```
[1] "Rafael Izbicki"
```

```
paste(x, y, sep = "+")
```

```
[1] "Rafael+Izbicki"
```

Vetores e Sequências

Usualmente declaramos vetores usando o operador `c` (concatenação):

```
x <- c(2, 5, 7, 1)
x
```

```
[1] 2 5 7 1
```

Para acessar seus componentes, fazemos:

```
x[2]
```

```
[1] 5
```

```
x[c(2, 3)]
```

```
[1] 5 7
```

```
x[-c(2, 3)]
```

```
[1] 2 1
```

As operações `x[c(2, 3)]` e `x[-c(2, 3)]` são chamadas de *subsetting*; subsetting é a seleção de um subconjunto de um objeto. O índice positivo indica quais elementos manter, e o negativo indica quais descartar. Veremos mais à frente outras maneiras de fazermos isso.

 **Atenção:** no R os índices começam em 1

O primeiro elemento de um vetor é `x[1]`, não `x[0]`. Se você também usa Python, cuidado: lá a contagem começa em 0.

Podemos mudar alguns dos valores deste vetor usando:

```
x[2:3] <- 0  
x
```

```
[1] 2 0 0 1
```

Operações aritméticas podem ser facilmente feitas para cada elemento de um vetor. Alguns exemplos:

```
x <- x - 1  
x
```

```
[1] 1 -1 -1 0
```

```
x <- 2 * x
x
```

```
[1] 2 -2 -2 0
```

Uma função útil para criar vetores é a `seq`:

```
x <- seq(from = 1, to = 100, by = 2)
x
```

```
[1] 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49
[26] 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97 99
```

```
y <- seq(from = 1, to = 100, length = 5)
y
```

```
[1] 1.00 25.75 50.50 75.25 100.00
```

Podemos também usar operadores lógicos em vetores:

```
x > 5
```

```
[1] FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[13] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[25] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[37] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[49] TRUE TRUE
```

Note que `x > 5` é um vetor lógico.

Podemos usar as operações lógicas em vetores lógicos:

```
x <- c(TRUE, TRUE, FALSE, FALSE)
y <- c(TRUE, FALSE, TRUE, FALSE)
x | y
```

```
[1] TRUE TRUE TRUE FALSE
```

```
x & y
```

```
[1] TRUE FALSE FALSE FALSE
```

Operadores lógicos também podem ser usados para fazer *subsetting*. A ideia é que `x[condicao]` seleciona os elementos de `x` nas posições em que `condicao` vale `TRUE`:

```
x <- c(2, 8, 0, 15)

x[x > 5] <- 0 # zera os elementos maiores que 5
x
```

```
[1] 2 0 0 0
```

```
x[!(x <= 0)] <- -1 # troca por -1 os elementos que não são menores ou iguais a 0
x
```

```
[1] -1 0 0 0
```

Podemos ter vetores de caracteres:

```
x <- c("a", "c", "fgh")
x[-c(2, 3)]
```

```
[1] "a"
```

```
paste(x, "add")
```

```
[1] "a add" "c add" "fgh add"
```

```
paste(x, "add", collapse = "+")
```

```
[1] "a add+c add+fgh add"
```

Algumas funções adicionais úteis:

```
rep(2, 5)
```

```
[1] 2 2 2 2 2
```

```
rep(c(1, 2), 5)
```

```
[1] 1 2 1 2 1 2 1 2 1 2
```

```
rep(c(1, 2), each = 5)
```

```
[1] 1 1 1 1 1 2 2 2 2 2
```

```
sort(c(5, 10, 0, 20))
```

```
[1] 0 5 10 20
```

```
order(c(5, 10, 0, 20))
```

```
[1] 3 1 2 4
```

Funções

Para declarar funções em R, fazemos:

```
minha_funcao <- function(argumento1, argumento2) {  
  # corpo da função  
}
```

Por exemplo:

```
potencia <- function(x, y) {  
  return(x^y)  
}  
potencia(2, 3)
```

```
[1] 8
```

Note que `x` e `y` podem ser vetores:

```
potencia(x = c(1, 2, 3), y = c(0, 1, 2))
```

```
[1] 1 2 9
```

Você pode inverter a ordem dos argumentos, desde que eles sejam nomeados:

```
potencia(y = c(0, 1, 2), x = c(1, 2, 3))
```

```
[1] 1 2 9
```

Argumentos com valores default

Os argumentos podem ter valores *default*. Por exemplo:

```
potencia <- function(x, y = rep(1, length(x))) {  
  return(x^y)  
}
```

Neste caso, se o argumento *y* não for fornecido, ele será um vetor de uns do tamanho de *x*:

```
potencia(2)
```

```
[1] 2
```

```
potencia(2, 3)
```

```
[1] 8
```

Laços

Para calcular o fatorial de um número *n*, podemos usar um laço **while**:

```
n <- 4
i <- 1
n_fatorial <- 1
while(i <= n) {
  n_fatorial <- n_fatorial * i
  i <- i + 1
}
n_fatorial
```

```
[1] 24
```

Ou podemos usar um laço for:

```
n <- 4
n_fatorial <- 1
for(i in 1:n) {
  n_fatorial <- n_fatorial * i
}
n_fatorial
```

```
[1] 24
```

Lembre-se de que laços podem ser lentos no R, especialmente se o tamanho do objeto não for previamente declarado. Vamos comparar o tempo de execução de diferentes abordagens usando a função `system.time`. Para isso, vamos calcular a soma acumulada de um vetor.

```
x <- 1:100000
```

1. Laço sem declaração de tamanho:

```
aux <- function(x) {
  soma_acumulada <- NULL
  soma_acumulada[1] <- x[1]
  for(i in 2:length(x)) {
    soma_acumulada[i] <- soma_acumulada[i - 1] + x[i]
  }
}
system.time(aux(x))[1]
```

```
user.self
0.029
```


2. Laço com declaração de tamanho:

```
aux <- function(x) {  
  soma_acumulada <- rep(NA, length(x))  
  soma_acumulada[1] <- x[1]  
  for(i in 2:length(x)) {  
    soma_acumulada[i] <- soma_acumulada[i - 1] + x[i]  
  }  
}  
system.time(aux(x))[1]
```

```
user.self  
0.015
```

3. Função nativa do R:

```
aux <- function(x) {  
  cumsum(x)  
}  
system.time(aux(x))[1]
```

```
user.self  
0.001
```

Note que, em geral, funções nativas do R são muito mais rápidas.

If/Else

O R permite o uso de condicionais `if` e `else` para controlar o fluxo de execução. A sintaxe básica é:

```
if (condição1) {  
  # código se a condição1 for verdadeira  
} else if (condição2) {  
  # código se a condição1 for falsa e condição2 verdadeira  
} else {  
  # código se todas as condições forem falsas  
}
```

Exemplo:

```

testa_x <- function(x) {
  if (!is.numeric(x)) {
    print("x não é um número")
  } else if (x > 0) {
    print("x é positivo")
  } else if (x < 0) {
    print("x é negativo")
  } else {
    print("x é nulo")
  }
}
testa_x(5)

```

```
[1] "x é positivo"
```

```
testa_x(-5)
```

```
[1] "x é negativo"
```

```
testa_x("5")
```

```
[1] "x não é um número"
```

Listas

Listas são como vetores, mas podem conter componentes de diferentes tipos (inclusive outras listas).

```

minha_lista <- list("um", TRUE, 3, c("q", "u", "a", "t", "r", "o"), "cinco")
minha_lista[[3]]

```

```
[1] 3
```

Você também pode atribuir nomes aos elementos de uma lista:

```

minha_lista <- list(primeiro = "um", segundo = TRUE, terceiro = 3,
                    quarto = c("q", "u", "a", "t", "r", "o"), quinto = "cinco")
minha_lista$quinto

```

```
[1] "cinco"
```

Listas são frequentemente usadas para retornar várias saídas de uma função. Por exemplo, ao ajustar um modelo de regressão linear, os resultados são armazenados em uma lista:

```
set.seed(42) # fixa a semente para que os números sorteados sejam sempre os mesmos
x <- rnorm(100)
y <- 1 + 2*x + rnorm(length(x), sd = 0.1)
ajuste <- lm(y ~ x)
typeof(ajuste)
```

```
[1] "list"
```

Os componentes da lista podem ser acessados com:

```
names(ajuste)
```

```
[1] "coefficients" "residuals"      "effects"        "rank"
[5] "fitted.values" "assign"          "qr"             "df.residual"
[9] "xlevels"       "call"           "terms"          "model"
```

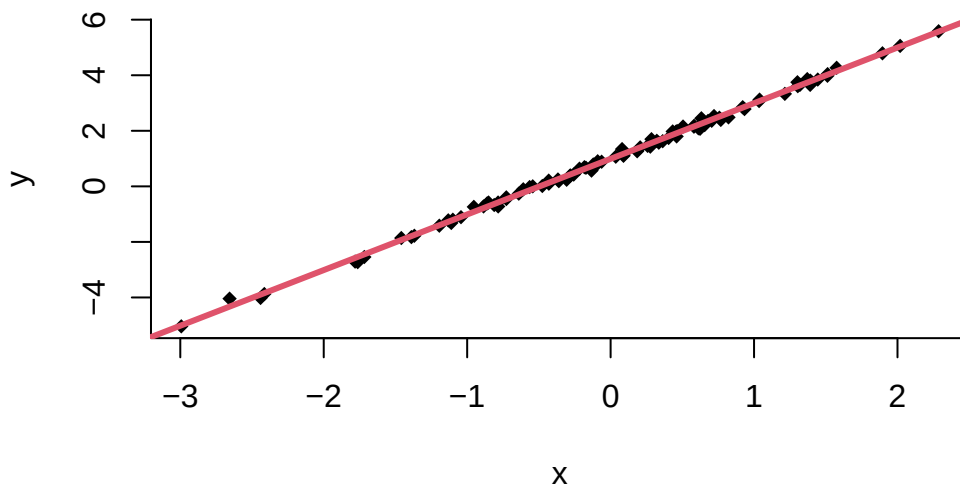
Para ver os coeficientes ajustados:

```
ajuste$coefficients
```

```
(Intercept)          x
  0.9911633    2.0027159
```

Para adicionar a reta estimada ao gráfico:

```
plot(x, y, pch = 18, bty = "l")
abline(ajuste, col = 2, lwd = 3)
```



Você também pode inicializar uma lista vazia:

```
minha_lista <- list()
```

Estatística Descritiva

O R é uma ferramenta poderosa para realizar análises descritivas. Vamos explorar alguns recursos para calcular estatísticas de variáveis quantitativas e qualitativas.

Variáveis quantitativas

```
x <- c(30, 1, 20, 5, 20, 60)
y <- c(12, 0, 2, 15, 22, 20)

mean(x)
```

```
[1] 22.66667
```

```
var(x)
```

```
[1] 448.6667
```

```
min(x)
```

```
[1] 1
```

```
which.min(x)
```

```
[1] 2
```

```
max(x)
```

```
[1] 60
```

```
which.max(x)
```

```
[1] 6
```

```
sort(x)
```

```
[1] 1 5 20 20 30 60
```

```
order(x)
```

```
[1] 2 4 3 5 1 6
```

```
median(x)
```

```
[1] 20
```

```
cor(x, y)
```

```
[1] 0.5229211
```

```
which(x > 10)
```

```
[1] 1 3 5 6
```

```
mean(y[x > 10])
```

```
[1] 14
```

Variáveis qualitativas

```
x <- c("S", "R", "P", "P", "Q", "P")  
y <- c("a", "a", "b", "a", "c", "d")  
  
x[y %in% c("a", "c")]
```

```
[1] "S" "R" "P" "Q"
```

```
table(x)
```

```
x  
P Q R S  
3 1 1 1
```

```
table(x, y)
```

```
      y  
x     a b c d  
P  1 1 0 1  
Q  0 0 1 0  
R  1 0 0 0  
S  1 0 0 0
```

Subconjuntos

Podemos utilizar subsetting para trabalhar com subconjuntos de interesse. Por exemplo, suponha que `dados` é um banco de dados com informações de alunos e queremos selecionar os alunos com idade menor que 20, altura maior que 1,70 e que estejam no primeiro ou segundo ano.

```
dados[dados$idade < 20 & dados$altura > 1.70 & dados$ano %in% c("Primeiro", "Segundo"), ]
```

Funções Apply

As funções da família `apply` (como `apply`, `lapply`, `sapply`) são úteis para aplicar uma função a um conjunto de dados. Vamos criar um banco de dados artificial para exemplificar:

```
set.seed(42)
dados <- data.frame(altura = rnorm(100, 1.7, 0.2), salario = rnorm(100, 3000, 500))
dados$peso <- dados$altura * 35 + rnorm(100, 0, 10)
```

`apply`

Para calcular a média de cada coluna de `dados`, usamos:

```
apply(dados, 2, mean)
```

	altura	salario	peso
	1.706503	2956.258146	59.623922

Aqui, o argumento 2 indica que a operação deve ser aplicada a cada coluna. Para calcular a soma de cada linha:

```
apply(dados, 1, sum)
```

[1]	3651.544	3582.847	2573.923	4010.593	2716.955	3101.684	2853.897	2988.803
[9]	3163.372	3118.475	3046.037	3152.070	2809.560	2806.243	2234.647	2874.986
[17]	2801.507	4407.787	2360.400	3126.497	2316.036	2309.642	3117.095	2560.945
[25]	3078.218	2842.231	2757.669	2033.823	2445.554	3158.853	3345.572	2829.316
[33]	3056.668	3613.599	3782.070	2496.371	3010.379	3655.595	2811.395	3025.802
[41]	3012.337	2624.742	2856.902	3053.734	2830.608	3641.509	2825.030	2854.746
[49]	3413.561	2528.023	3032.207	2280.274	3644.145	2930.906	2840.901	2433.726

```
[57] 3054.825 2662.171 2762.728 3703.256 2979.521 2536.338 3150.831 2871.393
[65] 3350.430 3797.437 2575.404 3294.094 3083.279 3514.783 2944.538 3479.318
[73] 2194.736 3903.380 3485.715 3003.095 2346.931 3373.617 3310.235 3062.148
[81] 3156.060 2754.376 3240.547 3214.012 2913.408 2400.968 3434.607 3328.799
[89] 2628.635 2272.560 3166.827 2889.688 3184.065 2446.335 2572.380 3593.116
[97] 3253.276 3352.563 3970.364 3114.058
```

Funções anônimas

Usando funções anônimas, podemos calcular várias estatísticas descritivas ao mesmo tempo:

```
estatisticas <- apply(dados, 2, function(x) {
  lista_resultados <- list()
  lista_resultados$media <- mean(x)
  lista_resultados$media_aparada <- mean(x, trim = 0.1)
  lista_resultados$mediana <- median(x)
  lista_resultados$maximo <- max(x)
  return(lista_resultados)
})

estatisticas$altura
```

```
$media
[1] 1.706503
```

```
$media_aparada
[1] 1.717016
```

```
$mediana
[1] 1.717959
```

```
$maximo
[1] 2.157329
```

O operador %>% (pipe)

O operador pipe foi uma das grandes revoluções recentes do R, tornando o código mais legível. Esse operador está definido no pacote **magrittr**, e vários outros pacotes, como o **dplyr**, fazem uso dele (veja a próxima seção).

A ideia é simples: o operador %>% usa o resultado do lado esquerdo como o primeiro argumento da função do lado direito. Um exemplo:

```
library(magrittr)
x <- c(1, 2, 3, 4)
x %>% sum() %>% sqrt()
```

```
[1] 3.162278
```

Isso é equivalente a:

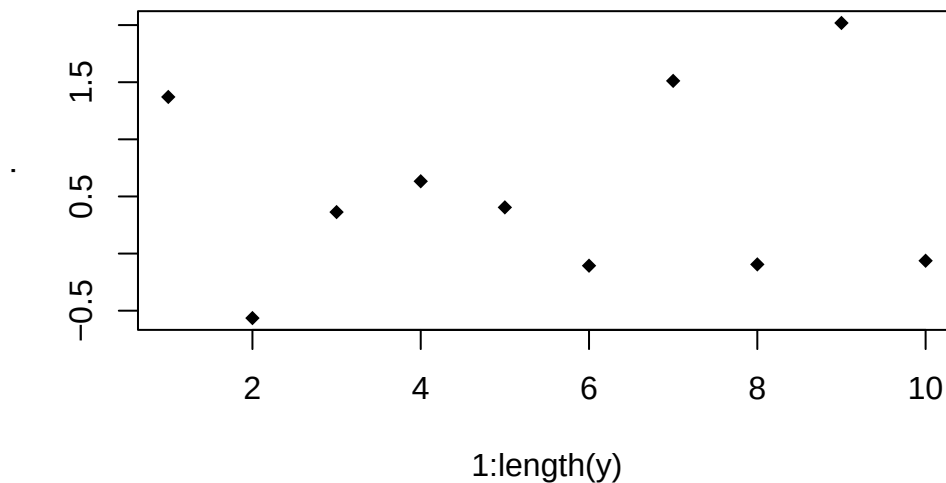
```
sqrt(sum(x))
```

```
[1] 3.162278
```

Mas a leitura com pipe é mais clara. No RStudio e no Positron, você pode inserir o pipe com o atalho `ctrl + shift + m`.

O pipe envia o valor à esquerda apenas para o primeiro argumento da função à direita. Para utilizar o valor da esquerda em outro argumento, utilize o `"."`:

```
set.seed(42)
y <- rnorm(10)
y %>% plot(x = 1:length(y), y = ., pch = 18)
```



O pacote dplyr

O pacote `dplyr` é muito útil para manipulação eficiente de data frames. Vamos demonstrar alguns exemplos usando o conjunto de dados `hflights`.

```
library(dplyr)
library(hflights)
data(hflights)
```

Primeiro, converta os dados para o formato `tibble`, uma variação mais amigável do `data.frame`:

```
flights <- as_tibble(hflights)
```

Filter

A função `filter` retorna todas as linhas que satisfazem uma condição. Por exemplo, para mostrar todos os voos do dia 1º de janeiro:

```
flights %>% filter(Month == 1, DayofMonth == 1)
```

```
# A tibble: 552 x 21
```

	Year	Month	DayofMonth	DayOfWeek	DepTime	ArrTime	UniqueCarrier	FlightNum
	<int>	<int>	<int>	<int>	<int>	<int>	<chr>	<int>
1	2011	1	1	6	1400	1500	AA	428
2	2011	1	1	6	728	840	AA	460
3	2011	1	1	6	1631	1736	AA	1121
4	2011	1	1	6	1756	2112	AA	1294
5	2011	1	1	6	1012	1347	AA	1700
6	2011	1	1	6	1211	1325	AA	1820
7	2011	1	1	6	557	906	AA	1994
8	2011	1	1	6	1824	2106	AS	731
9	2011	1	1	6	654	1124	B6	620
10	2011	1	1	6	1639	2110	B6	622

```
# i 542 more rows
```

```
# i 13 more variables: TailNum <chr>, ActualElapsedTime <int>, AirTime <int>,  
#   ArrDelay <int>, DepDelay <int>, Origin <chr>, Dest <chr>, Distance <int>,  
#   TaxiIn <int>, TaxiOut <int>, Cancelled <int>, CancellationCode <chr>,  
#   Diverted <int>
```

Para condições alternativas, podemos usar o operador | (ou) ou %in%:

```
flights %>% filter(UniqueCarrier %in% c("AA", "UA"))
```

```
# A tibble: 5,316 x 21
```

	Year	Month	DayofMonth	DayOfWeek	DepTime	ArrTime	UniqueCarrier	FlightNum
	<int>	<int>	<int>	<int>	<int>	<int>	<chr>	<int>
1	2011	1	1	6	1400	1500	AA	428
2	2011	1	2	7	1401	1501	AA	428
3	2011	1	3	1	1352	1502	AA	428
4	2011	1	4	2	1403	1513	AA	428
5	2011	1	5	3	1405	1507	AA	428
6	2011	1	6	4	1359	1503	AA	428
7	2011	1	7	5	1359	1509	AA	428
8	2011	1	8	6	1355	1454	AA	428
9	2011	1	9	7	1443	1554	AA	428
10	2011	1	10	1	1443	1553	AA	428

```
# i 5,306 more rows
```

```
# i 13 more variables: TailNum <chr>, ActualElapsedTime <int>, AirTime <int>,  
#   ArrDelay <int>, DepDelay <int>, Origin <chr>, Dest <chr>, Distance <int>,  
#   TaxiIn <int>, TaxiOut <int>, Cancelled <int>, CancellationCode <chr>,  
#   Diverted <int>
```

Select

A função `select` seleciona colunas específicas de um data frame. Para selecionar as colunas `DepTime`, `ArrTime`, e `FlightNum`:

```
flights %>% select(DepTime, ArrTime, FlightNum)
```

```
# A tibble: 227,496 x 3
```

	DepTime	ArrTime	FlightNum
	<int>	<int>	<int>
1	1400	1500	428
2	1401	1501	428
3	1352	1502	428
4	1403	1513	428
5	1405	1507	428
6	1359	1503	428
7	1359	1509	428
8	1355	1454	428

```

  9      1443      1554      428
10      1443      1553      428
# i 227,486 more rows

```

Para selecionar todas as colunas que contêm “Taxi” ou “Delay”:

```
flights %>% select(contains("Taxi"), contains("Delay"))
```

```

# A tibble: 227,496 x 4
   TaxiIn TaxiOut ArrDelay DepDelay
   <int>   <int>   <int>   <int>
1       7      13     -10         0
2       6       9      -9         1
3       5      17      -8        -8
4       9      22       3         3
5       9       9      -3         5
6       6      13      -7        -1
7      12      15      -1        -1
8       7      12     -16        -5
9       8      22      44        43
10      6      19      43        43
# i 227,486 more rows

```

Arrange

A função `arrange` ordena o data frame de acordo com algum critério. Para ordenar pelo atraso de partida (`DepDelay`):

```
flights %>% arrange(DepDelay)
```

```

# A tibble: 227,496 x 21
   Year Month DayOfMonth DayOfWeek DepTime ArrTime UniqueCarrier FlightNum
   <int> <int>      <int>      <int>   <int>   <int>   <chr>          <int>
1  2011    12         24          6   1112    1314  OO            5440
2  2011     2         14          1   1917    2027  MQ            3328
3  2011     4         10          7   2101    2206  XE            2669
4  2011     8          3          3   1741    1810  XE            2603
5  2011     1         18          2   1542    1936  CO            1688
6  2011    10          4          2   1438    1813  EV            5412
7  2011     1        26          3   2248    2343  XE            2450

```

```

 8 2011      3          8          2      953      1156 CO          1882
 9 2011      3         18          5     2103      2156 XE          2261
10 2011      4          3          7     1048      1307 MQ          3796
# i 227,486 more rows
# i 13 more variables: TailNum <chr>, ActualElapsedTime <int>, AirTime <int>,
#   ArrDelay <int>, DepDelay <int>, Origin <chr>, Dest <chr>, Distance <int>,
#   TaxiIn <int>, TaxiOut <int>, Cancelled <int>, CancellationCode <chr>,
#   Diverted <int>

```

Para ordem decrescente:

```
flights %>% arrange(desc(DepDelay))
```

```

# A tibble: 227,496 x 21
   Year Month DayOfMonth DayOfWeek DepTime ArrTime UniqueCarrier FlightNum
   <int> <int>      <int>      <int>   <int>   <int>   <chr>          <int>
1  2011      8          1          1     156     452 CO             1
2  2011     12         12          1     650     808 AA            1740
3  2011     11          8          2     721     948 MQ            3786
4  2011      6         21          2    2334     124 UA             855
5  2011      6          9          4    2029    2243 MQ            3859
6  2011      5         20          5     858    1027 MQ            3328
7  2011      1         20          4     635     807 CO              59
8  2011      6         22          3     908    1040 CO             595
9  2011     10         25          2    2310     149 DL            1215
10 2011     12         13          2     706     824 MQ            3328
# i 227,486 more rows
# i 13 more variables: TailNum <chr>, ActualElapsedTime <int>, AirTime <int>,
#   ArrDelay <int>, DepDelay <int>, Origin <chr>, Dest <chr>, Distance <int>,
#   TaxiIn <int>, TaxiOut <int>, Cancelled <int>, CancellationCode <chr>,
#   Diverted <int>

```

Mutate

A função `mutate` cria novas variáveis. Para criar a variável `Speed` (velocidade) e adicioná-la ao banco:

```
flights <- flights %>% mutate(Speed = Distance / AirTime * 60)
```

Summarise e Group_by

A função `summarise` calcula estatísticas resumo. Para calcular o atraso médio de chegada por destino:

```
flights %>% group_by(Dest) %>% summarise(avg_delay = mean(ArrDelay, na.rm = TRUE))
```

```
# A tibble: 116 x 2
  Dest   avg_delay
  <chr>   <dbl>
1 ABQ     7.23
2 AEX     5.84
3 AGS      4
4 AMA     6.84
5 ANC    26.1
6 ASE     6.79
7 ATL     8.23
8 AUS     7.45
9 AVL     9.97
10 BFL    -13.2
# i 106 more rows
```

A função `across`, usada dentro de `summarise`, aplica uma função a várias colunas ao mesmo tempo. Para calcular a média de `Cancelled` e `Diverted` por companhia aérea:

```
flights %>% group_by(UniqueCarrier) %>% summarise(across(c(Cancelled, Diverted), mean))
```

```
# A tibble: 15 x 3
  UniqueCarrier Cancelled Diverted
  <chr>          <dbl>   <dbl>
1 AA            0.0185  0.00185
2 AS            0      0.00274
3 B6            0.0259  0.00576
4 CO            0.00678  0.00263
5 DL            0.0159  0.00303
6 EV            0.0345  0.00318
7 F9            0.00716  0
8 FL            0.00982  0.00327
9 MQ            0.0290  0.00194
10 OO           0.0139  0.00349
11 UA           0.0164  0.00241
```

12	US	0.0113	0.00147
13	WN	0.0155	0.00229
14	XE	0.0155	0.00345
15	YV	0.0127	0

Também podemos aplicar várias funções a uma mesma coluna:

```
flights %>% group_by(UniqueCarrier) %>%
  summarise(across(c(Cancelled, Diverted), list(media = mean, variancia = var)))
```

```
# A tibble: 15 x 5
  UniqueCarrier Cancelled_media Cancelled_variancia Diverted_media
  <chr>          <dbl>          <dbl>          <dbl>
1 AA            0.0185          0.0182          0.00185
2 AS            0              0              0.00274
3 B6            0.0259          0.0253          0.00576
4 CO            0.00678         0.00674         0.00263
5 DL            0.0159          0.0157          0.00303
6 EV            0.0345          0.0333          0.00318
7 F9            0.00716         0.00712         0
8 FL            0.00982         0.00973         0.00327
9 MQ            0.0290          0.0282          0.00194
10 OO           0.0139          0.0138          0.00349
11 UA           0.0164          0.0161          0.00241
12 US           0.0113          0.0111          0.00147
13 WN           0.0155          0.0153          0.00229
14 XE           0.0155          0.0153          0.00345
15 YV           0.0127          0.0127          0
# i 1 more variable: Diverted_variancia <dbl>
```

Além disso, podemos usar funções anônimas (com ~ e .x) para passar argumentos extras:

```
flights %>% group_by(UniqueCarrier) %>%
  summarise(across(matches("Delay"),
    list(min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE))))
```

```
# A tibble: 15 x 5
  UniqueCarrier ArrDelay_min ArrDelay_max DepDelay_min DepDelay_max
  <chr>          <int>          <int>          <int>          <int>
1 AA           -39           978           -15           970
2 AS           -43           183           -15           172
```

3	B6	-44	335	-14	310
4	CO	-55	957	-18	981
5	DL	-32	701	-17	730
6	EV	-40	469	-18	479
7	F9	-24	277	-15	275
8	FL	-30	500	-14	507
9	MQ	-38	918	-23	931
10	OO	-57	380	-33	360
11	UA	-47	861	-11	869
12	US	-42	433	-17	425
13	WN	-44	499	-10	548
14	XE	-70	634	-19	628
15	YV	-32	72	-11	54

Por fim, podemos realizar agrupamentos múltiplos:

```
flights %>% group_by(UniqueCarrier, DayOfWeek) %>%
  summarise(across(matches("Delay"),
    list(min = ~min(.x, na.rm = TRUE), max = ~max(.x, na.rm = TRUE))),
    .groups = "drop")
```

A tibble: 105 x 6

	UniqueCarrier	DayOfWeek	ArrDelay_min	ArrDelay_max	DepDelay_min	DepDelay_max
	<chr>	<int>	<int>	<int>	<int>	<int>
1	AA	1	-30	978	-14	970
2	AA	2	-30	265	-12	286
3	AA	3	-33	179	-15	168
4	AA	4	-38	663	-15	653
5	AA	5	-28	255	-13	234
6	AA	6	-39	685	-13	677
7	AA	7	-34	507	-15	525
8	AS	1	-30	183	-12	172
9	AS	2	-34	92	-12	68
10	AS	3	-29	123	-12	138

i 95 more rows

O pacote ggplot2

Nota: Esta seção foi adaptada de curso-r.com.

O `ggplot2` é um pacote do R voltado para a criação de gráficos estatísticos. Ele é baseado na Gramática dos Gráficos (Grammar of Graphics) de Leland Wilkinson. Segundo essa gramática, um gráfico estatístico é um mapeamento dos dados por meio de atributos estéticos (cores, formas, tamanho) de formas geométricas (pontos, linhas, barras).

```
library(ggplot2)
```

Construindo gráficos

Vamos discutir os aspectos básicos para a construção de gráficos com o `ggplot2`, utilizando o conjunto de dados `mtcars`. Para visualizar as primeiras linhas:

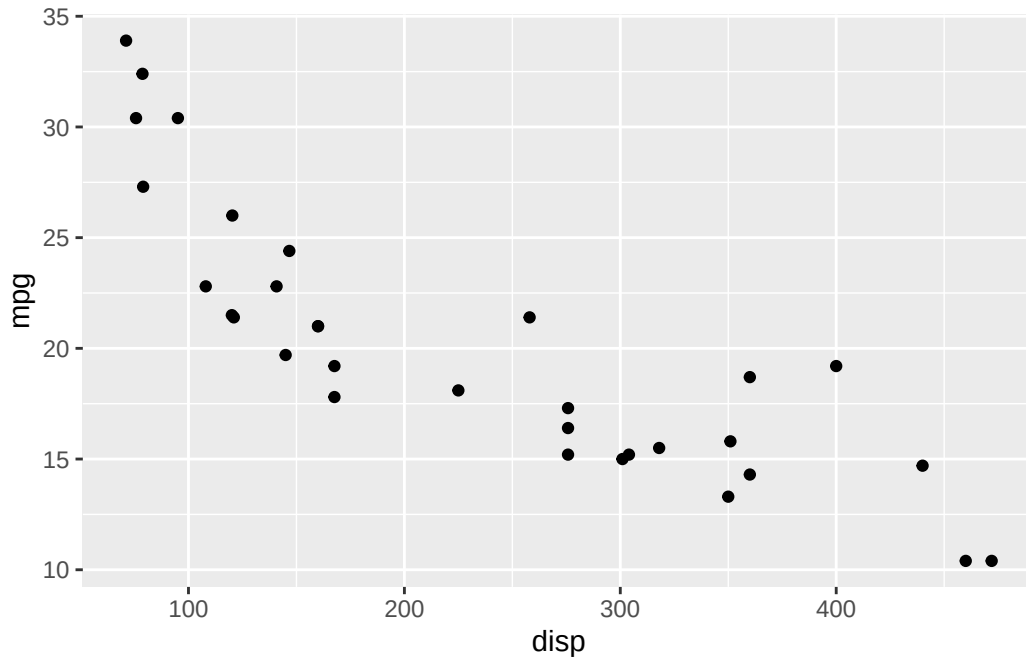
```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

As camadas de um gráfico

Os gráficos no `ggplot2` são construídos camada por camada. A primeira camada é criada com a função `ggplot`. Um exemplo de gráfico simples:

```
ggplot(data = mtcars) +  
  geom_point(aes(x = disp, y = mpg))
```

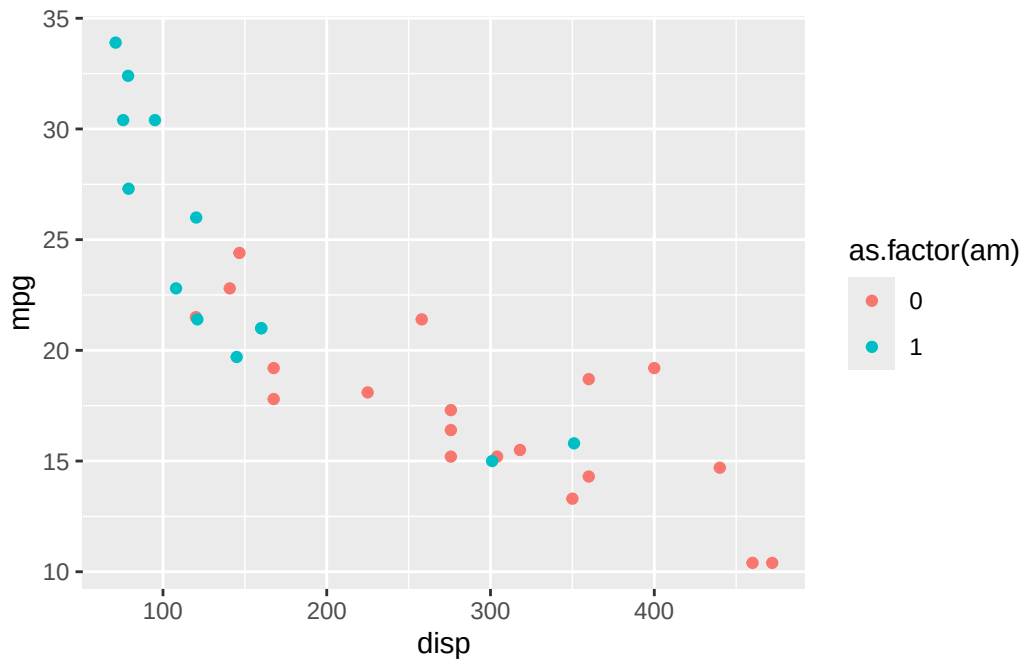


A função `aes` define o mapeamento entre as variáveis e os aspectos visuais. Neste caso, estamos criando um gráfico de dispersão (com `geom_point`) entre `disp` (cilindradas) e `mpg` (milhas por galão).

Aesthetics

Podemos mapear variáveis a diferentes aspectos estéticos, como cor e tamanho. Por exemplo, para mapear a variável `am` (transmissão) para a cor dos pontos:

```
ggplot(data = mtcars) +  
  geom_point(aes(x = disp, y = mpg, colour = as.factor(am)))
```



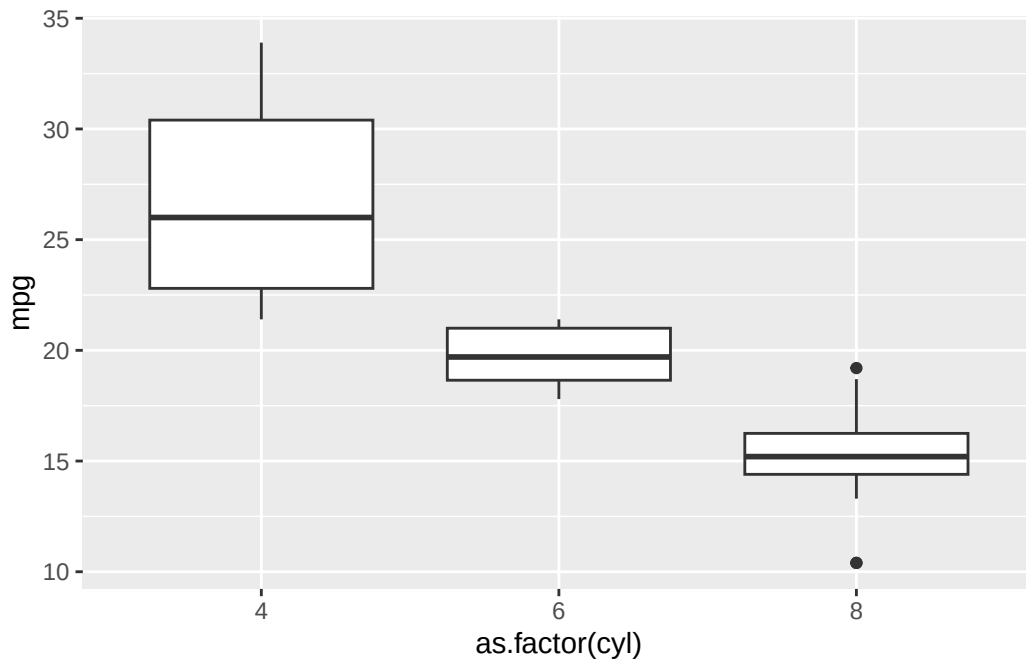
Geoms

Os geoms definem as formas geométricas usadas para a visualização dos dados. Além de `geom_point`, podemos usar:

- `geom_line` para linhas
- `geom_boxplot` para boxplots
- `geom_histogram` para histogramas

Exemplo de um boxplot:

```
ggplot(mtcars) +  
  geom_boxplot(aes(x = as.factor(cyl), y = mpg))
```

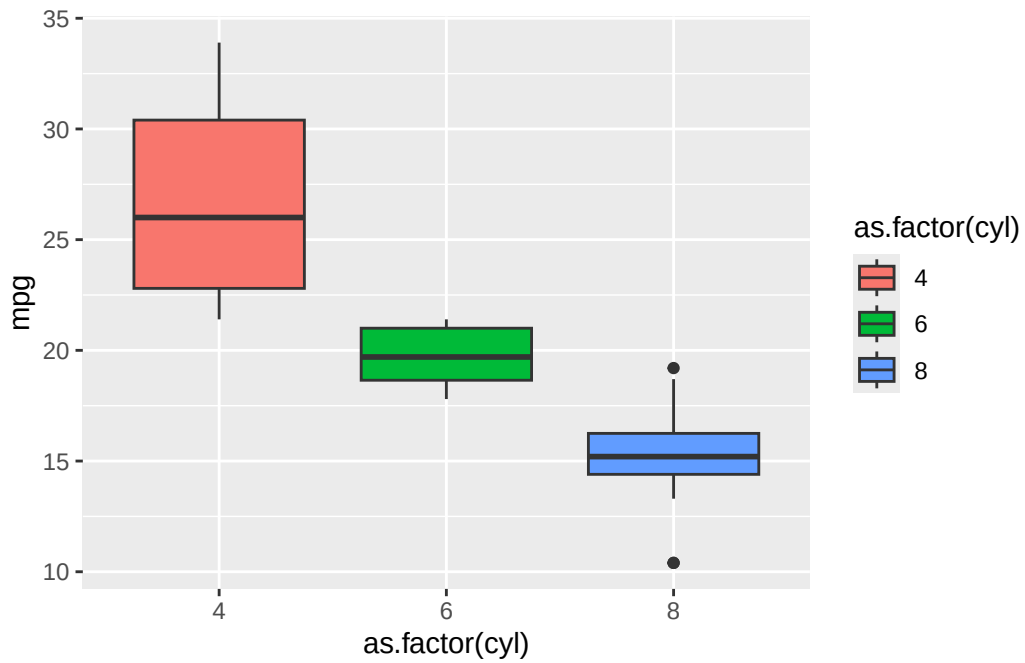


Personalizando os gráficos

Cores

Para mudar as cores do gráfico, podemos usar o argumento `colour` ou `fill`:

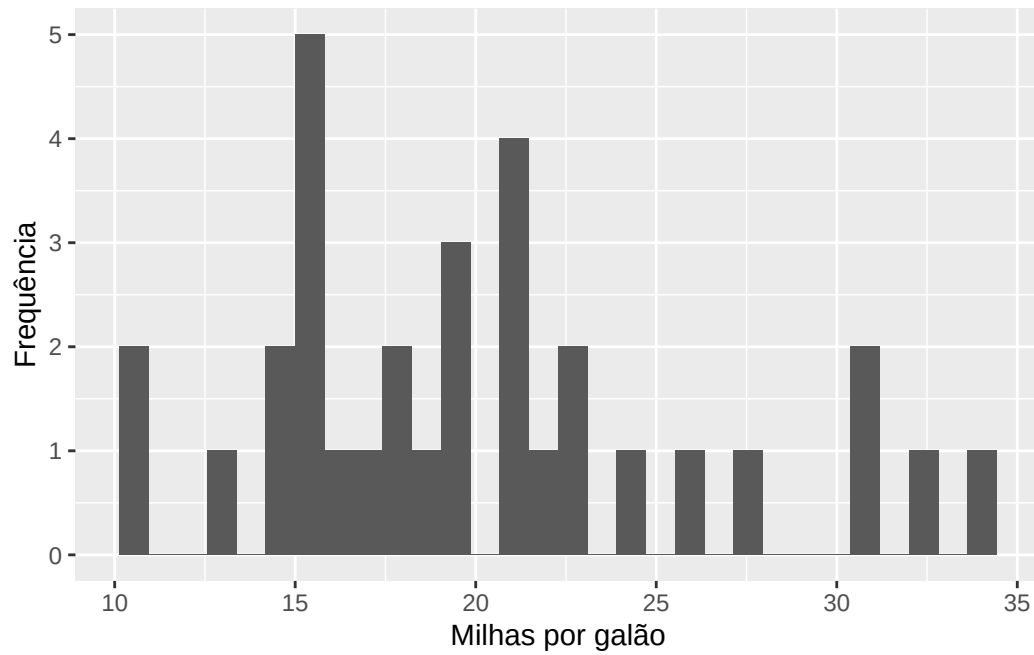
```
ggplot(mtcars) +  
  geom_boxplot(aes(x = as.factor(cyl), y = mpg, fill = as.factor(cyl)))
```



Eixos

Para alterar os rótulos dos eixos:

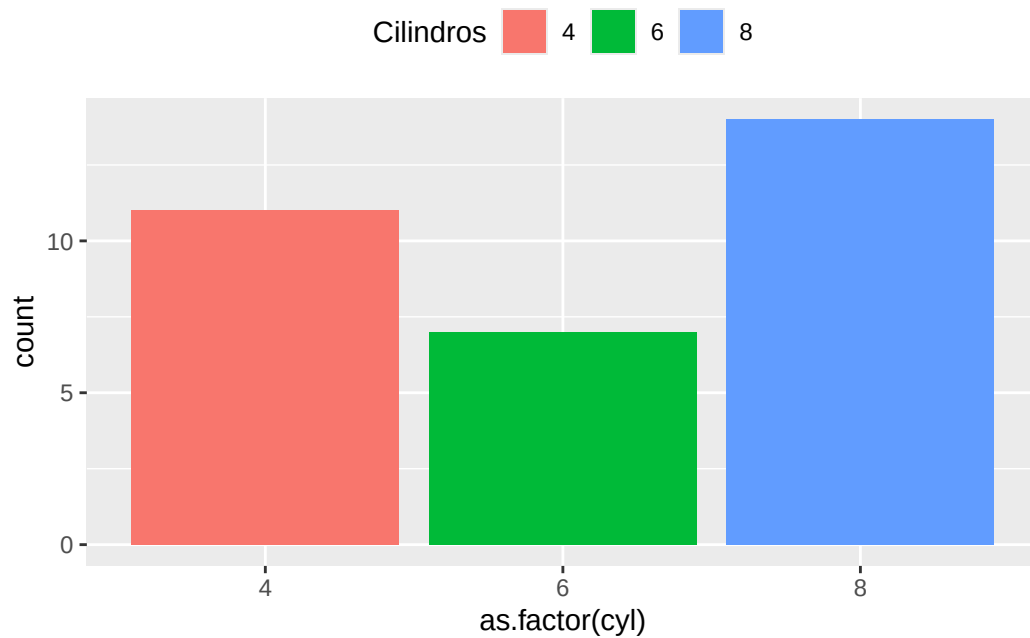
```
ggplot(mtcars) +  
  geom_histogram(aes(x = mpg)) +  
  xlab("Milhas por galão") +  
  ylab("Frequência")
```



Legendas

Podemos personalizar ou remover legendas facilmente:

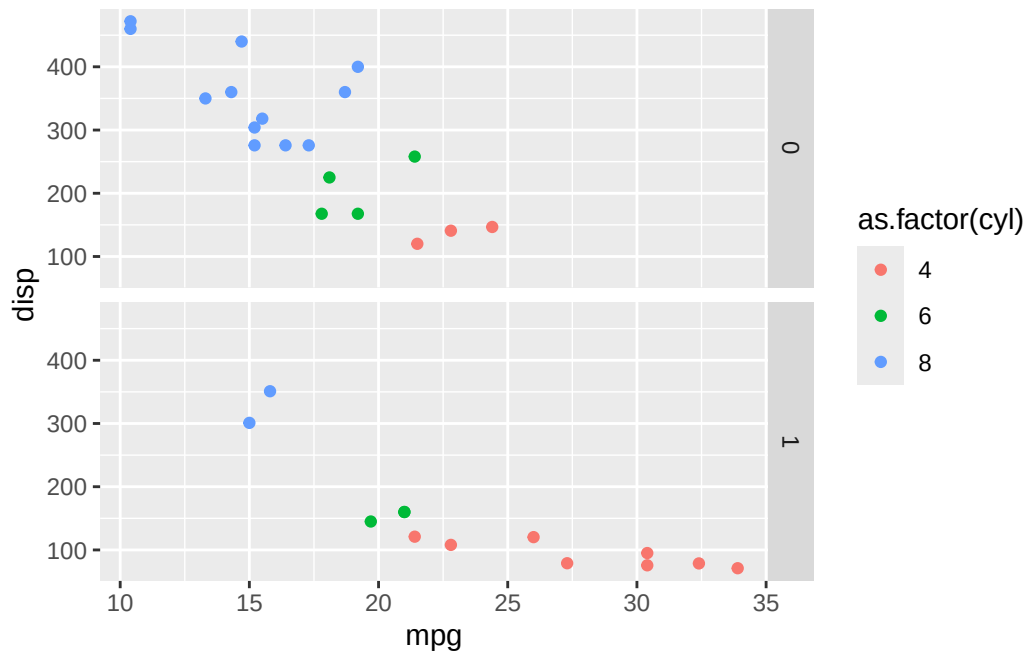
```
ggplot(mtcars) +  
  geom_bar(aes(x = as.factor(cyl), fill = as.factor(cyl))) +  
  labs(fill = "Cilindros") +  
  theme(legend.position = "top")
```



Facets

O `facet_grid` permite dividir o gráfico em subgráficos com base em uma variável:

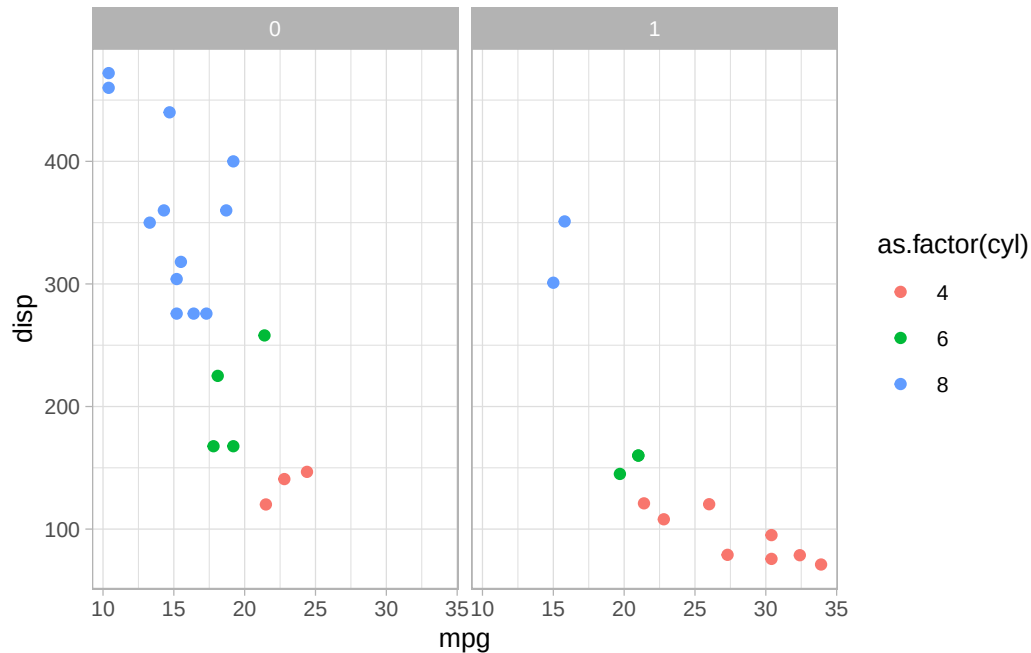
```
ggplot(mtcars) +  
  geom_point(aes(x = mpg, y = disp, colour = as.factor(cyl))) +  
  facet_grid(am ~ .)
```



Temas

Podemos mudar o tema de um gráfico com a função `theme_set`:

```
theme_set(theme_light(base_size = 10))
ggplot(mtcars) +
  geom_point(aes(x = mpg, y = disp, colour = as.factor(cyl))) +
  facet_grid(. ~ am)
```

O pacote tidyr

O pacote `tidyr` facilita a transformação e organização de dados no R. Para converter os dados de formato “wide” (largo) para “long” (longo), podemos utilizar a função `pivot_longer`, que substitui a antiga função `gather`. Considere os dados a seguir:

```
dados_originais <- data.frame(
  paciente = c("João", "Marcos", "Antonio"),
  pressao.antes = c(67, 80, 64),
  pressao.durante = c(54, 70, 60),
  pressao.depois = c(56, 90, 50)
)
dados_originais
```

	paciente	pressao.antes	pressao.durante	pressao.depois
1	João	67	54	56
2	Marcos	80	70	90
3	Antonio	64	60	50

Para reorganizar esses dados em um formato “long”, usando `pivot_longer`:

```
library(tidyr)
dados_novo_formato <- dados_originais %>%
  pivot_longer(cols = pressao.antes:pressao.depois,
               names_to = "instante",
               values_to = "pressao.arterial")
dados_novo_formato
```

```
# A tibble: 9 x 3
  paciente instante      pressao.arterial
  <chr>      <chr>          <dbl>
1 João      pressao.antes          67
2 João      pressao.durante        54
3 João      pressao.depois         56
4 Marcos    pressao.antes          80
5 Marcos    pressao.durante        70
6 Marcos    pressao.depois         90
7 Antonio   pressao.antes          64
8 Antonio   pressao.durante        60
9 Antonio   pressao.depois         50
```

A função `pivot_wider` é usada para converter dados de formato “long” (longo) para “wide” (largo), espalhando uma variável por várias colunas. Vamos começar com um conjunto de dados em formato “long” e então reorganizá-los para o formato “wide”.

Considere o conjunto de dados `dados_novo_formato` que organizamos anteriormente:

```
dados_novo_formato <- data.frame(
  paciente = c("João", "Marcos", "Antonio", "João", "Marcos", "Antonio", "João", "Marcos", "Antonio"),
  instante = c("pressao.antes", "pressao.antes", "pressao.antes", "pressao.durante", "pressao.durante", "pressao.durante", "pressao.depois", "pressao.depois", "pressao.depois"),
  pressao.arterial = c(67, 80, 64, 54, 70, 60, 56, 90, 50)
)
dados_novo_formato
```

```
  paciente      instante pressao.arterial
1   João  pressao.antes          67
2  Marcos  pressao.antes          80
3 Antonio  pressao.antes          64
4   João  pressao.durante          54
5  Marcos  pressao.durante          70
6 Antonio  pressao.durante          60
7   João  pressao.depois          56
```

8	Marcos	pressao.depois	90
9	Antonio	pressao.depois	50

Agora, vamos usar `pivot_wider` para transformar esses dados de volta ao formato “wide”:

```
dados_wide <- dados_novo_formato %>%
  pivot_wider(names_from = instante, values_from = pressao.arterial)
dados_wide
```

```
# A tibble: 3 x 4
  paciente pressao.antes pressao.durante pressao.depois
  <chr>      <dbl>      <dbl>      <dbl>
1 João         67         54         56
2 Marcos        80         70         90
3 Antonio       64         60         50
```

Temos os seguintes argumentos:

- `names_from` especifica a coluna cujos valores serão usados para criar novos nomes de colunas (neste caso, a variável `instante`).
- `values_from` especifica a coluna cujos valores serão preenchidos nas novas colunas criadas (neste caso, a variável `pressao.arterial`).

Exemplo: Manipulação e Visualização com `pivot_longer` e `ggplot2`

O pacote `tidyr` é particularmente útil em conjunto com o `ggplot2`. Vamos continuar com o exemplo dos dados de pressão arterial. Suponha que você queira visualizar as variações da pressão arterial dos pacientes em diferentes instantes.

Primeiro, vamos reorganizar os dados usando `pivot_longer` para colocar os dados no formato “long”, que é mais adequado para gráficos com o `ggplot2`:

```
library(tidyr)
library(ggplot2)

# Dados originais no formato "wide"
dados_originais <- data.frame(
  paciente = c("João", "Marcos", "Antonio"),
  pressao.antes = c(67, 80, 64),
  pressao.durante = c(54, 70, 60),
  pressao.depois = c(56, 90, 50)
```

```
)
dados_originais
```

```

  paciente pressao.antes pressao.durante pressao.depois
1      João           67             54             56
2      Marcos          80             70             90
3      Antonio         64             60             50

```

Para usá-los no ggplot, vamos reorganizá-los no formato “long”:

```

dados_long <- dados_originais %>%
  pivot_longer(cols = pressao.antes:pressao.depois,
               names_to = "instante",
               values_to = "pressao.arterial")
dados_long

```

```

# A tibble: 9 x 3
  paciente instante      pressao.arterial
  <chr>      <chr>          <dbl>
1 João     pressao.antes          67
2 João     pressao.durante        54
3 João     pressao.depois         56
4 Marcos   pressao.antes          80
5 Marcos   pressao.durante        70
6 Marcos   pressao.depois         90
7 Antonio  pressao.antes          64
8 Antonio  pressao.durante        60
9 Antonio  pressao.depois         50

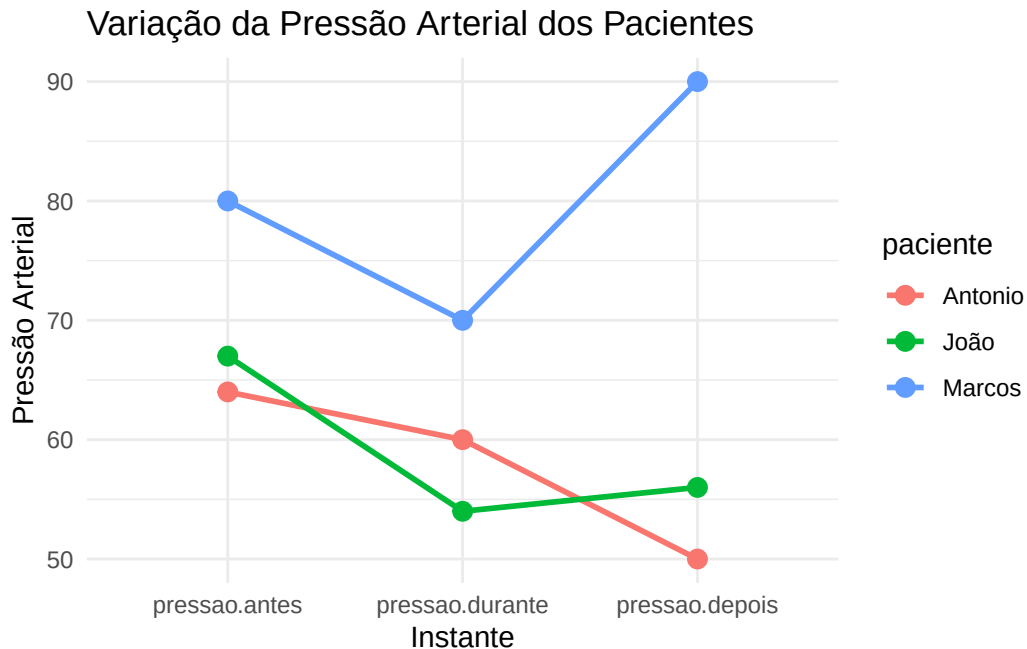
```

Agora, vamos criar um gráfico de linhas que mostra as mudanças na pressão arterial ao longo do tempo para cada paciente. Note que estamos reordenando o eixo x para levar em conta o caráter temporal do problema.

```

ggplot(dados_long, aes(x = factor(instante, levels = c("pressao.antes", "pressao.durante", "pressao.depois"),
                             y = pressao.arterial, group = paciente, color = paciente)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  labs(title = "Variação da Pressão Arterial dos Pacientes",
       x = "Instante",
       y = "Pressão Arterial") +
  theme_minimal()

```



Leitura de Arquivos com readr

O **readr** é um pacote eficiente para leitura de arquivos de dados no R. Vamos explorar como utilizar o **readr** para ler e manipular arquivos CSV. Um arquivo CSV (Comma-Separated Values) é um formato comum para armazenar dados tabulares.

Para começar, carregue o pacote **readr**:

```
library(readr)
```

Exemplo: Leitura de um CSV com Dados Públicos

Vamos usar um banco de dados público disponível na internet. Um exemplo muito comum é o conjunto de dados de passageiros do Titanic, disponível em formato CSV. Para carregar diretamente da internet:

```
url <- "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
dados_titanic <- read_csv(url)
```

Aqui, estamos carregando os dados diretamente de um link. Após a leitura, podemos visualizar os primeiros registros:

```
head(dados_titanic)
```

```
# A tibble: 6 x 12
  PassengerId Survived Pclass Name      Sex      Age SibSp Parch Ticket  Fare Cabin
    <dbl>      <dbl>  <dbl> <chr>    <chr>  <dbl>  <dbl>  <dbl> <chr>  <dbl> <chr>
1         1         0      3 Braund~ male    22      1      0 A/5 2~  7.25 <NA>
2         2         1      1 Cuming~ fema~   38      1      0 PC 17~ 71.3  C85
3         3         1      3 Heikki~ fema~   26      0      0 STON/~  7.92 <NA>
4         4         1      1 Futrel~ fema~   35      1      0 113803 53.1  C123
5         5         0      3 Allen,~ male    35      0      0 373450  8.05 <NA>
6         6         0      3 Moran,~ male    NA      0      0 330877  8.46 <NA>
# i 1 more variable: Embarked <chr>
```

Manipulando os Dados Lidos

O `read_csv` detecta automaticamente os tipos de dados de cada coluna. No exemplo dos dados do Titanic, podemos realizar algumas análises rápidas, como visualizar a distribuição de sobreviventes:

```
table(dados_titanic$Survived)
```

```
0    1
549 342
```

Salvando o Conjunto de Dados

Se você quiser salvar o conjunto de dados em seu computador após manipulações, basta usar o `write_csv`:

```
write_csv(dados_titanic, "titanic_local.csv")
```

Exercícios

Exercício 1. Execute as seguintes operações no R e interprete os resultados:

- $5 + 9$
- $\frac{0}{0}$
- 10^5

Utilize a função `log` para calcular o logaritmo natural de 10 e depois o logaritmo na base 10 de 1000.

Descubra a função que calcula a raiz quadrada de um número utilizando o símbolo de interrogação (?), e aplique-a ao número 144.

Exercício 2. Armazene o resultado de 15×7 em uma variável chamada `resultado`. Use `resultado` para calcular o valor de $2 \times \text{resultado}$.

Liste todas as variáveis no ambiente do R usando a função `ls()`. Exclua a variável `resultado` e verifique novamente as variáveis presentes.

Exercício 3. Crie um vetor `v` contendo os números 4, 8, 15, 16, 23, 42. Use subsetting para selecionar:

- O segundo e o quarto elemento.
- Todos os elementos exceto o terceiro.
- Os elementos que são maiores que 10.

Multiplique cada elemento do vetor `v` por 2 e armazene o resultado em um novo vetor `v2`.

Exercício 4. Verifique se 8 é maior que 5 e se 8 é igual a 10.

Crie dois vetores lógicos `a = c(TRUE, FALSE, TRUE)` e `b = c(FALSE, TRUE, FALSE)`. Aplique os operadores `&`, `|` e `xor()` nesses vetores.

Exercício 5. Escreva uma função `soma_quadrados` que receba dois números como argumentos e retorne a soma de seus quadrados. Teste sua função com os números 3 e 4.

Modifique a função `soma_quadrados` para que o segundo argumento tenha um valor padrão de 2. Teste a função chamando-a com apenas um argumento.

Exercício 6. Escreva um laço `for` que calcule a soma dos números de 1 a 100.

Crie um laço `while` que multiplique os números de 1 a 6 e retorne o resultado.

Exercício 7. Crie duas variáveis com o seu primeiro e último nome. Use a função `paste()` para juntar as duas variáveis em uma frase que diga “Meu nome completo é [nome completo]”.

Altere o separador na função `paste()` para um traço – e junte novamente as variáveis.

Exercício 8. Carregue o pacote `dplyr` e use o dataset `mtcars`. Selecione apenas as colunas `mpg`, `cyl`, e `hp`.

Filtre as observações onde `mpg` é maior que 20 e `hp` é menor que 150.

Ordene o dataset filtrado pela coluna `mpg` em ordem decrescente.

Exercício 9. Usando o dataset `mtcars`, crie um gráfico de dispersão (`geom_point`) que mostre a relação entre `hp` (horsepower) e `mpg` (milhas por galão).

No gráfico anterior, mapeie a variável `cyl` para a cor dos pontos.

Exercício 10. Utilize a base pública de dados de voos da [nycflights13](#) para responder às perguntas abaixo.

1. Carregue o pacote `nycflights13` e explore o dataset `flights`. Visualize as primeiras 6 linhas da base.
2. Filtre todos os voos que decolaram no mês de junho e aterrissaram com atraso maior que 1 hora.
3. Agrupe os dados por companhia aérea (`carrier`) e calcule o atraso médio de chegada (`arr_delay`) para cada companhia.
4. Crie um gráfico de barras que mostre o atraso médio de chegada por companhia aérea.

1 Por que Simular?

Boa parte das perguntas interessantes em probabilidade não tem resposta em fórmula fechada. Qual a chance de um álbum de figurinhas ficar completo com 800 pacotes? Quanto vale uma integral que nenhuma técnica de cálculo resolve? Qual a distribuição de probabilidades associada a uma estatística complicada?

Existe uma saída que funciona em todos esses casos e exige pouco mais do que saber programar: **repetir o experimento muitas vezes no computador e olhar para o que acontece**. É disso que trata este curso.

Este capítulo passa por quatro exemplos que mostram a ideia em funcionamento: estimar π contando pontos, simular um dado viciado com uma função pronta, refazer esse mesmo sorteio usando apenas números uniformes e, por fim, ver a aleatoriedade agindo dentro de um algoritmo de aprendizado de máquina. Nenhum deles será justificado aqui — o resto do livro é a justificativa. O que interessa agora é chegar até a pergunta levantada no terceiro exemplo, que organiza todos os capítulos seguintes.

1.1 Estimando π com pontos aleatórios

Considere um quadrado de lado 2 centrado na origem e o círculo de raio 1 inscrito nele. O quadrado tem área 4 e o círculo tem área π . Se sortearmos pontos uniformemente dentro do quadrado, a proporção que cai dentro do círculo deve ficar perto de $\pi/4$.

Ou seja: para estimar π , basta sortear pontos e contar.

2 R

```
set.seed(42)

# Definindo o número de pontos a serem gerados
n_pontos <- 5000

# Gerando pontos aleatórios (x, y) no quadrado [-1, 1] x [-1, 1]
x <- runif(n_pontos, -1, 1)
y <- runif(n_pontos, -1, 1)

# Verificando quais pontos caem dentro do círculo de raio 1
dentro_circulo <- x^2 + y^2 <= 1

# A proporção de pontos dentro do círculo estima pi/4
pi_estimado <- 4 * mean(dentro_circulo)

# Exibindo o valor estimado de Pi
# (sprintf("%.4f", x) formata x com exatamente 4 casas decimais)
cat("Valor estimado de :", sprintf("%.4f", pi_estimado), "\n")
```

Valor estimado de : 3.1144

```
cat("Valor verdadeiro :", sprintf("%.4f", pi), "\n")
```

Valor verdadeiro : 3.1416

```
# Visualizando a distribuição dos pontos
library(ggplot2)

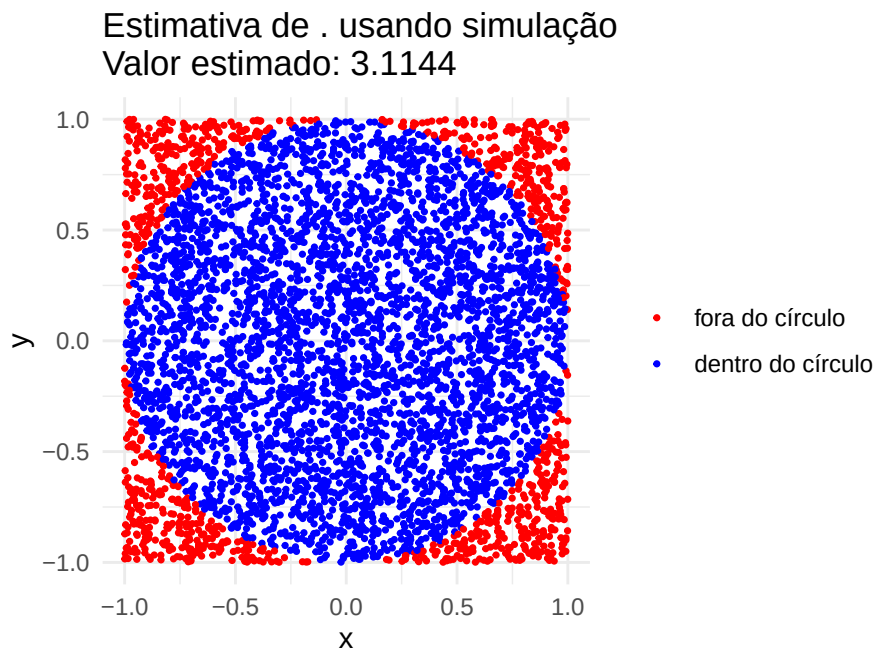
dados <- data.frame(x = x, y = y, dentro_circulo = dentro_circulo)

ggplot(dados, aes(x = x, y = y, color = dentro_circulo)) +
  geom_point(size = 0.6) +
  scale_color_manual(values = c("FALSE" = "red", "TRUE" = "blue"),
```

```

      labels = c("FALSE" = "fora do círculo",
                  "TRUE" = "dentro do círculo")) +
ggtitle(paste0("Estimativa de  usando simulação\nValor estimado: ",
               sprintf("%.4f", pi_estimado))) +
theme_minimal() +
coord_equal() +
labs(x = "x", y = "y", color = NULL)

```



3 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Definindo o número de pontos a serem gerados
n_pontos = 5000

# Gerando pontos aleatórios (x, y) no quadrado [-1, 1] x [-1, 1]
x = np.random.uniform(-1, 1, n_pontos)
y = np.random.uniform(-1, 1, n_pontos)

# Verificando quais pontos caem dentro do círculo de raio 1
dentro_circulo = x**2 + y**2 <= 1

# A proporção de pontos dentro do círculo estima pi/4
pi_estimado = 4 * np.mean(dentro_circulo)

# Exibindo o valor estimado de Pi
print(f"Valor estimado de : {pi_estimado:.4f}")
```

Valor estimado de : 3.1336

```
print(f"Valor verdadeiro : {np.pi:.4f}")
```

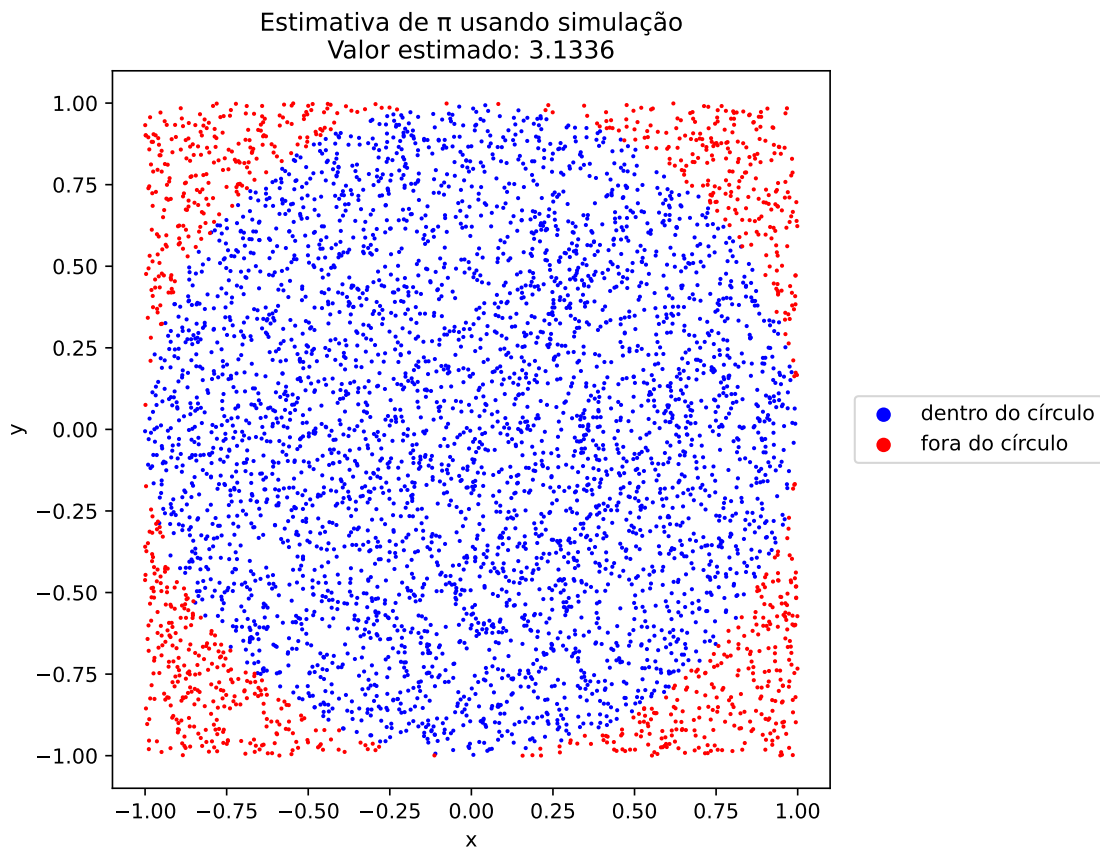
Valor verdadeiro : 3.1416

```
# Visualizando a distribuição dos pontos
# (usamos as mesmas cores da versão em R: azul dentro, vermelho fora)
plt.figure(figsize=(8, 6))
plt.scatter(x[dentro_circulo], y[dentro_circulo],
            color='blue', s=1, label='dentro do círculo')
plt.scatter(x[~dentro_circulo], y[~dentro_circulo],
```

```

        color='red', s=1, label='fora do círculo')
plt.title(f'Estimativa de  $\pi$  usando simulação\nValor estimado: {pi_estimado:.4f}')
plt.xlabel('x')
plt.ylabel('y')
# legenda fora do gráfico, para não cobrir os pontos
plt.legend(loc='center left', bbox_to_anchor=(1.02, 0.5), markerscale=6)
plt.gca().set_aspect('equal')
plt.tight_layout()
plt.show()

```



i Por que isso funciona?

Por enquanto o argumento é só geométrico: proporção de pontos $\approx$ proporção de áreas. A justificativa formal — por que essa proporção converge para $\pi/4$, e com que precisão para cada n — é o assunto do capítulo sobre o Método de Monte Carlo.

3.1 Simulando um dado viciado

Agora um experimento em que a resposta não vem da geometria, e sim de um sorteio com probabilidades desiguais. Imagine um dado em que as faces não são igualmente prováveis: as faces 5 e 6 saem com probabilidade 0,25 cada, enquanto a face 1 sai com probabilidade 0,05.

Tanto o R quanto o Python têm uma função pronta que sorteia de uma lista de valores com probabilidades dadas. Vamos usá-la para lançar esse dado 10 000 vezes e comparar as frequências obtidas com as probabilidades verdadeiras.

4 R

```
set.seed(123)

# Definindo as faces do dado e as probabilidades
faces <- 1:6
probabilidades <- c(0.05, 0.1, 0.15, 0.2, 0.25, 0.25) # Probabilidades associadas às faces

# Verificando que a soma das probabilidades é 1
cat("Soma das probabilidades:", sum(probabilidades), "\n")
```

Soma das probabilidades: 1

```
# Simulando 10000 lançamentos de um dado viciado
n_lancamentos <- 10000
resultados <- sample(faces, size = n_lancamentos, replace = TRUE, prob = probabilidades)

# Calculando a frequência relativa de cada face
frequencias <- sapply(faces, function(face) mean(resultados == face))

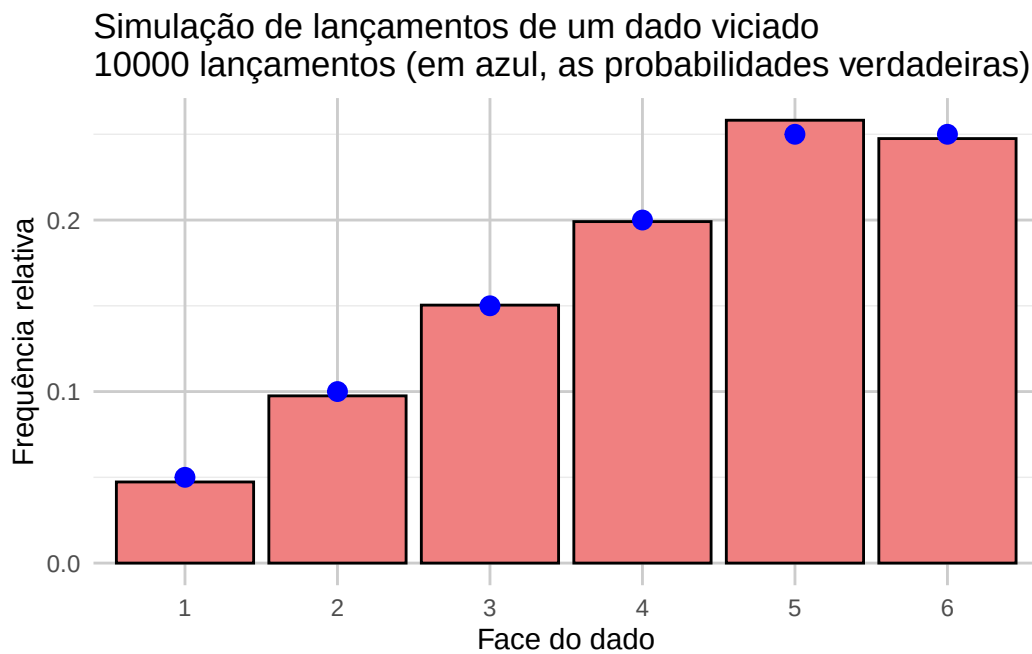
# Comparando o que saiu na simulação com o que era esperado
comparacao <- data.frame(
  face = faces,
  probabilidade = probabilidades,
  frequencia = frequencias
)
print(comparacao, row.names = FALSE)
```

face	probabilidade	frequencia
1	0.05	0.0473
2	0.10	0.0975
3	0.15	0.1504
4	0.20	0.1991
5	0.25	0.2582
6	0.25	0.2475

```
# Visualizando os resultados em um gráfico de barras
library(ggplot2)

dados <- data.frame(faces = as.factor(faces), frequencias = frequencias)

ggplot(dados, aes(x = faces, y = frequencias)) +
  geom_bar(stat = "identity", fill = "lightcoral", color = "black") +
  geom_point(aes(y = probabilidades), color = "blue", size = 3) +
  ggtitle(paste0("Simulação de lançamentos de um dado viciado\n",
                 n_lancamentos, " lançamentos (em azul, as probabilidades verdadeiras)")) +
  xlab("Face do dado") +
  ylab("Frequência relativa") +
  theme_minimal() +
  theme(panel.grid.major = element_line(color = "grey80"))
```



5 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)

# Definindo as faces do dado e as probabilidades
faces = [1, 2, 3, 4, 5, 6]
probabilidades = [0.05, 0.1, 0.15, 0.2, 0.25, 0.25] # Probabilidades associadas às faces do

# Verificando que a soma das probabilidades é 1
print(f"Soma das probabilidades: {sum(probabilidades)}")
```

Soma das probabilidades: 1.0

```
# Simulando 10000 lançamentos de um dado viciado
# (o sorteio é com reposição, que é o padrão de np.random.choice)
n_lancamentos = 10000
resultados = np.random.choice(faces, size=n_lancamentos, p=probabilidades)

# Calculando a frequência relativa de cada face
frequencias = [np.mean(resultados == face) for face in faces]

# Comparando o que saiu na simulação com o que era esperado
print(" face | probabilidade | frequência")
```

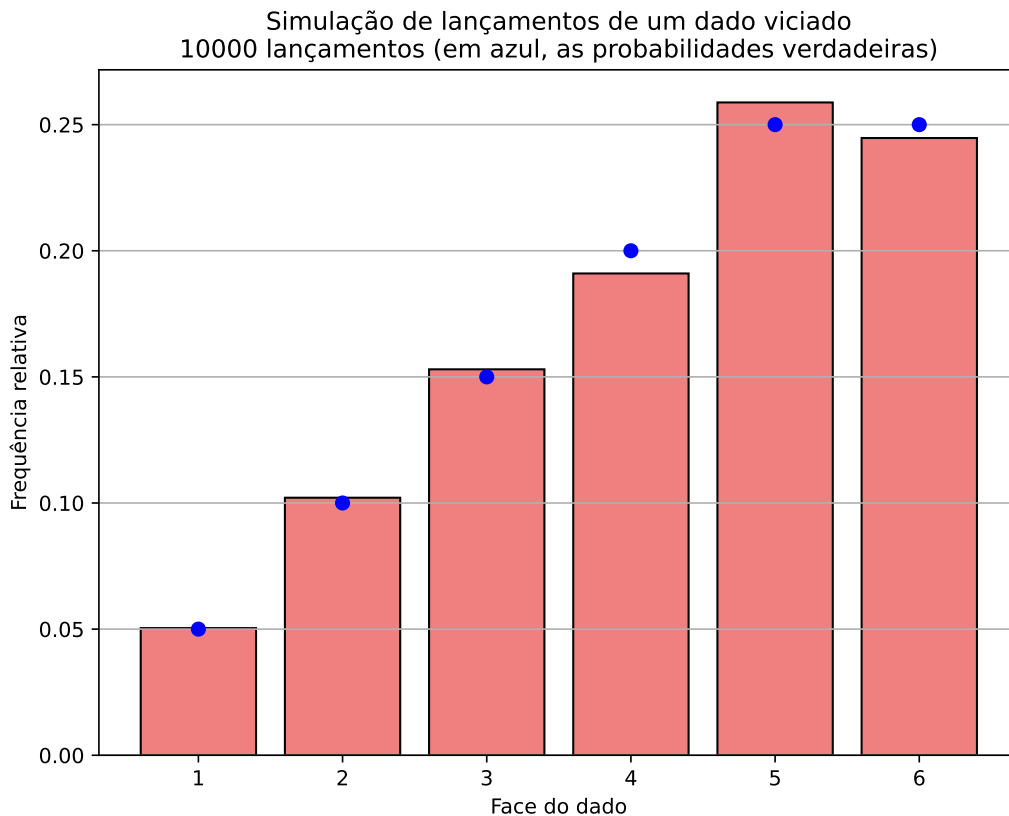
face | probabilidade | frequência

```
for face, p, freq in zip(faces, probabilidades, frequencias):
    print(f"{face:5d} | {p:13.4f} | {freq:10.4f}")
```

1	0.0500	0.0504
2	0.1000	0.1021

3	0.1500	0.1530
4	0.2000	0.1910
5	0.2500	0.2588
6	0.2500	0.2447

```
# Visualizando os resultados em um gráfico de barras
plt.figure(figsize=(8, 6))
plt.bar(faces, frequencias, color='lightcoral', edgecolor='black')
plt.scatter(faces, probabilidades, color='blue', s=40, zorder=3)
plt.title(f'Simulação de lançamentos de um dado viciado\n'
          f'{n_lancamentos} lançamentos (em azul, as probabilidades verdadeiras)')
plt.xlabel('Face do dado')
plt.ylabel('Frequência relativa')
plt.grid(True, axis='y')
plt.show()
```



5.1 E se tivéssemos apenas uma uniforme?

A função `sample` do R e a `np.random.choice` do Python resolveram o problema, mas elas são caixas-pretas: alguém já implementou o sorteio para nós. Vale perguntar o que existe dentro dessas caixas.

Acontece que basta saber gerar um número uniforme em $[0, 1)$. A ideia é dividir o intervalo $[0, 1]$ em pedaços com comprimentos iguais às probabilidades das faces:

$$\underbrace{[0; 0,05)}_{\text{face 1}} \quad \underbrace{[0,05; 0,15)}_{\text{face 2}} \quad \underbrace{[0,15; 0,30)}_{\text{face 3}} \quad \dots$$

Sorteamos u uniforme e devolvemos a face cujo pedaço contém u . Como u é uniforme, a chance de cair em cada pedaço é exatamente o comprimento dele — isto é, a probabilidade daquela face.

Escrito passo a passo, o procedimento é o seguinte.

Pseudo-algoritmo: sorteio pelos intervalos

Entradas: os valores possíveis $x_1, \dots, x_k$ e suas probabilidades $p_1, \dots, p_k$.

1. Gere $u \sim \text{Unif}(0, 1)$.
2. Faça `limite_inferior` $\leftarrow 0$.
3. Para $i = 1, 2, \dots, k$:
 - a. Faça `limite_superior` $\leftarrow$ `limite_inferior` $+$ p_i ;
 - b. Se `limite_inferior` $\leq u <$ `limite_superior`, devolva x_i e pare;
 - c. Caso contrário, faça `limite_inferior` $\leftarrow$ `limite_superior`.

O código abaixo é a tradução direta desse pseudo-algoritmo — cada passo vira uma linha.

6 R

```
set.seed(456)

# Definindo as faces do dado e as probabilidades associadas (não uniformes)
faces <- 1:6
probabilidades <- c(0.05, 0.1, 0.15, 0.2, 0.25, 0.25) # Probabilidades associadas às faces do dado

# Função para gerar uma amostra baseada em intervalos de probabilidades
gerar_amostra_por_intervalos <- function(probabilidades, faces) {
  u <- runif(1) # Gerando um número aleatório uniforme
  limite_inferior <- 0 # Limite inferior do intervalo

  # Percorrendo as probabilidades e verificando em qual intervalo o número caiu
  for (i in seq_along(probabilidades)) {
    limite_superior <- limite_inferior + probabilidades[i] # Definindo o limite superior do intervalo
    if (limite_inferior <= u && u < limite_superior) {
      return(faces[i]) # Retorna a face correspondente ao intervalo
    }
    limite_inferior <- limite_superior # Atualiza o limite inferior para o próximo intervalo
  }
}

# Simulando lançamentos do dado viciado utilizando a verificação dos intervalos
n_lancamentos <- 10000
resultados <- replicate(n_lancamentos, gerar_amostra_por_intervalos(probabilidades, faces))

# Calculando a frequência relativa de cada face
frequencias <- sapply(faces, function(face) mean(resultados == face))

# Comparando com as probabilidades verdadeiras
comparacao <- data.frame(
  face = faces,
  probabilidade = probabilidades,
  frequencia = frequencias
)
```

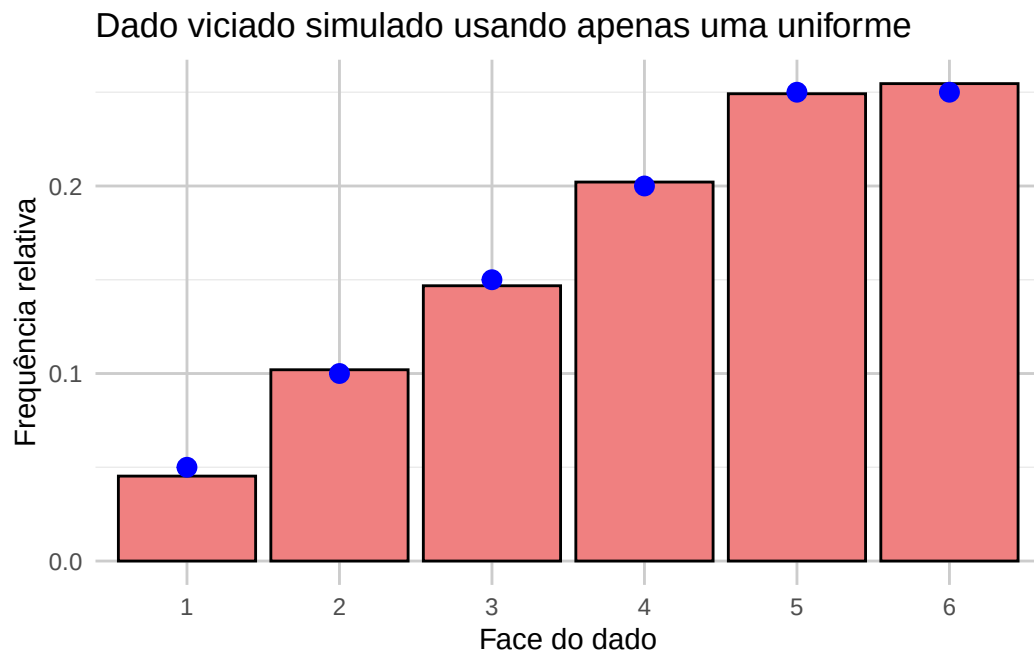
```
print(comparacao, row.names = FALSE)
```

face	probabilidade	frequencia
1	0.05	0.0453
2	0.10	0.1020
3	0.15	0.1468
4	0.20	0.2021
5	0.25	0.2492
6	0.25	0.2546

```
# Visualizando os resultados
library(ggplot2)

dados <- data.frame(faces = as.factor(faces), frequencias = frequencias)

ggplot(dados, aes(x = faces, y = frequencias)) +
  geom_bar(stat = "identity", fill = "lightcoral", color = "black") +
  geom_point(aes(y = probabilidades), color = "blue", size = 3) +
  ggtitle("Dado viciado simulado usando apenas uma uniforme") +
  xlab("Face do dado") +
  ylab("Frequência relativa") +
  theme_minimal() +
  theme(panel.grid.major = element_line(color = "grey80"))
```



7 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(456)

# Definindo as faces do dado e as probabilidades associadas (não uniformes)
faces = [1, 2, 3, 4, 5, 6]
probabilidades = [0.05, 0.1, 0.15, 0.2, 0.25, 0.25] # Probabilidades associadas às faces do

# Gerando um número aleatório e verificando em qual intervalo ele cai
def gerar_amostra_por_intervalos(probabilidades, faces):
    u = np.random.uniform(0, 1) # Gerando um número aleatório uniforme
    limite_inferior = 0 # Limite inferior do intervalo

    # Percorrendo as probabilidades e verificando em qual intervalo o número cai
    for i, p in enumerate(probabilidades):
        limite_superior = limite_inferior + p # Definindo o limite superior do intervalo
        if limite_inferior <= u < limite_superior:
            return faces[i] # Retorna a face correspondente ao intervalo
        limite_inferior = limite_superior # Atualiza o limite inferior para o próximo interv

# Simulando lançamentos do dado viciado utilizando a verificação dos intervalos
n_lancamentos = 10000
resultados = np.array([gerar_amostra_por_intervalos(probabilidades, faces)
                        for _ in range(n_lancamentos)])

# Calculando a frequência relativa de cada face
frequencias = [np.mean(resultados == face) for face in faces]

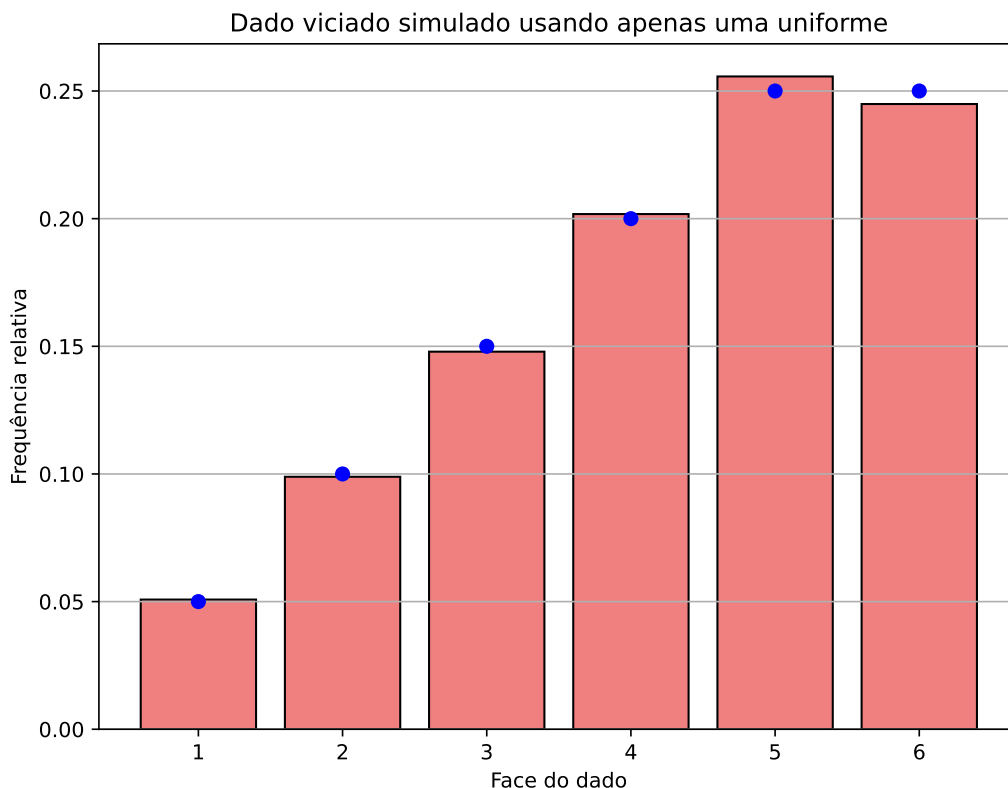
# Comparando com as probabilidades verdadeiras
print(" face | probabilidade | frequência")
```

```
face | probabilidade | frequência
```

```
for face, p, freq in zip(faces, probabilidades, frequencias):
    print(f"{face:5d} | {p:13.4f} | {freq:10.4f}")
```

1	0.0500	0.0508
2	0.1000	0.0989
3	0.1500	0.1479
4	0.2000	0.2018
5	0.2500	0.2557
6	0.2500	0.2449

```
# Visualizando os resultados
plt.figure(figsize=(8, 6))
plt.bar(faces, frequencias, color='lightcoral', edgecolor='black')
plt.scatter(faces, probabilidades, color='blue', s=40, zorder=3)
plt.title('Dado viciado simulado usando apenas uma uniforme')
plt.xlabel('Face do dado')
plt.ylabel('Frequência relativa')
plt.grid(True, axis='y')
plt.show()
```

i Você acabou de implementar um método

O procedimento acima tem nome: é a **técnica da inversão**, que estudaremos em detalhe nos capítulos sobre variáveis discretas e contínuas. Lá ele aparece escrito em termos da função de distribuição acumulada, mas a ideia é exatamente esta — cortar o intervalo $[0, 1]$ em pedaços e ver onde u caiu.

! A pergunta que organiza o curso

Se sabemos simular uma $\text{Unif}(0, 1)$, sabemos simular qualquer distribuição discreta. Isso vale de forma mais geral? Dá para gerar uma exponencial, uma normal, uma distribuição sem fórmula fechada para a acumulada — tudo a partir de números uniformes? A resposta é **sim**, e os próximos capítulos são as diferentes maneiras de fazer isso.

7.1 Aleatoriedade também aparece em aprendizado de máquina

Simulação não serve apenas para responder perguntas de probabilidade: vários algoritmos de aprendizado de máquina usam números aleatórios internamente. O Random Forest é um bom exemplo — ele constrói muitas árvores de decisão, cada uma treinada sobre um subconjunto sorteado das observações e das variáveis.

Isso tem uma consequência prática que costuma pegar quem está começando: **rodar o mesmo código duas vezes pode dar respostas diferentes**. Para ver o tamanho do efeito, vamos repetir exatamente o mesmo procedimento — dividir o conjunto `iris` em treino e teste, ajustar um Random Forest e medir a acurácia — mudando apenas a semente do gerador de números aleatórios.

8 R

```
library(randomForest)
library(ggplot2)

data(iris)

# Mesmo procedimento, mudando apenas a semente
acuracia_para_semente <- function(semente) {
  set.seed(semente)

  # Divisão aleatória em treino (70%) e teste (30%)
  indice <- sample(nrow(iris), 0.7 * nrow(iris))
  treino <- iris[indice, ]
  teste <- iris[-indice, ]

  # Random Forest: as árvores também são construídas de forma aleatória
  modelo <- randomForest(Species ~ ., data = treino, ntree = 100)

  # Proporção de acertos no conjunto de teste
  mean(predict(modelo, teste) == teste$Species)
}

acuracias <- sapply(1:20, acuracia_para_semente)

cat("Menor acurácia :", sprintf("%.4f", min(acuracias)), "\n")
```

Menor acurácia : 0.8889

```
cat("Maior acurácia :", sprintf("%.4f", max(acuracias)), "\n")
```

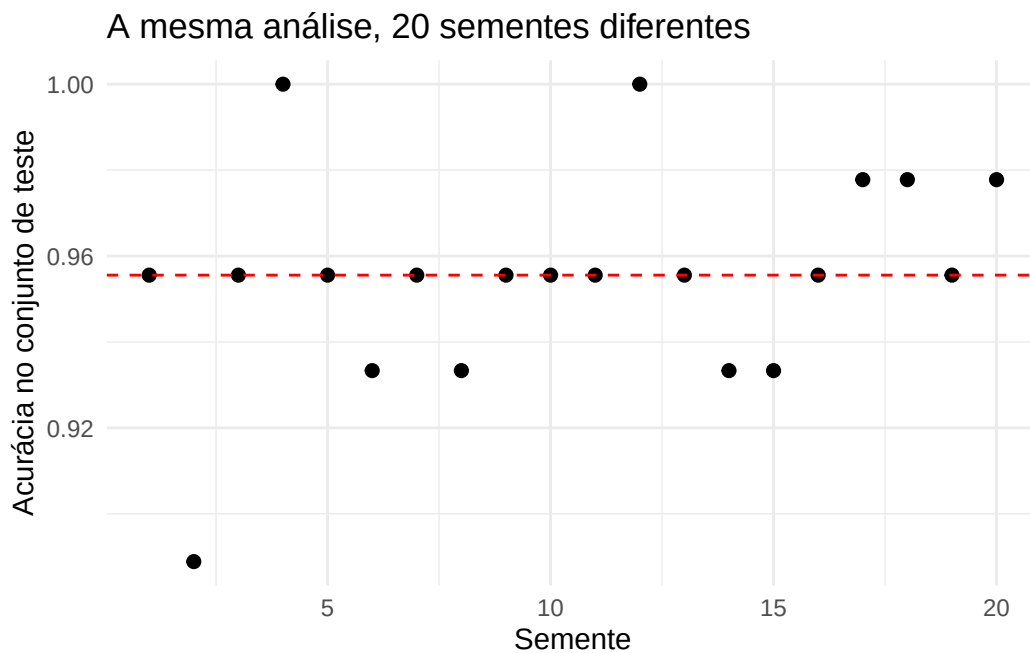
Maior acurácia : 1.0000

```
cat("Acurácia média :", sprintf("%.4f", mean(acuracias)), "\n")
```

Acurácia média : 0.9556

```
dados <- data.frame(semente = 1:20, acuracia = acuracias)

ggplot(dados, aes(x = semente, y = acuracia)) +
  geom_point(size = 2) +
  geom_hline(yintercept = mean(acuracias), color = "red", linetype = "dashed") +
  labs(title = "A mesma análise, 20 sementes diferentes",
       x = "Semente", y = "Acurácia no conjunto de teste") +
  theme_minimal()
```



9 Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

# Mesmo procedimento, mudando apenas a semente
def acuracia_para_semente(semente):
    # Divisão aleatória em treino (70%) e teste (30%)
    X_treino, X_teste, y_treino, y_teste = train_test_split(
        iris.data, iris.target, test_size=0.3, random_state=semente)

    # Random Forest: as árvores também são construídas de forma aleatória
    modelo = RandomForestClassifier(n_estimators=100, random_state=semente)
    modelo.fit(X_treino, y_treino)

    # Proporção de acertos no conjunto de teste
    return modelo.score(X_teste, y_teste)

sementes = np.arange(1, 21)
acuracias = np.array([acuracia_para_semente(s) for s in sementes])

print(f"Menor acurácia : {acuracias.min():.4f}")
```

Menor acurácia : 0.8889

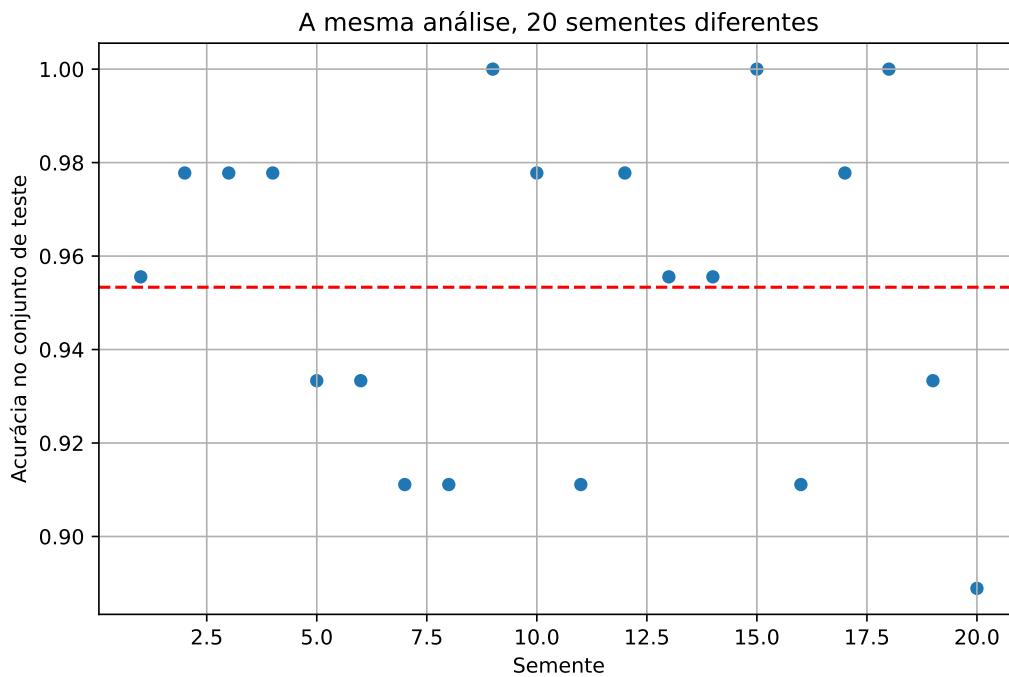
```
print(f"Maior acurácia : {acuracias.max():.4f}")
```

Maior acurácia : 1.0000

```
print(f"Acurácia média : {acuracias.mean():.4f}")
```

Acurácia média : 0.9533

```
plt.figure(figsize=(8, 5))
plt.scatter(sementes, acuracias, s=30)
plt.axhline(acuracias.mean(), color='red', linestyle='--')
plt.title('A mesma análise, 20 sementes diferentes')
plt.xlabel('Semente')
plt.ylabel('Acurácia no conjunto de teste')
plt.grid(True)
plt.show()
```



A acurácia varia mais de 10 pontos percentuais entre a pior e a melhor semente — e nada mudou no código além do número usado para inicializar o gerador. Reportar o resultado de uma única rodada, sem dizer qual semente foi usada, é reportar um número que ninguém consegue reproduzir.

 Sempre fixe a semente

`set.seed()` no R e `np.random.seed()` no Python fixam o ponto de partida do gerador, de modo que a mesma sequência de números “aleatórios” é produzida a cada execução. Todos os trechos de código deste livro fixam a semente por esse motivo: você deve conseguir rodar o código e obter exatamente os números impressos aqui. Isso não torna o resultado mais correto — apenas reproduzível. Quando o resultado depende demais da semente, como no exemplo acima, o certo é repetir a análise várias vezes e reportar a variabilidade.

9.1 O caminho daqui em diante

Os exemplos deste capítulo assumiram que sabemos gerar números uniformes em $[0, 1)$ e que a proporção observada se aproxima da probabilidade verdadeira. Os próximos capítulos tratam exatamente dessas duas lacunas:

- **Números pseudoaleatórios:** como um computador, que é determinístico, produz algo que se comporta como uma $\text{Unif}(0, 1)$.
- **Inversão, rejeição, transformações e misturas, Box-Muller:** as técnicas que transformam uniformes em qualquer outra distribuição.
- **Método de Monte Carlo:** por que a média de valores simulados aproxima uma esperança, e como quantificar o erro dessa aproximação com intervalos de confiança.
- **Redução de variância e amostragem por importância:** como obter a mesma precisão com menos simulações.

9.2 Exercícios

Todos os exercícios abaixo consistem em modificar algum dos códigos deste capítulo.

Exercício 1. No código que estima π , troque o número de pontos por $n = 100, 1\,000, 10\,000$ e $100\,000$.

- (a) Anote a estimativa obtida em cada caso e faça um gráfico da estimativa em função de n .
- (b) Quantos pontos foram necessários para acertar a primeira casa decimal de π ? E a segunda?

Exercício 2. Rode duas vezes seguidas o trecho que simula o dado viciado, primeiro apagando a linha `set.seed/np.random.seed` e depois com ela de volta. Explique o que muda em cada caso e por que fixar a semente importa quando você envia seu código para outra pessoa.

Exercício 3. Adapte a função `gerar_amostra_por_intervalos` para uma variável aleatória que assume os valores 10, 20, 30 com probabilidades 0,5, 0,2 e 0,3.

- (a) Gere uma amostra de tamanho 5 000 e compare as frequências obtidas com as probabilidades verdadeiras.
- (b) O que acontece se as probabilidades fornecidas não somarem 1? Teste e explique.

Exercício 4. O código da primeira seção estima a área do círculo, contando quantos pontos satisfazem $x^2 + y^2 \leq 1$. Troque essa condição por

$$|x|^3 + |y|^3 \leq 0,5$$

e estime a área dessa região (lembre-se de que a área do quadrado é 4). Refaça também o gráfico dos pontos, para ver o formato dela.

Diferentemente do círculo, essa região não tem área dada por uma fórmula em funções elementares — mas o código para estimá-la é exatamente o mesmo.

10 Números Pseudoaleatórios

No capítulo anterior, todos os exemplos começavam sorteando números uniformes em $[0, 1)$ — com `runif` no R e `np.random.uniform` no Python — e a partir deles construímos tudo o mais. Ficou pendente a pergunta de onde esses números uniformes vêm.

Números aleatórios têm muitas aplicações na computação, como em simulações, amostragem estatística, criptografia e jogos de azar. No entanto, os computadores, por serem sistemas determinísticos, não podem gerar números realmente aleatórios de forma autônoma. Em vez disso, utilizam algoritmos determinísticos que geram números que *parecem* aleatórios, e esses números são chamados de pseudoaleatórios.

Este capítulo trata de como um desses algoritmos funciona. Ele é a base de todo o resto do livro: os capítulos seguintes transformam uniformes em outras distribuições, e todos supõem que sabemos produzir uniformes.

i Definição: número pseudoaleatório

Um **número pseudoaleatório** é gerado a partir de uma fórmula matemática que, dada uma **semente** (um valor inicial), produz uma sequência de números com as propriedades desejadas de uma sequência aleatória. Essa sequência parece aleatória, mas é inteiramente determinada pela semente: se a mesma semente for usada, a sequência será exatamente a mesma.

É justamente por isso que `set.seed` e `np.random.seed` funcionam. Fixar a semente não torna a simulação “menos aleatória”; apenas permite que outra pessoa rode o mesmo código e obtenha os mesmos números.

10.1 O Gerador Linear Congruente (LCG)

O **Gerador Linear Congruente** (LCG) é um dos métodos mais antigos e simples para gerar números pseudoaleatórios (veja o [verbete na Wikipédia](#)). Ele parte de uma semente X_0 e produz os valores seguintes pela fórmula

$$X_{n+1} = (a \cdot X_n + c) \mod m,$$

onde:

- X_n é o número gerado na n -ésima iteração (e X_0 é a semente inicial),
- a é o multiplicador,
- c é o incremento,
- m é o módulo.

Como o resto da divisão por m está sempre entre 0 e $m - 1$, todo X_n é um inteiro nesse intervalo. Mas o que queremos são números em $[0, 1)$: para isso, basta dividir por m ,

$$U_n = \frac{X_n}{m}.$$

Pseudo-algoritmo: LCG

Entradas: os parâmetros a , c , m e a semente X_0 .

1. Faça $X \leftarrow X_0$.
2. Repita, a cada vez que um novo número for pedido:
 - a. Atualize $X \leftarrow (a \cdot X + c) \bmod m$;
 - b. Devolva $U = X/m$.

Note que o gerador guarda o valor de X entre uma chamada e a seguinte: cada número novo é calculado a partir do anterior. Toda a sequência fica, portanto, determinada por a , c , m e X_0 . Um conjunto mal escolhido desses parâmetros pode produzir uma sequência que se repete muito rapidamente, o que compromete a qualidade do gerador — voltaremos a esse ponto na seção [Escolhendo os parâmetros](#).

10.1.1 A função módulo

O único ingrediente novo da fórmula é a operação de resto.

Definição: função módulo

A **função módulo** (também conhecida como operação de resto) retorna o **resto da divisão** de um número por outro. Para inteiros y e $m > 0$,

$$r = y \bmod m,$$

onde y é o dividendo, m é o divisor e r é o resto da divisão de y por m . Vale sempre $r \in \{0, 1, \dots, m - 1\}$.

Por exemplo, a divisão de 17 por 5 dá quociente 3 e resto 2, então

$$17 \bmod 5 = 2.$$

No contexto do LCG, é a função módulo que garante que os números gerados fiquem dentro do intervalo $\{0, 1, \dots, m - 1\}$.

11 R

```
# Exemplo de uso da função módulo em R

# Definindo os valores
dividendo <- 17
divisor <- 5

# Calculando o resto da divisão (em R, o operador de módulo é %%)
resto <- dividendo %% divisor

# Exibindo o resultado
cat("O resto da divisão de", dividendo, "por", divisor, "é:", resto, "\n")
```

O resto da divisão de 17 por 5 é: 2

12 Python

```
# Exemplo de uso da função módulo em Python

# Definindo os valores
dividendo = 17
divisor = 5

# Calculando o resto da divisão (em Python, o operador de módulo é %)
resto = dividendo % divisor

# Exibindo o resultado
print(f"O resto da divisão de {dividendo} por {divisor} é: {resto}")
```

O resto da divisão de 17 por 5 é: 2

⚠ Atenção: operadores diferentes nas duas linguagens

O operador de módulo é %% no R e % no Python. Cuidado também com ^: no R ele é a exponenciação (2^{32} vale 2^{32}), mas em Python ^ é o **XOR bit a bit** — lá a exponenciação se escreve 2^{**32} . Escrever 2^{32} em Python não gera erro, apenas devolve silenciosamente o número errado (34).

12.1 Exemplo 1: implementando o LCG

Vamos implementar o pseudo-algoritmo acima e usá-lo para gerar 10 000 números. Como parâmetros, adotamos

- $m = 2^{32}$ (módulo com 32 bits),
- $a = 1103515245$ (multiplicador),
- $c = 12345$ (incremento),
- $X_0 = 5$ (semente inicial, que pode ser qualquer valor).

O multiplicador e o incremento são os do gerador sugerido no padrão da linguagem C (que usa $m = 2^{31}$; aqui tomamos $m = 2^{32}$). Eles não foram escolhidos ao acaso: satisfazem as condições matemáticas que garantem o período mais longo possível, como verificaremos na próxima seção.

Se o gerador funcionar bem, os 10 000 valores devem se espalhar de maneira aproximadamente uniforme em $[0, 1)$ — isto é, o histograma deve ter barras de alturas parecidas.

13 R

```
# Carregando os pacotes
library(ggplot2)
library(gmp) # inteiros de precisão arbitrária

# Classe para o Gerador Congruente Linear.
# Usamos setRefClass porque o gerador precisa *lembrar* o último valor de X
# entre uma chamada e a seguinte: cada objeto criado guarda a sua própria
# semente, que é atualizada a cada chamada de gerar().
#
# Atenção: a * semente chega a valores da ordem de 4.7e18, muito além dos
# 2^53 que um "numeric" do R representa exatamente. Por isso guardamos os
# parâmetros como inteiros grandes (bigz) do pacote gmp: sem isso, a conta
# modular seria feita com arredondamento e o gerador produziria outra sequência.
LinearCongruentialGenerator <- setRefClass(
  "LinearCongruentialGenerator",
  fields = list(a = "ANY", c = "ANY", m = "ANY", semente = "ANY"),
  methods = list(
    initialize = function(semente, a = 1103515245, c = 12345, m = as.bigz(2)^32) {
      .self$a <- as.bigz(a)
      .self$c <- as.bigz(c)
      .self$m <- as.bigz(m)
      .self$semente <- as.bigz(semente)
    },
    gerar = function() {
      # Atualizando a semente
      .self$semente <- (.self$a * .self$semente + .self$c) %% .self$m
      return(as.numeric(.self$semente) / as.numeric(.self$m)) # Normalizando para [0, 1)
    }
  )
)

# Inicializando o gerador com uma semente
lcg <- LinearCongruentialGenerator$new(semente = 5)
```

```
# Gerando 10000 números pseudoaleatórios
numeros_gerados <- sapply(1:10000, function(x) lcg$gerar())

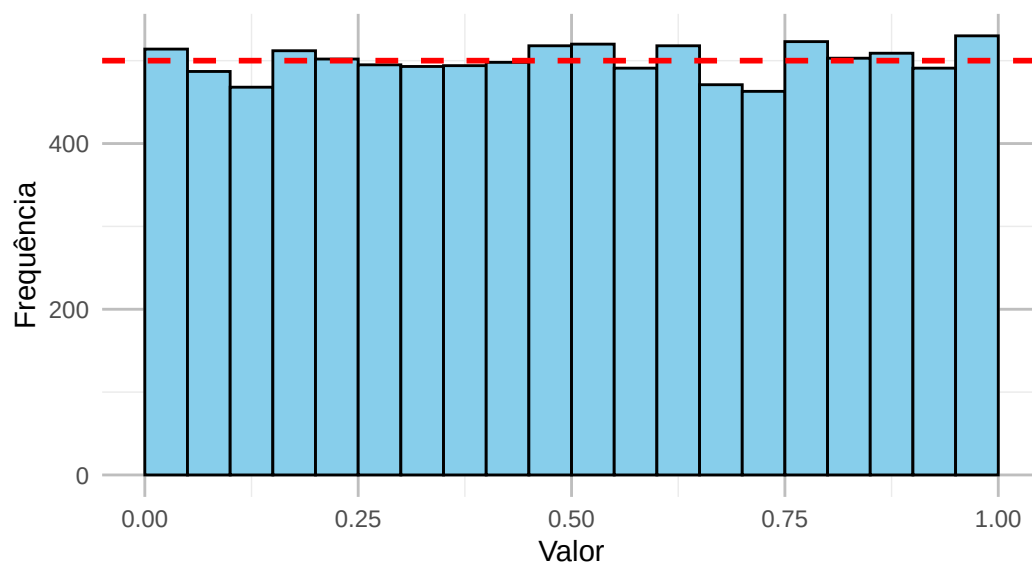
# Olhando os primeiros valores da sequência
print(round(head(numeros_gerados, 5), 6))
```

```
[1] 0.284664 0.629045 0.162000 0.486413 0.655533
```

```
# Convertendo para um data.frame
dados <- data.frame(numeros_gerados = numeros_gerados)

# Plotando o histograma dos números gerados
ggplot(dados, aes(x = numeros_gerados)) +
  # binwidth com boundary = 0 corta o intervalo em [0; 0,05), [0,05; 0,10), ...
  # Sem isso, as faixas das pontas sairiam mais estreitas e pareceriam ter
  # menos valores do que deveriam
  geom_histogram(binwidth = 0.05, boundary = 0,
                 fill = 'skyblue', color = 'black') +
  # Se os valores fossem mesmo Unif(0,1), cada uma das 20 faixas receberia, em
  # média, 10000/20 = 500 deles: é essa a frequência marcada em vermelho
  geom_hline(yintercept = length(numeros_gerados) / 20, color = 'red',
            linetype = 'dashed', linewidth = 1) +
  ggtitle('Histograma dos Números Pseudoaleatórios Gerados pelo LCG',
         subtitle = 'Linha vermelha: frequência esperada sob a Unif(0,1).') +
  xlab('Valor') +
  ylab('Frequência') +
  theme_minimal() +
  theme(panel.grid.major = element_line(color = "grey"))
```


Histograma dos Números Pseudoaleatórios Gerados pelo LCG
Linha vermelha: frequência esperada sob a Unif(0,1).



14 Python

```
import matplotlib.pyplot as plt

class LinearCongruentialGenerator:
    def __init__(self, semente, a=1103515245, c=12345, m=2**32):
        self.a = a
        self.c = c
        self.m = m
        self.semente = semente

    def gerar(self):
        # Atualizando a semente
        self.semente = (self.a * self.semente + self.c) % self.m
        return self.semente / self.m # Normalizando para [0, 1)

# Inicializando o gerador com uma semente
lcg = LinearCongruentialGenerator(semente=5)

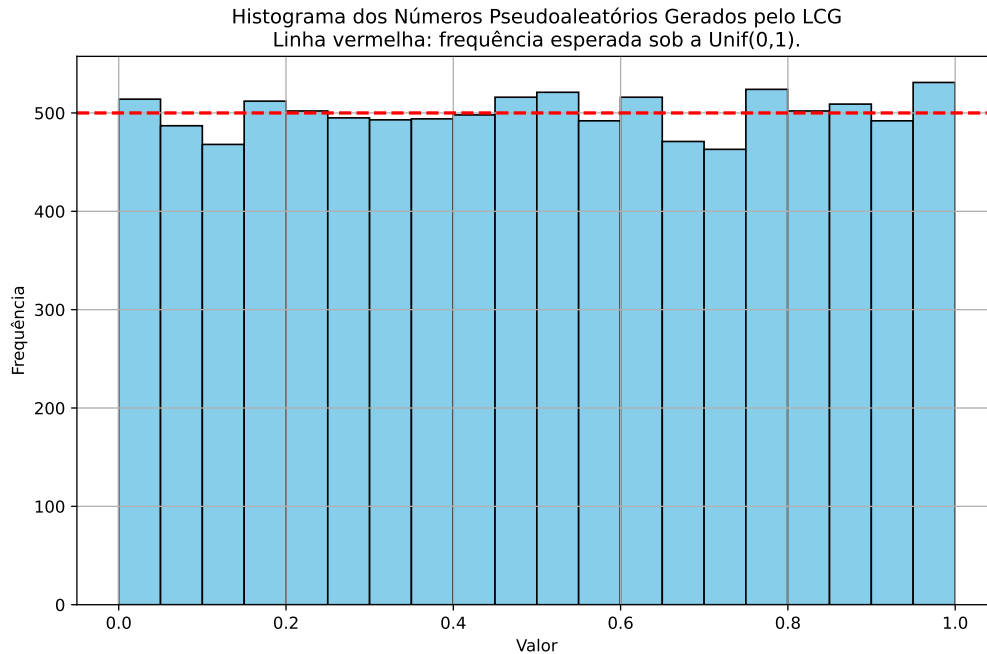
# Gerando 10000 números pseudoaleatórios
# (em Python os inteiros já têm precisão arbitrária, então a conta modular é exata)
numeros_gerados = [lcg.gerar() for _ in range(10000)]

# Olhando os primeiros valores da sequência
print([round(u, 6) for u in numeros_gerados[:5]])
```

[0.284664, 0.629045, 0.162, 0.486413, 0.655533]

```
# Plotando o histograma dos números gerados
plt.figure(figsize=(10, 6))
plt.hist(numeros_gerados, bins=20, color='skyblue', edgecolor='black')
# Se os valores fossem mesmo Unif(0,1), cada uma das 20 faixas receberia, em
# média, 10000/20 = 500 deles: é essa a frequência marcada em vermelho
plt.axhline(len(numeros_gerados) / 20, color='red', linestyle='--', linewidth=2)
plt.title('Histograma dos Números Pseudoaleatórios Gerados pelo LCG\n'
          'Linha vermelha: frequência esperada sob a Unif(0,1).')
```

```
plt.xlabel('Valor')
plt.ylabel('Frequência')
plt.grid(True)
plt.show()
```



As duas implementações produzem exatamente os mesmos cinco primeiros valores, e o histograma é compatível com uma $Unif(0,1)$. Vale notar que ser aproximadamente uniforme é uma condição **necessária**, mas longe de suficiente: a sequência 0,1; 0,2; ...; 1,0; 0,1; 0,2; ... também produz um histograma perfeitamente plano e não tem nada de aleatória. É por isso que a escolha dos parâmetros importa.

14.1 Escolhendo os parâmetros

A eficácia do LCG se baseia num bom equilíbrio entre a escolha dos parâmetros e as propriedades matemáticas que garantem uma sequência suficientemente “aleatória”. Antes de ver as condições, vale entender o que cada parâmetro faz.

1. **Módulo m :** define o intervalo no qual os números gerados estarão contidos, $\{0, 1, \dots, m-1\}$. Em muitos casos, m é escolhido como uma potência de 2 (por exemplo, $m = 2^{32}$ ou $m = 2^{64}$) porque cálculos modulares com potências de 2 são mais rápidos em hardware.

O valor de m também limita o quão longa a sequência pode ser antes de começar a se repetir.

2. **Multiplicador** a : é o parâmetro mais delicado. Se a não for bem escolhido, a sequência se repete rapidamente e passam a aparecer padrões visíveis entre valores consecutivos.
3. **Incremento** c : soma um valor fixo a cada passo e é um dos fatores que permite que todos os valores em $\{0, \dots, m-1\}$ sejam atingidos. Quando $c = 0$, o gerador é chamado de **multiplicativo**; nessa forma, o valor 0 é um ponto fixo (se $X_n = 0$, todos os seguintes também serão) e o período máximo possível é menor.
4. **Semente** X_0 : é o valor inicial da sequência. Mudar a semente resulta numa sequência diferente, mas com o mesmo período e as mesmas propriedades determinadas pelos outros parâmetros. É a semente que garante que a simulação possa ser **reproduzida**: dois programas com a mesma semente e os mesmos parâmetros produzem exatamente a mesma sequência.

14.1.1 Período

Definição: período

O **período** de um gerador é a quantidade de números produzidos antes que a sequência comece a se repetir. Como um LCG só depende do valor atual de X , e X assume no máximo m valores distintos, o período nunca pode ser maior que m .

Um período curto é o pior defeito que um gerador pode ter: se ele se repete a cada 32 valores, uma simulação com 10 000 repetições está, na verdade, usando as mesmas 32 observações várias vezes. O resultado deixa de ter qualquer relação com o que se queria estimar.

A boa notícia é que existe uma caracterização exata de quando o período máximo é atingido.

Condições de Hull–Dobell

Suponha $c \neq 0$. O LCG com parâmetros a , c e m tem período máximo m , **qualquer que seja a semente** X_0 , se e somente se:

1. c e m são **coprimos**, isto é, o maior divisor comum entre eles é 1: $\text{mdc}(c, m) = 1$;
2. $a - 1$ é divisível por todos os fatores primos de m ;
3. se m é divisível por 4, então $a - 1$ também é divisível por 4.

A intuição por trás de cada condição é a seguinte. A primeira garante que, ao somar c repetidamente, todos os valores possíveis de X_n possam ser atingidos: se c e m tivessem um fator comum, as somas ficariam presas a um subconjunto dos restos, e a sequência pularia valores. A segunda e a terceira controlam o efeito da multiplicação por a : elas garantem que

a parte multiplicativa não introduza ciclos curtos dentro do intervalo. Quando $m = 2^k$, seu único fator primo é 2, e as duas condições juntas se resumem a exigir que $a - 1$ seja divisível por 4.

Vamos verificar essas condições para os parâmetros usados no Exemplo 1.

15 R

```
# Carregando o pacote com a função gcd (maior divisor comum)
library(gmp)

# Parâmetros do Exemplo 1
# (evitamos chamar as variáveis de a, c e m porque, em R, "c" já é o nome
# da função que concatena valores)
modulo <- 2^32
multiplicador <- 1103515245
incremento <- 12345

# Verificando as três condições de Hull-Dobell.
# Como m = 2^32, seu único fator primo é 2.
condicoes <- c(
  "c e m são coprimos"      = as.logical(gcd(incremento, modulo) == 1),
  "a - 1 é divisível por 2" = (multiplicador - 1) %% 2 == 0,
  "a - 1 é divisível por 4" = (multiplicador - 1) %% 4 == 0
)

for (nome in names(condicoes)) {
  cat(nome, ":", condicoes[[nome]], "\n")
}
```

```
c e m são coprimos : TRUE
a - 1 é divisível por 2 : TRUE
a - 1 é divisível por 4 : TRUE
```

16 Python

```
import math

# Parâmetros do Exemplo 1
modulo = 2**32
multiplicador = 1103515245
incremento = 12345

# Verificando as três condições de Hull-Dobell.
# Como m = 2^32, seu único fator primo é 2.
condicoes = {
    "c e m são coprimos": math.gcd(incremento, modulo) == 1,
    "a - 1 é divisível por 2": (multiplicador - 1) % 2 == 0,
    "a - 1 é divisível por 4": (multiplicador - 1) % 4 == 0,
}

for nome, vale in condicoes.items():
    print(nome, ":", vale)
```

```
c e m são coprimos : True
a - 1 é divisível por 2 : True
a - 1 é divisível por 4 : True
```

As três condições valem, de modo que o gerador do Exemplo 1 percorre todos os $2^{32} \approx 4,3$ bilhões de valores possíveis antes de se repetir. Isso é mais do que suficiente para as simulações deste livro, mas modesto para os padrões atuais: geradores modernos como o Mersenne Twister — o que está por trás de `runif` e de `np.random.uniform` — têm períodos astronomicamente maiores.

i O que R e Python usam de verdade

Nenhuma das duas linguagens usa um LCG. Tanto `runif` quanto `np.random.uniform` são baseados no **Mersenne Twister**, publicado em 1997, cujo período é $2^{19937} - 1$ — um número com mais de seis mil algarismos. A diferença estrutural em relação ao LCG é o tamanho do estado interno: em vez de guardar um único X_n , ele guarda 624 inteiros

de 32 bits, e é isso que permite um período tão longo.

Você pode confirmar qual gerador está em uso com `RNGkind()` no R e com `np.random.get_state()[0]` no Python. O LCG continua valendo a pena estudar porque a ideia é exatamente a mesma — um estado que se atualiza por uma fórmula determinística, e uma semente que torna tudo reproduzível — e porque cabe em três linhas de código.

16.1 Exemplo 2: o efeito de parâmetros ruins

Para ver o que as condições de Hull–Dobell evitam, vamos comparar três geradores que diferem **apenas** em a e c . Todos usam $m = 64$ (pequeno, para que a repetição seja visível a olho nu) e semente $X_0 = 5$:

Gerador	a	c	Situação
bom	5	3	satisfaz as três condições
ruim 1	5	2	viola a condição 1: $\text{mdc}(2, 64) = 2$
ruim 2	6	2	viola também a condição 2: $a - 1 = 5$ é ímpar

Geramos 64 valores de cada um e contamos quantos são distintos. Como $m = 64$, o ideal é que os 64 valores sejam todos diferentes.

17 R

```
library(ggplot2)

# Função que devolve os n primeiros valores de um LCG.
# Aqui m é pequeno, então não há risco de perda de precisão e não precisamos do gmp.
# Chamar um argumento de "c" é seguro porque dentro da função não usamos a função
# c() do R; fora dela, é melhor evitar esse nome (como fizemos no trecho anterior).
gerar_lcg <- function(n, a, c, m, semente) {
  valores <- numeric(n)
  x <- semente
  for (i in 1:n) {
    x <- (a * x + c) %% m # mesma fórmula do pseudo-algoritmo
    valores[i] <- x
  }
  return(valores)
}

m <- 64
semente <- 5

seq_bom <- gerar_lcg(64, a = 5, c = 3, m = m, semente = semente)
seq_ruim1 <- gerar_lcg(64, a = 5, c = 2, m = m, semente = semente)
seq_ruim2 <- gerar_lcg(64, a = 6, c = 2, m = m, semente = semente)

# Quantos valores distintos cada gerador produziu em 64 iterações?
cat("a = 5, c = 3:", length(unique(seq_bom)), "valores distintos\n")
```

a = 5, c = 3: 64 valores distintos

```
cat("a = 5, c = 2:", length(unique(seq_ruim1)), "valores distintos\n")
```

a = 5, c = 2: 32 valores distintos

```
cat("a = 6, c = 2:", length(unique(seq_ruim2)), "valores distintos\n")
```

a = 6, c = 2: 6 valores distintos

```
# Os primeiros valores do pior gerador
cat("\nInício da sequência com a = 6, c = 2:", head(seq_ruim2, 10), "\n")
```

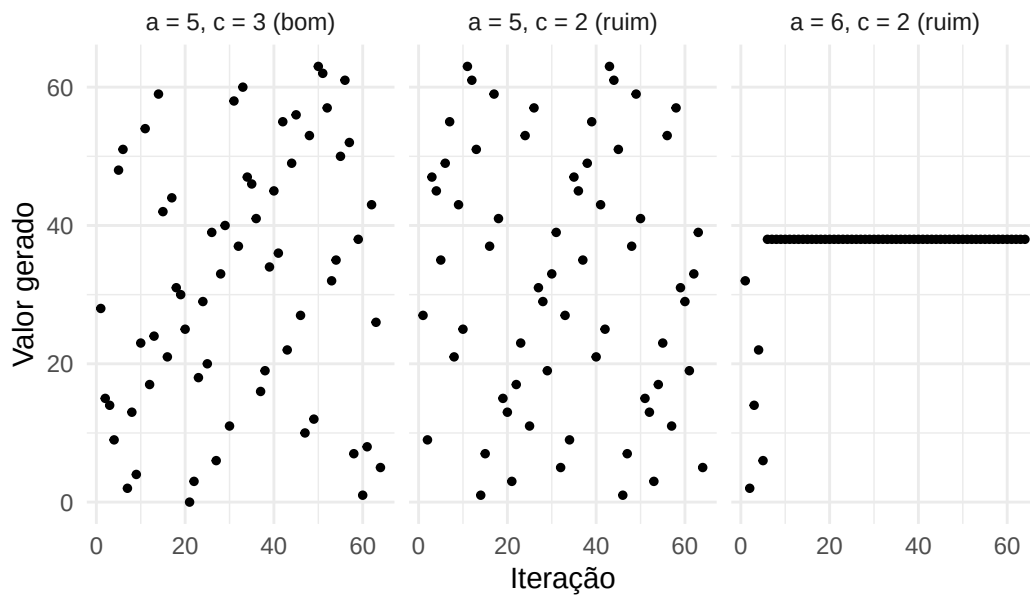
Início da sequência com a = 6, c = 2: 32 2 14 22 6 38 38 38 38 38

```
# Visualizando as três sequências lado a lado
rotulos <- c("a = 5, c = 3 (bom)", "a = 5, c = 2 (ruim)", "a = 6, c = 2 (ruim)")

dados <- data.frame(
  iteracao = rep(1:64, times = 3),
  valor = c(seq_bom, seq_ruim1, seq_ruim2),
  gerador = factor(rep(rotulos, each = 64), levels = rotulos)
)

ggplot(dados, aes(x = iteracao, y = valor)) +
  geom_point(size = 1) +
  facet_wrap(~ gerador) +
  labs(title = "64 iterações de um LCG com m = 64, mudando apenas a e c",
       x = "Iteração", y = "Valor gerado") +
  theme_minimal()
```

64 iterações de um LCG com $m = 64$, mudando apenas a e c



18 Python

```
import matplotlib.pyplot as plt

# Função que devolve os n primeiros valores de um LCG
def gerar_lcg(n, a, c, m, semente):
    valores = []
    x = semente
    for i in range(n):
        x = (a * x + c) % m    # mesma fórmula do pseudo-algoritmo
        valores.append(x)
    return valores

m = 64
semente = 5

seq_bom = gerar_lcg(64, a=5, c=3, m=m, semente=semente)
seq_ruim1 = gerar_lcg(64, a=5, c=2, m=m, semente=semente)
seq_ruim2 = gerar_lcg(64, a=6, c=2, m=m, semente=semente)

# Quantos valores distintos cada gerador produziu em 64 iterações?
print("a = 5, c = 3:", len(set(seq_bom)), "valores distintos")
```

a = 5, c = 3: 64 valores distintos

```
print("a = 5, c = 2:", len(set(seq_ruim1)), "valores distintos")
```

a = 5, c = 2: 32 valores distintos

```
print("a = 6, c = 2:", len(set(seq_ruim2)), "valores distintos")
```

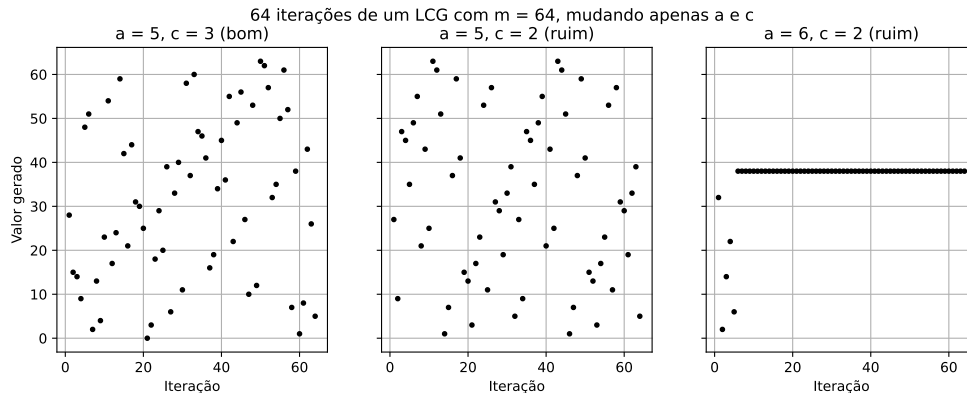
a = 6, c = 2: 6 valores distintos

```
# Os primeiros valores do pior gerador
print("\nInício da sequência com a = 6, c = 2:", seq_ruim2[:10])
```

Início da sequência com a = 6, c = 2: [32, 2, 14, 22, 6, 38, 38, 38, 38, 38]

```
# Visualizando as três sequências lado a lado
rotulos = ["a = 5, c = 3 (bom)", "a = 5, c = 2 (ruim)", "a = 6, c = 2 (ruim)"]
sequencias = [seq_bom, seq_ruim1, seq_ruim2]

fig, eixos = plt.subplots(1, 3, figsize=(12, 4), sharey=True)
for eixo, sequencia, rotulo in zip(eixos, sequencias, rotulos):
    eixo.plot(range(1, 65), sequencia, 'o', markersize=3, color='black')
    eixo.set_title(rotulo)
    eixo.set_xlabel('Iteração')
    eixo.grid(True)
eixos[0].set_ylabel('Valor gerado')
fig.suptitle('64 iterações de um LCG com m = 64, mudando apenas a e c')
plt.show()
```



O primeiro gerador percorre os 64 valores possíveis, como as condições de Hull–Dobell prometem. O segundo, que viola apenas a condição sobre c , atinge só metade deles: seu período é 32. O motivo é fácil de ver — como $5x + 2$ tem a mesma paridade de x , e 64 é par, todos os valores gerados herdam a paridade da semente. Partindo de $X_0 = 5$, o gerador só produz números ímpares. O terceiro é um desastre: depois de meia dúzia de iterações ele fica preso no valor 38 e passa a devolver sempre o mesmo número.

Repare que o defeito não apareceria num teste ingênuo: se olhássemos apenas a média dos valores gerados pelo segundo gerador, ela ficaria razoável. É o período, e não a média ou o histograma, que denuncia o problema.

18.1 Exemplo 3: a estrutura escondida nos pares

O exemplo anterior mostrou um defeito que o histograma não detecta, mas que o período detecta. Existe um defeito ainda mais sorrateiro: um gerador pode ter **período máximo** e histograma perfeitamente plano, e mesmo assim produzir uma sequência com estrutura evidente.

Para ver isso, comparamos dois geradores com o **mesmo** módulo $m = 2^{32}$ e a mesma semente, mudando apenas o multiplicador:

Gerador	a	c
bom	1103515245	12345
ruim	5	3

Os dois satisfazem as condições de Hull–Dobell — em ambos, $\text{mdc}(c, 2^{32}) = 1$ e $a - 1$ é divisível por 4 —, logo os dois têm período 2^{32} e percorrem todos os valores possíveis. Do ponto de vista dos testes que fizemos até agora, eles são indistinguíveis.

A ideia nova é olhar não para os valores isolados, mas para os **pares de valores consecutivos**: desenhamos no plano os pontos $(U_1, U_2), (U_2, U_3), (U_3, U_4), \dots$. Se a sequência fosse realmente aleatória, esses pontos deveriam preencher o quadrado $[0, 1) \times [0, 1)$ sem deixar buracos.

19 R

```
library(ggplot2)

# Reaproveitamos a classe definida no Exemplo 1. Os dois geradores usam o mesmo
# m = 2^32 (valor padrão da classe) e a mesma semente: só o multiplicador muda
lcg_bom <- LinearCongruentialGenerator$new(semente = 5, a = 1103515245, c = 12345)
lcg_ruim <- LinearCongruentialGenerator$new(semente = 5, a = 5, c = 3)

B <- 2000
u_bom <- sapply(1:(B + 1), function(i) lcg_bom$gerar())
u_ruim <- sapply(1:(B + 1), function(i) lcg_ruim$gerar())

# Antes do gráfico, os testes ingênuos aplicados ao gerador ruim
cat("Média dos valores do gerador ruim:", round(mean(u_ruim), 4), "\n")
```

Média dos valores do gerador ruim: 0.4948

```
cat("Contagens em 10 faixas de largura 0,1 (o ideal é 200 em cada):\n")
```

Contagens em 10 faixas de largura 0,1 (o ideal é 200 em cada):

```
print(as.vector(table(cut(u_ruim, breaks = seq(0, 1, by = 0.1)))))
```

```
[1] 197 203 197 208 190 222 201 204 210 169
```

```
# Cada ponto do gráfico é um par (U_n, U_{n+1}). head(u, -1) remove o último
# valor do vetor e tail(u, -1) remove o primeiro: assim os dois vetores ficam
# defasados em uma posição
rotulos <- c("a = 1103515245 (bom)", "a = 5 (ruim)")

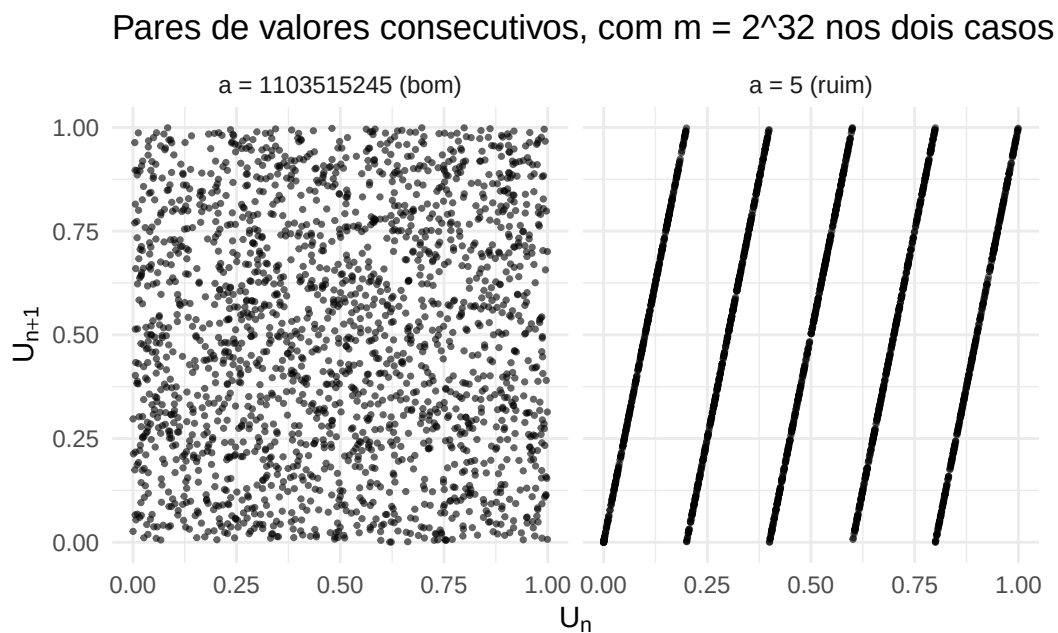
dados <- data.frame(
  atual = c(head(u_bom, -1), head(u_ruim, -1)),
```

```

    proximo = c(tail(u_bom, -1), tail(u_ruim, -1)),
    gerador = factor(rep(rotulos, each = B), levels = rotulos)
)

# coord_fixed deixa as duas escalas iguais, para que o quadrado pareça quadrado
ggplot(dados, aes(x = atual, y = proximo)) +
  geom_point(size = 0.6, alpha = 0.6) +
  facet_wrap(~ gerador) +
  coord_fixed() +
  labs(title = "Pares de valores consecutivos, com m = 2^32 nos dois casos",
       x = expression(U[n]), y = expression(U[n + 1])) +
  theme_minimal()

```



20 Python

```
import numpy as np
import matplotlib.pyplot as plt

# Reaproveitamos a classe definida no Exemplo 1. Os dois geradores usam o mesmo
# m = 2^32 (valor padrão da classe) e a mesma semente: só o multiplicador muda
lcg_bom = LinearCongruentialGenerator(semente=5, a=1103515245, c=12345)
lcg_ruim = LinearCongruentialGenerator(semente=5, a=5, c=3)

B = 2000
u_bom = [lcg_bom.gerar() for _ in range(B + 1)]
u_ruim = [lcg_ruim.gerar() for _ in range(B + 1)]

# Antes do gráfico, os testes ingênuos aplicados ao gerador ruim
print("Média dos valores do gerador ruim:", round(np.mean(u_ruim), 4))
```

Média dos valores do gerador ruim: 0.4948

```
print("Contagens em 10 faixas de largura 0,1 (o ideal é 200 em cada):")
```

Contagens em 10 faixas de largura 0,1 (o ideal é 200 em cada):

```
print(np.histogram(u_ruim, bins=10, range=(0, 1))[0])
```

[197 203 197 208 190 222 201 204 210 169]

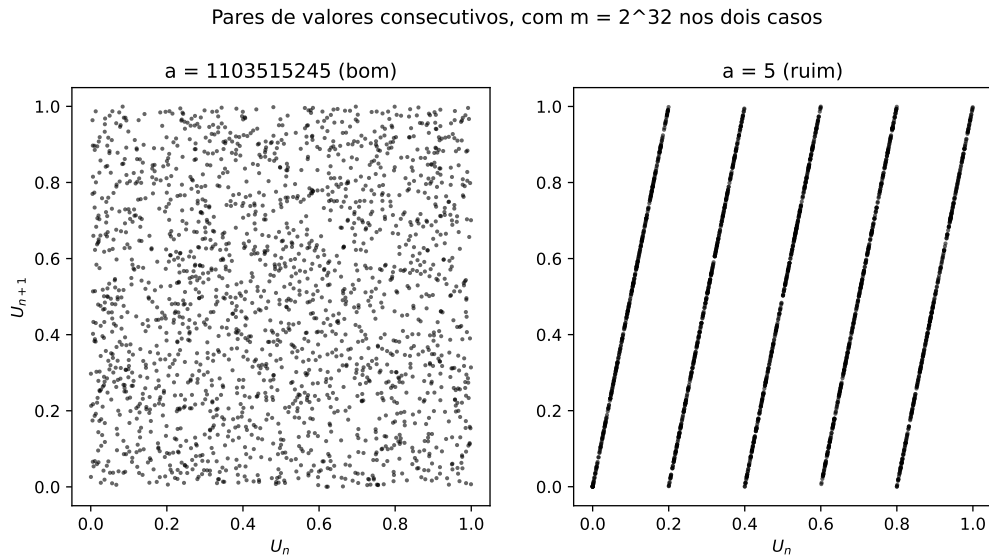
```
# Cada ponto do gráfico é um par (U_n, U_{n+1}): a fatia u[:-1] remove o último
# valor da lista e u[1:] remove o primeiro, deixando as duas defasadas em uma
# posição
rotulos = ["a = 1103515245 (bom)", "a = 5 (ruim)"]
sequencias = [u_bom, u_ruim]

fig, eixos = plt.subplots(1, 2, figsize=(10, 5))
```

```

for eixo, u, rotulo in zip(eixos, sequencias, rotulos):
    eixo.plot(u[:-1], u[1:], 'o', markersize=1.5, color='black', alpha=0.6)
    eixo.set_title(rotulo)
    eixo.set_xlabel('$U_n$')
    # set_aspect deixa as duas escalas iguais, para que o quadrado pareça quadrado
    eixo.set_aspect('equal')
eixos[0].set_ylabel('$U_{n+1}$')
fig.suptitle('Pares de valores consecutivos, com m = 2^32 nos dois casos')
plt.show()

```



O gerador ruim passa nos dois testes ingênuos — média próxima de 0,5 e cerca de 200 valores em cada faixa —, mas o gráfico dos pares é devastador: todos os pontos caem sobre **cinco retas paralelas**. Um par (U_n, U_{n+1}) produzido por esse gerador nunca cai na maior parte do quadrado. Se usássemos essas uniformes para sortear pontos no plano (como fizemos no Capítulo 1 para estimar π), sortearíamos sempre pontos das mesmas cinco retas.

O motivo é simples de ver na fórmula. Dividindo $X_{n+1} = (aX_n + c) \bmod m$ por m ,

$$U_{n+1} = \left(a U_n + \frac{c}{m} \right) \bmod 1,$$

ou seja, $U_{n+1} = aU_n + c/m - k$ para algum inteiro k . Cada valor de k define uma reta de inclinação a , e como $U_n < 1$ existem no máximo a retas possíveis. Com $a = 5$, são cinco retas — visíveis a olho nu. Com $a = 1103515245$, são mais de um bilhão de retas, separadas por uma distância da ordem de $1/a$: elas continuam lá, mas estão tão próximas umas das outras que preenchem o quadrado.

⚠ Atenção: nenhum teste sozinho basta

Já vimos três defeitos e três testes diferentes: período curto (detectado pela contagem de valores distintos), ponto fixo (detectado ao olhar a sequência) e estrutura em retas (detectada pelos pares). Nenhum dos testes detecta os defeitos que os outros detectam. É por isso que geradores modernos são avaliados por **baterias** de dezenas de testes estatísticos, e não por um critério só. A análise das retas formadas pelos pares — e, em dimensão maior, pelas trincas (U_n, U_{n+1}, U_{n+2}) — é conhecida como **teste espectral**, e é justamente o critério pelo qual os multiplicadores dos bons LCGs são escolhidos.

20.1 Exemplo 4: gerando números uniformes com uma moeda

O LCG produz uniformes a partir de contas com inteiros. Há uma construção diferente, e bem mais intuitiva, que produz uniformes a partir de uma fonte de bits aleatórios: lançamentos de uma moeda honesta.

A ideia é usar a **expansão binária** do número. Qualquer $u \in [0, 1)$ pode ser escrito como

$$u = \sum_{i=1}^{\infty} \frac{b_i}{2^i} = \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \dots,$$

onde cada $b_i \in \{0, 1\}$ é um dígito binário. Sortear u uniformemente equivale, então, a sortear os dígitos b_i : se $B_1, B_2, \dots$ são v.a. independentes com $\mathbb{P}(B_i = 1) = 1/2$ — ou seja, lançamentos de uma moeda honesta —, então

$$U = \sum_{i=1}^{\infty} \frac{B_i}{2^i} \sim \text{Unif}(0, 1).$$

Na prática, paramos após k lançamentos. Com k finito, U assume apenas os 2^k valores da forma $j/2^k$, com $j = 0, 1, \dots, 2^k - 1$, cada um com probabilidade 2^{-k} . Para $k = 32$ isso são mais de 4 bilhões de valores igualmente espaçados em $[0, 1)$, indistinguíveis de uma uniforme contínua para qualquer uso prático.

💡 Pseudo-algoritmo: uniforme a partir de uma moeda

Entrada: o número k de lançamentos (por exemplo, $k = 32$).

1. Faça $U \leftarrow 0$.
2. Para $i = 1, 2, \dots, k$:
 - a. Lance a moeda e chame o resultado de B_i (1 para cara, 0 para coroa);

b. Atualize $U \leftarrow U + B_i \cdot 2^{-i}$.

3. Devolva U .

21 R

```
# Carregando pacotes necessários
library(ggplot2)

# Função para simular o lançamento de uma moeda honesta
lancar_moeda <- function() {
  # Lançar moeda honesta: 0 para coroa (K) e 1 para cara (C)
  sample(c(0, 1), 1)
}

# Função para gerar um número uniformemente distribuído usando uma moeda
gerar_numero_uniforme <- function(n_bits = 32) {
  numero <- 0
  for (i in 1:n_bits) {
    bit <- lancar_moeda()
    # O i-ésimo lançamento vale  $2^{-i}$ : o primeiro decide a metade do
    # intervalo, o segundo o quarto, e assim por diante
    numero <- numero + bit * 2(-i)
  }
  return(numero)
}

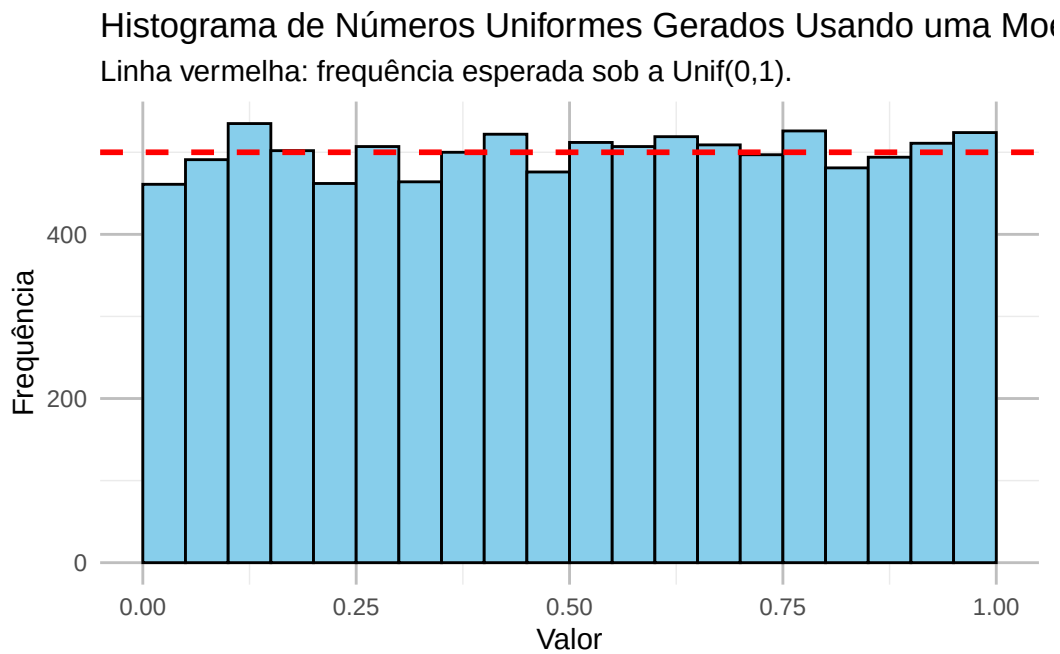
set.seed(42)

# Gerando 10000 números uniformemente distribuídos
numeros_uniformes <- sapply(1:10000, function(x) gerar_numero_uniforme())

# Convertendo para um data.frame
dados <- data.frame(numeros_uniformes = numeros_uniformes)

# Plotando o histograma dos números gerados
ggplot(dados, aes(x = numeros_uniformes)) +
  # Como antes, binwidth com boundary = 0 alinha as faixas em [0; 0,05), ...
  geom_histogram(binwidth = 0.05, boundary = 0,
    fill = 'skyblue', color = 'black') +
```

```
# Como antes, a linha vermelha marca a frequência esperada em cada faixa
geom_hline(yintercept = length(numeros_uniformes) / 20, color = 'red',
           linetype = 'dashed', linewidth = 1) +
ggtitle('Histograma de Números Uniformes Gerados Usando uma Moeda Honesta',
       subtitle = 'Linha vermelha: frequência esperada sob a Unif(0,1).') +
xlab('Valor') +
ylab('Frequência') +
theme_minimal() +
theme(panel.grid.major = element_line(color = "grey"))
```



22 Python

```
import numpy as np
import matplotlib.pyplot as plt

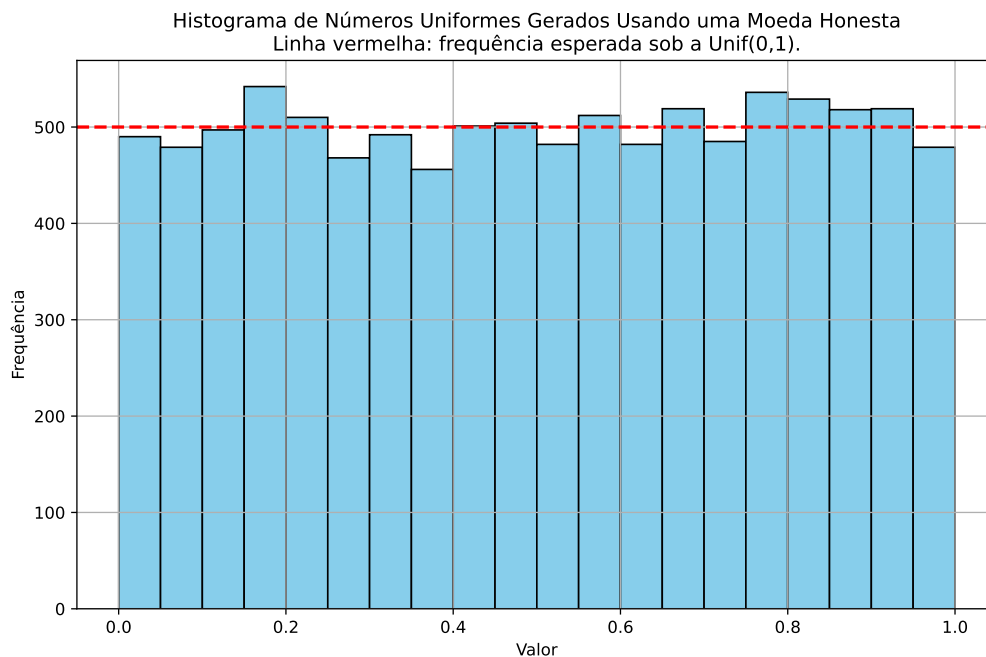
# Função para simular o lançamento de uma moeda honesta
def lancar_moeda():
    # Lançar moeda honesta: 0 para coroa (K) e 1 para cara (C)
    return np.random.choice([0, 1])

# Função para gerar um número uniformemente distribuído usando uma moeda
def gerar_numero_uniforme(n_bits=32):
    numero = 0
    for i in range(1, n_bits + 1):
        bit = lancar_moeda()
        # O i-ésimo lançamento vale  $2^{-i}$ : o primeiro decide a metade do
        # intervalo, o segundo o quarto, e assim por diante
        numero += bit * 2 ** (-i)
    return numero

np.random.seed(42)

# Gerando 10000 números uniformemente distribuídos
numeros_uniformes = [gerar_numero_uniforme() for _ in range(10000)]

# Plotando o histograma dos números gerados
plt.figure(figsize=(10, 6))
plt.hist(numeros_uniformes, bins=20, color='skyblue', edgecolor='black')
# Como antes, a linha vermelha marca a frequência esperada em cada faixa
plt.axhline(len(numeros_uniformes) / 20, color='red', linestyle='--', linewidth=2)
plt.title('Histograma de Números Uniformes Gerados Usando uma Moeda Honesta\n'
          'Linha vermelha: frequência esperada sob a Unif(0,1).')
plt.xlabel('Valor')
plt.ylabel('Frequência')
plt.grid(True)
plt.show()
```



Este exemplo tem valor conceitual: ele mostra que gerar uma $\text{Unif}(0,1)$ não exige nada além de uma fonte de bits independentes e honestos. Na prática, porém, ele é circular como gerador — `sample` e `np.random.choice` já são baseados num gerador pseudoaleatório. O que ele descreve bem é o que acontece quando existe uma fonte física de bits (ruído térmico, por exemplo), que é como sistemas operacionais produzem números *verdadeiramente* aleatórios.

Com uniformes na mão, o problema passa a ser outro: como transformá-las em amostras de qualquer distribuição que se queira. É o assunto dos próximos capítulos.

22.1 Exercícios

Exercício 1. Use a implementação do LCG do Exemplo 1.

- (a) Gere 5 números com semente $X_0 = 1$. Em seguida, crie um novo gerador com a mesma semente e gere outros 5. Compare as duas sequências e explique o resultado.
- (b) Repita com $X_0 = 2$. A sequência muda? E o histograma de 10 000 valores muda de forma perceptível?
- (c) Explique por que a semente precisa ser informada junto com qualquer resultado de simulação que se pretenda reproduzir.

Exercício 2. Escreva uma função `período(a, c, m, x0)` que devolva o período do LCG, isto é, o número de iterações até que a sequência comece a se repetir. (Dica: guarde os valores já visitados e pare quando um deles reaparecer.)

- (a) Use-a para conferir os períodos dos três geradores do Exemplo 2.
- (b) Fixe $m = 64$ e $c = 3$ e calcule o período para cada valor de a entre 1 e 63. Quais valores de a atingem o período máximo? Eles são exatamente os que satisfazem as condições de Hull–Dobell?

Exercício 3. Os parâmetros $a = 1664525$, $c = 1013904223$ e $m = 2^{32}$ aparecem no livro *Numerical Recipes*.

- (a) Verifique que eles satisfazem as três condições de Hull–Dobell.
- (b) Gere 10 000 números com esse gerador e compare o histograma com o do Exemplo 1.
- (c) (Desafio) Com esses parâmetros, o maior valor possível de $aX_n + c$ é da ordem de 7×10^{15} , menor que $2^{53} \approx 9 \times 10^{15}$. Por que isso significa que, neste caso, a versão em R pode dispensar o pacote `gmp`? Confira na prática que as duas implementações dão o mesmo resultado.

Exercício 4. Este exercício explora o gráfico dos pares consecutivos do Exemplo 3. Em todos os itens, use $m = 2^{32}$ e semente $X_0 = 5$.

- (a) Refaça o gráfico dos pares para o gerador do Exercício 3 ($a = 1664525$, $c = 1013904223$). Ele se parece mais com qual dos dois geradores do Exemplo 3?
- (b) Mantendo $c = 3$, faça o gráfico para $a = 9$, $a = 21$ e $a = 65$. Verifique antes que os três satisfazem as condições de Hull–Dobell. Conte as retas em cada gráfico e confira a previsão do texto: o número de retas é a .
- (c) Para o gerador com $a = 5$ e $c = 3$, verifique numericamente a relação $U_{n+1} = 5U_n + 3/m - k$: calcule $5U_n + 3/m - U_{n+1}$ para todos os pares gerados e confirme que o resultado é sempre um número inteiro (a menos de erro de arredondamento). Quantos valores distintos de k aparecem?
- (d) (Desafio) O gerador **RANDU** ($a = 65539$, $c = 0$, $m = 2^{31}$) foi muito usado nos anos 1960 e passa sem dificuldade no teste dos pares. Mostre que $a^2 = 6a - 9 + 2^{32}$ e conclua que

$$X_{n+2} = (6X_{n+1} - 9X_n) \mod 2^{31}.$$

Verifique essa igualdade numericamente (aqui os produtos são pequenos o bastante para dispensar o pacote `gmp`) e explique por que ela faz com que as trincas (U_n, U_{n+1}, U_{n+2}) fiquem confinadas a poucos planos no espaço.

Exercício 5. Sobre a construção com a moeda (Exemplo 4).

- (a) Refaça o exemplo com $k = 1, 2, 3$ e 8 bits, fazendo o histograma em cada caso. Descreva o que muda e explique por que k pequeno não produz algo parecido com uma $\text{Unif}(0, 1)$.
- (b) Qual é o maior valor que `gerar_numero_uniforme` pode devolver quando $k = 32$? Por que o resultado nunca é exatamente igual a 1?
- (c) (Desafio) Modifique a função para usar uma moeda viciada, com probabilidade $p \neq 1/2$ de dar cara. O número gerado ainda tem distribuição uniforme? Faça o histograma com $p = 0,3$ e descreva o formato obtido.

23 Técnica da Inversão para Variáveis Discretas

No capítulo anterior vimos como gerar números pseudoaleatórios com distribuição $\text{Unif}(0, 1)$. A partir de agora, esses números uniformes são a nossa única matéria-prima: todo o resto do livro consiste em transformá-los em v.a. com a distribuição que quisermos.

A **técnica da inversão** é a primeira dessas transformações, e é uma maneira poderosa de gerar v.a. com uma distribuição arbitrária. A ideia básica é usar a **função de distribuição acumulada (f.d.a.)** para mapear um número uniforme gerado entre 0 e 1 no valor correspondente da v.a. discreta.

23.1 Inversa da f.d.a.

O método da inversão recebe esse nome pois seu algoritmo também pode ser caracterizado pela função inversa da f.d.a. A **inversa da f.d.a.**, também conhecida como **função quantil**, é definida da seguinte forma:

i Definição: inversa da f.d.a.

Seja F a função de distribuição acumulada de uma v.a. X . A inversa de F , denotada por F^{-1} , é definida como

$$F^{-1}(p) = \inf\{x \in \mathbb{R} : F(x) \geq p\}, \quad \text{para } p \in (0, 1).$$

Em palavras: $F^{-1}(p)$ toma um número p , que representa uma probabilidade acumulada, e devolve o **menor** valor x cuja probabilidade acumulada já alcançou p . Assim, o menor valor x_i tal que $F(x_i) \geq u$ é justamente $F^{-1}(u)$.

⚠ Atenção: no caso discreto, a inversa não é a usual

Quando X é discreta, F é uma função escada: ela é constante entre os valores do suporte e dá saltos de tamanho $p(x_i)$ em cada x_i . Uma função assim não é injetora, e portanto não tem inversa no sentido usual — daí a necessidade do $\inf$ na definição acima.

Uma consequência prática: vale que $F^{-1}(F(x_i)) = x_i$ para os pontos x_i do suporte,

mas **não** para um x qualquer. Por exemplo, se X só assume os valores 0 e 1, então $F(0,7) = F(0)$, e portanto $F^{-1}(F(0,7)) = 0$, que é diferente de 0,7.

23.2 Geração de Variáveis Aleatórias Discretas Genéricas

Considere uma v.a. discreta X que assume os valores $x_1 < x_2 < \dots$. Dadas as probabilidades $p(x_i) = \mathbb{P}(X = x_i)$ de cada um desses valores, a f.d.a. avaliada em x_i é

$$F(x_i) = \sum_{j=1}^i p(x_j).$$

💡 Pseudo-algoritmo: inversão para v.a. discretas

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Encontre o menor valor x_i tal que $F(x_i) \geq U$.
3. Retorne x_i .

Na prática, o passo 2 é feito percorrendo os valores $x_1, x_2, \dots$ em ordem e acumulando as probabilidades até que a soma acumulada alcance U . O código a seguir faz exatamente isso e ilustra graficamente o que está acontecendo: a reta horizontal marca o valor sorteado de U , e a reta vertical marca o valor de X devolvido pelo algoritmo.

24 R

```
set.seed(42)

# Exemplo de valores e probabilidades de uma variável aleatória discreta
valores <- c(0, 1, 2, 3, 4, 5, 6)
probabilidades <- c(0, 0.1, 0.2, 0.3, 0.25, 0.15, 0)

# cumsum acumula as probabilidades: cdf[i] = p(x_1) + ... + p(x_i) = F(x_i)
cdf <- cumsum(probabilidades)

# Passo 1 do algoritmo: gerar um número aleatório uniforme
u <- runif(1)

# Passo 2: percorrer os valores em ordem e parar no primeiro com F(x_i) >= u
valor_gerado <- NA
for (i in seq_along(valores)) {
  if (u <= cdf[i]) {
    valor_gerado <- valores[i]
    break # encontramos o menor x_i; não precisamos olhar os seguintes
  }
}
valor_gerado
```

```
[1] 5
```

```
library(ggplot2)

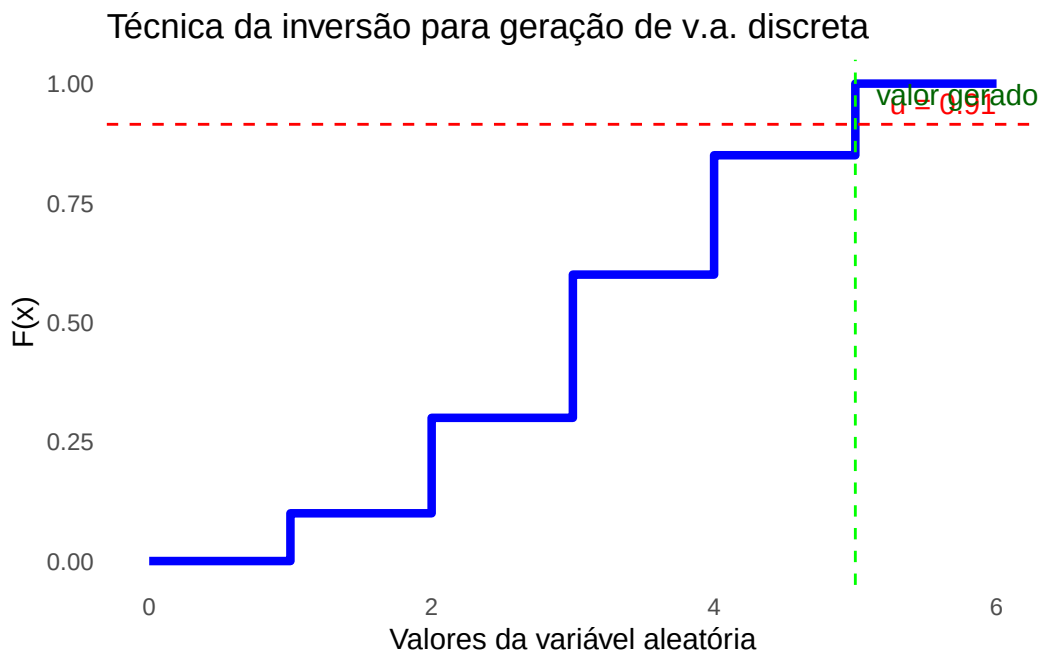
# Criando um data frame com os valores e a f.d.a. para o gráfico
df <- data.frame(valores = valores, cdf = cdf)

# Gráfico da f.d.a. com o número aleatório u e o valor gerado
ggplot(df, aes(x = valores, y = cdf)) +
  geom_step(direction = "hv", color = "blue", linewidth = 1.5) +
  geom_hline(yintercept = u, color = "red", linetype = "dashed") +
```

```

geom_vline(xintercept = valor_gerado, color = "green", linetype = "dashed") +
labs(title = "Técnica da inversão para geração de v.a. discreta",
     x = "Valores da variável aleatória",
     y = "F(x)") +
annotate("text", x = max(valores), y = u,
        label = sprintf("u = %.2f", u), hjust = 1, vjust = -0.5, color = "red") +
annotate("text", x = valor_gerado, y = max(cdf),
        label = sprintf("valor gerado = %d", valor_gerado),
        hjust = -0.1, vjust = 1, color = "darkgreen") +
theme_minimal() +
theme(panel.grid = element_blank())

```



25 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Exemplo de valores e probabilidades de uma variável aleatória discreta
valores = [0, 1, 2, 3, 4, 5, 6]
probabilidades = [0, 0.1, 0.2, 0.3, 0.25, 0.15, 0]

# np.cumsum acumula as probabilidades: cdf[i] = p(x_1) + ... + p(x_i) = F(x_i)
cdf = np.cumsum(probabilidades)

# Passo 1 do algoritmo: gerar um número aleatório uniforme
u = np.random.uniform(0, 1)

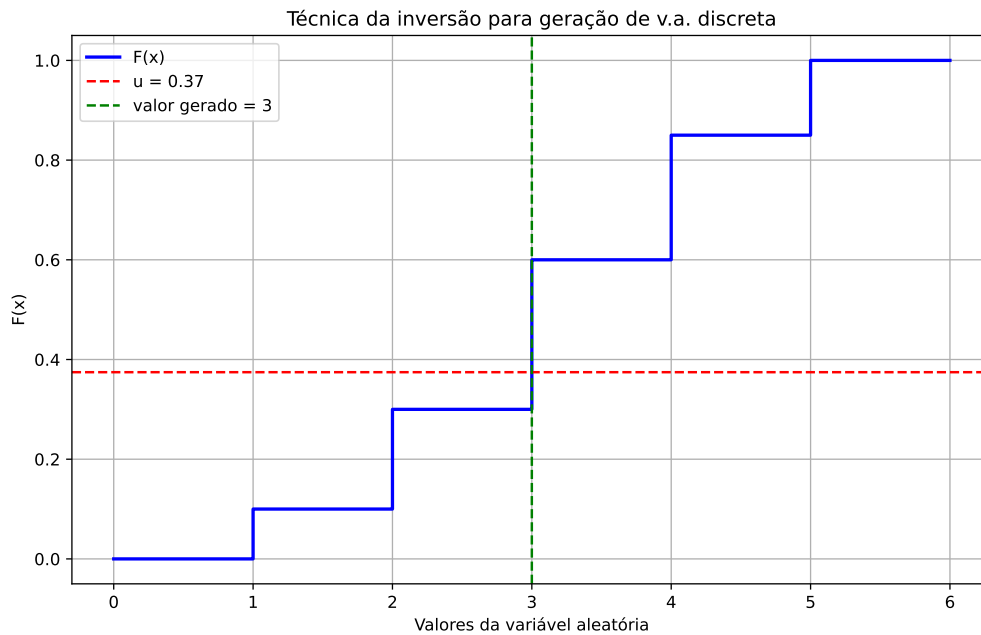
# Passo 2: percorrer os valores em ordem e parar no primeiro com F(x_i) >= u
valor_gerado = None
for i, valor in enumerate(valores):
    if u <= cdf[i]:
        valor_gerado = valor
        break # encontramos o menor x_i; não precisamos olhar os seguintes
print(valor_gerado)
```

3

```
plt.figure(figsize=(10, 6))

# Gráfico da f.d.a. com o número aleatório u e o valor gerado
plt.step(valores, cdf, label='F(x)', color='blue', linewidth=2, where='post')
plt.axhline(y=u, color='red', linestyle='--', label=f'u = {u:.2f}')
plt.axvline(x=valor_gerado, color='green', linestyle='--',
            label=f'valor gerado = {valor_gerado}')
plt.title('Técnica da inversão para geração de v.a. discreta')
```

```
plt.xlabel('Valores da variável aleatória')
plt.ylabel('F(x)')
plt.legend()
plt.grid(True)
plt.show()
```



25.1 Por que o método funciona

Até aqui, o algoritmo foi apresentado como uma receita. A proposição a seguir garante que ele de fato produz a distribuição desejada.

! Proposição

Considere uma v.a. discreta X com suporte ordenado $x_1 < x_2 < \dots$ e probabilidades $p_i = \mathbb{P}(X = x_i)$. Defina $F(x_i) = \sum_{j \leq i} p_j$ e $F(x) = 0$ para $x < x_1$. Seja $U \sim \text{Unif}(0, 1)$ e

$$\tilde{X} = \min\{x_i : F(x_i) \geq U\}.$$

Então $\tilde{X}$ tem a mesma distribuição de X .

Demonstração

O algoritmo devolve x_i exatamente quando a soma acumulada ainda não havia alcançado U no passo anterior, mas passa a alcançá-la em x_i . Ou seja, para cada $i \geq 1$,

$$\{\tilde{X} = x_i\} = \{F(x_{i-1}) < U \leq F(x_i)\},$$

onde convencionamos $F(x_0) = 0$. Como $U \sim \text{Unif}(0, 1)$, a probabilidade de U cair em um intervalo contido em $(0, 1)$ é o comprimento desse intervalo. Logo,

$$\mathbb{P}(\tilde{X} = x_i) = F(x_i) - F(x_{i-1}) = p_i.$$

Portanto, $\tilde{X}$ reproduz exatamente a função de probabilidade desejada. $\square$

25.2 Exemplo 1: Geração de uma Bernoulli por inversão

Para $X \sim \text{Bernoulli}(p)$, temos $\mathbb{P}(X = 0) = 1 - p$, $\mathbb{P}(X = 1) = p$ e

$$F(0) = 1 - p, \quad F(1) = 1.$$

Assim, o método da inversão consiste em gerar $U \sim \text{Unif}(0, 1)$ e devolver o menor valor cuja acumulada alcança U : se $U \leq F(0) = 1 - p$, o menor valor é 0; caso contrário, é 1. Ou seja,

$$X = F^{-1}(U) = \begin{cases} 0, & \text{se } U \leq 1 - p, \\ 1, & \text{se } U > 1 - p. \end{cases}$$

Pseudo-algoritmo: Bernoulli

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Se $U \leq 1 - p$, retorne 0; caso contrário, retorne 1.

Abaixo, simulamos n observações, calculamos a distribuição empírica associada a elas e a comparamos com a real de uma Bernoulli.

26 R

```
set.seed(123)

# parâmetros
p <- 0.3
n <- 10000

# simulação por inversão
u <- runif(n)
x <- as.integer(u > 1 - p) # 1 se U > 1-p, senão 0

# distribuição empírica (garantindo níveis 0 e 1)
emp <- prop.table(table(factor(x, levels = c(0, 1))))
real <- c(`0` = 1 - p, `1` = p)

# tabela de comparação
comp <- data.frame(
  valor = c(0, 1),
  real = as.numeric(real),
  estimada = as.numeric(emp)
)
comp
```

	valor	real	estimada
1	0	0.7	0.7048
2	1	0.3	0.2952

```
# gráfico comparando real vs estimada
library(tidyr); library(ggplot2)
comp_long <- pivot_longer(comp, cols = c(real, estimada),
  names_to = "tipo", values_to = "prob")

ggplot(comp_long, aes(x = factor(valor), y = prob, fill = tipo)) +
  geom_col(position = "dodge", color = "black") +
```

```
scale_x_discrete(name = "Valor de X") +
labs(title = sprintf("Bernoulli(p) com p = %.2f - n = %d", p, n),
     y = "Probabilidade") +
theme_minimal() +
theme(panel.grid.major = element_blank(), legend.title = element_blank())
```



27 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)

# parâmetros
p = 0.3
n = 10000

# simulação por inversão
u = np.random.uniform(0, 1, n)
x = (u > 1 - p).astype(int) # 1 se U > 1-p, senão 0

# distribuição empírica (garantindo níveis 0 e 1)
counts = np.bincount(x, minlength=2)
emp = counts / n
real = np.array([1 - p, p])

# tabela de comparação (impresso no console)
print("valor | real    | estimada")
```

```
valor | real    | estimada
```

```
for v in [0, 1]:
    print(f"{v:5d} | {real[v]:.4f} | {emp[v]:.4f}")
```

```
0 | 0.7000 | 0.6995
1 | 0.3000 | 0.3005
```

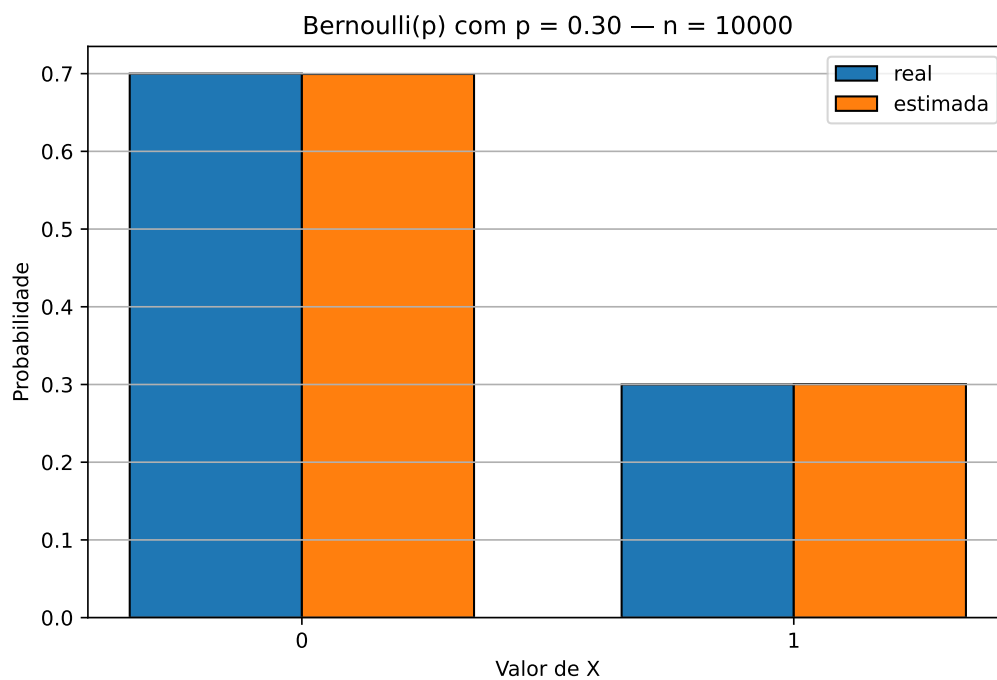
```
# gráfico comparando real vs estimada
vals = np.array([0, 1], dtype=float)
width = 0.35
```

```
plt.figure(figsize=(8, 5))
plt.bar(vals - width/2, real, width, edgecolor='black', label='real')
plt.bar(vals + width/2, emp, width, edgecolor='black', label='estimada')

plt.xticks(vals, ['0', '1'])
```

([<matplotlib.axis.XTick object at 0x70fc9f1647d0>, <matplotlib.axis.XTick object at 0x70fc9f1647d0>])

```
plt.xlabel('Valor de X')
plt.ylabel('Probabilidade')
plt.title(f'Bernoulli(p) com p = {p:.2f} - n = {n}')
plt.legend()
plt.grid(True, axis='y')
plt.show()
```



27.1 Exemplo 2: Geração de Variáveis Aleatórias com Distribuição Geométrica

A distribuição geométrica modela o número de falhas até o primeiro sucesso em uma sequência de experimentos de Bernoulli. Se a probabilidade de sucesso em cada tentativa é p , a função de probabilidade é dada por

$$\mathbb{P}(X = k) = (1 - p)^k p, \quad k = 0, 1, 2, \dots$$

onde k representa o número de falhas antes do primeiro sucesso.

Nos exemplos anteriores o suporte era finito e pequeno, então dava para percorrer os valores um a um. Aqui o suporte é infinito, e vale a pena obter uma fórmula fechada para F^{-1} . Começamos calculando a f.d.a., usando a soma da progressão geométrica $\sum_{x=0}^k r^x = \frac{1-r^{k+1}}{1-r}$ com $r = 1 - p$:

$$\begin{aligned} F(k) &= \mathbb{P}(X \leq k) = \sum_{x=0}^k (1-p)^x p \\ &= p \sum_{x=0}^k (1-p)^x \\ &= p \cdot \frac{1 - (1-p)^{k+1}}{1 - (1-p)} \\ &= p \cdot \frac{1 - (1-p)^{k+1}}{p} \\ &= 1 - (1-p)^{k+1}. \end{aligned}$$

Queremos agora a **inversa da f.d.a.**, ou seja, dado $u \in (0, 1)$, o menor inteiro k tal que $F(k) \geq u$. Note que a condição é uma **desigualdade**, e não uma igualdade: como X é discreta, em geral não existe k inteiro com $F(k) = u$ exatamente. Vamos resolver a desigualdade passo a passo.

Partimos de

$$1 - (1-p)^{k+1} \geq u.$$

Isolando o termo $(1-p)^{k+1}$ (subtraindo 1 dos dois lados e multiplicando por -1 , o que inverte a desigualdade):

$$(1-p)^{k+1} \leq 1 - u.$$

Aplicamos agora o logaritmo natural nos dois lados. Como o logaritmo é crescente, a desigualdade se mantém:

$$\log((1-p)^{k+1}) \leq \log(1-u).$$

Usando a propriedade que permite trazer o expoente para frente:

$$(k+1) \cdot \log(1-p) \leq \log(1-u).$$

 **Atenção:** a desigualdade se inverte aqui

Para $0 < p < 1$ temos $0 < 1-p < 1$, e portanto $\log(1-p) < 0$. Ao dividir os dois lados por $\log(1-p)$, estamos dividindo por um número **negativo**, e o sentido da desigualdade se inverte. Esquecer disso é um erro comum e leva a uma fórmula com $\lfloor \cdot \rfloor$ no lugar de $\lceil \cdot \rceil$.

Dividindo por $\log(1-p) < 0$ e isolando k :

$$k+1 \geq \frac{\log(1-u)}{\log(1-p)} \quad \Leftrightarrow \quad k \geq \frac{\log(1-u)}{\log(1-p)} - 1.$$

Como queremos o **menor** inteiro que satisfaz essa desigualdade, arredondamos para cima:

$$F^{-1}(u) = \left\lceil \frac{\log(1-u)}{\log(1-p)} - 1 \right\rceil.$$

 **Pseudo-algoritmo:** Geométrica

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Retorne $X = \left\lceil \frac{\log(1-U)}{\log(1-p)} - 1 \right\rceil$.

Podemos agora gerar v.a. com distribuição geométrica a partir de números uniformes.

28 R

```
# Função que calcula a inversa da f.d.a. da geométrica
inversa_cdf_geometrica <- function(p, u) {
  # k = número de falhas antes do 1º sucesso
  ceiling(log(1 - u) / log(1 - p) - 1)
}

# Parâmetro p da distribuição geométrica
p <- 0.5

set.seed(123)

# Gerando 1000 números uniformes
n <- 1000
uniformes <- runif(n)

# sapply aplica a função a cada elemento de `uniformes` e devolve um vetor com
# os 1000 resultados; é uma forma compacta de escrever um laço for
geometricas <- sapply(uniformes, inversa_cdf_geometrica, p = p)

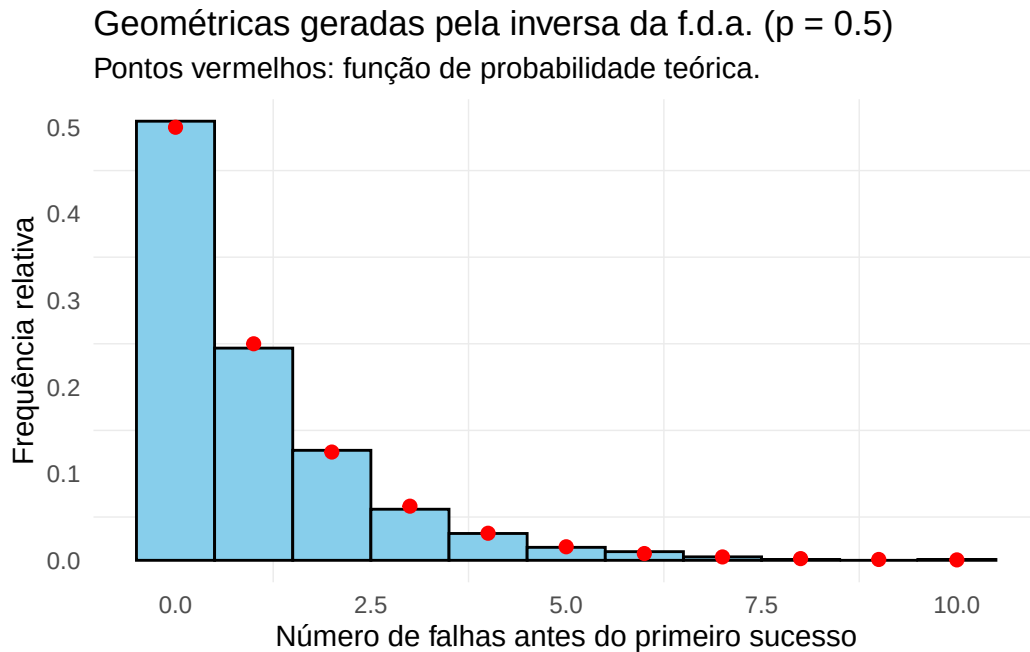
# Plotando um histograma das variáveis geométricas geradas
library(ggplot2)

# Função de probabilidade teórica da geométrica:  $P(X = k) = p (1 - p)^k$ 
k_teorico <- 0:max(geometricas)
df_teorico <- data.frame(k = k_teorico, prob = p * (1 - p)^k_teorico)

# binwidth = 1 com boundary = -0.5 deixa uma barra centrada em cada inteiro.
# Com after_stat(density) a altura de cada barra é a frequência relativa, que
# pode ser comparada diretamente com a função de probabilidade teórica
df <- data.frame(geometricas = geometricas)
ggplot(df, aes(x = geometricas)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 1, boundary = -0.5,
    closed = "left", color = "black", fill = "skyblue") +
  geom_point(data = df_teorico, aes(x = k, y = prob), color = "red", size = 2) +
```



```
labs(title = "Geométricas geradas pela inversa da f.d.a. (p = 0.5)",
     subtitle = "Pontos vermelhos: função de probabilidade teórica.",
     x = "Número de falhas antes do primeiro sucesso",
     y = "Frequência relativa") +
theme_minimal() +
theme(panel.grid.major = element_blank())
```

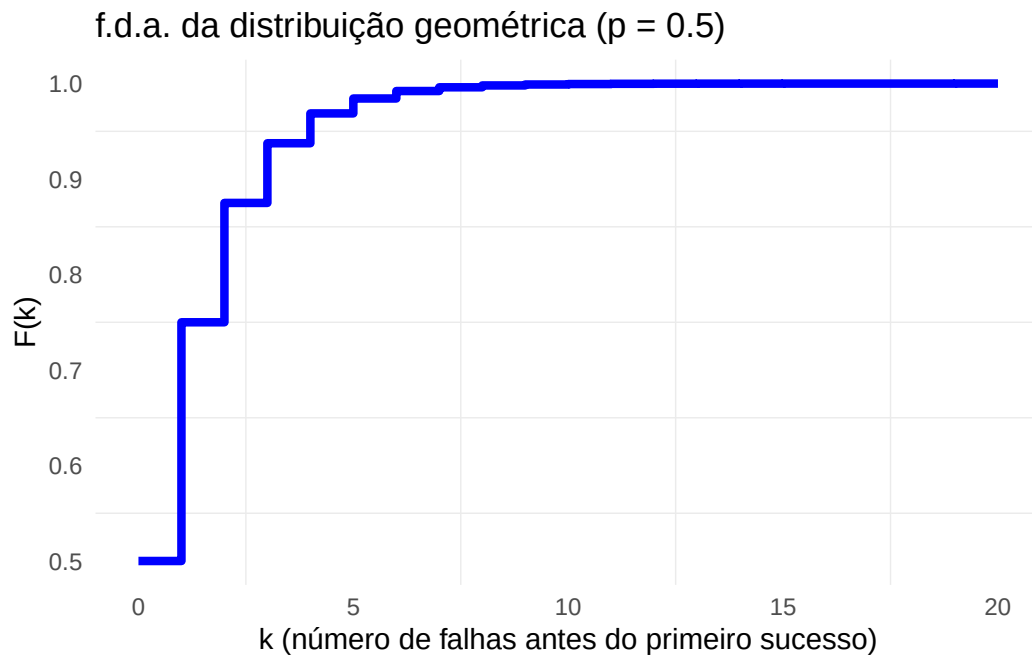


```
# Função para calcular a f.d.a. da distribuição geométrica
cdf_geometrica <- function(k, p) {
  1 - (1 - p)^(k + 1)
}

# Gerando valores de k para plotar a f.d.a.
k_values <- 0:20
cdf_values <- sapply(k_values, cdf_geometrica, p = p)

# Plotando a f.d.a. da distribuição geométrica
df_cdf <- data.frame(k_values = k_values, cdf_values = cdf_values)
ggplot(df_cdf, aes(x = k_values, y = cdf_values)) +
  geom_step(direction = "hv", color = "blue", linewidth = 1.5) +
  labs(title = "f.d.a. da distribuição geométrica (p = 0.5)",
       x = "k (número de falhas antes do primeiro sucesso)",
```

```
y = "F(k)" +  
theme_minimal() +  
theme(panel.grid.major = element_blank())
```



29 Python

```
import numpy as np
import math
import matplotlib.pyplot as plt

# Função que calcula a inversa da f.d.a. da geométrica
def inversa_cdf_geometrica(p, u):
    # k = número de falhas antes do 1º sucesso
    return math.ceil(math.log(1 - u) / math.log(1 - p) - 1)

# Parâmetro p da distribuição geométrica
p = 0.5

np.random.seed(123)

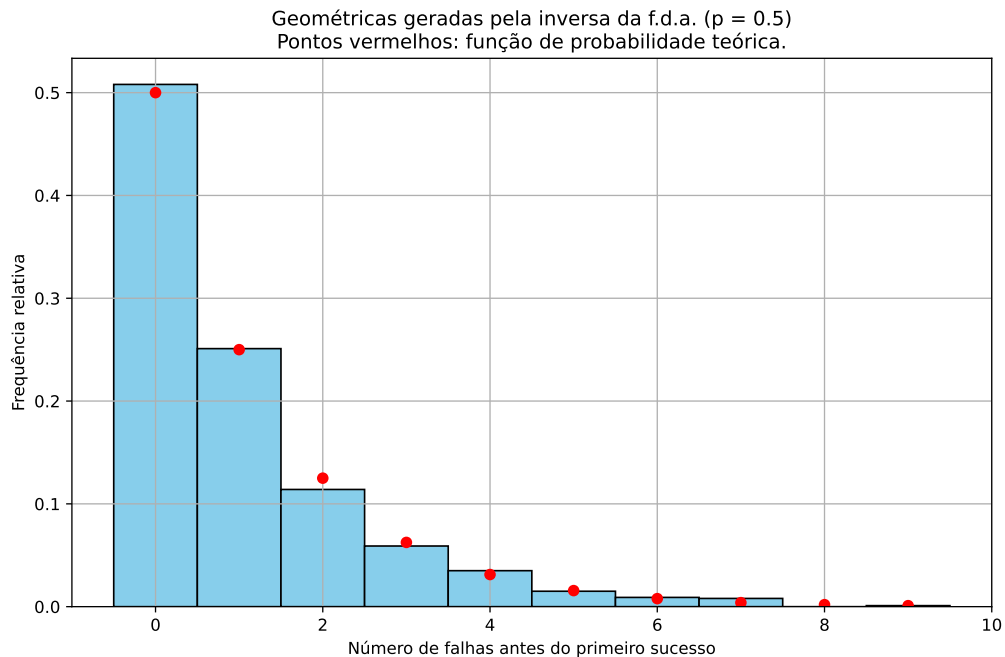
# Gerando 1000 números uniformes
n = 1000
uniformes = np.random.uniform(0, 1, n)

# A list comprehension abaixo aplica a função a cada elemento de `uniformes` e
# devolve uma lista com os 1000 resultados; é uma forma compacta de escrever um for
geometricas = [inversa_cdf_geometrica(p, u) for u in uniformes]

# Função de probabilidade teórica da geométrica:  $P(X = k) = p (1 - p)^k$ 
k_teorico = np.arange(0, max(geometricas) + 1)
prob_teorica = p * (1 - p)**k_teorico

# Plotando um histograma das variáveis geométricas geradas
# align='left' com bins inteiros deixa uma barra centrada em cada inteiro.
# Com density=True a altura de cada barra é a frequência relativa, que pode ser
# comparada diretamente com a função de probabilidade teórica
plt.figure(figsize=(10, 6))
plt.hist(geometricas,
        bins=range(0, max(geometricas) + 2), # +2 para incluir o último valor
        color='skyblue', edgecolor='black', align='left', density=True)
```

```
plt.plot(k_teorico, prob_teorica, 'o', color='red', markersize=6)
plt.title('Geométricas geradas pela inversa da f.d.a. (p = 0.5)\n'
        'Pontos vermelhos: função de probabilidade teórica.')
plt.xlabel('Número de falhas antes do primeiro sucesso')
plt.ylabel('Frequência relativa')
plt.grid(True)
plt.show()
```

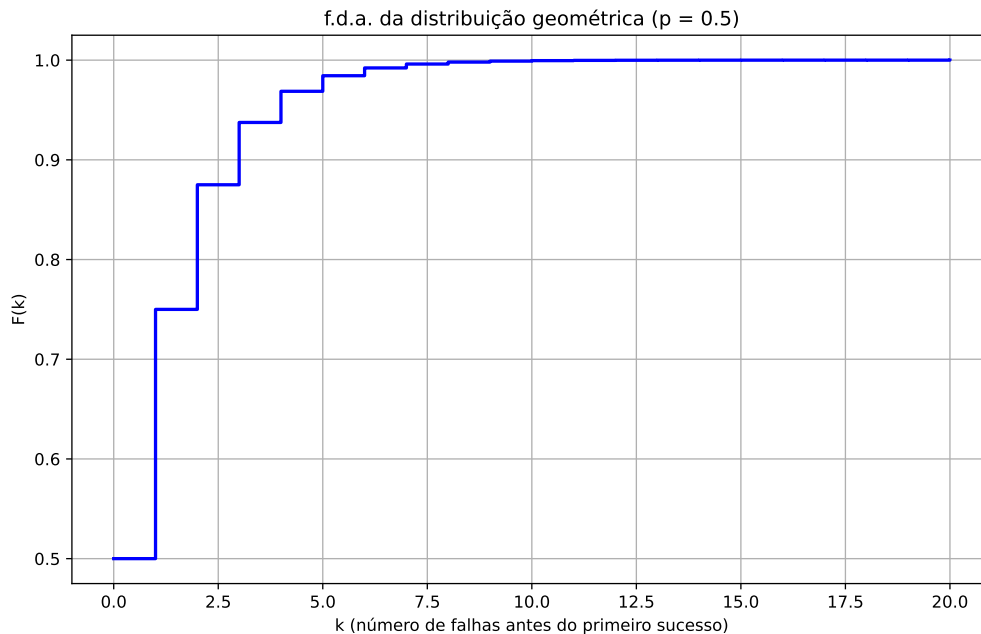


```
# Função para calcular a f.d.a. da distribuição geométrica
def cdf_geometrica(k, p):
    return 1 - (1 - p)**(k + 1)

# Gerando valores de k para plotar a f.d.a.
k_values = np.arange(0, 21)
cdf_values = [cdf_geometrica(k, p) for k in k_values]

# Plotando a f.d.a. da distribuição geométrica
plt.figure(figsize=(10, 6))
plt.step(k_values, cdf_values, where='post', color='blue', linewidth=2)
plt.title('f.d.a. da distribuição geométrica (p = 0.5)')
plt.xlabel('k (número de falhas antes do primeiro sucesso)')
```

```
plt.ylabel('F(k)')
plt.grid(True)
plt.show()
```



29.1 Exemplo 3: Geração de Variáveis Aleatórias com Distribuição Poisson

A distribuição de Poisson é usada para modelar o número de eventos que ocorrem em um intervalo de tempo ou espaço fixo, quando os eventos ocorrem com uma taxa constante λ e de forma independente.

A função de probabilidade da distribuição de Poisson é dada por

$$\mathbb{P}(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

Diferentemente da geométrica, aqui não há fórmula fechada simples para F^{-1} : vamos ter que voltar à ideia de acumular as probabilidades uma a uma até ultrapassar U . O problema é que calcular $\mathbb{P}(X = k)$ do zero para cada k exige recalculer λ^k e $k!$, o que é caro e, para k grande, chega a estourar a capacidade numérica do computador.

A saída é notar que as probabilidades da Poisson satisfazem a relação recursiva

$$\mathbb{P}(X = k + 1) = \frac{\lambda}{k + 1} \cdot \mathbb{P}(X = k), \quad \text{com } \mathbb{P}(X = 0) = e^{-\lambda},$$

que segue diretamente da função de probabilidade:

$$\frac{\mathbb{P}(X = k + 1)}{\mathbb{P}(X = k)} = \frac{\lambda^{k+1}e^{-\lambda}/(k+1)!}{\lambda^k e^{-\lambda}/k!} = \frac{\lambda}{k+1}.$$

Assim, cada probabilidade é obtida da anterior com uma única multiplicação e uma única divisão, sem calcular fatoriais nem potências.

29.1.1 Técnica da Inversão Usando a Fórmula Recursiva

Juntando a recursão com o algoritmo da inversão, obtemos:

Pseudo-algoritmo: Poisson

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Faça $i = 0$, $p = e^{-\lambda}$ e $F = p$.
3. Se $U \leq F$, faça $X = i$ e pare.
4. Caso contrário, atualize $i = i + 1$, $p = \frac{\lambda p}{i}$ e $F = F + p$.
5. Volte para o passo 3.

A seguir implementamos esse método:

30 R

```
# Gera um valor de Poisson por inversão, usando a fórmula recursiva
inversa_cdf_poisson_recursiva <- function(lam, u) {
  k <- 0
  p <- exp(-lam) # P(X = 0)
  F_acm <- p      # acumulada até k = 0

  # Continuamos somando até que F(k) >= u
  while (u > F_acm) {
    k <- k + 1
    p <- p * lam / k # atualiza P(X = k) a partir de P(X = k-1)
    F_acm <- F_acm + p
  }

  return(k)
}

# Parâmetro lambda da distribuição Poisson
lam <- 3

set.seed(123)

# Gerando 1000 números uniformes
n <- 1000
uniformes <- runif(n)

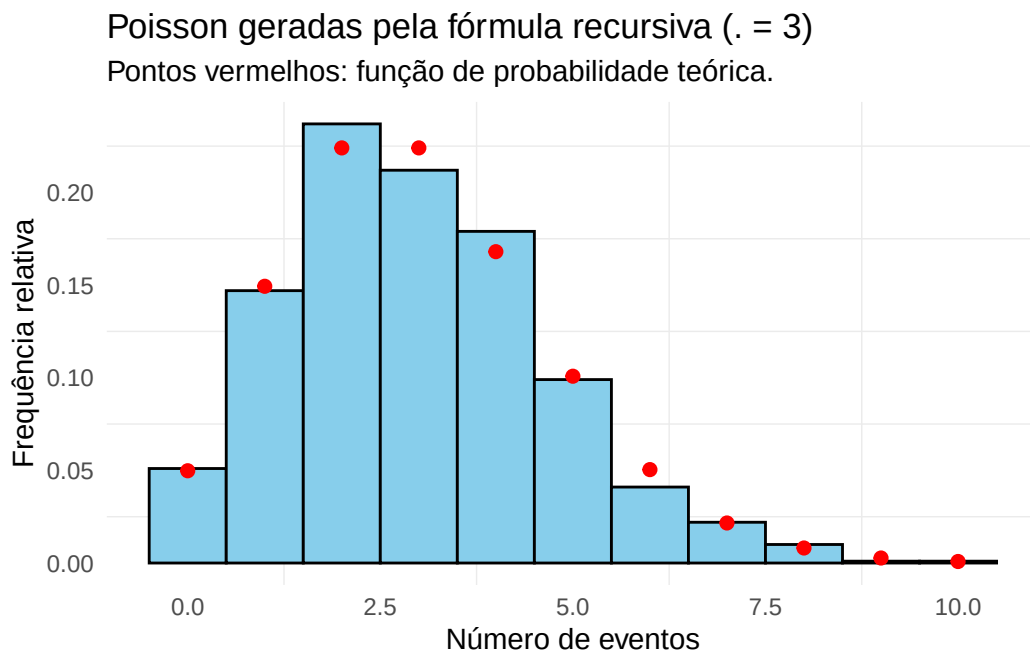
# sapply aplica a função a cada elemento de `uniformes` e devolve um vetor com
# os 1000 resultados; é uma forma compacta de escrever um laço for
poisson_vars <- sapply(uniformes, inversa_cdf_poisson_recursiva, lam = lam)

# Plotando o histograma das variáveis Poisson geradas
library(ggplot2)

# Função de probabilidade teórica da Poisson:  $P(X = k) = e^{-\lambda} \lambda^k / k!$ 
k_teorico <- 0:max(poisson_vars)
```

```
df_teorico <- data.frame(
  k = k_teorico,
  prob = exp(-lam) * lam^k_teorico / factorial(k_teorico)
)

# binwidth = 1 com boundary = -0.5 deixa uma barra centrada em cada inteiro.
# Com after_stat(density) a altura de cada barra é a frequência relativa, que
# pode ser comparada diretamente com a função de probabilidade teórica
df <- data.frame(poisson_vars = poisson_vars)
ggplot(df, aes(x = poisson_vars)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 1, boundary = -0.5,
    closed = "left", color = "black", fill = "skyblue") +
  geom_point(data = df_teorico, aes(x = k, y = prob), color = "red", size = 2) +
  labs(title = "Poisson geradas pela fórmula recursiva (λ = 3)",
    subtitle = "Pontos vermelhos: função de probabilidade teórica.",
    x = "Número de eventos",
    y = "Frequência relativa") +
  theme_minimal() +
  theme(panel.grid.major = element_blank())
```



```
# Função para calcular a f.d.a. da Poisson, também pela recursão
cdf_poisson <- function(k, lam) {
```



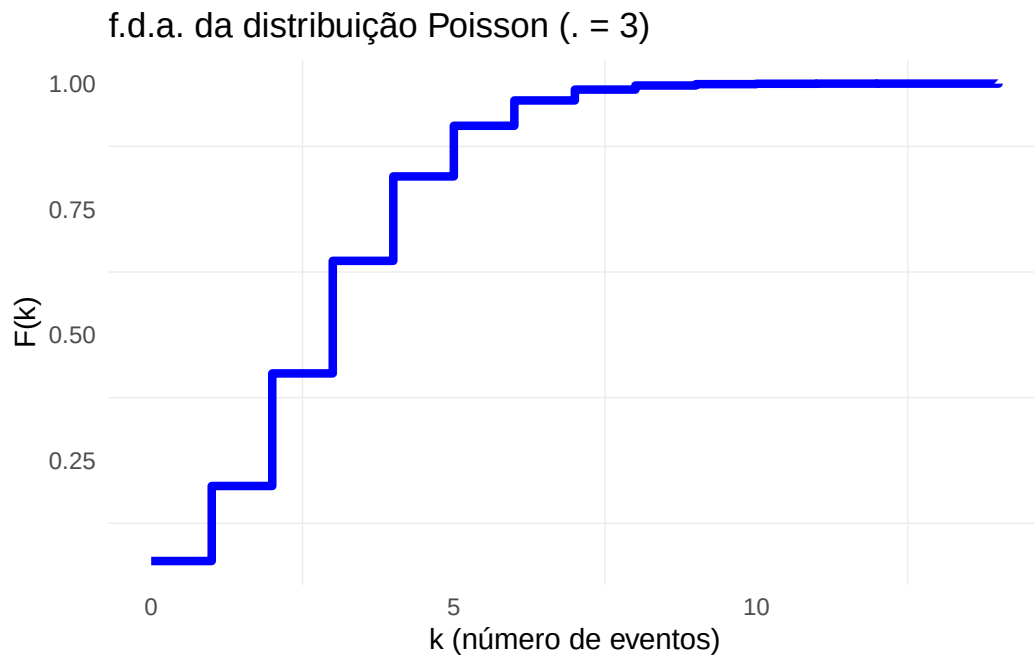
```

cdf <- 0
p <- exp(-lam) # P(X = 0)
for (i in 0:k) {
  cdf <- cdf + p # acumula P(X = i)
  if (i < k) {
    p <- p * lam / (i + 1) # atualiza para o próximo valor
  }
}
return(cdf)
}

# Gerando valores de k para a f.d.a.
k_values <- 0:14
cdf_values <- sapply(k_values, cdf_poisson, lam = lam)

# Plotando a f.d.a. da distribuição Poisson
df_cdf <- data.frame(k_values = k_values, cdf_values = cdf_values)
ggplot(df_cdf, aes(x = k_values, y = cdf_values)) +
  geom_step(direction = "hv", color = "blue", linewidth = 1.5) +
  labs(title = "f.d.a. da distribuição Poisson ( λ = 3)",
       x = "k (número de eventos)",
       y = "F(k)") +
  theme_minimal() +
  theme(panel.grid.major = element_blank())

```



31 Python

```
import numpy as np
import matplotlib.pyplot as plt
import math

# Gera um valor de Poisson por inversão, usando a fórmula recursiva
def inversa_cdf_poisson_recursiva(lam, u):
    k = 0
    p = math.exp(-lam) # P(X = 0)
    F_acm = p          # acumulada até k = 0

    # Continuamos somando até que F(k) >= u
    while u > F_acm:
        k += 1
        p = p * lam / k # atualiza P(X = k) a partir de P(X = k-1)
        F_acm += p

    return k

# Parâmetro lambda da distribuição Poisson
lam = 3

np.random.seed(123)

# Gerando 1000 números uniformes
n = 1000
uniformes = np.random.uniform(0, 1, n)

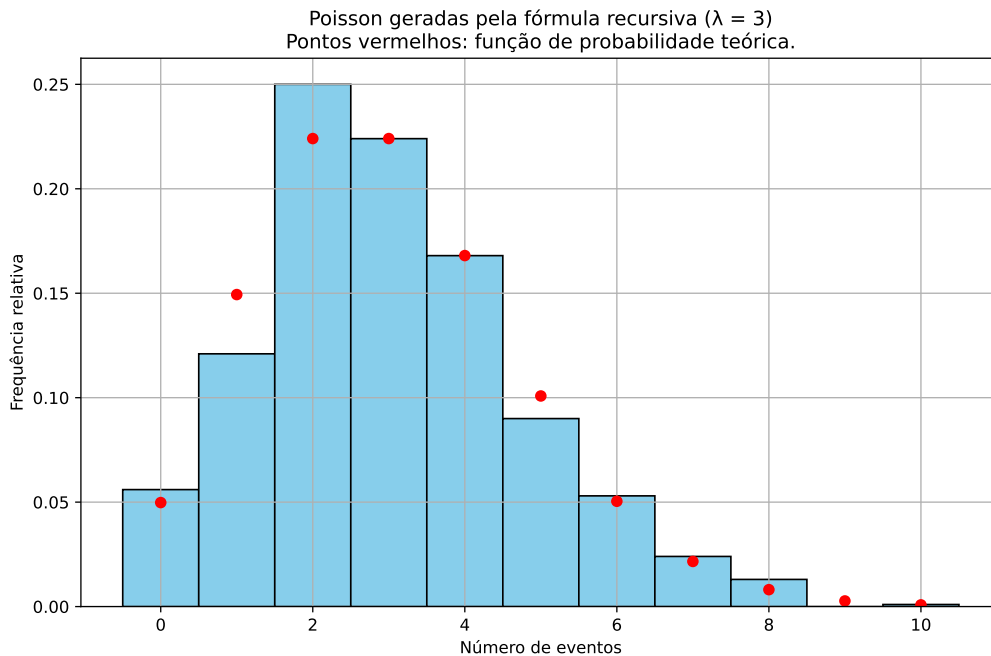
# A list comprehension abaixo aplica a função a cada elemento de `uniformes` e
# devolve uma lista com os 1000 resultados; é uma forma compacta de escrever um for
poisson_vars = [inversa_cdf_poisson_recursiva(lam, u) for u in uniformes]

# Função de probabilidade teórica da Poisson:  $P(X = k) = e^{-\lambda} \lambda^k / k!$ 
k_teorico = np.arange(0, max(poisson_vars) + 1)
prob_teorica = [math.exp(-lam) * lam**k / math.factorial(k) for k in k_teorico]
```

```

# Plotando o histograma das variáveis Poisson geradas
# align='left' com bins inteiros deixa uma barra centrada em cada inteiro.
# Com density=True a altura de cada barra é a frequência relativa, que pode ser
# comparada diretamente com a função de probabilidade teórica
plt.figure(figsize=(10, 6))
plt.hist(poisson_vars,
        bins=range(0, max(poisson_vars) + 2), # +2 para incluir o último valor
        color='skyblue', edgecolor='black', align='left', density=True)
plt.plot(k_teorico, probb_teorica, 'o', color='red', markersize=6)
plt.title('Poisson geradas pela fórmula recursiva (  $\lambda = 3$  )\n'
        'Pontos vermelhos: função de probabilidade teórica.')
plt.xlabel('Número de eventos')
plt.ylabel('Frequência relativa')
plt.grid(True)
plt.show()

```



```

# Função para calcular a f.d.a. da Poisson, também pela recursão
def cdf_poisson(k, lam):
    cdf = 0
    p = math.exp(-lam) # P(X = 0)
    for i in range(k + 1):

```

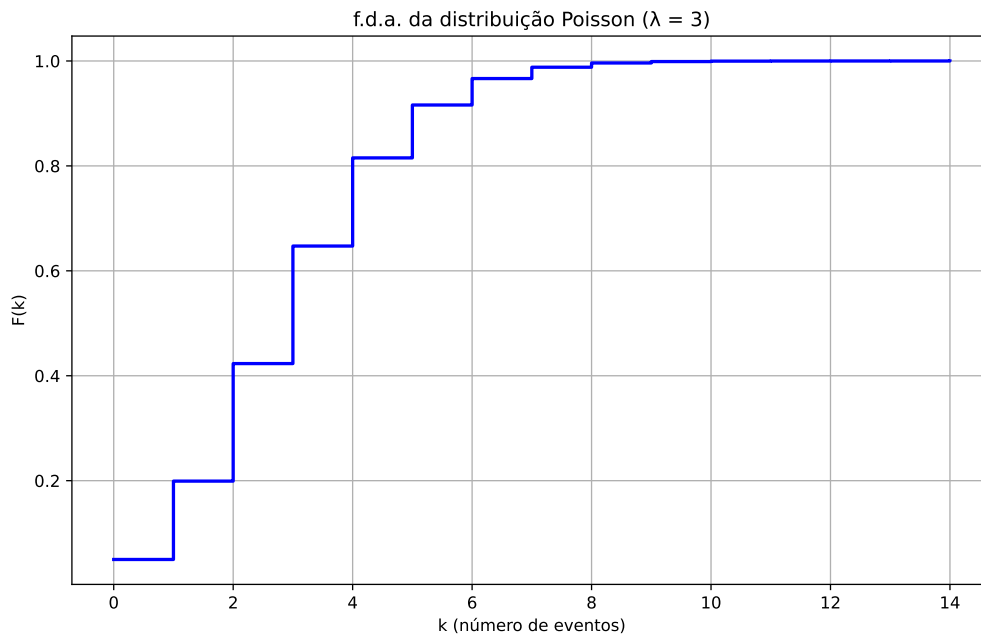
```

    cdf += p # acumula  $P(X = i)$ 
    if i < k:
        p = p * lam / (i + 1) # atualiza para o próximo valor
    return cdf

# Gerando valores de k para a f.d.a.
k_values = np.arange(0, 15)
cdf_values = [cdf_poisson(k, lam) for k in k_values]

# Plotando a f.d.a. da distribuição Poisson
plt.figure(figsize=(10, 6))
plt.step(k_values, cdf_values, where='post', color='blue', linewidth=2)
plt.title('f.d.a. da distribuição Poisson (  $\lambda = 3$  )')
plt.xlabel('k (número de eventos)')
plt.ylabel('F(k)')
plt.grid(True)
plt.show()

```



31.1 Exemplo 4: permutações e amostragem sem reposição

Nos três exemplos anteriores, cada chamada do algoritmo devolvia um número, e as chamadas eram independentes umas das outras. Este exemplo é diferente: o objeto sorteado é uma **permutação** — uma reordenação de n elementos, sorteada uniformemente entre as $n!$ possíveis — e, a partir dela, uma **amostra sem reposição**.

Essas duas operações aparecem o tempo todo: ao separar um conjunto de dados em treino e teste (como fizemos no Capítulo 1 com os dados `iris`), ao sortear a ordem em que os tratamentos são aplicados em um experimento, ou ao sortear quem será entrevistado em uma pesquisa. Em todos os casos, a exigência é a mesma: nenhum arranjo pode ser mais provável que outro.

31.1.1 O ingrediente básico: a uniforme discreta

O bloco de construção é a distribuição uniforme sobre $\{1, 2, \dots, k\}$, isto é, $\mathbb{P}(X = i) = 1/k$ para todo i . Aqui a inversão fica especialmente simples, porque a f.d.a. é $F(i) = i/k$: o menor i com $i/k \geq U$ é

$$X = \lceil kU \rceil,$$

onde $\lceil \cdot \rceil$ é o **teto**, o menor inteiro maior ou igual ao argumento. Diferentemente dos exemplos anteriores, não é preciso percorrer valor por valor: uma conta resolve.

i Por que $\lceil kU \rceil$ e não $\lfloor kU \rfloor$

Se usássemos o piso $\lfloor kU \rfloor$, obteríamos valores em $\{0, 1, \dots, k-1\}$ — o que também é uniforme, mas sobre o conjunto errado. A forma equivalente para começar em 1 é $\lfloor kU \rfloor + 1$. Note que $\lceil kU \rceil$ só difere dessa segunda expressão quando kU é exatamente um inteiro, o que tem probabilidade zero.

31.1.2 Embaralhamento de Fisher–Yates

Uma ideia que **não** funciona é sortear uma posição para cada elemento de forma independente: dois elementos podem cair na mesma posição. O algoritmo de Fisher–Yates resolve isso decidindo as posições uma de cada vez, sempre sorteadando entre os elementos que ainda não foram fixados.

💡 Pseudo-algoritmo: embaralhamento de Fisher–Yates

Entrada: um vetor $v = (v_1, \dots, v_n)$.

1. Para $i = n, n-1, \dots, 2$:
 - a. Sorteie j uniformemente em $\{1, 2, \dots, i\}$, isto é, gere $U \sim \text{Unif}(0, 1)$ e faça $j = \lceil iU \rceil$;
 - b. Troque v_i e v_j de lugar.
2. Devolva v .

O laço fixa a última posição primeiro: no passo $i = n$, o elemento que vai ocupar a posição n é sorteado uniformemente entre os n disponíveis. No passo seguinte, a posição $n-1$ recebe um dos $n-1$ elementos restantes, e assim por diante.

❗ Proposição

O algoritmo acima devolve cada uma das $n!$ permutações de v com probabilidade $1/n!$.

i Demonstração

O algoritmo faz $n-1$ sorteios independentes: no passo i , o valor de j tem i possibilidades igualmente prováveis. O número de sequências de sorteios $(j_n, j_{n-1}, \dots, j_2)$ é, portanto,

$$n \times (n-1) \times \dots \times 2 = n!,$$

e todas elas têm a mesma probabilidade $1/n!$, por independência.

Falta ver que sequências de sorteios diferentes produzem permutações diferentes. Isso vale porque o algoritmo pode ser desfeito: dada a permutação final, a última troca (a do passo $i = 2$) pode ser revertida, revelando j_2 ; depois a do passo $i = 3$, revelando j_3 ; e assim por diante. Ou seja, a permutação final determina a sequência de sorteios.

Temos então $n!$ sequências de sorteios, todas com a mesma probabilidade, em correspondência um a um com as $n!$ permutações possíveis. Logo cada permutação tem probabilidade $1/n!$. $\square$

O código abaixo implementa o pseudo-algoritmo e o verifica no menor caso em que a verificação é possível: com $n = 3$ existem $3! = 6$ permutações, e podemos contar quantas vezes cada uma aparece em 6000 embaralhamentos. O esperado é $1/6 \approx 0,1667$ para cada.

32 R

```
library(ggplot2)

set.seed(42)

embaralhar <- function(v) {
  n <- length(v)
  # O laço vai do fim para o começo: primeiro decidimos quem fica na última
  # posição, depois na penúltima, e assim por diante
  for (i in n:2) {
    # Sorteio uniforme em {1, ..., i}, pela inversão da uniforme discreta
    j <- ceiling(i * runif(1))
    # Troca de v[i] com v[j]; a variável temp guarda o valor que seria perdido
    temp <- v[i]
    v[i] <- v[j]
    v[j] <- temp
  }
  return(v)
}

# Verificação: as 6 permutações de (1, 2, 3) devem sair com a mesma frequência
B <- 6000
permutacoes <- character(B)
for (b in 1:B) {
  # paste(..., collapse = "") transforma o vetor c(2, 3, 1) no texto "231",
  # para que possamos contar as repetições com table()
  permutacoes[b] <- paste(embaralhar(1:3), collapse = "")
}

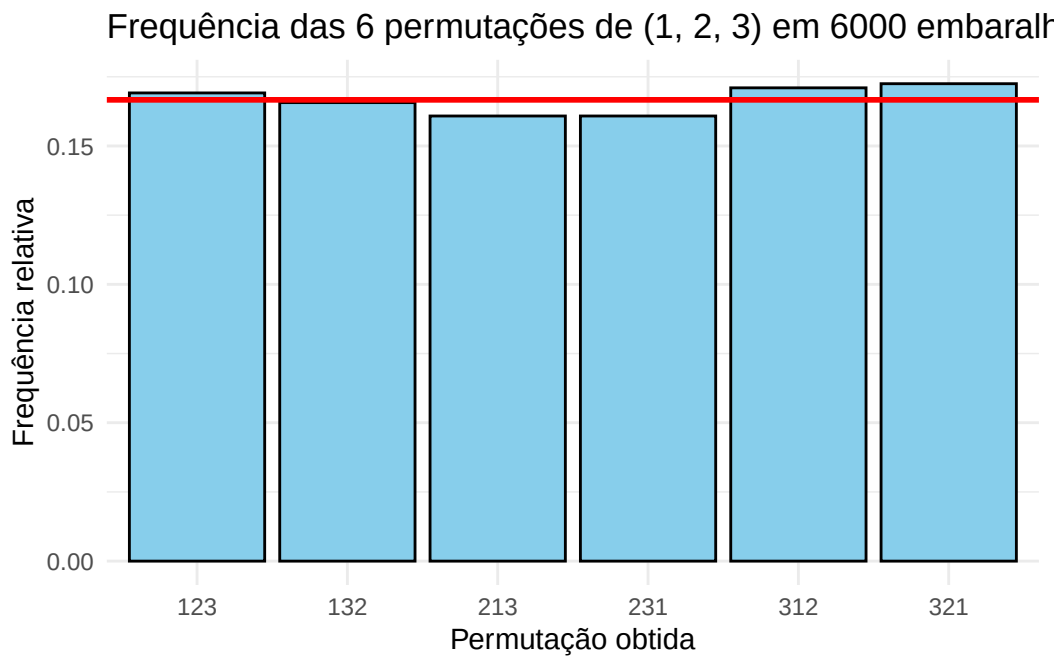
frequencias <- table(permutacoes) / B
print(round(frequencias, 4))
```

```
permutacoes
123    132    213    231    312    321
0.1692 0.1657 0.1608 0.1608 0.1710 0.1725
```



```
df <- data.frame(permutacao = names(frequencias),
                 frequencia = as.numeric(frequencias))

# A linha vermelha marca o valor teórico 1/6
ggplot(df, aes(x = permutacao, y = frequencia)) +
  geom_col(fill = "skyblue", color = "black") +
  geom_hline(yintercept = 1 / 6, color = "red", linewidth = 1) +
  labs(title = "Frequência das 6 permutações de (1, 2, 3) em 6000 embaralhamentos",
       x = "Permutação obtida", y = "Frequência relativa") +
  theme_minimal()
```



33 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def embaralhar(v):
    # Em Python, listas são passadas por referência: sem esta cópia, a função
    # alteraria a lista original de quem a chamou
    v = list(v)
    n = len(v)
    # O laço vai do fim para o começo: primeiro decidimos quem fica na última
    # posição, depois na penúltima, e assim por diante
    for i in range(n, 1, -1):
        # Sorteio uniforme em {1, ..., i}, pela inversão da uniforme discreta
        j = int(np.ceil(i * np.random.uniform()))
        # Troca de v[i] com v[j]. Atenção: as posições da lista começam em 0,
        # por isso subtraímos 1 dos índices
        v[i - 1], v[j - 1] = v[j - 1], v[i - 1]
    return v

# Verificação: as 6 permutações de (1, 2, 3) devem sair com a mesma frequência
B = 6000
contagem = {}
for b in range(B):
    # A permutação [2, 3, 1] vira o texto "231", para podermos contar repetições
    chave = "".join(str(x) for x in embaralhar([1, 2, 3]))
    contagem[chave] = contagem.get(chave, 0) + 1

permutacoes = sorted(contagem)
frequencias = [contagem[p] / B for p in permutacoes]

for p, f in zip(permutacoes, frequencias):
    print(p, round(f, 4))
```

```

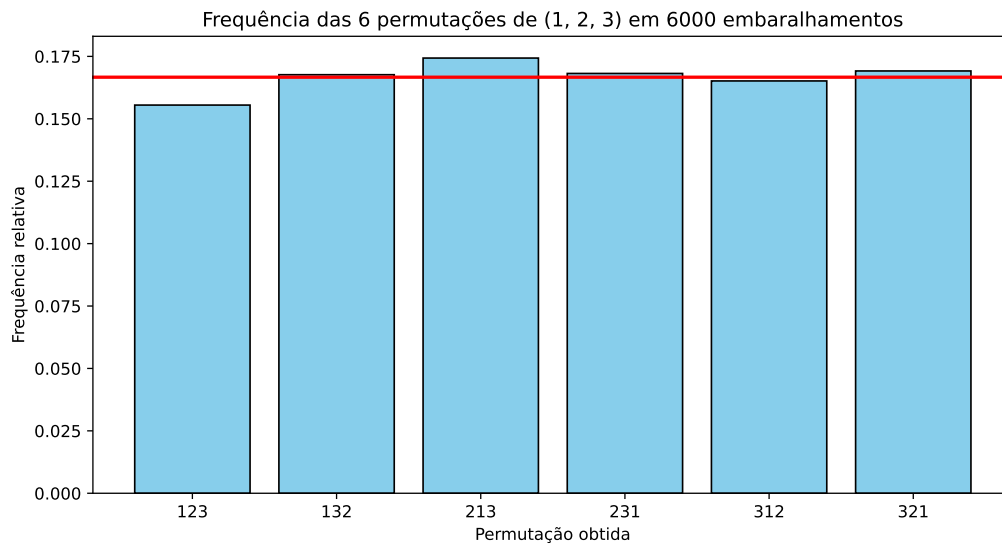
123 0.1555
132 0.1677
213 0.1743
231 0.1682
312 0.1652
321 0.1692

```

```

# A linha vermelha marca o valor teórico 1/6
plt.figure(figsize=(10, 5))
plt.bar(permutacoes, frequencias, color='skyblue', edgecolor='black')
plt.axhline(1 / 6, color='red', linewidth=2)
plt.title('Frequência das 6 permutações de (1, 2, 3) em 6000 embaralhamentos')
plt.xlabel('Permutação obtida')
plt.ylabel('Frequência relativa')
plt.show()

```



33.0.1 Amostragem sem reposição

Sortear k elementos entre n , sem repetir nenhum, é o mesmo algoritmo interrompido no meio. Depois do primeiro passo, a posição n guarda um elemento sorteado uniformemente entre os n ; depois do segundo, as posições n e $n - 1$ guardam dois elementos distintos; e assim por diante. Basta então rodar o laço k vezes e devolver as k últimas posições.

💡 Pseudo-algoritmo: amostra de tamanho k sem reposição

Entrada: um vetor $v = (v_1, \dots, v_n)$ e o tamanho $k \leq n$ da amostra.

1. Para $i = n, n-1, \dots, n-k+1$:
 - a. Sorteie j uniformemente em $\{1, \dots, i\}$;
 - b. Troque v_i e v_j de lugar.
2. Devolva $(v_{n-k+1}, \dots, v_n)$.

Para verificar, sorteamos 10 000 amostras de tamanho $k = 3$ entre $n = 10$ elementos e contamos com que frequência cada elemento aparece. Como todos os elementos têm a mesma chance de entrar na amostra, essa frequência deve ficar próxima de $k/n = 0,3$.

34 R

```
set.seed(42)

amostrar_sem_reposicao <- function(v, k) {
  n <- length(v)
  # Mesmo laço do embaralhamento, mas parando depois de k passos
  for (i in n:(n - k + 1)) {
    j <- ceiling(i * runif(1))
    temp <- v[i]
    v[i] <- v[j]
    v[j] <- temp
  }
  # As k últimas posições são a amostra
  return(v[(n - k + 1):n])
}

cat("Uma amostra de tamanho 3 entre 10 elementos:",
    amostrar_sem_reposicao(1:10, 3), "\n")
```

Uma amostra de tamanho 3 entre 10 elementos: 3 9 10

```
# Verificação: com que frequência cada elemento entra na amostra?
n <- 10
k <- 3
B <- 10000
contagem <- numeric(n)

for (b in 1:B) {
  amostra <- amostrar_sem_reposicao(1:n, k)
  contagem[amostra] <- contagem[amostra] + 1
}

cat("Proporção das amostras que contêm cada elemento (esperado k/n = 0,3):\n")
```

Proporção das amostras que contêm cada elemento (esperado $k/n = 0,3$):

```
print(round(contagem / B, 3))
```

```
[1] 0.306 0.307 0.291 0.304 0.298 0.304 0.292 0.301 0.296 0.298
```

35 Python

```
np.random.seed(42)

def amostrar_sem_reposicao(v, k):
    v = list(v)
    n = len(v)
    # Mesmo laço do embaralhamento, mas parando depois de k passos
    for i in range(n, n - k, -1):
        j = int(np.ceil(i * np.random.uniform()))
        v[i - 1], v[j - 1] = v[j - 1], v[i - 1]
    # As k últimas posições são a amostra
    return v[n - k:]

print("Uma amostra de tamanho 3 entre 10 elementos:",
      amostrar_sem_reposicao(range(1, 11), 3))
```

Uma amostra de tamanho 3 entre 10 elementos: [6, 9, 4]

```
# Verificação: com que frequência cada elemento entra na amostra?
n = 10
k = 3
B = 10000
contagem = np.zeros(n)

for b in range(B):
    amostra = amostrar_sem_reposicao(range(1, n + 1), k)
    for x in amostra:
        contagem[x - 1] += 1

print("Proporção das amostras que contêm cada elemento (esperado k/n = 0,3):")
```

Proporção das amostras que contêm cada elemento (esperado k/n = 0,3):

```
print(np.round(contagem / B, 3))
```

```
[0.303 0.296 0.301 0.302 0.304 0.304 0.292 0.3    0.306 0.293]
```

 **Atenção:** com reposição é outra coisa

Sortear k valores **com** reposição — o que `sample(v, k, replace = TRUE)` e `np.random.choice(v, k)` fazem — é o problema dos exemplos anteriores, repetido k vezes de forma independente, e pode devolver o mesmo elemento mais de uma vez. Sem reposição, os sorteios **não** são independentes: a cada elemento retirado, os demais ficam mais prováveis.

As funções prontas para o caso sem reposição são `sample(v)` e `sample(v, k)` no R, e `np.random.permutation(v)` e `np.random.choice(v, k, replace=False)` no Python. Todas implementam, por dentro, alguma variante do Fisher–Yates. Foi o que usamos no Capítulo 1 para separar os dados *iris* em treino e teste — lá, a separação precisava ser sem reposição, sob pena de a mesma flor aparecer nos dois conjuntos.

35.1 Exercícios

Exercício 1. Seja X uma v.a. tal que $\mathbb{P}(X = 1) = 0.3$, $\mathbb{P}(X = 3) = 0.1$ e $\mathbb{P}(X = 4) = 0.6$.

- (a) Escreva um pseudo-algoritmo para gerar um valor de X .
- (b) Implemente uma função para gerar n valores de X .
- (c) Compare a distribuição das frequências obtidas na amostra simulada com as probabilidades reais.

Exercício 2. Considere X uma v.a. tal que

$$\mathbb{P}(X = i) = \alpha \mathbb{P}(X_1 = i) + (1 - \alpha) \mathbb{P}(X_2 = i), \quad i = 0, 1, \dots$$

onde $0 \leq \alpha \leq 1$ e X_1, X_2 são v.a. discretas.

A distribuição de X é chamada de distribuição de mistura. Podemos escrever

$$X = \begin{cases} X_1, & \text{com probabilidade } \alpha, \\ X_2, & \text{com probabilidade } 1 - \alpha. \end{cases}$$

Implemente um algoritmo para gerar uma amostra de tamanho n da distribuição mistura de uma Poisson e de uma Geométrica, com base nas funções implementadas nos Exemplos 2 e 3.

Exercício 3. Seja $X \sim \text{Binomial}(m, p)$, isto é,

$$\mathbb{P}(X = k) = \binom{m}{k} p^k (1-p)^{m-k}, \quad k = 0, 1, \dots, m.$$

- (a) Mostre que as probabilidades satisfazem a relação recursiva

$$\mathbb{P}(X = k + 1) = \frac{m - k}{k + 1} \cdot \frac{p}{1 - p} \cdot \mathbb{P}(X = k), \quad \text{com } \mathbb{P}(X = 0) = (1 - p)^m.$$

- (b) Adapte a função do Exemplo 3 para gerar uma Binomial por inversão usando essa recursão. Assim como no caso da Poisson, você não deve calcular fatoriais nem potências dentro do laço.
- (c) Gere 1000 valores com $m = 10$ e $p = 0.3$ e compare o histograma obtido com as probabilidades teóricas.
- (d) Diferentemente da Poisson, aqui o laço do algoritmo sempre para em no máximo m passos. Por quê?

Exercício 4. Este exercício explora o embaralhamento do Exemplo 4.

- (a) Uma variante natural — e errada — do Fisher–Yates é sortear j uniformemente em $\{1, \dots, n\}$ a cada passo, em vez de em $\{1, \dots, i\}$. Implemente essa versão, gere 6000 permutações de $(1, 2, 3)$ e compare as frequências das seis permutações com $1/6$. Em seguida, explique por que ela **não pode** funcionar: quantas sequências de sorteios equiprováveis o algoritmo produz quando $n = 3$? Por que esse número não ser divisível por 6 já garante que alguma permutação sai com frequência diferente das outras?
- (b) Um **desarranjo** é uma permutação que não deixa nenhum elemento na posição original. Usando a função `embaralhar`, estime a probabilidade de que uma permutação aleatória de $(1, \dots, 10)$ seja um desarranjo e compare com o valor teórico, que é próximo de $1/e \approx 0,3679$. Repita com $n = 5$ e $n = 50$: a probabilidade muda muito com n ?
- (c) Separe os índices $1, \dots, 150$ em um conjunto de treino com 70% dos elementos e um de teste com os 30% restantes, usando `amostrar_sem_reposicao` e sem chamar `sample` nem `np.random.choice`. Confira que os dois conjuntos não têm elementos em comum e que juntos somam 150 índices.

- (d) Quantos números uniformes **embaralhar** consome para um vetor de tamanho n ? E **amostrar_sem_reposicao**, para uma amostra de tamanho k ? Compare com a alternativa ingênua de sortear k valores **com** reposição e recomeçar do zero sempre que houver repetição: para $n = 365$ e $k = 23$ (o problema do aniversário), estime por simulação a probabilidade de haver repetição em uma tentativa e o número médio de tentativas até obter 23 valores distintos.

Exercício 5. (Desafio) No algoritmo geral da inversão, os valores $x_1, x_2, \dots, x_m$ são percorridos em ordem até que $F(x_i) \geq U$. O número de comparações feitas até devolver x_i é, portanto, igual a i .

- (a) Mostre que o número esperado de comparações do algoritmo é

$$\sum_{i=1}^m i \cdot p(x_i).$$

- (b) Note que a soma acima depende da **ordem** em que listamos os valores, embora a distribuição gerada não dependa. Argumente que essa soma é mínima quando os valores estão listados em ordem decrescente de probabilidade.
- (c) Considere uma v.a. com probabilidades (0.05, 0.05, 0.1, 0.2, 0.6), nessa ordem. Modifique a função que você implementou no Exercício 1 para também contar quantas comparações foram feitas em cada geração. Gere 20000 valores com as probabilidades nessa ordem e depois em ordem decrescente, e compare a média empírica de comparações com o valor previsto pelo item (a).

Exercício 6. (Desafio) O **método do alias** gera uma v.a. discreta com m valores fazendo sempre o mesmo trabalho, qualquer que seja m — em vez do número crescente de comparações do Exercício 5. A ideia é escrever a distribuição como uma mistura de m distribuições, cada uma concentrada em no máximo **dois** pontos.

O gerador guarda dois vetores, calculados uma única vez: as probabilidades de corte $q_1, \dots, q_m$ e os *apelidos* (aliases) $a_1, \dots, a_m$. O algoritmo é:

1. Sorteie i uniformemente em $\{1, \dots, m\}$;
2. Gere $U \sim \text{Unif}(0, 1)$. Se $U \leq q_i$, devolva x_i ; caso contrário, devolva x_{a_i} .

- (a) Considere X com valores 1, 2, 3, 4 e probabilidades (0,1; 0,2; 0,3; 0,4), e a tabela

$$q = (0,4; 0,8; 0,6; 1,0), \quad a = (3, 4, 4, 4).$$

Verifique, enumerando os caminhos que levam a cada valor, que o algoritmo devolve exatamente essas probabilidades. Por exemplo, o valor 3 pode sair da coluna $i = 3$ (quando $U \leq 0,6$) ou da coluna $i = 1$ (quando $U > 0,4$, pois $a_1 = 3$), o que dá $\frac{1}{4}(0,6) + \frac{1}{4}(1 - 0,4) = 0,15 + 0,15 = 0,3$.

- (b) Implemente o algoritmo com essa tabela, gere 20 000 valores e compare as frequências obtidas com as probabilidades verdadeiras.
- (c) Explique por que o custo de gerar um valor não depende de m , e compare com o número esperado de comparações calculado no Exercício 5.
- (d) Falta construir a tabela. Multiplique todas as probabilidades por m , de modo que a média passe a ser 1. Mostre que, se nem todas forem iguais a 1, sempre existem um índice i com $m p_i \leq 1$ e um índice j com $m p_j \geq 1$. Faça então $q_i = m p_i$ e $a_i = j$: a coluna i fica completa, e o que faltava para enchê-la ($1 - q_i$) é descontado de $m p_j$. Repita o procedimento com as $m - 1$ colunas restantes. Implemente essa construção e verifique que ela reproduz a tabela do item (a).

36 Técnica da Inversão para Variáveis Contínuas

No capítulo anterior usamos a f.d.a. para transformar um número $\text{Unif}(0, 1)$ em uma v.a. discreta. A ideia agora é exatamente a mesma, mas o caso contínuo é até mais simples: como não há saltos na f.d.a., não precisamos percorrer valores acumulando probabilidades — quando conseguimos inverter F explicitamente, cada valor gerado sai de uma única conta.

Lembre que uma v.a. X é **contínua** quando sua função de distribuição acumulada (f.d.a.) pode ser escrita como

$$\mathbb{P}(X \leq a) = F(a) = \int_{-\infty}^a f(x) dx, \quad \forall a \in \mathbb{R},$$

em que $f : \mathbb{R} \rightarrow [0, \infty)$ é uma função integrável, chamada de função densidade de probabilidade.

36.1 A inversa da f.d.a. no caso contínuo

Quando X é contínua, F é uma função contínua: ela não dá saltos. Se, além disso, F for **estritamente crescente** no intervalo onde a densidade é positiva, então para cada $u \in (0, 1)$ existe um **único** x com $F(x) = u$. Nesse caso, F^{-1} é a função inversa no sentido usual, e a definição geral vista no capítulo anterior, $F^{-1}(u) = \inf\{x \in \mathbb{R} : F(x) \geq u\}$, coincide com ela.

 **Atenção:** contínua não quer dizer estritamente crescente

Uma f.d.a. contínua pode ser constante em um trecho: basta que a densidade seja zero ali. Por exemplo, se f é positiva em $(0, 1) \cup (2, 3)$ e nula em $[1, 2]$, então F é constante em $[1, 2]$ e não é injetora na reta toda.

Isso não atrapalha o método: F continua estritamente crescente **no suporte** de X , que é o único lugar de onde o algoritmo devolve valores. Em todos os exemplos deste capítulo o suporte é um intervalo e F é estritamente crescente nele, então podemos tratar F^{-1} como a inversa usual.

A figura a seguir ilustra a relação entre F e F^{-1} : entramos pelo eixo vertical com um valor u , caminhamos até a curva e descemos até o eixo horizontal, chegando em $F^{-1}(u)$.

37 R

```
library(ggplot2)

# f.d.a. usada como ilustração: a logística
F_ac <- function(x) {
  1 / (1 + exp(-x))
}

# Inversa da f.d.a. logística, obtida resolvendo  $u = 1 / (1 + e^{-x})$  em  $x$ 
F_inv <- function(u) {
  -log(1 / u - 1)
}

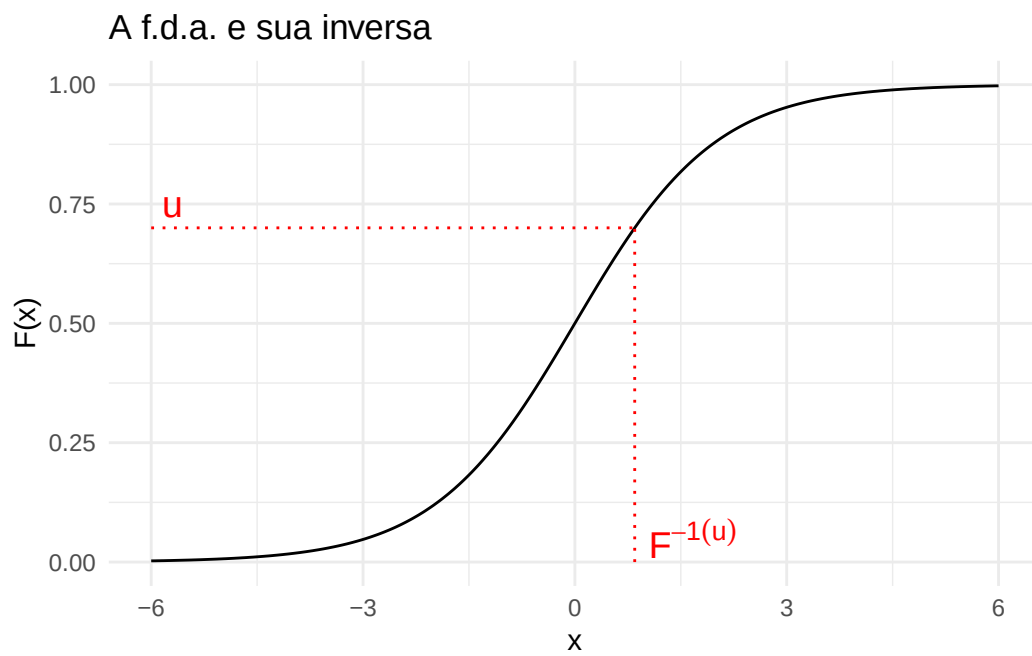
# Malha de pontos usada para desenhar a curva
x <- seq(-6, 6, length.out = 400)

# Valor de  $u$  escolhido para ilustrar o caminho  $u \rightarrow F^{-1}(u)$ 
u_valor <- 0.7
x_valor <- F_inv(u_valor)

df <- data.frame(x = x, F_x = F_ac(x))

ggplot(df, aes(x = x, y = F_x)) +
  geom_line(color = "black") +
  # Segmento horizontal: entramos com  $u$  pelo eixo vertical até tocar a curva
  annotate("segment", x = -6, xend = x_valor, y = u_valor, yend = u_valor,
    linetype = "dotted", color = "red") +
  # Segmento vertical: descemos da curva até o eixo horizontal
  annotate("segment", x = x_valor, xend = x_valor, y = 0, yend = u_valor,
    linetype = "dotted", color = "red") +
  annotate("text", x = x_valor + 0.2, y = 0.05, label = "F-1(u)",
    parse = TRUE, color = "red", hjust = 0, size = 5) +
  annotate("text", x = -5.7, y = u_valor + 0.05, label = "u",
    color = "red", size = 5) +
  labs(title = "A f.d.a. e sua inversa", x = "x", y = "F(x)") +
```

```
ylim(0, 1) +  
theme_minimal()
```



38 Python

```
import numpy as np
import matplotlib.pyplot as plt

# f.d.a. usada como ilustração: a logística
def F_ac(x):
    return 1 / (1 + np.exp(-x))

# Inversa da f.d.a. logística, obtida resolvendo  $u = 1 / (1 + e^{-x})$  em  $x$ 
def F_inv(u):
    return -np.log(1 / u - 1)

# Malha de pontos usada para desenhar a curva
x = np.linspace(-6, 6, 400)

# Valor de  $u$  escolhido para ilustrar o caminho  $u \rightarrow F^{-1}(u)$ 
u_valor = 0.7
x_valor = F_inv(u_valor)

plt.figure(figsize=(8, 6))
plt.plot(x, F_ac(x), color="black")

# Segmento horizontal: entramos com  $u$  pelo eixo vertical até tocar a curva
plt.hlines(u_valor, -6, x_valor, linestyle="dotted", colors="red")
# Segmento vertical: descemos da curva até o eixo horizontal
plt.vlines(x_valor, 0, u_valor, linestyle="dotted", colors="red")

plt.text(x_valor + 0.2, 0.05, r" $F^{-1}(u)$ ", fontsize=14, color="red")
plt.text(-5.7, u_valor + 0.03, r" $u$ ", fontsize=14, color="red")

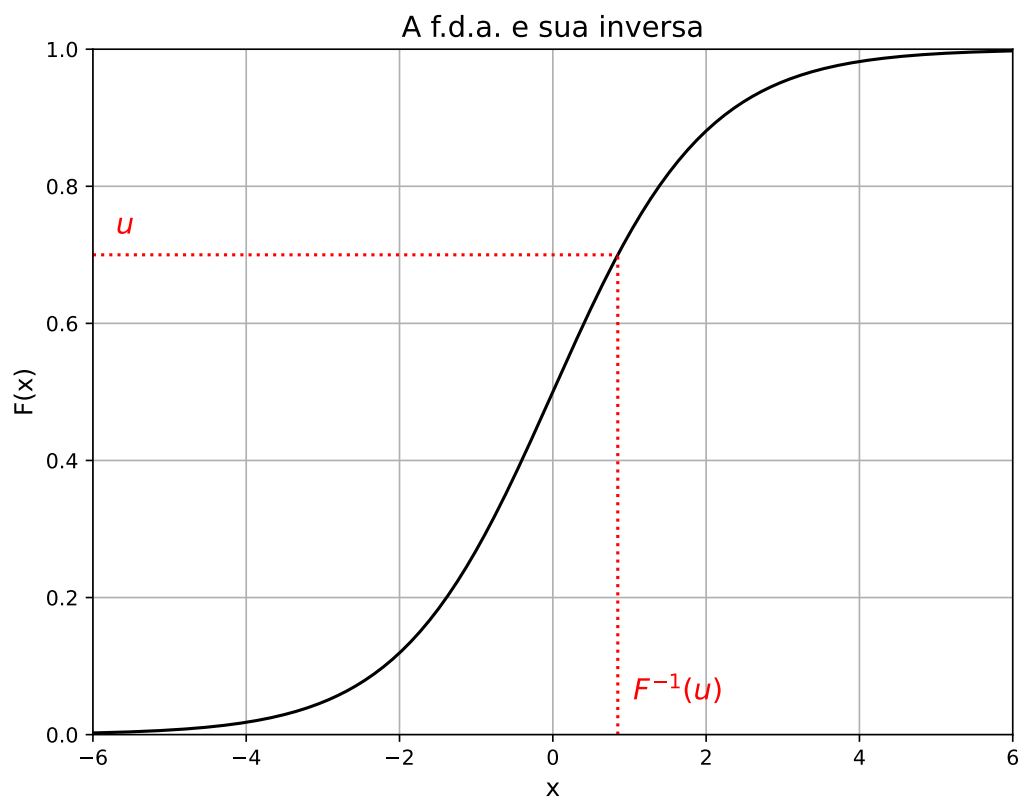
plt.title("A f.d.a. e sua inversa", fontsize=14)
plt.xlabel("x", fontsize=12)
plt.ylabel("F(x)", fontsize=12)
plt.ylim(0, 1)
```


(0.0, 1.0)

```
plt.xlim(-6, 6)
```

(-6.0, 6.0)

```
plt.grid(True)  
plt.show()
```



38.1 Método da Inversão

💡 Pseudo-algoritmo: inversão para v.a. contínuas

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Retorne $X = F^{-1}(U)$.

Compare com o caso discreto: lá o passo 2 era uma busca (percorrer os valores até que a acumulada alcançasse U); aqui ele é uma fórmula. Em compensação, precisamos conseguir escrever F^{-1} explicitamente — voltaremos a esse ponto no fim da seção.

! Proposição

Seja F a f.d.a. de uma v.a. contínua, estritamente crescente em seu suporte, e seja F^{-1} sua inversa. Se $U \sim \text{Unif}(0, 1)$, então

$$X = F^{-1}(U)$$

tem f.d.a. F .

i Demonstração

Fixe $x \in \mathbb{R}$. Como F é crescente, aplicar F aos dois lados de uma desigualdade preserva o seu sentido, e portanto

$$F^{-1}(U) \leq x \iff F(F^{-1}(U)) \leq F(x) \iff U \leq F(x),$$

onde na última equivalência usamos que $F(F^{-1}(u)) = u$. Os dois eventos são o mesmo, logo têm a mesma probabilidade:

$$\mathbb{P}(X \leq x) = \mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)).$$

Falta calcular essa última probabilidade. Como $U \sim \text{Unif}(0, 1)$, sua f.d.a. é $\mathbb{P}(U \leq u) = u$ para todo $u \in [0, 1]$. E $F(x)$ é um número em $[0, 1]$, por ser uma probabilidade. Portanto,

$$\mathbb{P}(U \leq F(x)) = F(x).$$

Ou seja, $\mathbb{P}(X \leq x) = F(x)$ para todo x , que é exatamente o que queríamos. $\square$

Vale a pena entender também **por que** o método funciona, e não só verificar a conta. A f.d.a. transforma “quanta probabilidade existe” em “até onde vamos no eixo x ”: um pedaço de comprimento p do eixo vertical é levado pela inversa em uma região do eixo x que tem probabilidade exatamente p . Onde a densidade é alta, F sobe rápido, e um intervalo curto de

x corresponde a um intervalo longo de u — por isso muitos dos U sorteados caem ali e são convertidos em valores dessa região.

38.2 Exemplo 1: uma potência

Seja X uma v.a. com f.d.a.

$$F(x) = x^n, \quad \text{para } 0 < x < 1,$$

em que n é um inteiro positivo conhecido. A densidade correspondente é $f(x) = F'(x) = nx^{n-1}$, para $0 < x < 1$.

Para obter a inversa, escrevemos $u = F(x)$ e isolamos x :

$$u = x^n \implies x = u^{1/n}.$$

Pseudo-algoritmo

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Retorne $X = U^{1/n}$.

No código abaixo geramos $B = 1000$ valores e comparamos o histograma com a densidade $f(x) = nx^{n-1}$, em vermelho. Usamos a letra B para o número de valores simulados, reservando n para o parâmetro da distribuição.

39 R

```
library(ggplot2)

set.seed(42)

n <- 3      # expoente da f.d.a.  $F(x) = x^n$ 
B <- 1000   # quantos valores queremos gerar

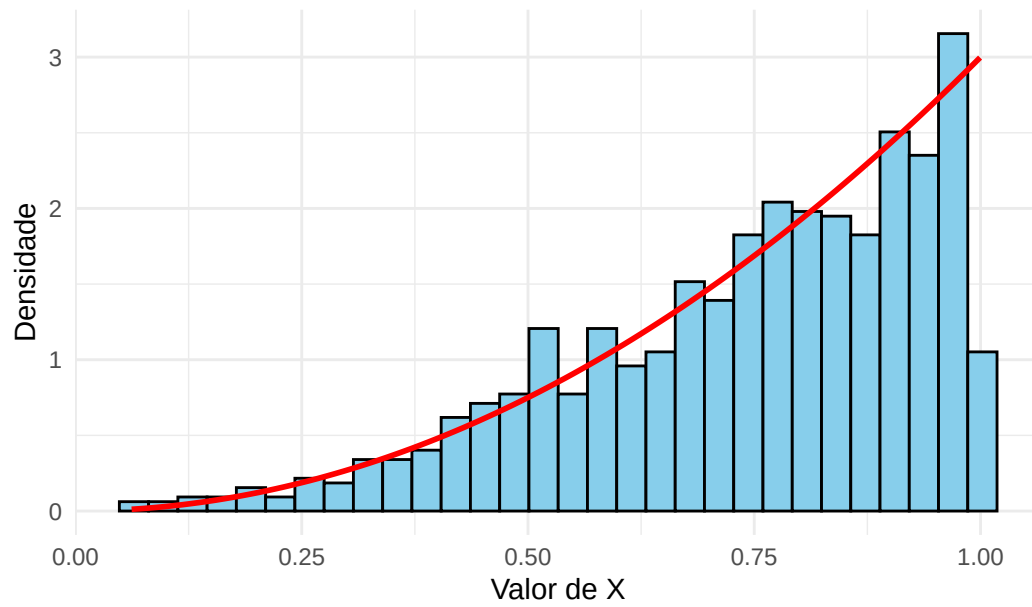
# Passo 1: gerar os uniformes
U <- runif(B, min = 0, max = 1)

# Passo 2: aplicar a inversa da f.d.a.,  $F^{-1}(u) = u^{1/n}$ 
X <- U^(1 / n)

df <- data.frame(X = X)

# O histograma usa a escala de densidade (e não de contagem) para poder ser
# comparado com a densidade teórica, desenhada por stat_function
ggplot(df, aes(x = X)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "skyblue", color = "black") +
  stat_function(fun = function(x) n * x^(n - 1), color = "red", linewidth = 1) +
  labs(title = "Inversão para  $F(x) = x^n$ , com  $n = 3$ ",
       x = "Valor de X", y = "Densidade") +
  theme_minimal()
```

Inversão para $F(x) = x^n$, com $n = 3$



40 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

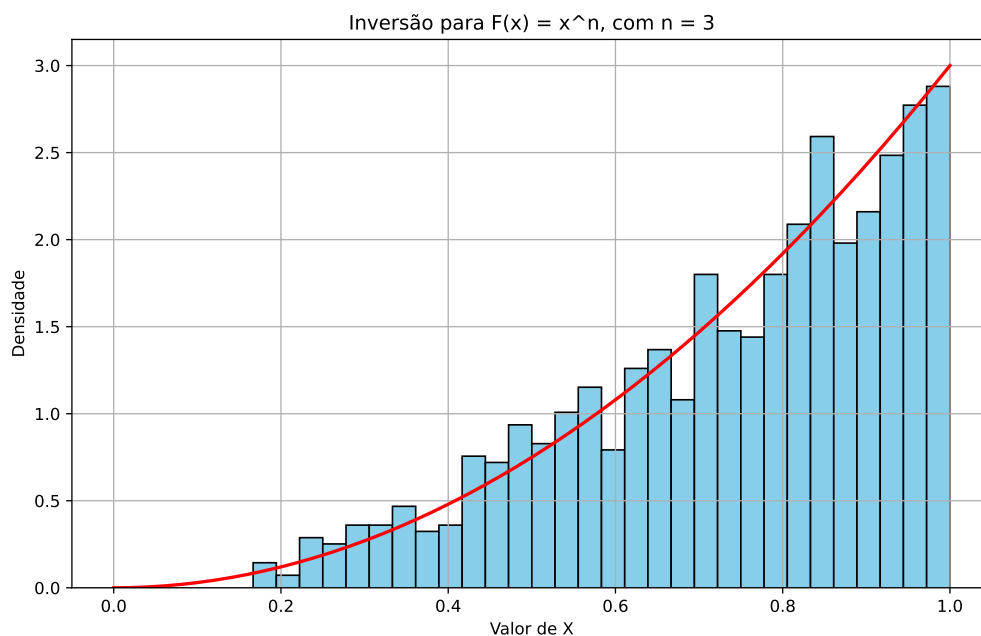
n = 3      # expoente da f.d.a.  $F(x) = x^n$ 
B = 1000   # quantos valores queremos gerar

# Passo 1: gerar os uniformes
U = np.random.uniform(0, 1, B)

# Passo 2: aplicar a inversa da f.d.a.,  $F^{-1}(u) = u^{1/n}$ 
X = U**(1 / n)

# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, 1, 200)

# O histograma usa a escala de densidade (density=True) para poder ser
# comparado com a densidade teórica
plt.figure(figsize=(10, 6))
plt.hist(X, bins=30, color='skyblue', edgecolor='black', density=True)
plt.plot(grade, n * grade**(n - 1), color='red', linewidth=2)
plt.title('Inversão para  $F(x) = x^n$ , com  $n = 3$ ')
plt.xlabel('Valor de X')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



40.1 Exemplo 2: distribuição exponencial

Seja $X \sim \text{Exp}(\lambda)$, cuja f.d.a. é

$$F(x) = 1 - e^{-\lambda x}, \quad \text{para } x > 0.$$

Novamente escrevemos $u = F(x)$ e isolamos x :

$$u = 1 - e^{-\lambda x} \implies e^{-\lambda x} = 1 - u \implies -\lambda x = \log(1 - u) \implies x = -\frac{\log(1 - u)}{\lambda}.$$

💡 Pseudo-algoritmo: exponencial

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Retorne $X = -\frac{\log(1 - U)}{\lambda}$.

41 R

```
library(ggplot2)

set.seed(42)

lambda <- 2 # parâmetro da distribuição exponencial
B <- 1000   # quantos valores queremos gerar

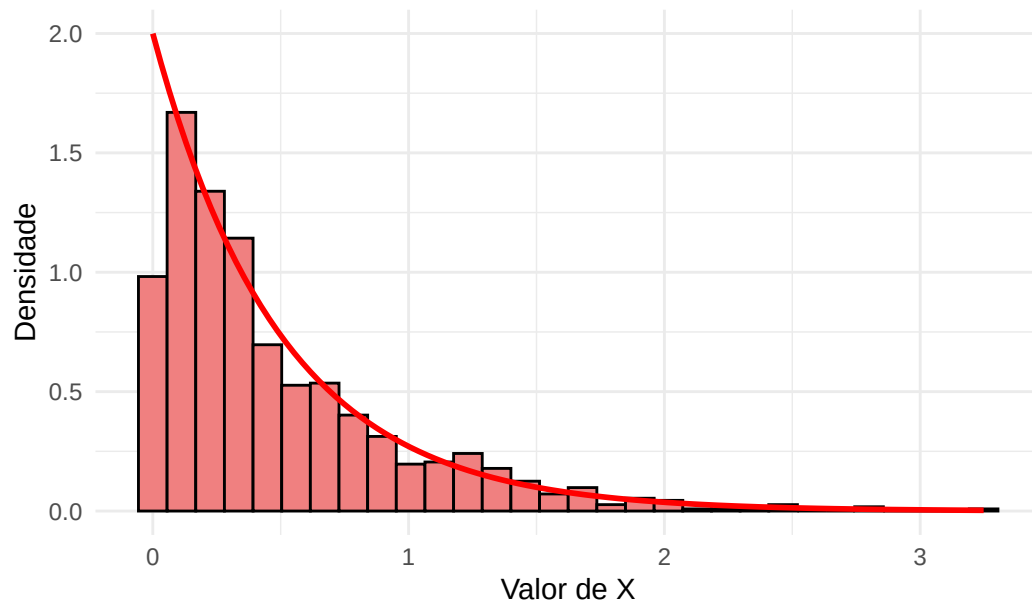
# Passo 1: gerar os uniformes
U <- runif(B, min = 0, max = 1)

# Passo 2: aplicar a inversa da f.d.a. da exponencial
X <- -log(1 - U) / lambda

df <- data.frame(X = X)

# Comparação com a densidade teórica  $f(x) = \lambda * e^{-\lambda x}$ 
ggplot(df, aes(x = X)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "lightcoral", color = "black") +
  stat_function(fun = function(x) lambda * exp(-lambda * x),
               color = "red", linewidth = 1) +
  labs(title = "Inversão para a distribuição Exponencial (lambda = 2)",
       x = "Valor de X", y = "Densidade") +
  theme_minimal()
```


Inversão para a distribuição Exponencial (lambda = 2)



42 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

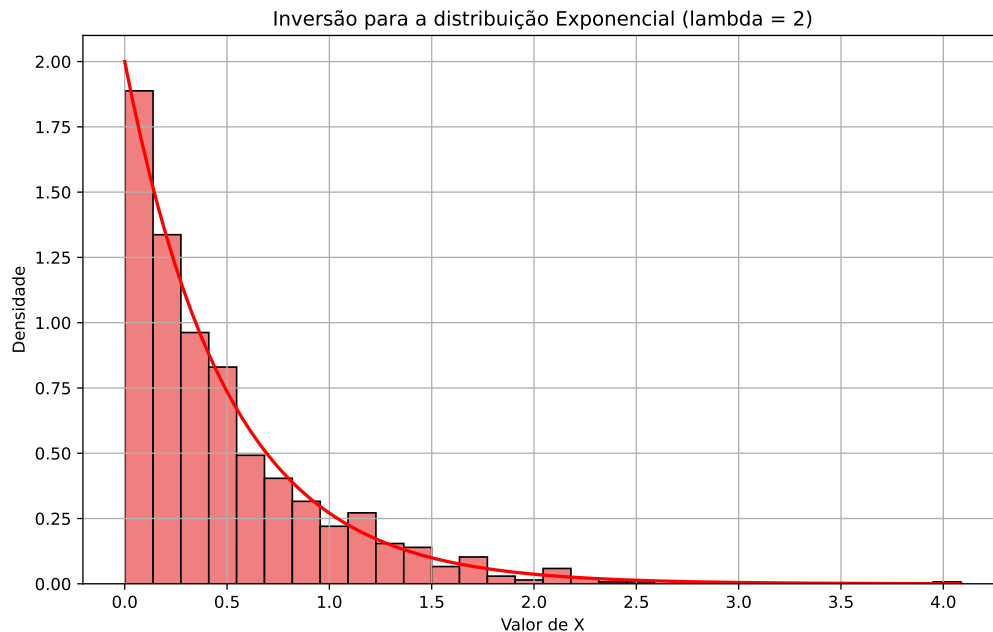
lamdb = 2 # parâmetro da distribuição exponencial
B = 1000 # quantos valores queremos gerar

# Passo 1: gerar os uniformes
U = np.random.uniform(0, 1, B)

# Passo 2: aplicar a inversa da f.d.a. da exponencial
X = -np.log(1 - U) / lamdb

# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, max(X), 200)

# Comparação com a densidade teórica  $f(x) = \lambda * e^{-\lambda x}$ 
plt.figure(figsize=(10, 6))
plt.hist(X, bins=30, color='lightcoral', edgecolor='black', density=True)
plt.plot(grade, lamdb * np.exp(-lamdb * grade), color='red', linewidth=2)
plt.title('Inversão para a distribuição Exponencial (lambda = 2)')
plt.xlabel('Valor de X')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



i Uma simplificação comum

Se $U \sim \text{Unif}(0, 1)$, então $1 - U$ também tem distribuição $\text{Unif}(0, 1)$. Por isso muitos textos escrevem o algoritmo da exponencial como $X = -\log(U)/\lambda$: a distribuição gerada é a mesma. Neste livro mantemos o $1 - U$, que é o que sai diretamente da inversão de F .

⚠ Atenção: nem sempre dá para inverter F na mão

Os dois exemplos acima têm f.d.a. que conseguimos inverter com álgebra simples. Isso é a exceção, não a regra: para a distribuição normal, por exemplo, nem mesmo F tem fórmula fechada, quanto mais F^{-1} . Nesses casos, ou usamos uma aproximação numérica de F^{-1} — como no Exemplo 4 —, ou trocamos de método — é o que faremos nos capítulos sobre o método da rejeição e sobre o algoritmo de Box-Muller.

42.1 Exemplo 3: distribuições truncadas

É comum precisarmos de uma variável restrita a um intervalo: o tempo de espera de quem já esperou pelo menos um minuto, o salário de quem ganha acima de um piso, a temperatura de um dia em que ela ficou entre dois valores. A distribuição correspondente é a distribuição **truncada**.

Restringir não é simplesmente ignorar o que está fora do intervalo: a densidade precisa ser reescalada, para que continue integrando 1 na região que sobrou.

Definição: distribuição truncada

Seja X uma v.a. contínua com densidade f e f.d.a. F , e seja (a, b) um intervalo com $F(b) > F(a)$. A distribuição de X **truncada** a (a, b) é a distribuição condicional de X dado $a < X < b$; sua densidade é

$$f_{a,b}(x) = \frac{f(x)}{F(b) - F(a)}, \quad a < x < b,$$

e sua f.d.a. é

$$F_{a,b}(x) = \frac{F(x) - F(a)}{F(b) - F(a)}, \quad a < x < b.$$

O denominador $F(b) - F(a) = \mathbb{P}(a < X < b)$ é a probabilidade que “sobra” depois do truncamento. Dividir por ele é o que devolve à densidade a área total igual a 1.

O ponto importante é que, se sabemos inverter F , sabemos inverter $F_{a,b}$ também — sem nenhum trabalho novo.

Proposição

Nas condições acima,

$$F_{a,b}^{-1}(u) = F^{-1}\left(F(a) + u[F(b) - F(a)]\right), \quad 0 < u < 1.$$

Demonstração

Basta resolver $u = F_{a,b}(x)$ em x :

$$u = \frac{F(x) - F(a)}{F(b) - F(a)} \implies F(x) = F(a) + u[F(b) - F(a)] \implies x = F^{-1}\left(F(a) + u[F(b) - F(a)]\right),$$

onde na última passagem aplicamos F^{-1} aos dois lados. $\square$

Pseudo-algoritmo: distribuição truncada a (a, b)

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Calcule $V = F(a) + U[F(b) - F(a)]$.

3. Retorne $X = F^{-1}(V)$.

Vale a pena ler o passo 2 geometricamente: V é uma uniforme no intervalo $(F(a), F(b))$, isto é, um sorteio da “altura” **dentro da faixa** da f.d.a. que corresponde a (a, b) . O passo 3 então converte essa altura em um valor de x , exatamente como na inversão comum. A diferença é só que agora sorteamos a altura em uma faixa, e não em $(0, 1)$ inteiro.

Vamos aplicar isso a uma $\text{Exp}(1)$ truncada ao intervalo $(0, 5; 2)$. Como $F(x) = 1 - e^{-x}$ e $F^{-1}(u) = -\log(1 - u)$, os dois passos são contas diretas.

43 R

```
library(ggplot2)

set.seed(42)

lambda <- 1      # parâmetro da exponencial original
a <- 0.5         # extremo inferior do truncamento
b <- 2           # extremo superior do truncamento
B <- 5000        # quantos valores queremos gerar

# f.d.a. da exponencial e sua inversa, do Exemplo 2
F_exp <- function(x) 1 - exp(-lambda * x)
F_inv_exp <- function(u) -log(1 - u) / lambda

# Passo 1: gerar os uniformes
U <- runif(B, min = 0, max = 1)

# Passo 2: levar U para a faixa (F(a), F(b)) da f.d.a.
V <- F_exp(a) + U * (F_exp(b) - F_exp(a))

# Passo 3: aplicar a inversa da f.d.a. original
X <- F_inv_exp(V)

cat("Menor valor gerado:", round(min(X), 4), "\n")
```

Menor valor gerado: 0.5002

```
cat("Maior valor gerado:", round(max(X), 4), "\n")
```

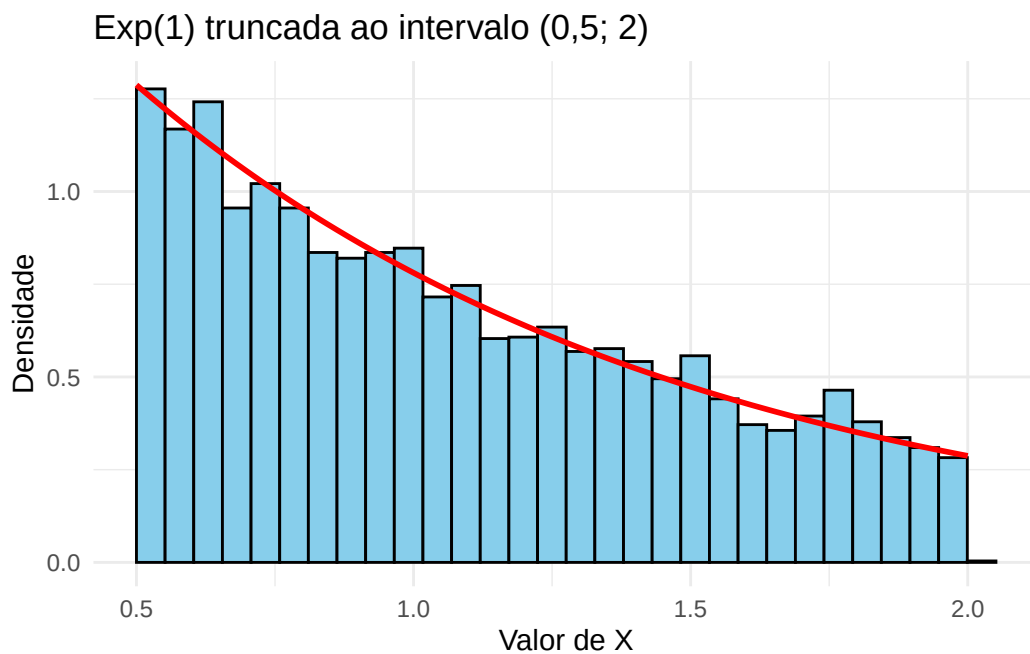
Maior valor gerado: 1.9994

```
# A constante que reescala a densidade
constante <- F_exp(b) - F_exp(a)
cat("P(a < X < b) na exponencial original:", round(constante, 4), "\n")
```

$P(a < X < b)$ na exponencial original: 0.4712

```
df <- data.frame(X = X)

# A densidade teórica é a da exponencial dividida pela constante, e existe
# apenas dentro do intervalo (a, b) - daí o argumento xlim de stat_function
ggplot(df, aes(x = X)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, boundary = a,
                 fill = "skyblue", color = "black") +
  stat_function(fun = function(x) lambda * exp(-lambda * x) / constante,
               xlim = c(a, b), color = "red", linewidth = 1) +
  labs(title = "Exp(1) truncada ao intervalo (0,5; 2)",
       x = "Valor de X", y = "Densidade") +
  theme_minimal()
```



44 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

lamdb = 1    # parâmetro da exponencial original
a = 0.5      # extremo inferior do truncamento
b = 2        # extremo superior do truncamento
B = 5000     # quantos valores queremos gerar

# f.d.a. da exponencial e sua inversa, do Exemplo 2
def F_exp(x):
    return 1 - np.exp(-lamdb * x)

def F_inv_exp(u):
    return -np.log(1 - u) / lamdb

# Passo 1: gerar os uniformes
U = np.random.uniform(0, 1, B)

# Passo 2: levar U para a faixa (F(a), F(b)) da f.d.a.
V = F_exp(a) + U * (F_exp(b) - F_exp(a))

# Passo 3: aplicar a inversa da f.d.a. original
X = F_inv_exp(V)

print("Menor valor gerado:", round(X.min(), 4))
```

Menor valor gerado: 0.5

```
print("Maior valor gerado:", round(X.max(), 4))
```

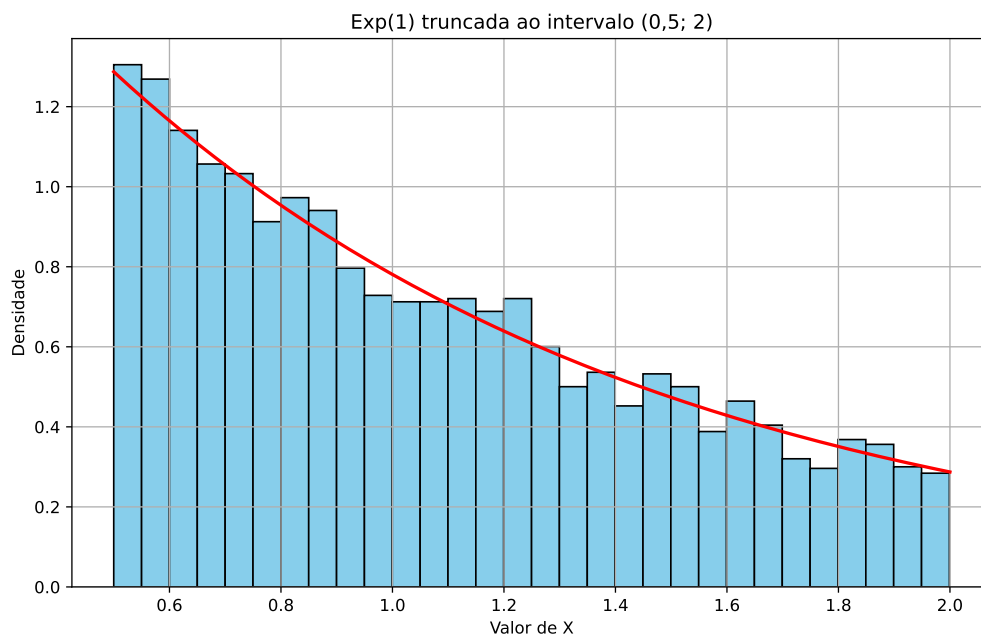
Maior valor gerado: 1.999


```
# A constante que reescala a densidade
constante = F_exp(b) - F_exp(a)
print("P(a < X < b) na exponencial original:", round(constante, 4))
```

P(a < X < b) na exponencial original: 0.4712

```
# Malha usada só para desenhar a densidade teórica, que só existe em (a, b)
grade = np.linspace(a, b, 200)

plt.figure(figsize=(10, 6))
plt.hist(X, bins=30, color='skyblue', edgecolor='black', density=True)
plt.plot(grade, lamdb * np.exp(-lamdb * grade) / constante,
         color='red', linewidth=2)
plt.title('Exp(1) truncada ao intervalo (0,5; 2)')
plt.xlabel('Valor de X')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



Todos os valores gerados caem dentro de (0,5; 2), e o histograma acompanha a densidade reescalada. Note que, na exponencial original, apenas 47% da massa está nesse intervalo — é por isso que a densidade truncada é pouco mais que o dobro da original em cada ponto.

⚠ Atenção: dois caminhos que parecem equivalentes

Diante de “quero uma exponencial entre 0,5 e 2”, duas outras ideias costumam aparecer. A primeira é **substituir** pelos extremos os valores que caem fora do intervalo (o que em programação se chama *clamp*). Isso está errado: o resultado tem massa de probabilidade concentrada exatamente em 0,5 e em 2, e portanto não é sequer uma distribuição contínua. A segunda é **descartar** os valores que caem fora, gerando novos até que um caia dentro. Esse caminho está correto — é um caso do método da rejeição, do Capítulo 6 — mas desperdiça trabalho: aqui, mais da metade dos valores gerados seria jogada fora. E o desperdício piora quanto mais raro for o intervalo: para gerar uma normal padrão truncada a $(4,5; \infty)$, como será preciso no Capítulo 11, seriam necessários cerca de 300 mil valores descartados para cada um aproveitado. Já a inversão do pseudo-algoritmo acima gasta exatamente um uniforme por valor gerado, seja qual for o intervalo.

44.1 Exemplo 4: quando não há fórmula para a inversa

Nos exemplos anteriores, F^{-1} saía com duas linhas de álgebra. Este exemplo trata do caso mais comum na prática: F é conhecida, mas a equação $F(x) = u$ não tem solução em forma fechada.

Considere um lote de componentes eletrônicos formado por dois tipos: uma fração p vem de uma linha de produção defeituosa, com tempo de vida $\text{Exp}(\lambda_1)$, e o restante de uma linha boa, com tempo de vida $\text{Exp}(\lambda_2)$. Sorteando um componente ao acaso, seu tempo de vida X tem densidade

$$f(x) = p \lambda_1 e^{-\lambda_1 x} + (1 - p) \lambda_2 e^{-\lambda_2 x}, \quad x > 0,$$

e, integrando,

$$F(x) = 1 - p e^{-\lambda_1 x} - (1 - p) e^{-\lambda_2 x}.$$

A f.d.a. é explícita, mas a equação

$$p e^{-\lambda_1 x} + (1 - p) e^{-\lambda_2 x} = 1 - u$$

envolve duas exponenciais com expoentes diferentes e não pode ser resolvida em x com as funções usuais. A saída é resolvê-la **numericamente**, para cada valor de u sorteado.

💡 Pseudo-algoritmo: inversão numérica

Entrada: a f.d.a. F e um intervalo $[0, L]$ que certamente contém a solução.

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Encontre numericamente a raiz x da equação $F(x) - U = 0$ no intervalo $[0, L]$.
3. Retorne x .

O passo 2 é resolvido por uma função pronta: `uniroot` no R e `brentq` (do `scipy.optimize`) no Python. As duas procuram uma raiz dentro de um intervalo em que a função troca de sinal, o que aqui é automático: $F(0) - U = -U < 0$ e $F(L) - U \approx 1 - U > 0$, desde que L seja grande.

45 R

```
library(ggplot2)

set.seed(42)

p <- 0.6          # proporção de componentes da linha defeituosa
lambda1 <- 1      # taxa dessa linha (vida curta)
lambda2 <- 5      # taxa da linha boa
B <- 2000         # quantos valores queremos gerar

# f.d.a. da mistura. Note que não escrevemos  $F^{-1}$ : ela não existe em forma
# fechada, e é justamente esse o ponto do exemplo
F_mistura <- function(x) 1 - p * exp(-lambda1 * x) - (1 - p) * exp(-lambda2 * x)

# Passo 1: gerar os uniformes
U <- runif(B, min = 0, max = 1)

# Passos 2 e 3: uma busca numérica para cada valor gerado.
# tol controla a precisão da raiz; o padrão de uniroot é bem mais frouxo
X <- numeric(B)
for (i in 1:B) {
  X[i] <- uniroot(function(x) F_mistura(x) - U[i],
                  interval = c(0, 50), tol = 1e-10)$root
}

cat("Média amostral:", round(mean(X), 4), "\n")
```

Média amostral: 0.6737

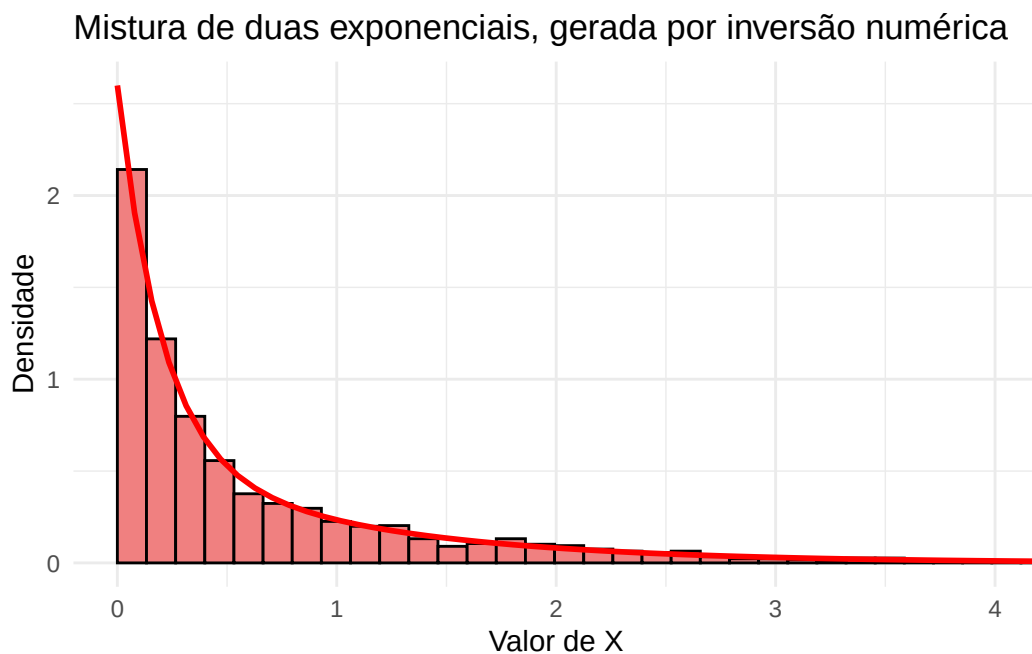
```
cat("Média teórica :", round(p / lambda1 + (1 - p) / lambda2, 4), "\n")
```

Média teórica : 0.68

```
df <- data.frame(X = X)

f_mistura <- function(x) {
  p * lambda1 * exp(-lambda1 * x) + (1 - p) * lambda2 * exp(-lambda2 * x)
}

# coord_cartesian apenas aproxima o gráfico da região onde está quase toda a
# massa; nenhum valor é descartado do histograma
ggplot(df, aes(x = X)) +
  geom_histogram(aes(y = after_stat(density)), bins = 60, boundary = 0,
                fill = "lightcoral", color = "black") +
  stat_function(fun = f_mistura, color = "red", linewidth = 1) +
  coord_cartesian(xlim = c(0, 4)) +
  labs(title = "Mistura de duas exponenciais, gerada por inversão numérica",
       x = "Valor de X", y = "Densidade") +
  theme_minimal()
```



46 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq

np.random.seed(42)

p = 0.6      # proporção de componentes da linha defeituosa
lambda1 = 1  # taxa dessa linha (vida curta)
lambda2 = 5  # taxa da linha boa
B = 2000     # quantos valores queremos gerar

# f.d.a. da mistura. Note que não escrevemos  $F^{-1}$ : ela não existe em forma
# fechada, e é justamente esse o ponto do exemplo
def F_mistura(x):
    return 1 - p * np.exp(-lambda1 * x) - (1 - p) * np.exp(-lambda2 * x)

# Passo 1: gerar os uniformes
U = np.random.uniform(0, 1, B)

# Passos 2 e 3: uma busca numérica para cada valor gerado
X = np.zeros(B)
for i in range(B):
    X[i] = brentq(lambda x: F_mistura(x) - U[i], 0, 50)

print("Média amostral:", round(X.mean(), 4))
```

Média amostral: 0.685

```
print("Média teórica :", round(p / lambda1 + (1 - p) / lambda2, 4))
```

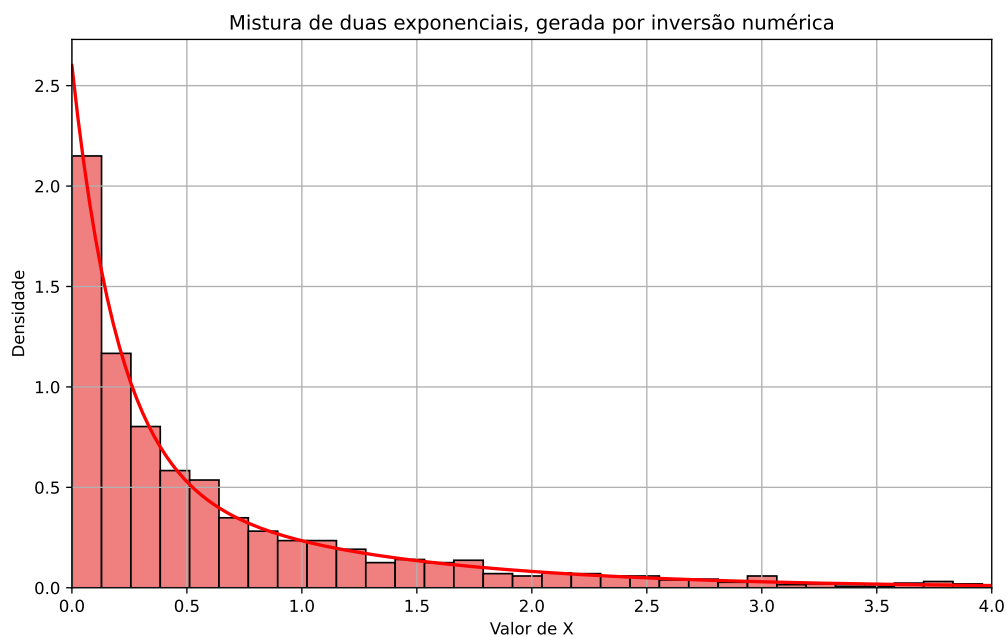
Média teórica : 0.68

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, X.max(), 300)
f_mistura = (p * lambda1 * np.exp(-lambda1 * grade) +
             (1 - p) * lambda2 * np.exp(-lambda2 * grade))

# plt.xlim apenas aproxima o gráfico da região onde está quase toda a massa;
# nenhum valor é descartado do histograma
plt.figure(figsize=(10, 6))
plt.hist(X, bins=60, color='lightcoral', edgecolor='black', density=True)
plt.plot(grade, f_mistura, color='red', linewidth=2)
plt.xlim(0, 4)
```

(0.0, 4.0)

```
plt.title('Mistura de duas exponenciais, gerada por inversão numérica')
plt.xlabel('Valor de X')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



O método funciona, mas cobra um preço: em vez de uma conta, cada valor gerado exige uma busca, que por sua vez avalia F várias vezes. Gerar um milhão de valores por esse caminho é bem mais lento do que pela fórmula do Exemplo 2.

 **Atenção:** o intervalo de busca

A busca numérica só encontra a raiz se ela estiver dentro do intervalo fornecido. Usamos $[0, 50]$ porque $F(50)$ é indistinguível de 1, mas isso não é uma garantia universal: se algum U sorteado for maior que $F(L)$, a função não troca de sinal no intervalo e o programa devolve um erro em vez de um número.

Quanto mais valores forem gerados, maior o U máximo sorteado, e maior precisa ser L . É um cuidado que a inversão com fórmula fechada simplesmente não exige.

Vale registrar que, neste exemplo específico, existe um caminho muito melhor. A densidade f é uma média ponderada de duas densidades exponenciais, e uma variável com essa estrutura pode ser gerada em dois passos — sorteia-se de qual das duas linhas de produção veio o componente e, depois, gera-se a exponencial correspondente por inversão. Nenhuma busca numérica é necessária. Esse é o **método da composição**, que veremos no capítulo sobre transformações e misturas. A inversão numérica continua sendo a alternativa quando nenhuma estrutura desse tipo está disponível.

46.1 Exercícios

Exercício 1. Utilizando o método da inversão, simule $X \sim \text{Unif}(1, 3)$.

Exercício 2.

- (a) Implemente uma função para gerar uma amostra de tamanho n da distribuição Exponencial de parâmetro λ .
- (b) Compare a distribuição empírica dos valores simulados com a densidade da Exponencial $f(x) = \lambda e^{-\lambda x}, x > 0$.

Exercício 3. Seja X uma v.a. com função densidade dada por

$$f(x) = \frac{1}{8}x, \quad 0 < x < 4.$$

- (a) Escreva um pseudo-algoritmo para simular um único valor da variável X pelo método da inversão.
- (b) Compare a distribuição empírica dos valores simulados com a densidade de X .

Exercício 4. Seja X uma v.a. com densidade

$$f(x) = \begin{cases} 4x, & 0 < x \leq 1/2, \\ 4(1-x), & 1/2 < x < 1, \\ 0, & \text{caso contrário.} \end{cases}$$

- (a) Encontre a função de distribuição acumulada F de X .
- (b) Mostre que

$$F^{-1}(u) = \begin{cases} \sqrt{u/2}, & 0 < u \leq 1/2, \\ 1 - \sqrt{(1-u)/2}, & 1/2 < u < 1. \end{cases}$$

- (c) Escreva um pseudo-algoritmo para simular um valor de X pelo método da inversão.
- (d) Implemente o algoritmo em R e Python e gere $B = 10\,000$ valores.
- (e) Compare o histograma dos valores simulados com a densidade teórica.
- (f) Usando a simulação, estime $\mathbb{E}[X]$ e $\mathbb{P}(X \leq 1/4)$. Compare com os valores exatos.

Exercício 5. Sobre as distribuições truncadas do Exemplo 3.

- (a) Implemente uma função que receba B , λ , a e b e devolva uma amostra de tamanho B da $\text{Exp}(\lambda)$ truncada a (a, b) . Gere $B = 5000$ valores com $\lambda = 2$, $a = 1$ e $b = 3$, e compare o histograma com a densidade truncada.
- (b) Quando o truncamento é só à esquerda ($b = \infty$), basta usar $F(b) = 1$ na fórmula. Mostre que, nesse caso, a exponencial truncada a (a, ∞) tem a mesma distribuição de $a + Y$, com $Y \sim \text{Exp}(\lambda)$ — é a propriedade de **falta de memória** da exponencial. Verifique numericamente, comparando os histogramas obtidos pelos dois caminhos.
- (c) Mostre que a $\text{Unif}(0, 1)$ truncada a (a, b) é exatamente a $\text{Unif}(a, b)$, tanto pela definição quanto aplicando o pseudo-algoritmo.
- (d) Gere 5000 valores de uma $N(0, 1)$ truncada a $(-1, 1)$. Você pode usar `qnorm` em R e `scipy.stats.norm.ppf` em Python no papel de F^{-1} (por dentro, essas funções fazem uma inversão numérica como a do Exemplo 4). Compare a variância amostral com 1 e explique, olhando o histograma, por que ela é bem menor.

Exercício 6. Sobre a inversão numérica do Exemplo 4.

- (a) Antes de confiar no método, é boa prática testá-lo num caso de resposta conhecida. Gere $B = 2000$ uniformes e transforme cada um em uma $\text{Exp}(2)$ de duas maneiras: pela fórmula $-\log(1 - U)/2$ e resolvendo numericamente $F(x) = U$. Confira que os dois vetores coincidem até a tolerância pedida na busca.

- (b) Meça o tempo das duas versões do item (a) com $B = 20\,000$ (use `system.time` em R ou `time.time` em Python). Quantas vezes mais lenta é a busca numérica?
- (c) Refaça o Exemplo 4 trocando o intervalo de busca de $[0, 50]$ para $[0, 2]$. O que acontece? Calcule $F(2)$ e determine a partir de qual valor sorteado de U o programa falha.
- (d) Estime $\mathbb{P}(X > 1)$ para a mistura do Exemplo 4 e compare com o valor exato $1 - F(1)$.
- (e) (Desafio) Implemente a geração de uma $N(0, 1)$ por inversão numérica, resolvendo $\Phi(x) = U$ com `pnorm/scipy.stats.norm.cdf` no papel de F (e sem usar `qnorm` nem `norm.ppf`). Compare o histograma com a densidade teórica e o tempo de execução com o de `rnorm/np.random.normal`. Qual intervalo de busca você usou, e o que acontece se U for muito próximo de 0 ou de 1?

47 Método da Rejeição para Variáveis Discretas

No Capítulo 3 vimos como gerar v.a. discretas pela técnica da inversão: acumulamos as probabilidades $p_1, p_2, \dots$ até ultrapassar um valor $U \sim \text{Unif}(0, 1)$. Esse método sempre funciona, mas exige que saibamos percorrer o suporte em ordem e calcular a f.d.a. do alvo.

O **método da rejeição** parte de uma ideia diferente. Em vez de construir X diretamente a partir de U , ele usa uma segunda distribuição — *fácil de simular* — para propor candidatos, e um sorteio adicional para decidir se cada candidato é aceito ou descartado. O preço a pagar é jogar fora parte dos valores gerados; em troca, basta saber calcular a **razão** entre as duas funções de probabilidade. Nunca precisamos da f.d.a. do alvo e, como veremos nos exercícios, nem mesmo da constante que normaliza p_j .

Neste capítulo tratamos o caso discreto. No próximo veremos que a versão contínua do método é essencialmente a mesma, trocando funções de probabilidade por densidades.

47.1 Algoritmo

Seja X uma v.a. discreta com $\mathbb{P}(X = x_j) = p_j$; essa é a **distribuição alvo**, aquela da qual queremos amostrar. O método supõe que sabemos simular uma segunda v.a. Y , com $\mathbb{P}(Y = x_j) = q_j$, chamada de **distribuição proposta**.

 **Atenção:** a proposta precisa cobrir o suporte do alvo

É indispensável que $q_j > 0$ sempre que $p_j > 0$. Se a proposta nunca sugere um valor que o alvo pode assumir, esse valor jamais aparecerá na amostra — e o algoritmo devolve, silenciosamente, uma distribuição errada.

Além disso, supomos conhecida uma constante c tal que

$$\frac{p_j}{q_j} \leq c \quad \text{para todo } j \text{ com } p_j > 0.$$

Quando o suporte é finito, a menor constante que serve é simplesmente $c = \max_j p_j/q_j$. Note ainda que necessariamente $c \geq 1$: somando a desigualdade $p_j \leq c q_j$ sobre os valores de j com $p_j > 0$, obtemos

$$1 = \sum_{j: p_j > 0} p_j \leq c \sum_{j: p_j > 0} q_j \leq c,$$

onde a última passagem usa que a soma de um subconjunto das probabilidades q_j é, no máximo, 1.

 Pseudo-algoritmo: rejeição para v.a. discretas

1. Gere Y a partir da distribuição proposta q .
2. Gere $U \sim \text{Unif}(0, 1)$, independente de Y .
3. Se $Y = x_j$ e $U \leq \frac{p_j}{c q_j}$, devolva $X = x_j$. Caso contrário, descarte Y e volte ao passo 1.

O passo 3 é apenas um sorteio de Bernoulli: aceitamos o candidato com probabilidade $p_j/(c q_j)$. É exatamente a escolha de c que garante que esse número esteja entre 0 e 1 e, portanto, seja de fato uma probabilidade.

47.2 Exemplo: uma distribuição sobre $\{1, 2, \dots, 10\}$

Vamos exemplificar a amostragem por rejeição gerando valores de uma distribuição alvo p_j , utilizando uma uniforme discreta como distribuição proposta.

Suponha que Y siga uma distribuição uniforme discreta em $\{1, 2, \dots, 10\}$, ou seja, $q_j = 1/10$ para todo j . A distribuição alvo p_j tem os seguintes valores:

j	1	2	3	4	5	6	7	8	9	10
p_j	0,11	0,12	0,09	0,08	0,12	0,10	0,09	0,09	0,10	0,10

Precisamos de uma constante c com $p_j/q_j \leq c$ para todo j . Como $q_j = 1/10$ para todos os valores, a razão $p_j/q_j = 10 p_j$ é máxima onde p_j é máximo, isto é, em $j = 2$ e $j = 5$. Logo,

$$c = \max_j \frac{p_j}{q_j} = 10 \times 0,12 = 1,2.$$

O gráfico a seguir ilustra o que o algoritmo faz. Para cada j , a haste preta vai de 0 até $c q_j$ (ponto verde), e o ponto vermelho marca a altura p_j . Sorteamos um valor do suporte com probabilidade uniforme e o aceitamos com probabilidade igual à **razão entre a altura do ponto vermelho e a do ponto verde** — que é justamente $p_j/(c q_j)$.

48 R

```
library(ggplot2)

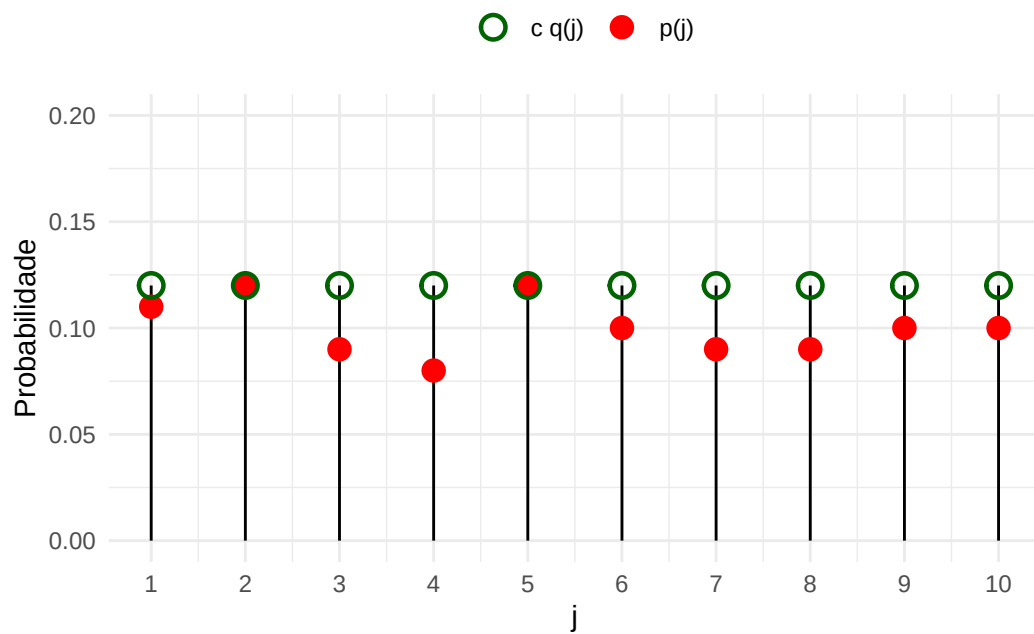
# Valores de j e probabilidades p_j (alvo) e q_j (proposta)
valores_j <- 1:10
p_j <- c(0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10)
q_j <- rep(1 / 10, length(p_j)) # proposta: uniforme discreta em {1, ..., 10}

# Menor constante que satisfaz p_j <= cte * q_j para todo j.
# Chamamos de `cte` (e não de `c`) para não esconder a função c() do R
cte <- max(p_j / q_j)

# Data frame no formato longo: uma linha para cada par (j, curva)
df <- data.frame(
  j = rep(valores_j, 2),
  prob = c(p_j, cte * q_j),
  curva = rep(c("p(j)", "c q(j)"), each = length(valores_j))
)

# As hastes vão de 0 até c*q_j, o "teto" da região de propostas
df_hastes <- data.frame(j = valores_j, teto = cte * q_j)

# O teto é desenhado como círculo vazado: assim, quando p_j = c*q_j (em j = 2 e
# j = 5) o ponto vermelho continua visível por dentro do círculo verde
ggplot(df, aes(x = j, y = prob, color = curva, shape = curva)) +
  geom_segment(data = df_hastes, aes(x = j, xend = j, y = 0, yend = teto),
    inherit.aes = FALSE, color = "black") +
  geom_point(size = 3.5, stroke = 1.2) +
  scale_color_manual(values = c("p(j)" = "red", "c q(j)" = "darkgreen"), name = "") +
  scale_shape_manual(values = c("p(j)" = 16, "c q(j)" = 1), name = "") +
  scale_x_continuous(breaks = valores_j) +
  ylim(0, 0.2) +
  labs(x = "j", y = "Probabilidade") +
  theme_minimal() +
  theme(legend.position = "top")
```



49 Python

```
import numpy as np
import matplotlib.pyplot as plt

# Valores de j e probabilidades p_j (alvo) e q_j (proposta)
valores_j = np.arange(1, 11)
p_j = np.array([0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10])
q_j = np.full_like(p_j, 1 / 10) # proposta: uniforme discreta em {1, ..., 10}

# Menor constante que satisfaz p_j <= cte * q_j para todo j
cte = max(p_j / q_j)

plt.figure(figsize=(8, 5))

# As hastes vão de 0 até c*q_j, o "teto" da região de propostas
plt.vlines(valores_j, 0, cte * q_j, color="black")

# O teto é desenhado como círculo vazado: assim, quando p_j = c*q_j (em j = 2 e
# j = 5) o ponto vermelho continua visível por dentro do círculo verde
plt.plot(valores_j, p_j, "o", color="red", markersize=6, label="p(j)")
plt.plot(valores_j, cte * q_j, "o", markerfacecolor="none",
         markeredgecolor="darkgreen", markeredgewidth=1.5, markersize=9,
         label="c q(j)")

plt.xticks(valores_j)
```

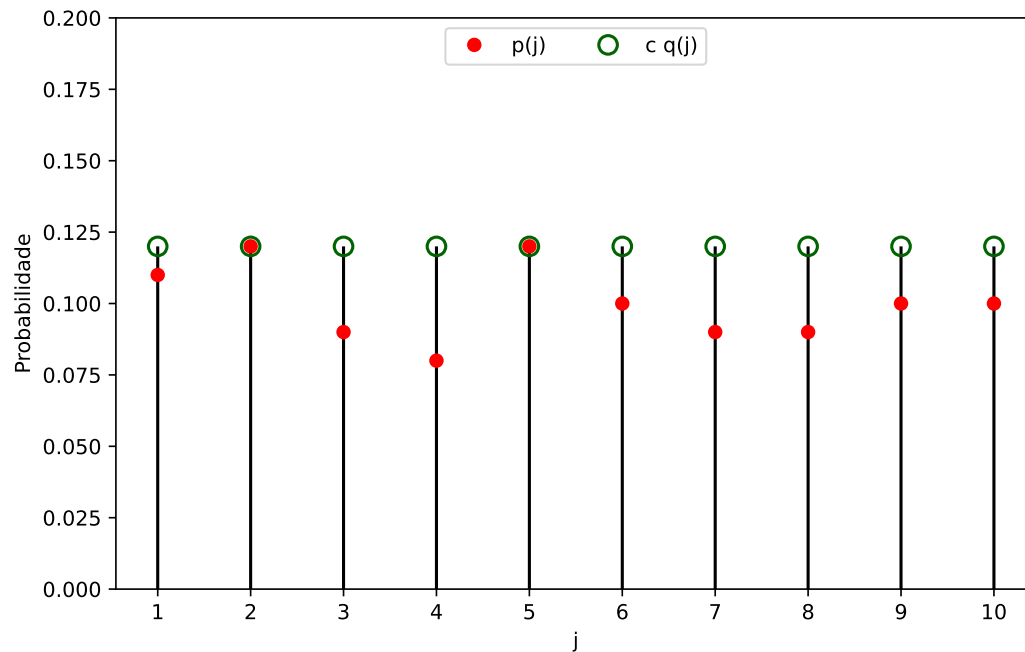
```
([<matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>, <matplotlib.axis.XTick object at 0x78633892bed0>], [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
plt.ylim(0, 0.2)
```

```
(0.0, 0.2)
```



```
plt.xlabel("j")
plt.ylabel("Probabilidade")
plt.legend(loc="upper center", ncol=2)
plt.show()
```



O código a seguir implementa o pseudo-algoritmo e gera 1000 valores dessa distribuição. Além das amostras aceitas, contamos **quantas propostas foram necessárias**: isso nos permite comparar a taxa de aceitação observada com o valor teórico $1/c$. Ao final, um gráfico põe lado a lado as frequências relativas obtidas e as probabilidades teóricas p_j .

50 R

```
library(ggplot2)

set.seed(42)

# --- Parâmetros do problema ---

valores_j <- 1:10
p_j <- c(0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10)
q_j <- rep(1 / 10, 10)
cte <- max(p_j / q_j)

# --- Método da rejeição ---

n_amostras <- 1000
amostras <- c()      # guarda os valores aceitos
n_propostas <- 0     # conta quantas vezes o passo 1 foi executado

while (length(amostras) < n_amostras) {
  # Passo 1: gerar Y da proposta (uniforme discreta em {1, ..., 10})
  y <- sample(valores_j, 1)
  n_propostas <- n_propostas + 1

  # Passo 2: gerar U ~ Unif(0,1)
  u <- runif(1)

  # Passo 3: aceitar y com probabilidade p_y / (c * q_y).
  # Como o suporte é 1, ..., 10, o próprio valor y serve de índice do vetor
  razao <- p_j[y] / (cte * q_j[y])

  if (u <= razao) {
    amostras <- c(amostras, y)
  }
}
```

```
# --- Resultados ---
```

```
cat("Primeiras 20 amostras geradas:\n")
```

Primeiras 20 amostras geradas:

```
print(amostras[1:20])
```

```
[1] 1 10 2 1 4 5 4 2 9 4 5 2 8 10 4 2 5 2 2 8
```

```
cat("\nPropostas geradas:", n_propostas, "\n")
```

Propostas geradas: 1209

```
cat("Taxa de aceitação observada:", round(n_amostras / n_propostas, 3), "\n")
```

Taxa de aceitação observada: 0.827

```
cat("Taxa de aceitação teórica (1/c):", round(1 / cte, 3), "\n")
```

Taxa de aceitação teórica (1/c): 0.833

```
# Tabela de frequência (factor com levels garante que todos os j apareçam,  
# mesmo os que porventura não tenham sido sorteados)
```

```
tabela_freq <- table(factor(amostras, levels = valores_j))
```

```
comparacao <- data.frame(  
  j = valores_j,  
  freq_absoluta = as.integer(tabela_freq),  
  freq_relativa = as.numeric(tabela_freq) / n_amostras,  
  p_j = p_j  
)
```

```
cat("\nFrequências dos", n_amostras, "valores gerados:\n")
```

Frequências dos 1000 valores gerados:

```
print(comparacao, row.names = FALSE)
```

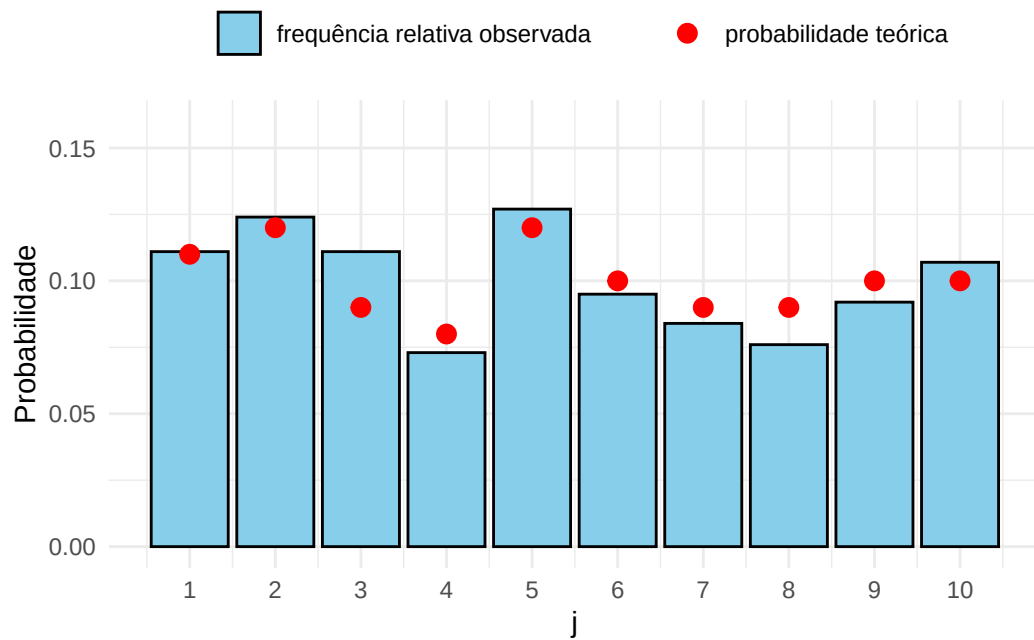
j	freq_absoluta	freq_relativa	p_j
1	111	0.111	0.11
2	124	0.124	0.12
3	111	0.111	0.09
4	73	0.073	0.08
5	127	0.127	0.12
6	95	0.095	0.10
7	84	0.084	0.09
8	76	0.076	0.09
9	92	0.092	0.10
10	107	0.107	0.10

```
# --- Comparação com a distribuição teórica ---
```

```
# As barras são o que saiu na simulação; os pontos vermelhos são as
```

```
# probabilidades p_j que queríamos reproduzir
```

```
ggplot(comparacao, aes(x = j)) +  
  geom_col(aes(y = freq_relativa, fill = "frequência relativa observada"),  
           color = "black") +  
  geom_point(aes(y = p_j, color = "probabilidade teórica"), size = 3) +  
  scale_fill_manual(values = c("frequência relativa observada" = "skyblue"),  
                   name = "") +  
  scale_color_manual(values = c("probabilidade teórica" = "red"), name = "") +  
  scale_x_continuous(breaks = valores_j) +  
  ylim(0, 0.16) +  
  labs(x = "j", y = "Probabilidade") +  
  theme_minimal() +  
  theme(legend.position = "top")
```



51 Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

# --- Parâmetros do problema ---

valores_j = np.arange(1, 11)
p_j = np.array([0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10])
q_j = np.full_like(p_j, 1 / 10)
cte = max(p_j / q_j)

# --- Método da rejeição ---

n_amostras = 1000
amostras = [] # guarda os valores aceitos
n_propostas = 0 # conta quantas vezes o passo 1 foi executado

while len(amostras) < n_amostras:
    # Passo 1: gerar Y da proposta (uniforme discreta em {1, ..., 10})
    y = np.random.choice(valores_j)
    n_propostas += 1

    # Passo 2: gerar U ~ Unif(0,1)
    u = np.random.uniform(0, 1)

    # Passo 3: aceitar y com probabilidade p_y / (c * q_y).
    # Usamos y - 1 porque em Python o primeiro elemento do vetor tem índice 0
    razao = p_j[y - 1] / (cte * q_j[y - 1])

    if u <= razao:
        amostras.append(y)
```

```
# --- Resultados ---
```

```
print("Primeiras 20 amostras geradas:")
```

Primeiras 20 amostras geradas:

```
print(np.array(amostras[:20])) # np.array só para imprimir de forma compacta
```

```
[ 8  7  7  8  8  6  6  5 10  9 10  3  4  7  7  2 10  4  7  8]
```

```
print("\nPropostas geradas:", n_propostas)
```

Propostas geradas: 1186

```
print("Taxa de aceitação observada:", round(n_amostras / n_propostas, 3))
```

Taxa de aceitação observada: 0.843

```
print("Taxa de aceitação teórica (1/c):", round(1 / cte, 3))
```

Taxa de aceitação teórica (1/c): 0.833

```
# Tabela de frequência (reindex garante que todos os j apareçam,
```

```
# mesmo os que porventura não tenham sido sorteados)
```

```
tabela_freq = pd.Series(amostras).value_counts().reindex(valores_j, fill_value=0)
```

```
comparacao = pd.DataFrame({
    "j": valores_j,
    "freq_absoluta": tabela_freq.values,
    "freq_relativa": tabela_freq.values / n_amostras,
    "p_j": p_j
})
```

```
print("\nFrequências dos", n_amostras, "valores gerados:")
```

Frequências dos 1000 valores gerados:

```
print(comparacao.to_string(index=False))
```

j	freq_absoluta	freq_relativa	p_j
1	121	0.121	0.11
2	131	0.131	0.12
3	72	0.072	0.09
4	81	0.081	0.08
5	131	0.131	0.12
6	98	0.098	0.10
7	85	0.085	0.09
8	95	0.095	0.09
9	100	0.100	0.10
10	86	0.086	0.10

```
# --- Comparação com a distribuição teórica ---
```

```
# As barras são o que saiu na simulação; os pontos vermelhos são as  
# probabilidades p_j que queríamos reproduzir
```

```
plt.figure(figsize=(8, 5))  
plt.bar(valores_j, comparacao["freq_relativa"], color="skyblue",  
        edgecolor="black", label="frequência relativa observada")  
plt.plot(valores_j, p_j, "o", color="red", markersize=7,  
         label="probabilidade teórica")
```

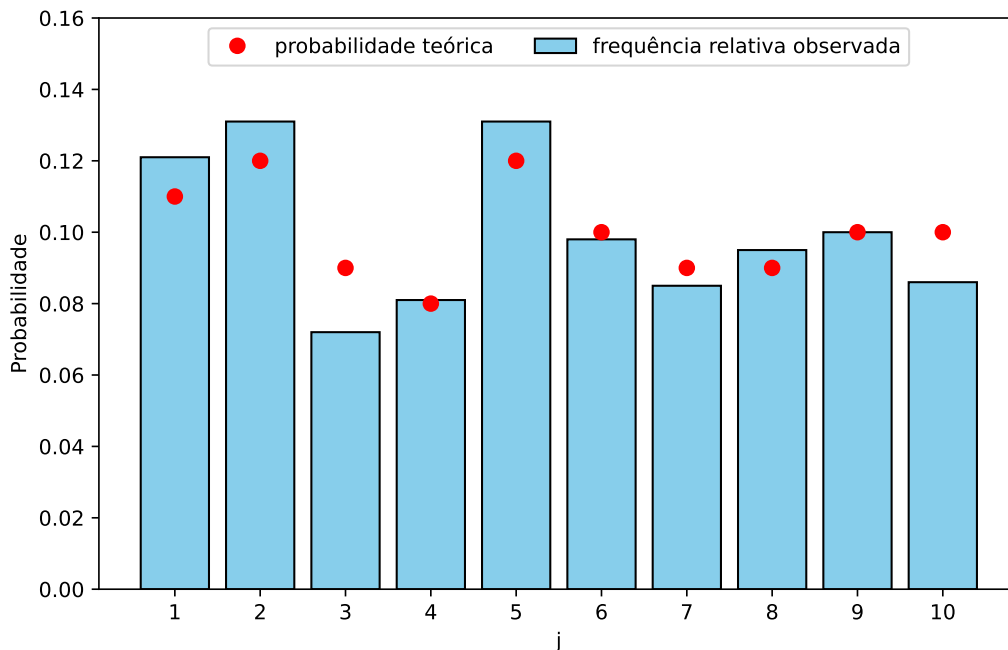
```
plt.xticks(valores_j)
```

```
([<matplotlib.axis.XTick object at 0x7863160d8910>, <matplotlib.axis.XTick object at 0x7863160d8910>],
```

```
plt.ylim(0, 0.16)
```

```
(0.0, 0.16)
```

```
plt.xlabel("j")  
plt.ylabel("Probabilidade")  
plt.legend(loc="upper center", ncol=2)  
plt.show()
```

⚠ Atenção: índices começam em 1 no R e em 0 no Python

No código acima o suporte é $\{1, 2, \dots, 10\}$, então em R o valor sorteado y serve diretamente como índice: $p_j[y]$ é exatamente p_y . Em Python o primeiro elemento de um vetor é $p_j[0]$, e por isso escrevemos $p_j[y - 1]$.

Esse deslocamento é fonte frequente de erros silenciosos: o programa roda sem reclamar, mas gera a distribuição errada. Se o suporte não fosse $\{1, \dots, 10\}$ — por exemplo, se fosse $\{0, 1, \dots, 9\}$ ou $\{2, 4, 6\}$ — o mais seguro seria usar um dicionário (Python) ou um vetor nomeado (R) associando cada valor à sua probabilidade.

51.1 Por que o método funciona

! Proposição

Suponha que $q_j > 0$ sempre que $p_j > 0$ e que $p_j/q_j \leq c$ para todo j com $p_j > 0$. Então:

- (i) o valor X devolvido pelo método da rejeição satisfaz $\mathbb{P}(X = x_j) = p_j$ para todo j ;
- (ii) o número N de propostas geradas até o primeiro aceite tem distribuição geométrica

com probabilidade de sucesso $1/c$. Em particular, $\mathbb{E}[N] = c$.

i Demonstração

Seja Y o valor proposto em um passo qualquer do algoritmo. Condição no valor de Y ,

$$\mathbb{P}(Y = x_j \text{ e aceitamos } Y) = \mathbb{P}(Y = x_j) \mathbb{P}(\text{aceitar} \mid Y = x_j) = q_j \cdot \frac{p_j}{c q_j} = \frac{p_j}{c}.$$

Somando sobre todos os valores possíveis de j , a probabilidade de um passo qualquer terminar em aceite é

$$\mathbb{P}(\text{aceitar}) = \sum_j \frac{p_j}{c} = \frac{1}{c}.$$

Como os passos são independentes e cada um aceita com probabilidade $1/c$, o número N de propostas até o primeiro aceite tem distribuição geométrica com probabilidade de sucesso $1/c$, e portanto $\mathbb{E}[N] = c$. Isso prova (ii).

Para (i), note que o algoritmo devolve x_j quando, para algum $n \geq 1$, as primeiras $n - 1$ propostas foram rejeitadas e a n -ésima propôs x_j e foi aceita. Chame esse evento de A_n . Os eventos $A_1, A_2, \dots$ são disjuntos (cada um especifica em qual passo ocorreu o primeiro aceite) e sua união é exatamente o evento $\{X = x_j\}$, de modo que

$$\begin{aligned} \mathbb{P}(X = x_j) &= \sum_{n=1}^{\infty} \mathbb{P}(A_n) \\ &= \sum_{n=1}^{\infty} \mathbb{P}(\text{os } n - 1 \text{ primeiros passos rejeitam}) \cdot \mathbb{P}(\text{o } n\text{-ésimo passo propõe } x_j \text{ e aceita}) \\ &= \sum_{n=1}^{\infty} \left(1 - \frac{1}{c}\right)^{n-1} \frac{p_j}{c} \\ &= \frac{p_j}{c} \cdot \frac{1}{1 - \left(1 - \frac{1}{c}\right)} \\ &= \frac{p_j}{c} \cdot c = p_j. \end{aligned}$$

Na segunda igualdade usamos que os passos são independentes; na terceira, as duas contas do parágrafo anterior (cada passo rejeita com probabilidade $1 - 1/c$ e propõe-e-aceita x_j com probabilidade p_j/c); e na quarta, a soma da série geométrica.

□

51.2 Eficiência e escolha da distribuição proposta

O item (ii) da proposição tem uma consequência prática importante: para gerar **um** valor da distribuição alvo, o algoritmo gasta, em média, c propostas. Equivalentemente, a fração de candidatos aceitos é $1/c$. Toda a eficiência do método está concentrada nessa única constante.

Como vimos, $c \geq 1$ sempre. O caso $c = 1$ ocorre exatamente quando $q_j = p_j$ para todo j — ou seja, quando a proposta é o alvo e nada é rejeitado. Isso não ajuda na prática (se soubéssemos simular de p , não precisaríamos do método), mas indica a direção certa: **quanto mais parecida a proposta for com o alvo, menor o c e mais eficiente o algoritmo.**

No exemplo acima, $c = 1,2$: aceitamos cerca de $1/1,2 \approx 83\%$ das propostas, e gastamos em média 1,2 propostas por valor gerado. Isso é excelente, e não é coincidência: o alvo é quase uniforme, então a proposta uniforme é quase igual a ele.

Agora imagine que, para esse mesmo alvo, usássemos como proposta uma uniforme em $\{1, 2, \dots, 100\}$. Ela cobre o suporte do alvo, então o método continua correto, mas agora $q_j = 1/100$ e

$$c = \max_j \frac{p_j}{q_j} = 100 \times 0,12 = 12,$$

de modo que apenas $1/12 \approx 8\%$ das propostas seriam aceitas: o mesmo resultado a um custo dez vezes maior. Note ainda que c depende apenas de p e q , e não do tamanho n da amostra que queremos gerar — uma proposta ruim é ruim para sempre.

51.3 Exemplo: um dado honesto a partir de moedas

No exemplo anterior, e em todos os métodos vistos até aqui, partimos de $U \sim \text{Unif}(0, 1)$. Suponha agora que a única fonte de aleatoriedade disponível seja uma **moeda honesta**: sabemos gerar bits independentes, cada um valendo 0 ou 1 com probabilidade $1/2$. Queremos simular o lançamento de um dado honesto, isto é, $p_j = 1/6$ para $j \in \{1, 2, \dots, 6\}$.

Aqui a inversão não está disponível como a usamos no Capítulo 3: não temos uma uniforme contínua para comparar com a f.d.a. do alvo. A rejeição, por outro lado, resolve o problema com naturalidade. Lendo **três bits** como um número em binário, obtemos Y uniforme em $\{1, 2, \dots, 8\}$, ou seja, $q_j = 1/8$ para $j = 1, \dots, 8$. Essa proposta cobre o suporte do alvo, e

$$c = \max_j \frac{p_j}{q_j} = \frac{1/6}{1/8} = \frac{4}{3}.$$

Como $p_7 = p_8 = 0$, a probabilidade de aceitação do passo 3 vale

$$\frac{p_j}{c q_j} = \begin{cases} 1, & j \in \{1, 2, \dots, 6\}, \\ 0, & j \in \{7, 8\}. \end{cases}$$

Ela só assume os valores 0 e 1, de modo que o sorteio de U do passo 2 se torna dispensável e o algoritmo se reduz a: *leia 3 bits; se o resultado for 7 ou 8, descarte-o e comece de novo*. É a mesma estrutura degenerada que aparecerá no Exercício 3, com uma distribuição condicional.

Pela proposição, aceitamos $1/c = 3/4$ das propostas e gastamos, em média, $c = 4/3$ propostas — isto é, $3 \times 4/3 = 4$ bits — por lançamento do dado. Vale comparar esse custo com o mínimo teórico dado pela entropia da distribuição, $\log_2 6 \approx 2,59$ bits: o esquema é simples e razoavelmente econômico, ainda que não seja ótimo.

 **Atenção:** reaproveitar as rejeições estraga a distribuição

É tentador evitar o desperdício mapeando os valores descartados de volta ao suporte, por exemplo $7 \mapsto 1$ e $8 \mapsto 2$. Isso produz um dado viciado: os valores 1 e 2 sairiam com probabilidade $2/8$ cada, e os demais com $1/8$.

Esse é exatamente o viés de `rand() % 6` — e é por isso que os geradores de inteiros uniformes de bibliotecas reais (`random.randrange` em Python, `Random.nextInt` em Java, `rand.Int31n` em Go) implementam, por dentro, um laço de rejeição como o desta seção.

O código a seguir implementa o amostrador usando apenas bits, e confere a taxa de aceitação e o custo em bits previstos pela teoria.

52 R

```
set.seed(42)

# Nossa única fonte de aleatoriedade: um bit justo (cara ou coroa)
gera_bit <- function() sample(0:1, 1)

# Alvo: uniforme em {1, ..., 6}. Proposta: uniforme em {1, ..., 8}, obtida
# lendo 3 bits como um número em binário
p_j <- 1 / 6
q_j <- 1 / 8
cte <- p_j / q_j    # c = 4/3

n_amstras <- 5000
amostras <- integer(n_amstras)
n_propostas <- 0

for (i in seq_len(n_amstras)) {
  repeat {
    # Passo 1: três bits formam Y uniforme em {1, ..., 8}
    y <- 1 + 4 * gera_bit() + 2 * gera_bit() + gera_bit()
    n_propostas <- n_propostas + 1

    # Passos 2 e 3: a probabilidade de aceitação p_y / (c q_y) vale 1 para
    # y <= 6 e 0 para y = 7 ou 8, então o sorteio de U é dispensável
    if (y <= 6) break
  }
  amostras[i] <- y
}

cat("Taxa de aceitação observada:", round(n_amstras / n_propostas, 3), "\n")
```

Taxa de aceitação observada: 0.75

```
cat("Taxa de aceitação teórica (1/c):", round(1 / cte, 3), "\n")
```

Taxa de aceitação teórica (1/c): 0.75

```
cat("Bits gastos por lançamento:", round(3 * n_propostas / n_amostras, 3), "\n\n")
```

Bits gastos por lançamento: 3.998

```
cat("Frequências relativas (teórico: 1/6 =", round(1 / 6, 3), "):\n")
```

Frequências relativas (teórico: 1/6 = 0.167):

```
print(round(table(amostras) / n_amostras, 3))
```

amostras

1	2	3	4	5	6
0.168	0.157	0.169	0.169	0.176	0.162

53 Python

```
import numpy as np
import pandas as pd

np.random.seed(42)

# Nossa única fonte de aleatoriedade: um bit justo (cara ou coroa)
def gera_bit():
    return np.random.randint(0, 2)

# Alvo: uniforme em {1, ..., 6}. Proposta: uniforme em {1, ..., 8}, obtida
# lendo 3 bits como um número em binário
p_j = 1 / 6
q_j = 1 / 8
cte = p_j / q_j # c = 4/3

n_amostras = 5000
amostras = []
n_propostas = 0

for _ in range(n_amostras):
    while True:
        # Passo 1: três bits formam Y uniforme em {1, ..., 8}
        y = 1 + 4 * gera_bit() + 2 * gera_bit() + gera_bit()
        n_propostas += 1

        # Passos 2 e 3: a probabilidade de aceitação p_y / (c q_y) vale 1 para
        # y <= 6 e 0 para y = 7 ou 8, então o sorteio de U é dispensável
        if y <= 6:
            break
        amostras.append(y)

print("Taxa de aceitação observada:", round(n_amostras / n_propostas, 3))
```

Taxa de aceitação observada: 0.752

```
print("Taxa de aceitação teórica (1/c):", round(1 / cte, 3))
```

Taxa de aceitação teórica (1/c): 0.75

```
print("Bits gastos por lançamento:", round(3 * n_propostas / n_amostras, 3), "\n")
```

Bits gastos por lançamento: 3.988

```
freq = pd.Series(amostras).value_counts().sort_index() / n_amostras
print("Frequências relativas (teórico: 1/6 =", round(1 / 6, 3), "):")
```

Frequências relativas (teórico: 1/6 = 0.167):

```
print(freq.round(3).to_string())
```

1	0.164
2	0.174
3	0.160
4	0.169
5	0.168
6	0.165

53.1 Quando a rejeição é preferível à inversão

A inversão do Capítulo 3 é simples e nunca descarta nada, então cabe perguntar por que alguém escolheria um método que joga trabalho fora. Há três situações típicas.

1. Quando não conhecemos a constante de normalização. É comum sabermos calcular pesos w_j proporcionais a p_j sem conhecer a soma $\sum_j w_j$ que os normaliza — o caso típico é uma distribuição a posteriori sobre um conjunto discreto de modelos ou configurações, em que cada w_j é fácil de avaliar mas a soma envolve um número proibitivo de termos. A inversão precisa da f.d.a., e portanto da normalização; a rejeição usa apenas a razão $w_j/(c' q_j)$, na qual a constante se cancela (Exercício 4). É essa mesma propriedade que torna possíveis os métodos de MCMC.

2. Quando o alvo é uma distribuição condicional. Se queremos amostrar W dado $W \in A$, então $p_j = \mathbb{P}(W = j)/\mathbb{P}(W \in A)$ para $j \in A$. Tomando como proposta a própria

distribuição de W , a razão p_j/q_j é constante em A , vale $c = 1/\mathbb{P}(W \in A)$, e o algoritmo se reduz a “gere valores de W até cair em A ” — sem nunca calcular $\mathbb{P}(W \in A)$ (Exercício 3). Distribuições truncadas, filas condicionadas a exceder a capacidade e sinistros acima de uma franquia entram todos nesse molde.

3. Quando percorrer o suporte é caro. A inversão do Capítulo 3 acumula probabilidades até ultrapassar U , o que custa, em média, um número de passos da ordem do tamanho do suporte relevante. Para uma Poisson com $\lambda = 500$ ou uma Binomial com n grande, isso pesa. Por isso os geradores de bibliotecas reais trocam de estratégia conforme o parâmetro: R e NumPy usam inversão para λ ou np pequenos e algoritmos de **rejeição** com envelopes ajustados quando esses valores crescem, obtendo um custo que não aumenta com o parâmetro.

 O outro lado: eventos raros

No caso condicional acima, $\mathbb{E}[N] = c = 1/\mathbb{P}(W \in A)$. Se o evento de interesse tem probabilidade 10^{-4} , são necessárias em média 10 mil propostas para cada valor aceito — e 99,99% do esforço vai para o lixo. A rejeição continua *correta*, mas se torna impraticável nesse regime.

É justamente essa dificuldade que motiva a amostragem por importância do Capítulo 11: em vez de descartar tudo que cai fora de A , deslocamos a distribuição proposta para a região de interesse e corrigimos o viés com pesos.

53.2 Exercícios

Exercício 1. Considere uma distribuição alvo X com suporte em $\{1, 2, \dots, 15\}$ e função de probabilidade

$$p_j = \frac{\ln(j+1)}{\sum_{k=1}^{15} \ln(k+1)}, \quad j = 1, 2, \dots, 15.$$

Use como distribuição proposta uma uniforme discreta em $\{1, 2, \dots, 15\}$.

- Determine a constante c . Em qual valor de j a razão p_j/q_j é máxima, e por quê?
- Implemente o método da rejeição e gere 1000 amostras. Compare graficamente as frequências relativas obtidas com as probabilidades teóricas p_j .
- Calcule a taxa de aceitação observada e compare com o valor teórico $1/c$.
- Modifique seu código para guardar, para **cada** valor aceito, quantas propostas foram necessárias até aceitá-lo. Compare o histograma desses valores com a função de probabilidade de uma Geométrica($1/c$), conforme o item (ii) da proposição.

Exercício 2. Seja $X \sim \text{Binomial}(10, p)$, com

$$p_j = \binom{10}{j} p^j (1-p)^{10-j}, \quad j = 0, 1, \dots, 10,$$

e use como proposta a uniforme discreta em $\{0, 1, \dots, 10\}$.

- (a) Mostre que a menor constante possível é $c = 11 \max_j p_j$, e calcule seu valor para $p = 0,5$ e para $p = 0,05$.
- (b) Para cada um dos dois valores de p , implemente o método e gere 5000 amostras, registrando a taxa de aceitação observada. Confira que ela bate com $1/c$.
- (c) A proposta uniforme é muito pior para $p = 0,05$ do que para $p = 0,5$. Explique o motivo comparando o formato das duas distribuições alvo com o da proposta.
- (d) Cuidado com a indexação: aqui o suporte começa em 0, e não em 1. Descreva o que aconteceria com a distribuição gerada se, em R, você escrevesse `p_j [y]` em vez de `p_j [y + 1]`.

Exercício 3. Seja $W \sim \text{Binomial}(20; 0,3)$ e considere a distribuição alvo dada por W **condicionada** a $W \geq 10$, isto é,

$$p_j = \frac{\mathbb{P}(W = j)}{\mathbb{P}(W \geq 10)}, \quad j = 10, 11, \dots, 20,$$

e $p_j = 0$ para $j < 10$. Como distribuição proposta, use a própria $\text{Binomial}(20; 0,3)$, ou seja, $q_j = \mathbb{P}(W = j)$ para $j = 0, 1, \dots, 20$.

- (a) Mostre que $p_j/q_j = 1/\mathbb{P}(W \geq 10)$ para $j \geq 10$ e que essa razão vale 0 para $j < 10$. Conclua que a menor constante possível é $c = 1/\mathbb{P}(W \geq 10)$.
- (b) Substitua esse c no critério do passo 3 e mostre que a probabilidade de aceitação vale 1 quando $y \geq 10$ e 0 caso contrário. Ou seja: o sorteio de U se torna irrelevante e o algoritmo se reduz a “gere valores da binomial até obter um maior ou igual a 10”.
- (c) Implemente o algoritmo (use `rbinom` em R ou `np.random.binomial` em Python para gerar as propostas) e gere 2000 valores.
- (d) Calcule o número médio de propostas por valor aceito e compare com $1/\mathbb{P}(W \geq 10)$. O método é eficiente neste caso? O que isso sugere sobre usar rejeição para amostrar de eventos raros?

Exercício 4. Muitas vezes conhecemos a função de probabilidade alvo apenas **a menos de uma constante de normalização**: sabemos calcular w_j , mas não a constante k tal que $p_j = k w_j$. O método da rejeição funciona mesmo assim. Se c' é tal que $w_j \leq c' q_j$ para todo j , o critério de aceitação $U \leq p_j/(c q_j)$ com $c = k c'$ vira

$$U \leq \frac{k w_j}{k c' q_j} = \frac{w_j}{c' q_j},$$

que não depende de k .

Considere $w_j = 1/j^2$ para $j = 1, 2, \dots, 50$ (de modo que $p_j = k/j^2$ com $k = 1/\sum_{j=1}^{50} j^{-2}$), e use como proposta a uniforme discreta em $\{1, 2, \dots, 50\}$.

- Determine $c' = \max_j w_j/q_j$.
- Implemente o método usando apenas w_j , q_j e c' , e gere 5000 amostras. Compare as frequências relativas obtidas com os valores p_j (que você pode calcular à parte, apenas para conferir).
- A taxa de aceitação observada é bem baixa. Explique por quê, comparando o formato de p_j com o da proposta uniforme.
- (Desafio) Pela proposição, a taxa de aceitação estima $1/c = 1/(k c')$. Use a taxa observada no item (b) para construir uma estimativa $\hat{k}$ da constante de normalização, e compare com o valor exato de k . Repare que acabamos de estimar uma soma sem nunca tê-la calculado.

Exercício 5. Neste exercício, o desafio é amostrar de uma distribuição com suporte infinito: uma Poisson **truncada em zero**. Isso é útil em cenários onde contamos eventos, mas apenas os casos em que *pelo menos um* evento ocorreu são registrados.

A distribuição alvo X é uma Poisson com parâmetro $\lambda = 4$ truncada em zero, cuja função de probabilidade, para $k \in \{1, 2, 3, \dots\}$, é

$$p_k = \mathbb{P}(X = k) = \frac{e^{-\lambda} \lambda^k}{k! (1 - e^{-\lambda})}.$$

Como distribuição proposta Y , use uma Geométrica(0,25) contando o número de tentativas até o primeiro sucesso, que também tem suporte em $\{1, 2, 3, \dots\}$:

$$q_k = \mathbb{P}(Y = k) = (1 - p)^{k-1} p, \quad p = 0,25.$$

- Implemente funções que calculem p_k e q_k para um dado k .

- (b) Ao contrário do caso de suporte finito, não podemos percorrer todos os valores para achar o máximo de p_k/q_k . Calcule a razão $r_k = p_k/q_k$ para $k = 1, \dots, \lceil 10\lambda \rceil$ e faça um gráfico de r_k contra k . O máximo observado é uma boa aproximação para c .
- (c) Implemente o amostrador e gere 5000 valores. Atenção: em R, `rgeom` conta o número de *falhas* antes do primeiro sucesso, então é preciso somar 1 ao resultado; em Python, `np.random.geometric` já conta o número de tentativas.
- (d) Construa um gráfico de barras comparando as frequências relativas observadas com as probabilidades teóricas p_k , e compare a taxa de aceitação observada com $1/c$.
- (e) (Desafio) Suponha que trocássemos os papéis: alvo geométrica e proposta Poisson truncada. Mostre que, nesse caso, a razão p_k/q_k é ilimitada e portanto **não existe** constante c válida. Qual característica das caudas das duas distribuições explica isso?

Exercício 6. Vamos comparar os dois métodos que já conhecemos para gerar v.a. discretas.

- (a) Implemente uma função que gere n valores da distribuição do exemplo deste capítulo (a tabela sobre $\{1, \dots, 10\}$) pela técnica da inversão do Capítulo 3.
- (b) Implemente outra função que gere n valores da mesma distribuição pelo método da rejeição.
- (c) Meça o tempo de execução das duas funções para $n = 10^5$ (use `system.time` em R ou `time.time` em Python). Qual é mais rápida?
- (d) Repita a comparação usando como proposta a uniforme discreta em $\{1, \dots, 100\}$ discutida na seção sobre eficiência. Como o tempo do método da rejeição se altera? E o da inversão?

Exercício 7. (Desafio) Este exercício explora o papel da constante c .

- (a) Mostre que qualquer constante válida satisfaz $c \geq 1$, com igualdade se e somente se $q_j = p_j$ para todo j .
- (b) Mostre que, se existe j com $p_j > 0$ e $q_j = 0$, então não existe c válido.
- (c) Usar um c **maior** que $\max_j p_j/q_j$ não compromete a corretude do método, apenas sua eficiência. Justifique reexaminando a demonstração da proposição, e verifique empiricamente gerando 5000 valores do exemplo do capítulo com $c = 5$ em vez de $c = 1, 2$.
- (d) Suponha agora que alguém use, por engano, um c **menor** que $\max_j p_j/q_j$, e que o código aceite sempre que $U \leq p_j/(c q_j)$ — o que para alguns j significa aceitar com probabilidade 1. Mostre que a distribuição gerada passa a ter probabilidades proporcionais a $q_j \min\{1, p_j/(c q_j)\}$, e verifique empiricamente com $c = 1$ no exemplo do capítulo. Quais valores do suporte ficam sub-representados?

54 Método da Rejeição para Variáveis Contínuas

No capítulo anterior usamos o método da rejeição para gerar v.a. discretas: sorteávamos candidatos a partir de uma distribuição fácil de simular e aceitávamos cada candidato com probabilidade proporcional à razão entre o alvo e a proposta. A versão contínua é a mesma ideia, palavra por palavra, trocando as funções de probabilidade p_j e q_j por densidades f e g .

Aqui o método é ainda mais valioso. No Capítulo 4 vimos que a inversão exige uma fórmula explícita para F^{-1} , e avisamos que isso é a exceção: para a normal, por exemplo, nem mesmo F tem forma fechada. O método da rejeição não precisa de F nem de F^{-1} — basta saber **calcular** a densidade alvo em um ponto e saber **simular** de alguma outra densidade parecida com ela. Como veremos no Exemplo 2, isso já é suficiente para gerar valores da normal.

54.1 Algoritmo

Seja X uma v.a. contínua com densidade f ; essa é a **distribuição alvo**. O método supõe que sabemos simular uma segunda v.a. Y , com densidade g , chamada de **distribuição proposta**.

 **Atenção:** a proposta precisa cobrir o suporte do alvo

É indispensável que $g(x) > 0$ sempre que $f(x) > 0$. Se a proposta nunca sugere valores de uma região onde o alvo tem densidade positiva, essa região jamais aparecerá na amostra — e o algoritmo devolve, silenciosamente, uma distribuição errada.

Além disso, supomos conhecida uma constante c tal que

$$\frac{f(x)}{g(x)} \leq c \quad \text{para todo } x \text{ com } f(x) > 0.$$

Exatamente como no caso discreto, necessariamente $c \geq 1$: integrando a desigualdade $f(x) \leq c g(x)$ sobre o conjunto onde f é positiva, obtemos

$$1 = \int_{\{f>0\}} f(x) dx \leq c \int_{\{f>0\}} g(x) dx \leq c,$$

pois a integral de g sobre um subconjunto da reta é, no máximo, 1.

💡 Pseudo-algoritmo: rejeição para v.a. contínuas

1. Gere Y a partir da densidade proposta g .
2. Gere $U \sim \text{Unif}(0, 1)$, independente de Y .
3. Se $U \leq \frac{f(Y)}{c g(Y)}$, devolva $X = Y$. Caso contrário, descarte Y e volte ao passo 1.

Como no caso discreto, o passo 3 é apenas um sorteio de Bernoulli: aceitamos o candidato com probabilidade $f(Y)/(c g(Y))$, e é a escolha de c que garante que esse número esteja entre 0 e 1.

54.2 Interpretação geométrica

A figura a seguir ilustra o passo 3 no caso em que a proposta é uma $\text{Unif}(0, 1)$, de modo que $c g(x)$ é uma reta horizontal na altura c . Sorteado um candidato Y , olhamos a haste vertical que vai de 0 até $c g(Y)$: a parte verde (abaixo de $f(Y)$) corresponde a aceitar, e a parte vermelha a rejeitar. O papel de U é sortear um ponto uniformemente ao longo dessa haste.

55 R

```
library(ggplot2)

# Densidade alvo:  $f(x) = 20 x (1-x)^3$ , para  $0 < x < 1$ 
f <- function(x) 20 * x * (1 - x)^3

# Densidade proposta: Unif(0,1). O rep() faz a função servir tanto para um
# número quanto para um vetor de valores
g <- function(x) rep(1, length(x))

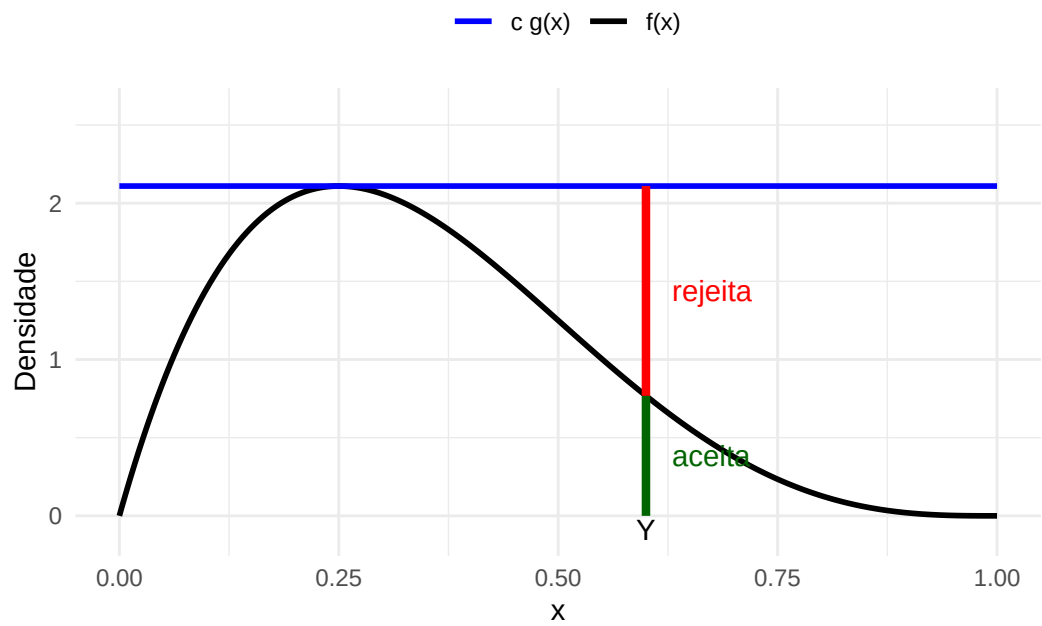
# Menor constante com  $f(x) \leq cte * g(x)$ . Chamamos de `cte` (e não de `c`)
# para não esconder a função c() do R
cte <- 135 / 64

# Candidato usado apenas para ilustrar o sorteio de aceitação
Y <- 0.6

grade <- seq(0, 1, length.out = 400)
df_curvas <- data.frame(x = grade, f = f(grade), teto = cte * g(grade))

ggplot(df_curvas, aes(x = x)) +
  geom_line(aes(y = f, color = "f(x)", linewidth = 1) +
  geom_line(aes(y = teto, color = "c g(x)", linewidth = 1) +
  # A haste em Y vai de 0 até o teto c*g(Y); abaixo de f(Y) aceitamos
  annotate("segment", x = Y, xend = Y, y = 0, yend = f(Y),
    color = "darkgreen", linewidth = 1.5) +
  annotate("segment", x = Y, xend = Y, y = f(Y), yend = cte * g(Y),
    color = "red", linewidth = 1.5) +
  annotate("text", x = Y + 0.03, y = f(Y) / 2, label = "aceita",
    color = "darkgreen", hjust = 0) +
  annotate("text", x = Y + 0.03, y = (f(Y) + cte) / 2, label = "rejeita",
    color = "red", hjust = 0) +
  annotate("text", x = Y, y = -0.09, label = "Y") +
  scale_color_manual(values = c("f(x)" = "black", "c g(x)" = "blue"), name = "") +
  coord_cartesian(ylim = c(-0.12, 2.6)) +
```

```
labs(x = "x", y = "Densidade") +
theme_minimal() +
theme(legend.position = "top")
```



56 Python

```
import numpy as np
import matplotlib.pyplot as plt

# Densidade alvo:  $f(x) = 20 x (1-x)^3$ , para  $0 < x < 1$ 
def f(x):
    return 20 * x * (1 - x)**3

# Densidade proposta: Unif(0,1). O ones_like faz a função servir tanto para um
# número quanto para um vetor de valores
def g(x):
    return np.ones_like(x, dtype=float)

# Menor constante com  $f(x) \leq cte * g(x)$ 
cte = 135 / 64

# Candidato usado apenas para ilustrar o sorteio de aceitação
Y = 0.6

grade = np.linspace(0, 1, 400)

plt.figure(figsize=(8, 5))
plt.plot(grade, f(grade), color="black", label="f(x)")
plt.plot(grade, cte * g(grade), color="blue", label="c g(x)")

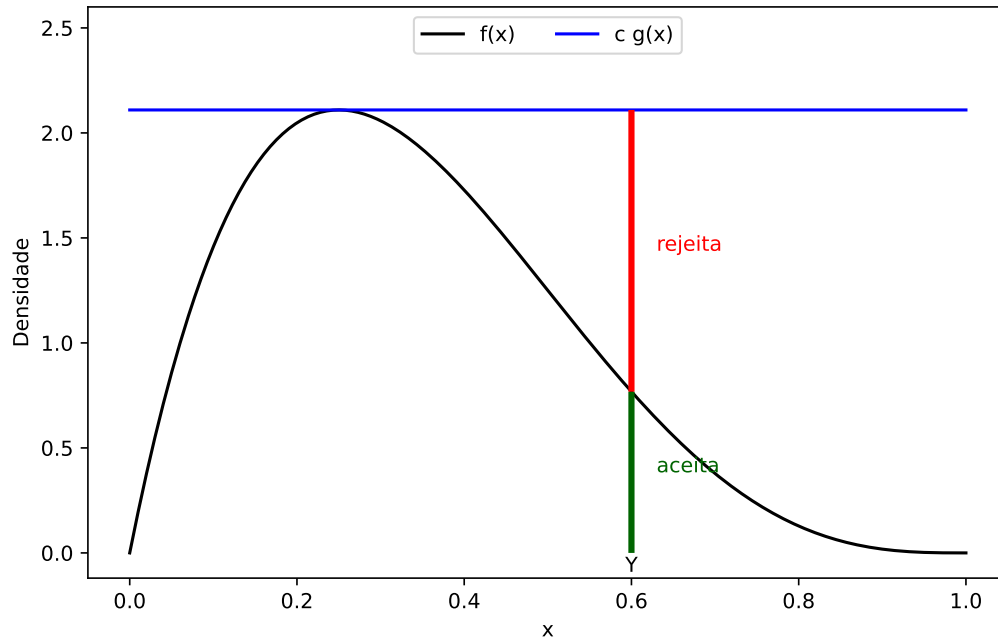
# A haste em Y vai de 0 até o teto  $c * g(Y)$ ; abaixo de  $f(Y)$  aceitamos
plt.vlines(Y, 0, f(Y), color="darkgreen", linewidth=3)
plt.vlines(Y, f(Y), cte * g(Y), color="red", linewidth=3)

plt.text(Y + 0.03, f(Y) / 2, "aceita", color="darkgreen")
plt.text(Y + 0.03, (f(Y) + cte) / 2, "rejeita", color="red")
plt.text(Y, -0.09, "Y", horizontalalignment="center")

plt.ylim(-0.12, 2.6)
```

(-0.12, 2.6)

```
plt.xlabel("x")
plt.ylabel("Densidade")
plt.legend(loc="upper center", ncol=2)
plt.show()
```



Essa leitura sugere uma segunda maneira de enxergar o método, que costuma ser mais esclarecedora do que a conta. Chame de $V = U \cdot c g(Y)$ a altura sorteada na haste. O par (Y, V) é um ponto sorteado uniformemente na região abaixo da curva $c g$, e o passo 3 mantém apenas os pontos que caem abaixo de f . O gráfico abaixo mostra 500 candidatos e o que acontece com cada um deles.

57 R

```
set.seed(42)

n_candidatos <- 500

# Passo 1: candidatos da proposta Unif(0,1)
Y <- runif(n_candidatos)

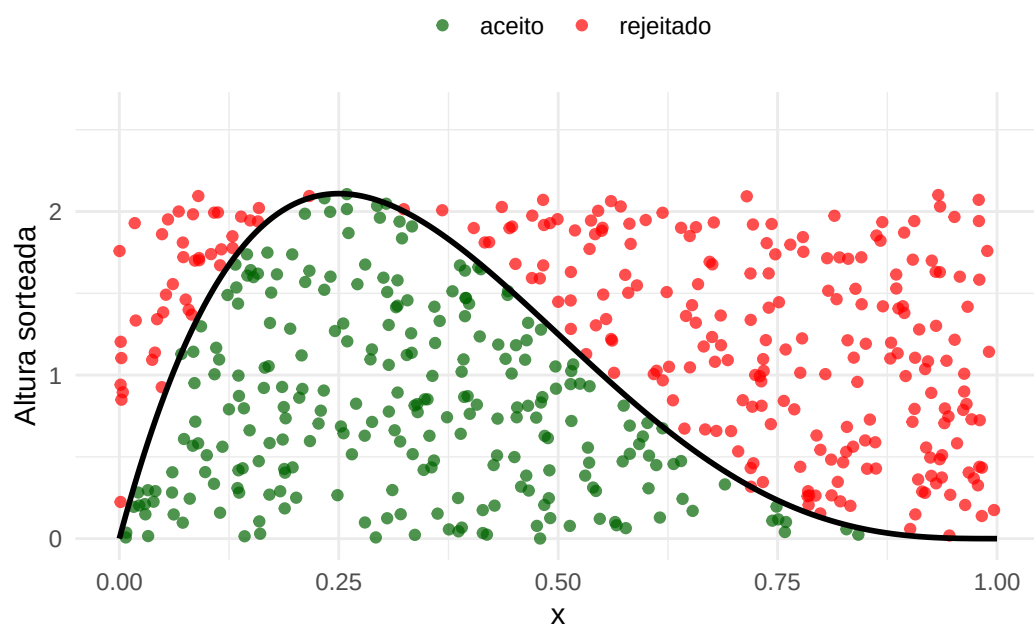
# Passo 2: os uniformes que decidem a aceitação
U <- runif(n_candidatos)

# Altura sorteada na haste: um ponto uniforme entre 0 e c*g(Y)
V <- U * cte * g(Y)

# Passo 3: ficamos com os pontos que caíram abaixo da curva f
aceito <- V <= f(Y)

df_pontos <- data.frame(
  x = Y,
  altura = V,
  status = ifelse(aceito, "aceito", "rejeitado")
)

ggplot(df_pontos, aes(x = x, y = altura, color = status)) +
  geom_point(size = 1.5, alpha = 0.7) +
  geom_line(data = df_curvas, aes(x = x, y = f), inherit.aes = FALSE,
            linewidth = 1) +
  scale_color_manual(values = c("aceito" = "darkgreen", "rejeitado" = "red"),
                     name = "") +
  coord_cartesian(ylim = c(0, 2.6)) +
  labs(x = "x", y = "Altura sorteada") +
  theme_minimal() +
  theme(legend.position = "top")
```



58 Python

```
np.random.seed(42)

n_candidatos = 500

# Passo 1: candidatos da proposta Unif(0,1)
Y = np.random.uniform(0, 1, n_candidatos)

# Passo 2: os uniformes que decidem a aceitação
U = np.random.uniform(0, 1, n_candidatos)

# Altura sorteada na haste: um ponto uniforme entre 0 e c*g(Y)
V = U * cte * g(Y)

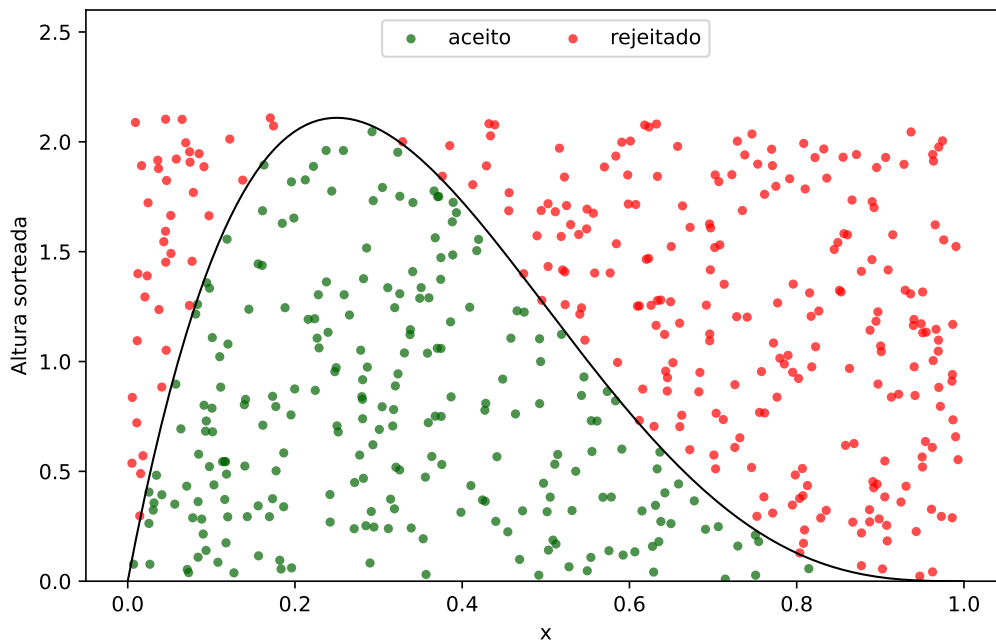
# Passo 3: ficamos com os pontos que caíram abaixo da curva f
aceito = V <= f(Y)

plt.figure(figsize=(8, 5))
plt.scatter(Y[aceito], V[aceito], s=12, alpha=0.7, color="darkgreen",
            label="aceito")
plt.scatter(Y[~aceito], V[~aceito], s=12, alpha=0.7, color="red",
            label="rejeitado")
plt.plot(grade, f(grade), color="black", linewidth=1)

plt.ylim(0, 2.6)
```

(0.0, 2.6)

```
plt.xlabel("x")
plt.ylabel("Altura sorteada")
plt.legend(loc="upper center", ncol=2)
plt.show()
```



i Outra maneira de ver o método

Os pontos verdes estão espalhados uniformemente na região sob a curva f . E um ponto uniforme sob o gráfico de uma densidade tem uma propriedade notável: sua **abscissa** tem exatamente essa densidade. A razão é que a região é mais alta onde f é maior, então mais pontos caem sobre esses valores de x — na proporção certa.

Ou seja, o método da rejeição pode ser lido assim: *para simular de f , sorteie um ponto uniforme sob o gráfico de f e devolva sua coordenada horizontal*. A proposta g e a constante c servem só para construir uma região maior, e fácil de sortear, que contenha a região de interesse. O Exercício 8 pede a demonstração desse fato.

58.1 Exemplo 1: uma densidade em $(0, 1)$ com proposta uniforme

Queremos gerar valores da densidade

$$f(x) = 20x(1-x)^3, \quad 0 < x < 1,$$

que é a densidade de uma $\text{Beta}(2, 4)$. Como o suporte é o intervalo $(0, 1)$, a escolha mais simples de proposta é $g(x) = 1$ para $0 < x < 1$, isto é, $Y \sim \text{Unif}(0, 1)$.

Nesse caso a razão $f(x)/g(x)$ é a própria $f(x)$, e a menor constante possível é o valor máximo da densidade. Derivando,

$$f'(x) = 20 [(1-x)^3 - 3x(1-x)^2] = 20(1-x)^2(1-4x),$$

que se anula em $x = 1/4$ (e em $x = 1$, onde f vale zero). Logo

$$c = f(1/4) = 20 \cdot \frac{1}{4} \cdot \left(\frac{3}{4}\right)^3 = \frac{135}{64} \approx 2,11.$$

💡 Pseudo-algoritmo: Beta(2,4) com proposta uniforme

1. Gere $Y \sim \text{Unif}(0, 1)$.
2. Gere $U \sim \text{Unif}(0, 1)$.
3. Se $U \leq \frac{20Y(1-Y)^3}{135/64}$, devolva $X = Y$. Caso contrário, volte ao passo 1.

O código a seguir gera $B = 1000$ valores. Além das amostras aceitas, contamos **quantos candidatos foram necessários**, para comparar a taxa de aceitação observada com o valor teórico $1/c$.

59 R

```
library(ggplot2)

set.seed(42)

f <- function(x) 20 * x * (1 - x)^3
g <- function(x) 1          # densidade da Unif(0,1)
cte <- 135 / 64

B <- 1000                  # quantos valores queremos gerar
amostras <- c()            # guarda os valores aceitos
n_propostas <- 0           # conta quantas vezes o passo 1 foi executado

while (length(amostras) < B) {
  # Passo 1: gerar Y da proposta Unif(0,1)
  Y <- runif(1)
  n_propostas <- n_propostas + 1

  # Passo 2: gerar U ~ Unif(0,1)
  U <- runif(1)

  # Passo 3: aceitar Y com probabilidade f(Y) / (cte * g(Y))
  if (U <= f(Y) / (cte * g(Y))) {
    amostras <- c(amostras, Y)
  }
}

cat("Candidatos gerados:", n_propostas, "\n")
```

Candidatos gerados: 2081

```
cat("Taxa de aceitação observada:", round(B / n_propostas, 3), "\n")
```

Taxa de aceitação observada: 0.481

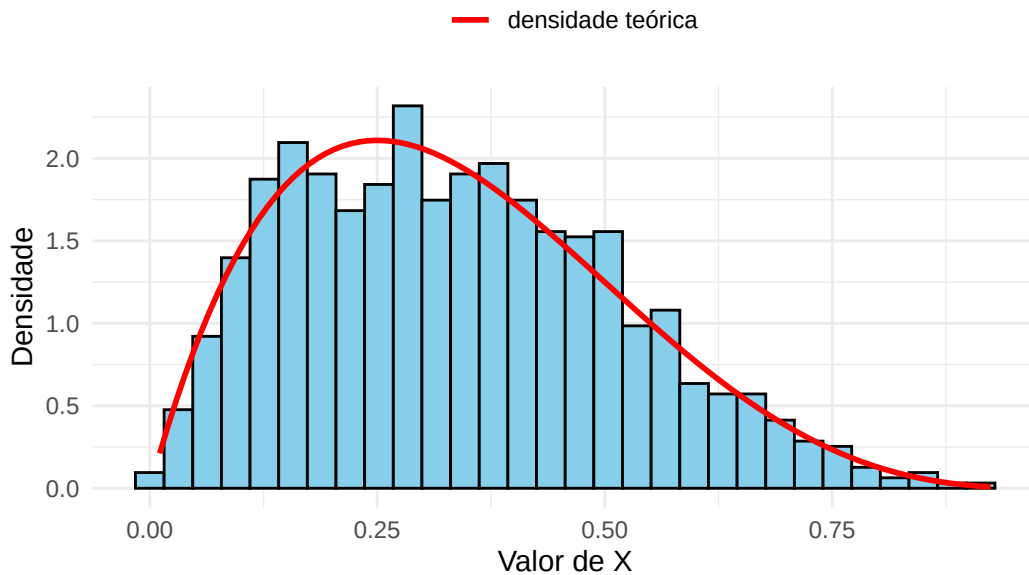

```
cat("Taxa de aceitação teórica (1/c):", round(1 / cte, 3), "\n")
```

Taxa de aceitação teórica (1/c): 0.474

```
df <- data.frame(x = amostras)

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "skyblue", color = "black") +
  # A curva vermelha é a densidade alvo: o histograma deve acompanhá-la
  stat_function(aes(color = "densidade teórica"), fun = f, linewidth = 1) +
  scale_color_manual(values = c("densidade teórica" = "red"), name = "") +
  labs(title = "Rejeição para  $f(x) = 20x(1-x)^3$ ",
       x = "Valor de X", y = "Densidade") +
  theme_minimal() +
  theme(legend.position = "top")
```

Rejeição para $f(x) = 20x(1-x)^3$



60 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def f(x):
    return 20 * x * (1 - x)**3

def g(x):
    return 1.0          # densidade da Unif(0,1)

cte = 135 / 64

B = 1000                # quantos valores queremos gerar
amostras = []           # guarda os valores aceitos
n_propostas = 0         # conta quantas vezes o passo 1 foi executado

while len(amostras) < B:
    # Passo 1: gerar Y da proposta Unif(0,1)
    Y = np.random.uniform(0, 1)
    n_propostas += 1

    # Passo 2: gerar U ~ Unif(0,1)
    U = np.random.uniform(0, 1)

    # Passo 3: aceitar Y com probabilidade f(Y) / (cte * g(Y))
    if U <= f(Y) / (cte * g(Y)):
        amostras.append(Y)

amostras = np.array(amostras)

print("Candidatos gerados:", n_propostas)
```

Candidatos gerados: 2084

```
print("Taxa de aceitação observada:", round(B / n_propostas, 3))
```

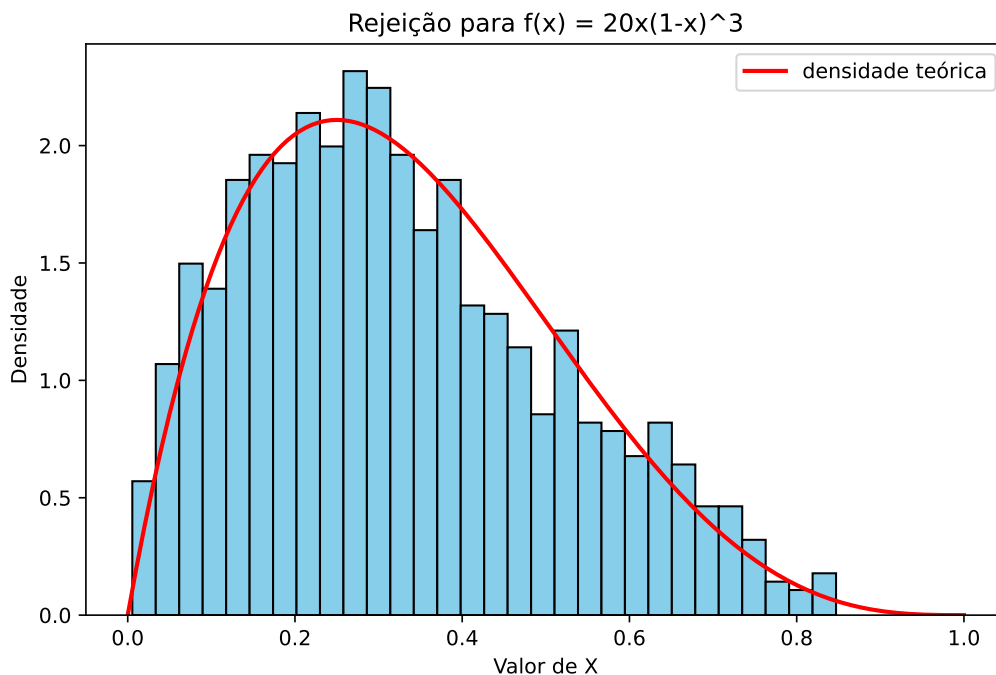
Taxa de aceitação observada: 0.48

```
print("Taxa de aceitação teórica (1/c):", round(1 / cte, 3))
```

Taxa de aceitação teórica (1/c): 0.474

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, 1, 400)

plt.figure(figsize=(8, 5))
plt.hist(amostras, bins=30, density=True, color="skyblue", edgecolor="black")
# A curva vermelha é a densidade alvo: o histograma deve acompanhá-la
plt.plot(grade, f(grade), color="red", linewidth=2, label="densidade teórica")
plt.title("Rejeição para  $f(x) = 20x(1-x)^3$ ")
plt.xlabel("Valor de X")
plt.ylabel("Densidade")
plt.legend()
plt.show()
```



60.1 Exemplo 2: gerando uma normal a partir de exponenciais

Este é o exemplo que a inversão não conseguia resolver. Queremos gerar $X \sim N(0, 1)$, cuja f.d.a. não tem forma fechada — muito menos sua inversa.

A ideia é gerar primeiro o **módulo** $|X|$ e depois sortear o sinal. Se $X \sim N(0, 1)$, a densidade de $|X|$ é o dobro da densidade da normal restrita aos valores positivos:

$$f(x) = \sqrt{\frac{2}{\pi}} e^{-x^2/2}, \quad x > 0.$$

Como proposta, usamos $Y \sim \text{Exp}(1)$, com densidade $g(x) = e^{-x}$ para $x > 0$ — uma distribuição que já sabemos simular por inversão (Capítulo 4) e que, como o alvo, tem suporte em $(0, \infty)$ e cai a zero na cauda direita. A razão entre as duas densidades é

$$\frac{f(x)}{g(x)} = \sqrt{\frac{2}{\pi}} e^{x-x^2/2}.$$

O expoente $x - x^2/2$ é máximo em $x = 1$, onde vale $1/2$. Portanto

$$c = \sqrt{\frac{2}{\pi}} e^{1/2} = \sqrt{\frac{2e}{\pi}} \approx 1,3155,$$

e a taxa de aceitação é $1/c \approx 0,76$: descartamos menos de um quarto dos candidatos.

O critério de aceitação fica particularmente simples. Substituindo c ,

$$\frac{f(Y)}{c g(Y)} = \frac{\sqrt{2/\pi} e^{Y-Y^2/2}}{\sqrt{2/\pi} e^{1/2}} = e^{-(Y-1)^2/2},$$

uma expressão que não envolve nem π nem raízes.

💡 Pseudo-algoritmo: normal padrão por rejeição

1. Gere $Y \sim \text{Exp}(1)$.
2. Gere $U \sim \text{Unif}(0, 1)$.
3. Se $U > e^{-(Y-1)^2/2}$, volte ao passo 1. Caso contrário, siga: Y é um valor de $|X|$.
4. Sorteie um sinal S , igual a $+1$ ou -1 com probabilidade $1/2$ cada, e devolva $X = S \cdot Y$.

O passo 4 é o que transforma o módulo em uma normal: como a densidade da $N(0,1)$ é simétrica em torno de zero, metade da massa está de cada lado.

61 R

```
library(ggplot2)

set.seed(42)

# A constante c só é usada no relatório final: o critério do passo 3 já está
# escrito na forma simplificada
cte <- sqrt(2 * exp(1) / pi)

B <- 5000          # quantos valores queremos gerar
modulos <- c()     # guarda os valores aceitos de |X|
n_propostas <- 0

while (length(modulos) < B) {
  # Passo 1: Y ~ Exp(1), gerada por inversão (Capítulo 4)
  U1 <- runif(1)
  Y <- -log(1 - U1)
  n_propostas <- n_propostas + 1

  # Passo 2: o uniforme que decide a aceitação
  U <- runif(1)

  # Passo 3: a razão f(Y) / (c * g(Y)) se simplifica para exp(-(Y-1)^2 / 2)
  if (U <= exp(-(Y - 1)^2 / 2)) {
    modulos <- c(modulos, Y)
  }
}

# Passo 4: cada módulo recebe um sinal + ou - com probabilidade 1/2
sinais <- sample(c(-1, 1), size = B, replace = TRUE)
X <- sinais * modulos

cat("Candidatos gerados:", n_propostas, "\n")
```

Candidatos gerados: 6610

```
cat("Taxa de aceitação observada:", round(B / n_propostas, 3), "\n")
```

Taxa de aceitação observada: 0.756

```
cat("Taxa de aceitação teórica (1/c):", round(1 / cte, 3), "\n")
```

Taxa de aceitação teórica (1/c): 0.76

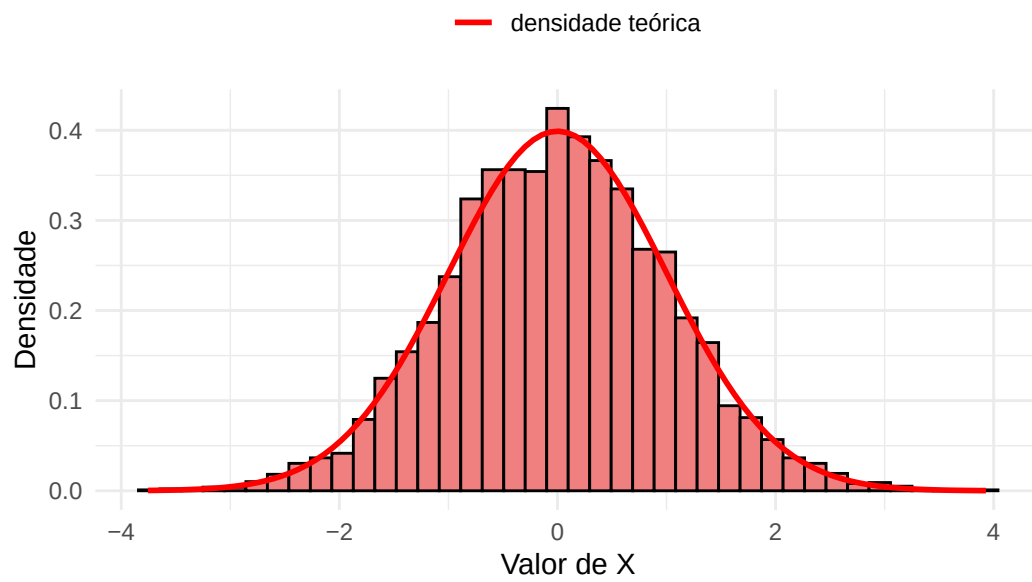
```
cat("Média e desvio padrão amostrais:", round(mean(X), 3), round(sd(X), 3), "\n")
```

Média e desvio padrão amostrais: 0 1.01

```
df <- data.frame(x = X)

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                 fill = "lightcoral", color = "black") +
  # A curva vermelha é a densidade da N(0,1), o alvo deste exemplo
  stat_function(aes(color = "densidade teórica"), fun = dnorm, linewidth = 1) +
  scale_color_manual(values = c("densidade teórica" = "red"), name = "") +
  labs(title = "Normal padrão gerada por rejeição",
       x = "Valor de X", y = "Densidade") +
  theme_minimal() +
  theme(legend.position = "top")
```

Normal padrão gerada por rejeição



62 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)

# A constante c só é usada no relatório final: o critério do passo 3 já está
# escrito na forma simplificada
cte = np.sqrt(2 * np.exp(1) / np.pi)

B = 5000          # quantos valores queremos gerar
modulos = []      # guarda os valores aceitos de |X|
n_propostas = 0

while len(modulos) < B:
    # Passo 1:  $Y \sim \text{Exp}(1)$ , gerada por inversão (Capítulo 4)
    U1 = np.random.uniform(0, 1)
    Y = -np.log(1 - U1)
    n_propostas += 1

    # Passo 2: o uniforme que decide a aceitação
    U = np.random.uniform(0, 1)

    # Passo 3: a razão  $f(Y) / (c * g(Y))$  se simplifica para  $\exp(-(Y-1)^2 / 2)$ 
    if U <= np.exp(-(Y - 1)**2 / 2):
        modulos.append(Y)

modulos = np.array(modulos)

# Passo 4: cada módulo recebe um sinal + ou - com probabilidade 1/2
sinais = np.random.choice([-1, 1], size=B)
X = sinais * modulos

print("Candidatos gerados:", n_propostas)
```

Candidatos gerados: 6558

```
print("Taxa de aceitação observada:", round(B / n_propostas, 3))
```

Taxa de aceitação observada: 0.762

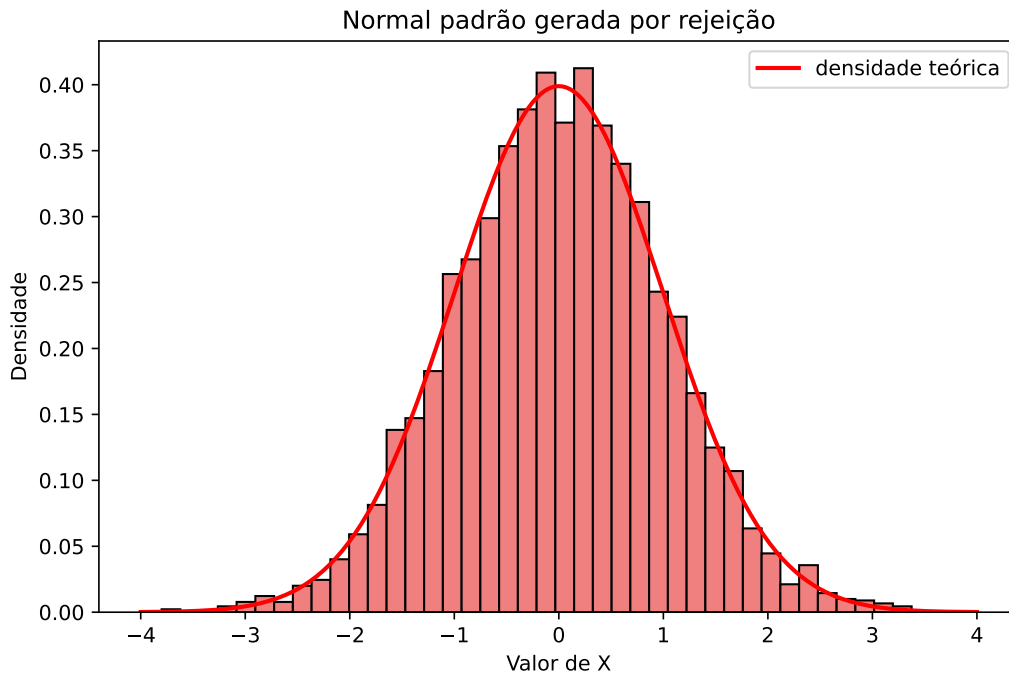
```
print("Taxa de aceitação teórica (1/c):", round(1 / cte, 3))
```

Taxa de aceitação teórica (1/c): 0.76

```
print("Média e desvio padrão amostrais:",  
      round(X.mean(), 3), round(X.std(ddof=1), 3))
```

Média e desvio padrão amostrais: 0.008 1.002

```
# Malha usada só para desenhar a densidade teórica  
grade = np.linspace(-4, 4, 400)  
  
plt.figure(figsize=(8, 5))  
plt.hist(X, bins=40, density=True, color="lightcoral", edgecolor="black")  
# A curva vermelha é a densidade da N(0,1), o alvo deste exemplo  
plt.plot(grade, norm.pdf(grade), color="red", linewidth=2,  
        label="densidade teórica")  
plt.title("Normal padrão gerada por rejeição")  
plt.xlabel("Valor de X")  
plt.ylabel("Densidade")  
plt.legend()  
plt.show()
```



62.1 Por que o método funciona

! Proposição

Suponha que $g(x) > 0$ sempre que $f(x) > 0$ e que $f(x)/g(x) \leq c$ para todo x com $f(x) > 0$. Então:

- (i) o valor X devolvido pelo método da rejeição tem densidade f ;
- (ii) o número N de candidatos gerados até o primeiro aceite tem distribuição geométrica com probabilidade de sucesso $1/c$. Em particular, $\mathbb{E}[N] = c$.

i Demonstração

Seja Y o candidato gerado em um passo qualquer do algoritmo e U o uniforme correspondente. Condicionando no valor de Y , a probabilidade de aceitar é

$$\mathbb{P}\left(U \leq \frac{f(Y)}{c g(Y)} \mid Y = y\right) = \frac{f(y)}{c g(y)},$$

porque $\mathbb{P}(U \leq a) = a$ para $a \in [0, 1]$. Logo, para qualquer x ,

$$\begin{aligned}\mathbb{P}(Y \leq x, \text{ aceitar } Y) &= \int_{-\infty}^x \frac{f(y)}{c g(y)} g(y) dy \\ &= \frac{1}{c} \int_{-\infty}^x f(y) dy = \frac{F(x)}{c},\end{aligned}$$

em que F é a f.d.a. do alvo. Fazendo $x \rightarrow \infty$ e usando que $F(\infty) = 1$,

$$\mathbb{P}(\text{aceitar}) = \frac{1}{c}.$$

Como os passos do algoritmo são independentes e cada um aceita com probabilidade $1/c$, o número N de candidatos até o primeiro aceite tem distribuição geométrica com probabilidade de sucesso $1/c$, e portanto $\mathbb{E}[N] = c$. Isso prova (ii).

Para (i), o valor devolvido é o candidato do passo em que houve aceite, ou seja, Y condicionado ao aceite. Pela definição de probabilidade condicional,

$$\mathbb{P}(X \leq x) = \mathbb{P}(Y \leq x \mid \text{aceitar}) = \frac{\mathbb{P}(Y \leq x, \text{ aceitar})}{\mathbb{P}(\text{aceitar})} = \frac{F(x)/c}{1/c} = F(x).$$

Portanto X tem f.d.a. F , isto é, densidade f . $\square$

Repare que a demonstração usa f e g apenas através da razão f/g . Isso tem uma consequência prática grande, explorada no Exercício 3: se conhecemos o alvo apenas **a menos de uma constante de normalização** — sabemos calcular f^* , mas não a constante k com $f = kf^*$ — o método continua funcionando. Basta achar c' com $f^*(x) \leq c'g(x)$ e usar o critério $U \leq f^*(Y)/(c'g(Y))$, que é idêntico ao original com $c = kc'$.

62.2 Eficiência e escolha da distribuição proposta

O item (ii) diz que, para gerar **um** valor do alvo, o algoritmo gasta em média c candidatos; equivalentemente, a fração aceita é $1/c$. Toda a eficiência do método está nessa única constante, e c depende só de f e g — não do tamanho da amostra que queremos gerar.

Na leitura geométrica da seção anterior, $1/c$ é uma razão de áreas: a região sob f tem área 1, a região sob $c g$ tem área c , e aceitamos os pontos que caem na primeira. **Quanto mais justo o “envelope” $c g$ ficar em volta de f , menos área desperdiçada e mais eficiente o algoritmo.** No Exemplo 1, $c \approx 2,11$: o retângulo de altura 2,11 tem mais que o dobro da área sob f , e por isso rejeitamos mais da metade dos candidatos. No Exemplo 2 a proposta exponencial acompanha bem melhor o formato do alvo, e $c \approx 1,32$.

Há duas armadilhas frequentes na escolha de g .

⚠ Atenção: a proposta precisa ter caudas mais pesadas que o alvo

Se g vai a zero mais rápido que f quando $|x| \rightarrow \infty$, a razão f/g explode e **não existe** constante c válida. É o que acontece, por exemplo, ao tentar usar uma normal como proposta para uma Cauchy: a densidade da normal decai como $e^{-x^2/2}$, e a da Cauchy apenas como $1/x^2$. Na direção contrária — Cauchy como proposta para gerar uma normal — a razão é limitada e o método funciona bem (Exercício 6).

⚠ Atenção: uma proposta espalhada demais custa caro

Mesmo quando c existe, ele pode ser grande. No Exemplo 1, se usássemos $Y \sim \text{Unif}(0, 3)$ — que também cobre o suporte de f — teríamos $g(x) = 1/3$ e $c = 3 \times 135/64 \approx 6,33$: o mesmo resultado a um custo três vezes maior, porque dois terços dos candidatos cairiam fora de $(0, 1)$ e seriam rejeitados na certa.

Por fim, nem sempre o máximo de f/g sai no papel. Quando não sair, calcule a razão em uma grade fina de pontos, ou use uma rotina de otimização numérica (`optimize` em R, `scipy.optimize.minimize_scalar` em Python), como pede o Exercício 7.

62.3 Exercícios

Exercício 1. Seja $X \sim \text{Gama}(3/2, 1)$, com densidade

$$f(x) = \frac{1}{\Gamma(3/2)} x^{1/2} e^{-x}, \quad x > 0.$$

Use como proposta a distribuição $\text{Exp}(2/3)$.

Observação: o parâmetro $2/3$ foi escolhido para que a proposta tenha a mesma média do alvo: $\mathbb{E}[X] = 3/2 = \mathbb{E}[Y]$.

- (a) Mostre que a razão $f(x)/g(x)$ é máxima em $x = 3/2$ e calcule a constante c .
- (b) Escreva um pseudo-algoritmo para simular um valor de X .
- (c) Implemente uma função que gere **um** valor de X e devolva também quantos candidatos foram necessários. Compare esse número com $\mathbb{E}[N] = c$.
- (d) Implemente uma função que gere uma amostra de tamanho B de X , e use-a com $B = 5000$.
- (e) Qual foi o número médio de candidatos por valor gerado? Compare com c .

- (f) Compare o histograma dos valores simulados com a densidade de X (`dgamma` em R, `scipy.stats.gamma.pdf` em Python).

Exercício 2. Queremos gerar valores da densidade

$$f(x) = \frac{1}{2} \sin(x), \quad 0 \leq x \leq \pi.$$

- (a) Usando como proposta a $\text{Unif}(0, \pi)$, isto é, $g(x) = 1/\pi$, encontre a menor constante c tal que $f(x) \leq c g(x)$ para todo $x \in [0, \pi]$.
- (b) Implemente o método e gere $B = 5000$ valores. Compare o histograma com a densidade teórica e a taxa de aceitação observada com $1/c$.
- (c) Considere agora a proposta

$$g_2(x) = \frac{6x(\pi - x)}{\pi^3}, \quad 0 \leq x \leq \pi,$$

que é uma $\text{Beta}(2, 2)$ reescalada para o intervalo $[0, \pi]$. Verifique que g_2 integra 1 e mostre que a razão $f(x)/g_2(x)$ é máxima em $x = \pi/2$, com $c_2 = \pi/3$.

- (d) Compare $1/c$ com $1/c_2$. Qual das duas propostas é mais eficiente, e por quê? Olhe o formato das duas propostas ao lado do de f .
- (e) Para usar g_2 seria preciso saber simular dela. Explique por que isso não é imediato, e sugira um caminho (dica: qual método deste livro gera uma $\text{Beta}(2, 2)$?).

Exercício 3. Muitas vezes conhecemos a densidade alvo apenas **a menos de uma constante de normalização**: sabemos calcular f^* , mas não a constante k tal que $f = k f^*$. Considere

$$f^*(x) = e^{-x} \sin^2(x), \quad x > 0,$$

e use como proposta a $\text{Exp}(1)$, ou seja, $g(x) = e^{-x}$ para $x > 0$.

- (a) Mostre que $f^*(x)/g(x) = \sin^2(x)$ e conclua que $c' = 1$ serve.
- (b) Escreva o pseudo-algoritmo usando apenas f^* , g e c' , e explique por que ele não depende de k .
- (c) Implemente o algoritmo e gere $B = 5000$ valores. Construa o histograma de densidade.
- (d) Calcule k exatamente, usando que $\sin^2 x = (1 - \cos 2x)/2$ e que $\int_0^\infty e^{-x} \cos(2x) dx = 1/5$. Sobreponha $f = k f^*$ ao histograma do item (c).

- (e) (Desafio) Pela proposição, a taxa de aceitação estima $1/c = 1/(kc')$. Use a taxa observada no item (c) para construir uma estimativa $\hat{k}$ e compare com o valor exato. Repare que acabamos de estimar uma integral sem nunca tê-la calculado.

Exercício 4. Seja $Z \sim N(0, 1)$ e considere como alvo a distribuição de Z **condicionada** a $Z \geq 2$, cuja densidade é

$$f(x) = \frac{\varphi(x)}{1 - \Phi(2)}, \quad x \geq 2,$$

onde φ e Φ são a densidade e a f.d.a. da $N(0, 1)$.

- Usando a própria $N(0, 1)$ como proposta, mostre que $f(x)/\varphi(x) = 1/(1 - \Phi(2))$ para $x \geq 2$ e o caso contrário. Conclua que $c = 1/(1 - \Phi(2))$ e calcule seu valor (`pnorm` em R, `scipy.stats.norm.cdf` em Python).
- Mostre que, com esse c , a probabilidade de aceitação vale 1 quando $y \geq 2$ e 0 caso contrário — ou seja, o algoritmo se reduz a “gere normais até obter uma maior ou igual a 2”. Implemente e gere $B = 2000$ valores, registrando o número médio de candidatos.
- Considere agora a proposta **exponencial deslocada**, com densidade $g_\lambda(x) = \lambda e^{-\lambda(x-2)}$ para $x \geq 2$, que se simula fazendo $Y = 2 + \text{Exp}(\lambda)$. Mostre que $f(x)/g_\lambda(x) \propto e^{-x^2/2 + \lambda x}$ e que essa razão é limitada para todo $\lambda > 0$. Onde está o máximo quando $\lambda \geq 2$?
- Calcule $c(\lambda)$ para λ em uma grade de 2 a 5 e faça um gráfico de $c(\lambda)$ contra λ . Implemente o método com o λ que minimiza c e compare a taxa de aceitação com a do item (b).
- (Desafio) Mostre analiticamente que o λ ótimo é $\lambda^* = 1 + \sqrt{2}$. O que esse exercício sugere sobre usar rejeição para amostrar de eventos raros?

Exercício 5. Este exercício explora diretamente a leitura geométrica do método: em vez de uma densidade na reta, vamos sortear um ponto uniformemente em uma região do plano.

Queremos gerar um ponto (X_1, X_2) uniformemente distribuído no disco unitário $D = \{(x_1, x_2) : x_1^2 + x_2^2 \leq 1\}$, isto é, com densidade conjunta $f(x_1, x_2) = 1/\pi$ em D . Como proposta, sorteamos um ponto uniforme no quadrado $[-1, 1]^2$, cuja densidade conjunta é $g(x_1, x_2) = 1/4$.

- Mostre que $f/g = 4/\pi$ dentro do disco e 0 fora dele, e conclua que $c = 4/\pi$. Verifique que o critério do passo 3 se reduz a “aceite se o ponto caiu dentro do disco”.
- Implemente o algoritmo, gere $B = 5000$ pontos e faça um gráfico de dispersão.
- A taxa de aceitação é $1/c = \pi/4$. Use isso ao contrário: a partir da taxa observada, construa um estimador $\hat{\pi}$ e compare com o valor de π .

- (d) (Desafio) O mesmo raciocínio vale em d dimensões: sorteie um ponto no cubo $[-1, 1]^d$ e aceite se ele cair na bola unitária. O volume da bola é $V_d = \pi^{d/2}/\Gamma(d/2 + 1)$, de modo que a taxa de aceitação é $V_d/2^d$. Calcule essa taxa para $d = 2, 5, 10$ e 20 . O que isso diz sobre usar rejeição em dimensão alta?

Exercício 6. Vamos verificar a afirmação sobre caudas feita no texto. Sejam f a densidade da $N(0, 1)$ e g a densidade da Cauchy padrão,

$$g(x) = \frac{1}{\pi(1+x^2)}, \quad x \in \mathbb{R}.$$

- (a) Mostre que

$$\frac{f(x)}{g(x)} = \sqrt{\frac{\pi}{2}} (1+x^2) e^{-x^2/2}$$

e que essa razão é máxima em $x = \pm 1$, com $c = \sqrt{2\pi/e} \approx 1,52$.

- (b) Sabendo que a f.d.a. da Cauchy é $G(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$, mostre que $G^{-1}(u) = \tan(\pi(u - 1/2))$ e escreva o pseudo-algoritmo completo (inversão para a proposta, rejeição para o alvo).
- (c) Implemente e gere $B = 5000$ valores. Compare o histograma com a densidade da normal e a taxa de aceitação com $1/c$.
- (d) Troque os papéis: alvo Cauchy, proposta $N(0, 1)$. Mostre que a razão $f(x)/g(x)$ é ilimitada e que, portanto, não existe c válido. Faça um gráfico dessa razão para $x \in [0, 6]$ para visualizar o problema.
- (e) Compare a eficiência do item (c) com a do Exemplo 2 do capítulo. Qual das duas propostas se ajusta melhor à normal?

Exercício 7. Nos exemplos do capítulo conseguimos maximizar f/g no papel. Aqui não. Considere a densidade conhecida a menos de constante

$$f^*(x) = e^{-x^2/2} (\sin^2(6x) + 3x^2 \cos^2(x) + 1), \quad x \in \mathbb{R},$$

e use como proposta uma $N(0, 2^2)$, isto é, uma normal de média 0 e desvio padrão 2.

- (a) Faça um gráfico de f^* no intervalo $[-6, 6]$. Quantos modos ela tem? Convença-se de que inverter a f.d.a. aqui está fora de questão.
- (b) Escreva a razão $r(x) = f^*(x)/g(x)$ e mostre que ela é limitada. (Dica: compare o expoente $-x^2/2$ da alvo com o $-x^2/8$ da proposta.)

- (c) Calcule $r(x)$ em uma grade fina de $[-10, 10]$ e tome c' como o máximo observado. Confirme o resultado com `optimize` (R) ou `scipy.optimize.minimize_scalar` (Python).
- (d) Implemente o método e gere $B = 5000$ valores. Sobreponha ao histograma a curva f^* **reescalada** de modo a integrar 1 (você pode obter a constante por integração numérica, com `integrate` em R ou `scipy.integrate.quad` em Python).
- (e) Qual a taxa de aceitação observada? Explique por que a $N(0, 1)$ **não** serviria como proposta aqui.
- (f) (Desafio) Se a grade do item (c) fosse grosseira, c' poderia ficar **abaixo** do máximo verdadeiro. O que aconteceria com a distribuição gerada? (Compare com o Exercício 7(d) do capítulo anterior.)

Exercício 8. (Desafio) Este exercício formaliza a leitura geométrica do método. Seja f uma densidade e

$$A = \{(x, v) \in \mathbb{R}^2 : 0 \leq v \leq f(x)\}$$

a região sob o seu gráfico, que tem área $\int f(x) dx = 1$.

- (a) Suponha que (X, V) seja um ponto sorteado uniformemente em A . Mostre que a densidade marginal de X é f . (Dica: a densidade conjunta de (X, V) é constante e igual a 1 em A ; integre em v .)
- (b) No algoritmo do capítulo, seja Y o candidato e $V = U \cdot c g(Y)$. Mostre que (Y, V) é uniforme na região sob $c g$.
- (c) Conclua que, condicionado ao aceite, o par (Y, V) é uniforme em A e que, portanto, o valor devolvido tem densidade f — uma demonstração alternativa da parte (i) da proposição.
- (d) Interprete $1/c$ como razão entre as áreas de A e da região sob $c g$.
- (e) Volte ao Exemplo 2 e considere a proposta $\text{Exp}(\lambda)$, com $\lambda > 0$ qualquer. Mostre que

$$c(\lambda) = \sqrt{\frac{2}{\pi}} \frac{e^{\lambda^2/2}}{\lambda},$$

e que essa função é minimizada em $\lambda = 1$ — ou seja, a escolha feita no capítulo é a melhor possível dentro da família exponencial. Faça o gráfico de $c(\lambda)$ para confirmar.

63 Transformações e Misturas

Os dois métodos vistos até aqui partem sempre de uniformes: a inversão aplica F^{-1} a uma $\text{Unif}(0, 1)$, e a rejeição sorteia candidatos de uma distribuição proposta até aceitar um. Este capítulo trata de uma terceira estratégia, mais oportunista: **construir a variável que queremos a partir de variáveis que já sabemos simular**.

Duas construções cobrem a maior parte dos casos úteis:

- **transformação**: encontrar uma função g tal que $X = g(Y)$, com Y (possivelmente um vetor) fácil de simular;
- **mistura**: descrever a distribuição de X em dois estágios — primeiro sortecemos uma variável auxiliar Y , depois sortecemos X usando o valor obtido de Y .

Nenhuma das duas é automática: não existe receita que, dada uma densidade qualquer, produza a transformação ou a mistura correspondente. O trabalho é reconhecer a relação, e ela vem da teoria de probabilidade, não do computador. Em compensação, quando essa relação existe, é difícil ganhar dela: não há candidatos descartados como na rejeição, nem f.d.a. para inverter como na inversão.

63.1 Transformação de v.a.

A situação é a seguinte:

- queremos simular valores de uma v.a. X ;
- sabemos simular valores de uma v.a. Y ;
- conhecemos uma função g tal que X e $g(Y)$ têm a **mesma distribuição**.

💡 Pseudo-algoritmo: método da transformação

1. Simule um valor de Y .
2. Devolva $X = g(Y)$.

Não há muito o que provar sobre o algoritmo: se X e $g(Y)$ têm a mesma distribuição, então uma amostra de $g(Y)$ é, por definição, uma amostra de X . Toda a dificuldade está em achar g e mostrar essa igualdade em distribuição — e é isso que os exemplos deste capítulo fazem.

Vale notar que já usamos o método sem lhe dar nome: a própria inversão é o caso particular em que $Y \sim \text{Unif}(0, 1)$ e $g = F^{-1}$. Os dois primeiros exemplos deste capítulo são as transformações mais simples que existem — uma translação e uma soma — e servem para fixar a ideia antes dos casos mais elaborados.

O segundo deles mostra que g **não precisa ser função de uma variável só**: podemos tomar $Y = (Y_1, \dots, Y_n)$ e $g : \mathbb{R}^n \rightarrow \mathbb{R}$, desde que saibamos simular todas as coordenadas. Somas, máximos, quocientes e somas de quadrados são as transformações mais frequentes nessa forma.

Para verificar que $g(Y)$ tem a distribuição desejada no caso de uma variável só, o caminho padrão passa pela f.d.a.:

Densidade de uma transformação monótona

Seja Y uma v.a. contínua com densidade f_Y e g uma função estritamente crescente e derivável. Então $X = g(Y)$ tem f.d.a.

$$F_X(x) = \mathbb{P}(g(Y) \leq x) = \mathbb{P}(Y \leq g^{-1}(x)) = F_Y(g^{-1}(x)),$$

e, derivando em x ,

$$f_X(x) = f_Y(g^{-1}(x)) \frac{d}{dx} g^{-1}(x).$$

O passo-chave é o segundo: como g é crescente, o evento $\{g(Y) \leq x\}$ é exatamente o evento $\{Y \leq g^{-1}(x)\}$.

63.2 Exemplo 1: simulando $Y \sim \text{Unif}(1, 2)$

Para gerar valores de $Y \sim \text{Unif}(1, 2)$, usamos o fato de que Y é uma simples translação de $U \sim \text{Unif}(0, 1)$:

$$Y = U + 1.$$

Pseudo-algoritmo

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Retorne $Y = U + 1$.

64 R

```
library(ggplot2)

set.seed(42)

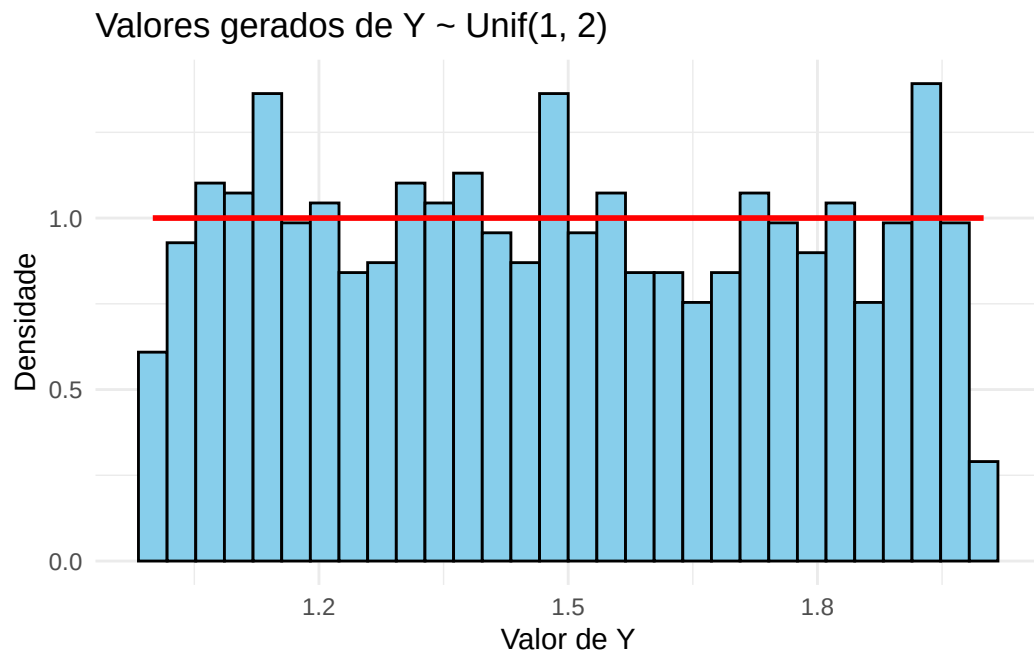
B <- 1000 # quantos valores queremos gerar

# Passo 1: gerar os uniformes
U <- runif(B, min = 0, max = 1)

# Passo 2: deslocar em uma unidade para obter  $Y \sim \text{Unif}(1, 2)$ 
Y <- U + 1

df <- data.frame(Y = Y)

# A densidade teórica é constante e igual a 1 no intervalo (1, 2)
ggplot(df, aes(x = Y)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "skyblue", color = "black") +
  annotate("segment", x = 1, xend = 2, y = 1, yend = 1,
          color = "red", linewidth = 1) +
  labs(title = "Valores gerados de  $Y \sim \text{Unif}(1, 2)$ ",
       x = "Valor de Y", y = "Densidade") +
  theme_minimal()
```



65 Python

```
import numpy as np
import matplotlib.pyplot as plt

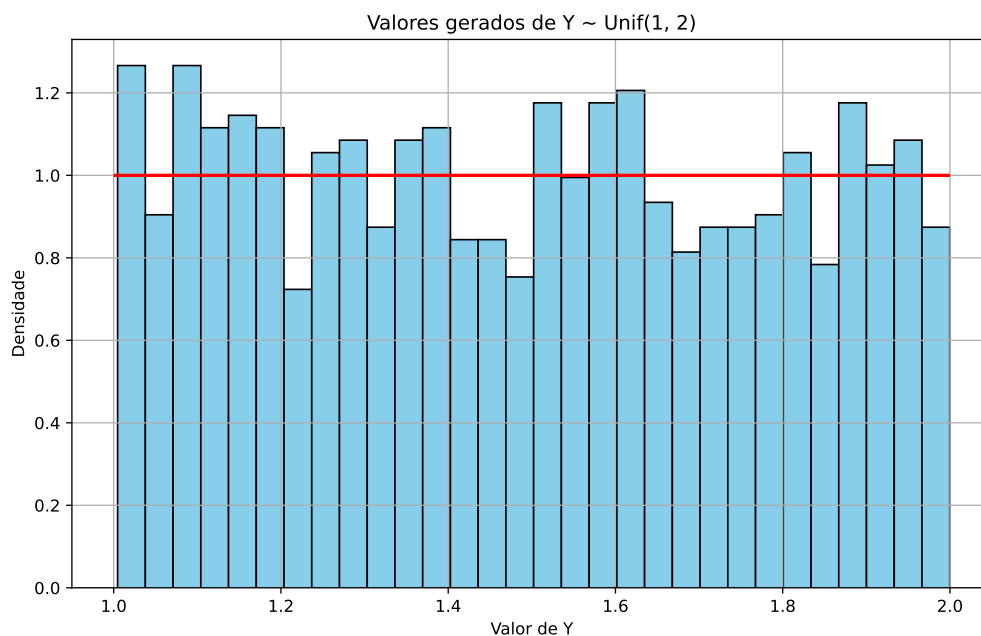
np.random.seed(42)

B = 1000 # quantos valores queremos gerar

# Passo 1: gerar os uniformes
U = np.random.uniform(0, 1, B)

# Passo 2: deslocar em uma unidade para obter  $Y \sim \text{Unif}(1, 2)$ 
Y = U + 1

# A densidade teórica é constante e igual a 1 no intervalo (1, 2)
plt.figure(figsize=(10, 6))
plt.hist(Y, bins=30, color='skyblue', edgecolor='black', density=True)
plt.hlines(1, 1, 2, colors='red', linewidth=2)
plt.title('Valores gerados de  $Y \sim \text{Unif}(1, 2)$ ')
plt.xlabel('Valor de Y')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



65.1 Exemplo 2: simulando $Y \sim \text{Gama}(n, \lambda)$

Aqui a transformação não é aplicada a um único valor, mas a vários: usamos o fato de que, se $X_1, \dots, X_n$ são independentes e $X_i \sim \text{Exp}(\lambda)$, então

$$Y = \sum_{i=1}^n X_i \sim \text{Gama}(n, \lambda).$$

Como já sabemos gerar exponenciais por inversão (Exemplo 2 do Capítulo 4), basta gerar n delas e somar.

💡 Pseudo-algoritmo: Gama com parâmetro de forma inteiro

1. Gere $U_1, \dots, U_n \sim \text{Unif}(0, 1)$ independentes.
2. Calcule $X_i = -\frac{\log(1 - U_i)}{\lambda}$, para $i = 1, \dots, n$.
3. Retorne $Y = X_1 + X_2 + \dots + X_n$.

Note que esse algoritmo só serve quando o parâmetro de forma n é um **inteiro** positivo: a soma de exponenciais tem que ter um número inteiro de parcelas.

Para gerar B valores de Y , precisamos de $B \times n$ uniformes. No código abaixo, guardamos esses uniformes em uma matriz com B linhas e n colunas: cada linha reúne as n exponenciais de um mesmo Y , e somar a linha produz um valor de Y .

66 R

```
library(ggplot2)

set.seed(42)

n <- 5      # parâmetro de forma da Gama (número de exponenciais somadas)
lambda <- 2 # parâmetro da distribuição exponencial
B <- 1000   # quantos valores de Y queremos gerar

# Passo 1: uma matriz de uniformes com B linhas e n colunas
U <- matrix(runif(B * n, min = 0, max = 1), nrow = B, ncol = n)

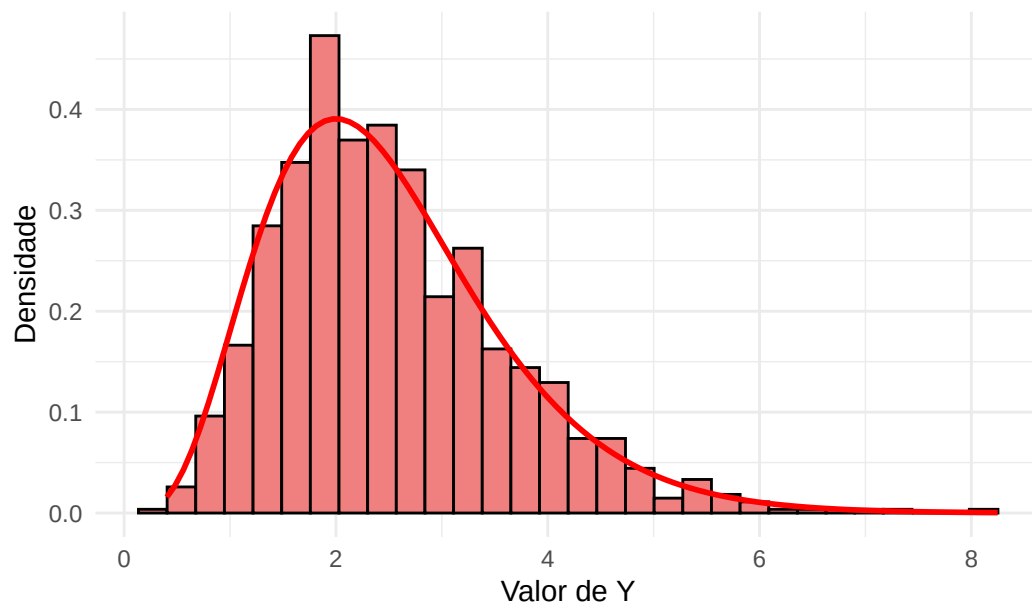
# Passo 2: a inversa da exponencial aplicada a cada entrada da matriz
X <- -log(1 - U) / lambda

# Passo 3: rowSums soma cada linha, devolvendo um vetor com os B valores de Y
Y <- rowSums(X)

df <- data.frame(Y = Y)

# dgamma é a densidade da Gama; shape é o parâmetro de forma e rate a taxa
ggplot(df, aes(x = Y)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "lightcoral", color = "black") +
  stat_function(fun = function(y) dgamma(y, shape = n, rate = lambda),
               color = "red", linewidth = 1) +
  labs(title = paste0("Valores gerados de Y ~ Gama(", n, ", ", lambda, ")"),
       x = "Valor de Y", y = "Densidade") +
  theme_minimal()
```

Valores gerados de $Y \sim \text{Gama}(5, 2)$



67 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gamma

np.random.seed(42)

n = 5      # parâmetro de forma da Gama (número de exponenciais somadas)
lambd = 2  # parâmetro da distribuição exponencial
B = 1000   # quantos valores de Y queremos gerar

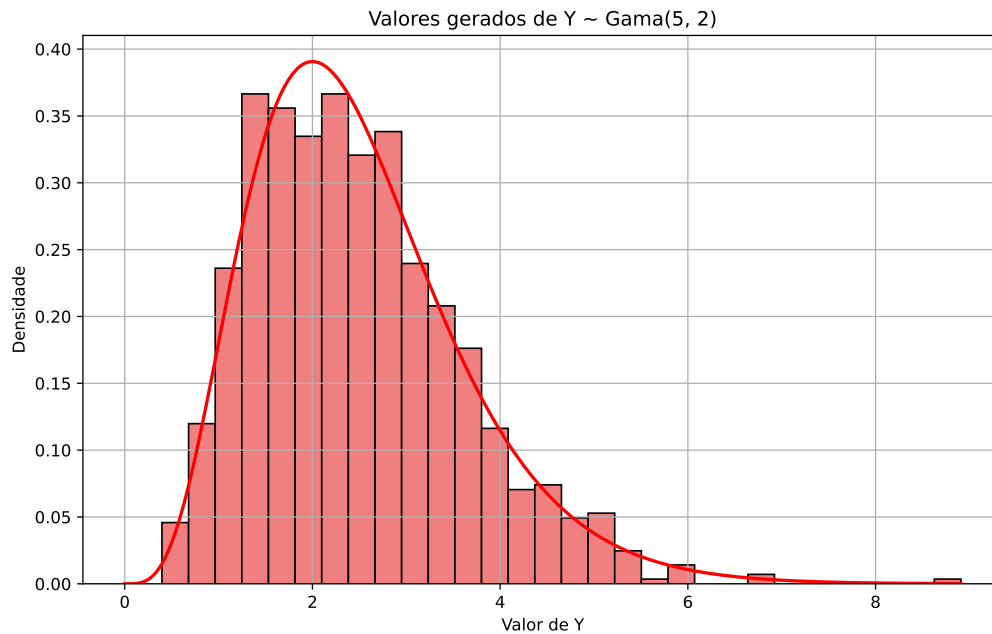
# Passo 1: uma matriz de uniformes com B linhas e n colunas
U = np.random.uniform(0, 1, (B, n))

# Passo 2: a inversa da exponencial aplicada a cada entrada da matriz
X = -np.log(1 - U) / lambd

# Passo 3: soma de cada linha (axis=1), devolvendo os B valores de Y
Y = np.sum(X, axis=1)

# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, max(Y), 200)

# gamma.pdf: a é o parâmetro de forma e scale = 1 / taxa
plt.figure(figsize=(10, 6))
plt.hist(Y, bins=30, color='lightcoral', edgecolor='black', density=True)
plt.plot(grade, gamma.pdf(grade, a=n, scale=1 / lambd), color='red', linewidth=2)
plt.title(f'Valores gerados de Y ~ Gama({n}, {lambd})')
plt.xlabel('Valor de Y')
plt.ylabel('Densidade')
plt.grid(True)
plt.show()
```



67.1 Exemplo 3: distribuição de Weibull

Seja X uma v.a. com distribuição de Weibull com parâmetro de forma $k > 0$ e parâmetro de escala $\lambda > 0$, cuja densidade é

$$f_X(x) = \frac{k}{\lambda^k} x^{k-1} e^{-(x/\lambda)^k}, \quad x > 0.$$

É uma distribuição muito usada em análise de sobrevivência e em engenharia de confiabilidade, para modelar tempos até a falha de um equipamento. O parâmetro k controla se a taxa de falha cresce ($k > 1$), decresce ($k < 1$) ou fica constante ($k = 1$) ao longo do tempo; quando $k = 1$, a Weibull é exatamente uma $\text{Exp}(1/\lambda)$.

Em vez de trabalhar diretamente com essa densidade, vamos escrever a Weibull como uma transformação simples de uma exponencial — distribuição que já sabemos simular desde o Capítulo 4.

! Proposição

Se $Y \sim \text{Exp}(1/\lambda^k)$, isto é, Y é exponencial com **taxa** $1/\lambda^k$, então

$$X = Y^{1/k} \sim \text{Weibull}(\lambda, k).$$

i Demonstração

A densidade de Y é

$$f_Y(y) = \frac{1}{\lambda^k} e^{-y/\lambda^k}, \quad y > 0.$$

A função $g(y) = y^{1/k}$ é estritamente crescente em $(0, \infty)$, com inversa $g^{-1}(x) = x^k$. Logo, para $x > 0$,

$$\mathbb{P}(X \leq x) = \mathbb{P}(Y^{1/k} \leq x) = \mathbb{P}(Y \leq x^k) = F_Y(x^k),$$

e, derivando em x ,

$$\begin{aligned} f_X(x) &= \frac{dF_Y(x^k)}{dx} = kx^{k-1} f_Y(x^k) \\ &= kx^{k-1} \frac{1}{\lambda^k} e^{-x^k/\lambda^k} \\ &= \frac{k}{\lambda^k} x^{k-1} e^{-(x/\lambda)^k}, \end{aligned}$$

que é exatamente a densidade da Weibull. $\square$

Falta simular Y . Pelo método da inversão, uma exponencial de taxa θ é gerada por $Y = -\log(1 - U)/\theta$; aqui $\theta = 1/\lambda^k$, de modo que $Y = -\lambda^k \log(1 - U)$.

i Trocar $1 - U$ por U

Se $U \sim \text{Unif}(0, 1)$, então $1 - U$ também é $\text{Unif}(0, 1)$. Por isso podemos escrever $Y = -\lambda^k \log U$ no lugar de $Y = -\lambda^k \log(1 - U)$: as duas expressões geram valores com a mesma distribuição (embora, para um mesmo U , produzam números diferentes).

💡 Pseudo-algoritmo: Weibull

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Faça $Y = -\lambda^k \log U$, de modo que $Y \sim \text{Exp}(1/\lambda^k)$.
3. Devolva $X = Y^{1/k}$.

Juntando os passos 2 e 3, o algoritmo inteiro cabe em uma linha: $X = \lambda(-\log U)^{1/k}$.

i Aqui, transformação e inversão coincidem

A f.d.a. da Weibull é $F(x) = 1 - e^{-(x/\lambda)^k}$; isolando x em $u = F(x)$, obtemos $F^{-1}(u) = \lambda(-\log(1-u))^{1/k}$, que é exatamente a expressão a que chegamos (com $1-U$ no lugar de U). Não é coincidência: quando g é monótona e Y é gerada por inversão, aplicar g dá no mesmo que inverter a f.d.a. de X .

A transformação só ganha da inversão quando F_X é intratável mas a relação entre X e Y é simples — como nos Exemplos 2 e 4, em que nem sequer há uma única variável Y a inverter.

O código a seguir gera $B = 5000$ valores com $k = 1,5$ e $\lambda = 2$, e compara o histograma com a densidade teórica. Como conferência adicional, comparamos a média amostral com a média teórica $\mathbb{E}[X] = \lambda \Gamma(1 + 1/k)$.

68 R

```
library(ggplot2)

set.seed(42)

k <- 1.5      # parâmetro de forma
lambda <- 2   # parâmetro de escala
B <- 5000     # quantos valores queremos gerar

# Passo 1: os uniformes
U <- runif(B)

# Passo 2:  $Y \sim \text{Exp}(1/\text{lambda}^k)$ , pelo método da inversão
Y <- -lambda^k * log(U)

# Passo 3: a transformação que leva a exponencial na Weibull
X <- Y^(1 / k)

# gamma() em R é a função Gama, e não a densidade da distribuição Gama
cat("Média amostral:", round(mean(X), 3), "\n")
```

Média amostral: 1.799

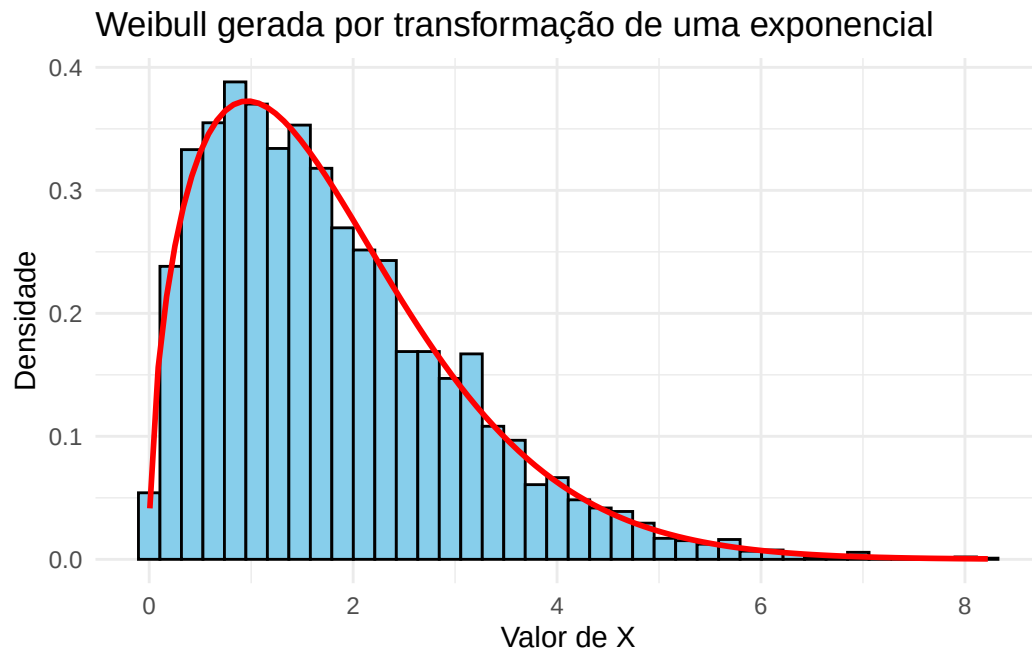
```
cat("Média teórica:", round(lambda * gamma(1 + 1 / k), 3), "\n")
```

Média teórica: 1.805

```
df <- data.frame(x = X)

# dweibull é a densidade da Weibull: shape é a forma e scale a escala
ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                 fill = "skyblue", color = "black") +
  stat_function(fun = function(x) dweibull(x, shape = k, scale = lambda),
```

```
color = "red", linewidth = 1) +  
labs(title = "Weibull gerada por transformação de uma exponencial",  
x = "Valor de X", y = "Densidade") +  
theme_minimal()
```



69 Python

```
import math
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import weibull_min

np.random.seed(42)

k = 1.5      # parâmetro de forma
lambd = 2    # parâmetro de escala
B = 5000     # quantos valores queremos gerar

# Passo 1: os uniformes
U = np.random.uniform(0, 1, B)

# Passo 2:  $Y \sim \text{Exp}(1/\lambda^k)$ , pelo método da inversão
Y = -lambd**k * np.log(U)

# Passo 3: a transformação que leva a exponencial na Weibull
X = Y**(1 / k)

# math.gamma é a função Gama, e não a densidade da distribuição Gama
print("Média amostral:", round(X.mean(), 3))
```

Média amostral: 1.821

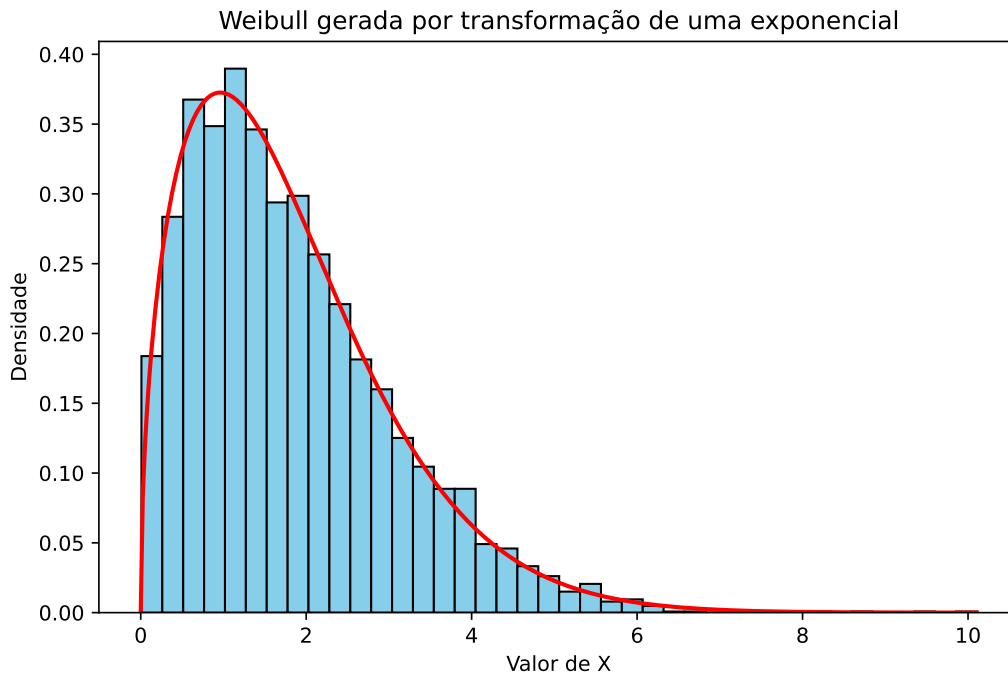
```
print("Média teórica:", round(lambd * math.gamma(1 + 1 / k), 3))
```

Média teórica: 1.805

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, X.max(), 400)

# weibull_min.pdf: c é a forma e scale a escala
```

```
plt.figure(figsize=(8, 5))
plt.hist(X, bins=40, density=True, color="skyblue", edgecolor="black")
plt.plot(grade, weibull_min.pdf(grade, c=k, scale=lambd), color="red",
        linewidth=2)
plt.title("Weibull gerada por transformação de uma exponencial")
plt.xlabel("Valor de X")
plt.ylabel("Densidade")
plt.show()
```



69.1 Exemplo 4: distribuição qui-quadrado

Este exemplo usa uma transformação de **várias** variáveis, e será a peça que falta para o Exemplo 7.

Por definição, se $Z_1, \dots, Z_k$ são independentes e $Z_i \sim N(0, 1)$, então

$$X = Z_1^2 + Z_2^2 + \dots + Z_k^2 \sim \chi_k^2.$$

Ou seja, $g(z_1, \dots, z_k) = z_1^2 + \dots + z_k^2$ e $Y = (Z_1, \dots, Z_k)$. Como já sabemos gerar normais pelo método da rejeição (Exemplo 2 do Capítulo 6), o algoritmo é imediato.

💡 Pseudo-algoritmo: qui-quadrado com k graus de liberdade

1. Gere $Z_1, \dots, Z_k$ independentes, todas $N(0, 1)$.
2. Devolva $X = \sum_{i=1}^k Z_i^2$.

i Um caminho alternativo

A χ_k^2 é o mesmo que uma $\text{Gama}(k/2, 1/2)$. Quando k é **par**, $k/2$ é inteiro e podemos gerá-la somando $k/2$ variáveis $\text{Exp}(1/2)$, como no Exemplo 2 — sem precisar de nenhuma normal. Em particular, $\chi_2^2 = \text{Exp}(1/2)$, fato que reaparecerá no capítulo sobre o método de Box-Muller.

No código abaixo usamos as funções prontas `rnorm` e `np.random.normal` para gerar as normais, em vez de repetir o algoritmo de rejeição do capítulo anterior.

70 R

```
library(ggplot2)

set.seed(42)

k <- 5      # graus de liberdade
B <- 5000   # quantos valores queremos gerar

# Passo 1: uma matriz de normais com B linhas e k colunas. Cada linha reúne as
# k normais de um mesmo valor de X
Z <- matrix(rnorm(B * k), nrow = B, ncol = k)

# Passo 2: rowSums soma cada linha, devolvendo os B valores de X
X <- rowSums(Z^2)

cat("Média amostral:", round(mean(X), 3), " (teórica:", k, ")\n")
```

Média amostral: 5.086 (teórica: 5)

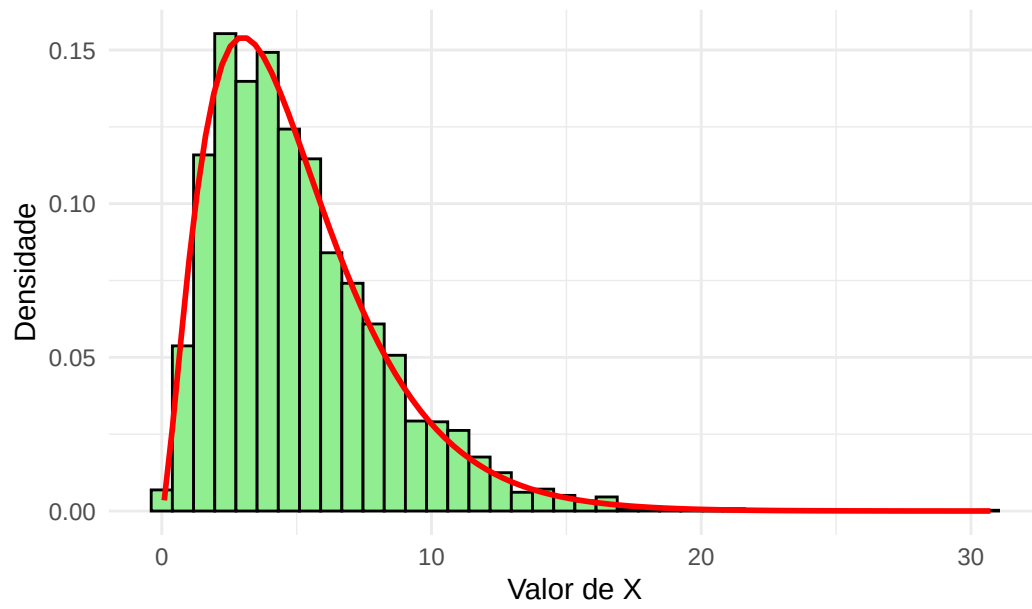
```
cat("Variância amostral:", round(var(X), 3), " (teórica:", 2 * k, ")\n")
```

Variância amostral: 10.19 (teórica: 10)

```
df <- data.frame(x = X)

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                 fill = "lightgreen", color = "black") +
  stat_function(fun = function(x) dchisq(x, df = k),
               color = "red", linewidth = 1) +
  labs(title = "Qui-quadrado como soma de quadrados de normais",
       x = "Valor de X", y = "Densidade") +
  theme_minimal()
```

Qui-quadrado como soma de quadrados de normais



71 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2

np.random.seed(42)

k = 5      # graus de liberdade
B = 5000   # quantos valores queremos gerar

# Passo 1: uma matriz de normais com B linhas e k colunas. Cada linha reúne as
# k normais de um mesmo valor de X
Z = np.random.normal(0, 1, (B, k))

# Passo 2: soma de cada linha (axis=1), devolvendo os B valores de X
X = np.sum(Z**2, axis=1)

print("Média amostral:", round(X.mean(), 3), " (teórica:", k, ")")
```

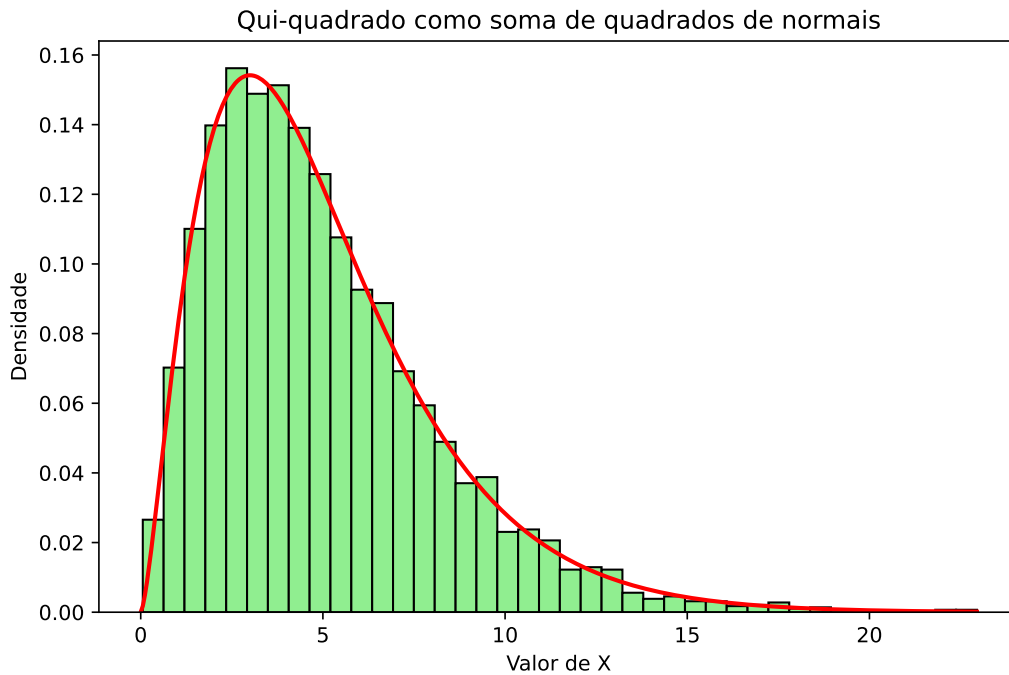
Média amostral: 4.985 (teórica: 5)

```
print("Variância amostral:", round(X.var(ddof=1), 3), " (teórica:", 2 * k, ")")
```

Variância amostral: 9.827 (teórica: 10)

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(0, X.max(), 400)

plt.figure(figsize=(8, 5))
plt.hist(X, bins=40, density=True, color="lightgreen", edgecolor="black")
plt.plot(grade, chi2.pdf(grade, df=k), color="red", linewidth=2)
plt.title("Qui-quadrado como soma de quadrados de normais")
plt.xlabel("Valor de X")
plt.ylabel("Densidade")
plt.show()
```



71.1 Exemplo 5: um vetor de normais correlacionadas

Nos dois exemplos anteriores, g recebia uma ou várias variáveis e devolvia um **número**. Neste exemplo, g devolve um **vetor** — e é essa a construção usada sempre que se quer simular várias medidas de um mesmo indivíduo, que não são independentes entre si: altura e peso de uma pessoa, notas de um aluno em disciplinas diferentes, preços de ações em uma carteira.

Queremos gerar um vetor $X = (X_1, \dots, X_d)$ com distribuição **normal multivariada**, isto é, com vetor de médias μ e matriz de covariâncias Σ dados. O que sabemos fazer é gerar $Z = (Z_1, \dots, Z_d)$ com coordenadas independentes $N(0, 1)$ — um vetor com médias nulas e matriz de covariâncias igual à identidade.

A transformação que procuramos é afim: $X = \mu + LZ$, para alguma matriz L de dimensão $d \times d$. Falta descobrir qual. Como somar μ só desloca as médias, o trabalho está em achar L que produza as covariâncias certas.

i Definição: decomposição de Cholesky

Seja Σ uma matriz simétrica e **positiva definida** (o que toda matriz de covariâncias de um vetor não degenerado é). Existe uma única matriz L triangular inferior, com todos

os elementos da diagonal positivos, tal que

$$\Sigma = LL^\top.$$

Essa matriz é a **decomposição de Cholesky** de Σ , e pode ser vista como uma “raiz quadrada” de Σ . Ela é calculada por `chol` no R e por `np.linalg.cholesky` no Python.

! Proposição

Sejam $Z_1, \dots, Z_d$ independentes e $N(0, 1)$, $\mu \in \mathbb{R}^d$ e $\Sigma = LL^\top$. Então

$$X = \mu + LZ$$

é um vetor normal multivariado com médias μ e matriz de covariâncias Σ .

i Demonstração

Que X é normal multivariado segue de um resultado padrão: transformações afins de vetores normais são normais. Restam as duas primeiras características.

Para as médias, como $\mathbb{E}[Z] = 0$,

$$\mathbb{E}[X] = \mu + L \mathbb{E}[Z] = \mu.$$

Para as covariâncias, escrevemos a matriz de covariâncias na forma $\text{Cov}(X) = \mathbb{E}[(X - \mu)(X - \mu)^\top]$. Como $X - \mu = LZ$,

$$\text{Cov}(X) = \mathbb{E}[(LZ)(LZ)^\top] = \mathbb{E}[LZZ^\top L^\top] = L \mathbb{E}[ZZ^\top] L^\top,$$

onde na última passagem tiramos L e $L^\top$ de dentro da esperança, por serem constantes. Agora, $\mathbb{E}[ZZ^\top]$ é a matriz de covariâncias de Z : como as coordenadas são independentes e têm variância 1, ela é a identidade I . Logo,

$$\text{Cov}(X) = L I L^\top = LL^\top = \Sigma. \quad \square$$

💡 Pseudo-algoritmo: normal multivariada

Entradas: o vetor de médias μ e a matriz de covariâncias Σ .

1. Calcule a decomposição de Cholesky $\Sigma = LL^\top$ (uma única vez, fora do laço).
2. Gere $Z_1, \dots, Z_d$ independentes, todas $N(0, 1)$.
3. Devolva $X = \mu + LZ$.

Vamos simular três medidas de uma pessoa — altura, peso e circunferência da cintura —, com médias 170 cm, 70 kg e 85 cm, desvios padrão 8, 12 e 10, e correlações 0,6 entre altura e peso, 0,3 entre altura e cintura e 0,8 entre peso e cintura.

É mais natural especificar desvios padrão e correlações do que a matriz de covariâncias diretamente. A conversão é $\Sigma = DRD$, em que R é a matriz de correlações e D é a matriz diagonal com os desvios padrão — afinal, $\text{Cov}(X_i, X_j) = \sigma_i \sigma_j \rho_{ij}$.

 **Atenção:** `chol` do R devolve a transposta

As duas linguagens usam convenções diferentes. O `np.linalg.cholesky` do Python devolve a triangular **inferior** L , com $\Sigma = LL^\top$, que é exatamente a matriz do algoritmo. Já o `chol` do R devolve a triangular **superior** U , com $\Sigma = U^\top U$; a matriz L do algoritmo é, portanto, `t(chol(Sigma))`.

Esquecer a transposta não gera erro: o programa roda normalmente e devolve vetores normais com as médias certas, mas com **variâncias e correlações diferentes das pedidas**. É um bug silencioso, do tipo que só aparece quando se conferem os desvios e as correlações amostrais — como fazemos no código abaixo.

72 R

```
library(ggplot2)

set.seed(42)

# Médias e desvios padrão de altura (cm), peso (kg) e cintura (cm)
mu <- c(170, 70, 85)
desvios <- c(8, 12, 10)

correlacoes <- matrix(c(1.0, 0.6, 0.3,
                        0.6, 1.0, 0.8,
                        0.3, 0.8, 1.0), nrow = 3, byrow = TRUE)

# Sigma = D R D, em que D = diag(desvios). O operador %*% é a multiplicação
# de matrizes; o * comum multiplicaria elemento a elemento, o que seria errado
Sigma <- diag(desvios) %*% correlacoes %*% diag(desvios)

# Passo 1: Cholesky, feito uma única vez. O t() é a transposta, necessária
# porque chol() devolve a triangular superior
L <- t(chol(Sigma))
cat("Matriz L:\n")
```

Matriz L:

```
print(round(L, 2))
```

```
      [,1] [,2] [,3]
[1,]  8.0  0.00  0.00
[2,]  7.2  9.60  0.00
[3,]  3.0  7.75  5.56
```

```
B <- 2000
X <- matrix(0, nrow = B, ncol = 3)
```

```

for (b in 1:B) {
  Z <- rnorm(3)          # Passo 2: três normais padrão independentes
  X[b, ] <- mu + L %*% Z  # Passo 3: a transformação afim
}

cat("\nMédias amostrais :", round(colMeans(X), 2), "\n")

```

Médias amostrais : 170.03 69.85 84.82

```
cat("Médias teóricas :", mu, "\n")
```

Médias teóricas : 170 70 85

```
cat("Desvios amostrais:", round(apply(X, 2, sd), 2), "\n")
```

Desvios amostrais: 8.02 11.9 10.04

```
cat("Desvios teóricos :", desvios, "\n")
```

Desvios teóricos : 8 12 10

```
cat("\nCorrelações amostrais:\n")
```

Correlações amostrais:

```
print(round(cor(X), 3))
```

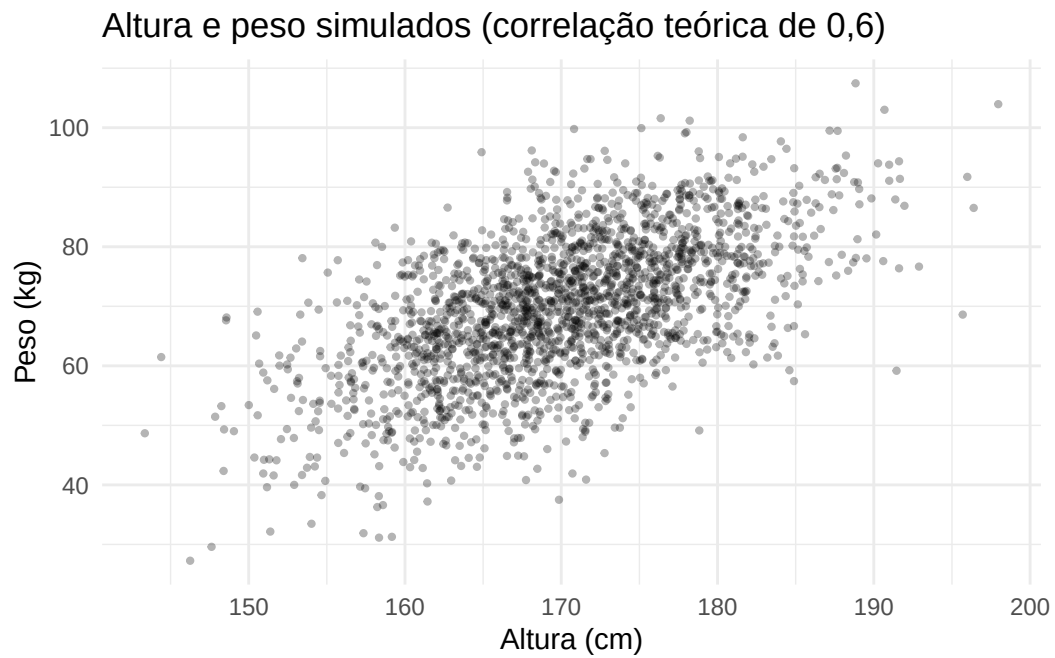
```

      [,1] [,2] [,3]
[1,] 1.000 0.585 0.293
[2,] 0.585 1.000 0.796
[3,] 0.293 0.796 1.000

```

```
df <- data.frame(altura = X[, 1], peso = X[, 2])

ggplot(df, aes(x = altura, y = peso)) +
  geom_point(alpha = 0.3, size = 0.8) +
  labs(title = "Altura e peso simulados (correlação teórica de 0,6)",
       x = "Altura (cm)", y = "Peso (kg)") +
  theme_minimal()
```



73 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Médias e desvios padrão de altura (cm), peso (kg) e cintura (cm)
mu = np.array([170, 70, 85])
desvios = np.array([8, 12, 10])

correlacoes = np.array([[1.0, 0.6, 0.3],
                        [0.6, 1.0, 0.8],
                        [0.3, 0.8, 1.0]])

# Sigma = D R D, em que D = diag(desvios). O operador @ é a multiplicação de
# matrizes (o equivalente ao %*% do R); o * comum multiplicaria elemento a
# elemento, o que seria errado
Sigma = np.diag(desvios) @ correlacoes @ np.diag(desvios)

# Passo 1: Cholesky, feito uma única vez. Aqui já vem a triangular inferior
L = np.linalg.cholesky(Sigma)
print("Matriz L:")
```

Matriz L:

```
print(np.round(L, 2))
```

```
[[8.    0.    0. ]
 [7.2   9.6   0. ]
 [3.    7.75 5.56]]
```

```
B = 2000
X = np.zeros((B, 3))
```

```

for b in range(B):
    Z = np.random.normal(size=3)    # Passo 2: três normais padrão independentes
    X[b, :] = mu + L @ Z           # Passo 3: a transformação afim

print("\nMédias amostrais :", np.round(X.mean(axis=0), 2))

```

```

Médias amostrais : [169.92  69.64  84.91]

```

```

print("Médias teóricas :", mu)

```

```

Médias teóricas : [170  70  85]

```

```

print("Desvios amostrais:", np.round(X.std(axis=0, ddof=1), 2))

```

```

Desvios amostrais: [ 8.02 11.74  9.81]

```

```

print("Desvios teóricos :", desvios)

```

```

Desvios teóricos : [ 8 12 10]

```

```

print("\nCorrelações amostrais:")

```

```

Correlações amostrais:

```

```

print(np.round(np.corrcoef(X, rowvar=False), 3))

```

```

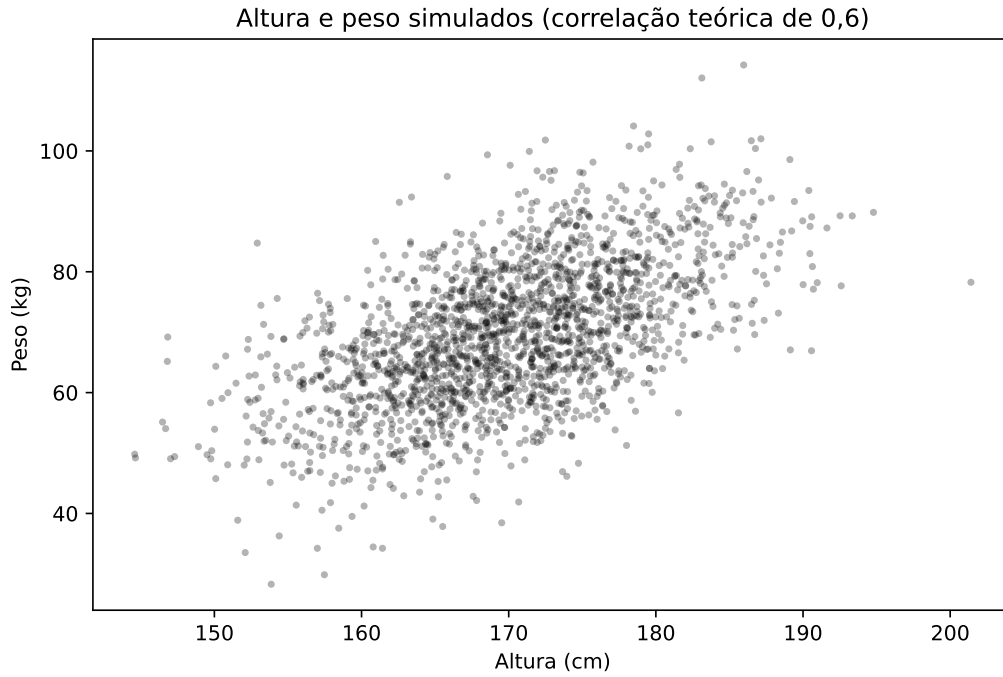
[[1.    0.576 0.255]
 [0.576 1.    0.789]
 [0.255 0.789 1.    ]]

```

```

plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], alpha=0.3, s=6, color="black")
plt.title("Altura e peso simulados (correlação teórica de 0,6)")
plt.xlabel("Altura (cm)")
plt.ylabel("Peso (kg)")
plt.show()

```



As médias, os desvios padrão e as correlações amostrais reproduzem os valores pedidos, e a nuvem de pontos tem a inclinação esperada de duas variáveis positivamente correlacionadas. Note que o passo caro — a decomposição de Cholesky — é feito **uma única vez**, fora do laço: gerar mais um vetor custa apenas d normais padrão e uma multiplicação por L .

No próximo capítulo veremos o caso $d = 2$ construído à mão, sem matrizes, diretamente a partir do método de Box-Muller.

73.1 Misturas

Agora a segunda construção. Suponha que saibamos simular uma v.a. Y e que, **dado** o valor de Y , saibamos também simular X . Se a densidade de X puder ser escrita como

$$f_X(x) = \int_{-\infty}^{\infty} f_{X|Y}(x | y) f_Y(y) dy \quad (\text{caso } Y \text{ contínua})$$

ou como

$$f_X(x) = \sum_y f_{X|Y}(x | y) \mathbb{P}(Y = y) \quad (\text{caso } Y \text{ discreta}),$$

dizemos que a distribuição de X é uma **distribuição de mistura**. A variável Y é chamada de variável de mistura, e simulá-la é o primeiro passo do algoritmo.

As duas fórmulas se distinguem apenas por integrar ou somar sobre os valores de Y ; a natureza de X é indiferente. Quando X também é discreta, basta trocar as densidades f por funções de probabilidade — é o que acontece nos Exercícios 6 e 7.

 Pseudo-algoritmo: método da mistura

1. Simule um valor y a partir da distribuição de Y .
2. Simule X a partir da distribuição condicional de X dado $Y = y$, e devolva esse valor (descartando y).

 Proposição

O valor X devolvido pelo algoritmo acima tem densidade $f_X(x) = \int f_{X|Y}(x | y) f_Y(y) dy$.

 Demonstração

Pela definição de densidade condicional, a densidade **conjunta** do par (X, Y) é

$$f_{X,Y}(x, y) = f_{X|Y}(x | y) f_Y(y).$$

O algoritmo produz exatamente um par com essa conjunta: o passo 1 gera Y com densidade f_Y , e o passo 2 gera, condicionalmente a $Y = y$, um valor com densidade $f_{X|Y}(\cdot | y)$. Descartar y e ficar apenas com X corresponde a tomar a densidade marginal, ou seja, a integrar a conjunta em y :

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dy = \int_{-\infty}^{\infty} f_{X|Y}(x | y) f_Y(y) dy. \quad \square$$

Como no caso da transformação, a demonstração é curta e o trabalho de verdade é o inverso dela: dada uma densidade f_X que queremos simular, **reconhecer** quais Y e $X | Y$ a produzem. O Exemplo 7 mostra um caso em que essa decomposição não é nada óbvia.

Quando Y é discreta e assume apenas os valores $1, \dots, m$, a mistura toma a forma particularmente simples

$$f_X(x) = w_1 f_1(x) + w_2 f_2(x) + \dots + w_m f_m(x), \quad w_j = \mathbb{P}(Y = j),$$

com $w_j \geq 0$ e $\sum_j w_j = 1$: a densidade de X é uma **média ponderada** de m densidades. Nesse caso o método também é chamado de **método da composição**, e o algoritmo é: sorteie qual das m distribuições usar (com probabilidades $w_1, \dots, w_m$) e gere um valor dela.

 Atenção: misturar não é fazer média

São as **densidades** que entram na combinação linear, não as variáveis. Se $X_1 \sim N(-3, 1)$ e $X_2 \sim N(3, 1)$ são independentes, a mistura com pesos $1/2$ é a variável que vale X_1 ou X_2 conforme o resultado de um cara ou coroa — e tem densidade **bimodal**, com picos em -3 e 3 . Já a média $(X_1 + X_2)/2$ é uma $N(0, 1/2)$: unimodal, concentrada em torno de zero, e sem nenhuma massa perto dos picos. As duas construções não têm nada a ver uma com a outra.

Assim como no caso das transformações, já usamos misturas sem lhes dar nome: no Exemplo 2 do Capítulo 6, geramos por rejeição um valor Y com a distribuição de $|X|$, sorteamos um sinal $S = \pm 1$ e devolvemos $S \cdot Y$. Aquilo era uma mistura de duas componentes com pesos $1/2$ — a normal restrita aos valores positivos e a restrita aos negativos.

73.2 Exemplo 6: mistura de duas normais

O caso mais comum de mistura discreta aparece quando a população estudada tem dois grupos com comportamentos diferentes: peças produzidas por duas máquinas, alunos que estudaram e que não estudaram, pacientes que responderam e que não responderam ao tratamento. Se a proporção do primeiro grupo é w e as duas subpopulações são normais, a densidade da população inteira é

$$f_X(x) = w \cdot \varphi(x; \mu_1, \sigma_1) + (1 - w) \cdot \varphi(x; \mu_2, \sigma_2),$$

em que $\varphi(\cdot; \mu, \sigma)$ denota a densidade da $N(\mu, \sigma^2)$.

Vamos simular o caso $w = 0,7$, com $N(0, 1)$ no primeiro grupo e $N(4, 0,5^2)$ no segundo. Aqui a variável de mistura é $Y \sim \text{Bernoulli}(w)$, que sorteia o grupo.

 Pseudo-algoritmo: mistura de duas normais

1. Gere $U \sim \text{Unif}(0, 1)$.
2. Se $U \leq w$, gere e devolva $X \sim N(\mu_1, \sigma_1^2)$.
3. Caso contrário, gere e devolva $X \sim N(\mu_2, \sigma_2^2)$.

O laço abaixo é escrito passo a passo, seguindo o pseudo-algoritmo: para cada um dos B valores, primeiro sorteamos o grupo e só depois geramos a normal correspondente.

74 R

```
library(ggplot2)

set.seed(42)

w <- 0.7          # peso da primeira componente
mu1 <- 0; sigma1 <- 1
mu2 <- 4; sigma2 <- 0.5
B <- 5000         # quantos valores queremos gerar

X <- numeric(B)   # vetor que guardará os valores gerados
grupo <- numeric(B) # guarda de qual componente veio cada valor

for (i in 1:B) {
  # Passo 1: o uniforme que sorteia a componente
  U <- runif(1)

  # Passos 2 e 3: geramos da normal correspondente ao grupo sorteado
  if (U <= w) {
    grupo[i] <- 1
    X[i] <- rnorm(1, mean = mu1, sd = sigma1)
  } else {
    grupo[i] <- 2
    X[i] <- rnorm(1, mean = mu2, sd = sigma2)
  }
}

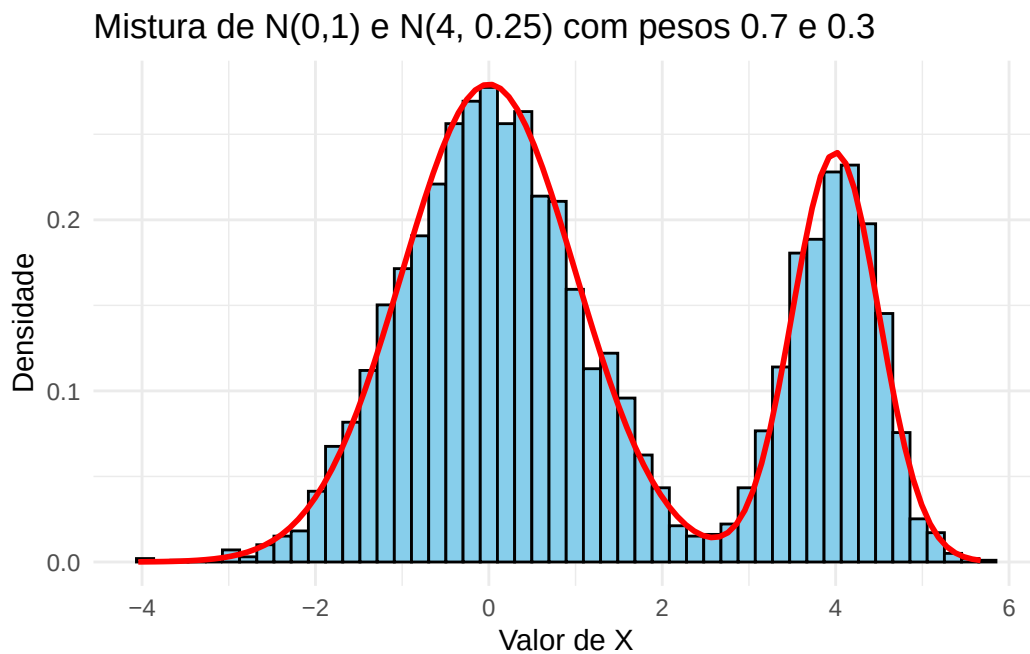
# A densidade da mistura é a média ponderada das duas densidades
densidade_mistura <- function(x) {
  w * dnorm(x, mu1, sigma1) + (1 - w) * dnorm(x, mu2, sigma2)
}

cat("Proporção sorteada do primeiro grupo:", round(mean(grupo == 1), 3),
    " (peso w =", w, ")\n")
```

Proporção sorteada do primeiro grupo: 0.694 (peso $w = 0.7$)

```
df <- data.frame(x = X)

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50,
    fill = "skyblue", color = "black") +
  stat_function(fun = densidade_mistura, color = "red", linewidth = 1) +
  labs(title = "Mistura de N(0,1) e N(4, 0.25) com pesos 0.7 e 0.3",
    x = "Valor de X", y = "Densidade") +
  theme_minimal()
```



75 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)

w = 0.7          # peso da primeira componente
mu1, sigma1 = 0, 1
mu2, sigma2 = 4, 0.5
B = 5000        # quantos valores queremos gerar

X = np.zeros(B)    # vetor que guardará os valores gerados
grupo = np.zeros(B) # guarda de qual componente veio cada valor

for i in range(B):
    # Passo 1: o uniforme que sorteia a componente
    U = np.random.uniform(0, 1)

    # Passos 2 e 3: geramos da normal correspondente ao grupo sorteado
    if U <= w:
        grupo[i] = 1
        X[i] = np.random.normal(mu1, sigma1)
    else:
        grupo[i] = 2
        X[i] = np.random.normal(mu2, sigma2)

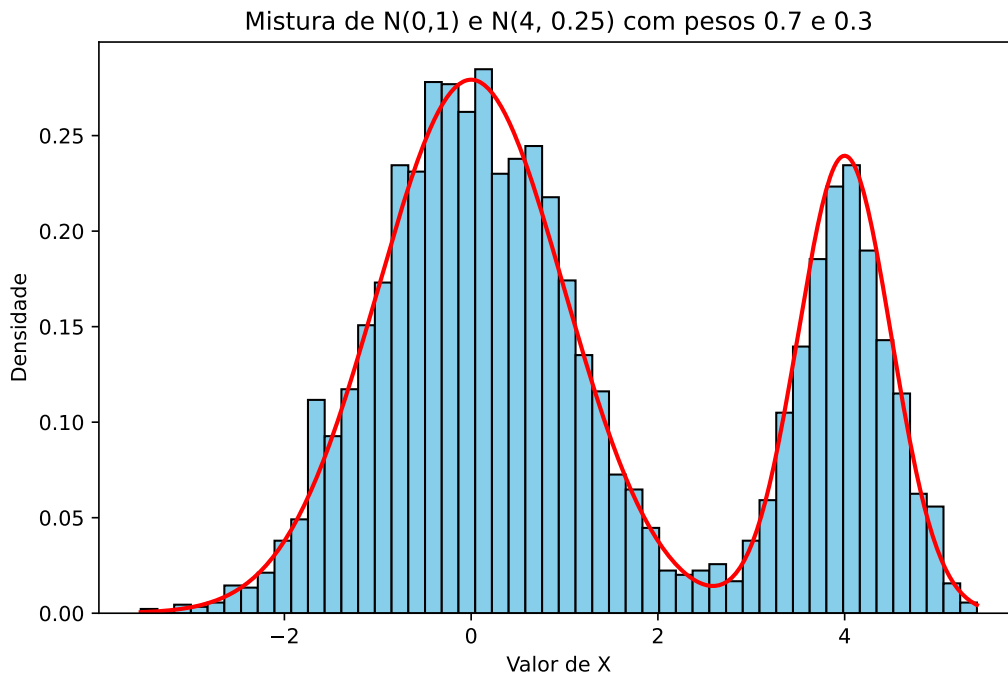
# A densidade da mistura é a média ponderada das duas densidades
def densidade_mistura(x):
    return w * norm.pdf(x, mu1, sigma1) + (1 - w) * norm.pdf(x, mu2, sigma2)

print("Proporção sorteada do primeiro grupo:", round(np.mean(grupo == 1), 3),
      " (peso w =", w, ")")
```

Proporção sorteada do primeiro grupo: 0.714 (peso w = 0.7)

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(X.min(), X.max(), 400)

plt.figure(figsize=(8, 5))
plt.hist(X, bins=50, density=True, color="skyblue", edgecolor="black")
plt.plot(grade, densidade_mistura(grade), color="red", linewidth=2)
plt.title("Mistura de N(0,1) e N(4, 0.25) com pesos 0.7 e 0.3")
plt.xlabel("Valor de X")
plt.ylabel("Densidade")
plt.show()
```



Repare que a densidade resultante tem **dois picos**, e que nenhuma das duas componentes, sozinha, se parece com ela. Nenhum método baseado em inverter F seria confortável aqui; a mistura, ao contrário, praticamente lê o algoritmo na própria fórmula da densidade.

75.1 Exemplo 7: distribuição t de Student

Neste exemplo a mistura é contínua, e a decomposição está longe de ser óbvia: partimos de uma densidade complicada e descobrimos que ela esconde uma normal cuja variância é, ela própria, aleatória.

Seja $X \sim t_k$, com densidade

$$f_X(x) = \frac{\Gamma\left(\frac{k+1}{2}\right)}{\Gamma\left(\frac{k}{2}\right)} \frac{1}{\sqrt{k\pi}} \frac{1}{(1+x^2/k)^{(k+1)/2}}, \quad x \in \mathbb{R}.$$

! Proposição

Se $Y \sim \chi_k^2$ e $X | Y = y \sim N(0, k/y)$, então $X \sim t_k$.

i Demonstração

Temos

$$f_{X|Y=y}(x | y) = \frac{1}{\sqrt{k/y} \sqrt{2\pi}} e^{-\frac{y}{2k} x^2} \quad \text{e} \quad f_Y(y) = \frac{1}{2^{k/2} \Gamma(k/2)} y^{k/2-1} e^{-y/2},$$

esta última para $y > 0$. Logo,

$$\begin{aligned} f_X(x) &= \int_0^\infty f_{X|Y}(x | y) f_Y(y) dy \\ &= \int_0^\infty \frac{\sqrt{y}}{\sqrt{k} \sqrt{2\pi}} e^{-\frac{y}{2k} x^2} \frac{1}{2^{k/2} \Gamma(k/2)} y^{k/2-1} e^{-y/2} dy \\ &= \frac{1}{\sqrt{k} \sqrt{2\pi}} \frac{1}{2^{k/2} \Gamma(k/2)} \int_0^\infty y^{\frac{k+1}{2}-1} e^{-y\left(\frac{x^2}{2k} + \frac{1}{2}\right)} dy. \end{aligned}$$

A integral que sobrou é a da densidade de uma Gama, a menos de constantes: para $a > 0$ e $b > 0$, $\int_0^\infty y^{a-1} e^{-by} dy = \Gamma(a)/b^a$. Com $a = (k+1)/2$ e $b = x^2/(2k) + 1/2$,

$$f_X(x) = \frac{1}{\sqrt{k} \sqrt{2\pi}} \frac{1}{2^{k/2} \Gamma(k/2)} \frac{\Gamma\left(\frac{k+1}{2}\right)}{\left(\frac{x^2}{2k} + \frac{1}{2}\right)^{(k+1)/2}}.$$

Por fim, colocando $1/2$ em evidência no denominador, $\left(\frac{x^2}{2k} + \frac{1}{2}\right)^{(k+1)/2} = 2^{-(k+1)/2} \left(\frac{x^2}{k} + 1\right)^{(k+1)/2}$, e os fatores $2^{(k+1)/2}/(2^{k/2} \sqrt{2\pi})$ se simplificam para $1/\sqrt{\pi}$, resultando em

$$f_X(x) = \frac{\Gamma\left(\frac{k+1}{2}\right)}{\Gamma\left(\frac{k}{2}\right)} \frac{1}{\sqrt{k\pi}} \frac{1}{(1+x^2/k)^{(k+1)/2}},$$

que é a densidade da t_k . $\square$

💡 Pseudo-algoritmo: t de Student com k graus de liberdade

1. Gere $Y \sim \chi_k^2$ (pelo Exemplo 4, somando os quadrados de k normais padrão).
2. Gere e devolva $X \sim N(0, k/Y)$, isto é, uma normal de média 0 e desvio padrão $\sqrt{k/Y}$.

i A mesma construção, vista como transformação

O passo 2 equivale a fazer $X = \sqrt{k/Y} Z$, com $Z \sim N(0, 1)$ independente de Y — ou, reorganizando,

$$X = \frac{Z}{\sqrt{Y/k}},$$

que é a definição da t_k vista em cursos de inferência. Mistura e transformação são, aqui, duas leituras da mesma construção: sortear uma normal cuja variância é aleatória é o mesmo que dividir uma normal por uma raiz de qui-quadrado.

O código abaixo gera $B = 5000$ valores de uma t_5 seguindo o pseudo-algoritmo, e compara o histograma com a densidade teórica. A curva tracejada é a densidade da $N(0, 1)$: ela ajuda a ver que a t tem **caudas mais pesadas**, que é justamente o efeito de deixar a variância variar.

76 R

```
library(ggplot2)

set.seed(42)

k <- 5      # graus de liberdade
B <- 5000   # quantos valores queremos gerar

X <- numeric(B)

for (i in 1:B) {
  # Passo 1: Y ~ qui-quadrado com k graus de liberdade (Exemplo 4)
  Z <- rnorm(k)
  Y <- sum(Z^2)

  # Passo 2: a normal cuja variância depende do valor sorteado de Y
  X[i] <- rnorm(1, mean = 0, sd = sqrt(k / Y))
}

# Quanto da massa está além de 3 desvios? Na t é bem mais que na normal
cat("P(|X| > 3) observada:", round(mean(abs(X) > 3), 4), "\n")
```

P(|X| > 3) observada: 0.0276

```
cat("P(|X| > 3) na t_5:", round(2 * pt(-3, df = k), 4), "\n")
```

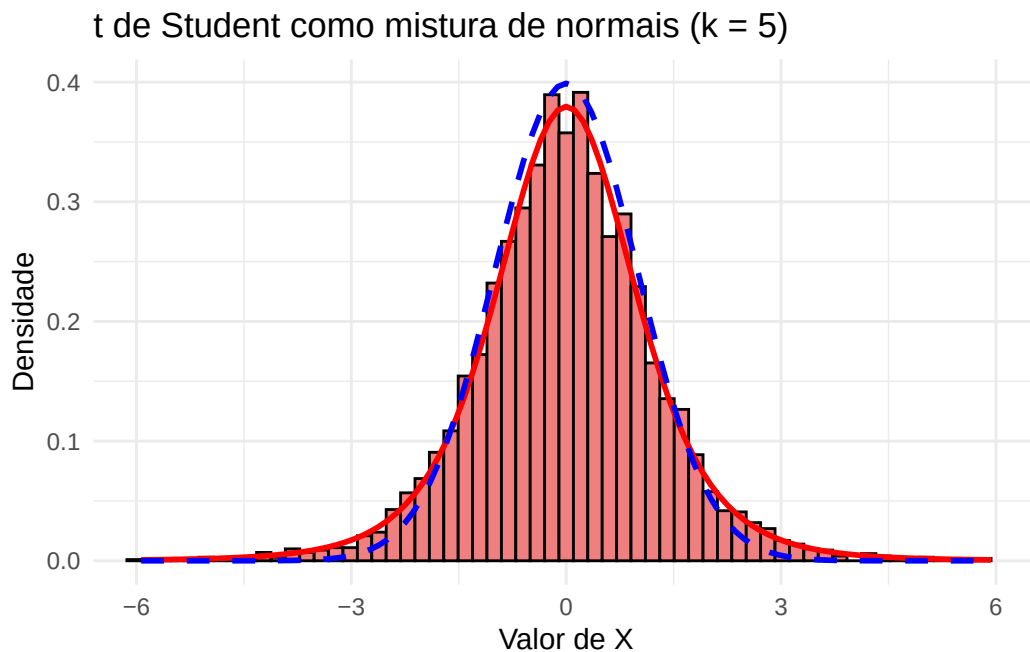
P(|X| > 3) na t_5: 0.0301

```
cat("P(|X| > 3) na N(0,1):", round(2 * pnorm(-3), 4), "\n")
```

P(|X| > 3) na N(0,1): 0.0027


```
# Para o histograma, olhamos só o intervalo [-6, 6]: a t tem caudas longas, e
# uns poucos valores extremos deixariam todas as barras espremidas no centro
df <- data.frame(x = X[abs(X) <= 6])

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 60,
    fill = "lightcoral", color = "black") +
  stat_function(fun = function(x) dt(x, df = k), color = "red",
    linewidth = 1) +
  stat_function(fun = dnorm, color = "blue", linewidth = 1,
    linetype = "dashed") +
  coord_cartesian(xlim = c(-6, 6)) +
  labs(title = "t de Student como mistura de normais (k = 5)",
    x = "Valor de X", y = "Densidade") +
  theme_minimal()
```



77 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import t, norm

np.random.seed(42)

k = 5      # graus de liberdade
B = 5000   # quantos valores queremos gerar

X = np.zeros(B)

for i in range(B):
    # Passo 1: Y ~ qui-quadrado com k graus de liberdade (Exemplo 4)
    Z = np.random.normal(0, 1, k)
    Y = np.sum(Z**2)

    # Passo 2: a normal cuja variância depende do valor sorteado de Y
    X[i] = np.random.normal(0, np.sqrt(k / Y))

# Quanto da massa está além de 3 desvios? Na t é bem mais que na normal
print("P(|X| > 3) observada:", round(np.mean(np.abs(X) > 3), 4))
```

P(|X| > 3) observada: 0.0264

```
print("P(|X| > 3) na t_5:", round(2 * t.cdf(-3, df=k), 4))
```

P(|X| > 3) na t_5: 0.0301

```
print("P(|X| > 3) na N(0,1):", round(2 * norm.cdf(-3), 4))
```

P(|X| > 3) na N(0,1): 0.0027

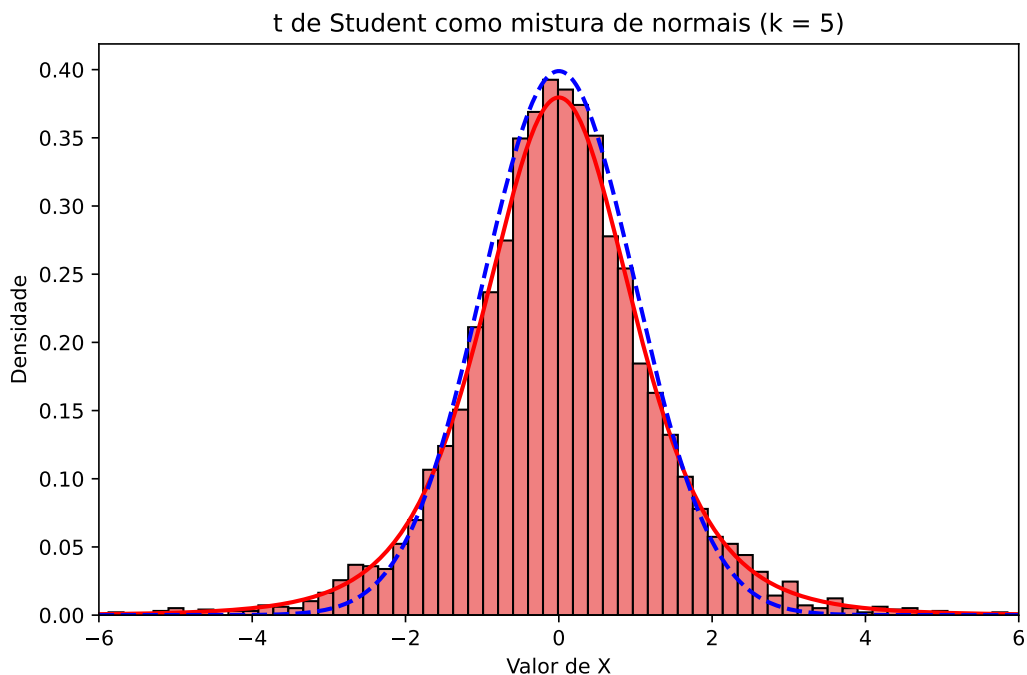
```
# Malha usada só para desenhar as densidades teóricas
grade = np.linspace(-6, 6, 400)

# Para o histograma, olhamos só o intervalo [-6, 6]: a t tem caudas longas, e
# uns poucos valores extremos deixariam todas as barras espremidas no centro
X_grafico = X[np.abs(X) <= 6]

plt.figure(figsize=(8, 5))
plt.hist(X_grafico, bins=60, density=True,
        color="lightcoral", edgecolor="black")
plt.plot(grade, t.pdf(grade, df=k), color="red", linewidth=2)
plt.plot(grade, norm.pdf(grade), color="blue", linewidth=2, linestyle="--")
plt.xlim(-6, 6)
```

(-6.0, 6.0)

```
plt.title("t de Student como mistura de normais (k = 5)")
plt.xlabel("Valor de X")
plt.ylabel("Densidade")
plt.show()
```



77.1 Exemplo 8: um modelo hierárquico

No exemplo anterior, partimos de uma densidade dada e **descobrimos** a mistura escondida nela. Este exemplo percorre o caminho oposto, que é o mais comum na prática: o modelo já **nasce** em dois estágios, porque é assim que o fenômeno está sendo descrito. Modelos com essa estrutura são chamados de **hierárquicos**.

Suponha que queremos modelar o número de consultas médicas que uma pessoa faz em um ano. Uma primeira tentativa seria dizer que esse número é $\text{Poisson}(\lambda)$, com o mesmo λ para todo mundo. Mas isso é implausível: pessoas têm estados de saúde diferentes, e portanto taxas diferentes. O modelo hierárquico incorpora exatamente essa ideia:

$$\Lambda \sim \text{Gama}(r, \beta), \quad N \mid \Lambda = \ell \sim \text{Poisson}(\ell).$$

Primeiro sorteamos a taxa da pessoa, depois sorteamos quantas consultas ela faz dada essa taxa. A distribuição Gama é uma escolha natural para Λ por ser positiva e flexível, e — como veremos — por levar a uma resposta conhecida.

💡 Pseudo-algoritmo: modelo hierárquico Poisson–Gama

1. Gere $\Lambda \sim \text{Gama}(r, \beta)$.
2. Gere e devolva $N \sim \text{Poisson}(\Lambda)$.

O algoritmo é o método da mistura sem nenhuma novidade: a variável de mistura é Λ , e $N \mid \Lambda$ é a distribuição condicional. O que surpreende é a distribuição marginal que resulta disso.

❗ Proposição

Se $\Lambda \sim \text{Gama}(r, \beta)$ e $N \mid \Lambda = \ell \sim \text{Poisson}(\ell)$, então N tem distribuição Binomial Negativa com parâmetros r e $p = \frac{\beta}{1 + \beta}$, isto é,

$$\mathbb{P}(N = n) = \frac{\Gamma(n + r)}{n! \Gamma(r)} p^r (1 - p)^n, \quad n = 0, 1, 2, \dots$$

i Demonstração

Aplicamos a fórmula da mistura com $Y = \Lambda$ contínua. Como

$$\mathbb{P}(N = n \mid \Lambda = \ell) = \frac{e^{-\ell} \ell^n}{n!} \quad \text{e} \quad f_{\Lambda}(\ell) = \frac{\beta^r}{\Gamma(r)} \ell^{r-1} e^{-\beta \ell},$$

temos

$$\begin{aligned}\mathbb{P}(N = n) &= \int_0^\infty \frac{e^{-\ell} \ell^n}{n!} \frac{\beta^r}{\Gamma(r)} \ell^{r-1} e^{-\beta \ell} d\ell \\ &= \frac{\beta^r}{n! \Gamma(r)} \int_0^\infty \ell^{n+r-1} e^{-(1+\beta)\ell} d\ell.\end{aligned}$$

A integral que sobrou é a mesma que apareceu no Exemplo 7: para $a > 0$ e $b > 0$, $\int_0^\infty \ell^{a-1} e^{-b\ell} d\ell = \Gamma(a)/b^a$. Com $a = n + r$ e $b = 1 + \beta$,

$$\mathbb{P}(N = n) = \frac{\beta^r}{n! \Gamma(r)} \cdot \frac{\Gamma(n+r)}{(1+\beta)^{n+r}} = \frac{\Gamma(n+r)}{n! \Gamma(r)} \left(\frac{\beta}{1+\beta} \right)^r \left(\frac{1}{1+\beta} \right)^n,$$

onde na última igualdade separamos $(1+\beta)^{n+r}$ em $(1+\beta)^r$ e $(1+\beta)^n$. Reconhecendo $p = \beta/(1+\beta)$ e $1-p = 1/(1+\beta)$, chegamos à expressão do enunciado. $\square$

No código abaixo usamos $r = 3$ e $\beta = 1,5$, de modo que a taxa média é $\mathbb{E}[\Lambda] = r/\beta = 2$ consultas por ano. Comparamos as frequências observadas com as probabilidades da Binomial Negativa e, para deixar clara a diferença, também com as de uma Poisson(2) — que tem exatamente a mesma média.

78 R

```
library(ggplot2)

set.seed(42)

r <- 3      # parâmetro de forma da Gama
taxa <- 1.5  # parâmetro de taxa da Gama (o beta do texto; evitamos o nome
             # "beta" porque em R já existe uma função com esse nome)
B <- 5000

N <- numeric(B)

for (b in 1:B) {
  # Passo 1: a taxa daquela pessoa
  lambda_pessoa <- rgamma(1, shape = r, rate = taxa)
  # Passo 2: quantas consultas ela faz, dada a sua taxa
  N[b] <- rpois(1, lambda = lambda_pessoa)
}

cat("Média amostral   :", round(mean(N), 3),
    " (teórica:", r / taxa, ")\n")
```

Média amostral : 2.001 (teórica: 2)

```
cat("Variância amostral:", round(var(N), 3),
    " (teórica:", round(r / taxa + r / taxa^2, 3), ")\n")
```

Variância amostral: 3.32 (teórica: 3.333)

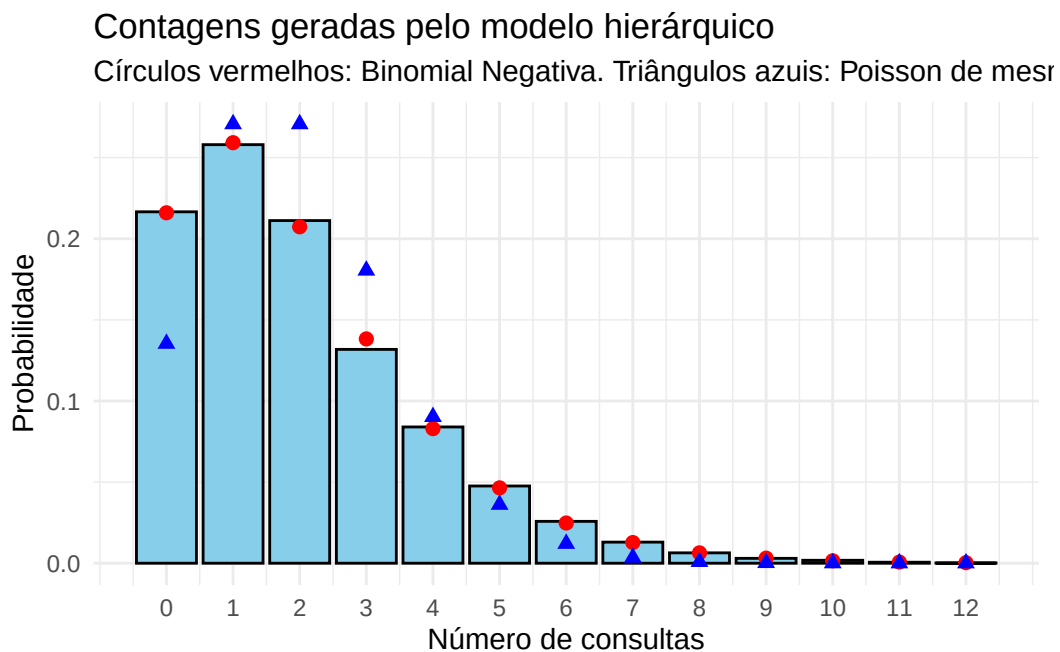
```
valores <- 0:12
p_bn <- taxa / (1 + taxa)

# factor com levels garante que todos os valores apareçam, mesmo os que
# porventura não tenham sido sorteados
```

```
frequencias <- as.numeric(table(factor(N, levels = valores))) / B

df <- data.frame(
  valor = valores,
  observada = frequencias,
  binomial_negativa = dnbinom(valores, size = r, prob = p_bn),
  poisson = dpois(valores, lambda = r / taxa)
)

ggplot(df, aes(x = valor)) +
  geom_col(aes(y = observada), fill = "skyblue", color = "black") +
  geom_point(aes(y = binomial_negativa), color = "red", size = 2) +
  geom_point(aes(y = poisson), color = "blue", size = 2, shape = 17) +
  scale_x_continuous(breaks = valores) +
  labs(title = "Contagens geradas pelo modelo hierárquico",
       subtitle = paste("Círculos vermelhos: Binomial Negativa.",
                        "Triângulos azuis: Poisson de mesma média."),
       x = "Número de consultas", y = "Probabilidade") +
  theme_minimal()
```



79 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import nbinom, poisson

np.random.seed(42)

r = 3          # parâmetro de forma da Gama
taxa = 1.5     # parâmetro de taxa da Gama (o beta do texto)
B = 5000

N = np.zeros(B, dtype=int)

for b in range(B):
    # Passo 1: a taxa daquela pessoa. Atenção: np.random.gamma recebe a escala,
    # que é o inverso da taxa
    lambda_pessoa = np.random.gamma(shape=r, scale=1 / taxa)
    # Passo 2: quantas consultas ela faz, dada a sua taxa
    N[b] = np.random.poisson(lambda_pessoa)

print("Média amostral      :", round(N.mean(), 3),
      " (teórica:", r / taxa, ")")
```

Média amostral : 2.017 (teórica: 2.0)

```
print("Variância amostral:", round(N.var(ddof=1), 3),
      " (teórica:", round(r / taxa + r / taxa**2, 3), ")")
```

Variância amostral: 3.201 (teórica: 3.333)

```
valores = np.arange(0, 13)
p_bn = taxa / (1 + taxa)

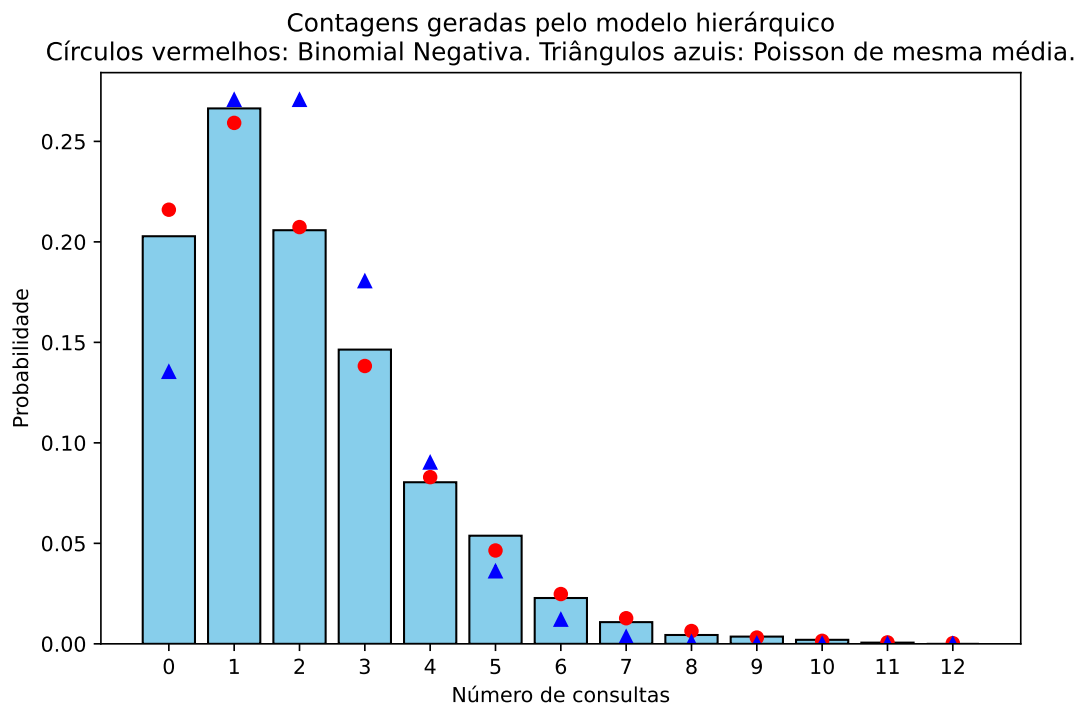
frequencias = np.array([np.mean(N == v) for v in valores])
```



```
plt.figure(figsize=(8, 5))
plt.bar(valores, frequencias, color="skyblue", edgecolor="black")
plt.plot(valores, nbinom.pmf(valores, n=r, p=p_bn), 'o',
         color="red", markersize=6)
plt.plot(valores, poisson.pmf(valores, mu=r / taxa), '^',
         color="blue", markersize=6)
plt.xticks(valores)
```

([<matplotlib.axis.XTick object at 0x7b614d9c7750>, <matplotlib.axis.XTick object at 0x7b614d9c7750>], [
<matplotlib.axis.YTick object at 0x7b614d9c7750>, <matplotlib.axis.YTick object at 0x7b614d9c7750>])

```
plt.title("Contagens geradas pelo modelo hierárquico\n"
         "Círculos vermelhos: Binomial Negativa. "\n"
         "Triângulos azuis: Poisson de mesma média.")
plt.xlabel("Número de consultas")
plt.ylabel("Probabilidade")
plt.show()
```



As frequências observadas seguem de perto a Binomial Negativa. Já a Poisson de mesma média erra em dois lugares que se compensam: ela dá probabilidade pequena demais ao valor

0 e probabilidade grande demais aos valores intermediários. Em dados reais de contagem, é exatamente essa a assinatura de que uma Poisson é simples demais para o problema.

i Superdispersão

A variância teórica do modelo é maior que a média — 3,33 contra 2 —, enquanto na Poisson as duas coincidem. Esse excesso é chamado de **superdispersão**, e a lei da variância total mostra de onde ele vem:

$$\text{Var}(N) = \mathbb{E}[\text{Var}(N \mid \Lambda)] + \text{Var}(\mathbb{E}[N \mid \Lambda]) = \mathbb{E}[\Lambda] + \text{Var}(\Lambda),$$

usando que a Poisson tem média e variância iguais a Λ . A primeira parcela é a variabilidade que existiria se todas as pessoas tivessem a mesma taxa; a segunda é a variabilidade **entre** as pessoas, que a Poisson simples ignora.

i A mesma estrutura em inferência bayesiana

Vale registrar que o modelo deste exemplo é, palavra por palavra, um modelo bayesiano: a distribuição de Λ é o que se chama de **priori**, e a distribuição de $N \mid \Lambda$ é a **verossimilhança**. Gerar valores pelo pseudo-algoritmo acima é simular da **distribuição preditiva a priori** — as contagens que o modelo considera plausíveis antes de ver qualquer dado. Simular de modelos hierárquicos é, por isso, uma das operações mais frequentes em estatística bayesiana, e o algoritmo é sempre o mesmo: percorrer a hierarquia de cima para baixo, usando em cada estágio o valor sorteado no anterior.

79.1 Exercícios

Exercício 1. Este exercício explora a Weibull do Exemplo 3, cuja f.d.a. é $F(x) = 1 - e^{-(x/\lambda)^k}$ para $x > 0$.

- (a) Mostre que a mediana da Weibull é $\lambda(\log 2)^{1/k}$.
- (b) Implemente o pseudo-algoritmo do Exemplo 3 em uma função que receba B , k e λ e devolva uma amostra de tamanho B .
- (c) Com $\lambda = 2$ fixo, gere amostras de tamanho $B = 5000$ para $k = 0,5$, $k = 1$ e $k = 3$, e faça os três histogramas. Descreva como o formato da densidade muda com k .
- (d) Para cada uma das três amostras, compare a mediana amostral com o valor obtido em (a).
- (e) Verifique numericamente que, quando $k = 1$, a Weibull coincide com uma $\text{Exp}(1/\lambda)$: sobreponha ao histograma correspondente a densidade da exponencial.

Exercício 2. Continuando o Exemplo 7, implemente uma função que gere uma amostra de tamanho B de uma t_k pelo método da mistura.

- (a) Gere $B = 5000$ valores com $k = 3$ e com $k = 30$, e compare cada histograma com a densidade teórica (`dt` em R, `scipy.stats.t.pdf` em Python).
- (b) Sobreponha aos dois histogramas a densidade da $N(0, 1)$. O que acontece com a t_k quando k cresce? Explique o que ocorre com a variável de mistura Y/k quando $k \rightarrow \infty$ (dica: lei dos grandes números).
- (c) Estime $\mathbb{P}(X > 2)$ nos dois casos e compare com o valor correspondente para a normal padrão.
- (d) Usando a lei da variância total, $\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y])$, mostre que $\text{Var}(X) = k/(k - 2)$ para $k > 2$. Compare com a variância amostral obtida em (a). Por que a variância é maior do que 1, mesmo que $\text{Var}(X | Y = y)$ possa ser menor?

Exercício 3. Existe uma relação clássica entre as distribuições Poisson e Exponencial: se $X_1, X_2, \dots$ são independentes com $X_i \sim \text{Exp}(\lambda)$, e definimos N como o maior inteiro tal que $X_1 + \dots + X_N \leq 1$ (com $N = 0$ se $X_1 > 1$), então $N \sim \text{Poisson}(\lambda)$. Em outras palavras,

$$\mathbb{P}(N = j) = \mathbb{P}(X_1 + \dots + X_j \leq 1 < X_1 + \dots + X_{j+1}).$$

- (a) Escreva um pseudo-algoritmo que gere um valor de N somando exponenciais até que a soma ultrapasse 1.
- (b) Implemente o algoritmo, gere $B = 5000$ valores com $\lambda = 5$ e compare as frequências observadas com as probabilidades da Poisson (`dpois` em R, `scipy.stats.poisson.pmf` em Python).
- (c) Usando que $X_i = -\log(U_i)/\lambda$, mostre que a condição $X_1 + \dots + X_j \leq 1$ é equivalente a $U_1 U_2 \dots U_j \geq e^{-\lambda}$. Reescreva o algoritmo usando apenas produtos de uniformes, sem calcular logaritmos.
- (d) Quantos uniformes o algoritmo consome, em média, por valor gerado? Compare o valor observado com $\lambda + 1$.
- (e) (Desafio) Demonstre a relação enunciada acima. Use que $X_1 + \dots + X_j \sim \text{Gama}(j, \lambda)$ e escreva $\mathbb{P}(N = j) = \mathbb{P}(S_j \leq 1) - \mathbb{P}(S_{j+1} \leq 1)$, em que $S_j = X_1 + \dots + X_j$.

Exercício 4. O Exemplo 2 gera uma $\text{Gama}(n, \lambda)$ com parâmetro de forma inteiro, somando n exponenciais.

- (a) Implemente uma função que receba B , a e b e devolva uma amostra de tamanho B da distribuição $\text{Gama}(a, b)$, com a inteiro.
- (b) Compare a distribuição empírica dos valores simulados com a densidade da Gama $f(x) = \frac{b^a}{\Gamma(a)} x^{a-1} e^{-bx}$, $x > 0$.

Exercício 5. Sejam G_1 e G_2 independentes, com $G_1 \sim \text{Gama}(a, 1)$ e $G_2 \sim \text{Gama}(b, 1)$. Um resultado clássico afirma que

$$X = \frac{G_1}{G_1 + G_2} \sim \text{Beta}(a, b).$$

Note que essa é uma transformação de duas variáveis.

- Escreva um pseudo-algoritmo para gerar uma $\text{Beta}(2, 4)$ usando esse resultado. Como $a = 2$ e $b = 4$ são inteiros, cada Gama pode ser gerada somando exponenciais (Exemplo 2).
- Implemente o algoritmo, gere $B = 5000$ valores e compare o histograma com a densidade $f(x) = 20x(1-x)^3$ da $\text{Beta}(2, 4)$.
- Essa mesma distribuição foi gerada por rejeição no Exemplo 1 do Capítulo 6. Quantos uniformes o método daquele capítulo consome, em média, por valor gerado? E este aqui? Qual dos dois é mais eficiente?
- O que acontece com este método quando a ou b não são inteiros? E com o método da rejeição?

Exercício 6. O **modelo da normal contaminada** é uma mistura muito usada para representar dados com valores atípicos: com probabilidade $1 - \epsilon$ a observação vem de uma $N(0, 1)$ (“dados bem comportados”) e, com probabilidade ϵ , de uma $N(0, \sigma^2)$ com σ grande (“contaminação”). Use $\epsilon = 0,05$ e $\sigma = 5$.

- Escreva a densidade da mistura e o pseudo-algoritmo correspondente.
- Gere $B = 5000$ valores e compare o histograma com a densidade da mistura e com a densidade da $N(0, 1)$. Onde está a diferença entre as duas curvas?
- Mostre que $\text{Var}(X) = (1 - \epsilon) + \epsilon\sigma^2$ e compare com a variância amostral.
- Gere 1000 amostras de tamanho 30 dessa distribuição. Para cada uma, calcule a média e a mediana amostrais. Faça o histograma dos 1000 valores de cada estatística e compare suas variâncias. Qual das duas é menos afetada pela contaminação?
- Compare a amostra do item (b) com uma amostra de $0,95X_1 + 0,05X_2$, com $X_1 \sim N(0, 1)$ e $X_2 \sim N(0, 25)$ independentes. As duas construções produzem a mesma distribuição? (Compare os histogramas e releia o aviso da seção sobre misturas.)

Exercício 7. Em contagens reais é comum observar zeros demais para uma Poisson: pense no número de cigarros fumados por dia em uma amostra da população, em que boa parte das pessoas simplesmente não fuma. O modelo **Poisson inflacionada de zeros** trata disso como uma mistura: com probabilidade p a observação é o valor 0 (o indivíduo não é fumante) e, com probabilidade $1 - p$, ela vem de uma $\text{Poisson}(\lambda)$.

- (a) Mostre que $\mathbb{P}(X = 0) = p + (1 - p)e^{-\lambda}$ e que, para $j \geq 1$, $\mathbb{P}(X = j) = (1 - p)e^{-\lambda}\lambda^j/j!$.
- (b) Escreva o pseudo-algoritmo e implemente-o. Gere $B = 5000$ valores com $p = 0,3$ e $\lambda = 4$ (você pode usar o gerador do Exercício 3, ou as funções prontas `rpois` e `np.random.poisson`).
- (c) Compare as frequências relativas observadas com as probabilidades do item (a), usando um gráfico de barras.
- (d) Mostre que $\mathbb{E}[X] = (1 - p)\lambda$ e $\text{Var}(X) = (1 - p)\lambda(1 + p\lambda)$. Compare com a média e a variância amostrais. Por que dizemos que esse modelo apresenta **superdispersão** em relação à Poisson?
- (e) Ajuste uma Poisson aos dados simulados, isto é, calcule $\hat{\lambda} = \bar{X}$ e desenhe as probabilidades da $\text{Poisson}(\hat{\lambda})$ sobre o gráfico do item (c). Onde o ajuste falha?

Exercício 8. O Exemplo 8 construiu uma contagem a partir de uma **taxa** aleatória. Este exercício faz o mesmo com uma **proporção** aleatória. Suponha que cada aluno de uma turma acerte cada uma das m questões de uma prova com probabilidade P , e que essa probabilidade varie de aluno para aluno:

$$P \sim \text{Beta}(a, b), \quad X \mid P = p \sim \text{Binomial}(m, p).$$

Use $m = 10$, $a = 2$ e $b = 3$.

- (a) Escreva o pseudo-algoritmo para gerar X . Explique por que a Beta é uma escolha natural para P . (Para gerar a Beta você pode usar o método do Exercício 5, ou as funções prontas `rbeta` e `np.random.beta`.)
- (b) Implemente-o e gere $B = 5000$ valores. Compare as frequências observadas com as probabilidades da distribuição **Beta-Binomial**,

$$\mathbb{P}(X = k) = \binom{m}{k} \frac{B(k + a, m - k + b)}{B(a, b)},$$

em que $B(\cdot, \cdot)$ é a função beta (`beta` em R, `scipy.special.beta` em Python).

- (c) Compare a média e a variância amostrais com as de uma amostra de $\text{Binomial}(m, a/(a + b))$, que tem a mesma média teórica. Qual das duas é mais dispersa?
- (d) Use a lei da variância total para mostrar que

$$\text{Var}(X) = m \mathbb{E}[P] (1 - \mathbb{E}[P]) + m(m-1) \text{Var}(P),$$

e explique em uma frase de onde vem a parcela extra em relação à Binomial.

Exercício 9. Sejam $U_1, \dots, U_n$ independentes e $\text{Unif}(0, 1)$, e seja $M = \max(U_1, \dots, U_n)$.

- Mostre que $F_M(x) = x^n$ para $0 < x < 1$ e conclua, pelo método da inversão, que M tem a mesma distribuição de $U^{1/n}$, com $U \sim \text{Unif}(0, 1)$.
- Gere $B = 10\,000$ valores de M com $n = 10$ pelos dois caminhos — tomando o máximo de 10 uniformes, e aplicando $U^{1/n}$ a um único uniforme — e compare os histogramas.
- Quanto uniformes cada caminho consome? Meça o tempo de execução dos dois para $n = 1000$ (com `system.time` em R ou `time.time` em Python) e comente.
- O mesmo raciocínio vale para o mínimo: mostre que $\min(U_1, \dots, U_n)$ tem a mesma distribuição de $1 - U^{1/n}$.
- Mais geralmente, a j -ésima menor observação de n uniformes tem distribuição $\text{Beta}(j, n-j+1)$. Use o método do Exercício 5 para gerar diretamente a 3ª menor de 10 uniformes, e compare com o resultado de ordenar 10 uniformes e tomar a terceira.

Exercício 10. Uma seguradora quer estudar o total pago em sinistros durante um mês. O número de sinistros é $N \sim \text{Poisson}(\lambda)$ e, dado $N = n$, os valores individuais $X_1, \dots, X_n$ são independentes e $\text{Exp}(1/\mu)$ (isto é, com média μ). O total é

$$S = \sum_{i=1}^N X_i, \quad \text{com } S = 0 \text{ se } N = 0.$$

Essa é uma mistura (primeiro sortecemos N) combinada com uma transformação (somamos os X_i). Use $\lambda = 3$ e $\mu = 1000$.

- Escreva o pseudo-algoritmo e implemente-o, gerando $B = 5000$ valores de S .
- Faça o histograma de S . Por que ele tem uma barra isolada em zero? Qual é o valor teórico de $\mathbb{P}(S = 0)$? Compare com a proporção observada.
- Estime $\mathbb{E}[S]$ e compare com o valor teórico $\mathbb{E}[S] = \lambda\mu$ (dica: $\mathbb{E}[S] = \mathbb{E}[\mathbb{E}[S | N]]$).
- Estime $\mathbb{P}(S > 5000)$, a probabilidade de o mês custar mais de 5000 à seguradora.
- Mostre que, condicionalmente a $N = n \geq 1$, $S \sim \text{Gama}(n, 1/\mu)$, e escreva a densidade de S na região $s > 0$ como uma soma infinita. Sobreponha essa densidade (truncando a soma em $n = 30$) ao histograma do item (b), lembrando de descartar os valores nulos.

Exercício 11. Sobre a normal multivariada do Exemplo 5.

- (a) Implemente uma função que receba B , o vetor μ e a matriz Σ e devolva uma matriz com B linhas e d colunas, em que cada linha é um vetor gerado. Use-a para reproduzir o exemplo e confira as médias, os desvios padrão e as correlações amostrais.
- (b) O que acontece se você esquecer a transposta, usando `chol(Sigma)` no lugar de `t(chol(Sigma))` em R (ou `np.linalg.cholesky(Sigma).T` em Python)? Gere 2000 vetores dessa forma e compare as médias, os desvios e as correlações amostrais com os valores pedidos. Quais das três características saem erradas?
- (c) Estime $\mathbb{P}(X_1 > 180 \text{ e } X_2 > 80)$, a probabilidade de a pessoa ser ao mesmo tempo alta e pesada. Compare com o produto $\mathbb{P}(X_1 > 180)\mathbb{P}(X_2 > 80)$, calculado a partir das marginais, e explique a diferença.
- (d) Verifique numericamente que qualquer combinação linear das coordenadas ainda é normal: faça o histograma de $X_1 + X_2 + X_3$ e sobreponha a densidade da normal de média $\mu_1 + \mu_2 + \mu_3$ e variância igual à soma de **todas** as entradas de Σ .
- (e) Troque a correlação entre peso e cintura de 0,8 para 0,9 e depois para $-0,9$, mantendo as outras duas. Em um dos casos a decomposição de Cholesky falha. Qual? Calcule o determinante das duas matrizes de correlação e explique o que a falha significa: por que não pode existir um vetor aleatório com essas três correlações ao mesmo tempo?

Exercício 12. (Desafio) O Exemplo 2 usa o fato de que a soma de n exponenciais independentes de taxa λ tem distribuição Gama(n, λ), mas não o demonstra.

- (a) Prove esse resultado por indução em n . Para o passo indutivo, escreva a densidade de $S_{n+1} = S_n + X_{n+1}$ como a convolução

$$f_{S_{n+1}}(s) = \int_0^s f_{S_n}(t) f_{X_{n+1}}(s-t) dt.$$

- (b) Conclua que a Gama($1, \lambda$) é a própria Exp(λ) e que $\chi_2^2 = \text{Gama}(1, 1/2) = \text{Exp}(1/2)$ — fato usado no próximo capítulo.
- (c) Explique por que o argumento não diz nada sobre Gama(a, λ) com a não inteiro, e cite um método deste livro que resolveria esse caso.

Exercício 13. (Desafio) A distribuição de Laplace (ou dupla exponencial) tem densidade

$$f(x) = \frac{1}{2}e^{-|x|}, \quad x \in \mathbb{R}.$$

Ela pode ser construída de duas maneiras completamente diferentes.

- (a) **Como transformação de uma exponencial com sinal aleatório.** Sejam $E \sim \text{Exp}(1)$ e S independente de E , com $\mathbb{P}(S = 1) = \mathbb{P}(S = -1) = 1/2$. Mostre que $X = S \cdot E$ tem densidade f .
- (b) **Como mistura de escala de normais.** Sejam $W \sim \text{Exp}(1)$ e $Z \sim N(0, 1)$ independentes. Mostre que $X = \sqrt{2W} Z$ também tem densidade f . (Dica: calcule a função geradora de momentos condicionando em W , use que $\mathbb{E}[e^{tX} \mid W = w] = e^{t^2 w}$ e verifique que $\mathbb{E}[e^{tX}] = 1/(1 - t^2)$ para $|t| < 1$, que é a f.g.m. da Laplace.)
- (c) Implemente as duas construções, gere $B = 5000$ valores por cada uma e compare os histogramas com f .
- (d) Quantos uniformes cada construção consome por valor gerado? Qual você usaria na prática?
- (e) Compare a estrutura do item (b) com a do Exemplo 7. Em ambos, $X \mid V$ é normal de média zero e variância aleatória; o que muda é a distribuição de V . O que isso sugere sobre a origem das caudas pesadas nas duas distribuições?

80 Método de Box-Muller

No Capítulo 6 já geramos valores de uma $N(0,1)$ pelo método da rejeição: propúnhamos uma $\text{Exp}(1)$, aceitávamos cerca de 76% dos candidatos e sorteávamos um sinal no final. O algoritmo funciona, mas cobra caro — são necessários, em média, cerca de 3,6 uniformes para cada normal produzida, e há um laço que pode, em princípio, demorar.

Este capítulo apresenta um método que faz melhor: a partir de **exatamente dois** uniformes, ele devolve **duas** normais padrão independentes, sem rejeitar nada. É o método de Box-Muller, e ele é um caso particular do método da transformação do capítulo anterior — com uma escolha de transformação que ninguém adivinharia olhando apenas para a densidade da normal.

O truque é justamente esse: em vez de atacar uma normal de cada vez, atacamos **duas ao mesmo tempo**.

80.1 A ideia: olhar para o par, não para a coordenada

Por que olhar para duas normais ajudaria, se nem uma sabemos gerar por inversão? Porque o **par** tem uma estrutura que a coordenada isolada não tem. Se $X \sim N(0,1)$ e $Y \sim N(0,1)$ são independentes, a densidade conjunta é

$$f_{X,Y}(x,y) = \frac{1}{\sqrt{2\pi}}e^{-x^2/2} \cdot \frac{1}{\sqrt{2\pi}}e^{-y^2/2} = \frac{1}{2\pi}e^{-(x^2+y^2)/2}.$$

Repare que (x,y) aparece **somente** através de $x^2 + y^2$, ou seja, através da distância do ponto à origem. Dois pontos que estejam à mesma distância da origem têm exatamente a mesma densidade, não importa em que direção estejam. A nuvem de pontos (X,Y) é circularmente simétrica: não tem direção preferida.

E a maneira natural de descrever algo que só depende da distância à origem é trocar as coordenadas cartesianas (x,y) pelas **coordenadas polares**: a distância r e o ângulo θ . É o que faremos a seguir — e a surpresa é que, nessas coordenadas, as duas variáveis resultantes são fáceis de simular.

80.2 Representação em coordenadas polares

Considere duas variáveis aleatórias $X \sim N(0, 1)$ e $Y \sim N(0, 1)$, independentes entre si. No plano cartesiano, podemos representar o ponto (X, Y) e descrevê-lo pelo par (R, Θ) , em que R é a distância do ponto à origem e Θ é o ângulo formado com o eixo horizontal.

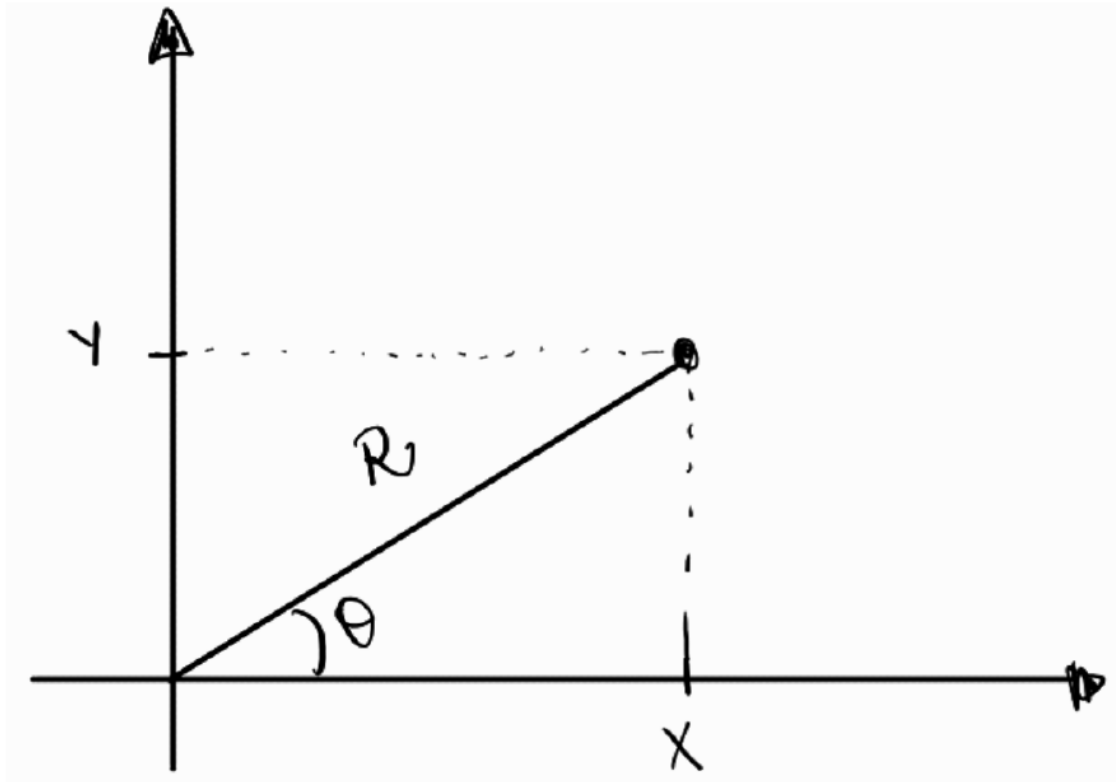


Figure 80.1: Representação em Coordenadas Polares

As relações trigonométricas para Θ são

$$\sin \Theta = \frac{\text{cateto oposto}}{\text{hipotenusa}} = \frac{Y}{R} \quad \text{e} \quad \cos \Theta = \frac{\text{cateto adjacente}}{\text{hipotenusa}} = \frac{X}{R}.$$

Dessa forma, obtemos as duas relações que usaremos o capítulo inteiro:

$$X = R \cos \Theta, \quad Y = R \sin \Theta, \quad \text{com} \quad R^2 = X^2 + Y^2.$$

80.3 A distribuição de R e Θ

As variáveis R e Θ são uma transformação do par (X, Y) . Para obter a densidade conjunta de (R, Θ) , usamos a fórmula de mudança de variáveis em duas dimensões:

$$f_{R,\Theta}(r, \theta) = f_{X,Y}(h_1(r, \theta), h_2(r, \theta)) |J_h(r, \theta)|,$$

em que $h_1(r, \theta) = r \cos \theta$ e $h_2(r, \theta) = r \sin \theta$ são as transformações inversas (as que expressam X e Y em termos de R e Θ), e J_h é o Jacobiano dessas funções:

$$J_h(r, \theta) = \begin{vmatrix} \frac{\partial h_1}{\partial r} & \frac{\partial h_1}{\partial \theta} \\ \frac{\partial h_2}{\partial r} & \frac{\partial h_2}{\partial \theta} \end{vmatrix} = \begin{vmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{vmatrix} = r(\cos^2 \theta + \sin^2 \theta) = r.$$

! Proposição: a versão polar de duas normais

Sejam X e Y independentes, ambas $N(0, 1)$, e sejam $R \geq 0$ e $\Theta \in (0, 2\pi)$ suas coordenadas polares. Então R e Θ são **independentes**, com

$$\Theta \sim \text{Unif}(0, 2\pi) \quad \text{e} \quad R^2 \sim \text{Exp}(1/2).$$

i Demonstração

Substituindo $f_{X,Y}(x, y) = \frac{1}{2\pi} e^{-(x^2+y^2)/2}$ na fórmula da mudança de variáveis, e usando que $(r \cos \theta)^2 + (r \sin \theta)^2 = r^2$, obtemos

$$f_{R,\Theta}(r, \theta) = \frac{1}{2\pi} e^{-r^2/2} r = \underbrace{\frac{1}{2\pi}}_{f_{\Theta}(\theta)} \cdot \underbrace{r e^{-r^2/2}}_{f_R(r)}, \quad 0 < r < \infty, \quad 0 < \theta < 2\pi.$$

A densidade conjunta **fatorou** em um termo que só depende de θ e outro que só depende de r : é exatamente isso que significa R e Θ serem independentes. Como cada fator é uma densidade legítima em seu domínio, lemos diretamente que

$$f_{\Theta}(\theta) = \frac{1}{2\pi}, \quad 0 < \theta < 2\pi, \quad \text{e} \quad f_R(r) = r e^{-r^2/2}, \quad r > 0,$$

isto é, $\Theta \sim \text{Unif}(0, 2\pi)$ e $R \sim \text{Rayleigh}(1)$.

Falta passar de R para R^2 . Para $w > 0$,

$$\mathbb{P}(R^2 \leq w) = \mathbb{P}(R \leq \sqrt{w}) = \int_0^{\sqrt{w}} r e^{-r^2/2} dr = 1 - e^{-w/2},$$

que é a f.d.a. de uma $\text{Exp}(1/2)$ (exponencial de taxa 1/2, e portanto de média 2). $\square$

Rayleigh, qui-quadrado e exponencial

A distribuição de R tem nome: Rayleigh(1). E a de R^2 tem dois: como $R^2 = X^2 + Y^2$ é a soma dos quadrados de duas normais padrão independentes, pela definição do Capítulo 7 temos $R^2 \sim \chi_2^2$. A demonstração acima mostra que essa χ_2^2 é a mesma coisa que uma $\text{Exp}(1/2)$ — fato que também aparece no Exercício 12 do Capítulo 7, por outro caminho. Não é uma coincidência sem graça: é justamente ela que torna o método viável, porque exponenciais nós sabemos gerar por inversão desde o Capítulo 4.

80.4 O algoritmo

A proposição acima descreve o caminho de ida: **das** normais **para** as coordenadas polares. O método de Box-Muller percorre esse caminho ao contrário. Se conseguirmos gerar R e Θ com as distribuições certas, as fórmulas $X = R \cos \Theta$ e $Y = R \sin \Theta$ nos devolvem o par de normais.

E gerar R e Θ é fácil, porque as duas são independentes e cada uma sai de um único uniforme:

- $\Theta \sim \text{Unif}(0, 2\pi)$ é apenas uma mudança de escala: $\Theta = 2\pi U_2$;
- $R^2 \sim \text{Exp}(1/2)$ sai por inversão (Capítulo 4): como $F(w) = 1 - e^{-w/2}$, temos $R^2 = -2 \log(1 - U_1)$, ou, trocando $1 - U_1$ por U_1 (a mesma observação feita no Capítulo 7), $R^2 = -2 \log U_1$. Daí $R = \sqrt{-2 \log U_1}$.

Pseudo-algoritmo: Box-Muller

1. Gere $U_1 \sim \text{Unif}(0, 1)$ e $U_2 \sim \text{Unif}(0, 1)$, independentes.
2. Calcule o raio $R = \sqrt{-2 \log U_1}$ e o ângulo $\Theta = 2\pi U_2$.
3. Devolva o par

$$X = R \cos \Theta = \sqrt{-2 \log U_1} \cos(2\pi U_2), \quad Y = R \sin \Theta = \sqrt{-2 \log U_1} \sin(2\pi U_2).$$

Proposição: validade do método

As variáveis X e Y devolvidas pelo algoritmo são independentes e ambas têm distribuição $N(0, 1)$.

Demonstração

O passo 2 produz $\Theta \sim \text{Unif}(0, 2\pi)$ e $R^2 \sim \text{Exp}(1/2)$, independentes (porque U_1 e U_2 o são). Logo, o par (R, Θ) gerado tem exatamente a densidade conjunta

$$f_{R,\Theta}(r, \theta) = \frac{1}{2\pi} r e^{-r^2/2}$$

obtida na proposição anterior.

Falta ver que aplicar $x = r \cos \theta$, $y = r \sin \theta$ a esse par devolve a densidade conjunta de duas normais padrão. Usamos de novo a fórmula de mudança de variáveis, agora no sentido contrário: o Jacobiano da transformação que leva (x, y) em (r, θ) é o inverso do anterior, ou seja, $1/r$. Como $r = \sqrt{x^2 + y^2}$,

$$f_{X,Y}(x, y) = f_{R,\Theta}(r, \theta) \cdot \frac{1}{r} = \frac{1}{2\pi} r e^{-r^2/2} \cdot \frac{1}{r} = \frac{1}{2\pi} e^{-(x^2+y^2)/2} = \frac{e^{-x^2/2}}{\sqrt{2\pi}} \cdot \frac{e^{-y^2/2}}{\sqrt{2\pi}}.$$

A densidade conjunta fatorou no produto de duas densidades $N(0, 1)$, o que significa que X e Y são independentes e cada uma é $N(0, 1)$. $\square$

Vale registrar o que o método tem de notável: ele é **exato** (não é uma aproximação, como somar doze uniformes e apelar para o Teorema Central do Limite), **não rejeita nada** (todo par de uniformes vira um par de normais) e consome exatamente **um uniforme por normal gerada** — contra os 3,6 do método da rejeição do Capítulo 6.

Atenção: três armadilhas de implementação

- **log é logaritmo natural** tanto em R quanto em Python (`np.log`). Se você usar `log10`, o método simplesmente gera outra distribuição, sem dar erro.
- **Os senos e cossenos estão em radianos.** É por isso que $\Theta = 2\pi U_2$ entra direto em `cos()` e `sin()`, sem conversão para graus.
- U_1 **não pode valer 0**, senão $\log U_1 = -\infty$ e o algoritmo devolve `Inf` ou `NaN`. O `runif` do R e o `np.random.uniform` do Python não devolvem esse valor na prática, mas um gerador escrito por você (como o LCG do Capítulo 2) pode devolver — veja o Exercício 11.
- Em Python, **^ não é exponenciação**: use `**`. Escrever `V1^2` não dá erro, mas calcula um “ou exclusivo” bit a bit.

80.5 Exemplo 1: gerando normais padrão

O código abaixo implementa o pseudo-algoritmo em uma função que devolve B pares. Como cada par traz duas normais, geramos $B = 2500$ pares para obter 5000 valores.

Além de comparar o histograma com a densidade da $N(0, 1)$, fazemos duas conferências que dizem respeito ao par: a correlação entre X e Y (que deve ser próxima de zero) e o gráfico de dispersão dos pares, que deve exibir a nuvem circular discutida no início do capítulo.

81 R

```
library(ggplot2)

set.seed(42)

# Gera B pares de normais padrão pelo método de Box-Muller.
# Devolve um data.frame com colunas x e y: cada linha é um par (X, Y)
box_muller <- function(B) {
  X <- numeric(B)
  Y <- numeric(B)

  for (i in 1:B) {
    # Passo 1: dois uniformes independentes
    U1 <- runif(1)
    U2 <- runif(1)

    # Passo 2: o raio e o ângulo. Em R, log() é o logaritmo natural
    R <- sqrt(-2 * log(U1))
    Theta <- 2 * pi * U2

    # Passo 3: de coordenadas polares de volta para as cartesianas.
    # cos e sin esperam o ângulo em radianos, que é o que Theta já é
    X[i] <- R * cos(Theta)
    Y[i] <- R * sin(Theta)
  }

  return(data.frame(x = X, y = Y))
}

B <- 2500 # número de PARES: no fim teremos 2 * B = 5000 valores normais
pares <- box_muller(B)

cat("X: média", round(mean(pares$x), 3),
    "e desvio padrão", round(sd(pares$x), 3), "\n")
```

X: média 0.04 e desvio padrão 0.99

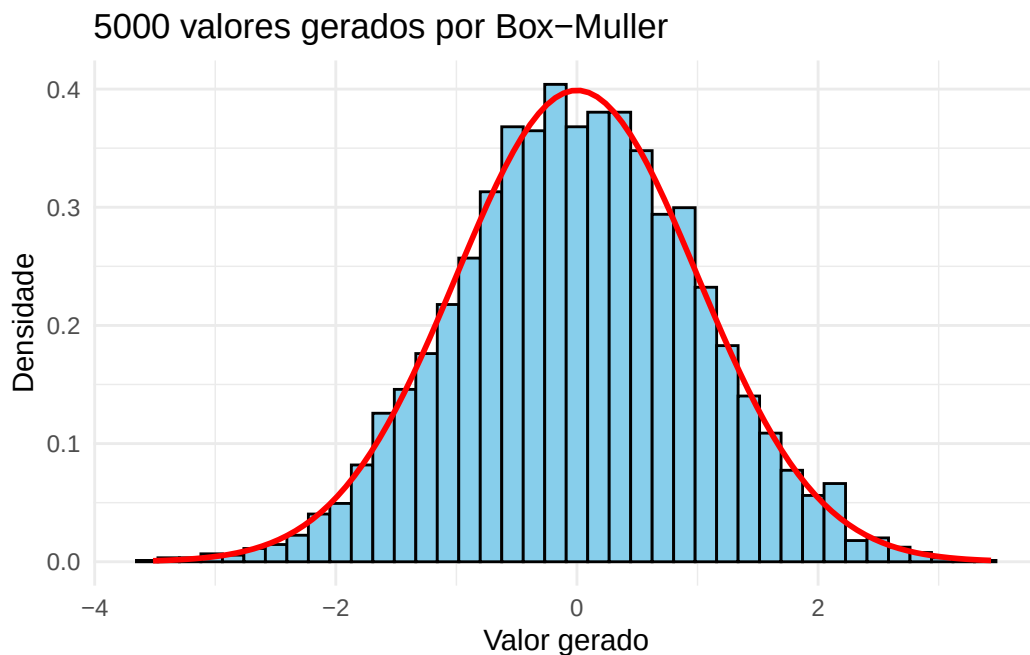
```
cat("Y: média", round(mean(pares$y), 3),  
    "e desvio padrão", round(sd(pares$y), 3), "\n")
```

Y: média -0.02 e desvio padrão 1.015

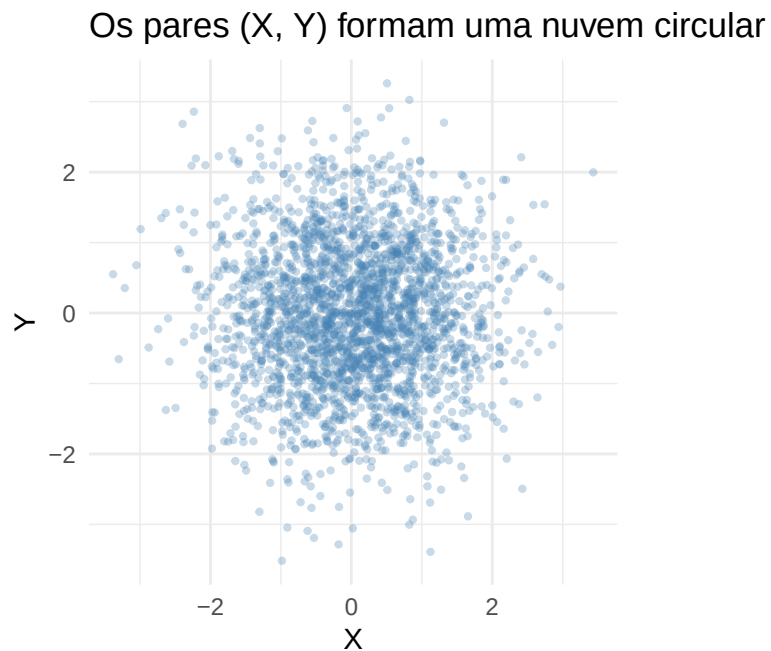
```
cat("Correlação entre X e Y:", round(cor(pares$x, pares$y), 3), "\n")
```

Correlação entre X e Y: -0.009

```
# As duas colunas juntas: 2B valores, todos N(0,1)  
df <- data.frame(z = c(pares$x, pares$y))  
  
ggplot(df, aes(x = z)) +  
  geom_histogram(aes(y = after_stat(density)), bins = 40,  
                 fill = "skyblue", color = "black") +  
  stat_function(fun = dnorm, color = "red", linewidth = 1) +  
  labs(title = "5000 valores gerados por Box-Muller",  
       x = "Valor gerado", y = "Densidade") +  
  theme_minimal()
```




```
# coord_fixed deixa as duas escalas iguais: sem isso, um círculo pareceria
# uma ellipse
ggplot(pares, aes(x = x, y = y)) +
  geom_point(alpha = 0.3, size = 0.8, color = "steelblue") +
  coord_fixed() +
  labs(title = "Os pares (X, Y) formam uma nuvem circular",
       x = "X", y = "Y") +
  theme_minimal()
```



82 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)

# Gera B pares de normais padrão pelo método de Box-Muller.
# Devolve dois vetores de tamanho B: o i-ésimo par é (X[i], Y[i])
def box_muller(B):
    X = np.zeros(B)
    Y = np.zeros(B)

    for i in range(B):
        # Passo 1: dois uniformes independentes
        U1 = np.random.uniform(0, 1)
        U2 = np.random.uniform(0, 1)

        # Passo 2: o raio e o ângulo. np.log é o logaritmo natural
        R = np.sqrt(-2 * np.log(U1))
        Theta = 2 * np.pi * U2

        # Passo 3: de coordenadas polares de volta para as cartesianas.
        # cos e sin esperam o ângulo em radianos, que é o que Theta já é
        X[i] = R * np.cos(Theta)
        Y[i] = R * np.sin(Theta)

    return X, Y

B = 2500 # número de PARES: no fim teremos 2 * B = 5000 valores normais
X, Y = box_muller(B)

print("X: média", round(X.mean(), 3),
      "e desvio padrão", round(X.std(ddof=1), 3))
```

X: média 0.009 e desvio padrão 1.036

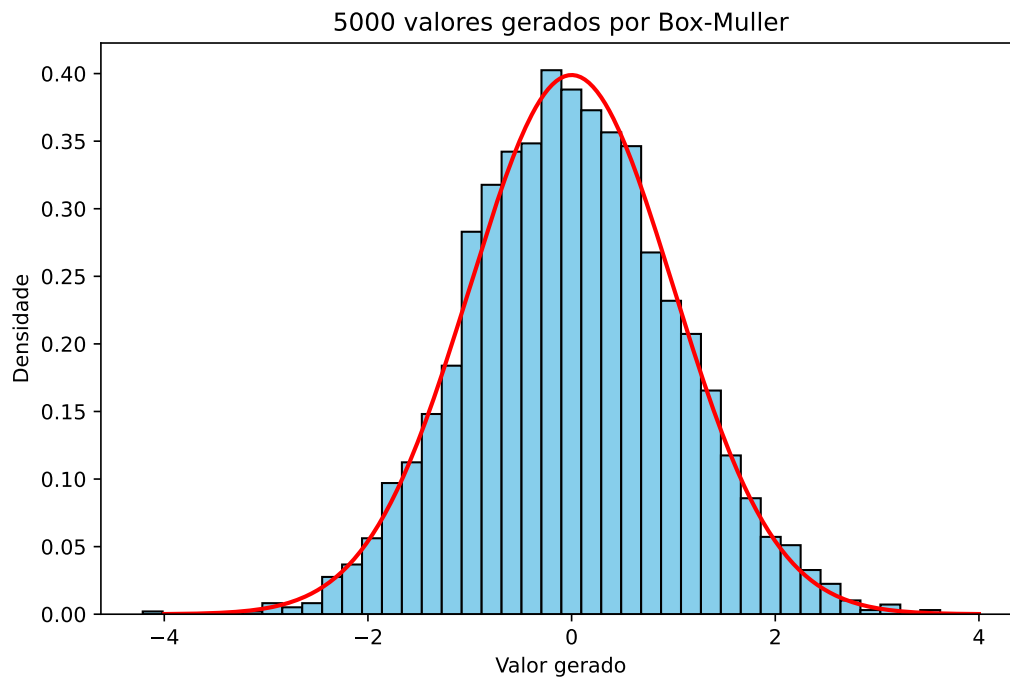
```
print("Y: média", round(Y.mean(), 3),  
      "e desvio padrão", round(Y.std(ddof=1), 3))
```

Y: média 0.006 e desvio padrão 0.987

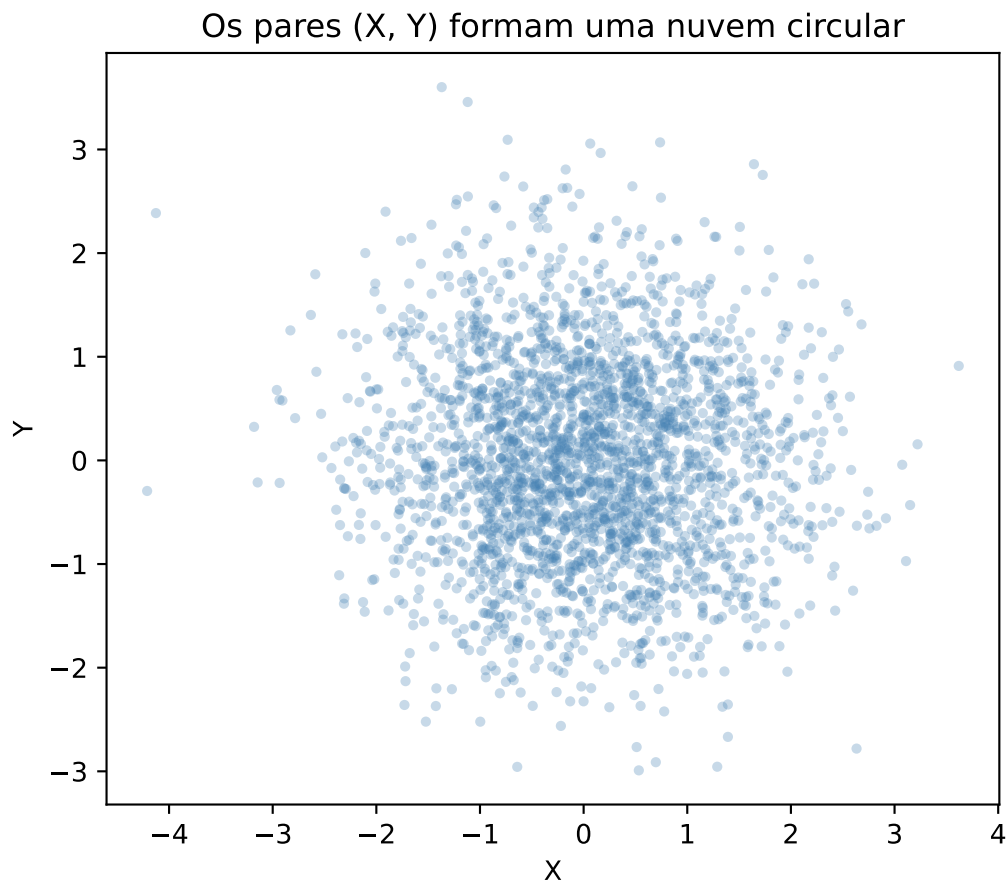
```
print("Correlação entre X e Y:", round(np.corrcoef(X, Y)[0, 1], 3))
```

Correlação entre X e Y: -0.039

```
# Os dois vetores juntos: 2B valores, todos N(0,1)  
Z = np.concatenate([X, Y])  
  
# Malha usada só para desenhar a densidade teórica  
grade = np.linspace(-4, 4, 400)  
  
plt.figure(figsize=(8, 5))  
plt.hist(Z, bins=40, density=True, color="skyblue", edgecolor="black")  
plt.plot(grade, norm.pdf(grade), color="red", linewidth=2)  
plt.title("5000 valores gerados por Box-Muller")  
plt.xlabel("Valor gerado")  
plt.ylabel("Densidade")  
plt.show()
```



```
# set_aspect deixa as duas escalas iguais: sem isso, um círculo pareceria
# uma elipse
plt.figure(figsize=(6, 6))
plt.scatter(X, Y, alpha=0.3, s=8, color="steelblue")
plt.gca().set_aspect("equal")
plt.title("Os pares (X, Y) formam uma nuvem circular")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



82.1 Exemplo 2: de $N(0, 1)$ para $N(\mu, \sigma^2)$

O método entrega normais **padrão**, mas na prática quase sempre queremos uma normal com média e variância dadas. A ponte é uma transformação afim, que já é um uso do método do Capítulo 7.

! Proposição

Se $Z \sim N(0, 1)$, então $X = \mu + \sigma Z \sim N(\mu, \sigma^2)$, para quaisquer $\mu \in \mathbb{R}$ e $\sigma > 0$.

i Demonstração

A função $g(z) = \mu + \sigma z$ é estritamente crescente (pois $\sigma > 0$), com inversa $g^{-1}(x) = (x - \mu)/\sigma$. Pela fórmula da densidade de uma transformação monótona (Capítulo 7),

$$f_X(x) = f_Z\left(\frac{x - \mu}{\sigma}\right) \cdot \frac{1}{\sigma} = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x - \mu}{\sigma}\right)^2},$$

que é a densidade da $N(\mu, \sigma^2)$. $\square$

Note que σ é o **desvio padrão**, não a variância: essa é uma fonte frequente de erro, agravada pelo fato de que R e Python discordam da notação usual — `rnorm` e `np.random.normal` pedem o desvio padrão, enquanto a notação $N(\mu, \sigma^2)$ exhibe a variância.

Vamos simular alturas de uma população adulta, modeladas por uma $N(170, 8^2)$ (em centímetros), usando a função `box_muller` do Exemplo 1.

83 R

```
set.seed(2024)

mu <- 170      # média desejada
sigma <- 8     # desvio padrão desejado

pares <- box_muller(2500)
Z <- c(pares$x, pares$y)  # 5000 valores N(0,1)

# A transformação afim que desloca e reescala
X <- mu + sigma * Z

cat("Média amostral:", round(mean(X), 2), " (teórica:", mu, ")\n")
```

Média amostral: 169.91 (teórica: 170)

```
cat("Desvio padrão amostral:", round(sd(X), 2), " (teórico:", sigma, ")\n")
```

Desvio padrão amostral: 7.86 (teórico: 8)

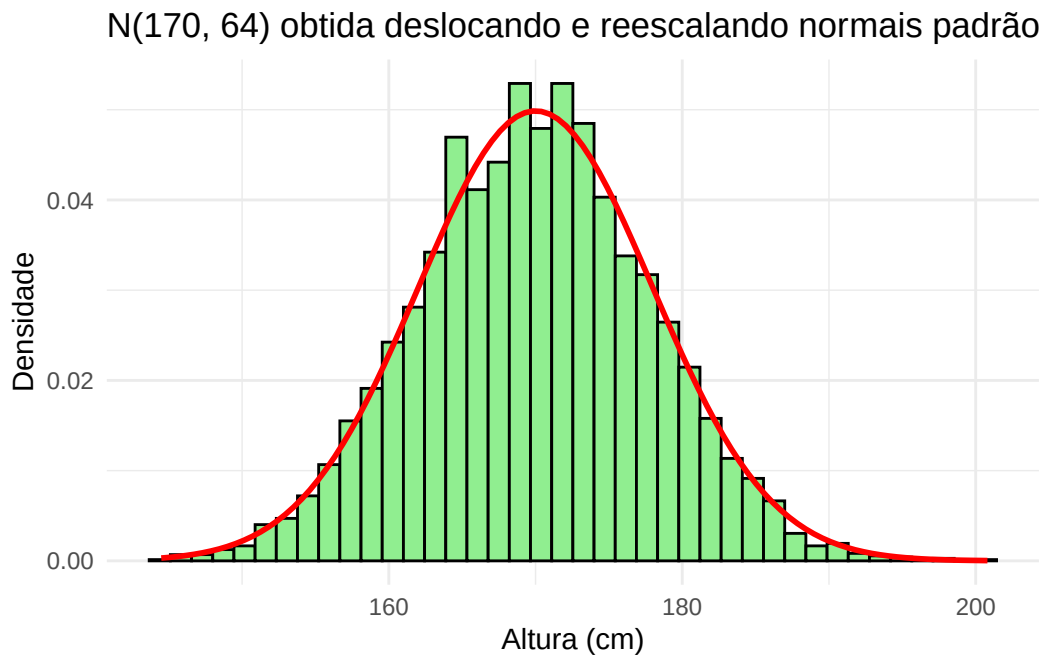
```
cat("Proporção de valores acima de 186 cm:", round(mean(X > 186), 4), "\n")
```

Proporção de valores acima de 186 cm: 0.0182

```
cat("Valor teórico:", round(1 - pnorm(186, mean = mu, sd = sigma), 4), "\n")
```

Valor teórico: 0.0228

```
ggplot(data.frame(x = X), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
    fill = "lightgreen", color = "black") +
  stat_function(fun = function(x) dnorm(x, mean = mu, sd = sigma),
    color = "red", linewidth = 1) +
  labs(title = "N(170, 64) obtida deslocando e reescalando normais padrão",
    x = "Altura (cm)", y = "Densidade") +
  theme_minimal()
```



84 Python

```
np.random.seed(2024)

mu = 170      # média desejada
sigma = 8     # desvio padrão desejado

X1, Y1 = box_muller(2500)
Z = np.concatenate([X1, Y1]) # 5000 valores N(0,1)

# A transformação afim que desloca e reescala
X = mu + sigma * Z

print("Média amostral:", round(X.mean(), 2), " (teórica:", mu, ")")
```

Média amostral: 169.93 (teórica: 170)

```
print("Desvio padrão amostral:", round(X.std(ddof=1), 2),
      " (teórico:", sigma, ")")
```

Desvio padrão amostral: 8.12 (teórico: 8)

```
print("Proporção de valores acima de 186 cm:", round(np.mean(X > 186), 4))
```

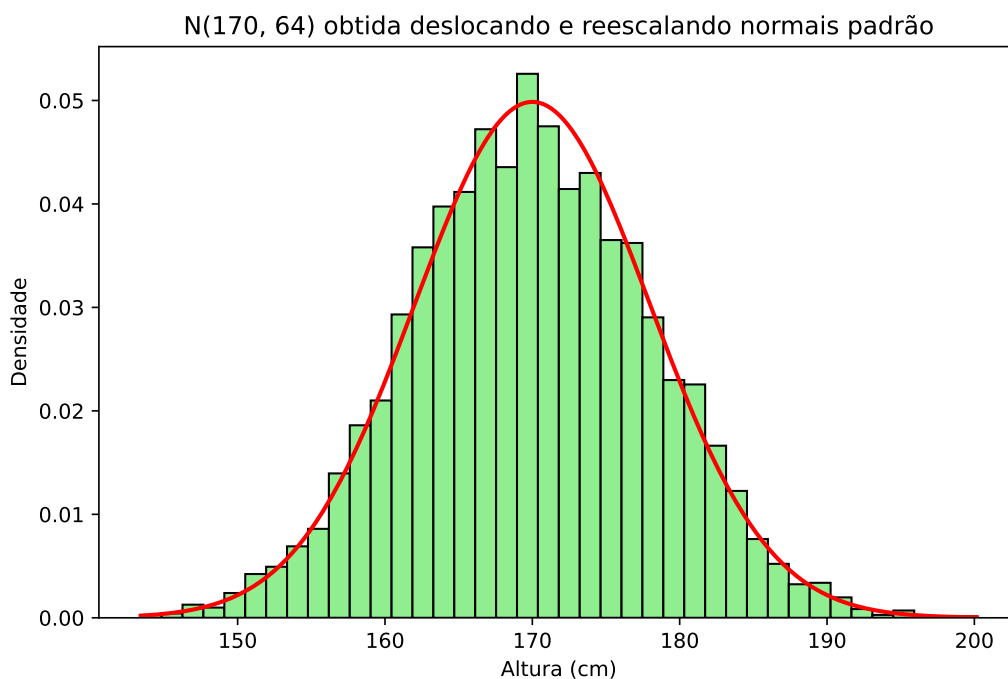
Proporção de valores acima de 186 cm: 0.0228

```
print("Valor teórico:", round(1 - norm.cdf(186, loc=mu, scale=sigma), 4))
```

Valor teórico: 0.0228

```
# Malha usada só para desenhar a densidade teórica
grade = np.linspace(X.min(), X.max(), 400)

plt.figure(figsize=(8, 5))
plt.hist(X, bins=40, density=True, color="lightgreen", edgecolor="black")
plt.plot(grade, norm.pdf(grade, loc=mu, scale=sigma), color="red", linewidth=2)
plt.title("N(170, 64) obtida deslocando e reescalando normais padrão")
plt.xlabel("Altura (cm)")
plt.ylabel("Densidade")
plt.show()
```



84.1 Exemplo 3: um par de normais correlacionadas

Até aqui tratamos as duas normais devolvidas pelo método como valores separados. Mas há situações em que queremos justamente um **par** — duas medidas do mesmo indivíduo, como peso e altura, que não são independentes. Nesses casos, o fato de o Box-Muller já produzir pares é uma conveniência e tanto.

Queremos gerar (X, Y) com distribuição normal bivariada, ambas com média 0 e variância 1, e com $\text{corr}(X, Y) = \rho$. A construção é uma transformação do par de normais independentes (Z_1, Z_2) :

$$X = Z_1, \quad Y = \rho Z_1 + \sqrt{1 - \rho^2} Z_2.$$

A intuição é que Y empresta uma fração ρ do valor de X e completa o resto com ruído independente; o coeficiente $\sqrt{1 - \rho^2}$ é exatamente o que faz a variância de Y voltar a valer 1:

$$\text{Var}(Y) = \rho^2 \text{Var}(Z_1) + (1 - \rho^2) \text{Var}(Z_2) = 1, \quad \text{Cov}(X, Y) = \rho \text{Var}(Z_1) = \rho.$$

💡 Pseudo-algoritmo: normal bivariada com correlação ρ

1. Gere Z_1 e Z_2 independentes, ambas $N(0, 1)$, por Box-Muller.
2. Devolva $X = Z_1$ e $Y = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$.

Como conferência, além da correlação amostral, comparamos a proporção de pontos no primeiro quadrante com o valor teórico, que para a normal bivariada padrão é

$$\mathbb{P}(X > 0, Y > 0) = \frac{1}{4} + \frac{\arcsin \rho}{2\pi}.$$

Quando $\rho = 0$ essa expressão vale $1/4$, como esperado para variáveis independentes e simétricas; quanto maior ρ , mais os pontos se concentram nos quadrantes em que X e Y têm o mesmo sinal.

85 R

```
set.seed(7)

rho <- 0.8 # correlação desejada
B <- 2000

pares <- box_muller(B)
Z1 <- pares$x
Z2 <- pares$y

# X é a primeira normal; Y mistura as duas para criar a correlação
X <- Z1
Y <- rho * Z1 + sqrt(1 - rho^2) * Z2

cat("Correlação amostral:", round(cor(X, Y), 3), " (teórica:", rho, ")\n")
```

Correlação amostral: 0.808 (teórica: 0.8)

```
cat("Desvio padrão de Y:", round(sd(Y), 3), " (teórico: 1)\n")
```

Desvio padrão de Y: 0.999 (teórico: 1)

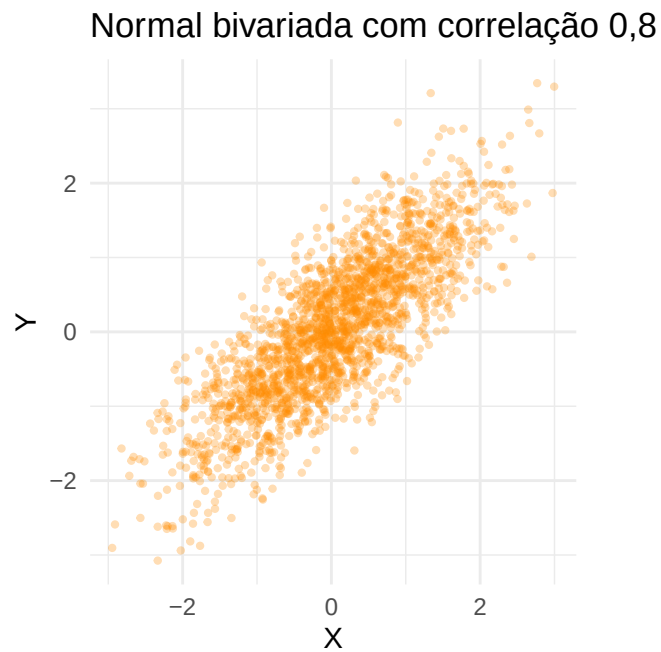
```
prob_teorica <- 1 / 4 + asin(rho) / (2 * pi)
cat("P(X > 0 e Y > 0) observada:", round(mean(X > 0 & Y > 0), 4), "\n")
```

P(X > 0 e Y > 0) observada: 0.4105

```
cat("P(X > 0 e Y > 0) teórica:", round(prob_teorica, 4), "\n")
```

P(X > 0 e Y > 0) teórica: 0.3976

```
ggplot(data.frame(x = X, y = Y), aes(x = x, y = y)) +  
  geom_point(alpha = 0.3, size = 0.8, color = "darkorange") +  
  coord_fixed() +  
  labs(title = "Normal bivariada com correlação 0,8",  
        x = "X", y = "Y") +  
  theme_minimal()
```



86 Python

```
np.random.seed(7)

rho = 0.8 # correlação desejada
B = 2000

Z1, Z2 = box_muller(B)

# X é a primeira normal; Y mistura as duas para criar a correlação
X = Z1
Y = rho * Z1 + np.sqrt(1 - rho**2) * Z2

print("Correlação amostral:", round(np.corrcoef(X, Y)[0, 1], 3),
      " (teórica:", rho, ")")
```

Correlação amostral: 0.798 (teórica: 0.8)

```
print("Desvio padrão de Y:", round(Y.std(ddof=1), 3), " (teórico: 1)")
```

Desvio padrão de Y: 0.997 (teórico: 1)

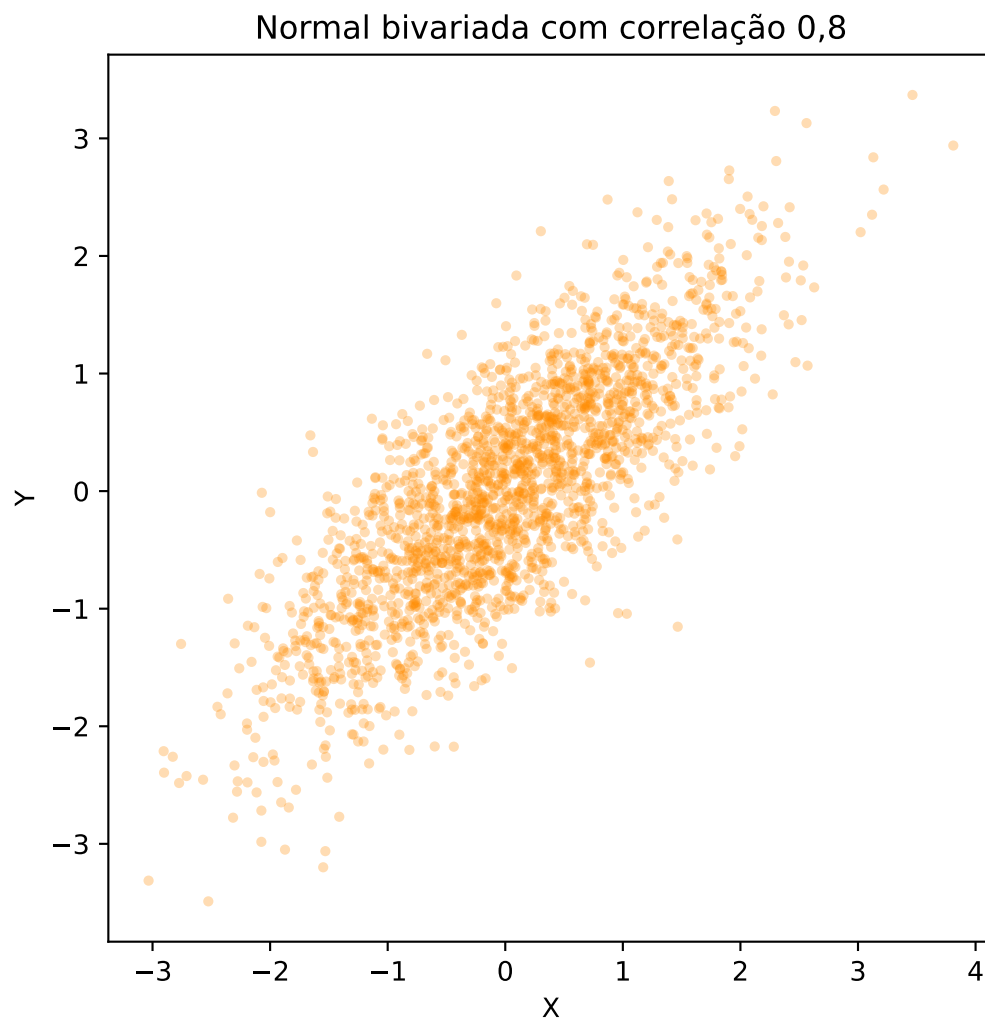
```
prob_teorica = 1 / 4 + np.arcsin(rho) / (2 * np.pi)
print("P(X > 0 e Y > 0) observada:", round(np.mean((X > 0) & (Y > 0)), 4))
```

P(X > 0 e Y > 0) observada: 0.402

```
print("P(X > 0 e Y > 0) teórica:", round(prob_teorica, 4))
```

P(X > 0 e Y > 0) teórica: 0.3976

```
plt.figure(figsize=(6, 6))
plt.scatter(X, Y, alpha=0.3, s=8, color="darkorange")
plt.gca().set_aspect("equal")
plt.title("Normal bivariada com correlação 0,8")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



Compare este gráfico com o do Exemplo 1: a nuvem deixou de ser circular e virou uma elipse inclinada. A simetria que motivou o capítulo inteiro é uma propriedade de normais

independentes e de mesma variância; quando elas se correlacionam, as coordenadas polares deixam de ajudar.

86.1 O método polar de Marsaglia

O Box-Muller tem um custo escondido: cada par exige um logaritmo, uma raiz, um seno e um cosseno. Funções trigonométricas são caras — historicamente, bem mais caras que uma rejeição ocasional. Marsaglia propôs uma variante que produz exatamente a mesma distribuição sem calcular nenhum seno ou cosseno, ao preço de descartar alguns candidatos. É um híbrido bonito dos dois métodos: rejeição para sortear a direção, transformação para gerar o raio.

A ideia é que não precisamos do **ângulo**; precisamos apenas de seu seno e seu cosseno. E um ângulo uniforme com seu seno e cosseno é o que se obtém sorteando um ponto uniformemente dentro do círculo de raio 1: se (V_1, V_2) é esse ponto e $S = V_1^2 + V_2^2$, então

$$\cos \Theta = \frac{V_1}{\sqrt{S}} \quad \text{e} \quad \sin \Theta = \frac{V_2}{\sqrt{S}},$$

sem nunca calcular Θ . Sortear um ponto uniforme no círculo, por sua vez, é um caso do método da rejeição do Capítulo 6: sorteia-se um ponto no quadrado $[-1, 1] \times [-1, 1]$ e descarta-se quem cair fora do círculo.

O que fecha o método é um fato menos óbvio: condicionado à aceitação, S é $\text{Unif}(0, 1)$ e é **independente** do ângulo. Logo, o próprio S pode fazer o papel do U_1 do Box-Muller, e nenhum uniforme extra é necessário para o raio.

Pseudo-algoritmo: método polar

1. Gere $V_1, V_2 \sim \text{Unif}(-1, 1)$ independentes, e faça $S = V_1^2 + V_2^2$.
2. Se $S \geq 1$ ou $S = 0$, volte ao passo 1 (o ponto caiu fora do círculo).
3. Devolva o par

$$X = V_1 \sqrt{\frac{-2 \log S}{S}}, \quad Y = V_2 \sqrt{\frac{-2 \log S}{S}}.$$

O fator $\sqrt{-2 \log S}$ é o raio R , exatamente como no Box-Muller; a divisão extra por $\sqrt{S}$ é o que normaliza (V_1, V_2) para virar $(\cos \Theta, \sin \Theta)$.

Como a área do círculo de raio 1 é π e a do quadrado é 4, a taxa de aceitação é $\pi/4 \approx 0,785$: descartamos cerca de um em cada cinco pares.

87 R

```
set.seed(42)

B <- 2500
X <- numeric(B)
Y <- numeric(B)
n_sorteios <- 0 # conta quantos pares (V1, V2) foram sorteados ao todo
i <- 1

while (i <= B) {
  # Passo 1: um ponto uniforme no quadrado [-1, 1] x [-1, 1]
  V1 <- runif(1, min = -1, max = 1)
  V2 <- runif(1, min = -1, max = 1)
  n_sorteios <- n_sorteios + 1

  S <- V1^2 + V2^2

  # Passo 2: só aceitamos os pontos que caíram dentro do círculo de raio 1
  if (S < 1 && S > 0) {
    # Passo 3: o fator comum faz o papel de R / sqrt(S)
    fator <- sqrt(-2 * log(S) / S)
    X[i] <- V1 * fator
    Y[i] <- V2 * fator
    i <- i + 1
  }
}

cat("Pares sorteados:", n_sorteios, "para", B, "aceitos\n")
```

Pares sorteados: 3212 para 2500 aceitos

```
cat("Taxa de aceitação observada:", round(B / n_sorteios, 3), "\n")
```

Taxa de aceitação observada: 0.778

```
cat("Taxa de aceitação teórica (pi/4):", round(pi / 4, 3), "\n")
```

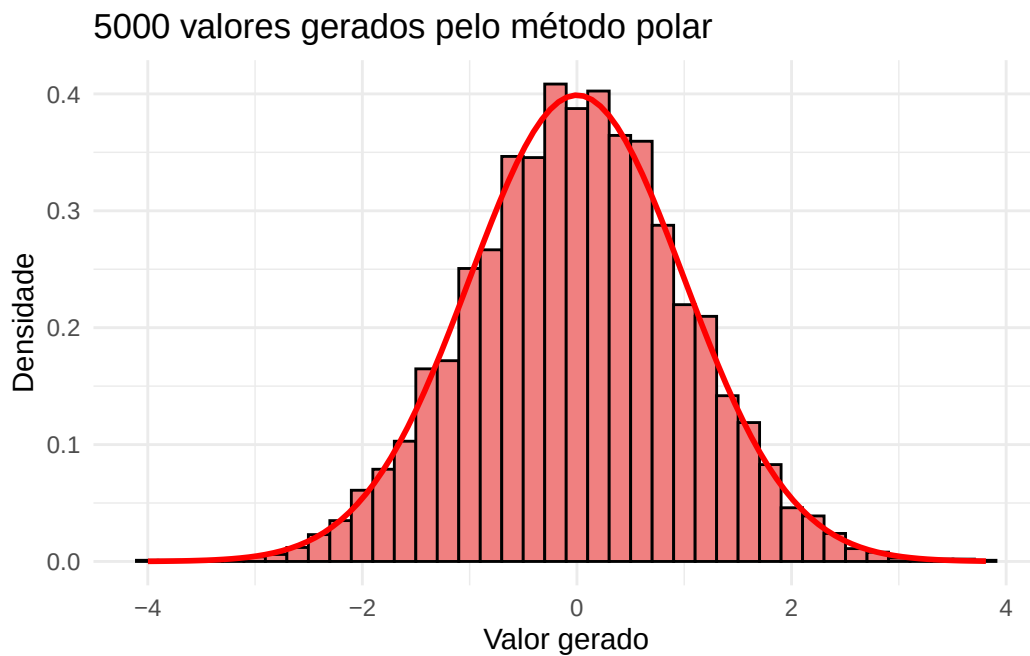
Taxa de aceitação teórica (pi/4): 0.785

```
cat("Média e desvio padrão de X:", round(mean(X), 3), round(sd(X), 3), "\n")
```

Média e desvio padrão de X: 0.003 0.99

```
df <- data.frame(z = c(X, Y))

ggplot(df, aes(x = z)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40,
                 fill = "lightcoral", color = "black") +
  stat_function(fun = dnorm, color = "red", linewidth = 1) +
  labs(title = "5000 valores gerados pelo método polar",
       x = "Valor gerado", y = "Densidade") +
  theme_minimal()
```



88 Python

```
np.random.seed(42)

B = 2500
X = np.zeros(B)
Y = np.zeros(B)
n_sorteios = 0 # conta quantos pares (V1, V2) foram sorteados ao todo
i = 0

while i < B:
    # Passo 1: um ponto uniforme no quadrado [-1, 1] x [-1, 1]
    V1 = np.random.uniform(-1, 1)
    V2 = np.random.uniform(-1, 1)
    n_sorteios += 1

    S = V1**2 + V2**2

    # Passo 2: só aceitamos os pontos que caíram dentro do círculo de raio 1
    if S < 1 and S > 0:
        # Passo 3: o fator comum faz o papel de R / sqrt(S)
        fator = np.sqrt(-2 * np.log(S) / S)
        X[i] = V1 * fator
        Y[i] = V2 * fator
        i += 1

print("Pares sorteados:", n_sorteios, "para", B, "aceitos")
```

Pares sorteados: 3168 para 2500 aceitos

```
print("Taxa de aceitação observada:", round(B / n_sorteios, 3))
```

Taxa de aceitação observada: 0.789

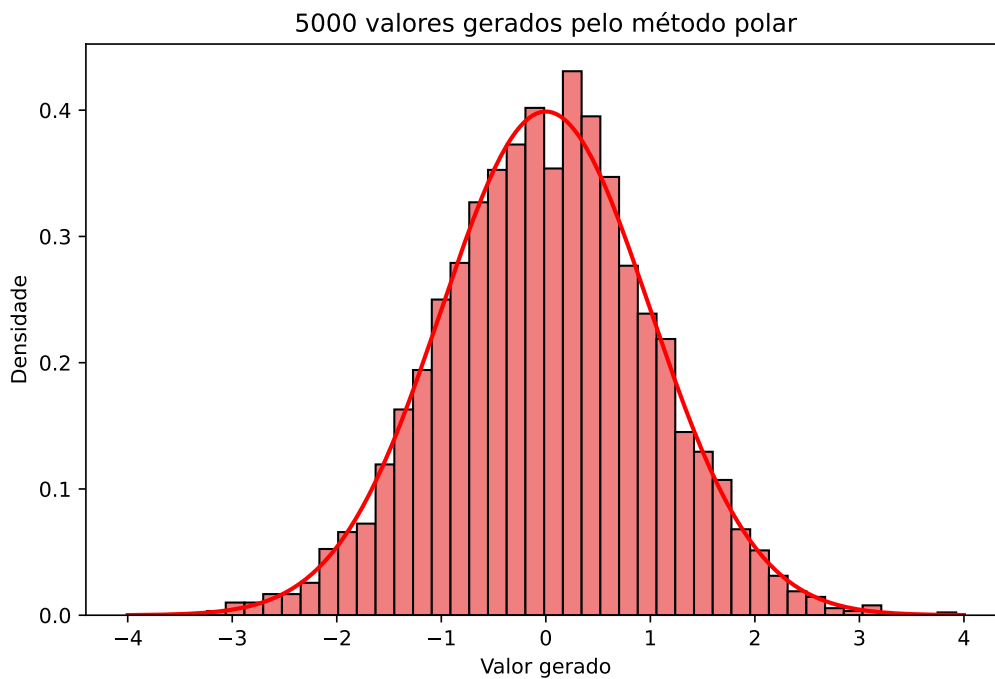
```
print("Taxa de aceitação teórica (pi/4):", round(np.pi / 4, 3))
```

Taxa de aceitação teórica (pi/4): 0.785

```
print("Média e desvio padrão de X:", round(X.mean(), 3),  
      round(X.std(ddof=1), 3))
```

Média e desvio padrão de X: 0.013 1.01

```
Z = np.concatenate([X, Y])  
grade = np.linspace(-4, 4, 400)  
  
plt.figure(figsize=(8, 5))  
plt.hist(Z, bins=40, density=True, color="lightcoral", edgecolor="black")  
plt.plot(grade, norm.pdf(grade), color="red", linewidth=2)  
plt.title("5000 valores gerados pelo método polar")  
plt.xlabel("Valor gerado")  
plt.ylabel("Densidade")  
plt.show()
```



i E o que R e Python usam de verdade?

Nenhum dos dois usa o Box-Muller por padrão. O R gera normais por **inversão** da f.d.a., com uma aproximação numérica muito precisa de Φ^{-1} : rode `RNGkind()` e veja "Inversion" na segunda posição. (Dá para pedir o Box-Muller com `RNGkind(normal.kind = "Box-Muller")`.)

Já o gerador legado do NumPy — o `np.random.normal` que usamos no livro inteiro — implementa exatamente o **método polar** desta seção. A API mais nova (`np.random.default_rng`) usa um terceiro algoritmo, o *ziggurat*, mais rápido e bem mais complicado de explicar.

88.1 Exercícios

Exercício 1. Implemente o método de Box-Muller em uma função que receba B e devolva B pares de valores independentes com distribuição $N(0, 1)$.

- (a) Gere $B = 5000$ pares e faça o histograma dos 10 000 valores obtidos, sobrepondo a densidade da $N(0, 1)$.
- (b) Faça um gráfico quantil-quantil (`qqnorm` seguido de `qqline` em R, `scipy.stats.probplot` em Python) e comente o que ele mostra nas caudas.
- (c) Verifique a independência do par de três maneiras: calcule a correlação amostral, faça o gráfico de dispersão e compare a proporção de pontos em cada um dos quatro quadrantes com $1/4$.
- (d) Estime $\mathbb{P}(|X| > 1,96)$ e compare com o valor teórico 0,05.

Exercício 2. Este exercício verifica, na mão, os fatos sobre R usados no capítulo. Lembre que $f_R(r) = re^{-r^2/2}$ para $r > 0$.

- (a) Mostre que a f.d.a. de R é $F_R(r) = 1 - e^{-r^2/2}$ e que, portanto, $R^2 \sim \text{Exp}(1/2)$.
- (b) Mostre, invertendo F_R , que $R = \sqrt{-2 \log(1 - U)}$ com $U \sim \text{Unif}(0, 1)$. Conclua que o passo 2 do Box-Muller é o método da inversão aplicado à Rayleigh.
- (c) Verifique que $\mathbb{E}[R^2] = 2$, coerente com $R^2 \sim \chi_2^2$, e que $\mathbb{E}[R] = \sqrt{\pi/2}$.
- (d) Gere $B = 5000$ valores de R (você pode usar o Box-Muller e calcular $\sqrt{X^2 + Y^2}$, ou gerar R diretamente pelo item (b)) e compare o histograma com f_R . Compare também as médias amostrais de R e R^2 com os valores do item (c).

Exercício 3. A partir das normais padrão do Exercício 1:

- (a) Gere $B = 5000$ valores de uma $N(-2, 9)$ e compare o histograma com a densidade teórica. Cuidado: 9 é a variância.
- (b) A distribuição **log-normal** é a de $W = e^X$, com $X \sim N(\mu, \sigma^2)$; ela é muito usada para modelar salários e preços, que são positivos e assimétricos. Gere $B = 5000$ valores de W com $\mu = 0$ e $\sigma = 1$ e compare o histograma com a densidade da log-normal (`dlnorm` em R, `scipy.stats.lognorm.pdf(x, s=sigma)` em Python).
- (c) Estime $\mathbb{E}[W]$ e compare com e^μ e com $e^{\mu+\sigma^2/2}$. Qual dos dois é o valor correto? Explique por que $\mathbb{E}[e^X] \neq e^{\mathbb{E}[X]}$.
- (d) Compare a média e a mediana amostrais de W . Por que elas são tão diferentes, se para X elas coincidem?

Exercício 4. Continuando o Exemplo 3.

- (a) Verifique, por conta própria, que $\text{Var}(Y) = 1$ e $\text{Cov}(X, Y) = \rho$ para $X = Z_1$ e $Y = \rho Z_1 + \sqrt{1 - \rho^2} Z_2$.
- (b) Gere $B = 2000$ pares para $\rho = -0,9$, $\rho = 0$ e $\rho = 0,9$, e faça os três gráficos de dispersão lado a lado. Descreva como a nuvem muda.
- (c) Para cada um dos três valores de ρ , estime $\mathbb{P}(X > 0, Y > 0)$ e compare com $1/4 + \arcsin(\rho)/(2\pi)$.
- (d) Adapte a construção para gerar um par com médias μ_1, μ_2 e desvios padrão σ_1, σ_2 quaisquer. Gere $B = 2000$ pares “peso e altura” com $\mu = (70, 170)$, $\sigma = (12, 8)$ e $\rho = 0,6$, e confira as médias, os desvios e a correlação amostrais.
- (e) O Exemplo 5 do Capítulo 7 gera vetores normais em dimensão qualquer, transformando um vetor Z de normais padrão em LZ , com $\Sigma = LL^\top$. Verifique que, em dimensão 2, as duas construções são exatamente a mesma: calcule a decomposição de Cholesky de

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

(com `t(chol(Sigma))` em R e `np.linalg.cholesky` em Python) e confira que a matriz obtida é

$$L = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix},$$

de modo que LZ reproduz linha por linha as fórmulas de X e Y usadas neste exemplo.

Exercício 5. A mesma mudança de variáveis do capítulo resolve uma integral famosa. Seja $I = \int_{-\infty}^{\infty} e^{-x^2/2} dx$.

- (a) Escreva I^2 como a integral dupla $\int \int e^{-(x^2+y^2)/2} dx dy$ e passe para coordenadas polares, lembrando que o elemento de área vira $r dr d\theta$ — o mesmo Jacobiano da seção sobre a distribuição de R e Θ .

- (b) Conclua que $I = \sqrt{2\pi}$, isto é, que a constante que aparece na densidade da normal é exatamente a que a faz integrar 1.
- (c) Confira o resultado numericamente (`integrate` em R, `scipy.integrate.quad` em Python).
- (d) Em uma frase: qual é a relação entre esse cálculo e o método de Box-Muller?

Exercício 6. Sobre o método polar.

- (a) Implemente-o em uma função e verifique, com $B = 5000$ pares, que os valores gerados são $N(0, 1)$ e que a correlação entre as duas coordenadas é próxima de zero.
- (b) Registre a proporção de pares aceitos e compare com $\pi/4$. Use essa proporção para estimar π : por que $4 \times$ (taxa de aceitação) é uma estimativa de π ?
- (c) Guarde os valores de S dos pares **aceitos** e faça o histograma. Ele é compatível com uma $\text{Unif}(0, 1)$? Explique geometricamente por que $\mathbb{P}(S \leq s \mid S < 1) = s$ para $0 < s < 1$ (dica: compare áreas de círculos).
- (d) Quantos uniformes o método consome, em média, por normal gerada? Compare com o 1 do Box-Muller e com o 3,6 do método da rejeição do Capítulo 6.

Exercício 7. Agora temos três geradores de normais: a rejeição do Capítulo 6, o Box-Muller e o método polar.

- (a) Meça o tempo que cada um leva para gerar 10^5 valores (`system.time` em R, `time.time` em Python). Rode cada medição algumas vezes: os tempos variam.
- (b) Compare com o tempo das funções prontas `rnorm` e `np.random.normal`. A diferença é grande? O que ela sugere sobre a implementação dessas funções?
- (c) Em R, rode `RNGkind()` e descubra qual método o `rnorm` usa por padrão. Em seguida troque para `RNGkind(normal.kind = "Box-Muller")`, gere uma amostra e confira que continua sendo normal. Volte ao padrão com `RNGkind(normal.kind = "Inversion")` ao final.
- (d) O método polar rejeita cerca de 21% dos pares, mas costuma ser mais rápido que o Box-Muller. Como isso é possível?

Exercício 8. Um erro tentador é economizar um uniforme, usando o **mesmo** U nas duas fórmulas:

$$X = \sqrt{-2 \log U} \cos(2\pi U), \quad Y = \sqrt{-2 \log U} \sin(2\pi U).$$

- (a) Gere $B = 2000$ pares assim e faça o gráfico de dispersão de (X, Y) . O que aconteceu com a nuvem circular?

- (b) Calcule a correlação amostral entre X e Y e o desvio padrão de cada uma. Faça o histograma de X sozinho e compare com a densidade da $N(0, 1)$: o defeito aparece de forma tão evidente quanto no gráfico de dispersão?
- (c) Qual hipótese da proposição de validade do método foi violada? Responda em termos de R e Θ .
- (d) Conclua que olhar apenas para o histograma de X é insuficiente para atestar que um gerador de **pares** está correto. Que gráfico você faria primeiro, ao desconfiar de um gerador desse tipo?

Exercício 9. Um dardo é atirado em um alvo. O erro horizontal X e o erro vertical Y (em centímetros, em relação ao centro) são independentes e $N(0, \sigma^2)$, com $\sigma = 3$. A distância do dardo ao centro é $D = \sqrt{X^2 + Y^2}$.

- (a) Use o Box-Muller para gerar $B = 5000$ pares (X, Y) e obtenha os valores correspondentes de D .
- (b) Pela teoria do capítulo, $D = \sigma R$ com $R \sim \text{Rayleigh}(1)$. Mostre que $\mathbb{P}(D \leq d) = 1 - e^{-d^2/(2\sigma^2)}$ e compare essa expressão com a f.d.a. empírica dos valores simulados.
- (c) Estime a probabilidade de o dardo cair a menos de 2 cm do centro e a probabilidade de cair a mais de 6 cm, comparando com os valores exatos do item (b).
- (d) Qual é a distância **mediana** ao centro? Compare o valor simulado com o teórico $\sigma\sqrt{2\log 2}$.
- (e) Note que $\mathbb{P}(D \leq d)$ cresce a partir de 0 e que a densidade de D vale zero em $d = 0$: acertar exatamente o centro é o resultado *menos* provável, mesmo sendo o centro o ponto de maior densidade de (X, Y) . Explique essa aparente contradição.

Exercício 10. (Desafio) Todo gerador uniforme produz, na verdade, um número finito de valores distintos. Seja $u_{\min}$ o menor valor estritamente positivo que o seu gerador pode devolver.

- (a) Mostre que o Box-Muller nunca produz um valor com $|X| > \sqrt{-2\log u_{\min}}$, qualquer que seja U_2 .
- (b) Suponha $u_{\min} = 2^{-32}$ (resolução típica de um gerador de 32 bits) e depois $u_{\min} = 2^{-53}$ (precisão de um **double**). Calcule o limite em cada caso. A resposta está entre 6 e 9 desvios padrão.
- (c) Calcule $\mathbb{P}(|X| > 6,66)$ para $X \sim N(0, 1)$. Para quais aplicações essa truncagem da cauda seria irrelevante, e para quais ela poderia importar?
- (d) Descubra experimentalmente se o **runif** do R (ou o **np.random.uniform** do Python) pode devolver exatamente 0: gere alguns milhões de valores e olhe o mínimo. O que aconteceria com o algoritmo se ele devolvesse?

- (e) O método polar sofre do mesmo problema? Onde entra o $u_{\min}$ nele?

Exercício 11. (Desafio) Este exercício mostra que um gerador uniforme “razoável” pode virar um gerador normal péssimo — um fenômeno descoberto por Neave em 1973.

Use a função `gerar_lcg(n, a, c, m, semente)` do Capítulo 2 para produzir os uniformes, dividindo os valores por m . Tome os pares $(U_1, U_2), (U_3, U_4), \dots$ de valores **consecutivos** do gerador e aplique o Box-Muller a cada par.

- (a) Comece com $m = 2^{10}$, $a = 5$, $c = 3$ e semente 5, gerando 10 000 uniformes (ou seja, 5000 pares). Atenção: esse gerador **pode** devolver o valor 0, e $\log 0 = -\infty$ — é exatamente a armadilha do callout de atenção deste capítulo. Decida o que fazer com esses casos e justifique.
- (b) Faça o gráfico de dispersão dos 5000 pares (X, Y) , com escalas iguais nos dois eixos. Compare com o gráfico do Exemplo 1 e descreva o que vê.
- (c) Faça o histograma de X sozinho, sobrepondo a densidade da $N(0, 1)$. Ele denuncia o problema com a mesma clareza? Compare também o maior $|Y|$ obtido com o maior $|Z|$ de uma amostra de 5000 valores de `rnorm` (ou `np.random.normal`): o gerador ruim consegue produzir valores na cauda?
- (d) Explique o que aconteceu. Que propriedade da **sequência** de uniformes o Box-Muller exige, e que o histograma de um único U não enxerga? (Duas dicas: os pares consecutivos de um LCG caem sobre poucas retas do quadrado $[0, 1] \times [0, 1]$; e o período desse gerador é $m = 1024$.)
- (e) Repita o experimento com o famoso gerador RANDU ($a = 65539$, $c = 0$, $m = 2^{31}$, semente **ímpar**). Desta vez o gráfico de dispersão parece bem comportado. Isso prova que o RANDU é um bom gerador? (O defeito dele aparece em **ternos**: vale $x_{n+2} = 6x_{n+1} - 9x_n \bmod 2^{31}$, de modo que os pontos (U_n, U_{n+1}, U_{n+2}) se acumulam em poucos planos do cubo. Um teste que só usa pares nunca veria isso.)

89 Método de Monte Carlo

Os capítulos anteriores foram todos sobre a mesma pergunta: **como** gerar valores de uma distribuição. Inversão, rejeição, transformações, Box-Muller — cada método resolvia um caso. Este capítulo muda o foco e pergunta **para quê**: uma vez que sabemos gerar $X_1, X_2, \dots$, o que fazemos com esses valores?

A resposta é o **Método de Monte Carlo** (MMC): usar valores simulados para calcular, de forma aproximada, esperanças, probabilidades e integrais que não sabemos (ou não queremos) resolver no papel. A ideia é simples a ponto de parecer boa demais — trocar uma conta difícil por uma média de números sorteados — mas ela é a base de boa parte da estatística computacional moderna.

Seja X uma v.a. discreta com função de probabilidade $p(x)$, ou uma v.a. contínua com densidade $f(x)$. Queremos calcular

$$\theta = \mathbb{E}[g(X)] = \begin{cases} \int_{-\infty}^{\infty} g(x)f(x) dx & \text{se } X \text{ é contínua,} \\ \sum_x g(x)p(x) & \text{se } X \text{ é discreta.} \end{cases}$$

89.1 A ideia do método

Suponha que saibamos gerar valores com a distribuição de X . Geramos $X_1, \dots, X_B$ independentes, aplicamos g em cada um deles e tiramos a média.

i Definição: estimador de Monte Carlo

Sejam $X_1, \dots, X_B$ v.a.'s i.i.d. com a mesma distribuição de X . O **estimador de Monte Carlo** de $\theta = \mathbb{E}[g(X)]$ é

$$\hat{\theta}_B = \frac{1}{B} \sum_{i=1}^B g(X_i).$$

 Pseudo-algoritmo: Monte Carlo

1. Gere $X_1, \dots, X_B$ valores independentes da v.a. X .
2. Calcule

$$\hat{\theta}_B = \frac{1}{B} \sum_{i=1}^B g(X_i).$$

Note que o algoritmo tem dois ingredientes, e apenas um deles é novo: gerar os X_i é exatamente o assunto dos capítulos anteriores; calcular a média é o passo trivial. Toda a dificuldade prática de Monte Carlo está em saber **quem é X** e **quem é g** — isto é, em reescrever a quantidade de interesse como uma esperança.

89.1.1 Por que funciona?

 Proposição: propriedades do estimador de Monte Carlo

Suponha que $\mathbb{E}[|g(X)|] < \infty$ e seja $\sigma^2 = \text{Var}(g(X))$. Então:

- (i) $\mathbb{E}[\hat{\theta}_B] = \theta$, ou seja, $\hat{\theta}_B$ é não viesado;
- (ii) se $\sigma^2 < \infty$, $\text{Var}(\hat{\theta}_B) = \frac{\sigma^2}{B}$;
- (iii) $\hat{\theta}_B \rightarrow \theta$ quase certamente, quando $B \rightarrow \infty$.

 Demonstração

Como os X_i têm a mesma distribuição de X , cada $g(X_i)$ tem esperança θ . Pela linearidade da esperança,

$$\mathbb{E}[\hat{\theta}_B] = \frac{1}{B} \sum_{i=1}^B \mathbb{E}[g(X_i)] = \frac{1}{B} \cdot B \theta = \theta,$$

o que prova (i). Para (ii), usamos que os X_i são independentes, de modo que a variância da soma é a soma das variâncias:

$$\text{Var}(\hat{\theta}_B) = \frac{1}{B^2} \sum_{i=1}^B \text{Var}(g(X_i)) = \frac{1}{B^2} \cdot B \sigma^2 = \frac{\sigma^2}{B}.$$

Finalmente, (iii) é exatamente a Lei Forte dos Grandes Números aplicada às v.a.'s i.i.d. $Y_i = g(X_i)$, que têm esperança θ finita. $\square$

O item (iii) é a garantia de que o método faz sentido: aumentando B , chegamos tão perto de θ quanto quisermos. Já o item (ii) é o que diz **quão rápido** chegamos — voltaremos a ele na seção *Quão preciso é o método?*.

i Por que B , e não n ?

Neste capítulo o número de valores simulados é chamado de B , e não de n . A razão é que, em estatística, n costuma denotar o tamanho de uma amostra de **dados reais** — que é o que ele é, por exemplo, no Exercício 6. Já B é o número de repetições que **nós** decidimos fazer no computador: podemos aumentá-lo à vontade, ao custo de tempo de máquina.

89.2 Exemplo 1: Estimativa de uma Integral

Queremos obter uma estimativa para

$$\theta = \int_0^1 e^{-x} dx.$$

Para isso, basta observar que se $U \sim \text{Unif}(0, 1)$, então sua densidade é $f(u) = 1$ no intervalo $(0, 1)$, e portanto

$$\theta = \int_0^1 e^{-x} dx = \int_0^1 e^{-u} f(u) du = \mathbb{E}[e^{-U}].$$

Ou seja, estamos no caso $X = U \sim \text{Unif}(0, 1)$ e $g(x) = e^{-x}$. O estimador de Monte Carlo desta integral é então a média de e^{-U_i} :

90 R

```
set.seed(58)
B <- 100
u <- runif(B, min = 0, max = 1)

# Estimativa de theta usando Monte Carlo
theta_hat <- mean(exp(-u))
cat("Estimativa de theta:", theta_hat, "\n")
```

Estimativa de theta: 0.6672518

```
# Valor real da integral
valor_real <- 1 - exp(-1)
cat("Valor real:", valor_real, "\n")
```

Valor real: 0.6321206

91 Python

```
import numpy as np

np.random.seed(58)

# Número de simulações
B = 100

# Geração de valores uniformes
u = np.random.uniform(0, 1, B)

# Estimativa de theta usando Monte Carlo
theta_hat = np.mean(np.exp(-u))
print("Estimativa de theta:", theta_hat)
```

Estimativa de theta: 0.6445501026553044

```
# Valor real da integral
valor_real = 1 - np.exp(-1)
print("Valor real:", valor_real)
```

Valor real: 0.6321205588285577

Vamos verificar como o valor de B influencia na aproximação. Para isso, calculamos $\hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_{2000}$, isto é, a estimativa obtida com o primeiro valor gerado, com os dois primeiros, e assim por diante:

92 R

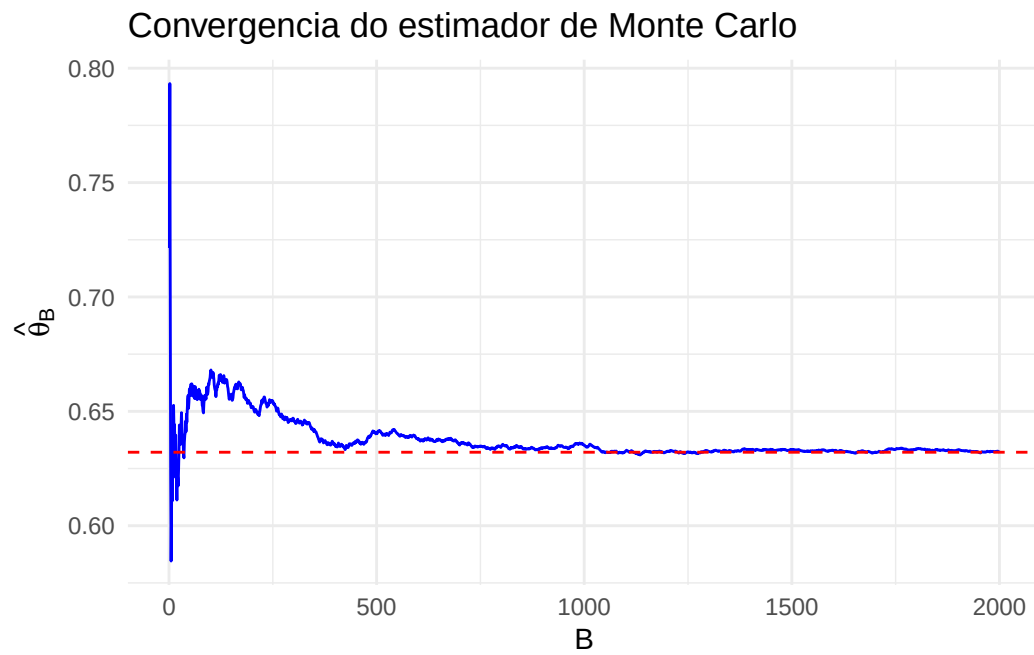
```
library(ggplot2)

set.seed(58)
B <- 2000
u <- runif(B)

# cumsum(x)[i] é a soma dos i primeiros elementos de x. Dividindo pelo índice,
# obtemos a média dos i primeiros valores, ou seja, a estimativa com i simulações
theta_hat_parcial <- cumsum(exp(-u)) / (1:B)

dados <- data.frame(b = 1:B, theta_hat = theta_hat_parcial)

ggplot(dados, aes(x = b, y = theta_hat)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = 1 - exp(-1), color = "red", linetype = "dashed") +
  labs(x = "B", y = expression(hat(theta)[B]),
       title = "Convergencia do estimador de Monte Carlo") +
  theme_minimal()
```



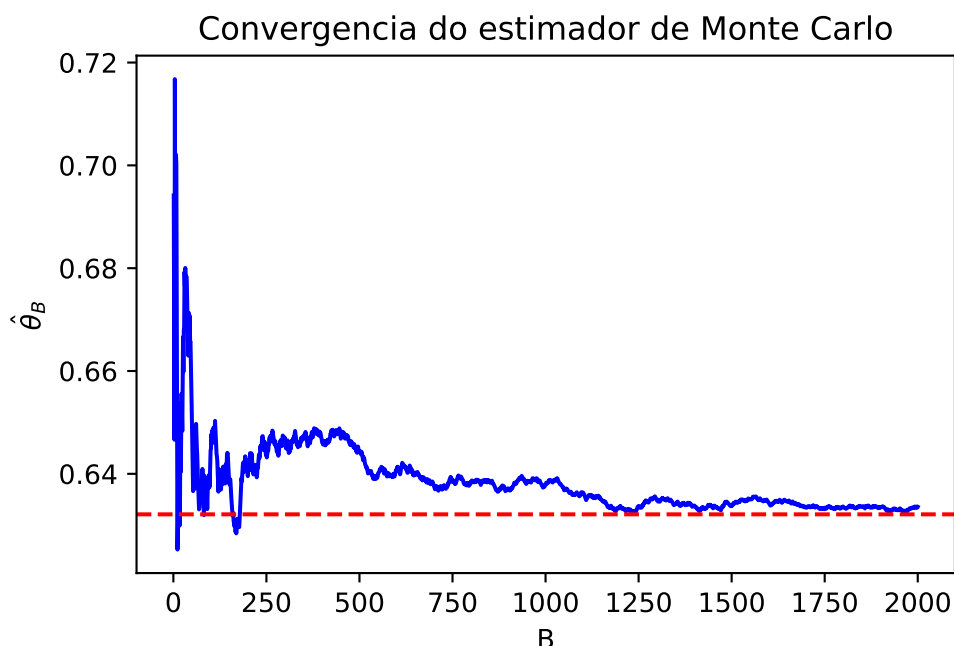
93 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(58)
B = 2000
u = np.random.uniform(0, 1, B)

# np.cumsum(x)[i] é a soma dos i+1 primeiros elementos de x. Dividindo pelo
# índice, obtemos a estimativa baseada nas i+1 primeiras simulações
theta_hat_parcial = np.cumsum(np.exp(-u)) / np.arange(1, B + 1)

plt.plot(np.arange(1, B + 1), theta_hat_parcial, color='blue')
plt.axhline(y=1 - np.exp(-1), color='red', linestyle='--')
plt.xlabel('B')
plt.ylabel(r'$\hat{\theta}_B$')
plt.title('Convergencia do estimador de Monte Carlo')
plt.show()
```



O gráfico mostra a evolução da estimativa à medida que o número de valores gerados B aumenta, comparada com o valor real da integral. Repare em dois aspectos: a estimativa se aproxima do valor verdadeiro, mas de forma **errática** (ela não melhora a cada passo — apenas na média), e a oscilação vai diminuindo devagar. Quantificar esse “devagar” é o assunto da seção *Quão preciso é o método?*.

93.1 Escolhendo a densidade: integrais em outros domínios

No Exemplo 1 tivemos sorte: a integral era em $(0, 1)$ e a densidade da uniforme vale exatamente 1 ali, então a integral já era uma esperança. O caso geral pede um passo a mais. Suponha que queremos calcular

$$\theta = \int_A g(x) dx$$

para uma região A qualquer. O truque é multiplicar e dividir o integrando por uma densidade f escolhida por nós:

$$\theta = \int_A g(x) dx = \int_A \frac{g(x)}{f(x)} f(x) dx = \mathbb{E} \left[\frac{g(X)}{f(X)} \right], \quad X \sim f,$$

o que é válido desde que $f(x) > 0$ em todo ponto de A em que $g(x) \neq 0$. Temos então liberdade total: qualquer densidade que saibamos simular e cujo suporte cubra A serve.

A escolha mais simples, quando $A = [a, b]$ é limitado, é a uniforme nesse intervalo: $f(x) = 1/(b-a)$, de modo que $g(x)/f(x) = (b-a)g(x)$ e

$$\int_a^b g(x) dx = (b-a) \mathbb{E}[g(U)], \quad U \sim \text{Unif}(a, b).$$

💡 Pseudo-algoritmo: integral em um intervalo $[a, b]$

1. Gere $U_1, \dots, U_B \sim \text{Unif}(a, b)$ independentes.
2. Calcule

$$\hat{\theta}_B = (b-a) \frac{1}{B} \sum_{i=1}^B g(U_i).$$

Note que o Exemplo 1 é o caso particular $a = 0, b = 1$, em que o fator $(b-a)$ vale 1 e passa despercebido.

93.2 Exemplo 2: Uma integral em um intervalo qualquer

Queremos estimar

$$\theta = \int_0^\pi x \sin(x) dx,$$

cujo valor exato é π (integrando por partes). Aqui $a = 0, b = \pi$ e $g(x) = x \sin(x)$, de modo que o estimador é

$$\hat{\theta}_B = \pi \cdot \frac{1}{B} \sum_{i=1}^B U_i \sin(U_i), \quad U_i \stackrel{iid}{\sim} \text{Unif}(0, \pi).$$

94 R

```
set.seed(58)
B <- 10000
a <- 0
b <- pi

u <- runif(B, min = a, max = b)

# Não esqueça do fator (b - a): a densidade da Unif(a,b) vale 1/(b-a), e não 1
theta_hat <- (b - a) * mean(u * sin(u))
cat("Estimativa de theta:", theta_hat, "\n")
```

Estimativa de theta: 3.12135

```
cat("Valor real:", pi, "\n")
```

Valor real: 3.141593

95 Python

```
import numpy as np

np.random.seed(58)
B = 10000
a = 0
b = np.pi

u = np.random.uniform(a, b, B)

# Não esqueça do fator (b - a): a densidade da Unif(a,b) vale 1/(b-a), e não 1
theta_hat = (b - a) * np.mean(u * np.sin(u))
print("Estimativa de theta:", theta_hat)
```

Estimativa de theta: 3.141924435234792

```
print("Valor real:", np.pi)
```

Valor real: 3.141592653589793

 Atenção: o fator $(b - a)$

Esquecer de multiplicar por $(b - a)$ é o erro mais comum ao aplicar Monte Carlo a uma integral. Uma forma de se proteger dele é testar o código com $g \equiv 1$: a estimativa deve dar $b - a$, que é o comprimento do intervalo, e não 1.

95.1 Exemplo 3: Uma integral em um domínio ilimitado

Quando o domínio é ilimitado, a uniforme não serve — não existe distribuição uniforme em $(0, \infty)$. Precisamos então de uma densidade f com o suporte certo. Queremos estimar

$$\theta = \int_0^\infty e^{-x^2} dx,$$

cujo valor exato é $\theta = \sqrt{\pi}/2$. Uma densidade com suporte em $(0, \infty)$ que já sabemos simular é a da $\text{Exp}(1)$, dada por $f(x) = e^{-x}$ para $x \geq 0$. Aplicando a identidade da seção anterior com essa escolha:

$$\theta = \int_0^\infty e^{-x^2} dx = \int_0^\infty \frac{e^{-x^2}}{e^{-x}} e^{-x} dx = \int_0^\infty e^{-(x^2-x)} e^{-x} dx = \mathbb{E} \left[e^{-(X^2-X)} \right], \quad X \sim \text{Exp}(1).$$

Logo, o estimador de Monte Carlo é

$$\hat{\theta}_B = \frac{1}{B} \sum_{i=1}^B e^{-(X_i^2-X_i)}, \quad X_i \stackrel{iid}{\sim} \text{Exp}(1).$$

96 R

```
set.seed(58)
B <- 1000
x <- rexp(B, rate = 1)

theta_hat <- mean(exp(-(x^2 - x)))
cat("Estimativa de theta:", theta_hat, "\n")
```

Estimativa de theta: 0.891637

```
valor_real <- sqrt(pi) / 2
cat("Valor real:", valor_real, "\n")
```

Valor real: 0.8862269

97 Python

```
import numpy as np

np.random.seed(58)
B = 1000
x = np.random.exponential(scale=1.0, size=B)

theta_hat = np.mean(np.exp(-(x**2 - x)))
print("Estimativa de theta:", theta_hat)
```

Estimativa de theta: 0.9110120774968984

```
valor_real = np.sqrt(np.pi) / 2
print("Valor real:", valor_real)
```

Valor real: 0.8862269254527579

Novamente, podemos acompanhar a convergência do estimador conforme B cresce:

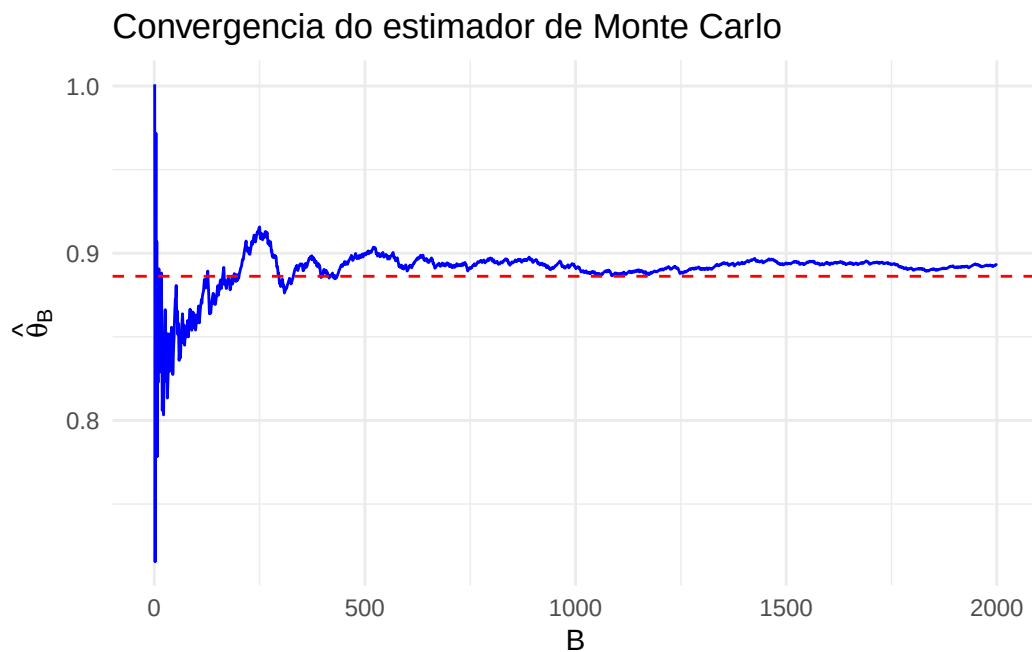
98 R

```
set.seed(58)
B <- 2000
x <- rexp(B, rate = 1)

theta_hat_parcial <- cumsum(exp(-(x^2 - x))) / (1:B)

dados <- data.frame(b = 1:B, theta_hat = theta_hat_parcial)

ggplot(dados, aes(x = b, y = theta_hat)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = sqrt(pi) / 2, color = "red", linetype = "dashed") +
  labs(x = "B", y = expression(hat(theta)[B]),
       title = "Convergencia do estimador de Monte Carlo") +
  theme_minimal()
```



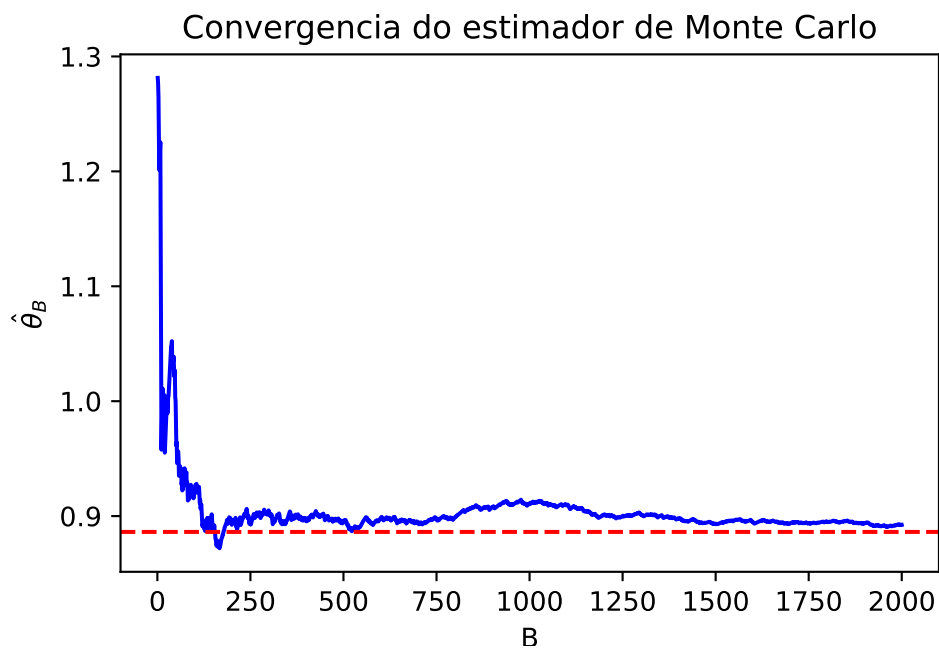
99 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(58)
B = 2000
x = np.random.exponential(scale=1.0, size=B)

theta_hat_parcial = np.cumsum(np.exp(-(x**2 - x))) / np.arange(1, B + 1)

plt.plot(np.arange(1, B + 1), theta_hat_parcial, color='blue')
plt.axhline(y=np.sqrt(np.pi) / 2, color='red', linestyle='--')
plt.xlabel('B')
plt.ylabel(r'$\hat{\theta}_B$')
plt.title('Convergencia do estimador de Monte Carlo')
plt.show()
```



i A escolha de f é livre — e importa

No exemplo acima poderíamos ter usado qualquer outra densidade com suporte em $(0, \infty)$: uma $\text{Exp}(2)$, uma Gama, uma meia-normal. Todas dariam estimadores **não viesados** do mesmo θ , mas com **variâncias diferentes** — isto é, umas exigiriam muito menos simulações que outras para a mesma precisão. Escolher f de propósito para reduzir a variância é o assunto do Capítulo 11 (amostragem por importância).

99.1 Exemplo 4: Aproximando uma Probabilidade

Seja $X \sim \text{Gama}(2, 3)$. Queremos aproximar o valor de $\mathbb{P}(X \geq 0,4)$ pelo método de Monte Carlo. À primeira vista isso não parece uma esperança, mas é: note que

$$\theta := \mathbb{P}(X \geq 0,4) = \int g(x)f(x) dx = \mathbb{E}[g(X)],$$

em que $g(x) = I(x \geq 0,4)$ é a função indicadora e f é a densidade da $\text{Gama}(2, 3)$. Isso vale porque a esperança de uma indicadora é justamente a probabilidade do evento indicado: $\mathbb{E}[I(X \in A)] = \mathbb{P}(X \in A)$.

Neste caso, o algoritmo corresponde a gerar $X_i \sim \text{Gama}(2, 3)$ e definir $Y_i = I(X_i \geq 0,4)$, que vale 1 quando $X_i \geq 0,4$ e 0 caso contrário. A estimativa de Monte Carlo é a média dos Y_i , ou seja, a **proporção de valores gerados que caíram no evento**:

100 R

```
set.seed(58)

# Número de simulações
B <- 50000

# Geração de valores da distribuição Gama(2,3)
x <- rgamma(B, shape = 2, rate = 3)

# Indicadora do evento de interesse: TRUE vira 1 e FALSE vira 0
y <- as.integer(x >= 0.4)

# Estimativa via Monte Carlo
valor_aproximado <- mean(y)
cat("Valor aproximado via Monte Carlo:", valor_aproximado, "\n")
```

Valor aproximado via Monte Carlo: 0.66282

```
# Valor real, usando a função de distribuição acumulada (f.d.a.)
valor_real <- 1 - pgamma(0.4, shape = 2, rate = 3)
cat("Valor real (f.d.a.):", valor_real, "\n")
```

Valor real (f.d.a.): 0.6626273

101 Python

```
import numpy as np
from scipy.stats import gamma

np.random.seed(58)

# Número de simulações
B = 50000

# Geração de valores da distribuição Gama(2,3): no scipy, o segundo
# parâmetro entra como escala, isto é, como 1/taxa
x = gamma.rvs(2, scale=1/3, size=B)

# Indicadora do evento de interesse: True vira 1 e False vira 0
y = (x >= 0.4).astype(int)

# Estimativa via Monte Carlo
valor_aproximado = np.mean(y)
print("Valor aproximado via Monte Carlo:", valor_aproximado)
```

Valor aproximado via Monte Carlo: 0.66114

```
# Valor real, usando a função de distribuição acumulada (f.d.a.)
valor_real = 1 - gamma.cdf(0.4, 2, scale=1/3)
print("Valor real (f.d.a.):", valor_real)
```

Valor real (f.d.a.): 0.6626272662068446

i Toda probabilidade é uma esperança

O que fizemos aqui vale sempre: para estimar $\mathbb{P}(X \in A)$, basta gerar valores de X e calcular a proporção deles que cai em A . É o caso $g = I(\cdot \in A)$ do método geral, e por isso não precisamos de nenhuma teoria nova para estimar probabilidades — nem mesmo

quando A é um evento complicado, descrito por várias variáveis ao mesmo tempo, como no Exemplo 8.

101.1 Exemplo 5: Aproximando o valor de π

Neste exemplo, queremos aproximar o valor de π utilizando o método de Monte Carlo. A ideia é gerar pontos aleatórios em um quadrado e contar quantos caem dentro de um círculo inscrito no quadrado. Vamos seguir o raciocínio a partir da geometria básica.

101.1.1 Geometria

- Considere um quadrado com lado 2 centrado na origem, ou seja, o quadrado vai de $(-1, -1)$ até $(1, 1)$.
- Dentro deste quadrado, inscreva um círculo de raio 1, também centrado na origem.
- A área do quadrado é 4 (já que $2 \times 2 = 4$) e a área do círculo é $\pi \cdot r^2 = \pi \cdot 1^2 = \pi$.

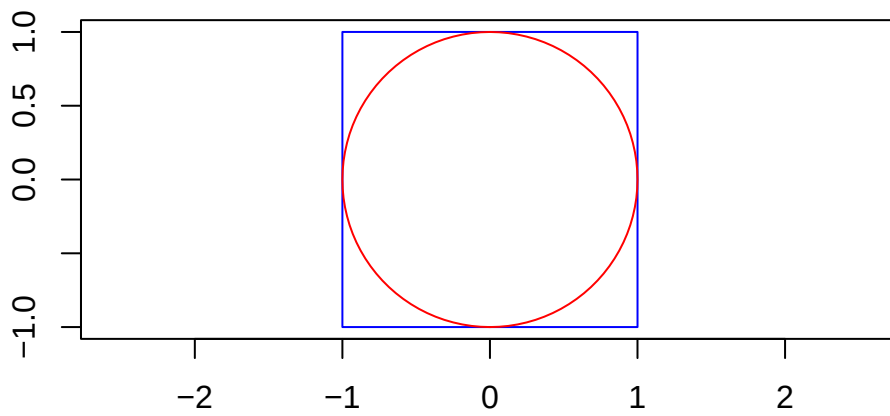
A razão entre a área do círculo e a área do quadrado é dada por:

$$\frac{\text{Área do círculo}}{\text{Área do quadrado}} = \frac{\pi}{4}$$

102 R

```
require(plotrix)

plot(c(-1, 1), c(-1, 1), type = "n", asp = 1, xlab = "", ylab = "")
rect(-1, -1, 1, 1, border = "blue")
draw.circle(0, 0, 1, border = "red")
```



103 Python

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Circle

# Configura o gráfico
fig, ax = plt.subplots()
ax.set_aspect('equal') # Define o aspecto como 1:1 (quadrado)
ax.set_xlim(-1, 1)
```

(-1.0, 1.0)

```
ax.set_ylim(-1, 1)
```

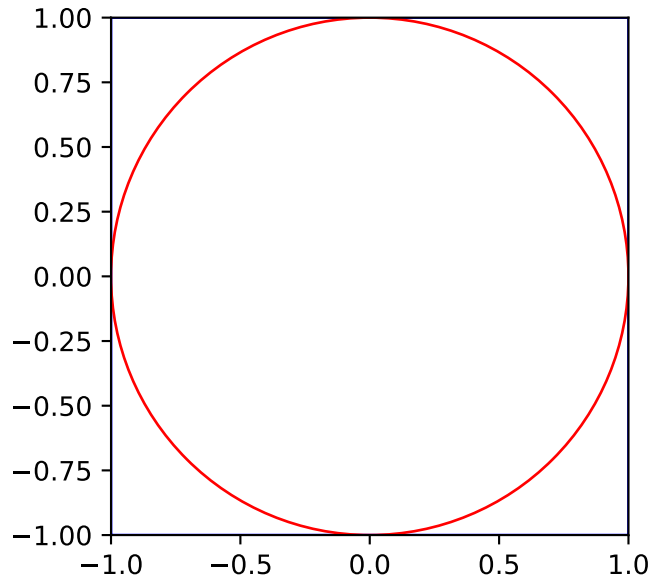
(-1.0, 1.0)

```
ax.set_xlabel('')
ax.set_ylabel('')

# Desenha o retângulo
rect = Rectangle((-1, -1), 2, 2, edgecolor='blue', facecolor='none')
ax.add_patch(rect)

# Desenha o círculo
circle = Circle((0, 0), 1, edgecolor='red', facecolor='none')
ax.add_patch(circle)

# Mostra o gráfico
plt.show()
```

Para estimar π usando Monte Carlo, procedemos da seguinte forma:

1. Geramos pontos aleatórios (x, y) no quadrado $[-1, 1] \times [-1, 1]$.
2. Verificamos se cada ponto está dentro do círculo, o que ocorre se $x^2 + y^2 \leq 1$.
3. A fração de pontos que caem dentro do círculo aproxima a razão $\frac{\pi}{4}$.
4. Multiplicamos essa fração por 4 para obter uma estimativa de π .

103.0.1 Matemática do estimador

Formalmente, se (X, Y) é um ponto com $X \sim \text{Unif}(-1, 1)$ e $Y \sim \text{Unif}(-1, 1)$ independentes, e $g(x, y) = I((x, y) \text{ está no círculo})$, temos que

$$\theta := \int g(x, y) f(x, y) dx dy = \frac{\text{Área do círculo}}{\text{Área do quadrado}} = \frac{\pi}{4}.$$

Assim, se (X_i, Y_i) é um ponto gerado uniformemente dentro do quadrado, o estimador de Monte Carlo para $\frac{\pi}{4}$ é dado por

$$\hat{\theta}_B = \frac{1}{B} \sum_{i=1}^B g(X_i, Y_i) = \frac{1}{B} \sum_{i=1}^B Z_i,$$

em que $Z_i = 1$ se o ponto i está dentro do círculo (i.e., se $X_i^2 + Y_i^2 \leq 1$) e $Z_i = 0$ caso contrário.

Multiplicando por 4, obtemos a estimativa de π :

$$\hat{\pi}_B = 4 \cdot \hat{\theta}_B = 4 \cdot \frac{1}{B} \sum_{i=1}^B Z_i.$$

Pela Lei Forte dos Grandes Números,

$$\hat{\pi}_B \longrightarrow \pi \quad (\text{quase certamente, quando } B \rightarrow \infty).$$

Ou seja, à medida que o número de pontos simulados B aumenta, a estimativa $\hat{\pi}_B$ converge para o valor verdadeiro de π .

O código abaixo simula esse processo, gerando B pontos e calculando a aproximação de π com base nos que caem dentro do círculo. O gráfico mostra como a estimativa melhora conforme o número de simulações aumenta.

104 R

```
set.seed(459)

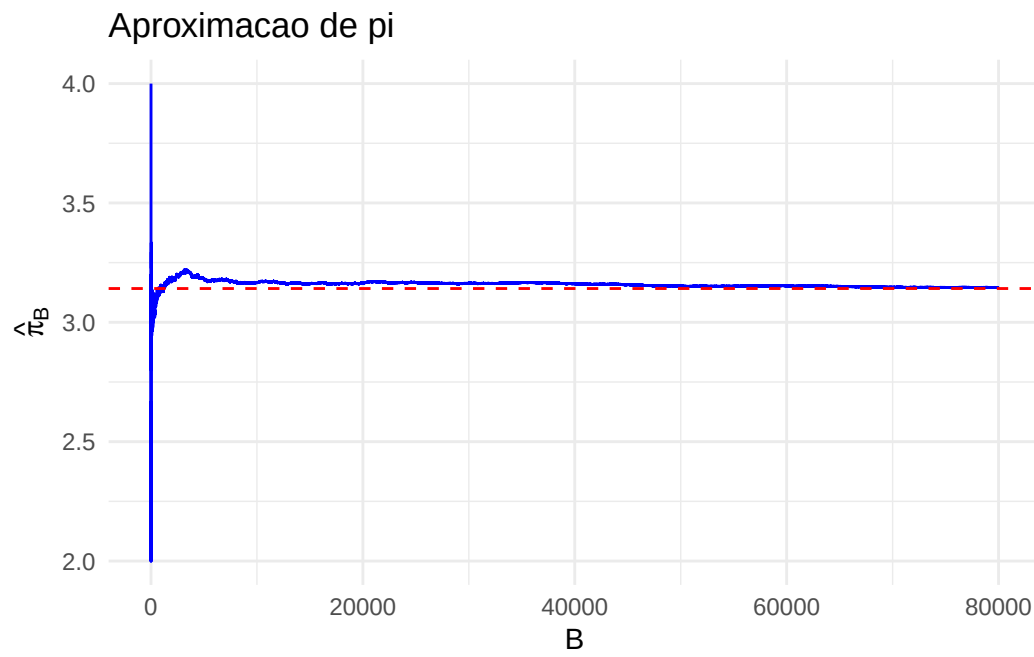
B <- 80000
z <- numeric(B)

# Loop para gerar os pontos e verificar se estão dentro do círculo.
# Como  $U \sim \text{Unif}(0,1)$ ,  $2*U - 1$  tem distribuição  $\text{Unif}(-1,1)$ 
for (i in 1:B) {
  x <- 2 * runif(1) - 1
  y <- 2 * runif(1) - 1
  z[i] <- (x^2 + y^2 <= 1)
}

# Estimativa de pi usando os i primeiros pontos, para cada i
theta_hat <- cumsum(z) / (1:B)
pi_hat <- theta_hat * 4

dados <- data.frame(b = 1:B, pi_hat = pi_hat)

ggplot(dados, aes(x = b, y = pi_hat)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = pi, color = "red", linetype = "dashed") +
  labs(x = "B", y = expression(hat(pi)[B]),
       title = "Aproximacao de pi") +
  theme_minimal()
```



105 Python

```
import numpy as np
import matplotlib.pyplot as plt

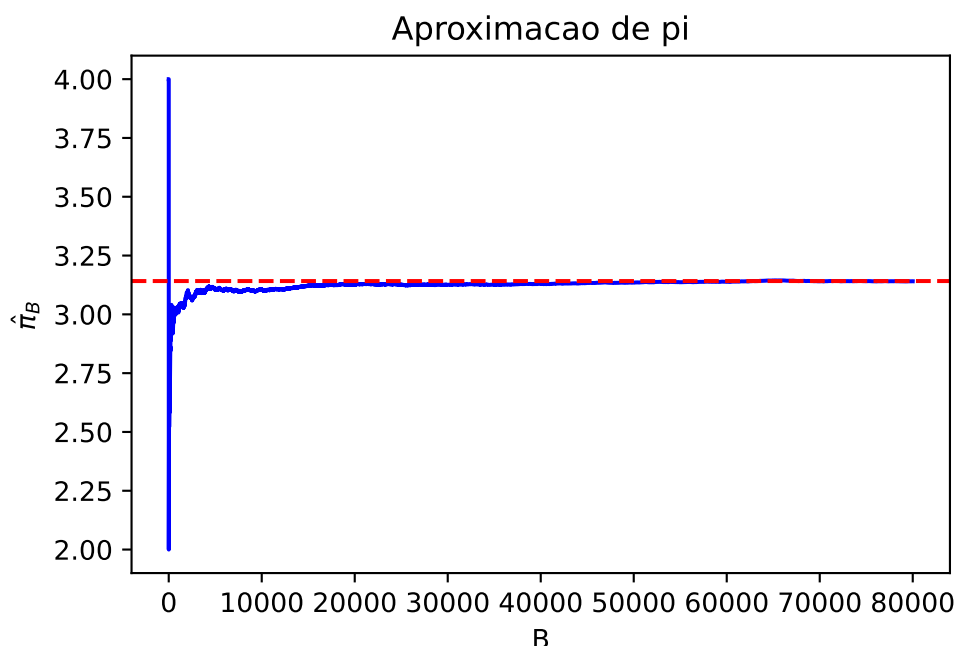
np.random.seed(459)

B = 80000
z = np.zeros(B)

# Loop para gerar os pontos e verificar se estão dentro do círculo.
# Como  $U \sim \text{Unif}(0,1)$ ,  $2U - 1$  tem distribuição  $\text{Unif}(-1,1)$ 
for i in range(B):
    x = 2 * np.random.uniform(0, 1) - 1
    y = 2 * np.random.uniform(0, 1) - 1
    z[i] = (x**2 + y**2 <= 1)

# Estimativa de pi usando os i primeiros pontos, para cada i
theta_hat = np.cumsum(z) / np.arange(1, B + 1)
pi_hat = theta_hat * 4

plt.plot(np.arange(1, B + 1), pi_hat, color='blue')
plt.axhline(y=np.pi, color='red', linestyle='--')
plt.xlabel('B')
plt.ylabel(r' $\hat{\pi}_B$ ')
plt.title('Aproximacao de pi')
plt.show()
```



Repare na escala do eixo horizontal: foram necessários dezenas de milhares de pontos para estabilizar as duas primeiras casas decimais de π . Monte Carlo é um método geral, não um método rápido — como veremos a seguir.

105.1 Quão preciso é o método?

O item (ii) da proposição diz que $\text{Var}(\hat{\theta}_B) = \sigma^2/B$, com $\sigma^2 = \text{Var}(g(X))$. Tomando a raiz quadrada, o **erro padrão** do estimador é

$$\text{ep}(\hat{\theta}_B) = \frac{\sigma}{\sqrt{B}}.$$

O $\sqrt{B}$ no denominador é a característica mais importante do método, e vale a pena olhar para ela com atenção:

B	erro padrão
100	$\sigma/10$
10 000	$\sigma/100$
1 000 000	$\sigma/1000$

Ou seja: **para ganhar uma casa decimal, é preciso multiplicar B por 100**. Foi exatamente isso que vimos no Exemplo 5, em que 80 000 pontos ainda deixavam a terceira casa de π indefinida. Não adianta insistir: a taxa $1/\sqrt{B}$ é inerente ao método, e a única forma de melhorá-la é reduzir σ — o assunto dos Capítulos 10 e 11.

Note também o que **não** aparece na fórmula: nada sobre a dimensão do problema, nada sobre a suavidade de g . Essa indiferença é o que torna o método tão útil.

i Monte Carlo e a maldição da dimensionalidade

Para aproximar $\int_0^1 g(x) dx$ em dimensão 1, métodos determinísticos como a regra do trapézio são muito melhores que Monte Carlo: com B pontos igualmente espaçados, o erro do trapézio é da ordem de B^{-2} , contra $B^{-1/2}$ do Monte Carlo.

A situação se inverte em dimensão alta. Para integrar em $[0, 1]^d$ com uma grade de m pontos por eixo, são necessárias $B = m^d$ avaliações, e o erro fica da ordem de $m^{-2} = B^{-2/d}$. Esse expoente **piora com d** : em dimensão 10, o erro cai como $B^{-1/5}$, muito mais devagar que o $B^{-1/2}$ do Monte Carlo, que não muda com a dimensão.

Em problemas de dimensão moderada ou alta — o caso típico em estatística, em que integramos sobre todos os parâmetros de um modelo — Monte Carlo não é uma alternativa entre outras: é frequentemente a única viável.

105.2 Intervalos de confiança para estimativas de Monte Carlo

Saber que o erro é da ordem de $\sigma/\sqrt{B}$ é pouco útil na prática, porque não conhecemos σ (se conhecêssemos a distribuição de $g(X)$ tão bem, talvez nem precisássemos simular). A saída é **estimar σ com os mesmos valores simulados** e usar essa estimativa para construir um **intervalo de confiança**.

Pelo Teorema Central do Limite, para B suficientemente grande,

$$\hat{\theta}_B \stackrel{\text{aprox.}}{\sim} N\left(\theta, \frac{\sigma^2}{B}\right),$$

em que $\sigma^2 = \text{Var}(g(X))$. Substituindo σ por sua estimativa $\hat{\sigma}$, obtemos o intervalo de confiança aproximado para θ , com nível de confiança $(1 - \alpha) \times 100\%$:

$$\hat{\theta}_B \pm z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{B}},$$

em que $z_{\alpha/2}$ é o quantil da $N(0, 1)$ que deixa $\alpha/2$ de probabilidade à direita (por exemplo, $z_{0,025} \approx 1,96$ para 95% de confiança).

💡 Pseudo-algoritmo: intervalo de confiança para θ

1. Gere $X_1, \dots, X_B$ i.i.d. com a distribuição de X e calcule $Y_i = g(X_i)$.

2. Calcule a estimativa pontual

$$\hat{\theta}_B = \frac{1}{B} \sum_{i=1}^B Y_i.$$

3. Estime o desvio padrão de $g(X)$:

$$\hat{\sigma} = \sqrt{\frac{1}{B-1} \sum_{i=1}^B (Y_i - \hat{\theta}_B)^2}.$$

4. Calcule o erro padrão $\hat{\sigma}/\sqrt{B}$ e devolva

$$\left[\hat{\theta}_B - z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{B}}, \hat{\theta}_B + z_{\alpha/2} \frac{\hat{\sigma}}{\sqrt{B}} \right].$$

⚠️ Atenção: desvio padrão e erro padrão são coisas diferentes

$\hat{\sigma}$ estima a dispersão de **uma** observação $g(X_i)$, e não muda quando B cresce. O que encolhe com B é o **erro padrão** $\hat{\sigma}/\sqrt{B}$, que mede a dispersão da **média**. Trocar um pelo outro produz intervalos absurdamente largos (ou, pior, um código que parece funcionar).

105.3 Exemplo 6: Intervalo de Confiança para a Estimativa de π

Vamos construir um intervalo de confiança de 95% para a estimativa de π do Exemplo 5. Aqui $g(X_i, Y_i) = Z_i$ é uma indicadora, e o intervalo é construído para $\theta = \pi/4$; ao final, multiplicamos os dois extremos por 4 para obter um intervalo para π .

106 R

```
set.seed(0)
B <- 10000 # número de simulações
z <- numeric(B)

# Loop para gerar os pontos e verificar se estão dentro do círculo
for (i in 1:B) {
  x <- 2 * runif(1) - 1
  y <- 2 * runif(1) - 1
  z[i] <- (x^2 + y^2 <= 1)
}

# Estimativa pontual
theta_hat <- mean(z)
pi_hat <- theta_hat * 4
cat("Estimativa de pi:", pi_hat, "\n")
```

Estimativa de pi: 3.1308

```
# Desvio padrão de uma observação e erro padrão da média
sigma_hat <- sd(z)
erro_padrao <- sigma_hat / sqrt(B)

alpha <- 0.05 # nível de significância
z_alpha2 <- qnorm(1 - alpha / 2)

# Intervalo de confiança para theta = pi/4 e, multiplicando por 4, para pi
ic_theta <- c(theta_hat - z_alpha2 * erro_padrao,
             theta_hat + z_alpha2 * erro_padrao)
ic_pi <- 4 * ic_theta
cat("Intervalo de confiança para pi:", ic_pi, "\n")
```

Intervalo de confiança para pi: 3.098466 3.163134

107 Python

```
import numpy as np
from scipy.stats import norm

np.random.seed(0)
B = 10000 # número de simulações
z = np.zeros(B)

# Loop para gerar os pontos e verificar se estão dentro do círculo
for i in range(B):
    x = 2 * np.random.uniform(0, 1) - 1
    y = 2 * np.random.uniform(0, 1) - 1
    z[i] = (x**2 + y**2 <= 1)

# Estimativa pontual
theta_hat = np.mean(z)
pi_hat = theta_hat * 4
print("Estimativa de pi:", pi_hat)
```

Estimativa de pi: 3.1228

```
# Desvio padrão de uma observação e erro padrão da média.
# ddof=1 faz o numpy dividir por B-1, como o sd() do R
sigma_hat = np.std(z, ddof=1)
erro_padrao = sigma_hat / np.sqrt(B)

alpha = 0.05 # nível de significância
z_alpha2 = norm.ppf(1 - alpha / 2)

# Intervalo de confiança para theta = pi/4 e, multiplicando por 4, para pi
ic_theta = np.array([theta_hat - z_alpha2 * erro_padrao,
                     theta_hat + z_alpha2 * erro_padrao])
ic_pi = 4 * ic_theta
print("Intervalo de confiança para pi:", ic_pi)
```

Intervalo de confiança para pi: [3.09035923 3.15524077]

O intervalo obtido tem semi-amplitude próxima de 0,03, ou seja, ele localiza π com incerteza já na segunda casa decimal — 10 000 simulações não bastam nem para garantir o “3,14”. E, ao contrário do gráfico do Exemplo 5, esse diagnóstico foi obtido **sem conhecer o valor verdadeiro**, usando apenas os valores simulados. É assim que se reporta uma estimativa de Monte Carlo: nunca sozinha, sempre acompanhada do erro padrão ou de um intervalo.

107.0.1 Quantas simulações são necessárias?

O intervalo também responde à pergunta prática do capítulo: se queremos que a semi-amplitude do intervalo seja no máximo ε , precisamos de

$$z_{\alpha/2} \frac{\sigma}{\sqrt{B}} \leq \varepsilon \quad \Longleftrightarrow \quad B \geq \left(\frac{z_{\alpha/2} \sigma}{\varepsilon} \right)^2.$$

Como σ é desconhecido, a receita usual tem duas etapas: rodamos uma simulação **piloto**, pequena, só para estimar σ ; com $\hat{\sigma}$ em mãos, calculamos o B necessário e rodamos a simulação de verdade. Vejamos quantas simulações seriam necessárias para determinar π com erro de, no máximo, 0,001:

108 R

```
set.seed(1)

# Etapa 1: simulação piloto, apenas para estimar sigma
B_piloto <- 1000
x <- runif(B_piloto, -1, 1)
y <- runif(B_piloto, -1, 1)
z <- as.integer(x^2 + y^2 <= 1)
sigma_hat <- sd(z)

# Etapa 2: B necessário para a semi-amplitude desejada.
# Como estimamos  $\pi = 4\theta$ , o erro em  $\pi$  é 4 vezes o erro em  $\theta$ 
epsilon <- 0.001
z_alpha2 <- qnorm(0.975)
B_necessario <- (z_alpha2 * 4 * sigma_hat / epsilon)^2

cat("Desvio padrão estimado:", sigma_hat, "\n")
```

Desvio padrão estimado: 0.4210431

```
cat("B necessário:", ceiling(B_necessario), "\n")
```

B necessário: 10896054

109 Python

```
import numpy as np
from scipy.stats import norm

np.random.seed(1)

# Etapa 1: simulação piloto, apenas para estimar sigma
B_piloto = 1000
x = np.random.uniform(-1, 1, B_piloto)
y = np.random.uniform(-1, 1, B_piloto)
z = (x**2 + y**2 <= 1).astype(int)
sigma_hat = np.std(z, ddof=1)

# Etapa 2: B necessário para a semi-amplitude desejada.
# Como estimamos  $\pi = 4\theta$ , o erro em  $\pi$  é 4 vezes o erro em  $\theta$ 
epsilon = 0.001
z_alpha2 = norm.ppf(0.975)
B_necessario = (z_alpha2 * 4 * sigma_hat / epsilon)**2

print("Desvio padrão estimado:", sigma_hat)
```

Desvio padrão estimado: 0.42358402168096876

```
print("B necessário:", int(np.ceil(B_necessario)))
```

B necessário: 11027964

O número que sai é da ordem de dez milhões de simulações — e ainda assim para apenas três casas decimais de π . Arquimedes, com polígonos inscritos e circunscritos e sem computador algum, já garantia as duas primeiras casas no século III a.C.

⚠ Atenção: o intervalo de confiança pode falhar

Todo o raciocínio acima depende de duas hipóteses:

- $\sigma^2 = \text{Var}(g(X))$ deve ser **finita**. Se não for, o Teorema Central do Limite não se aplica, e o intervalo não tem o nível de confiança prometido — por mais que o código rode sem erro e devolva um intervalo de aparência inocente (veja o Exercício 13).
- B deve ser grande o bastante para que a aproximação normal valha. Isso costuma ser inofensivo, exceto quando estimamos a probabilidade de um **evento raro**: se $\theta = 10^{-6}$ e $B = 10^5$, o mais provável é não observar nenhuma ocorrência do evento, obter $\hat{\theta}_B = 0$ e, pior, um intervalo de largura zero. Esse problema motiva o Capítulo 11.

109.1 Exemplo 7: Quando o método falha

A hipótese $\mathbb{E}[|g(X)|] < \infty$ da proposição não é decorativa. Considere a distribuição de **Cauchy**, cuja densidade é

$$f(x) = \frac{1}{\pi(1+x^2)}, \quad x \in \mathbb{R}.$$

Ela é simétrica em torno de zero, então seria natural esperar que a média de valores gerados convergisse para 0. Mas a Cauchy tem caudas tão pesadas que $\mathbb{E}[|X|] = \infty$: a Lei dos Grandes Números não vale, e a média amostral simplesmente **não converge**. O gráfico abaixo compara a média corrente de valores $N(0, 1)$ com a de valores Cauchy:

110 R

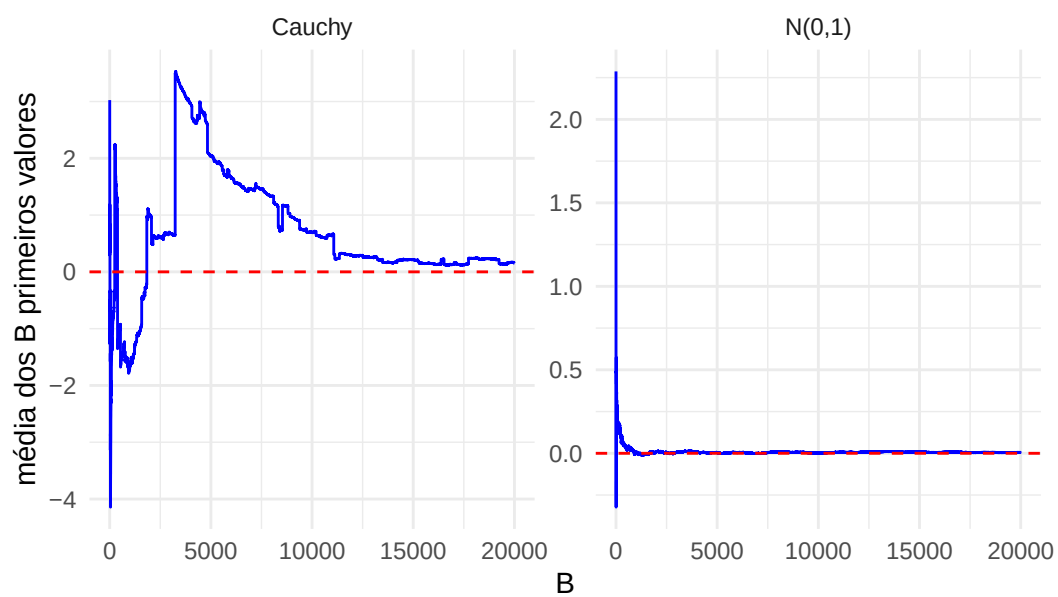
```
set.seed(7)
B <- 20000

# Média dos i primeiros valores gerados, para cada i, nas duas distribuições
media_normal <- cumsum(rnorm(B)) / (1:B)
media_cauchy <- cumsum(rcauchy(B)) / (1:B)

dados <- data.frame(
  b = rep(1:B, 2),
  media = c(media_normal, media_cauchy),
  distribuicao = rep(c("N(0,1)", "Cauchy"), each = B)
)

ggplot(dados, aes(x = b, y = media)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  facet_wrap(~ distribuicao, scales = "free_y") +
  labs(x = "B", y = "média dos B primeiros valores",
       title = "A Lei dos Grandes Numeros precisa de esperanca finita") +
  theme_minimal()
```

A Lei dos Grandes Numeros precisa de esperanca finita



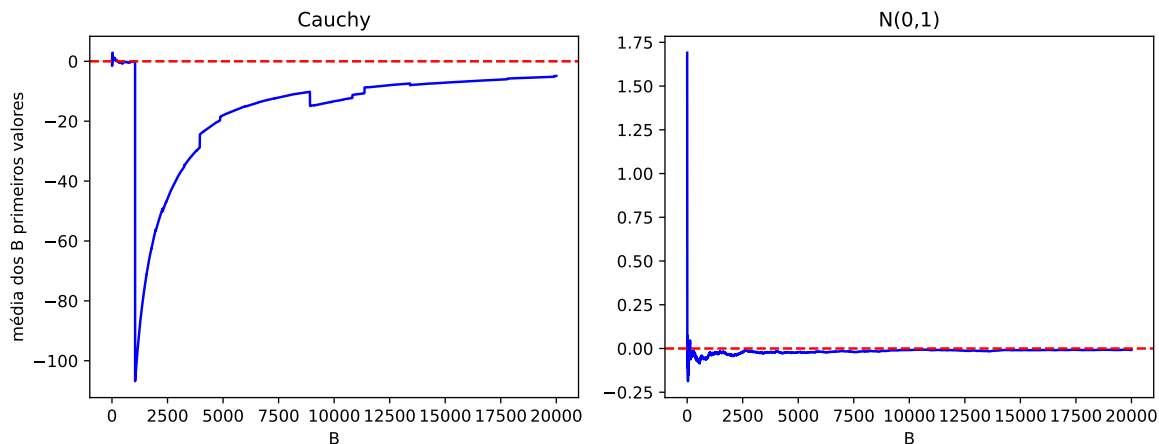
111 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import cauchy

np.random.seed(7)
B = 20000

# Média dos i primeiros valores gerados, para cada i, nas duas distribuições
indices = np.arange(1, B + 1)
media_normal = np.cumsum(np.random.normal(0, 1, B)) / indices
media_cauchy = np.cumsum(cauchy.rvs(size=B)) / indices

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, media, nome in zip(axes, [media_cauchy, media_normal],
                           ["Cauchy", "N(0,1)"]):
    ax.plot(indices, media, color='blue')
    ax.axhline(y=0, color='red', linestyle='--')
    ax.set_title(nome)
    ax.set_xlabel('B')
axes[0].set_ylabel('média dos B primeiros valores')
plt.tight_layout()
plt.show()
```



No painel da normal, a média se cola no zero. No da Cauchy, ela dá saltos que não diminuem: de tempos em tempos aparece um valor gigantesco, que sozinho desloca a média inteira. Rodar mais simulações não resolve — e, o que é mais perigoso, **nada no código avisa que algo deu errado**. Cabe a quem simula verificar antes se a esperança existe.

111.1 Exemplo 8: Problema das Figurinhas

Um colecionador está juntando figurinhas para completar um álbum da Copa. Qual é a probabilidade de que, ao comprar n pacotes, pelo menos um deles contenha duas ou mais figurinhas iguais? Essa probabilidade pode ser calculada analiticamente (veja o Exercício 12), mas o caminho por Monte Carlo dispensa qualquer conta: simulamos a compra de n pacotes, cada um com 5 figurinhas sorteadas entre as 640 do álbum, repetimos essa simulação $B = 1000$ vezes e contamos em quantas delas houve pelo menos um pacote com repetição. A proporção obtida é a estimativa de Monte Carlo da probabilidade desejada.

112 R

```
set.seed(42)
n_figurinhas <- 640
B <- 1000
n_pacotes <- 1:50

probab_coincidencia <- numeric(length(n_pacotes))

for (ii in seq_along(n_pacotes)) {
  tem_repetida <- numeric(B)
  for (jj in 1:B) {
    # Cada linha da matriz é um pacote com 5 figurinhas sorteadas com reposição
    figurinhas <- matrix(sample(1:n_figurinhas, n_pacotes[ii] * 5, replace = TRUE),
                        nrow = n_pacotes[ii], ncol = 5)
    # apply(m, 1, f) aplica a função f a cada linha da matriz m; aqui, f verifica
    # se o pacote tem alguma figurinha repetida
    repetida_no_pacote <- apply(figurinhas, 1, function(pacote) any(duplicated(pacote)))
    tem_repetida[jj] <- any(repetida_no_pacote)
  }
  probab_coincidencia[ii] <- mean(tem_repetida)
}
```

113 Python

```
import numpy as np

np.random.seed(42)
n_figurinhas = 640
B = 1000
n_pacotes = np.arange(1, 51)

prob_coincidencia = np.zeros(len(n_pacotes))

for ii in range(len(n_pacotes)):
    tem_repetida = np.zeros(B)
    for jj in range(B):
        # Cada linha da matriz é um pacote com 5 figurinhas sorteadas com reposição
        figurinhas = np.random.randint(1, n_figurinhas + 1,
                                         size=(n_pacotes[ii], 5))

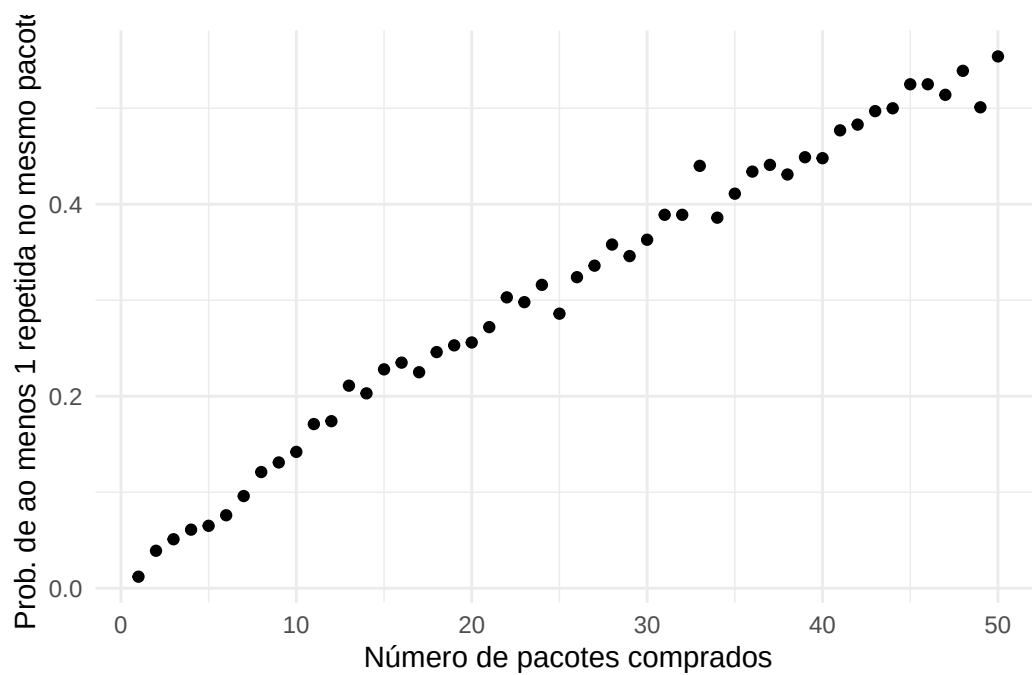
        # len(set(pacote)) < 5 indica que o pacote tem figurinha repetida
        repetida_no_pacote = [len(set(pacote)) < 5 for pacote in figurinhas]
        tem_repetida[jj] = any(repetida_no_pacote)
    prob_coincidencia[ii] = np.mean(tem_repetida)
```

Agora podemos visualizar os resultados obtidos por meio do gráfico a seguir:

114 R

```
dados <- data.frame(n_pacotes = n_pacotes, prob_coincidencia = prob_coincidencia)

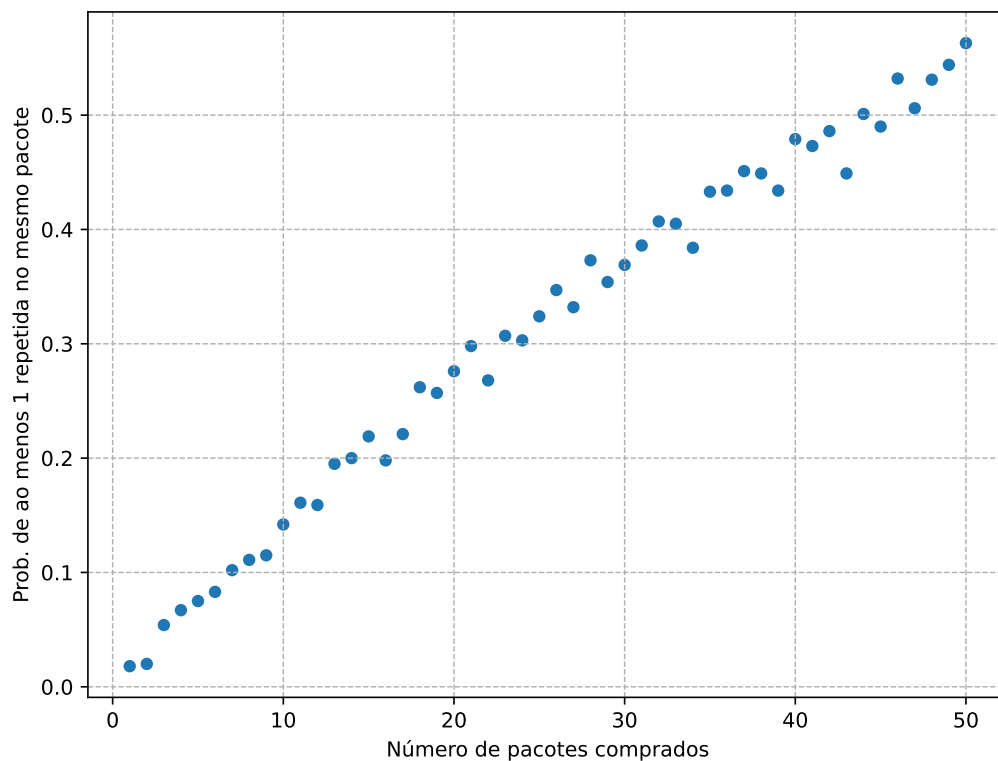
ggplot(dados, aes(x = n_pacotes, y = prob_coincidencia)) +
  geom_point() +
  labs(x = "Número de pacotes comprados",
       y = "Prob. de ao menos 1 repetida no mesmo pacote") +
  theme_minimal()
```



115 Python

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(n_pacotes, probab_coincidencia, s=25)
plt.xlabel('Número de pacotes comprados')
plt.ylabel('Prob. de ao menos 1 repetida no mesmo pacote')
plt.grid(True, linestyle='--', linewidth=0.7)
plt.show()
```



Simulação para completar o álbum

Uma segunda pergunta que também podemos responder via simulação é: quantos pacotes são necessários, em média, para completar o álbum? Aqui o número de pacotes é aleatório, e não fixo: compramos pacotes até que todas as 640 figurinhas tenham aparecido pelo menos uma vez. Para isso, guardamos em um vetor de `TRUE/FALSE` quais figurinhas já temos.

116 R

```
set.seed(42)
numero_pacotes <- numeric(B)

for (jj in 1:B) {
  ja_tenho <- rep(FALSE, n_figurinhas) # ja_tenho[k]: a figurinha k já apareceu?
  n_distintas <- 0
  contador_pacotes <- 0

  while (n_distintas < n_figurinhas) {
    pacote <- sample(1:n_figurinhas, 5, replace = TRUE)
    for (figurinha in pacote) {
      if (!ja_tenho[figurinha]) {
        ja_tenho[figurinha] <- TRUE
        n_distintas <- n_distintas + 1
      }
    }
    contador_pacotes <- contador_pacotes + 1
  }

  numero_pacotes[jj] <- contador_pacotes
}
```


117 Python

```
import numpy as np

np.random.seed(42)
numero_pacotes = np.zeros(B)

for jj in range(B):
    ja_tenho = np.zeros(n_figurinhas, dtype=bool) # ja_tenho[k]: já apareceu?
    n_distintas = 0
    contador_pacotes = 0

    while n_distintas < n_figurinhas:
        # em Python os índices começam em 0, então numeramos as figurinhas
        # de 0 a 639 para usá-las diretamente como índice de ja_tenho
        pacote = np.random.randint(0, n_figurinhas, size=5)
        for figurinha in pacote:
            if not ja_tenho[figurinha]:
                ja_tenho[figurinha] = True
                n_distintas += 1
        contador_pacotes += 1

    numero_pacotes[jj] = contador_pacotes
```

O número médio de pacotes necessários para completar o álbum é:

118 R

```
mean(numero_pacotes)
```

```
[1] 896.684
```

119 Python

```
np.mean(numero_pacotes)
```

```
np.float64(902.748)
```

Com essa mesma simulação, podemos ainda estimar outras quantidades de interesse, sem gerar nada de novo — basta calcular a proporção das repetições em que o evento ocorreu:

120 R

```
cat("Probabilidade de precisar de mais de 800 pacotes: ",  
    mean(numero_pacotes > 800) * 100, "%\n", sep = "")
```

Probabilidade de precisar de mais de 800 pacotes: 69.5%

```
cat("Probabilidade de precisar de mais de 1000 pacotes: ",  
    mean(numero_pacotes > 1000) * 100, "%\n", sep = "")
```

Probabilidade de precisar de mais de 1000 pacotes: 21.8%

121 Python

```
prob_mais_800 = np.mean(numero_pacotes > 800) * 100
prob_mais_1000 = np.mean(numero_pacotes > 1000) * 100

print(f"Probabilidade de precisar de mais de 800 pacotes: {prob_mais_800}%")
```

Probabilidade de precisar de mais de 800 pacotes: 71.8%

```
print(f"Probabilidade de precisar de mais de 1000 pacotes: {prob_mais_1000}%")
```

Probabilidade de precisar de mais de 1000 pacotes: 22.900000000000002%

Repare no que este exemplo tem de diferente dos anteriores: não há integral nenhuma à vista, e seria trabalhoso escrever a densidade da v.a. “número de pacotes até completar o álbum”. Ainda assim, o método se aplica sem alteração, porque tudo o que ele exige é saber **simular** o experimento — não descrevê-lo em fórmulas.

121.1 Exercícios

Exercício 1. Considere a integral

$$\theta = \int_0^{10} \sin(x^2) dx.$$

- (a) Escreva θ na forma $(b - a) \mathbb{E}[g(U)]$, explicitando quem são U e g .
- (b) Estime θ por Monte Carlo com $B = 10^4$ e forneça um intervalo de confiança de 95%.
- (c) Compare sua estimativa com o valor obtido por integração numérica (`integrate` em R, `scipy.integrate.quad` em Python).
- (d) Quantas simulações seriam necessárias para que a semi-amplitude do intervalo fosse menor que 0,01? Verifique rodando com esse B .

Exercício 2. Considere a integral

$$\theta = \int_1^{\infty} \frac{1}{x^3} dx.$$

- (a) Calcule θ analiticamente.
- (b) A integral não é sobre um intervalo limitado, de modo que não podemos usar a uniforme diretamente. Um caminho é a mudança de variável $u = 1/x$: verifique que ela leva a $\theta = \int_0^1 u du = \mathbb{E}[U]$, com $U \sim \text{Unif}(0, 1)$. Estime θ assim, com $B = 10^4$, e forneça um intervalo de confiança.
- (c) Outro caminho é escolher uma densidade com suporte em $(1, \infty)$, como a de $X = 1 + Y$ com $Y \sim \text{Exp}(1)$, que é $f(x) = e^{-(x-1)}$ para $x > 1$. Escreva o estimador correspondente e mostre que $\mathbb{E}[(g(X)/f(X))^2] = \infty$, ou seja, que ele tem variância infinita. Rode-o mesmo assim, com $B = 10^4$, e note que ele *parece* funcionar perfeitamente. Explique por quê: a partir de que valor de X a razão $g(X)/f(X)$ começa a explodir, e qual é a probabilidade de observar um valor desses? Moral: ter o suporte certo não basta para uma densidade ser uma boa escolha, e o problema pode não aparecer na simulação.
- (d) Voltando ao estimador do item (b), construa 10 000 estimativas de θ , cada uma baseada em $B = 100$ simulações, com seus respectivos intervalos de confiança de 95%. Qual a proporção dos intervalos que contém o valor verdadeiro de θ ?
- (e) Repita o item anterior com $B = 1000$. A proporção ficou mais próxima de 95%? Por quê?

Exercício 3. Seja $X \sim \text{Exp}(\lambda)$ com $\lambda = 2$.

- (a) Estime $\mathbb{E}[X^2]$ por Monte Carlo com $B = 10^4$, forneça um intervalo de confiança e compare com o valor exato $2/\lambda^2$.
- (b) Repita para $\mathbb{E}[X^4]$, cujo valor exato é $24/\lambda^4$.
- (c) Compare as larguras relativas dos dois intervalos (isto é, a semi-amplitude dividida pela estimativa). Por que estimar $\mathbb{E}[X^4]$ é mais difícil, mesmo gerando exatamente os mesmos valores de X ?

Exercício 4. Sejam $X \sim N(0, 1)$ e $Y \sim \text{Gama}(1, 1)$ independentes.

- (a) Estime $\mathbb{P}(X \cdot Y > 3)$ por Monte Carlo e forneça um intervalo de confiança.
- (b) Estime também $\mathbb{E}[X \cdot Y]$ e compare com o valor teórico $\mathbb{E}[X]\mathbb{E}[Y]$.
- (c) Este é um exemplo em que a densidade de XY não é imediata. Comente: o que o método de Monte Carlo exigiu que você soubesse sobre XY ?

Exercício 5. Seja $Z \sim N(0, 1)$. Mostre que sua função geradora de momentos é $\mathbb{E}[e^{tZ}] = e^{t^2/2}$. Para $t \in \{-1, 0.5, 1, 1.5\}$, estime $\mathbb{E}[e^{tZ}]$ por Monte Carlo, forneça intervalos de confiança e compare com a fórmula fechada. Estude como o erro decai quando B cresce e comente por que os valores maiores de t dão mais trabalho.

Exercício 6. Seja $X_1, \dots, X_n$ uma amostra i.i.d. de $X \sim \text{Gama}(2, 1)$, cuja média é 2. Para $n \in \{5, 20, 100\}$, calcule:

- (a) uma cota superior para $\mathbb{P}(\bar{X}_n - 2 > 0.3)$ usando a desigualdade de Markov;
- (b) uma cota superior para $\mathbb{P}(\bar{X}_n - 2 > 0.3)$ usando a desigualdade de Chebyshev;
- (c) uma aproximação para $\mathbb{P}(\bar{X}_n - 2 > 0.3)$ usando o Teorema Central do Limite;
- (d) uma aproximação para $\mathbb{P}(\bar{X}_n - 2 > 0.3)$ por Monte Carlo, com $B = 10\,000$ repetições (em cada repetição, gere uma amostra de tamanho n e calcule sua média).

Comente quem é mais informativo em cada n . Note que aqui n e B têm papéis completamente diferentes: qual deles você poderia aumentar à vontade se este fosse um problema real?

Exercício 7. O volume da bola unitária em dimensão d , $\{x \in \mathbb{R}^d : \|x\| \leq 1\}$, pode ser estimado exatamente como no Exemplo 5: sorteando pontos uniformemente no cubo $[-1, 1]^d$ (que tem volume 2^d) e contando quantos caem na bola.

- (a) Implemente o estimador $\hat{V}_d = 2^d \cdot \hat{p}$, em que $\hat{p}$ é a proporção de pontos com $\|x\| \leq 1$. Verifique-o em $d = 2$ (deve dar π) e em $d = 3$ (deve dar $4\pi/3$).
- (b) Estime V_{10} com $B = 10^5$ e compare com o valor exato $V_d = \pi^{d/2}/\Gamma(d/2 + 1)$ (use `gamma` em R e `math.gamma` em Python).
- (c) Faça um gráfico da proporção $\hat{p}$ de pontos aceitos em função de $d = 1, 2, \dots, 15$. O que acontece? Interprete: onde estão quase todos os pontos de um cubo em dimensão alta?
- (d) Explique por que o erro *relativo* do estimador piora com d , mesmo com B fixo. (Dica: para uma indicadora, $\text{Var}(\hat{p}) = p(1-p)/B$; escreva o erro padrão relativo em função de p .)

Exercício 8. (Agulha de Buffon) Uma agulha de comprimento 1 é jogada ao acaso sobre um piso com linhas paralelas separadas por uma distância 1. Pode-se mostrar que a probabilidade de a agulha cruzar alguma linha é $2/\pi$. Para simular um lançamento, gere a distância D do centro da agulha à linha mais próxima, com $D \sim \text{Unif}(0, 1/2)$, e o ângulo Θ da agulha com as linhas, com $\Theta \sim \text{Unif}(0, \pi/2)$, independentes; a agulha cruza uma linha se $D \leq \frac{1}{2} \sin(\Theta)$.

- (a) Simule $B = 10^5$ lançamentos, estime $2/\pi$ e obtenha uma estimativa de π a partir dela.
- (b) Forneça um intervalo de confiança para π . Cuidado: o intervalo sai naturalmente para $p = 2/\pi$; para passá-lo a π , aplique a transformação $p \mapsto 2/p$ aos dois extremos (e note que ela inverte a ordem).

- (c) Compare a precisão obtida com a do Exemplo 5 para o mesmo B . Qual dos dois métodos estima π com menos simulações?

Exercício 9. Este exercício verifica empiricamente a taxa $1/\sqrt{B}$. Considere $\theta = \mathbb{E}[e^{-U}]$ com $U \sim \text{Unif}(0, 1)$, cujo valor exato é $1 - e^{-1}$.

- (a) Para cada $B \in \{10, 10^2, 10^3, 10^4\}$, repita 500 vezes a estimação de θ e calcule a raiz do erro quadrático médio (REQM) dessas 500 estimativas em relação ao valor verdadeiro.
- (b) Faça um gráfico de $\log(\text{REQM})$ contra $\log(B)$. O que a inclinação da reta deveria ser? Estime-a (por exemplo, com `lm` em R ou `np.polyfit` em Python).
- (c) Compare a REQM obtida com o valor teórico $\sigma/\sqrt{B}$, calculando $\sigma = \sqrt{\text{Var}(e^{-U})}$ analiticamente.

Exercício 10. (Ruína do jogador) Um jogador começa com 10 fichas e, a cada rodada, ganha 1 ficha com probabilidade p ou perde 1 ficha com probabilidade $1 - p$, de forma independente. Ele para quando fica sem fichas ou quando chega a 20 fichas.

- (a) Simule $B = 10\,000$ partidas com $p = 0,5$ e estime a probabilidade de o jogador chegar a 20 fichas, com intervalo de confiança. Compare com o valor teórico $1/2$.
- (b) Estime também a duração média de uma partida e compare com o valor teórico $10 \times (20 - 10) = 100$ rodadas.
- (c) Repita com $p = 0,48$ (um cassino com pequena vantagem). A probabilidade de vitória cai muito mais do que os 2 pontos percentuais de desvantagem por rodada? Comente.
- (d) Ainda com $p = 0,48$, estime a probabilidade de vitória partindo de 10 fichas e indo até 20, e depois partindo de 100 fichas e indo até 200. O que acontece quando se joga “a mesma partida”, mas com mais fichas?

Exercício 11. Monte Carlo também serve para **comparar estimadores**, e não apenas para calcular integrais. Suponha uma amostra $X_1, \dots, X_n$ com $n = 25$, e considere dois estimadores do centro da distribuição: a média amostral e a mediana amostral.

- (a) Com $X_i \sim N(0, 1)$, gere $B = 10\,000$ amostras de tamanho 25; para cada uma, calcule a média e a mediana. Estime a variância de cada estimador e compare com os valores teóricos aproximados $1/n$ e $\pi/(2n)$.
- (b) Qual dos dois é preferível sob normalidade? E qual é a perda relativa de eficiência ao usar a mediana?
- (c) Repita o experimento com $X_i \sim t_2$ (uma distribuição de caudas pesadas, `rt(n, df = 2)` em R e `scipy.stats.t.rvs(2, size=n)` em Python). A conclusão se inverte?
- (d) Repita agora com $X_i \sim \text{Cauchy}$. Uma das duas variâncias estimadas vai crescer sem controle conforme você aumenta B . Qual, e por quê? (Compare com o Exemplo 7.)

Exercício 12. Este exercício confere as contas do Exemplo 8 contra a teoria.

- (a) Mostre que a probabilidade de um pacote de 5 figurinhas (sorteadas com reposição entre 640) ter pelo menos uma repetição é $p = 1 - \prod_{k=1}^4 (1 - \frac{k}{640})$, e calcule seu valor.
- (b) Conclua que, com n pacotes, a probabilidade de haver pelo menos um pacote com repetição é $1 - (1 - p)^n$. Sobreponha essa curva ao gráfico do Exemplo 8.
- (c) O número esperado de figurinhas que é preciso comprar para completar um álbum de N figurinhas é $N \sum_{k=1}^N \frac{1}{k}$ (o “problema do colecionador de cupons”). Divida por 5 para obter o número esperado de pacotes e compare com a média simulada no Exemplo 8.
- (d) Forneça um intervalo de confiança para o número médio de pacotes obtido na simulação. O valor teórico do item (c) cai dentro dele?

Exercício 13. (Desafio) Considere

$$\theta = \int_0^1 x^{-3/4} dx = 4.$$

- (a) Usando $U \sim \text{Unif}(0, 1)$ e $g(u) = u^{-3/4}$, mostre que $\mathbb{E}[g(U)] = 4 < \infty$, mas $\mathbb{E}[g(U)^2] = \infty$. Conclua que o estimador de Monte Carlo é não viesado e consistente, mas tem variância infinita.
- (b) Faça o gráfico da média corrente para $B = 10^5$. Ela converge? Como são as oscilações, comparadas às do Exemplo 1?
- (c) Construa 1000 intervalos de confiança de 95%, cada um com $B = 1000$, e calcule a proporção que contém $\theta = 4$. Ela é próxima de 95%? Explique o que deu errado.
- (d) Agora escolha $f(x) = \frac{1}{4}x^{-3/4}$ em $(0, 1)$ como densidade geradora (verifique que ela integra 1). Mostre que, com essa escolha, o estimador $\frac{1}{B} \sum g(X_i)/f(X_i)$ tem variância **zero**. Este é o caso extremo da ideia do Capítulo 11.

Exercício 14. (Desafio) Seja $\theta = \mathbb{P}(Z > 4,5)$, com $Z \sim N(0, 1)$, cujo valor é aproximadamente $3,4 \times 10^{-6}$.

- (a) Para o estimador $\hat{\theta}_B$ baseado na indicadora, mostre que o erro padrão **relativo** é

$$\frac{\text{ep}(\hat{\theta}_B)}{\theta} = \sqrt{\frac{1 - \theta}{\theta B}}.$$

- (b) Quantas simulações são necessárias para que esse erro relativo seja de no máximo 10%? E se θ fosse 10^{-9} ?

- (c) Rode a simulação com $B = 10^5$ algumas vezes (mudando a semente). Com que frequência você obtém $\hat{\theta}_B = 0$? O que o intervalo de confiança devolve nesse caso, e por que ele é enganoso?
- (d) Compare com a situação do Exemplo 4, em que $\theta \approx 0,66$. Por que lá o mesmo B produzia uma estimativa excelente?

Exercício 15. (Desafio) Sejam $X_1, \dots, X_n \stackrel{iid}{\sim} \text{Exp}(\lambda)$.

- (a) Mostre que a esperança de $X_{(k)}$ (a k -ésima estatística de ordem) é $\sum_{j=1}^k \frac{1}{\lambda(n-j+1)}$.

Dica: pense nos espaçamentos $Y_i = X_{(i)} - X_{(i-1)}$, com $X_{(0)} = 0$. Mostre que $\min(X_1, \dots, X_n) \sim \text{Exp}(n\lambda)$ e, depois, use a falta de memória: depois que o mínimo “sai”, restam $n-1$ exponenciais i.i.d. $\text{Exp}(\lambda)$. Conclua que $Y_i \sim \text{Exp}((n-i+1)\lambda)$ e, portanto, $\mathbb{E}[Y_i] = \frac{1}{(n-i+1)\lambda}$. Finalmente, note que $X_{(k)} = \sum_{i=1}^k Y_i$ e some as esperanças.

- (b) Estime $\mathbb{E}[X_{(5)}]$ para $(n, k, \lambda) = (50, 5, 2)$ por Monte Carlo e compare com o valor teórico. Forneça um intervalo de confiança.
- (c) Use a mesma simulação para estimar $\mathbb{E}[X_{(50)}]$, isto é, a esperança do **máximo**. Compare com a fórmula do item (a) e observe qual dos espaçamentos contribui mais para o total.

122 Técnicas de Redução de Variância

O capítulo anterior terminou com um diagnóstico incômodo: o erro padrão do estimador de Monte Carlo é $\sigma/\sqrt{B}$, e o $\sqrt{B}$ no denominador é implacável — para ganhar uma casa decimal é preciso multiplicar o número de simulações por 100. A taxa $1/\sqrt{B}$ é inerente ao método e não há como melhorá-la. Resta atacar o outro fator: σ .

É isso que fazem as técnicas deste capítulo. Todas seguem o mesmo roteiro: dado $\theta = \mathbb{E}[g(X)]$, construir um estimador **diferente** do usual, que continue não viesado — isto é, que continue estimando o mesmo θ — mas com variância menor. Como o número de simulações não muda, o ganho sai de graça em tempo de máquina; o preço é pago em outra moeda: cada técnica exige que saibamos algo a mais sobre o problema. Uma simetria, uma quantidade parecida com $g(X)$ cuja esperança já conhecemos, ou uma esperança condicional que sabemos calcular no papel.

Veremos quatro técnicas:

- **variáveis antitéticas**, que geram os valores aos pares, de modo que os erros de um se cancelem com os do outro;
- **variáveis de controle**, que corrigem a estimativa usando uma quantidade auxiliar de média conhecida;
- **condicionamento**, que substitui parte da simulação por uma conta feita no papel;
- **amostragem estratificada**, que espalha os pontos de propósito, em vez de deixá-los inteiramente ao acaso.

O capítulo termina com uma ideia de natureza um pouco diferente, os **números aleatórios comuns**: ela não reduz a variância de uma estimativa isolada, e sim a da **comparação** entre duas. A amostragem por importância, a última das técnicas clássicas, é o assunto do Capítulo 11.

122.1 Medindo o ganho

Antes de reduzir a variância, precisamos combinar como comparar dois estimadores.

i Definição: fator de redução de variância

Sejam $\hat{\theta}$ o estimador de Monte Carlo usual e $\tilde{\theta}$ um estimador alternativo, ambos não viesados para θ e ambos baseados no mesmo número B de avaliações de g . O **fator de redução de variância** de $\tilde{\theta}$ é

$$\frac{\text{Var}(\tilde{\theta})}{\text{Var}(\hat{\theta})},$$

e a **redução percentual** é 1 menos esse fator.

Um fator de 0,05 (redução de 95%) significa que o estimador novo tem variância vinte vezes menor. Como a variância do estimador usual cai com $1/B$, isso quer dizer que $\hat{\theta}$ precisaria de **vinte vezes mais simulações** para atingir a mesma precisão — é assim que o ganho deve ser lido.

A Atenção: compare a custo igual

Comparar variâncias só faz sentido se os dois estimadores custarem o mesmo. O erro mais comum é comparar um estimador antitético que usa $2n$ avaliações de g com um estimador usual que usa apenas n : metade do “ganho” observado seria obtida simplesmente por simular mais.

Neste capítulo, todas as comparações fixam o **número total de avaliações de g** , denotado por B . Quando as técnicas têm custos por avaliação muito diferentes (por exemplo, quando uma delas exige calcular uma Φ a cada passo), o certo é comparar $1/(\text{variância} \times \text{tempo})$, quantidade conhecida como **eficiência** do estimador.

i Como as variâncias são estimadas neste capítulo

Para exibir o ganho, precisamos das variâncias dos estimadores. Elas aparecem adiante de duas formas:

- **repetindo a estimação inteira** R vezes e calculando a variância amostral das R estimativas obtidas. É caro, mas é a forma mais direta de *ilustrar* o ganho;
- **a partir de uma única rodada**, com o erro padrão $\hat{\sigma}/\sqrt{B}$ do Capítulo 9, em que $\hat{\sigma}$ é o desvio padrão amostral das B parcelas cuja média estamos tomando. É o que se faz na prática.

122.2 Técnica 1: Variáveis Antitéticas

Queremos estimar $\theta = \mathbb{E}[X]$, e o estimador de Monte Carlo usual, com $X_1, \dots, X_B$ i.i.d.,

$$\hat{\theta} = \frac{1}{B} \sum_{i=1}^B X_i, \quad \text{Var}(\hat{\theta}) = \frac{\text{Var}(X)}{B},$$

trata todos os valores gerados como independentes. A ideia das **variáveis antitéticas** é abrir mão dessa independência de propósito: geramos os valores **aos pares**, de modo que, quando o primeiro elemento do par cai acima de θ , o segundo tenda a cair abaixo. A média do par fica mais próxima de θ do que cada elemento isoladamente, e é essa compensação que reduz a variância.

! Proposição: variância do estimador antitético

Sejam $(X_1, Y_1), \dots, (X_n, Y_n)$ pares independentes entre si, em que X_i e Y_i têm a mesma distribuição, com média θ , variância σ^2 e correlação $\rho = \text{Corr}(X_i, Y_i)$. O **estimador antitético**

$$\hat{\theta}_A = \frac{1}{2n} \sum_{i=1}^n (X_i + Y_i)$$

satisfaz $\mathbb{E}[\hat{\theta}_A] = \theta$ e, escrevendo $B = 2n$ para o número total de valores gerados,

$$\text{Var}(\hat{\theta}_A) = \frac{\sigma^2(1 + \rho)}{B}.$$

i Demonstração

Como cada X_i e cada Y_i tem média θ , a linearidade da esperança dá

$$\mathbb{E}[\hat{\theta}_A] = \frac{1}{2n} \sum_{i=1}^n (\mathbb{E}[X_i] + \mathbb{E}[Y_i]) = \frac{2n\theta}{2n} = \theta.$$

Para a variância, os n pares são independentes entre si, então a variância da soma é a soma das variâncias:

$$\text{Var}(\hat{\theta}_A) = \frac{1}{4n^2} \sum_{i=1}^n \text{Var}(X_i + Y_i) = \frac{1}{4n^2} \sum_{i=1}^n (\text{Var}(X_i) + \text{Var}(Y_i) + 2\text{Cov}(X_i, Y_i)).$$

Cada parcela vale $2\sigma^2 + 2\rho\sigma^2$, de modo que

$$\text{Var}(\hat{\theta}_A) = \frac{n \cdot 2\sigma^2(1+\rho)}{4n^2} = \frac{\sigma^2(1+\rho)}{2n} = \frac{\sigma^2(1+\rho)}{B}. \quad \square$$

Compare com o estimador usual baseado nas mesmas B observações, cuja variância é σ^2/B : o fator de redução de variância é exatamente

$$\frac{\text{Var}(\hat{\theta}_A)}{\text{Var}(\hat{\theta})} = 1 + \rho.$$

Ou seja, tudo depende do sinal de ρ . Se $\rho < 0$, ganhamos; se $\rho = 0$ (pares independentes), estamos de volta ao Monte Carlo usual; e se $\rho > 0$, **perdemos**. Em particular, quando $\rho = 1$ o estimador antitético é tão ruim quanto o estimador usual com metade dos pontos.

122.2.1 Como gerar pares negativamente correlacionados

Falta a parte prática: como construir Y_i com a mesma distribuição de X_i e correlação negativa com ele? A receita usa o método da inversão do Capítulo 4. Suponha que $X = h(U)$, com $U \sim \text{Unif}(0,1)$ — o que sempre é possível, tomando $h = F^{-1}$, ainda que nem sempre seja simples.

A observação-chave é que $1 - U$ também tem distribuição $\text{Unif}(0,1)$. Portanto $h(1 - U)$ tem exatamente a mesma distribuição de X , e em particular a mesma média θ — o estimador continua não viesado. Além disso, U grande corresponde a $1 - U$ pequeno, e a monotonicidade de h transmite essa oposição para os valores gerados:

! Proposição: monotonicidade garante correlação negativa

Se h é uma função monótona (não decrescente ou não crescente) e $U \sim \text{Unif}(0,1)$, então

$$\text{Cov}(h(U), h(1 - U)) \leq 0.$$

i Demonstração

Suponha h não decrescente (o caso não crescente é análogo) e defina $f(u) = h(u)$ e $\ell(u) = h(1 - u)$. Então f é não decrescente e ℓ é não crescente, de modo que, para quaisquer u e u' , os fatores de

$$(f(u) - f(u'))(\ell(u) - \ell(u'))$$

têm sinais opostos (ou algum deles é zero): esse produto é sempre ≤ 0 .

Sejam agora U e U' independentes, ambas $\text{Unif}(0,1)$. Tomando a esperança do produto

acima com $u = U$ e $u' = U'$, e usando que U e U' são independentes e de mesma distribuição,

$$0 \geq \mathbb{E}[(f(U) - f(U'))(\ell(U) - \ell(U'))] = 2\mathbb{E}[f(U)\ell(U)] - 2\mathbb{E}[f(U)]\mathbb{E}[\ell(U)] = 2\text{Cov}(f(U), \ell(U)),$$

o que conclui a demonstração. $\square$

Pseudo-algoritmo: variáveis antitéticas

Para estimar $\theta = \mathbb{E}[g(X)]$, com $X = h(U)$ e $U \sim \text{Unif}(0, 1)$, usando $B = 2n$ avaliações de g :

1. Gere $U_1, \dots, U_n \sim \text{Unif}(0, 1)$ independentes.
2. Para cada i , defina $X_i = h(U_i)$ e $Y_i = h(1 - U_i)$.
3. Calcule

$$\hat{\theta}_A = \frac{1}{2n} \sum_{i=1}^n (g(X_i) + g(Y_i)).$$

Note que o passo 1 gera apenas n uniformes para produzir $2n$ valores: além de reduzir a variância, o método economiza metade dos números aleatórios.

Atenção: a monotonicidade é de $g \circ h$, não de h

Quando estimamos $\mathbb{E}[g(X)]$, os valores que entram na média são $g(h(U))$ e $g(h(1 - U))$. A proposição acima se aplica à composição $g \circ h$, e é ela que precisa ser monótona. Se $g \circ h$ for simétrica em torno de $u = 1/2$ — como acontece com $g(x) = x^2$ e $h(u) = 2u - 1$ —, teremos $\rho = 1$ e o método **dobra** a variância a custo igual, em vez de reduzi-la (Exercício 2).

122.2.2 Exemplo 1: Aproximação de uma integral

Queremos estimar

$$\theta = \int_0^\infty \log(1 + x^2) e^{-x} dx = \mathbb{E}[\log(1 + X^2)], \quad X \sim \text{Exp}(1).$$

Pelo método da inversão, $X = -\log(U)$ tem distribuição $\text{Exp}(1)$ quando $U \sim \text{Unif}(0, 1)$; logo $h(u) = -\log(u)$, e o par antitético de $X = -\log(U)$ é $Y = -\log(1 - U)$. Como h é decrescente

e $g(x) = \log(1 + x^2)$ é crescente em $x > 0$, a composição $g \circ h$ é monótona e a proposição garante correlação negativa.

123 R

```
set.seed(42)
B <- 20000 # número total de avaliações de g, para os dois métodos

# --- Monte Carlo usual: B valores independentes
x_mc <- rexp(B, rate = 1)
theta_mc <- mean(log(1 + x_mc^2))

# --- Antitético: n = B/2 uniformes geram B valores, aos pares
u <- runif(B / 2)
x <- -log(u)      # X = h(U) ~ Exp(1)
y <- -log(1 - u)  # Y = h(1 - U) ~ Exp(1), o par antitético de X

g_x <- log(1 + x^2)
g_y <- log(1 + y^2)
theta_anti <- mean((g_x + g_y) / 2)

cat("Monte Carlo usual:  ", theta_mc, "\n")
```

Monte Carlo usual: 0.6959782

```
cat("Antitético:      ", theta_anti, "\n")
```

Antitético: 0.6890659

```
cat("Correlação do par:  ", cor(g_x, g_y), "\n")
```

Correlação do par: -0.666576

```
# Valor de referência, obtido por integração numérica
integral <- integrate(function(x) log(1 + x^2) * exp(-x), 0, Inf)
cat("Valor de referência:", integral$value, "\n")
```

Valor de referência: 0.6867559

124 Python

```
import numpy as np
from scipy.integrate import quad

np.random.seed(42)
B = 20000 # número total de avaliações de g, para os dois métodos

# --- Monte Carlo usual: B valores independentes
x_mc = np.random.exponential(scale=1.0, size=B)
theta_mc = np.mean(np.log(1 + x_mc**2))

# --- Antitético: n = B/2 uniformes geram B valores, aos pares
u = np.random.uniform(0, 1, B // 2)
x = -np.log(u) # X = h(U) ~ Exp(1)
y = -np.log(1 - u) # Y = h(1 - U) ~ Exp(1), o par antitético de X

g_x = np.log(1 + x**2)
g_y = np.log(1 + y**2)
theta_anti = np.mean((g_x + g_y) / 2)

print("Monte Carlo usual: ", theta_mc)
```

Monte Carlo usual: 0.685042173881501

```
print("Antitético: ", theta_anti)
```

Antitético: 0.6824565876024888

```
print("Correlação do par: ", np.corrcoef(g_x, g_y)[0, 1])
```

Correlação do par: -0.6622699303881481

```
# Valor de referência, obtido por integração numérica
integral, _ = quad(lambda x: np.log(1 + x**2) * np.exp(-x), 0, np.inf)
print("Valor de referência:", integral)
```

Valor de referência: 0.6867559231128539

As duas estimativas estão próximas do valor de referência, como deveriam: as duas são não viesadas. O que muda é a **dispersão** delas. Para enxergá-la, repetimos todo o processo R vezes e olhamos a variância das estimativas obtidas:

125 R

```
set.seed(42)
B <- 2000 # avaliações de g em cada estimativa
R <- 2000 # número de repetições, apenas para estimar as variâncias

theta_mc <- numeric(R)
theta_anti <- numeric(R)

for (r in 1:R) {
  # Estimativa usual, com B valores independentes
  theta_mc[r] <- mean(log(1 + rexp(B)^2))

  # Estimativa antitética, com B/2 pares (também B avaliações de g)
  u <- runif(B / 2)
  g_x <- log(1 + (-log(u))^2)
  g_y <- log(1 + (-log(1 - u))^2)
  theta_anti[r] <- mean((g_x + g_y) / 2)
}

cat("Variância do estimador usual: ", var(theta_mc), "\n")
```

Variância do estimador usual: 0.000293334

```
cat("Variância do antitético:      ", var(theta_anti), "\n")
```

Variância do antitético: 9.608894e-05

```
cat("Fator de redução:              ", var(theta_anti) / var(theta_mc), "\n")
```

Fator de redução: 0.3275752

126 Python

```
import numpy as np

np.random.seed(42)
B = 2000 # avaliações de g em cada estimativa
R = 2000 # número de repetições, apenas para estimar as variâncias

theta_mc = np.zeros(R)
theta_anti = np.zeros(R)

for r in range(R):
    # Estimativa usual, com B valores independentes
    theta_mc[r] = np.mean(np.log(1 + np.random.exponential(1.0, B)**2))

    # Estimativa antitética, com B/2 pares (também B avaliações de g)
    u = np.random.uniform(0, 1, B // 2)
    g_x = np.log(1 + (-np.log(u))**2)
    g_y = np.log(1 + (-np.log(1 - u))**2)
    theta_anti[r] = np.mean((g_x + g_y) / 2)

print("Variância do estimador usual: ", np.var(theta_mc, ddof=1))
```

Variância do estimador usual: 0.000300386628580812

```
print("Variância do antitético: ", np.var(theta_anti, ddof=1))
```

Variância do antitético: 0.00010025745787497008

```
print("Fator de redução: ",
      np.var(theta_anti, ddof=1) / np.var(theta_mc, ddof=1))
```

Fator de redução: 0.3337613872782561

O fator de redução observado é próximo de $1 + \rho$, com o ρ estimado no bloco anterior — exatamente o que a proposição prevê.

126.1 Técnica 2: Variáveis de Controle

A segunda técnica parte de uma pergunta diferente: e se, além da quantidade que queremos estimar, soubéssemos simular uma outra quantidade **parecida com ela** e cuja média fosse conhecida?

Concretamente, queremos estimar $\theta = \mathbb{E}[X]$ e dispomos de uma v.a. Y , simulada junto com X , cuja média $\mathbb{E}[Y] = \mu$ **conhecemos**. Chamamos Y de **variável de controle**. A cada simulação, observamos o erro que o controle cometeu, $Y - \mu$, e usamos essa informação para corrigir X :

i Definição: estimador com variável de controle

Seja Y uma v.a. com $\mathbb{E}[Y] = \mu$ conhecida. Para uma constante c , defina

$$Z = X + c(Y - \mu).$$

Como $\mathbb{E}[Y - \mu] = 0$, temos $\mathbb{E}[Z] = \mathbb{E}[X] = \theta$ para **qualquer** c : a média de B cópias independentes de Z é um estimador não viesado de θ .

A ideia é intuitiva: se X e Y variam juntos e nesta simulação Y saiu maior que sua média, é razoável suspeitar que X também tenha saído grande demais — e então descontamos um pouco. Resta escolher **quanto** descontar.

! Proposição: constante ótima

A variância de $Z = X + c(Y - \mu)$ é

$$\text{Var}(Z) = \text{Var}(X) + c^2 \text{Var}(Y) + 2c \text{Cov}(X, Y),$$

minimizada em

$$c^* = -\frac{\text{Cov}(X, Y)}{\text{Var}(Y)},$$

e, para essa escolha,

$$\text{Var}(Z) = \text{Var}(X) - \frac{[\text{Cov}(X, Y)]^2}{\text{Var}(Y)} = \text{Var}(X) (1 - [\text{Corr}(X, Y)]^2).$$

i Demonstração

A expressão da variância é a fórmula usual de $\text{Var}(A+B)$ aplicada a $A = X$ e $B = c(Y - \mu)$, lembrando que constantes não alteram variâncias nem covariâncias. Como função de c ,

ela é uma parábola com concavidade para cima (o coeficiente de c^2 é $\text{Var}(Y) > 0$), de modo que o mínimo é o ponto em que a derivada se anula:

$$\frac{d}{dc} \text{Var}(Z) = 2c \text{Var}(Y) + 2 \text{Cov}(X, Y) = 0 \quad \Leftrightarrow \quad c = -\frac{\text{Cov}(X, Y)}{\text{Var}(Y)}.$$

Substituindo c^* na expressão da variância:

$$\text{Var}(Z) = \text{Var}(X) + \frac{[\text{Cov}(X, Y)]^2}{\text{Var}(Y)} - 2 \frac{[\text{Cov}(X, Y)]^2}{\text{Var}(Y)} = \text{Var}(X) - \frac{[\text{Cov}(X, Y)]^2}{\text{Var}(Y)}.$$

A última igualdade do enunciado segue de $\text{Cov}(X, Y) = \text{Corr}(X, Y) \sqrt{\text{Var}(X) \text{Var}(Y)}$. $\square$

O fator de redução de variância é, portanto,

$$\frac{\text{Var}(Z)}{\text{Var}(X)} = 1 - [\text{Corr}(X, Y)]^2.$$

Duas leituras importantes. Primeiro, com c^* ótimo **nunca se perde**: o fator é sempre ≤ 1 , e o pior caso (Y não correlacionada com X) apenas devolve o estimador usual. Segundo, o que importa é a correlação **em módulo**: um controle fortemente negativo é tão bom quanto um fortemente positivo, porque c^* ajusta o sinal automaticamente. Um controle com $|\text{Corr}| = 0,9$ reduz a variância em 81%; com 0,99, em 98%.

126.1.1 Estimando c^* na prática

Em geral não conhecemos $\text{Cov}(X, Y)$ nem $\text{Var}(Y)$ — se conhecêssemos, provavelmente saberíamos também θ . A saída é estimá-los com as próprias B simulações:

$$\hat{c}^* = -\frac{\widehat{\text{Cov}}(X, Y)}{\widehat{\text{Var}}(Y)}, \quad \widehat{\text{Cov}}(X, Y) = \frac{1}{B-1} \sum_{i=1}^B (X_i - \bar{X})(Y_i - \bar{Y}), \quad \widehat{\text{Var}}(Y) = \frac{1}{B-1} \sum_{i=1}^B (Y_i - \bar{Y})^2.$$

 Pseudo-algoritmo: variável de controle

1. Gere os pares $(X_1, Y_1), \dots, (X_B, Y_B)$ independentes, em que Y tem média μ conhecida.
2. Estime a constante ótima:

$$\hat{c}^* = -\frac{\widehat{\text{Cov}}(X, Y)}{\widehat{\text{Var}}(Y)}.$$

3. Calcule $Z_i = X_i + \hat{c}^*(Y_i - \mu)$ e devolva

$$\hat{\theta}_C = \frac{1}{B} \sum_{i=1}^B Z_i,$$

com erro padrão $\hat{\sigma}_Z/\sqrt{B}$, em que $\hat{\sigma}_Z$ é o desvio padrão amostral dos Z_i .

i Variável de controle é regressão linear

Repare que $\hat{c}^*$ é (a menos do sinal) exatamente o coeficiente angular da reta de regressão de X contra Y . E $Z_i = X_i + \hat{c}^*(Y_i - \mu)$ é o resíduo dessa reta, deslocado por uma constante.

É por isso que o método funciona: a regressão remove de X a parte que pode ser **explicada linearmente** por Y , e sobra apenas o que Y não sabia prever. O fator $1 - [\text{Corr}(X, Y)]^2$ é o velho conhecido $1 - R^2$.

⚠ Atenção: estimar c^* introduz um pequeno viés

Como $\hat{c}^*$ é calculado com a mesma amostra que fornece os X_i , o produto $\hat{c}^*(Y_i - \mu)$ não tem média exatamente zero, e o estimador deixa de ser rigorosamente não viesado. O viés é da ordem de $1/B$, desprezível diante do erro padrão (que é da ordem de $1/\sqrt{B}$) quando B é grande.

Se você quiser eliminá-lo de vez, estime c^* em uma simulação **piloto** separada e use esse valor, fixo, na simulação principal (Exercício 6).

126.1.2 Exemplo 2: Estimando $\mathbb{E}[e^U]$

Considere $X = e^U$, com $U \sim \text{Unif}(0, 1)$, cuja média é $\theta = e - 1 \approx 1,7183$. A escolha natural de controle é a própria uniforme, $Y = U$, com $\mu = \mathbb{E}[U] = 1/2$ e $\text{Var}(U) = 1/12$ conhecidas — e a correlação é alta porque e^u é quase uma reta no intervalo $(0, 1)$.

Para saber o que esperar, façamos as contas. Integrando por partes com $u = x$ e $dv = e^x dx$, temos $\int_0^1 x e^x dx = [x e^x - e^x]_0^1 = 1$, de modo que

$$\text{Cov}(e^U, U) = \mathbb{E}[U e^U] - \mathbb{E}[U] \mathbb{E}[e^U] = 1 - \frac{e-1}{2} \approx 0,14086.$$

Logo $c^* = -\text{Cov}(e^U, U)/\text{Var}(U) = -12 \times 0,14086 \approx -1,690$ e, usando $\text{Var}(e^U) = \frac{e^2-1}{2} - (e-1)^2 \approx 0,2420$,

$$\text{Var}(Z) = 0,2420 - 12 \times (0,14086)^2 \approx 0,00394, \quad \frac{\text{Var}(Z)}{\text{Var}(e^U)} \approx 0,0163.$$

Ou seja, esperamos uma redução de cerca de 98,4% — o estimador usual precisaria de mais de sessenta vezes mais simulações para igualar essa precisão.

127 R

```
set.seed(42)
B <- 10000

u <- runif(B)
x <- exp(u) # o que queremos promediar
y <- u      # variável de controle, com média mu conhecida
mu <- 0.5

# Constante ótima estimada com a própria amostra
c_hat <- -cov(x, y) / var(y)
z <- x + c_hat * (y - mu)

cat("c* teórico: ", -12 * (1 - (exp(1) - 1) / 2), "\n")
```

c* teórico: -1.690309

```
cat("c* estimado:", c_hat, "\n\n")
```

c* estimado: -1.687179

```
cat("Monte Carlo usual: ", mean(x), " (EP:", sd(x) / sqrt(B), ")\n")
```

Monte Carlo usual: 1.717407 (EP: 0.004945457)

```
cat("Com controle: ", mean(z), " (EP:", sd(z) / sqrt(B), ")\n")
```

Com controle: 1.719195 (EP: 0.000630611)

```
cat("Valor exato: ", exp(1) - 1, "\n\n")
```

Valor exato: 1.718282

```
cat("Correlação entre X e Y:", cor(x, y), "\n")
```

Correlação entre X e Y: 0.9918369

```
cat("Fator de redução:      ", var(z) / var(x), "\n")
```

Fator de redução: 0.01625961

128 Python

```
import numpy as np

np.random.seed(42)
B = 10000

u = np.random.uniform(0, 1, B)
x = np.exp(u)    # o que queremos promediar
y = u            # variável de controle, com média mu conhecida
mu = 0.5

# Constante ótima estimada com a própria amostra
c_hat = -np.cov(x, y, ddof=1)[0, 1] / np.var(y, ddof=1)
z = x + c_hat * (y - mu)

print("c* teórico: ", -12 * (1 - (np.e - 1) / 2))
```

c* teórico: -1.6903090292457295

```
print("c* estimado:", c_hat, "\n")
```

c* estimado: -1.6864493521453634

```
print("Monte Carlo usual: ", np.mean(x),
      " (EP:", np.std(x, ddof=1) / np.sqrt(B), ")")
```

Monte Carlo usual: 1.707932345121554 (EP: 0.004890981710407133)

```
print("Com controle:      ", np.mean(z),
      " (EP:", np.std(z, ddof=1) / np.sqrt(B), ")")
```

Com controle: 1.7177819552811089 (EP: 0.0006261456835013693)

```
print("Valor exato:      ", np.e - 1, "\n")
```

Valor exato: 1.718281828459045

```
print("Correlação entre X e Y:", np.corrcoef(x, y)[0, 1])
```

Correlação entre X e Y: 0.9917715282123958

```
print("Fator de redução:      ", np.var(z, ddof=1) / np.var(x, ddof=1))
```

Fator de redução: 0.016389235827249004

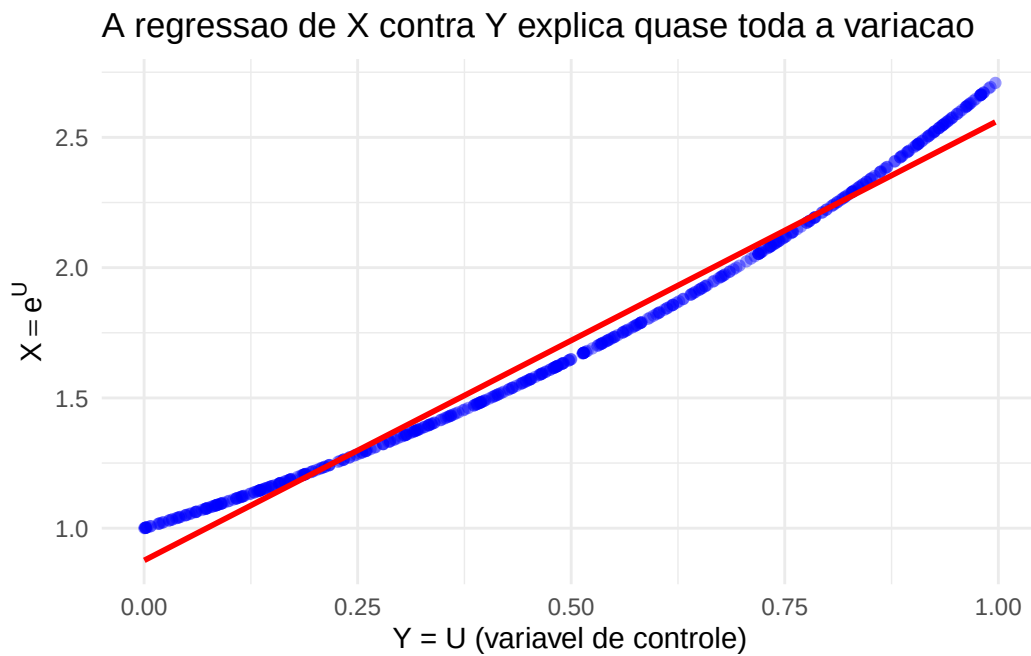
O gráfico abaixo mostra o que aconteceu. Cada ponto é um par (U_i, e^{U_i}) , e a reta é a regressão de X contra Y : como os pontos quase caem sobre ela, os resíduos — que é o que o estimador com controle promedia — são muito menos dispersos que os próprios e^{U_i} .

129 R

```
library(ggplot2)

# Usa apenas os 500 primeiros pontos, para o gráfico não ficar carregado
dados <- data.frame(y = y[1:500], x = x[1:500])

ggplot(dados, aes(x = y, y = x)) +
  geom_point(alpha = 0.4, color = "blue") +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(x = "Y = U (variavel de controle)", y = expression(X == e^U),
       title = "A regressao de X contra Y explica quase toda a variacao") +
  theme_minimal()
```



130 Python

```
import matplotlib.pyplot as plt

# Usa apenas os 500 primeiros pontos, para o gráfico não ficar carregado
y_plot, x_plot = y[:500], x[:500]

# Coeficientes da reta de regressão de X contra Y
inclinacao, intercepto = np.polyfit(y_plot, x_plot, 1)
grade = np.linspace(0, 1, 100)

plt.scatter(y_plot, x_plot, alpha=0.4, color='blue')
```

<matplotlib.collections.PathCollection object at 0x706e53a874d0>

```
plt.plot(grade, intercepto + inclinacao * grade, color='red')
```

[<matplotlib.lines.Line2D object at 0x706e53ae1d10>]

```
plt.xlabel('Y = U (variavel de controle)')
```

Text(0.5, 0, 'Y = U (variavel de controle)')

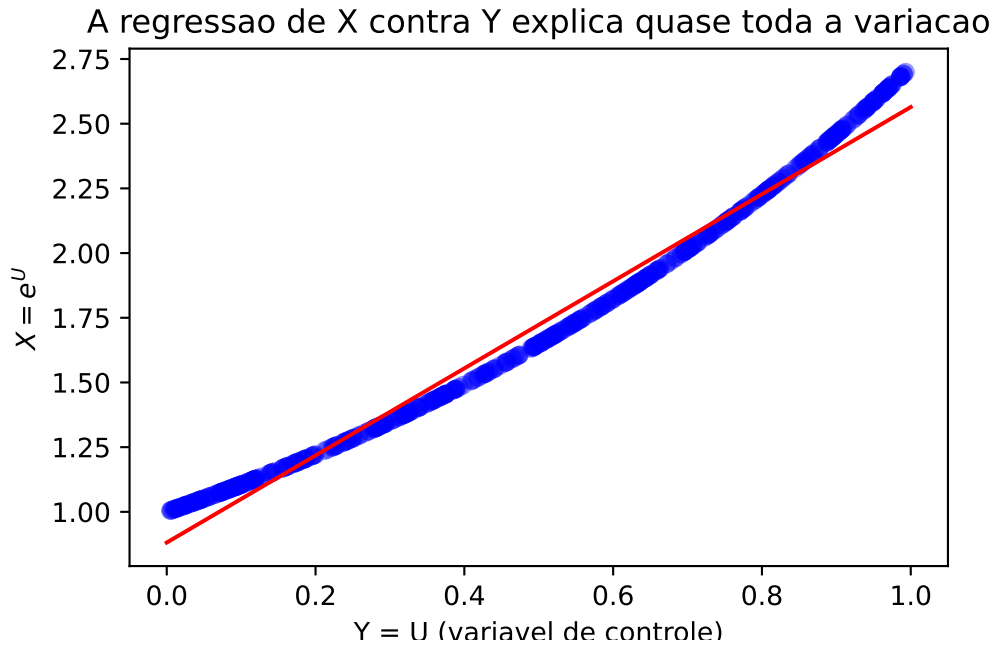
```
plt.ylabel(r'$X = e^U$')
```

Text(0, 0.5, '\$X = e^U\$')

```
plt.title('A regressao de X contra Y explica quase toda a variacao')
```

Text(0.5, 1.0, 'A regressao de X contra Y explica quase toda a variacao')

```
plt.show()
```



130.0.1 Exemplo 3: Um controle vindo do próprio integrando

No exemplo anterior o controle era a própria uniforme, mas a receita geral é outra, e mais poderosa: **substitua a parte difícil do integrando por uma aproximação cuja integral você saiba calcular**. Essa aproximação vira o controle.

Queremos estimar

$$\theta = \int_0^1 \frac{e^{-x}}{1+x^2} dx = \mathbb{E} \left[\frac{e^{-U}}{1+U^2} \right], \quad U \sim \text{Unif}(0, 1),$$

que não tem primitiva elementar. Mas o fator e^{-x} varia pouco em $(0, 1)$ (entre 1 e 0,37); trocando-o por uma constante, sobra algo que sabemos integrar:

$$\mathbb{E} \left[\frac{1}{1+U^2} \right] = \int_0^1 \frac{dx}{1+x^2} = \arctan(1) = \frac{\pi}{4}.$$

Tomamos então $Y = 1/(1+U^2)$ como variável de controle, com $\mu = \pi/4$. Note que não precisamos nos preocupar com a constante multiplicativa que descartamos: qualquer múltiplo de Y leva ao mesmo estimador, porque c^* se ajusta na mesma proporção.

131 R

```
set.seed(42)
B <- 10000

u <- runif(B)
x <- exp(-u) / (1 + u^2) # integrando
y <- 1 / (1 + u^2)       # controle: o integrando sem o fator e^{-x}
mu <- pi / 4             # E[Y], calculada analiticamente

c_hat <- -cov(x, y) / var(y)
z <- x + c_hat * (y - mu)

cat("Monte Carlo usual:", mean(x), " (EP:", sd(x) / sqrt(B), ")\n")
```

Monte Carlo usual: 0.5261481 (EP: 0.002471774)

```
cat("Com controle:      ", mean(z), " (EP:", sd(z) / sqrt(B), ")\n")
```

Com controle: 0.5258419 (EP: 0.0005616811)

```
cat("Integração numérica:",
    integrate(function(x) exp(-x) / (1 + x^2), 0, 1)$value, "\n\n")
```

Integração numérica: 0.5247971

```
cat("Correlação entre X e Y:", cor(x, y), "\n")
```

Correlação entre X e Y: 0.9738392

```
cat("Fator de redução:      ", var(z) / var(x), "\n")
```

Fator de redução: 0.05163715

132 Python

```
import numpy as np
from scipy.integrate import quad

np.random.seed(42)
B = 10000

u = np.random.uniform(0, 1, B)
x = np.exp(-u) / (1 + u**2)    # integrando
y = 1 / (1 + u**2)            # controle: o integrando sem o fator e^{-x}
mu = np.pi / 4                # E[Y], calculada analiticamente

c_hat = -np.cov(x, y, ddof=1)[0, 1] / np.var(y, ddof=1)
z = x + c_hat * (y - mu)

print("Monte Carlo usual:", np.mean(x),
      " (EP:", np.std(x, ddof=1) / np.sqrt(B), ")")
```

Monte Carlo usual: 0.529447122022146 (EP: 0.0024472445327840906)

```
print("Com controle:      ", np.mean(z),
      " (EP:", np.std(z, ddof=1) / np.sqrt(B), ")")
```

Com controle: 0.5244625395727385 (EP: 0.000559710964957539)

```
print("Integração numérica:", quad(lambda x: np.exp(-x) / (1 + x**2), 0, 1)[0], "\n")
```

Integração numérica: 0.5247971432602705

```
print("Correlação entre X e Y:", np.corrcoef(x, y)[0, 1])
```

Correlação entre X e Y: 0.9734944397145877

```
print("Fator de redução: ", np.var(z, ddof=1) / np.var(x, ddof=1))
```

Fator de redução: 0.052308575844781255

132.1 Técnica 3: Condicionamento

A terceira técnica é, em certo sentido, a mais radical: em vez de melhorar a forma de combinar os valores simulados, ela **simula menos**. A ideia é que toda parte do problema que soubermos resolver analiticamente não precisa ser sorteada — e tudo o que não é sorteado não contribui com variabilidade.

Queremos estimar $\theta = \mathbb{E}[X]$. Suponha que exista uma v.a. Y , que sabemos simular, tal que a esperança condicional

$$g(y) := \mathbb{E}[X \mid Y = y]$$

tenha fórmula fechada. Então, em vez de gerar X , geramos apenas Y e usamos $g(Y)$ no lugar de X .

! Proposição: o estimador condicionado é melhor

Sejam $Y_1, \dots, Y_B$ i.i.d. com a distribuição de Y e $g(y) = \mathbb{E}[X \mid Y = y]$. O estimador

$$\hat{\theta}_{\text{cond}} = \frac{1}{B} \sum_{i=1}^B g(Y_i)$$

é não viesado para θ e satisfaz

$$\text{Var}(\hat{\theta}_{\text{cond}}) \leq \text{Var}(\hat{\theta}),$$

em que $\hat{\theta}$ é o estimador de Monte Carlo usual com as mesmas B simulações.

i Demonstração

Pela lei da esperança total, $\mathbb{E}[g(Y_i)] = \mathbb{E}[\mathbb{E}[X \mid Y]] = \mathbb{E}[X] = \theta$, o que dá

$$\mathbb{E}[\hat{\theta}_{\text{cond}}] = \frac{1}{B} \sum_{i=1}^B \mathbb{E}[g(Y_i)] = \theta.$$

Para a variância, usamos a fórmula da variância condicional (também conhecida como lei da variância total):

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]).$$

Como o primeiro termo do lado direito é uma esperança de uma variância, ele é ≥ 0 , e portanto

$$\text{Var}(g(Y)) = \text{Var}(\mathbb{E}[X | Y]) \leq \text{Var}(X).$$

Dividindo os dois lados por B , obtemos $\text{Var}(\hat{\theta}_{\text{cond}}) \leq \text{Var}(\hat{\theta})$. $\square$

A demonstração diz mais do que o enunciado: a redução de variância é **exatamente** $\mathbb{E}[\text{Var}(X | Y)]$, isto é, toda a variabilidade que sobraria em X depois de conhecido Y . Ela some porque essa parte do sorteio deixou de ser feita.

Rao-Blackwellização

Trocar um estimador por sua esperança condicional é a mesma operação do **Teorema de Rao-Blackwell**, visto em cursos de inferência: condicionar nunca piora e costuma melhorar. Por isso a técnica também é chamada de *Rao-Blackwellização* do estimador de Monte Carlo.

Pseudo-algoritmo: condicionamento

1. Escolha Y tal que $g(y) = \mathbb{E}[X | Y = y]$ seja conhecida em forma fechada.
2. Gere $Y_1, \dots, Y_B$ independentes.
3. Devolva

$$\hat{\theta}_{\text{cond}} = \frac{1}{B} \sum_{i=1}^B g(Y_i),$$

com erro padrão $\hat{\sigma}_g / \sqrt{B}$, em que $\hat{\sigma}_g$ é o desvio padrão amostral dos $g(Y_i)$.

Atenção: é preciso saber calcular $\mathbb{E}[X | Y]$

A técnica não custa nada em variância, mas cobra caro em álgebra: se $\mathbb{E}[X | Y]$ tiver que ser estimada por simulação, não sobrou ganho algum. Na prática, ela se aplica quando o modelo é construído em etapas — sorteia Y , depois sorteia X dado Y — e a segunda etapa é uma distribuição conhecida, cuja média ou probabilidade acumulada temos em uma função pronta.

132.1.1 Exemplo 4: Estimando π

Considere o algoritmo do Capítulo 9 para estimar π : geramos pontos uniformes no quadrado $[-1, 1]^2$ e contamos a proporção que cai no círculo de raio 1. Em símbolos, com $X, Y \sim \text{Unif}(-1, 1)$ independentes,

$$Z = I(X^2 + Y^2 \leq 1), \quad \mathbb{E}[Z] = \frac{\pi}{4}.$$

Aqui há dois sorteios, e podemos dispensar o segundo. Fixado $X = x$, o ponto cai no círculo se $|Y| \leq \sqrt{1 - x^2}$, e como $Y \sim \text{Unif}(-1, 1)$ tem densidade $1/2$,

$$\mathbb{E}[Z \mid X = x] = \mathbb{P}(X^2 + Y^2 \leq 1 \mid X = x) = \int_{-\sqrt{1-x^2}}^{\sqrt{1-x^2}} \frac{1}{2} dy = \sqrt{1 - x^2}.$$

O estimador condicionado troca a indicadora (que só sabe dizer “dentro” ou “fora”) pela **fração exata** da faixa vertical que está dentro do círculo:

$$\hat{\pi}_{\text{cond}} = 4 \cdot \frac{1}{B} \sum_{i=1}^B \sqrt{1 - X_i^2}.$$

133 R

```
set.seed(42)

# Estima pi e o erro padrão - sem condicionamento
estimate_pi_simple <- function(B) {
  u1 <- runif(B, -1, 1)
  u2 <- runif(B, -1, 1)
  z <- ifelse(u1^2 + u2^2 <= 1, 1, 0)
  p_hat <- mean(z)
  se <- 4 * sqrt(p_hat * (1 - p_hat) / B) # EP por binomial
  list(estimate = 4 * p_hat, se = se)
}

# Estima pi e o erro padrão - com condicionamento em X
estimate_pi_conditioning <- function(B) {
  x <- runif(B, -1, 1)
  g <- sqrt(1 - x^2) # E[Z | X]
  se <- 4 * sd(g) / sqrt(B) # EP via amostra de g
  list(estimate = 4 * mean(g), se = se)
}

B <- 10000

res_simple <- estimate_pi_simple(B)
res_cond <- estimate_pi_conditioning(B)

cat("Sem condicionamento: est =", res_simple$estimate, "| EP =", res_simple$se, "\n")
```

Sem condicionamento: est = 3.1272 | EP = 0.01652096

```
cat("Com condicionamento: est =", res_cond$estimate, "| EP =", res_cond$se, "\n")
```

Com condicionamento: est = 3.130729 | EP = 0.009053532

```
cat("Fator de redução:", (res_cond$se / res_simple$se)^2, "\n")
```

Fator de redução: 0.3003071

134 Python

```
import numpy as np

np.random.seed(42)

# Estima pi e o erro padrão - sem condicionamento
def estimate_pi_simple(B):
    u1 = np.random.uniform(-1, 1, B)
    u2 = np.random.uniform(-1, 1, B)
    z = (u1**2 + u2**2 <= 1).astype(int)
    p_hat = z.mean()
    se = 4 * np.sqrt(p_hat * (1 - p_hat) / B) # EP por binomial
    return 4 * p_hat, se

# Estima pi e o erro padrão - com condicionamento em X
def estimate_pi_conditioning(B):
    x = np.random.uniform(-1, 1, B)
    g = np.sqrt(1 - x**2) # E[Z | X]
    se = 4 * g.std(ddof=1) / np.sqrt(B) # EP via amostra de g
    return 4 * g.mean(), se

B = 10000

pi_simple, se_simple = estimate_pi_simple(B)
pi_cond, se_cond = estimate_pi_conditioning(B)

print("Sem condicionamento: est =", pi_simple, "| EP =", se_simple)
```

Sem condicionamento: est = 3.1348 | EP = 0.016468846225525333

```
print("Com condicionamento: est =", pi_cond, " | EP =", se_cond)
```

Com condicionamento: est = 3.1540094743355818 | EP = 0.008885501290884038


```
print("Fator de redução:", (se_cond / se_simple)**2)
```

Fator de redução: 0.2910968592795422

As contas confirmam o que se observa. Para a indicadora, $\text{Var}(Z) = p(1-p) \approx 0,1685$ com $p = \pi/4$; já para a versão condicionada,

$$\text{Var}(\sqrt{1-X^2}) = \mathbb{E}[1-X^2] - \left(\frac{\pi}{4}\right)^2 = \frac{2}{3} - \frac{\pi^2}{16} \approx 0,0498,$$

um fator de redução de cerca de 0,30. E note o bônus: o estimador condicionado usa **uma** uniforme por simulação, em vez de duas.

134.0.1 Exemplo 5: Estimando $\mathbb{P}(X > 1)$

Seja $Y \sim \text{Exp}(1)$ e suponha que $X | Y = y \sim N(y, 4)$. Queremos estimar $\theta = \mathbb{P}(X > 1)$. Este é o caso típico descrito acima: o modelo já vem em duas etapas, e a segunda é uma normal, cuja acumulada temos pronta.

Escrevendo $\theta = \mathbb{E}[Z]$ com $Z = I(X > 1)$, condicionamos em Y :

$$\mathbb{E}[Z | Y = y] = \mathbb{P}(X > 1 | Y = y) = \mathbb{P}\left(\frac{X-y}{2} > \frac{1-y}{2} \mid Y = y\right) = 1 - \Phi\left(\frac{1-y}{2}\right),$$

em que Φ é a f.d.a. da $N(0, 1)$. O estimador condicionado é

$$\hat{\theta}_{\text{cond}} = \frac{1}{B} \sum_{i=1}^B \left[1 - \Phi\left(\frac{1-Y_i}{2}\right) \right], \quad Y_i \sim \text{Exp}(1).$$

135 R

```
set.seed(42)

# Estima  $P(X > 1)$  e o EP - sem condicionamento (indicadora)
estimate_prob_simple <- function(B) {
  y <- rexp(B, rate = 1)          #  $Y \sim \text{Exp}(1)$ 
  x <- rnorm(B, mean = y, sd = 2)  #  $X | Y \sim N(Y, 4)$ 
  z <- ifelse(x > 1, 1, 0)
  p_hat <- mean(z)
  list(estimate = p_hat, se = sqrt(p_hat * (1 - p_hat) / B))
}

# Estima  $P(X > 1)$  e o EP - com condicionamento em Y
estimate_prob_conditioning <- function(B) {
  y <- rexp(B, rate = 1)
  g <- 1 - pnorm((1 - y) / 2)      #  $E[Z | Y]$ 
  list(estimate = mean(g), se = sd(g) / sqrt(B))
}

B <- 100000

res_simple <- estimate_prob_simple(B)
res_cond   <- estimate_prob_conditioning(B)

cat("Sem condicionamento: est =", res_simple$estimate, "| EP =", res_simple$se, "\n")
```

Sem condicionamento: est = 0.4905 | EP = 0.001580853

```
cat("Com condicionamento: est =", res_cond$estimate, "| EP =", res_cond$se, "\n")
```

Com condicionamento: est = 0.4903664 | EP = 0.0005183444

```
cat("Fator de redução:", (res_cond$se / res_simple$se)^2, "\n")
```

Fator de redução: 0.1075112

136 Python

```
import numpy as np
from scipy.stats import norm

np.random.seed(42)

# Estima  $P(X > 1)$  e o EP - sem condicionamento (indicadora)
def estimate_prob_simple(B):
    y = np.random.exponential(1, B)      #  $Y \sim \text{Exp}(1)$ 
    x = np.random.normal(y, 2, B)        #  $X | Y \sim N(Y, 4)$ 
    z = (x > 1).astype(int)
    p_hat = z.mean()
    return p_hat, np.sqrt(p_hat * (1 - p_hat) / B)

# Estima  $P(X > 1)$  e o EP - com condicionamento em Y
def estimate_prob_conditioning(B):
    y = np.random.exponential(1, B)
    g = 1 - norm.cdf((1 - y) / 2)        #  $E[Z | Y]$ 
    return g.mean(), g.std(ddof=1) / np.sqrt(B)

B = 100000

prob_simple, se_simple = estimate_prob_simple(B)
prob_cond, se_cond = estimate_prob_conditioning(B)

print("Sem condicionamento: est =", prob_simple, "| EP =", se_simple)
```

Sem condicionamento: est = 0.48977 | EP = 0.0015808078539152062

```
print("Com condicionamento: est =", prob_cond, "| EP =", se_cond)
```

Com condicionamento: est = 0.4900730050240222 | EP = 0.0005167915603236426

```
print("Fator de redução:", (se_cond / se_simple)**2)
```

Fator de redução: 0.10687414548573845

136.1 Técnica 4: Amostragem Estratificada

As três técnicas anteriores mexem no que fazemos **com** os pontos sorteados. A última mexe em **onde** eles caem.

O problema do sorteio puro é que ele é desigual: em B uniformes independentes, nada impede que sobrem pontos perto de 0 e faltem pontos perto de 1. Essa irregularidade é pura variabilidade desperdiçada. A **amostragem estratificada** elimina boa parte dela impondo, por construção, quantos pontos caem em cada pedaço do intervalo.

Queremos estimar $\theta = \mathbb{E}[g(U)]$, com $U \sim \text{Unif}(0, 1)$. Dividimos $(0, 1)$ em k intervalos de mesmo comprimento, chamados **estratos**,

$$I_j = \left(\frac{j-1}{k}, \frac{j}{k} \right], \quad j = 1, \dots, k,$$

e observamos que, como $\mathbb{P}(U \in I_j) = 1/k$ para todo j , a lei da esperança total dá

$$\theta = \sum_{j=1}^k \mathbb{P}(U \in I_j) \mathbb{E}[g(U) \mid U \in I_j] = \frac{1}{k} \sum_{j=1}^k \theta_j, \quad \theta_j := \mathbb{E}[g(U) \mid U \in I_j].$$

Cada θ_j é estimado separadamente, com m pontos sorteados dentro do estrato j — o que é fácil, porque uma uniforme em I_j se obtém de uma $\text{Unif}(0, 1)$ com a transformação $u \mapsto (j-1 + u)/k$.

! Proposição: a estratificação nunca piora

Sejam $U_{j1}, \dots, U_{jm}$ uniformes independentes no estrato I_j , para $j = 1, \dots, k$, num total de $B = km$ pontos. O estimador estratificado

$$\hat{\theta}_{\text{est}} = \frac{1}{k} \sum_{j=1}^k \left(\frac{1}{m} \sum_{i=1}^m g(U_{ji}) \right)$$

é não viesado para θ e tem variância

$$\text{Var}(\hat{\theta}_{\text{est}}) = \frac{\sigma^2 - \tau^2}{B}, \quad \tau^2 := \frac{1}{k} \sum_{j=1}^k (\theta_j - \theta)^2,$$

em que $\sigma^2 = \text{Var}(g(U))$. Como $\tau^2 \geq 0$, tem-se sempre $\text{Var}(\hat{\theta}_{\text{est}}) \leq \sigma^2/B$.

Demonstração

Cada média interna estima θ_j sem viés, e a média das k delas é θ pela identidade acima. Escrevendo $\sigma_j^2 = \text{Var}(g(U) \mid U \in I_j)$ e usando a independência de todos os pontos,

$$\text{Var}(\hat{\theta}_{\text{est}}) = \frac{1}{k^2} \sum_{j=1}^k \frac{\sigma_j^2}{m} = \frac{1}{km} \left(\frac{1}{k} \sum_{j=1}^k \sigma_j^2 \right).$$

Seja agora J o índice do estrato em que cai uma uniforme, isto é, $J = j$ quando $U \in I_j$; note que J é uniforme em $\{1, \dots, k\}$. A lei da variância total aplicada a $g(U)$, condicionando em J , dá

$$\sigma^2 = \mathbb{E}[\text{Var}(g(U) \mid J)] + \text{Var}(\mathbb{E}[g(U) \mid J]) = \frac{1}{k} \sum_{j=1}^k \sigma_j^2 + \tau^2.$$

Substituindo na expressão anterior e usando $B = km$:

$$\text{Var}(\hat{\theta}_{\text{est}}) = \frac{\sigma^2 - \tau^2}{B}. \quad \square$$

A demonstração revela o parentesco entre esta técnica e a anterior: estratificar é condicionar no estrato J . O que desaparece da variância, τ^2 , é justamente a variabilidade **entre** estratos — aquela que se deve apenas ao fato de U ter caído mais para a esquerda ou mais para a direita. Sobra só a variabilidade **dentro** de cada estrato, que é pequena quando os estratos são estreitos.

Pseudo-algoritmo: amostragem estratificada

Para estimar $\theta = \mathbb{E}[g(U)]$ com $B = km$ avaliações de g :

1. Escolha o número de estratos k e o número de pontos por estrato m .
2. Para $j = 1, \dots, k$ e $i = 1, \dots, m$: gere $V_{ji} \sim \text{Unif}(0, 1)$ e faça

$$U_{ji} = \frac{j - 1 + V_{ji}}{k},$$

que é uniforme no estrato I_j .

3. Devolva

$$\hat{\theta}_{\text{est}} = \frac{1}{k} \sum_{j=1}^k \frac{1}{m} \sum_{i=1}^m g(U_{ji}).$$

i Estratificando uma variável qualquer

O pseudo-algoritmo está escrito para $\theta = \mathbb{E}[g(U)]$, com U uniforme, mas serve para qualquer v.a. contínua X com f.d.a. F : basta aplicar o método da inversão do Capítulo 4 aos uniformes já estratificados,

$$X_{ji} = F^{-1}(U_{ji}).$$

Como U_{ji} é uniforme em $(\frac{j-1}{k}, \frac{j}{k}]$, o valor X_{ji} cai entre os quantis $F^{-1}(\frac{j-1}{k})$ e $F^{-1}(\frac{j}{k})$ de X . Ou seja, os estratos deixam de ser faixas de mesmo **comprimento** e passam a ser faixas de mesma **probabilidade** $1/k$ — que é exatamente o que a demonstração acima usa, e por isso a proposição continua valendo palavra por palavra.

Para estratificar uma $N(0, 1)$ em 10 estratos, por exemplo, basta trocar `(estrato + v)/k` por `qnorm((estrato + v)/k)` em R, ou por `scipy.stats.norm.ppf((estrato + v)/k)` em Python.

i Quanto se ganha, e por quê o ganho é tão grande

Se g for suave, dentro de um estrato de largura $1/k$ ela é quase uma reta, e $\sigma_j^2 \approx g'(x_j)^2/(12k^2)$. Somando sobre os estratos,

$$\text{Var}(\hat{\theta}_{\text{est}}) \approx \frac{1}{12 k^2 B} \int_0^1 g'(x)^2 dx,$$

isto é, o ganho cresce com k^2 : dobrar o número de estratos divide a variância por quatro, sem simular um ponto a mais. Se ainda por cima fizemos m pequeno e k crescer junto com B (por exemplo, um ponto por estrato, $k = B$), a variância cai como B^{-3} e o erro padrão como $B^{-3/2}$, muito mais rápido que o $B^{-1/2}$ do Monte Carlo usual.

Nada disso é de graça, porém. Em dimensão d , estratificar cada eixo em k pedaços exige k^d estratos: a técnica sofre da maldição da dimensionalidade discutida no Capítulo 9, e é sobretudo útil em dimensão baixa. Uma saída parcial é o **hipercubo latino**, que estratifica cada eixo separadamente em vez de estratificar o cubo inteiro, e por isso mantém B estratos por eixo qualquer que seja d (Exercício 19).

⚠ Atenção: com um ponto por estrato não há erro padrão

Com $m = 1$ não é possível estimar σ_j^2 dentro de cada estrato, e o estimador fica sem barra de erro. Na prática, usa-se $m \geq 2$ e o erro padrão

$$\widehat{\text{ep}} = \frac{1}{k} \sqrt{\sum_{j=1}^k \frac{\hat{\sigma}_j^2}{m}},$$

com $\hat{\sigma}_j^2$ a variância amostral dentro do estrato j . Usar a fórmula usual $\hat{\sigma}/\sqrt{B}$ sobre todos os pontos juntos **superestima** o erro, porque ela ignora que os pontos não foram sorteados de forma independente.

136.1.1 Exemplo 6: Estratificando $\mathbb{E}[e^U]$

Voltemos a $\theta = \mathbb{E}[e^U] = e - 1$, agora com $k = 10$ estratos e $m = 100$ pontos em cada um ($B = 1000$ avaliações, como no Monte Carlo usual com que comparamos).

137 R

```
set.seed(42)
k <- 10      # número de estratos
m <- 100     # pontos por estrato
B <- k * m

# Ponto i do estrato j: desloca uma Unif(0,1) para dentro de I_j
v <- runif(B)
estrato <- rep(0:(k - 1), each = m)
u <- (estrato + v) / k

# Cada coluna da matriz reúne os m pontos de um mesmo estrato
valores <- matrix(exp(u), nrow = m, ncol = k)
medias <- colMeans(valores)
variancias <- apply(valores, 2, var)

theta_est <- mean(medias)
ep_est <- sqrt(sum(variancias / m)) / k

# Monte Carlo usual, com o mesmo número de avaliações
x <- exp(runif(B))

cat("Monte Carlo usual: ", mean(x), " (EP:", sd(x) / sqrt(B), ")\n")
```

Monte Carlo usual: 1.716613 (EP: 0.01587679)

```
cat("Estratificado:      ", theta_est, " (EP:", ep_est, ")\n")
```

Estratificado: 1.715901 (EP: 0.001651776)

```
cat("Valor exato:        ", exp(1) - 1, "\n")
```

Valor exato: 1.718282

```
cat("Fator de redução: ", (ep_est / (sd(x) / sqrt(B)))^2, "\n")
```

Fator de redução: 0.01082373

138 Python

```
import numpy as np

np.random.seed(42)
k = 10      # número de estratos
m = 100     # pontos por estrato
B = k * m

# Ponto i do estrato j: desloca uma Unif(0,1) para dentro de I_j
v = np.random.uniform(0, 1, B)
estrato = np.repeat(np.arange(k), m)
u = (estrato + v) / k

# Cada linha da matriz reúne os m pontos de um mesmo estrato
valores = np.exp(u).reshape(k, m)
medias = valores.mean(axis=1)
variancias = valores.var(axis=1, ddof=1)

theta_est = medias.mean()
ep_est = np.sqrt((variancias / m).sum()) / k

# Monte Carlo usual, com o mesmo número de avaliações
x = np.exp(np.random.uniform(0, 1, B))

print("Monte Carlo usual: ", np.mean(x),
      " (EP:", np.std(x, ddof=1) / np.sqrt(B), ")")
```

Monte Carlo usual: 1.7316256444846885 (EP: 0.01572800685558926)

```
print("Estratificado:      ", theta_est, " (EP:", ep_est, ")")
```

Estratificado: 1.7162017557712563 (EP: 0.0016291034225254023)

```
print("Valor exato:      ", np.e - 1)
```

Valor exato: 1.718281828459045

```
print("Fator de redução:  ", (ep_est / (np.std(x, ddof=1) / np.sqrt(B)))**2)
```

Fator de redução: 0.010728769936899486

138.1 Colocando as técnicas lado a lado

Para comparar as quatro técnicas de forma justa, aplicamos todas ao **mesmo** problema — $\theta = \mathbb{E}[e^U] = e - 1$ — com o mesmo número de avaliações de g , e estimamos a variância de cada estimador repetindo a estimação R vezes.

139 R

```
set.seed(42)
B <- 1000 # avaliações de g em cada estimativa
R <- 2000 # repetições, apenas para estimar as variâncias

mc_simples <- function(B) {
  mean(exp(runif(B)))
}

antitetica <- function(B) {
  u <- runif(B / 2)
  mean(c(exp(u), exp(1 - u)))
}

com_controle <- function(B) {
  u <- runif(B)
  x <- exp(u)
  c_hat <- -cov(x, u) / var(u)
  mean(x + c_hat * (u - 0.5))
}

estratificada <- function(B, k) {
  v <- runif(B)
  estrato <- rep(0:(k - 1), each = B / k)
  mean(exp((estrato + v) / k))
}

# Repete cada estimador R vezes. replicate(R, expr) avalia expr R vezes
# e devolve o vetor dos R resultados
estimativas <- list(
  "Monte Carlo simples" = replicate(R, mc_simples(B)),
  "Antitética"          = replicate(R, antitetica(B)),
  "Variável de controle" = replicate(R, com_controle(B)),
  "Estratificada (k = 10)" = replicate(R, estratificada(B, 10)),
  "Estratificada (k = 100)" = replicate(R, estratificada(B, 100))
)
```

```

)

variancias <- sapply(estimativas, var)
referencia <- variancias["Monte Carlo simples"]

for (metodo in names(variancias)) {
  # Completa o nome com espaços até 24 caracteres, para alinhar as colunas
  rotulo <- paste0(metodo, strrep(" ", 24 - nchar(metodo)))
  cat(rotulo, sprintf(" var = %.3e   fator = %.4f\n",
                      variancias[metodo], variancias[metodo] / referencia),
      sep = "")
}

```

Monte Carlo simples	var = 2.300e-04	fator = 1.0000
Antitética	var = 7.963e-06	fator = 0.0346
Variável de controle	var = 3.773e-06	fator = 0.0164
Estratificada (k = 10)	var = 2.716e-06	fator = 0.0118
Estratificada (k = 100)	var = 2.648e-08	fator = 0.0001

140 Python

```
import numpy as np

np.random.seed(42)
B = 1000 # avaliações de g em cada estimativa
R = 2000 # repetições, apenas para estimar as variâncias

def mc_simples(B):
    return np.mean(np.exp(np.random.uniform(0, 1, B)))

def antitetica(B):
    u = np.random.uniform(0, 1, B // 2)
    return np.mean(np.concatenate((np.exp(u), np.exp(1 - u))))

def com_controle(B):
    u = np.random.uniform(0, 1, B)
    x = np.exp(u)
    c_hat = -np.cov(x, u, ddof=1)[0, 1] / np.var(u, ddof=1)
    return np.mean(x + c_hat * (u - 0.5))

def estratificada(B, k):
    v = np.random.uniform(0, 1, B)
    estrato = np.repeat(np.arange(k), B // k)
    return np.mean(np.exp((estrato + v) / k))

estimativas = {
    "Monte Carlo simples": [mc_simples(B) for _ in range(R)],
    "Antitética": [antitetica(B) for _ in range(R)],
    "Variável de controle": [com_controle(B) for _ in range(R)],
    "Estratificada (k = 10)": [estratificada(B, 10) for _ in range(R)],
    "Estratificada (k = 100)": [estratificada(B, 100) for _ in range(R)],
}

variancias = {nome: np.var(v, ddof=1) for nome, v in estimativas.items()}
referencia = variancias["Monte Carlo simples"]
```

```
for metodo, variancia in variancias.items():
    print(f"{metodo:<24s} var = {variancia:.3e}    fator = {variancia/referencia:.4f}")
```

Monte Carlo simples	var = 2.384e-04	fator = 1.0000
Antitética	var = 7.764e-06	fator = 0.0326
Variável de controle	var = 4.018e-06	fator = 0.0169
Estratificada (k = 10)	var = 2.553e-06	fator = 0.0107
Estratificada (k = 100)	var = 2.660e-08	fator = 0.0001

Três lições ficam desta tabela.

Primeiro, **todas as técnicas ganham**, e ganham muito: os fatores de redução são da ordem de 10^{-2} ou menos, ou seja, o Monte Carlo usual precisaria de dezenas a milhares de vezes mais simulações para empatar.

Segundo, **elas ganham por motivos diferentes**. A antitética e o controle se aproveitam de e^u ser quase uma reta em $(0, 1)$; a estratificação se aproveita de e^u ser suave. Em um problema em que g oscilasse muito, as três perderiam boa parte da vantagem — e a comparação poderia se inverter.

Terceiro, **nenhuma delas é universal**. A antitética exige monotonicidade, o controle exige uma variável auxiliar de média conhecida, o condicionamento exige uma esperança condicional em forma fechada e a estratificação exige dimensão baixa. Vale a pena tentar mais de uma — e, muitas vezes, combiná-las (Exercício 13).

140.1 Números aleatórios comuns

As quatro técnicas anteriores respondem sempre à mesma pergunta: *quanto vale θ* ? Mas muitas vezes a pergunta é outra — *qual das duas alternativas é melhor*? Dois algoritmos, dois desenhos de experimento, duas políticas de estoque, o mesmo modelo com dois valores de um parâmetro. Aí o alvo não é uma esperança, e sim uma **diferença**:

$$\Delta = \theta_1 - \theta_2.$$

O estimador natural é $\hat{\Delta} = \hat{\theta}_1 - \hat{\theta}_2$, e sua variância é

$$\text{Var}(\hat{\Delta}) = \text{Var}(\hat{\theta}_1) + \text{Var}(\hat{\theta}_2) - 2 \text{Cov}(\hat{\theta}_1, \hat{\theta}_2).$$

Se rodarmos as duas simulações de forma independente — o que acontece por padrão, se simplesmente executarmos dois programas —, a covariância é zero e as duas variâncias se

somam. Mas nada nos obriga a isso. Podemos alimentar as duas simulações com **os mesmos números aleatórios**: se os dois sistemas reagem de forma parecida aos mesmos sorteios, a covariância fica positiva e a variância da diferença cai.

! Proposição: variância da diferença

Sejam $\hat{\theta}_1$ e $\hat{\theta}_2$ estimadores não viesados de θ_1 e θ_2 , ambos com variância σ^2/B e correlação $\rho = \text{Corr}(\hat{\theta}_1, \hat{\theta}_2)$. Então $\hat{\Delta} = \hat{\theta}_1 - \hat{\theta}_2$ é não viesado para Δ e

$$\text{Var}(\hat{\Delta}) = \frac{2\sigma^2(1 - \rho)}{B}.$$

Em particular, o fator de redução em relação a duas simulações independentes é $1 - \rho$.

i Demonstração

A esperança sai da linearidade: $\mathbb{E}[\hat{\Delta}] = \theta_1 - \theta_2 = \Delta$. Para a variância,

$$\text{Var}(\hat{\theta}_1 - \hat{\theta}_2) = \frac{\sigma^2}{B} + \frac{\sigma^2}{B} - 2\rho \frac{\sigma^2}{B} = \frac{2\sigma^2(1 - \rho)}{B},$$

usando $\text{Cov}(\hat{\theta}_1, \hat{\theta}_2) = \rho \sigma^2/B$. Com simulações independentes, $\rho = 0$ e a variância é $2\sigma^2/B$; a razão entre as duas é $1 - \rho$. $\square$

i É a antitética com o sinal trocado

Vale a pena colocar as duas técnicas lado a lado, porque a ideia é a mesma — induzir correlação de propósito — e só o sinal desejado muda:

Técnica	O que promediamos	Queremos	Fator
Variáveis antitéticas	uma soma , $X_i + Y_i$	$\rho < 0$	$1 + \rho$
Números aleatórios comuns	uma diferença , $\hat{\theta}_1 - \hat{\theta}_2$	$\rho > 0$	$1 - \rho$

Na antitética, alimentamos o segundo valor com $1 - U$ para forçar oposição; aqui alimentamos o segundo sistema com o próprio U para forçar semelhança.

💡 Pseudo-algoritmo: números aleatórios comuns

Para estimar $\Delta = \theta_1 - \theta_2$ com B avaliações de cada sistema:

1. Gere $U_1, \dots, U_B \sim \text{Unif}(0, 1)$ **uma única vez**.

2. Para cada i , calcule a diferença **pareada**

$$D_i = g_1(h_1(U_i)) - g_2(h_2(U_i)),$$

usando o mesmo U_i nos dois sistemas.

3. Devolva

$$\hat{\Delta} = \frac{1}{B} \sum_{i=1}^B D_i,$$

com erro padrão $\hat{\sigma}_D/\sqrt{B}$, em que $\hat{\sigma}_D$ é o desvio padrão amostral dos D_i .

⚠ Atenção: o erro padrão vem das diferenças pareadas

O passo 3 é onde o método é mais fácil de estragar. Se você calcular os erros padrão dos dois sistemas separadamente e combiná-los como $\sqrt{\widehat{\text{ep}}_1^2 + \widehat{\text{ep}}_2^2}$, jogará fora exatamente o ganho obtido: essa fórmula pressupõe independência, que é o que acabamos de destruir de propósito. O erro padrão precisa ser calculado a partir dos D_i .

É a mesma distinção entre o teste t para duas amostras independentes e o teste t **pareado**: quando as observações vêm aos pares, é a variabilidade das diferenças que interessa, não a de cada grupo.

140.1.1 Exemplo 7: qual o efeito de mudar um parâmetro?

Voltemos à integral do Exemplo 1,

$$\theta(\lambda) = \mathbb{E} [\log(1 + X^2)], \quad X \sim \text{Exp}(\lambda),$$

e perguntemos quanto θ muda quando a taxa aumenta 1%, isto é, ao passar de $\lambda = 1$ para $\lambda = 1,01$. Queremos

$$\Delta = \theta(1) - \theta(1,01) \approx 0,0075.$$

Este é o caso difícil, e o mais frequente na prática: a diferença que queremos medir é pequena, muito menor que a variabilidade de cada simulação isolada.

Pelo método da inversão, $X = -\log(U)/\lambda$. Usar números comuns significa usar o mesmo U nos dois valores de λ — e note o que isso produz: $X_2 = X_1 \cdot (\lambda_1/\lambda_2)$. Os dois sistemas enxergam exatamente o mesmo sorteio, apenas reescalado.

141 R

```
set.seed(42)
B <- 10000
lambda1 <- 1.00 # sistema 1
lambda2 <- 1.01 # sistema 2: taxa 1% maior

g <- function(x) log(1 + x^2)

# --- Simulações independentes: cada sistema com seus próprios uniformes
g1 <- g(-log(runif(B)) / lambda1)
g2 <- g(-log(runif(B)) / lambda2)

dif_indep <- mean(g1) - mean(g2)
# Sendo independentes, as variâncias se somam
ep_indep <- sqrt(var(g1) / B + var(g2) / B)

# --- Números aleatórios comuns: os MESMOS uniformes nos dois sistemas
u <- runif(B)
d <- g(-log(u) / lambda1) - g(-log(u) / lambda2) # diferenças pareadas

dif_comum <- mean(d)
ep_comum <- sd(d) / sqrt(B) # erro padrão calculado sobre as diferenças

# Valor de referência, por integração numérica
theta <- function(lambda) {
  integrate(function(x) g(x) * lambda * exp(-lambda * x), 0, Inf)$value
}
cat("Diferença verdadeira:", theta(lambda1) - theta(lambda2), "\n\n")
```

Diferença verdadeira: 0.007505881

```
cat("Independentes:", dif_indep, " (EP:", ep_indep, ")\n")
```

Independentes: 0.007054988 (EP: 0.01093854)

```
cat("  IC de 95%: [", dif_indep - 1.96 * ep_indep, ",",  
    dif_indep + 1.96 * ep_indep, "]\n\n")
```

IC de 95%: [-0.01438455 , 0.02849452]

```
cat("Comuns:      ", dif_comum, " (EP:", ep_comum, ")\n")
```

Comuns: 0.007543918 (EP: 6.18685e-05)

```
cat("  IC de 95%: [", dif_comum - 1.96 * ep_comum, ",",  
    dif_comum + 1.96 * ep_comum, "]\n\n")
```

IC de 95%: [0.007422656 , 0.00766518]

```
cat("Correlação entre os dois sistemas:",  
    cor(g(-log(u) / lambda1), g(-log(u) / lambda2)), "\n")
```

Correlação entre os dois sistemas: 0.999996

```
cat("Fator de redução:", (ep_comum / ep_indep)^2, "\n")
```

Fator de redução: 3.199047e-05

142 Python

```
import numpy as np
from scipy.integrate import quad

np.random.seed(42)
B = 10000
lambda1 = 1.00 # sistema 1
lambda2 = 1.01 # sistema 2: taxa 1% maior

def g(x):
    return np.log(1 + x**2)

# --- Simulações independentes: cada sistema com seus próprios uniformes
g1 = g(-np.log(np.random.uniform(0, 1, B)) / lambda1)
g2 = g(-np.log(np.random.uniform(0, 1, B)) / lambda2)

dif_indep = g1.mean() - g2.mean()
# Sendo independentes, as variâncias se somam
ep_indep = np.sqrt(np.var(g1, ddof=1) / B + np.var(g2, ddof=1) / B)

# --- Números aleatórios comuns: os MESMOS uniformes nos dois sistemas
u = np.random.uniform(0, 1, B)
d = g(-np.log(u) / lambda1) - g(-np.log(u) / lambda2) # diferenças pareadas

dif_comum = d.mean()
ep_comum = np.std(d, ddof=1) / np.sqrt(B) # erro padrão sobre as diferenças

# Valor de referência, por integração numérica
def theta(lam):
    return quad(lambda x: g(x) * lam * np.exp(-lam * x), 0, np.inf)[0]

print("Diferença verdadeira:", theta(lambda1) - theta(lambda2), "\n")
```

Diferença verdadeira: 0.007505881378814472

```
print("Independentes:", dif_indep, " (EP:", ep_indep, ")")
```

Independentes: 0.02777707687024855 (EP: 0.010795428333020782)

```
print(" IC de 95%: [", dif_indep - 1.96 * ep_indep, ",",  
      dif_indep + 1.96 * ep_indep, "]\n")
```

IC de 95%: [0.006618037337527818 , 0.048936116402969285]

```
print("Comuns:      ", dif_comum, " (EP:", ep_comum, ")")
```

Comuns: 0.007489180143101634 (EP: 6.13051592035667e-05)

```
print(" IC de 95%: [", dif_comum - 1.96 * ep_comum, ",",  
      dif_comum + 1.96 * ep_comum, "]\n")
```

IC de 95%: [0.007369022031062643 , 0.007609338255140624]

```
print("Correlação entre os dois sistemas:",  
      np.corrcoef(g(-np.log(u) / lambda1), g(-np.log(u) / lambda2))[0, 1])
```

Correlação entre os dois sistemas: 0.9999960693825236

```
print("Fator de redução:", (ep_comum / ep_indep)**2)
```

Fator de redução: 3.224885443299369e-05

O contraste é brutal. Com simulações independentes, o intervalo de confiança tem largura de cerca de 0,043 — quase seis vezes a diferença que se queria medir. Dependendo da semente, ele chega a conter o zero: com esses números não dá nem para afirmar com segurança qual dos dois valores é maior. Com números comuns, a correlação entre os sistemas passa de 0,999, o fator de redução fica na casa de 10^{-5} , e a margem de erro do intervalo cai de quase 300% da diferença para menos de 2% dela — com exatamente o mesmo esforço computacional.

A explicação é a mesma de sempre: a variabilidade de $g(X)$ é enorme comparada ao efeito de mexer 1% em λ . Rodando as duas simulações separadamente, medimos essa variabilidade duas vezes e tentamos enxergar o efeito por trás dela. Rodando com os mesmos sorteios, ela aparece nos dois lados da subtração e se cancela, sobrando só o que de fato muda quando λ muda.

⚠ Atenção: os números precisam ser usados para a mesma coisa

O ganho depende de os dois sistemas reagirem de forma parecida ao mesmo sorteio, e para isso cada número aleatório precisa desempenhar o **mesmo papel** nos dois. Se um dos sistemas consumir uma quantidade variável de uniformes — o que acontece em qualquer algoritmo de rejeição, dos Capítulos 5 e 6 —, os dois fluxos se desalinham na primeira vez em que o número de tentativas diferir, e a partir dali os sistemas passam a ver sorteios essencialmente independentes. Esse cuidado, chamado de **sincronização**, é mais um argumento a favor da inversão, que consome exatamente um uniforme por valor gerado.

E o ganho não é garantido: se os dois sistemas reagirem em sentidos **opostos** ao mesmo sorteio, teremos $\rho < 0$ e a variância da diferença **aumenta** — o espelho exato do que acontece com as antitéticas quando $g \circ h$ é simétrica (Exercício 14).

142.1 Exercícios

Exercício 1. Implemente a técnica das variáveis antitéticas para estimar $\mathbb{E}[X^2]$, em que $X \sim \text{Unif}(0, \pi)$.

- (a) Escreva X como $h(U)$, com $U \sim \text{Unif}(0, 1)$, e identifique a função $g \circ h$ cuja monotonicidade garante o ganho.
- (b) Estime $\mathbb{E}[X^2]$ pelos dois métodos com $B = 10^4$ avaliações em cada um e compare as estimativas com o valor exato $\pi^2/3$.
- (c) Repita a estimação $R = 1000$ vezes e calcule o fator de redução de variância observado.
- (d) Mostre que $\text{Corr}(U^2, (1 - U)^2) = -7/8$ (use $\mathbb{E}[U^2(1 - U)^2] = 1/30$) e verifique que o fator obtido em (c) está próximo de $1 + \rho$.

Exercício 2. Seja $U \sim \text{Unif}(0, 1)$ e $\theta = \mathbb{E}[\sin(\pi U)]$.

- (a) Mostre que $\sin(\pi(1 - u)) = \sin(\pi u)$ para todo u e conclua que, neste caso, a correlação do par antitético vale exatamente 1.
- (b) Conclua da proposição que o estimador antitético com B avaliações é idêntico ao estimador usual com $B/2$ avaliações. Verifique isso empiricamente, estimando as variâncias com $R = 1000$ repetições.
- (c) A moral é que a simetria de g em torno de $1/2$ é o pior cenário possível para a antitética. Proponha uma **variável de controle** para este problema — por exemplo, $Y = U(1 - U)$, cuja média é $1/6$ — e verifique quanta variância ela reduz.

Exercício 3. Sejam $X = -\log U$ e $Y = -\log(1 - U)$, com $U \sim \text{Unif}(0, 1)$.

- (a) Mostre que X e Y têm ambas distribuição $\text{Exp}(1)$.
- (b) Argumente que $h(u) = -\log u$ é monótona e conclua o sinal de $\text{Corr}(X, Y)$.
- (c) Considere $\theta = \mathbb{E}[e^{-X}]$, cujo valor exato é $1/2$. Mostre que $e^{-X} = U$ e que, portanto, o estimador antitético tem variância **zero** — ele acerta θ exatamente, em qualquer simulação. Confirme rodando o código.
- (d) Estime $\mathbb{E}[X^2] = 2$ pelos dois métodos e compare as variâncias. Por que aqui o ganho é grande, mas não infinito?

Exercício 4. Seja $Z \sim N(0, 1)$. Como $-Z$ também tem distribuição $N(0, 1)$, os pares $(Z_i, -Z_i)$ são a versão antitética natural para a normal.

- (a) Justifique essa afirmação escrevendo $Z = \Phi^{-1}(U)$ e verificando que $\Phi^{-1}(1 - U) = -\Phi^{-1}(U)$.
- (b) Estime $\mathbb{E}[e^Z] = e^{1/2}$ pelos dois métodos e calcule o fator de redução observado.
- (c) Estime $\mathbb{P}(Z > 1,5)$ pelos dois métodos. O ganho é bem menor que em (b): mostre que, neste caso, $\rho = -p/(1 - p)$ com $p = \mathbb{P}(Z > 1,5)$, e conclua que, para eventos raros, a antitética é quase inútil. (Este é um dos problemas que motivam o Capítulo 11.)
- (d) Estime $\mathbb{E}[Z^2] = 1$ pelos dois métodos. O que acontece, e por quê?

Exercício 5. Considere a integral

$$\theta = \int_0^{\pi/4} \int_0^{\pi/4} x^2 y^2 \sin(x + y) \log(x + y) dx dy.$$

- (a) Escreva θ como $(\pi/4)^2 \mathbb{E}[g(X, Y)]$, com X e Y uniformes independentes em $(0, \pi/4)$, e implemente o estimador de Monte Carlo usual.
- (b) Use $X^2 Y^2$ como variável de controle, calculando $\mathbb{E}[X^2 Y^2]$ analiticamente (lembre-se de que X e Y são independentes).
- (c) Estime a correlação entre o integrando e o controle e compare o fator de redução observado com o valor previsto, $1 - \text{Corr}^2$.

Exercício 6. Volte ao Exemplo 2, em que $\theta = \mathbb{E}[e^U] = e - 1$.

- (a) Use como controle a aproximação de Taylor de segunda ordem $Y = 1 + U + U^2/2$, cuja média é $5/3$. Compare o fator de redução com o obtido no Exemplo 2, em que o controle era $Y = U$.
- (b) Explique, com base na interpretação de regressão, por que este controle é melhor.

- (c) Investigue o viés causado por estimar c^* com a própria amostra: com $B = 20$, repita a estimação $R = 10^5$ vezes e compare a média das estimativas com $e - 1$. O viés é detectável? E com $B = 200$?
- (d) Repita (c) estimando c^* em uma simulação piloto separada, com $B = 20$ pontos próprios, e usando esse valor fixo na simulação principal. O viés desaparece?

Exercício 7. Sejam X e Y independentes com distribuição $\text{Exp}(1)$, e $\theta = \mathbb{P}(X + Y > 3)$.

- (a) Mostre que

$$\mathbb{E}[I(X + Y > 3) \mid Y = y] = \begin{cases} e^{-(3-y)}, & \text{se } y < 3, \\ 1, & \text{caso contrário.} \end{cases}$$

- (b) Estime θ com e sem condicionamento, com $B = 10^4$, e compare os erros padrão.
- (c) O valor exato é $\mathbb{P}(W > 3)$ com $W \sim \text{Gama}(2, 1)$, que vale $4e^{-3}$. Verifique se ele cai nos intervalos de confiança dos dois estimadores.

Exercício 8. Seja $N \sim \text{Poisson}(3)$ e, dado $N = n$, sejam $X_1, \dots, X_n$ i.i.d. $\text{Exp}(1)$ independentes de N . Defina a soma aleatória $S = \sum_{i=1}^N X_i$ (com $S = 0$ se $N = 0$).

- (a) Simule S e estime $\mathbb{E}[S]$ por Monte Carlo usual.
- (b) Mostre que $\mathbb{E}[S \mid N] = N$ e use isso para construir o estimador condicionado. Compare as variâncias com o valor teórico: $\text{Var}(S) = 6$ e $\text{Var}(\mathbb{E}[S \mid N]) = 3$.
- (c) Estime agora $\mathbb{P}(S > 5)$ com e sem condicionamento, usando que, dado $N = n \geq 1$, $S \sim \text{Gama}(n, 1)$ (use `pgamma` em R e `scipy.stats.gamma` em Python). Não se esqueça do caso $N = 0$.

Exercício 9. Seja $Y \sim \text{Exp}(1)$ e $X \mid Y = y \sim N(y, 4)$, como no Exemplo 5.

- (a) Reproduza a estimativa de $\mathbb{P}(X > 1)$ pelo método do condicionamento.
- (b) Proponha uma melhoria combinando condicionamento com variáveis antitéticas: gere $Y_i = -\log(U_i)$ e $Y'_i = -\log(1 - U_i)$ e use ambos no estimador condicionado. Compare as variâncias dos dois estimadores.
- (c) A função $y \mapsto 1 - \Phi((1 - y)/2)$ é monótona? O que isso permite prever sobre o resultado de (b)?

Exercício 10. Seja $B \sim \text{Bernoulli}(p)$ e $X \mid B = b \sim N(\mu_b, 1)$, com $\mu_1 = 2$ e $\mu_0 = -1$. Queremos estimar $\theta = \mathbb{P}(X > 1)$.

- (a) Estime θ por Monte Carlo usual, com $p = 0,5$.
- (b) Mostre que $\mathbb{E}[I(X > 1) \mid B = b] = 1 - \Phi(1 - \mu_b)$ e implemente o estimador condicionado.

- (c) Repita para $p \in \{0,1; 0,5; 0,9\}$ e explique por que o ganho do condicionamento depende de p . (Dica: pense em quanto da variabilidade de X vem do sorteio de B e quanto vem do sorteio da normal.)
- (d) Acrescente variáveis antitéticas ao estimador do item (a), gerando o par $B_i = I(U_i \leq p)$ e $B'_i = I(1 - U_i \leq p)$, e o par $Z_i, -Z_i$ para a normal. Compare com os itens anteriores.

Exercício 11. Seja $X = Y + \varepsilon$, com $Y \sim \text{Unif}(0,1)$ e $\varepsilon \sim N(0, \sigma^2)$ independentes, e tome $\sigma = 0,5$.

- (a) Calcule $\text{Var}(X)$, $\mathbb{E}[X | Y]$ e $\text{Var}(\mathbb{E}[X | Y])$.
- (b) Verifique empiricamente a lei da variância total, isto é, que $\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]) = \sigma^2 + 1/12$.
- (c) Explique como essa decomposição justifica a redução de variância por condicionamento, e diga qual das duas parcelas o estimador condicionado elimina.
- (d) Estime $\mathbb{P}(X > 1)$ com e sem condicionamento e verifique que o fator de redução observado é compatível com a decomposição do item (b). Repita com $\sigma = 2$: o ganho aumenta ou diminui? Por quê?

Exercício 12. Considere novamente $\theta = \int_0^1 e^x dx = e - 1$, com $B = 1000$ avaliações fixas.

- (a) Implemente o estimador estratificado para $k \in \{2, 5, 10, 50, 100\}$ e estime a variância de cada um repetindo a estimação $R = 1000$ vezes.
- (b) Faça o gráfico de $\log(\text{variância})$ contra $\log(k)$. A inclinação está próxima de -2 , como prevê a aproximação da seção?
- (c) Compare o caso $k = 2$ com o estimador antitético. Qual dos dois é melhor, e por quê? A partir de qual k a estratificação passa a vencer?
- (d) O que impede, na prática, tomar k tão grande quanto quisermos?

Exercício 13. Volte à integral do Exemplo 3, $\theta = \int_0^1 e^{-x}/(1+x^2) dx$. Compare, com o mesmo número $B = 10^4$ de avaliações e $R = 1000$ repetições, os estimadores:

- (a) Monte Carlo usual;
- (b) antitético;
- (c) com variável de controle $Y = 1/(1 + U^2)$;
- (d) antitético e com variável de controle ao mesmo tempo (aplique o controle sobre as médias dos pares);
- (e) estratificado com $k = 20$.

Monte uma tabela com os fatores de redução e discuta: as técnicas se somam ou uma “rouba” o ganho da outra?

Exercício 14. Sobre os números aleatórios comuns do Exemplo 7.

- (a) Refaça o exemplo com $\lambda_2 = 1,1$, $\lambda_2 = 1,5$ e $\lambda_2 = 3$, anotando em cada caso a correlação entre os sistemas e o fator de redução. Explique por que o ganho é tanto maior quanto **menor** for a diferença entre os dois sistemas.
- (b) A derivada $\theta'(1)$ pode ser aproximada por $[\theta(1+h) - \theta(1)]/h$ para h pequeno. Estime-a com $B = 10^4$ para $h \in \{0,1; 0,01; 0,001\}$, pelos dois esquemas, e faça o gráfico do erro padrão contra h em escala logarítmica nos dois eixos. Mostre que, com simulações independentes, o erro padrão da aproximação cresce como $1/h$ — de modo que diminuir h para reduzir o erro de aproximação **augmenta** o erro de simulação — e explique por que isso não acontece com números comuns.
- (c) Quantos uniformes o método da inversão consome por valor gerado? E o método da rejeição do Capítulo 6? Explique, com base nisso, por que a técnica exigiria cuidado extra se as exponenciais fossem geradas por rejeição.
- (d) Um caso em que a técnica **piora** as coisas. Seja $X \sim \text{Exp}(1)$ e

$$\Delta = \mathbb{E}[X] - \mathbb{E}[e^{-X}] = 1 - \frac{1}{2}.$$

Escrevendo $X = -\log U$, note que o primeiro sistema é decrescente em U e o segundo, $e^{-X} = U$, é crescente. Mostre que $\text{Cov}(-\log U, U) = -1/4$ e conclua que os números comuns **augmentam** a variância da diferença em cerca de 46%. Confirme empiricamente com $B = 10^5$.

Exercício 15. (Desafio) Seja $U \sim \text{Unif}(0, 1)$ e $\theta = \mathbb{E}[U^p]$, com $p \in \{1, 2, 3, 4\}$.

- (a) Calcule θ analiticamente.
- (b) Mostre que $\hat{\theta}_A = \frac{1}{2n} \sum_{i=1}^n (U_i^p + (1 - U_i)^p)$ é não viesado.
- (c) Derive $\text{Var}(\hat{\theta})$ e $\text{Var}(\hat{\theta}_A)$ em função de $\mathbb{E}[U^{2p}]$ e $\mathbb{E}[U^p(1-U)^p]$. (Dica: a segunda esperança é uma integral Beta.)
- (d) Compare as variâncias teóricas com as empíricas e descreva como o ganho da antitética varia com p .

Exercício 16. (Desafio) Seja $g : [0, 1] \rightarrow [0, 1]$ e $\theta = \int_0^1 g(x) dx$. Compare dois estimadores: o de **acerto ou erro**, que gera $U, V \sim \text{Unif}(0, 1)$ independentes e usa $W = I(V \leq g(U))$; e o da **média amostral**, que usa $g(U)$.

- (a) Mostre que $\mathbb{E}[W] = \theta$, de modo que os dois são não viesados.

- (b) Mostre que $\mathbb{E}[W | U] = g(U)$. Conclua que o estimador da média amostral é exatamente o condicionamento do estimador de acerto ou erro e que, portanto, nunca é pior.
- (c) Mostre que $\text{Var}(W) - \text{Var}(g(U)) = \theta - \int_0^1 g(x)^2 dx \geq 0$ e explique por que a última desigualdade vale.
- (d) Aplique a $g(x) = \sqrt{1 - x^2}$ e compare com o Exemplo 4: em que sentido a estimativa de π do Capítulo 9 já era um estimador de acerto ou erro?

Exercício 17. (Desafio) Na amostragem estratificada, nada obriga a colocar o mesmo número de pontos em cada estrato. Suponha B_j pontos no estrato j , com $\sum_j B_j = B$.

- (a) Mostre que a variância do estimador é $\frac{1}{k^2} \sum_{j=1}^k \sigma_j^2 / B_j$.
- (b) Minimize essa expressão sujeita a $\sum_j B_j = B$ e conclua que a alocação ótima é $B_j \propto \sigma_j$ (**alocação de Neyman**). (Dica: multiplicadores de Lagrange, ou a desigualdade de Cauchy-Schwarz.)
- (c) Implemente a alocação de Neyman para $\theta = \int_0^1 e^{5x} dx$ com $k = 10$ e $B = 2000$, estimando os σ_j em uma simulação piloto. Compare com a alocação igual.
- (d) Por que o ganho da alocação ótima é pequeno para $g(x) = e^x$ e grande para $g(x) = e^{5x}$?

Exercício 18. (Desafio) A técnica das variáveis de controle admite q controles simultâneos: dado $\mathbf{Y} = (Y_1, \dots, Y_q)$ com médias conhecidas μ , tomamos $Z = X + \mathbf{c}^\top (\mathbf{Y} - \mu)$.

- (a) Mostre que o vetor ótimo é $\mathbf{c}^* = -\text{Var}(\mathbf{Y})^{-1} \text{Cov}(\mathbf{Y}, X)$ e que, com ele, $\text{Var}(Z) = \text{Var}(X)(1 - R^2)$, em que R^2 é o coeficiente de determinação da regressão de X sobre $\mathbf{Y}$.
- (b) Conclua que $\hat{\mathbf{c}}^*$ pode ser obtido diretamente de uma regressão linear múltipla (1m em R, `np.linalg.lstsq` em Python).
- (c) Estime $\theta = \mathbb{E}[e^U]$ usando os controles U , U^2 e U^3 (cuas médias são 1/2, 1/3 e 1/4). Compare o fator de redução com o do Exemplo 2.
- (d) Acrescentar controles nunca aumenta a variância teórica, mas cada controle a mais é um parâmetro a mais estimado da amostra. Investigue empiricamente, com B pequeno, o que acontece ao acrescentar controles inúteis (por exemplo, uma $\text{Unif}(0, 1)$ independente de U).

Exercício 19. (Desafio) O **hipercubo latino** contorna o problema dos k^d estratos estratificando cada eixo **separadamente**. Em dimensão $d = 2$ e com B pontos, o algoritmo é:

1. Divida cada eixo em B faixas de comprimento $1/B$;
2. Gere duas permutações aleatórias π_1 e π_2 de $(1, \dots, B)$ pelo método do Exemplo 4 do Capítulo 3;

3. Para $i = 1, \dots, B$, gere $V_{i1}, V_{i2} \sim \text{Unif}(0, 1)$ e faça

$$U_{i1} = \frac{\pi_1(i) - 1 + V_{i1}}{B}, \quad U_{i2} = \frac{\pi_2(i) - 1 + V_{i2}}{B};$$

4. Devolva os B pontos (U_{i1}, U_{i2}) .

Cada faixa de cada eixo recebe exatamente um ponto; as permutações é que decidem quais faixas dos dois eixos se encontram.

- (a) Implemente o algoritmo com $B = 100$ e faça o gráfico dos pontos. Verifique que cada uma das 100 faixas de cada eixo contém exatamente um ponto.
- (b) Mostre que U_{i1} é marginalmente $\text{Unif}(0, 1)$ e conclua que o estimador $\frac{1}{B} \sum_i g(U_{i1}, U_{i2})$ continua não viesado.
- (c) Estime $\theta = \int_0^1 \int_0^1 e^{x+y} dx dy = (e - 1)^2$ com $B = 100$ e $R = 1000$ repetições por três métodos: Monte Carlo usual, estratificação em uma grade 10×10 (que também usa 100 pontos) e hipercubo latino. Qual tem a menor variância?
- (d) Repita com $g(x, y) = e^x$, que não depende de y . Agora o hipercubo latino ganha de longe. Explique por quê, contando em quantas faixas **do eixo** x cada um dos dois métodos estratificados divide o intervalo.
- (e) Junte os itens (c) e (d): para que tipo de função o hipercubo latino é a melhor escolha, e para que tipo a grade completa ainda vence? Explique também por que, em dimensão $d = 10$, a grade deixa de ser uma opção e o hipercubo latino não.
- (f) O que aconteceria se tomássemos $\pi_1 = \pi_2 = \text{id}$ identidade, sem sortear as permutações? Faça o gráfico dos pontos nesse caso e explique por que o estimador resultante seria péssimo.

143 Amostragem por Importância

As quatro técnicas do Capítulo 10 são bem diferentes entre si, mas todas têm algo em comum: os valores continuam sendo sorteados da distribuição f do problema. O que muda é o que fazemos **com** eles — combinamos aos pares, corrigimos por uma variável auxiliar, substituímos por uma esperança condicional, espalhamos entre estratos.

A **amostragem por importância** abandona essa restrição. Ela sorteia os valores de uma outra densidade g , escolhida por nós, e desfaz o viés que isso introduziria atribuindo a cada valor um **peso**. Trocar a densidade de onde se sorteia é uma alavanca muito mais poderosa do que qualquer uma das anteriores, e resolve dois problemas que elas não resolvem:

- **eventos raros**: quando a quantidade de interesse depende de uma região que f quase nunca visita, o Monte Carlo usual gasta praticamente todas as simulações onde não há informação alguma;
- **densidades difíceis de simular**: se sabemos calcular $f(x)$ mas não sabemos gerar valores de f , ainda assim podemos estimar $\mathbb{E}[h(X)]$ — basta saber simular de alguma outra densidade.

Como bônus, uma mesma amostra pode ser reaproveitada para estimar esperanças sob várias densidades diferentes, o que é útil em análises de sensibilidade (Exercício 10).

Começamos pelo primeiro problema, que é o mais gritante.

143.1 Quando o Monte Carlo usual falha

143.1.1 Exemplo 1: a cauda da normal

Seja $Z \sim N(0, 1)$ e considere

$$p = \mathbb{P}(Z > 4,5) \approx 3,3977 \times 10^{-6}.$$

Escrevendo $p = \mathbb{E}[h(Z)]$ com $h(z) = I(z > 4,5)$, o estimador de Monte Carlo do Capítulo 9 é simplesmente a proporção de valores simulados que superam 4,5. Vamos aplicá-lo com $B = 100\,000$ simulações.

144 R

```
set.seed(0)
B <- 100000

p_exato <- pnorm(4.5, lower.tail = FALSE)

# Monte Carlo usual: proporção de valores simulados acima de 4,5
z <- rnorm(B)
indicadoras <- 1 * (z > 4.5)

p_mc <- mean(indicadoras)
ep_mc <- sd(indicadoras) / sqrt(B)

cat("Valor exato:      ", p_exato, "\n")
```

Valor exato: 3.397673e-06

```
cat("Monte Carlo usual:  ", p_mc, "\n")
```

Monte Carlo usual: 0

```
cat("Erro padrão:      ", ep_mc, "\n")
```

Erro padrão: 0

```
cat("Valores acima de 4,5:", sum(indicadoras), "de", B, "\n")
```

Valores acima de 4,5: 0 de 1e+05

145 Python

```
import numpy as np
from scipy.stats import norm

np.random.seed(0)
B = 100000

p_exato = norm.sf(4.5) # sf é a função de sobrevivência, igual a 1 - cdf

# Monte Carlo usual: proporção de valores simulados acima de 4,5
z = np.random.normal(0, 1, B)
indicadoras = (z > 4.5).astype(float)

p_mc = indicadoras.mean()
ep_mc = indicadoras.std(ddof=1) / np.sqrt(B)

print("Valor exato:      ", p_exato)
```

```
Valor exato:      3.3976731247300535e-06
```

```
print("Monte Carlo usual:  ", p_mc)
```

```
Monte Carlo usual:  0.0
```

```
print("Erro padrão:      ", ep_mc)
```

```
Erro padrão:      0.0
```

```
print("Valores acima de 4,5:", int(indicadoras.sum()), "de", B)
```

```
Valores acima de 4,5: 0 de 100000
```

Nenhum dos cem mil valores simulados ultrapassou 4,5, e a estimativa é exatamente zero — errada por completo, já que $p > 0$.

⚠ Atenção: o erro padrão também mente

Quando todas as indicadoras valem zero, a variância amostral delas também vale zero, e o erro padrão estimado é 0. O intervalo de confiança do Capítulo 9 seria $[0, 0]$: uma estimativa errada acompanhada de uma promessa de precisão absoluta.

O problema é que o erro padrão é estimado **com a mesma amostra** que não viu o evento. Sempre que uma quantidade depende de uma região raramente visitada, o diagnóstico usual perde a validade — é preciso desconfiar antes, olhando quantas simulações efetivamente caíram na região de interesse.

Não é questão de azar. Para o estimador $\hat{p}$ da proporção, $\text{Var}(\hat{p}) = p(1-p)/B$, de modo que o **erro relativo** é

$$\frac{\sqrt{\text{Var}(\hat{p})}}{\hat{p}} = \sqrt{\frac{1-p}{Bp}} \approx \frac{1}{\sqrt{Bp}}.$$

O que importa não é B , e sim o produto Bp — o número esperado de simulações que caem na região de interesse. Para obter um erro relativo de 10% são necessárias $B \approx 100/p \approx 2,9 \times 10^7$ simulações; para $p = 10^{-9}$, seriam 10^{11} . O método não escala.

A raiz do problema é o **sorteio**, não o estimador: praticamente todas as B simulações caem onde h vale zero e não carregam informação nenhuma sobre p . A saída é sortear em outro lugar.

145.1 A ideia da amostragem por importância

Queremos estimar $\theta = \mathbb{E}[h(X)]$, com X de densidade f . Seja g uma outra densidade, chamada **proposta** (ou densidade de importância). Multiplicando e dividindo o integrando por $g(x)$,

$$\theta = \int h(x)f(x) dx = \int h(x) \frac{f(x)}{g(x)} g(x) dx = \mathbb{E}_g \left[h(Y) \frac{f(Y)}{g(Y)} \right],$$

em que Y tem densidade g e o índice em $\mathbb{E}_g$ lembra sob qual densidade a esperança é tomada. A conta é elementar, mas a consequência não é: θ , que era uma esperança sob f , virou uma esperança sob g — e podemos estimá-la simulando de g .

i Definição: pesos de importância

Dadas a densidade alvo f e a proposta g , o **peso de importância** de um ponto y é

$$w(y) = \frac{f(y)}{g(y)}.$$

O peso mede o quanto o ponto y foi sorteado “em excesso” ou “em falta” em relação a f : onde $g > f$, os pontos aparecem demais e são descontados ($w < 1$); onde $g < f$, aparecem de menos e são inflados ($w > 1$).

! Proposição: o estimador por importância

Sejam $Y_1, \dots, Y_B$ i.i.d. com densidade g , e suponha que $g(y) > 0$ sempre que $h(y)f(y) \neq 0$.
O **estimador por amostragem por importância**

$$\hat{\theta}_{\text{IS}} = \frac{1}{B} \sum_{i=1}^B w(Y_i) h(Y_i), \quad w(y) = \frac{f(y)}{g(y)},$$

é não viesado para θ e tem variância

$$\text{Var}(\hat{\theta}_{\text{IS}}) = \frac{1}{B} \left\{ \int \frac{h(x)^2 f(x)^2}{g(x)} dx - \theta^2 \right\}.$$

i Demonstração

As parcelas $w(Y_i)h(Y_i)$ são i.i.d., e pela identidade acima cada uma tem média

$$\mathbb{E}_g \left[h(Y) \frac{f(Y)}{g(Y)} \right] = \theta,$$

o que dá $\mathbb{E}[\hat{\theta}_{\text{IS}}] = \theta$. Como as parcelas são independentes, a variância da média é a variância de uma parcela dividida por B , e

$$\text{Var}_g \left[h(Y) \frac{f(Y)}{g(Y)} \right] = \mathbb{E}_g \left[h(Y)^2 \frac{f(Y)^2}{g(Y)^2} \right] - \theta^2 = \int h(x)^2 \frac{f(x)^2}{g(x)^2} g(x) dx - \theta^2. \quad \square$$

Comparando com a variância do Monte Carlo usual, $\frac{1}{B} \{ \int h(x)^2 f(x) dx - \theta^2 \}$, vê-se que os dois estimadores diferem apenas na integral do segundo momento: onde um tem $h^2 f$, o outro tem $h^2 f^2/g$. Tudo se resume, portanto, a escolher g que torne $\int h^2 f^2/g$ pequena — e é a isso que voltaremos na próxima seção.

⚠ Atenção: o suporte de g não pode ter buracos

A condição “ $g(y) > 0$ sempre que $h(y)f(y) \neq 0$ ” não é uma tecnicidade. Se existir uma região onde hf não se anula mas g vale zero, essa região **nunca será sorteada**, e a

contribuição dela para a integral desaparece silenciosamente: o estimador converge, com erro padrão pequeno e ar de confiabilidade, para o valor errado. Um caso concreto: usar $g = \text{Unif}(0, 1)$ para estimar $\mathbb{E}[X]$ com $X \sim \text{Exp}(1)$. Todo o intervalo $(1, \infty)$ é ignorado.

💡 Pseudo-algoritmo: amostragem por importância

Para estimar $\theta = \mathbb{E}[h(X)]$, com X de densidade f :

1. Escolha uma proposta g que saiba simular, com $g > 0$ onde $hf \neq 0$.
2. Gere $Y_1, \dots, Y_B$ independentes com densidade g .
3. Calcule os pesos $w_i = f(Y_i)/g(Y_i)$ e as parcelas $A_i = w_i h(Y_i)$.
4. Devolva

$$\hat{\theta}_{\text{IS}} = \frac{1}{B} \sum_{i=1}^B A_i,$$

com erro padrão $\hat{\sigma}_A/\sqrt{B}$, em que $\hat{\sigma}_A$ é o desvio padrão amostral dos A_i .

Note que o passo 4 é o de sempre: uma média de B valores i.i.d., com o erro padrão do Capítulo 9. A única novidade está em **quais** valores entram na média.

i Parentesco com o método da rejeição

O método da rejeição do Capítulo 6 também parte de uma proposta g e de valores simulados dela. A diferença está no que se faz com cada valor:

- a **rejeição** decide, no sorteio de uma moeda, se aceita ou descarta cada ponto; os pontos aceitos formam uma amostra genuína de f , e todo o resto do trabalho é jogado fora;
- a **amostragem por importância** não descarta nada: cada ponto entra na conta, ponderado por w_i .

A rejeição exige que f/g seja limitada por uma constante M conhecida; a amostragem por importância não exige nada além do suporte, embora funcione muito melhor quando f/g é razoavelmente limitada. Em compensação, a rejeição devolve uma amostra de f , que serve para qualquer finalidade, enquanto a amostragem por importância devolve apenas uma estimativa (Exercício 7).

145.2 A escolha da proposta

Nem toda proposta serve, e a diferença entre uma boa e uma ruim é enorme. Há um resultado que diz exatamente qual é a melhor.

! Proposição: a proposta ótima

Entre todas as densidades g admissíveis, a que minimiza $\text{Var}(\hat{\theta}_{\text{IS}})$ é

$$g^*(x) = \frac{|h(x)| f(x)}{\int |h(u)| f(u) du}.$$

Se, além disso, $h \geq 0$, então $g^* = hf/\theta$ e o estimador tem **variância zero**: todos os pesos ficam iguais e $\hat{\theta}_{\text{IS}} = \theta$ em qualquer simulação.

i Demonstração

Para qualquer g admissível, o segundo momento da parcela satisfaz, pela desigualdade de Jensen (ou simplesmente porque variância é não negativa),

$$\mathbb{E}_g \left[\left(h(Y) \frac{f(Y)}{g(Y)} \right)^2 \right] \geq \left(\mathbb{E}_g \left[h(Y) \frac{f(Y)}{g(Y)} \right] \right)^2 = \left(\int |h(x)| f(x) dx \right)^2,$$

e note que o lado direito **não depende de g** . Basta então verificar que g^* atinge esse limite. Escrevendo $c = \int |h|f$, temos $g^* = |h|f/c$ e

$$\mathbb{E}_{g^*} \left[\left(h(Y) \frac{f(Y)}{g^*(Y)} \right)^2 \right] = \int \frac{h(x)^2 f(x)^2}{g^*(x)} dx = \int \frac{h(x)^2 f(x)^2 c}{|h(x)| f(x)} dx = c \int |h(x)| f(x) dx = c^2.$$

Logo g^* minimiza o segundo momento e, como θ é o mesmo para todas as propostas, também minimiza a variância. Quando $h \geq 0$ tem-se $c = \int hf = \theta$, e o segundo momento vale θ^2 : a variância é $\theta^2 - \theta^2 = 0$. $\square$

O resultado é ao mesmo tempo perfeito e inútil. Perfeito porque exhibe uma proposta com variância zero; inútil porque essa proposta depende de $\int |h|f$, que é essencialmente a quantidade que queremos estimar. Se soubéssemos g^* , não precisaríamos simular.

O valor da proposição é servir de **bússola**. Ela diz que a proposta deve ter formato parecido com $|h|f$: colocar massa onde o produto $|h(x)|f(x)$ é grande e pouca massa onde ele é pequeno. Na prática, escolhemos uma família de densidades que saibamos simular — exponenciais, normais, gamas — e ajustamos os parâmetros para imitar $|h|f$.

⚠ Atenção: a cauda de g não pode ser leve demais

A variância do estimador envolve $\int h^2 f^2 / g$, com g **no denominador**. Se g decair mais rápido que $|h|f$ na cauda, essa razão explode e a integral pode divergir: a variância fica **infinita**, mesmo que a variância do Monte Carlo usual seja finita.

E o pior é o sintoma. Com variância infinita o estimador continua não viesado e a Lei dos Grandes Números continua valendo, de modo que as estimativas parecem se estabilizar — até que um único ponto muito longe na cauda receba um peso enorme e desloque a média inteira. O Teorema Central do Limite não vale, e o erro padrão calculado pela fórmula usual não significa nada.

A regra prática: **a proposta deve ter cauda mais pesada que a do alvo**. Na dúvida, prefira uma g dispersa demais a uma g concentrada demais.

145.2.1 Exemplo 2: quando a cauda da proposta é leve demais

Vamos estimar algo perfeitamente comum: $\theta = \mathbb{E}[X^2] = 1$, com $X \sim N(0, 1)$. Aqui não há evento raro nenhum, e o Monte Carlo usual funciona bem, com $\text{Var}(X^2) = \mathbb{E}[X^4] - 1 = 2$. Usaremos como proposta uma $N(0, \sigma^2)$, para dois valores de σ .

O segundo momento da parcela é, com φ_σ denotando a densidade da $N(0, \sigma^2)$,

$$\int x^4 \frac{\varphi(x)^2}{\varphi_\sigma(x)} dx \propto \int x^4 \exp\left\{-x^2 \left(1 - \frac{1}{2\sigma^2}\right)\right\} dx,$$

que converge se e somente se $1 - \frac{1}{2\sigma^2} > 0$, isto é, se $\sigma > 1/\sqrt{2} \approx 0,707$. Tomamos $\sigma = 0,5$ (variância infinita) e $\sigma = 1,5$ (variância finita) e olhamos a **média corrente**, ou seja, a estimativa que teríamos após cada número de simulações.

146 R

```
library(ggplot2)

set.seed(2)
B <- 20000

# Devolve as parcelas  $A_i = Y_i^2 * w_i$  do estimador por importância,
# com proposta  $N(0, \sigma^2)$  e alvo  $N(0,1)$ 
parcelas_is <- function(B, sigma) {
  y <- rnorm(B, mean = 0, sd = sigma)
  w <- dnorm(y) / dnorm(y, mean = 0, sd = sigma) # pesos  $f(y)/g(y)$ 
  y^2 * w
}

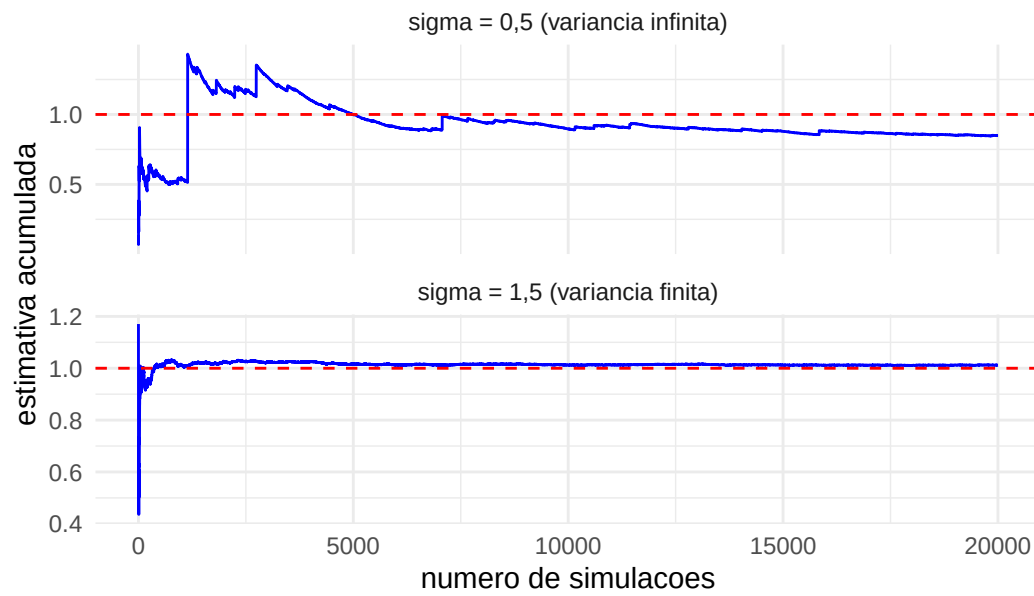
a_leve <- parcelas_is(B, sigma = 0.5) # cauda leve demais
a_ok <- parcelas_is(B, sigma = 1.5) # cauda suficientemente pesada

# Média corrente: a estimativa que teríamos após i simulações
media_corrente <- function(a) cumsum(a) / seq_along(a)

dados <- data.frame(
  i = rep(1:B, 2),
  media = c(media_corrente(a_leve), media_corrente(a_ok)),
  proposta = rep(c("sigma = 0,5 (variancia infinita)",
                    "sigma = 1,5 (variancia finita)"), each = B)
)

ggplot(dados, aes(x = i, y = media)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = 1, color = "red", linetype = "dashed") +
  facet_wrap(~ proposta, ncol = 1, scales = "free_y") +
  labs(x = "numero de simulacoes", y = "estimativa acumulada",
       title = "Media corrente do estimador por importancia") +
  theme_minimal()
```

Media corrente do estimador por importancia



147 Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(2)
B = 20000

# Devolve as parcelas  $A_i = Y_i^2 * w_i$  do estimador por importância,
# com proposta  $N(0, \sigma^2)$  e alvo  $N(0,1)$ 
def parcelas_is(B, sigma):
    y = np.random.normal(0, sigma, B)
    w = norm.pdf(y) / norm.pdf(y, 0, sigma)    # pesos  $f(y)/g(y)$ 
    return y**2 * w

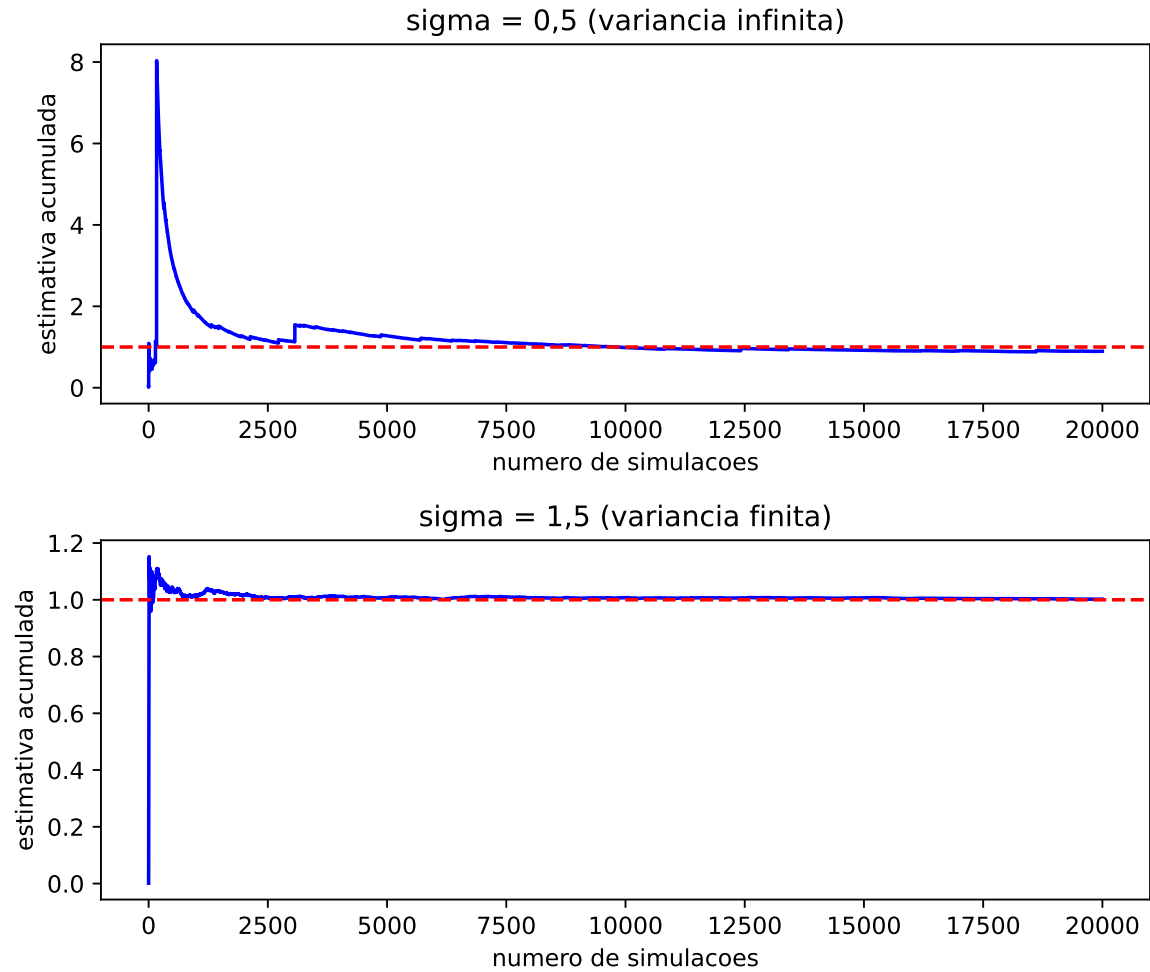
a_leve = parcelas_is(B, 0.5)    # cauda leve demais
a_ok    = parcelas_is(B, 1.5)    # cauda suficientemente pesada

# Média corrente: a estimativa que teríamos após i simulações
def media_corrente(a):
    return np.cumsum(a) / np.arange(1, len(a) + 1)

fig, eixos = plt.subplots(2, 1, figsize=(7, 6))
titulos = ["sigma = 0,5 (variancia infinita)",
           "sigma = 1,5 (variancia finita)"]

for eixo, a, titulo in zip(eixos, [a_leve, a_ok], titulos):
    eixo.plot(media_corrente(a), color='blue')
    eixo.axhline(1, color='red', linestyle='--')
    eixo.set_title(titulo)
    eixo.set_xlabel('numero de simulacoes')
    eixo.set_ylabel('estimativa acumulada')

fig.tight_layout()
plt.show()
```

O painel de cima é a assinatura da variância infinita: longos trechos de aparente estabilidade, interrompidos por saltos que jogam a estimativa para longe. O painel de baixo é o comportamento esperado de uma média de variáveis com variância finita.

E há um segundo recado, menos óbvio. Para $\sigma = 1,5$ a variância da parcela é aproximadamente 0,49, contra 2 do Monte Carlo usual — uma redução de quatro vezes. A amostragem por importância **não serve apenas para eventos raros**: aqui ela ganha porque $x^2 f(x)$ é uma função com duas corcovas afastadas da origem, e uma normal mais dispersa se parece mais com ela do que a própria $N(0, 1)$. O valor de σ que minimiza a variância é objeto do Exercício 5.

147.1 De volta aos eventos raros

147.1.1 Exemplo 3: $\mathbb{P}(Z > 4,5)$ por amostragem por importância

Voltemos ao Exemplo 1. Aqui $h(z) = I(z > 4,5) \geq 0$, então a proposição diz que a proposta ótima é

$$g^*(y) = \frac{\varphi(y) I(y > 4,5)}{p},$$

a normal padrão **truncada** em 4,5 — que não sabemos simular sem conhecer p , justamente o que queremos estimar (Exercício 12). Mas a bússola já ajuda: a proposta deve viver inteiramente em $(4,5, \infty)$ e ter, ali, o formato de φ . Escrevendo $y = 4,5 + s$,

$$\frac{\varphi(4,5 + s)}{\varphi(4,5)} = e^{-4,5 s - s^2/2} \approx e^{-4,5 s} \quad \text{para } s \text{ pequeno,}$$

isto é, logo acima do limiar a normal decai como uma **exponencial de taxa** 4,5. Isso sugere a família de propostas

$$g_\lambda(y) = \lambda e^{-\lambda(y-4,5)}, \quad y > 4,5,$$

— uma exponencial de taxa λ deslocada para 4,5 — e sugere ainda a escolha $\lambda = 4,5$. Vamos comparar $\lambda = 1$ (a escolha ingênua) com $\lambda = 4,5$.

Como $h(y) = 1$ para todo y gerado, o estimador é simplesmente a média dos pesos:

$$\hat{p}_{\text{IS}} = \frac{1}{B} \sum_{i=1}^B \frac{\varphi(Y_i)}{g_\lambda(Y_i)}, \quad Y_i = 4,5 + S_i, \quad S_i \sim \text{Exp}(\lambda).$$

148 R

```
set.seed(1)
B <- 100000
limiar <- 4.5

# Amostragem por importância com proposta exponencial de taxa lambda,
# deslocada para começar no limiar
is_exp_deslocada <- function(B, lambda, limiar) {
  s <- rexp(B, rate = lambda)      # S ~ Exp(lambda)
  y <- limiar + s                  # Y = limiar + S, sempre acima do limiar
  g <- lambda * exp(-lambda * s)   # densidade da proposta avaliada em y
  w <- dnorm(y) / g                # pesos; h(y) = 1, pois y > limiar
  list(est = mean(w), ep = sd(w) / sqrt(B), pesos = w)
}

is_1 <- is_exp_deslocada(B, lambda = 1, limiar)
is_45 <- is_exp_deslocada(B, lambda = 4.5, limiar)

# Monte Carlo usual, com o mesmo número de simulações
z <- rnorm(B)
p_mc <- mean(1 * (z > limiar))
ep_mc <- sd(1 * (z > limiar)) / sqrt(B)

mostra <- function(rotulo, est, ep) {
  # quando a estimativa é zero, o erro relativo não está definido
  rel <- if (est > 0) sprintf("%.4f", ep / est) else "---"
  cat(sprintf("%-20s est = %.4e   EP = %.4e   EP/est = %s\n",
              rotulo, est, ep, rel))
}

cat(sprintf("Valor exato: %.4e\n\n", p_exato))
```

Valor exato: 3.3977e-06

```
mostra("Monte Carlo usual", p_mc, ep_mc)
```

Monte Carlo usual est = 0.0000e+00 EP = 0.0000e+00 EP/est = ---

```
mostra("IS, lambda = 1", is_1$est, is_1$ep)
```

IS, lambda = 1 est = 3.3992e-06 EP = 1.3987e-08 EP/est = 0.0041

```
mostra("IS, lambda = 4,5", is_45$est, is_45$ep)
```

IS, lambda = 4,5 est = 3.3983e-06 EP = 9.1122e-10 EP/est = 0.0003

149 Python

```
import numpy as np
from scipy.stats import norm

np.random.seed(1)
B = 100000
limiar = 4.5

# Amostragem por importância com proposta exponencial de taxa lambda,
# deslocada para começar no limiar
def is_exp_deslocada(B, lam, limiar):
    s = np.random.exponential(scale=1/lam, size=B) # S ~ Exp(lam)
    y = limiar + s # Y = limiar + S, sempre acima do limiar
    g = lam * np.exp(-lam * s) # densidade da proposta avaliada em y
    w = norm.pdf(y) / g # pesos; h(y) = 1, pois y > limiar
    return {"est": w.mean(), "ep": w.std(ddof=1) / np.sqrt(B), "pesos": w}

is_1 = is_exp_deslocada(B, 1.0, limiar)
is_45 = is_exp_deslocada(B, 4.5, limiar)

# Monte Carlo usual, com o mesmo número de simulações
z = np.random.normal(0, 1, B)
ind = (z > limiar).astype(float)
p_mc, ep_mc = ind.mean(), ind.std(ddof=1) / np.sqrt(B)

def mostra(rotulo, est, ep):
    # quando a estimativa é zero, o erro relativo não está definido
    rel = f"{ep/est:.4f}" if est > 0 else "---"
    print(f"{rotulo:<20s} est = {est:.4e} EP = {ep:.4e} EP/est = {rel}")

print(f"Valor exato: {p_exato:.4e}\n")
```

Valor exato: 3.3977e-06

```
mostra("Monte Carlo usual", p_mc, ep_mc)
```

```
Monte Carlo usual      est = 1.0000e-05   EP = 1.0000e-05   EP/est = 1.0000
```

```
mostra("IS, lambda = 1", is_1["est"], is_1["ep"])
```

```
IS, lambda = 1        est = 3.4198e-06   EP = 1.4012e-08   EP/est = 0.0041
```

```
mostra("IS, lambda = 4,5", is_45["est"], is_45["ep"])
```

```
IS, lambda = 4,5      est = 3.3982e-06   EP = 9.0241e-10   EP/est = 0.0003
```

A coluna EP/est é o erro relativo, e é onde a história aparece. As duas propostas produzem estimativas corretas, mas com $\lambda = 4,5$ o erro relativo é mais de dez vezes menor que com $\lambda = 1$ — ou seja, mais de cem vezes menos variância, apenas por trocar um parâmetro da mesma família. Ambas são incomparavelmente melhores que o Monte Carlo usual, que com as mesmas 10^5 simulações acerta no máximo a ordem de grandeza, quando acerta.

i Deslocar a proposta é a receita para caudas

O padrão do Exemplo 3 se repete em quase todo problema de evento raro: se a região de interesse é $\{X > a\}$, procure uma proposta **centrada em** a , e não na média de f . Isso vale tanto para a família exponencial deslocada usada aqui quanto para propostas do tipo $N(\mu, 1)$ com $\mu \approx a$ (Exercício 13) ou para somas de variáveis (Exercício 8). A técnica geral por trás disso chama-se *tilting* exponencial, e consiste em trocar $f(x)$ por $f(x)e^{tx}/\mathbb{E}[e^{tX}]$, escolhendo t de modo que a nova densidade tenha média a .

149.1 Diagnóstico: o tamanho amostral efetivo

Como saber, olhando apenas para a saída da simulação, se a proposta foi boa? O sintoma de uma proposta ruim é sempre o mesmo: **poucos pesos muito grandes** dominam a soma, e as demais simulações praticamente não contribuem. Se dez pontos entre um milhão respondem por quase toda a estimativa, a amostra efetiva tem tamanho dez, não um milhão.

i Definição: tamanho amostral efetivo

Dados os pesos $w_1, \dots, w_B$, o **tamanho amostral efetivo** (TAE) é

$$\text{TAE} = \frac{\left(\sum_{i=1}^B w_i\right)^2}{\sum_{i=1}^B w_i^2}.$$

Ele satisfaz $1 \leq \text{TAE} \leq B$, com $\text{TAE} = B$ quando todos os pesos são iguais e $\text{TAE} = 1$ quando um único peso é não nulo.

A interpretação é que a estimativa por importância com B pontos tem precisão comparável à de uma estimativa por Monte Carlo usual com TAE pontos sorteados de f (Exercício 14). É uma regra de bolso, não um teorema — mas é um alarme barato e eficaz: um TAE muito menor que B diz que a proposta não cobre bem o alvo.

150 R

```
tae <- function(w) sum(w)^2 / sum(w^2)

cat("TAE com lambda = 1: ", round(tae(is_1$pesos), 1), "de", B, "\n")
```

TAE com lambda = 1: 37130 de 1e+05

```
cat("TAE com lambda = 4,5:", round(tae(is_45$pesos), 1), "de", B, "\n")
```

TAE com lambda = 4,5: 99286.2 de 1e+05

151 Python

```
def tae(w):  
    return w.sum()**2 / (w**2).sum()  
  
print("TAE com lambda = 1: ", round(tae(is_1["pesos"]), 1), "de", B)
```

TAE com lambda = 1: 37329.6 de 100000

```
print("TAE com lambda = 4,5:", round(tae(is_45["pesos"]), 1), "de", B)
```

TAE com lambda = 4,5: 99299.8 de 100000

O diagnóstico reproduz o que já sabíamos do erro padrão: com $\lambda = 1$ pouco mais de um terço das simulações é efetivamente aproveitado, enquanto com $\lambda = 4,5$ a amostra efetiva é quase B inteiro.

 Atenção: o TAE não detecta buracos no suporte

O TAE só enxerga os pontos que foram sorteados. Se g ignora completamente uma região onde hf é grande, nenhum peso grande aparece — e o TAE fica confortavelmente alto, apontando para uma estimativa errada. Ele detecta pesos desbalanceados, não regiões esquecidas.

151.1 Ajustando os parâmetros da proposta

Quando a proposta pertence a uma família paramétrica, às vezes é possível escolher o parâmetro **analiticamente**, minimizando a variância. O exemplo a seguir faz essa conta até o fim.

151.1.1 Exemplo 4: $\mathbb{E}[\sqrt{X}]$ com proposta exponencial

Queremos estimar

$$\theta = \mathbb{E}[\sqrt{X}], \quad X \sim \text{Exp}(1),$$

que equivale à integral

$$\theta = \int_0^\infty \sqrt{x} e^{-x} dx = \Gamma\left(\frac{3}{2}\right) = \frac{\sqrt{\pi}}{2} \approx 0,8862.$$

O Monte Carlo direto — simular $X \sim \text{Exp}(1)$ e promediar $\sqrt{X}$ — tem variância

$$\text{Var}(\sqrt{X}) = \mathbb{E}[X] - \theta^2 = 1 - \frac{\pi}{4} \approx 0,2146.$$

Tomemos como proposta $g_\lambda(x) = \lambda e^{-\lambda x}$, $x > 0$, isto é, uma $\text{Exp}(\lambda)$. As parcelas do estimador são

$$A_\lambda(x) = \frac{h(x)f(x)}{g_\lambda(x)} = \frac{\sqrt{x} e^{-x}}{\lambda e^{-\lambda x}} = \frac{1}{\lambda} \sqrt{x} e^{-(1-\lambda)x},$$

e o estimador é $\hat{\theta}_{\text{IS}} = \frac{1}{B} \sum_{i=1}^B A_\lambda(X_i)$, com $X_i \sim \text{Exp}(\lambda)$. Note que $\lambda = 1$ devolve $g = f$, $A_1(x) = \sqrt{x}$, e portanto o Monte Carlo direto: ele é um caso particular da amostragem por importância.

Para escolher λ , calculamos o segundo momento das parcelas sob g_λ :

$$\mathbb{E}_{g_\lambda}[A_\lambda(X)^2] = \int_0^\infty \frac{x e^{-2(1-\lambda)x}}{\lambda^2} \lambda e^{-\lambda x} dx = \frac{1}{\lambda} \int_0^\infty x e^{-(2-\lambda)x} dx = \frac{1}{\lambda(2-\lambda)^2},$$

finito apenas para $0 < \lambda < 2$ — mais uma vez, a cauda da proposta não pode ser leve demais. Logo

$$\text{Var}(\hat{\theta}_{\text{IS}}) = \frac{1}{B} \left(\frac{1}{\lambda(2-\lambda)^2} - \theta^2 \right).$$

Minimizar essa expressão é maximizar $\lambda(2-\lambda)^2$; derivando,

$$\frac{d}{d\lambda} \lambda(2-\lambda)^2 = (2-\lambda)(2-3\lambda) = 0 \quad \implies \quad \lambda^* = \frac{2}{3},$$

com $1/[\lambda^*(2-\lambda^*)^2] = 27/32 = 0,84375$ e, portanto,

$$\text{Var}(\hat{\theta}_{\text{IS}, \lambda^*}) = \frac{1}{B} \left(0,84375 - \frac{\pi}{4} \right) = \frac{0,05835}{B},$$

um ganho de $0,2146/0,05835 \approx 3,68$ vezes em relação ao Monte Carlo direto. O sentido da resposta é o esperado: $\lambda^* < 1$ significa uma exponencial mais dispersa que a original, deslocando massa para valores maiores de x — que é onde $\sqrt{x}e^{-x}$ tem seu máximo.

💡 Pseudo-algoritmo: o estimador do Exemplo 4

1. Fixe λ (use $\lambda = 2/3$ para o mínimo da variância).
2. Gere $X_1, \dots, X_B \stackrel{iid}{\sim} \text{Exp}(\lambda)$.
3. Calcule $A_i = \frac{\sqrt{X_i} e^{-X_i}}{\lambda e^{-\lambda X_i}}$.
4. Devolva $\hat{\theta} = \frac{1}{B} \sum_{i=1}^B A_i$, com erro padrão $\hat{\sigma}_A/\sqrt{B}$.

152 R

```
set.seed(1)

theta_exato <- sqrt(pi) / 2
B <- 10000

is_exp <- function(B, lambda) {
  x <- rexp(B, rate = lambda)
  a <- sqrt(x) * exp(-x) / (lambda * exp(-lambda * x))
  list(est = mean(a), ep = sd(a) / sqrt(B))
}

out_1 <- is_exp(B, lambda = 1)      # equivale ao Monte Carlo direto
out_opt <- is_exp(B, lambda = 2/3)  # proposta ótima na família

cat(sprintf("Valor exato          : %.6f\n", theta_exato))
```

Valor exato : 0.886227

```
cat(sprintf("IS Exp(1)    (= MC)    : %.6f    EP = %.5f\n", out_1$est, out_1$ep))
```

IS Exp(1) (= MC) : 0.882998 EP = 0.00468

```
cat(sprintf("IS Exp(2/3) (ótima) : %.6f    EP = %.5f\n", out_opt$est, out_opt$ep))
```

IS Exp(2/3) (ótima) : 0.889350 EP = 0.00240

```
cat(sprintf("Fator de redução      : %.4f    (teórico: %.4f)\n",
            (out_opt$ep / out_1$ep)^2, 0.05835 / 0.21460))
```

Fator de redução : 0.2631 (teórico: 0.2719)

153 Python

```
import numpy as np

np.random.seed(1)

theta_exato = np.sqrt(np.pi) / 2
B = 10000

def is_exp(B, lam):
    x = np.random.exponential(scale=1/lam, size=B)
    a = np.sqrt(x) * np.exp(-x) / (lam * np.exp(-lam * x))
    return {"est": a.mean(), "ep": a.std(ddof=1) / np.sqrt(B)}

out_1 = is_exp(B, 1.0)      # equivale ao Monte Carlo direto
out_opt = is_exp(B, 2/3)    # proposta ótima na família

print(f"Valor exato          : {theta_exato:.6f}")
```

Valor exato : 0.886227

```
print(f"IS Exp(1)    (= MC)    : {out_1['est']:.6f}    EP = {out_1['ep']:.5f}")
```

IS Exp(1) (= MC) : 0.881945 EP = 0.00459

```
print(f"IS Exp(2/3) (ótima)  : {out_opt['est']:.6f}    EP = {out_opt['ep']:.5f}")
```

IS Exp(2/3) (ótima) : 0.887836 EP = 0.00243

```
print(f"Fator de redução      : {(out_opt['ep']/out_1['ep'])**2:.4f}"
      f"    (teórico: {0.05835/0.21460:.4f})")
```

Fator de redução : 0.2797 (teórico: 0.2719)

153.1 Amostragem por importância autonormalizada

Até aqui supusemos que sabemos calcular $f(x)$. Em muitas aplicações, porém, só conhecemos f **a menos de uma constante multiplicativa**: temos uma função $q(x) \geq 0$ com

$$f(x) = \frac{q(x)}{Z}, \quad Z = \int q(u) du \text{ desconhecida.}$$

Isso é a regra, não a exceção. Densidades condicionais, distribuições truncadas e sobretudo distribuições a posteriori em inferência bayesiana quase sempre chegam nessa forma, porque a constante Z é justamente uma integral que não sabemos calcular.

A saída é estimar Z com a mesma amostra. Definindo os pesos **não normalizados** $w_i = q(Y_i)/g(Y_i)$, temos, pela Lei dos Grandes Números,

$$\frac{1}{B} \sum_{i=1}^B w_i \rightarrow \mathbb{E}_g \left[\frac{q(Y)}{g(Y)} \right] = \int q(u) du = Z, \quad \frac{1}{B} \sum_{i=1}^B w_i h(Y_i) \rightarrow Z \theta,$$

e o quociente das duas médias faz Z desaparecer.

i Definição: estimador autonormalizado

Com $Y_1, \dots, Y_B$ i.i.d. de densidade g e $w_i = q(Y_i)/g(Y_i)$, o **estimador autonormalizado** de $\theta = \mathbb{E}_f[h(X)]$ é

$$\hat{\theta}_{\text{AN}} = \frac{\sum_{i=1}^B w_i h(Y_i)}{\sum_{i=1}^B w_i} = \sum_{i=1}^B \bar{w}_i h(Y_i), \quad \bar{w}_i = \frac{w_i}{\sum_{j=1}^B w_j}.$$

Ele é uma **média ponderada** dos valores $h(Y_i)$, com pesos $\bar{w}_i$ que somam 1.

Por ser um quociente de duas médias, $\hat{\theta}_{\text{AN}}$ converge para θ pela Lei dos Grandes Números, mas **não é não viesado**: a esperança de um quociente não é o quociente das esperanças. O viés é da ordem de $1/B$ — desprezível diante do erro padrão, que é da ordem de $1/\sqrt{B}$, exatamente como acontecia com a constante estimada das variáveis de controle no Capítulo 10.

💡 Pseudo-algoritmo: amostragem por importância autonormalizada

1. Escolha g que saiba simular, com $g > 0$ onde $q > 0$.
2. Gere $Y_1, \dots, Y_B$ independentes com densidade g .
3. Calcule os pesos não normalizados $w_i = q(Y_i)/g(Y_i)$ e normalize: $\bar{w}_i = w_i / \sum_j w_j$.

4. Devolva $\hat{\theta}_{\text{AN}} = \sum_{i=1}^B \bar{w}_i h(Y_i)$.
5. Verifique o TAE, $(\sum_i w_i)^2 / \sum_i w_i^2$, antes de acreditar no resultado.

i Um bônus: probabilidades saem de graça

Tomando $h(x) = I(x \in A)$, o estimador vira a soma dos pesos normalizados dos pontos que caíram em A . Ou seja, uma única amostra ponderada permite estimar a média, a variância, quantis e qualquer probabilidade sob f — sem repetir a simulação.

153.1.1 Exemplo 5: uma densidade conhecida a menos de constante

Considere a densidade sobre $(0, 1)$ dada por

$$f(\theta) = \frac{q(\theta)}{Z}, \quad q(\theta) = \theta^{13}(1 - \theta)^7,$$

e suponha que queiramos $\mathbb{E}_f[\theta]$ e $\mathbb{P}_f(\theta > 0,7)$.

i De onde vem essa densidade

Ela é a distribuição a posteriori de uma probabilidade de sucesso θ com distribuição a priori $\text{Unif}(0, 1)$, depois de observar 13 sucessos em 20 ensaios de Bernoulli: $q(\theta)$ é exatamente a verossimilhança. Neste caso sabemos que $Z = B(14, 8)$ e que f é uma $\text{Beta}(14, 8)$ — o que nos dá um valor exato para conferir. Em problemas reais, Z é uma integral intratável, e é aí que o método é indispensável.

Vamos comparar duas propostas: a $\text{Unif}(0, 1)$, que ignora tudo o que sabemos, e uma $N(0,65; 0,1^2)$, ajustada grosseiramente ao formato de q (o máximo de q está em $13/20 = 0,65$).

154 R

```
set.seed(3)
B <- 10000

# Densidade alvo, conhecida apenas a menos da constante Z
q <- function(theta) ifelse(theta > 0 & theta < 1, theta^13 * (1 - theta)^7, 0)

# Estimador autonormalizado: devolve média, P(theta > 0,7) e o TAE
is_autonormalizada <- function(y, g_y) {
  w <- q(y) / g_y
  list(media = sum(w * y) / sum(w),
        prob = sum(w * (y > 0.7)) / sum(w),
        tae = sum(w)^2 / sum(w^2))
}

# Proposta 1: Unif(0,1)
y1 <- runif(B)
res1 <- is_autonormalizada(y1, dunif(y1))

# Proposta 2: N(0,65 ; 0,1^2)
y2 <- rnorm(B, mean = 0.65, sd = 0.1)
res2 <- is_autonormalizada(y2, dnorm(y2, mean = 0.65, sd = 0.1))

# Valores exatos, pois f é uma Beta(14, 8)
media_exata <- 14 / 22
prob_exata <- pbeta(0.7, 14, 8, lower.tail = FALSE)

cat(sprintf("%-18s %10s %10s %8s\n", "Proposta", "media", "P(>0,7)", "TAE"))
```

Proposta	media	P(>0,7)	TAE
----------	-------	---------	-----

```
cat(sprintf("%-18s %10.4f %10.4f %8s\n", "valor exato", media_exata, prob_exata, "-"))
```

valor exato	0.6364	0.2770	-
-------------	--------	--------	---


```
cat(sprintf("%-18s %10.4f %10.4f %8.0f\n", "Unif(0,1)",
           res1$media, res1$prob, res1$tae))
```

```
Unif(0,1)          0.6349    0.2659    3588
```

```
cat(sprintf("%-18s %10.4f %10.4f %8.0f\n", "Normal ajustada",
           res2$media, res2$prob, res2$tae))
```

```
Normal ajustada    0.6353    0.2691    9711
```

155 Python

```
import numpy as np
from scipy.stats import norm, uniform, beta

np.random.seed(3)
B = 10000

# Densidade alvo, conhecida apenas a menos da constante Z
def q(theta):
    dentro = (theta > 0) & (theta < 1)
    return np.where(dentro, theta**13 * (1 - theta)**7, 0.0)

# Estimador autonormalizado: devolve média, P(theta > 0,7) e o TAE
def is_autonormalizada(y, g_y):
    w = q(y) / g_y
    return {"media": np.sum(w * y) / np.sum(w),
            "prob": np.sum(w * (y > 0.7)) / np.sum(w),
            "tae": np.sum(w)**2 / np.sum(w**2)}

# Proposta 1: Unif(0,1)
y1 = np.random.uniform(0, 1, B)
res1 = is_autonormalizada(y1, uniform.pdf(y1))

# Proposta 2: N(0,65 ; 0,1^2)
y2 = np.random.normal(0.65, 0.1, B)
res2 = is_autonormalizada(y2, norm.pdf(y2, 0.65, 0.1))

# Valores exatos, pois f é uma Beta(14, 8)
media_exata = 14 / 22
prob_exata = beta.sf(0.7, 14, 8)

print(f"{'Proposta':<18s} {'media':>10s} {'P(>0,7)':>10s} {'TAE':>8s}")
```

Proposta	media	P(>0,7)	TAE
----------	-------	---------	-----

```
print(f'{"valor exato":<18s} {"media_exata:>10.4f} {"prob_exata:>10.4f} {"-":>8s}')
```

```
valor exato          0.6364      0.2770      -
```

```
print(f'{"Unif(0,1)":<18s} {"res1['media']:>10.4f} {"res1['prob']:>10.4f} {"res1['tae']:>8.0f}')
```

```
Unif(0,1)           0.6360      0.2790      3541
```

```
print(f'{"Normal ajustada":<18s} {"res2['media']:>10.4f} {"res2['prob']:>10.4f} {"res2['tae']:>8.0f}')
```

```
Normal ajustada     0.6360      0.2729      9725
```

As duas propostas acertam a média com boa precisão, mas o TAE denuncia a diferença de qualidade: a $\text{Unif}(0,1)$ desperdiça a maior parte das simulações em regiões onde q é praticamente zero, enquanto a normal ajustada aproveita quase todas. A diferença fica mais visível na estimativa de $\mathbb{P}_f(\theta > 0,7)$, que depende de uma região de menor probabilidade e por isso é mais sensível ao desperdício.

⚠ Atenção: autonormalizar não conserta uma proposta ruim

A normalização pelos pesos garante que a estimativa seja uma média ponderada legítima — em particular, uma probabilidade estimada nunca sai de $[0,1]$. Isso dá uma falsa sensação de segurança: se g não cobre bem q , o resultado continua ruim, apenas de forma menos escandalosa. O TAE é o que revela o problema.

155.1 Exercícios

Exercício 1. Seja $Z \sim N(0,1)$ e $p = \mathbb{P}(Z > 5)$.

- Estime p por Monte Carlo usual com $B = 10^6$. Quantas simulações caíram acima de 5? Quantas seriam necessárias, em média, para que caíssem 100?
- Estime p por amostragem por importância com a proposta exponencial deslocada $g_\lambda(y) = \lambda e^{-\lambda(y-5)}$, $y > 5$, para $\lambda \in \{1, 3, 5, 8\}$, sempre com $B = 10^5$. Compare os erros relativos.
- Compare as estimativas com o valor exato (`pnorm(5, lower.tail = FALSE)` em R, `scipy.stats.norm.sf(5)` em Python) e calcule o TAE de cada proposta.
- O melhor λ do item (b) é próximo de 5. Explique por quê, usando o argumento de que $\varphi(5+s)/\varphi(5) \approx e^{-5s}$.

Exercício 2. Estime $\theta = \int_0^2 \sqrt{x} e^{-x} dx$ por amostragem por importância com duas propostas diferentes:

- (a) $g_1 = \text{Unif}(0, 2)$;
- (b) g_2 = a densidade $\text{Exp}(1)$ **truncada** ao intervalo $(0, 2)$, isto é, $g_2(x) = e^{-x}/(1 - e^{-2})$ para $0 < x < 2$. (Simule-a pelo método da inversão do Capítulo 4.)
- (c) Justifique, antes de rodar o código, qual das duas deve ser melhor, esboçando $|h(x)|f(x)$ com $h(x) = \sqrt{x}$ e f a densidade que cada proposta imita. Depois compare os erros padrão obtidos e verifique se a previsão se confirmou.
- (d) Use `integrate` (R) ou `scipy.integrate.quad` (Python) como valor de referência.

Exercício 3. Seja $X \sim \text{Exp}(1)$ e $\theta = \mathbb{E}[\log(1 + X)]$.

- (a) Estime θ por Monte Carlo usual com $B = 10^4$ e calcule o erro padrão.
- (b) Faça o gráfico de $h(x)f(x) = \log(1 + x)e^{-x}$ e verifique que o máximo está em torno de $x \approx 0,76$ (a equação a resolver é $1/(1 + x) = \log(1 + x)$).
- (c) Proponha uma densidade g com formato parecido — por exemplo uma $\text{Gama}(2, \beta)$, cuja moda é $1/\beta$ — e implemente o estimador por importância.
- (d) O ganho aqui é modesto, bem menor que nos exemplos de eventos raros. Explique por quê, comparando o formato de hf com o de f .

Exercício 4. Sejam f e g densidades com $g > 0$ onde $f > 0$, e $w(Y) = f(Y)/g(Y)$ com $Y \sim g$.

- (a) Mostre que $\mathbb{E}_g[w(Y)] = 1$.
- (b) Mostre que $\text{Var}_g(w(Y)) = \int \frac{f(x)^2}{g(x)} dx - 1$.
- (c) A quantidade do item (b) é a **divergência do qui-quadrado** entre f e g , e vale zero se e somente se $f = g$. Verifique numericamente os itens (a) e (b) para $f = N(0, 1)$ e $g = N(0, \sigma^2)$, com $\sigma \in \{0,8; 1; 2\}$.
- (d) Explique por que um valor grande em (b) implica um TAE pequeno.

Exercício 5. Volte ao Exemplo 2: $\theta = \mathbb{E}[X^2] = 1$ com $X \sim N(0, 1)$ e proposta $N(0, \sigma^2)$.

- (a) Mostre que, escrevendo $c = 1 - \frac{1}{2\sigma^2}$, o segundo momento da parcela é

$$\mathbb{E}_g[A^2] = \frac{3\sigma}{4c^2\sqrt{2c}}, \quad \sigma > 1/\sqrt{2}.$$

(Dica: $\int_{-\infty}^{\infty} x^4 e^{-ax^2} dx = \frac{3}{4a^2} \sqrt{\pi/a}$.)

- (b) Minimize essa expressão e conclua que $\sigma^* = \sqrt{3}$.
- (c) Calcule o fator de redução de variância em relação ao Monte Carlo usual e verifique-o empiricamente com $B = 10^5$.
- (d) Faça o gráfico de $\text{Var}(A)$ contra $\sigma \in (0,71; 4)$ e observe o que acontece quando $\sigma \rightarrow 1/\sqrt{2}$.

Exercício 6. Uma forma de nunca cair no problema da cauda leve demais é usar uma proposta de cauda **muito** pesada. Tome $f = N(0, 1)$, $h(x) = x^4$ e como proposta a densidade de Cauchy padrão, $g(x) = \frac{1}{\pi(1+x^2)}$.

- (a) Mostre que os pesos $w(x) = \pi(1+x^2)\varphi(x)$ são limitados e conclua que a variância do estimador é finita.
- (b) Estime $\mathbb{E}[X^4] = 3$ com $B = 10^5$ e compare o erro padrão com o do Monte Carlo usual.
- (c) A proposta é segura, mas é melhor ou pior que o Monte Carlo usual neste problema? Explique o resultado em termos do formato de $|h|f$.

Exercício 7. Seja f a densidade da **meia-normal**, $f(x) = \sqrt{2/\pi} e^{-x^2/2}$ para $x > 0$, e $g(x) = e^{-x}$ a densidade $\text{Exp}(1)$, como no Capítulo 6.

- (a) Mostre que $f(x)/g(x) \leq M = \sqrt{2e/\pi} \approx 1,3155$ e que a igualdade ocorre em $x = 1$.
- (b) Estime $\mathbb{E}_f[X] = \sqrt{2/\pi}$ de duas maneiras, ambas partindo de $B = 10^4$ valores gerados de g : pelo método da rejeição (aceitando cerca de B/M deles e tirando a média simples) e por amostragem por importância (usando todos os B).
- (c) Compare os erros padrão. Qual método aproveita melhor o mesmo esforço computacional?
- (d) Cite uma situação em que ainda assim se prefere a rejeição.

Exercício 8. Sejam $X_1, \dots, X_{10}$ i.i.d. $\text{Exp}(1)$ e $S = \sum_{i=1}^{10} X_i$. Queremos $\theta = \mathbb{P}(S > 40)$.

- (a) Estime θ por Monte Carlo usual com $B = 10^6$ e comente o resultado.
- (b) Considere a proposta que gera $X_1, \dots, X_{10}$ i.i.d. $\text{Exp}(\lambda)$, com $\lambda < 1$. Mostre que o peso de uma configuração $(x_1, \dots, x_{10})$ é

$$w(x_1, \dots, x_{10}) = \prod_{i=1}^{10} \frac{e^{-x_i}}{\lambda e^{-\lambda x_i}} = \lambda^{-10} e^{-(1-\lambda)\sum_i x_i}.$$

- (c) Implemente o estimador com $\lambda = 1/4$ e $B = 10^5$. Compare com o valor exato, $\mathbb{P}(W > 40)$ com $W \sim \text{Gama}(10, 1)$ (`pgamma` em R, `scipy.stats.gamma` em Python).
- (d) Explique a escolha $\lambda = 10/40$: sob a proposta, qual é a média de S ? Repita com $\lambda \in \{1/2; 1/4; 1/8\}$ e veja qual dá o menor erro relativo.

Exercício 9. Volte ao Exemplo 5, com $q(\theta) = \theta^{13}(1 - \theta)^7$.

- (a) Reproduza as estimativas com as duas propostas do exemplo.
- (b) Acrescente uma terceira proposta, $N(0,5; 0,05^2)$, e compare o TAE e o erro da média estimada. O que deu errado?
- (c) Acrescente uma quarta proposta, $\text{Beta}(10,6)$, e compare com as demais.
- (d) Repita o exercício com $q(\theta) = \theta^{130}(1 - \theta)^{70}$ (o mesmo formato, com 200 ensaios em vez de 20). O que acontece com o TAE da proposta $\text{Unif}(0,1)$? Que lição isso dá sobre amostragem por importância em problemas com muita informação?

Exercício 10. Uma vantagem prática da amostragem por importância é reaproveitar uma **única** amostra para vários alvos. Sejam f_μ a densidade da $N(\mu, 1)$ e $h(x) = x^2$, de modo que $\theta(\mu) = \mathbb{E}_{f_\mu}[X^2] = 1 + \mu^2$.

- (a) Gere uma única amostra $Y_1, \dots, Y_B$ de $g = N(0, 2^2)$, com $B = 10^4$.
- (b) Usando **essa mesma amostra**, estime $\theta(\mu)$ para $\mu \in \{-1, 0, 1, 2, 3, 4\}$, recalculando apenas os pesos $w_i = f_\mu(Y_i)/g(Y_i)$.
- (c) Faça o gráfico das estimativas contra μ , junto com a curva exata $1 + \mu^2$ e com as bandas de ± 2 erros padrão.
- (d) A partir de que valor de μ a estimativa deixa de ser confiável? Relacione com o TAE de cada μ .

Exercício 11. Suponha que a proposta g tenha suporte **menor** que o de f : tome $f = \text{Exp}(1)$, $g = \text{Unif}(0, 3)$ e $h(x) = x$.

- (a) Calcule analiticamente para qual valor o estimador por importância converge, e mostre que ele não é $\mathbb{E}[X] = 1$.
- (b) Confirme numericamente com $B = 10^6$ e observe que o erro padrão é pequeno — o estimador converge confiantemente para o valor errado.
- (c) Calcule o TAE e verifique que ele **não** detecta o problema.
- (d) Como você detectaria esse erro na prática, sem conhecer a resposta?

Exercício 12. Considere de novo $p = \mathbb{P}(Z > 4,5)$, com $Z \sim N(0, 1)$, e a proposta ótima $g^*(y) = \varphi(y)I(y > 4,5)/p$, isto é, a normal padrão truncada em 4,5.

- (a) Mostre que, se $U \sim \text{Unif}(0, 1)$, então $Y = \Phi^{-1}(1 - U(1 - \Phi(4,5)))$ tem densidade g^* . (Em R, use `qnorm(u * pnorm(4.5, lower.tail = FALSE), lower.tail = FALSE)`, que é numericamente mais estável.)

- (b) Gere $B = 1000$ valores de g^* , calcule os pesos $w_i = \varphi(Y_i)/g^*(Y_i)$ e verifique que todos são iguais a p : a variância é exatamente zero.
- (c) Explique por que essa proposta é inútil na prática, apesar de ótima.
- (d) Compare o histograma dos valores gerados em (b) com o dos valores gerados pela proposta exponencial deslocada com $\lambda = 4,5$ do Exemplo 3. Por que a segunda funciona tão bem?

Exercício 13. (Desafio) Estime $p = \mathbb{P}(Z > 6)$, com $Z \sim N(0, 1)$, usando propostas da família $g_\mu = N(\mu, 1)$.

- (a) Mostre que o estimador é $\hat{p} = \frac{1}{B} \sum_{i=1}^B I(Y_i > 6) \frac{\varphi(Y_i)}{\varphi(Y_i - \mu)}$, com $Y_i \sim N(\mu, 1)$, e que os pesos valem $e^{-\mu Y_i + \mu^2/2}$.
- (b) Mostre que o segundo momento das parcelas é

$$m(\mu) = e^{\mu^2} \mathbb{P}(Z > 6 + \mu).$$

(Dica: verifique que $\varphi(y - \mu) e^{-2\mu y} = \varphi(y + \mu)$.)

- (c) Encontre numericamente o μ^* que minimiza $m(\mu)$ e mostre que a condição de primeira ordem é $2\mu = \varphi(6 + \mu)/\mathbb{P}(Z > 6 + \mu)$. Argumente que $\mu^* \approx 6$ quando o limiar é grande.
- (d) Compare, em erro relativo, o Monte Carlo usual, a amostragem por importância com $\mu \in \{3, 6, 9\}$ e com μ^* .

Exercício 14. (Desafio) Este exercício justifica o tamanho amostral efetivo.

- (a) Mostre que

$$\text{TAE} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2} = \frac{B}{1 + \widehat{\text{cv}}^2(w)},$$

onde $\widehat{\text{cv}}^2(w)$ é o quadrado do coeficiente de variação amostral dos pesos (use a versão com denominador B na variância amostral).

- (b) Conclua, usando o Exercício 4, que TAE/B converge para $1/(1 + \chi^2(f\|g))$, em que $\chi^2(f\|g) = \int f^2/g - 1$.
- (c) Considere o estimador autonormalizado de $\theta = \mathbb{E}_f[h(X)]$ e suponha h pouco correlacionada com os pesos. Argumente que sua variância é aproximadamente $\text{Var}_f(h(X))/\text{TAE}$, o que justifica a leitura “amostra efetiva de tamanho TAE”.
- (d) Verifique (c) numericamente para $f = N(0, 1)$, $g = N(0, \sigma^2)$ e $h(x) = x$, repetindo a estimação $R = 1000$ vezes.

Exercício 15. (Desafio) A amostragem por importância defensiva protege contra o desastre do Exemplo 2. Dada uma proposta qualquer g_0 e um $\alpha \in (0, 1)$, use a mistura

$$g(x) = \alpha f(x) + (1 - \alpha)g_0(x).$$

- (a) Descreva como simular de g (sorteie primeiro de qual das duas componentes o valor virá) e escreva os pesos.
- (b) Mostre que $w(x) = f(x)/g(x) \leq 1/\alpha$ para todo x .
- (c) Conclua que $\mathbb{E}_g[h^2 w^2] \leq \frac{1}{\alpha} \mathbb{E}_f[h(X)^2]$, de modo que a variância é finita sempre que $\text{Var}_f(h(X))$ for finita — qualquer que seja g_0 .
- (d) Aplique ao Exemplo 2 com $g_0 = N(0; 0,5^2)$ e $\alpha = 0,2$, e refaça o gráfico da média corrente. O desastre sumiu? Compare a variância resultante com a do Monte Carlo usual e comente: o que exatamente se comprou com a mistura?

Exercício 16. (Desafio) Seja $X \sim \text{Exp}(1)$ e $\theta = \mathbb{P}(X > t) = e^{-t}$, e considere a família de propostas $\text{Exp}(\lambda)$ com $0 < \lambda < 2$.

- (a) Mostre que o segundo momento das parcelas é

$$m(\lambda) = \frac{e^{-(2-\lambda)t}}{\lambda(2-\lambda)}.$$

- (b) Mostre que a condição de primeira ordem para minimizar m leva à equação $t\lambda^2 - 2(t+1)\lambda + 2 = 0$ e conclua que

$$\lambda^* = \frac{t+1 - \sqrt{t^2+1}}{t}.$$

- (c) Verifique que $\lambda^* \rightarrow 1/t$ quando $t \rightarrow \infty$ e interprete: sob a proposta ótima, qual é a média de X ?
- (d) Para $t = 10$, compare numericamente ($B = 10^5$) o erro relativo do Monte Carlo usual, da proposta com $\lambda = 1/t$ e da proposta com λ^* . A diferença entre as duas últimas é relevante?

References

Ross, Sheldon M. 2006. *Simulation, Fourth Edition*. USA: Academic Press, Inc.